Computer Systems

A Programmer's Perspective

Randal E. Bryant
Carnegie Mellon University

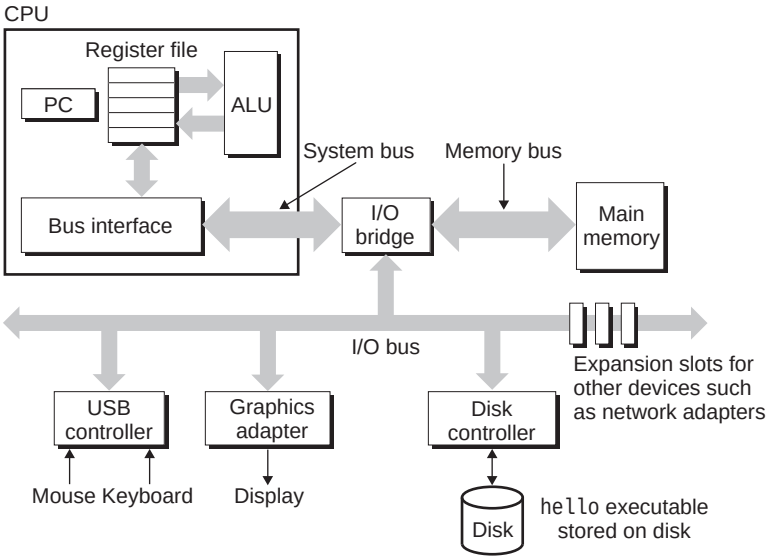
David R. O'Hallaron
Carnegie Mellon University

RS N

Contents

1

Figure 1.4
Hardware organization of a typical system. CPU: tral r o i t, ALU: ar t ti /l o i t, PC: r o ra o t r, USB: U i r al S ri al B .



assumes that it is the name of an executable file that it should load and run. So in this case, the shell loads and runs the `hello` program and then waits for it to terminate. The `hello` program prints its message to the screen and then terminates. The shell then prints a prompt and waits for the next input command line.

1.4.1 Hardware Organization of a System

To understand what happens to our `hello` program when we run it, we need to understand the hardware organization of a typical system, which is shown in Figure 1.4. This particular picture is modeled after the family of recent Intel systems, but all systems have a similar look and feel. Don't worry about the complexity of this figure just now. We will get to its various details in stages throughout the course of the book.

B

Running throughout the system is a collection of electrical conduits called buses that carry bytes of information back and forth between the components. Buses are typically designed to transfer fixed-size chunks of bytes known as words. The number of bytes in a word (the word size) is a fundamental system parameter that varies across systems. Most machines today have word sizes of either 4 bytes (32 bits) or 8 bytes (64 bits). In this book, we do not assume any fixed definition of word size. Instead, we will specify what we mean by a "word" in any context that requires this to be defined.

I/O D i

Input/output (I/O) devices are the system's connection to the external world. Our example system has four I/O devices: a keyboard and mouse for user input, a display for user output, and a disk drive (or simply disk) for long-term storage of data and programs. Initially, the executable `hello` program resides on the disk.

Each I/O device is connected to the I/O bus by either a controller or an adapter. The distinction between the two is mainly one of packaging. Controllers are chip sets in the device itself or on the system's main printed circuit board (often called the motherboard). An adapter is a card that plugs into a slot on the motherboard. Regardless, the purpose of each is to transfer information back and forth between the I/O bus and an I/O device.

Chapter 6 has more to say about how I/O devices such as disks work. In Chapter 10, you will learn how to use the Unix I/O interface to access devices from your application programs. We focus on the especially interesting class of devices known as networks, but the techniques generalize to other kinds of devices as well.

Mai M or

The main memory is a temporary storage device that holds both a program and the data it manipulates while the processor is executing the program. Physically, main memory consists of a collection of dynamic random access memory (DRAM) chips. Logically, memory is organized as a linear array of bytes, each with its own unique address (array index) starting at zero. In general, each of the machine instructions that constitute a program can consist of a variable number of bytes. The sizes of data items that correspond to C program variables vary according to type. For example, on an x86-64 machine running Linux, data of type `short` require 2 bytes, types `int` and `float` 4 bytes, and types `long` and `double` 8 bytes.

Chapter 6 has more to say about how memory technologies such as DRAM chips work, and how they are combined to form main memory.

Pro or

The central processing unit (CPU), or simply processor, is the engine that interprets (or executes) instructions stored in main memory. At its core is a word-size storage device (or register) called the program counter (PC). At any point in time, the PC points at (contains the address of) some machine-language instruction in main memory.[2]

From the time that power is applied to the system until the time that the power is shut off, a processor repeatedly executes the instruction pointed at by the program counter and updates the program counter to point to the next instruction. A processor appears to operate according to a very simple instruction execution model, defined by its instruction set architecture. In this model, instructions execute

2. PC is also a commonly used acronym for "personal computer." However, the distinction between the two should be clear from the context.

in strict sequence, and executing a single instruction involves performing a series of steps. The processor reads the instruction from memory pointed at by the program counter (PC), interprets the bits in the instruction, performs some simple operation dictated by the instruction, and then updates the PC to point to the next instruction, which may or may not be contiguous in memory to the instruction that was just executed.

There are only a few of these simple operations, and they revolve around main memory, the register file, and the arithmetic/logic unit (ALU). The register file is a small storage device that consists of a collection of word-size registers, each with its own unique name. The ALU computes new data and address values. Here are some examples of the simple operations that the CPU might carry out at the request of an instruction:

- Load: Copy a byte or a word from main memory into a register, overwriting the previous contents of the register.
- Store: Copy a byte or a word from a register to a location in main memory, overwriting the previous contents of that location.
- Operate: Copy the contents of two registers to the ALU, perform an arithmetic operation on the two words, and store the result in a register, overwriting the previous contents of that register.
- Jump: Extract a word from the instruction itself and copy that word into the program counter (PC), overwriting the previous value of the PC.

We say that a processor appears to be a simple implementation of its instruction set architecture, but in fact modern processors use far more complex mechanisms to speed up program execution. Thus, we can distinguish the processor's instruction set architecture, describing the effect of each machine-code instruction, from its microarchitecture, describing how the processor is actually implemented. When we study machine code in Chapter 3, we will consider the abstraction provided by the machine's instruction set architecture. Chapter 4 has more to say about how processors are actually implemented. Chapter 5 describes a model of how modern processors work that enables predicting and optimizing the performance of machine-language programs.

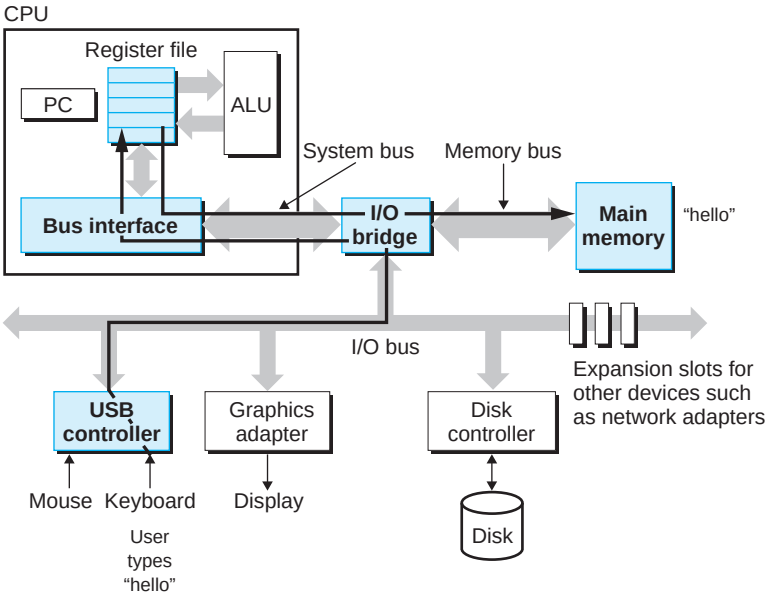
1.4.2 Running the `hello` Program

Given this simple view of a system's hardware organization and operation, we can begin to understand what happens when we run our example program. We must omit a lot of details here that will be filled in later, but for now we will be content with the big picture.

Initially, the shell program is executing its instructions, waiting for us to type a command. As we type the characters `./hello` at the keyboard, the shell program reads each one into a register and then stores it in memory, as shown in Figure 1.5.

When we hit the `enter` key on the keyboard, the shell knows that we have finished typing the command. The shell then loads the executable `hello` file by executing a sequence of instructions that copies the code and data in the `hello`

Figure 1.5
Reading the `hello` command from the keyboard.



object file from disk to main memory. The data includes the string of characters `hello, world\n` that will eventually be printed out.

Using a technique known as direct memory access (DMA, discussed in Chapter 6), the data travel directly from disk to main memory, without passing through the processor. This step is shown in Figure 1.6.

Once the code and data in the `hello` object file are loaded into memory, the processor begins executing the machine-language instructions in the `hello` program's `main` routine. These instructions copy the bytes in the `hello, world\n` string from memory to the register file, and from there to the display device, where they are displayed on the screen. This step is shown in Figure 1.7.

1.5 Caches Matter

An important lesson from this simple example is that a system spends a lot of time moving information from one place to another. The machine instructions in the `hello` program are originally stored on disk. When the program is loaded, they are copied to main memory. As the processor runs the program, instructions are copied from main memory into the processor. Similarly, the data string `hello,world\n`, originally on disk, is copied to main memory and then copied from main memory to the display device. From a programmer's perspective, much of this copying is overhead that slows down the "real work" of the program. Thus, a major goal for system designers is to make these copy operations run as fast as possible.

Because of physical laws, larger storage devices are slower than smaller storage devices. And faster devices are more expensive to build than their slower

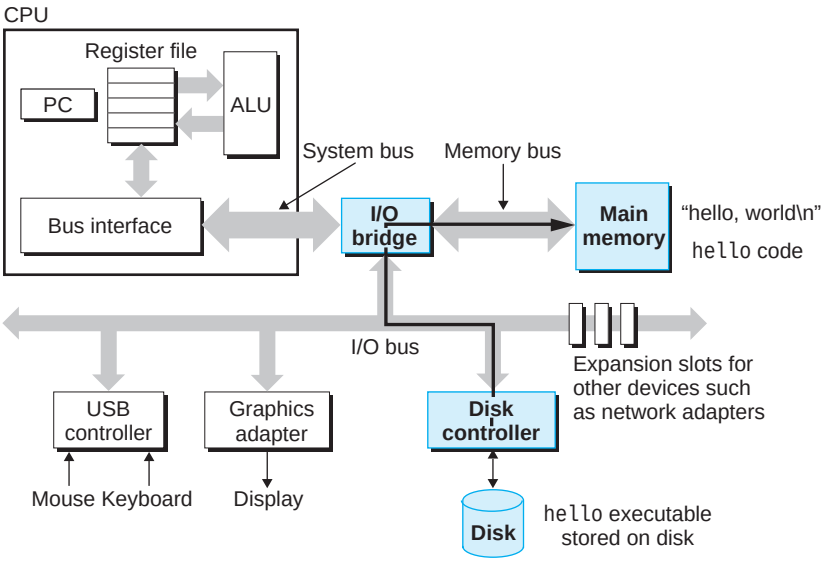


Figure 1.6 Loading the executable from disk into main memory.

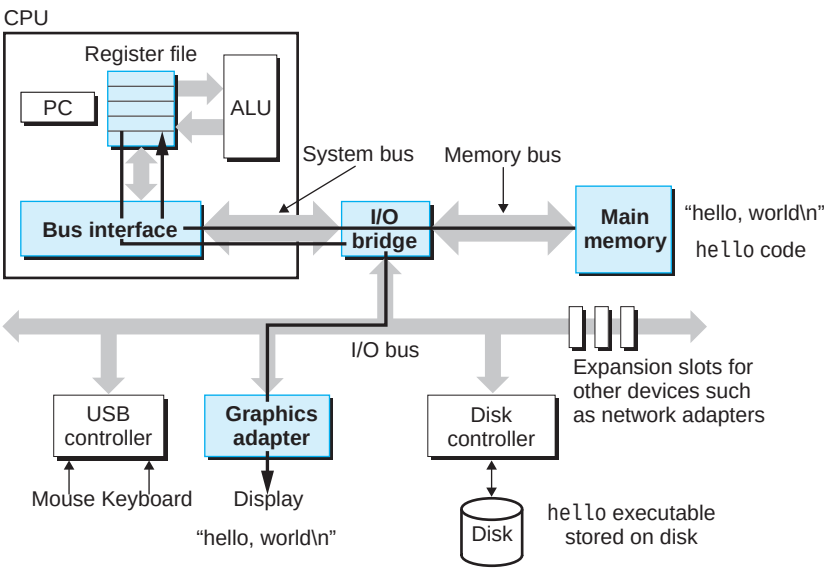
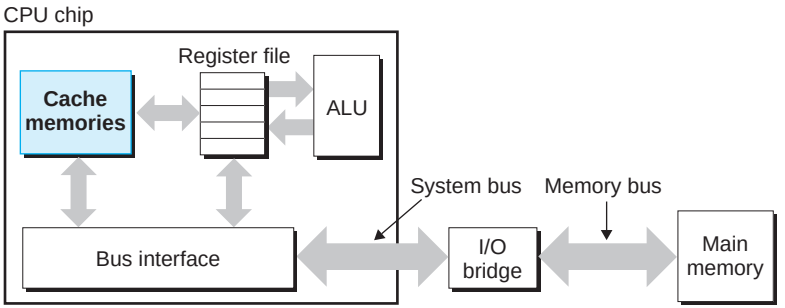


Figure 1.7 Writing the output string from memory to the display.

Figure 1.8
Cache memories.



counterparts. For example, the disk drive on a typical system might be 1,000 times larger than the main memory, but it might take the processor 10,000,000 times longer to read a word from disk than from memory.

Similarly, a typical register file stores only a few hundred bytes of information, as opposed to billions of bytes in the main memory. However, the processor can read data from the register file almost 100 times faster than from memory. Even more troublesome, as semiconductor technology progresses over the years, this processor–memory gap continues to increase. It is easier and cheaper to make processors run faster than it is to make main memory run faster.

To deal with the processor–memory gap, system designers include smaller, faster storage devices called cache memories (or simply caches) that serve as temporary staging areas for information that the processor is likely to need in the near future. Figure 1.8 shows the cache memories in a typical system. An L1 cache on the processor chip holds tens of thousands of bytes and can be accessed nearly as fast as the register file. A larger L2 cache with hundreds of thousands to millions of bytes is connected to the processor by a special bus. It might take 5 times longer for the processor to access the L2 cache than the L1 cache, but this is still 5 to 10 times faster than accessing the main memory. The L1 and L2 caches are implemented with a hardware technology known as static random access memory (SRAM). Newer and more powerful systems even have three levels of cache: L1, L2, and L3. The idea behind caching is that a system can get the effect of both a very large memory and a very fast one by exploiting locality, the tendency for programs to access data and code in localized regions. By setting up caches to hold data that are likely to be accessed often, we can perform most memory operations using the fast caches.

One of the most important lessons in this book is that application programmers who are aware of cache memories can exploit them to improve the performance of their programs by an order of magnitude. You will learn more about these important devices and how to exploit them in Chapter 6.

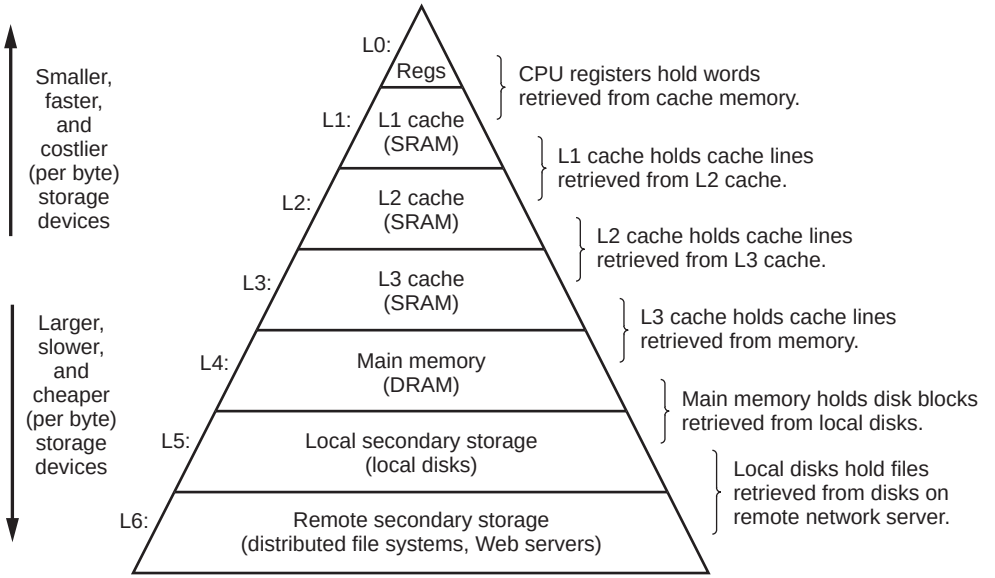


Figure 1.9 An example of a memory hierarchy.

1.6 Storage Devices Form a Hierarchy

This notion of inserting a smaller, faster storage device (e.g., cache memory) between the processor and a larger, slower device (e.g., main memory) turns out to be a general idea. In fact, the storage devices in every computer system are organized as a memory hierarchy similar to Figure 1.9. As we move from the top of the hierarchy to the bottom, the devices become slower, larger, and less costly per byte. The register file occupies the top level in the hierarchy, which is known as level 0 or L0. We show three levels of caching L1 to L3, occupying memory hierarchy levels 1 to 3. Main memory occupies level 4, and so on.

The main idea of a memory hierarchy is that storage at one level serves as a cache for storage at the next lower level. Thus, the register file is a cache for the L1 cache. Caches L1 and L2 are caches for L2 and L3, respectively. The L3 cache is a cache for the main memory, which is a cache for the disk. On some networked systems with distributed file systems, the local disk serves as a cache for data stored on the disks of other systems.

Just as programmers can exploit knowledge of the different caches to improve performance, programmers can exploit their understanding of the entire memory hierarchy. Chapter 6 will have much more to say about this.

1.7

Figure 1.10
Layered view of a computer system.

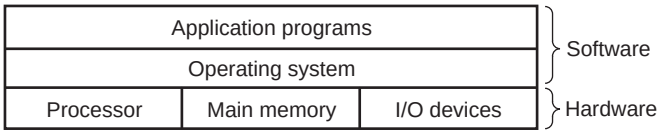
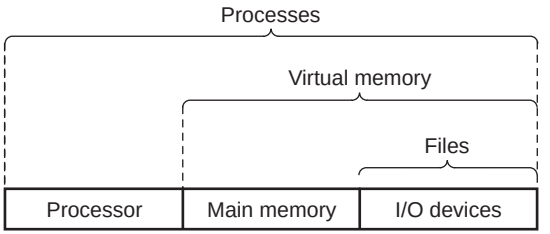


Figure 1.11
Abstractions provided by an operating system.



keyboard, display, disk, or main memory directly. Rather, they relied on the services provided by the operating system. We can think of the operating system as a layer of software interposed between the application program and the hardware, as shown in Figure 1.10. All attempts by an application program to manipulate the hardware must go through the operating system.

The operating system has two primary purposes: (1) to protect the hardware from misuse by runaway applications and (2) to provide applications with simple and uniform mechanisms for manipulating complicated and often wildly different low-level hardware devices. The operating system achieves both goals via the fundamental abstractions shown in Figure 1.11: processes, virtual memory, and files. As this figure suggests, files are abstractions for I/O devices, virtual memory is an abstraction for both the main memory and disk I/O devices, and processes are abstractions for the processor, main memory, and I/O devices. We will discuss each in turn.

1.7.1 Processes

When a program such as `hello` runs on a modern system, the operating system provides the illusion that the program is the only one running on the system. The program appears to have exclusive use of both the processor, main memory, and I/O devices. The processor appears to execute the instructions in the program, one after the other, without interruption. And the code and data of the program appear to be the only objects in the system's memory. These illusions are provided by the notion of a process, one of the most important and successful ideas in computer science.

A process is the operating system's abstraction for a running program. Multiple processes can run concurrently on the same system, and each process appears to have exclusive use of the hardware. By concurrently, we mean that the instructions of one process are interleaved with the instructions of another process. In most systems, there are more processes to run than there are CPUs to run them.

Aside Unix, Posix, and the Standard Unix Specification

The 1960s was an era of huge, complex operating systems, such as IBM's OS/360 and Honeywell's Multics systems. While OS/360 was one of the most successful software projects in history, Multics dragged on for years and never achieved wide-scale use. Bell Laboratories was an original partner in the Multics project but dropped out in 1969 because of concern over the complexity of the project and the lack of progress. In reaction to their unpleasant Multics experience, a group of Bell Labs researchers—Ken Thompson, Dennis Ritchie, Doug McIlroy, and Joe Ossanna—began work in 1969 on a simpler operating system for a Digital Equipment Corporation PDP-7 computer, written entirely in machine language. Many of the ideas in the new system, such as the hierarchical file system and the notion of a shell as a user-level process, were borrowed from Multics but implemented in a smaller, simpler package. In 1970, Brian Kernighan dubbed the new system "Unix" as a pun on the complexity of "Multics." The kernel was rewritten in C in 1973, and Unix was announced to the outside world in 1974 [93].

Because Bell Labs made the source code available to schools with generous terms, Unix developed a large following at universities. The most influential work was done at the University of California at Berkeley in the late 1970s and early 1980s, with Berkeley researchers adding virtual memory and the Internet protocols in a series of releases called Unix 4.xBSD (Berkeley Software Distribution). Concurrently, Bell Labs was releasing their own versions, which became known as System V Unix. Versions from other vendors, such as the Sun Microsystems Solaris system, were derived from these original BSD and System V versions.

Trouble arose in the mid 1980s as Unix vendors tried to differentiate themselves by adding new and often incompatible features. To combat this trend, IEEE (Institute for Electrical and Electronics Engineers) sponsored an effort to standardize Unix, later dubbed "Posix" by Richard Stallman. The result was a family of standards, known as the Posix standards, that cover such issues as the C language interface for Unix system calls, shell programs and utilities, threads, and network programming. More recently, a separate standardization effort, known as the "Standard Unix Specification," has joined forces with Posix to create a single, unified standard for Unix systems. As a result of these standardization efforts, the differences between Unix versions have largely disappeared.

Traditional systems could only execute one program at a time, while newer multi-core processors can execute several programs simultaneously. In either case, a single CPU can appear to execute multiple processes concurrently by having the processor switch among them. The operating system performs this interleaving with a mechanism known as context switching. To simplify the rest of this discussion, we consider only a uniprocessor system containing a single CPU. We will return to the discussion of multiprocessor systems in Section 1.9.2.

The operating system keeps track of all the state information that the process needs in order to run. This state, which is known as the context, includes information such as the current values of the PC, the register file, and the contents of main memory. At any point in time, a uniprocessor system can only execute the code for a single process. When the operating system decides to transfer control from the current process to some new process, it performs a context switch by saving the context of the current process, restoring the context of the new process, and

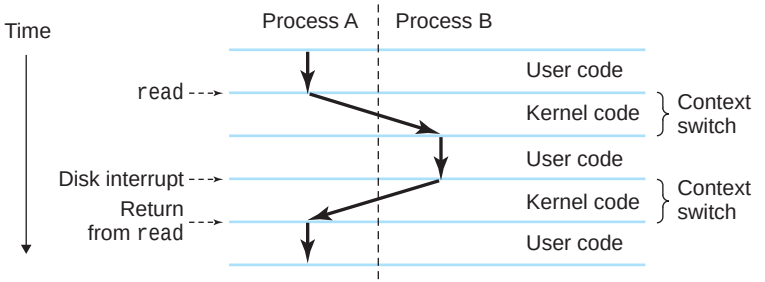


Figure 1.12
Process context switching.

then passing control to the new process. The new process picks up exactly where it left off. Figure 1.12 shows the basic idea for our example `hello` scenario.

There are two concurrent processes in our example scenario: the shell process and the `hello` process. Initially, the shell process is running alone, waiting for input on the command line. When we ask it to run the `hello` program, the shell carries out our request by invoking a special function known as a system call that passes control to the operating system. The operating system saves the shell's context, creates a new `hello` process and its context, and then passes control to the new `hello` process. After `hello` terminates, the operating system restores the context of the shell process and passes control back to it, where it waits for the next command-line input.

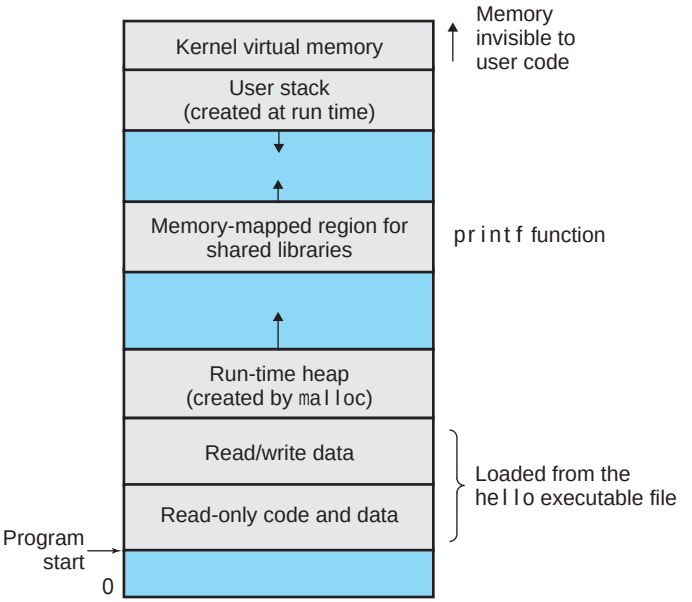
As Figure 1.12 indicates, the transition from one process to another is managed by the operating system kernel. The kernel is the portion of the operating system code that is always resident in memory. When an application program requires some action by the operating system, such as to read or write a file, it executes a special system call instruction, transferring control to the kernel. The kernel then performs the requested operation and returns back to the application program. Note that the kernel is not a separate process. Instead, it is a collection of code and data structures that the system uses to manage all the processes.

Implementing the process abstraction requires close cooperation between both the low-level hardware and the operating system software. We will explore how this works, and how applications can create and control their own processes, in Chapter 8.

1.7.2 Threads

Although we normally think of a process as having a single control flow, in modern systems a process can actually consist of multiple execution units, called threads, each running in the context of the process and sharing the same code and global data. Threads are an increasingly important programming model because of the requirement for concurrency in network servers, because it is easier to share data between multiple threads than between multiple processes, and because threads are typically more efficient than processes. Multi-threading is also one way to make programs run faster when multiple processors are available, as we will discuss in

Figure 1.13
Process virtual address space. (Typical)



Section 1.9.2. You will learn the basic concepts of concurrency, including how to write threaded programs, in Chapter 12.

1.7.3 Virtual Memory

Virtual memory is an abstraction that provides each process with the illusion that it has exclusive use of the main memory. Each process has the same uniform view of memory, which is known as its virtual address space. The virtual address space for Linux processes is shown in Figure 1.13. (Other Unix systems use a similar layout.) In Linux, the topmost region of the address space is reserved for code and data in the operating system that is common to all processes. The lower region of the address space holds the code and data defined by the user's process. Note that addresses in the figure increase from the bottom to the top.

The virtual address space seen by each process consists of a number of well-defined areas, each with a specific purpose. You will learn more about these areas later in the book, but it will be helpful to look briefly at each, starting with the lowest addresses and working our way up:

- Program code and data. Code begins at the same fixed address for all processes, followed by data locations that correspond to global C variables. The code and data areas are initialized directly from the contents of an executable object file—in our case, the `hello` executable. You will learn more about this part of the address space when we study linking and loading in Chapter 7.
- Heap. The code and data areas are followed immediately by the run-time heap. Unlike the code and data areas, which are fixed in size once the process begins

running, the heap expands and contracts dynamically at run time as a result of calls to C standard library routines such as `malloc` and `free`. We will study heaps in detail when we learn about managing virtual memory in Chapter 9.

- Shared libraries. Near the middle of the address space is an area that holds the code and data for shared libraries such as the C standard library and the math library. The notion of a shared library is a powerful but somewhat difficult concept. You will learn how they work when we study dynamic linking in Chapter 7.
- Stack. At the top of the user's virtual address space is the user stack that the compiler uses to implement function calls. Like the heap, the user stack expands and contracts dynamically during the execution of the program. In particular, each time we call a function, the stack grows. Each time we return from a function, it contracts. You will learn how the compiler uses the stack in Chapter 3.
- Kernel virtual memory. The top region of the address space is reserved for the kernel. Application programs are not allowed to read or write the contents of this area or to directly call functions defined in the kernel code. Instead, they must invoke the kernel to perform these operations.

For virtual memory to work, a sophisticated interaction is required between the hardware and the operating system software, including a hardware translation of every address generated by the processor. The basic idea is to store the contents of a process's virtual memory on disk and then use the main memory as a cache for the disk. Chapter 9 explains how this works and why it is so important to the operation of modern systems.

1.7.4 Files

A file is a sequence of bytes, nothing more and nothing less. Every I/O device, including disks, keyboards, displays, and even networks, is modeled as a file. All input and output in the system is performed by reading and writing files, using a small set of system calls known as Unix I/O.

This simple and elegant notion of a file is nonetheless very powerful because it provides applications with a uniform view of all the varied I/O devices that might be contained in the system. For example, application programmers who manipulate the contents of a disk file are blissfully unaware of the specific disk technology. Further, the same program will run on different systems that use different disk technologies. You will learn about Unix I/O in Chapter 10.

1.8 Systems Communicate with Other Systems Using Networks

Up to this point in our tour of systems, we have treated a system as an isolated collection of hardware and software. In practice, modern systems are often linked to other systems by networks. From the point of view of an individual system, the

Aside T Li r$_0$ t

In August 1991, a Finnish graduate student named Linus Torvalds modestly announced a new Unix-like operating system kernel:

```
From: torvalds@klaava.Helsinki.FI (Linus Benedict Torvalds)
Newsgroups: comp.os.minix
Subject: What would you like to see most in minix?
Summary: small poll for my new operating system
Date: 25 Aug 91 20:57:08 GMT

Hello everybody out there using minix -
I'm doing a (free) operating system (just a hobby, won't be big and
professional like gnu) for 386(486) AT clones. This has been brewing
since April, and is starting to get ready. I'd like any feedback on
things people like/dislike in minix, as my OS resembles it somewhat
(same physical layout of the file-system (due to practical reasons)
among other things).

I've currently ported bash(1.08) and gcc(1.40), and things seem to work.
This implies that I'll get something practical within a few months, and
I'd like to know what features most people would want. Any suggestions
are welcome, but I won't promise I'll implement them :-)

Linus (torvalds@kruuna.helsinki.fi)
```

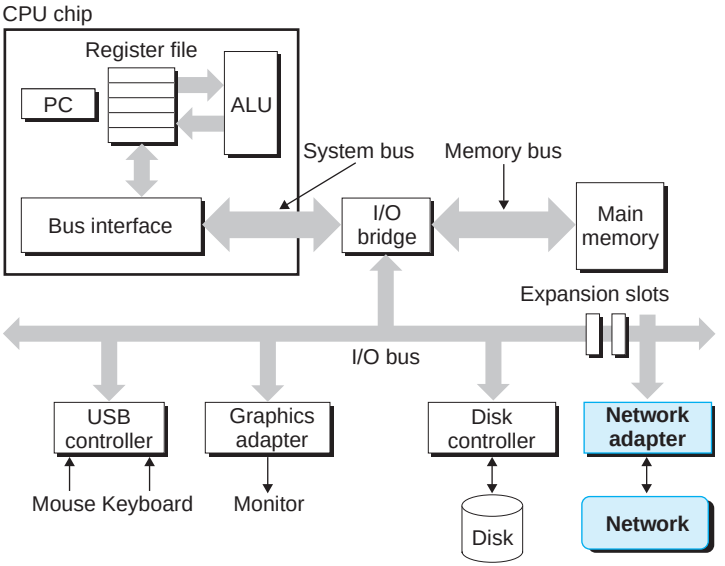
As Torvalds indicates, his starting point for creating Linux was Minix, an operating system developed by Andrew S. Tanenbaum for educational purposes [113].

The rest, as they say, is history. Linux has evolved into a technical and cultural phenomenon. By combining forces with the GNU project, the Linux project has developed a complete, Posix-compliant version of the Unix operating system, including the kernel and all of the supporting infrastructure. Linux is available on a wide array of computers, from handheld devices to mainframe computers. A group at IBM has even ported Linux to a wristwatch!

network can be viewed as just another I/O device, as shown in Figure 1.14. When the system copies a sequence of bytes from main memory to the network adapter, the data flow across the network to another machine, instead of, say, to a local disk drive. Similarly, the system can read data sent from other machines and copy these data to its main memory.

With the advent of global networks such as the Internet, copying information from one machine to another has become one of the most important uses of computer systems. For example, applications such as email, instant messaging, the World Wide Web, FTP, and telnet are all based on the ability to copy information over a network.

Figure 1.14
A network is another I/O device.



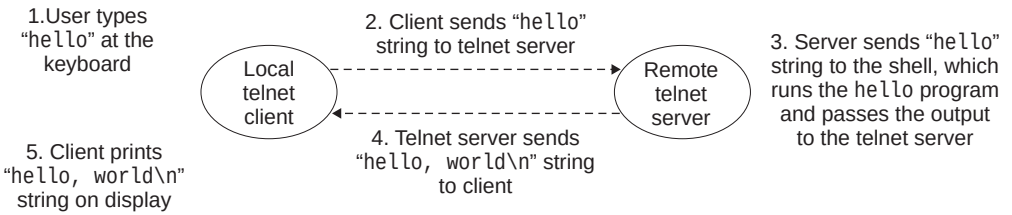


Figure 1.15 Using telnet to run `hello` remotely over a network.

Returning to our `hello` example, we could use the familiar telnet application to run `hello` on a remote machine. Suppose we use a telnet client running on our local machine to connect to a telnet server on a remote machine. After we log in to the remote machine and run a shell, the remote shell is waiting to receive an input command. From this point, running the `hello` program remotely involves the five basic steps shown in Figure 1.15.

After we type in the `hello` string to the telnet client and hit the `enter` key, the client sends the string to the telnet server. After the telnet server receives the string from the network, it passes it along to the remote shell program. Next, the remote shell runs the `hello` program and passes the output line back to the telnet server. Finally, the telnet server forwards the output string across the network to the telnet client, which prints the output string on our local terminal.

This type of exchange between clients and servers is typical of all network applications. In Chapter 11 you will learn how to build network applications and apply this knowledge to build a simple Web server.

1.9 Important Themes

This concludes our initial whirlwind tour of systems. An important idea to take away from this discussion is that a system is more than just hardware. It is a collection of intertwined hardware and systems software that must cooperate in order to achieve the ultimate goal of running application programs. The rest of this book will fill in some details about the hardware and the software, and it will show how, by knowing these details, you can write programs that are faster, more reliable, and more secure.

To close out this chapter, we highlight several important concepts that cut across all aspects of computer systems. We will discuss the importance of these concepts at multiple places within the book.

1.9.1 Amdahl's Law

Gene Amdahl, one of the early pioneers in computing, made a simple but insightful observation about the effectiveness of improving the performance of one part of a system. This observation has come to be known as Amdahl's law. The main idea is that when we speed up one part of a system, the effect on the overall system performance depends on both how significant this part was and how much it sped up. Consider a system in which executing some application requires time T_{old}. Suppose some part of the system requires a fraction α of this time, and that we improve its performance by a factor of k. That is, the component originally required time αT_{old}, and it now requires time $(\alpha T_{old})/k$. The overall execution time would thus be

$$\begin{aligned} T_{new} &= (1 - \alpha)T_{old} + (\alpha T_{old})/k \\ &= T_{old}[(1 - \alpha) + \alpha/k] \end{aligned}$$

From this, we can compute the speedup $S = T_{old}/T_{new}$ as

$$S = \frac{1}{(1 - \alpha) + \alpha/k} \tag{1.1}$$

As an example, consider the case where a part of the system that initially consumed 60% of the time ($\alpha = 0.6$) is sped up by a factor of 3 ($k = 3$). Then we get a speedup of $1/[0.4 + 0.6/3] = 1.67\times$. Even though we made a substantial improvement to a major part of the system, our net speedup was significantly less than the speedup for the one part. This is the major insight of Amdahl's law—to significantly speed up the entire system, we must improve the speed of a very large fraction of the overall system.

Practice Problem 1.1 (solution page 64)

Suppose you work as a truck driver, and you have been hired to carry a load of potatoes from Boise, Idaho, to Minneapolis, Minnesota, a total distance of 2,500 kilometers. You estimate you can average 100 km/hr driving within the speed limits, requiring a total of 25 hours for the trip.

Aside: Expressing relative change

The best way to express a performance improvement is as a ratio of the form T_{old}/T_{new}, where T_{old} is the time required for the original version and T_{new} is the time required by the modified version. This will be a number greater than 1.0 if any real improvement occurred. We use the suffix '×' to indicate such a ratio, where the factor "2.2×" is expressed verbally as "2.2 times."

The more traditional way of expressing relative change as a percentage works well when the change is small, but its definition is ambiguous. Should it be $100 \cdot (T_{old} - T_{new})/T_{new}$, or possibly $100 \cdot (T_{old} - T_{new})/T_{old}$, or something else? In addition, it is less instructive for large changes. Saying that "performance improved by 120%" is more difficult to comprehend than simply saying that the performance improved by 2.2×.

A. You hear on the news that Montana has just abolished its speed limit, which constitutes 1,500 km of the trip. Your truck can travel at 150 km/hr. What will be your speedup for the trip?

B. You can buy a new turbocharger for your truck at www.fasttrucks.com. They stock a variety of models, but the faster you want to go, the more it will cost. How fast must you travel through Montana to get an overall speedup for your trip of 1.67×?

Practice Problem 1.2 (solution page 64)

A car manufacturing company has promised their customers that the next release of a new engine will show a 4× performance improvement. You have been assigned the task of delivering on that promise. You have determined that only 90% of the engine can be improved. How much (i.e., what value of k) would you need to improve this part to meet the overall performance target of the engine?

One interesting special case of Amdahl's law is to consider the effect of setting k to ∞. That is, we are able to take some part of the system and speed it up to the point at which it takes a negligible amount of time. We then get

$$S_\infty = \frac{1}{(1 - \alpha)} \tag{1.2}$$

So, for example, if we can speed up 60% of the system to the point where it requires close to no time, our net speedup will still only be $1/0.4 = 2.5\times$.

Amdahl's law describes a general principle for improving any process. In addition to its application to speeding up computer systems, it can guide a company trying to reduce the cost of manufacturing razor blades, or a student trying to improve his or her grade point average. Perhaps it is most meaningful in the world

of computers, where we routinely improve performance by factors of 2 or more. Such high factors can only be achieved by optimizing large parts of a system.

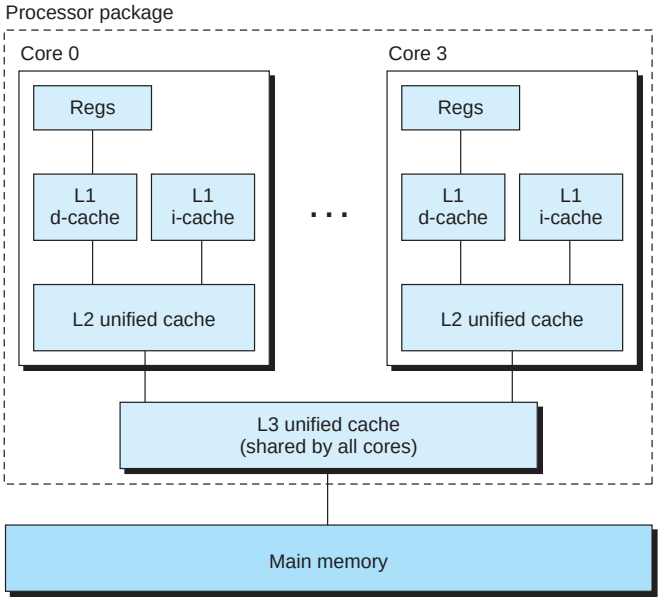
1.9.2 Concurrency and Parallelism

Throughout the history of digital computers, two demands have been constant forces in driving improvements: we want them to do more, and we want them to run faster. Both of these factors improve when the processor does more things at once. We use the term concurrency to refer to the general concept of a system with multiple, simultaneous activities, and the term parallelism to refer to the use of concurrency to make a system run faster. Parallelism can be exploited at multiple levels of abstraction in a computer system. We highlight three levels here, working from the highest to the lowest level in the system hierarchy.

T r a -L I C rr

Building on the process abstraction, we are able to devise systems where multiple programs execute at the same time, leading to concurrency. With threads, we can even have multiple control flows executing within a single process. Support for concurrent execution has been fo204.r /T1 (parall)r /Tise r /Tii(perfr /T(at)-3 /Tadve

Figure 1.17
Multi-core processor organization. For illustrative purposes.



typical multi-core processor, where the chip has four CPU cores, each with its own L1 and L2 caches, and with each L1 cache split into two parts—one to hold recently fetched instructions and one to hold data. The cores share higher levels of cache as well as the interface to main memory. Industry experts predict that they will be able to have dozens, and ultimately hundreds, of cores on a single chip.

Hyperthreading, sometimes called simultaneous multi-threading, is a technique that allows a single CPU to execute multiple flows of control. It involves having multiple copies of some of the CPU hardware, such as program counters and register files, while having only single copies of other parts of the hardware, such as the units that perform floating-point arithmetic. Whereas a conventional processor requires around 20,000 clock cycles to shift between different threads, a hyperthreaded processor decides which of its threads to execute on a cycle-by-cycle basis. It enables the CPU to take better advantage of its processing resources.

the hardware. Chapter 12 will look much more deeply into concurrency and its use to provide a sharing of processing resources and to enable more parallelism in program execution.

Instruction-Level Parallelism

At a much lower level of abstraction, modern processors can execute multiple instructions at one time, a property known as instruction-level parallelism. For example, early microprocessors, such as the 1978-vintage Intel 8086, required multiple (typically 3–10) clock cycles to execute a single instruction. More recent processors can sustain execution rates of 2–4 instructions per clock cycle. Any given instruction requires much longer from start to finish, perhaps 20 cycles or more, but the processor uses a number of clever tricks to process as many as 100 instructions at a time. In Chapter 4, we will explore the use of pipelining, where the actions required to execute an instruction are partitioned into different steps and the processor hardware is organized as a series of stages, each performing one of these steps. The stages can operate in parallel, working on different parts of different instructions. We will see that a fairly simple hardware design can sustain an execution rate close to 1 instruction per clock cycle.

Processors that can sustain execution rates faster than 1 instruction per cycle are known as superscalar processors. Most modern processors support superscalar operation. In Chapter 5, we will describe a high-level model of such processors. We will see that application programmers can use this model to understand the performance of their programs. They can then write programs such that the generated code achieves higher degrees of instruction-level parallelism and therefore runs faster.

SIMD Parallelism

At the lowest level, many modern processors have special hardware that allows a single instruction to cause multiple operations to be performed in parallel, a mode known as single-instruction, multiple-data (SIMD) parallelism. For example, recent generations of Intel and AMD processors have instructions that can add 8 pairs of single-precision floating-point numbers (C data type `float`) in parallel.

These SIMD instructions are provided mostly to speed up applications that process image, sound, and video data. Although some compilers attempt to automatically extract SIMD parallelism from C programs, a more reliable method is to write programs using special vector data types supported in compilers such as GCC. We describe this style of programming in Web Aside OPT:SIMD, as a supplement to the more general presentation on program optimization found in Chapter 5.

1.9.3 The Importance of Abstractions in Computer Systems

The use of abstractions is one of the most important concepts in computer science. For example, one aspect of good programming practice is to formulate a simple application program interface (API) for a set of functions that allow programmers to use the code without having to delve into its inner workings. Different program-

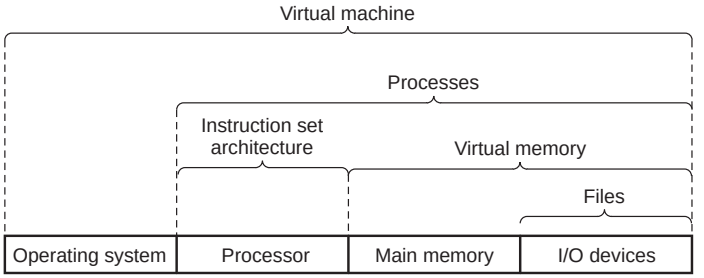


Figure 1.18
Some abstractions provided by a computer system. A appropriate abstraction is provided by every system component that is visible to a higher level.

ming languages provide different forms and levels of support for abstraction, such as Java class declarations and C function prototypes.

We have already been introduced to several of the abstractions seen in computer systems, as indicated in Figure 1.18. On the processor side, the instruction set architecture provides an abstraction of the actual processor hardware. With this abstraction, a machine-code program behaves as if it were executed on a processor that performs just one instruction at a time. The underlying hardware is far more elaborate, executing multiple instructions in parallel, but always in a way that is consistent with the simple, sequential model. By keeping the same execution model, different processor implementations can execute the same machine code while offering a range of cost and performance.

On the operating system side, we have introduced three abstractions: files as an abstraction of I/O devices, virtual memory as an abstraction of program memory, and processes as an abstraction of a running program. To these abstractions we add a new one: the virtual machine, providing an abstraction of the entire computer, including the operating system, the processor, and the programs. The idea of a virtual machine was introduced by IBM in the 1960s, but it has become more prominent recently as a way to manage computers that must be able to run programs designed for multiple operating systems (such as Microsoft Windows, Mac OS X, and Linux) or different versions of the same operating system.

We will return to these abstractions in subsequent sections of the book.

1.10 Summary

A computer system consists of hardware and systems software that cooperate to run application programs. Information inside the computer is represented as groups of bits that are interpreted in different ways, depending on the context. Programs are translated by other programs into different forms, beginning as ASCII text and then translated by compilers and linkers into binary executable files.

Processors read and interpret binary instructions that are stored in main memory. Since computers spend most of their time copying data between memory, I/O devices, and the CPU registers, the storage devices in a system are arranged in a hierarchy, with the CPU registers at the top, followed by multiple levels of hardware cache memories, DRAM main memory, and disk storage. Storage devices that are higher in the hierarchy are faster and more costly per bit than those lower in the

hierarchy. Storage devices that are higher in the hierarchy serve as caches for devices that are lower in the hierarchy. Programmers can optimize the performance of their C programs by understanding and exploiting the memory hierarchy.

The operating system kernel serves as an intermediary between the application and the hardware. It provides three fundamental abstractions: (1) Files are abstractions for I/O devices. (2) Virtual memory is an abstraction for both main memory and disks. (3) Processes are abstractions for the processor, main memory, and I/O devices.

Finally, networks provide ways for computer systems to communicate with one another. From the viewpoint of a particular system, the network is just another I/O device.

Bibliographic Notes

Ritchie has written interesting firsthand accounts of the early days of C and Unix [91, 92]. Ritchie and Thompson presented the first published account of Unix [93]. Silberschatz, Galvin, and Gagne [102] provide a comprehensive history of the different flavors of Unix. The GNU (www.gnu.org) and Linux (www.linux.org) Web pages have loads of current and historical information. The Posix standards are available online at (www.unix.org).

Solutions to Practice Problems

Solution to Problem 1.1 (page 58)

This problem illustrates that Amdahl's law applies to more than just computer systems.

A. In terms of Equation 1.1, we have $\alpha = 0.6$ and $k = 1.5$. More directly, traveling the 1,500 kilometers through Montana will require 10 hours, and the rest of the trip also requires 10 hours. This will give a speedup of $25/(10 + 10) = 1.25\times$.

B. In terms of Equation 1.1, we have $\alpha = 0.6$, and we require $S = 1.67$, from which we can solve for k. More directly, to speed up the trip by $1.67\times$, we must decrease the overall time to 15 hours. The parts outside of Montana will still require 10 hours, so we must drive through Montana in 5 hours. This requires traveling at 300 km/hr, which is pretty fast for a truck!

Solution to Problem 1.2 (page 59)

Amdahl's law is best understood by working through some examples. This one requires you to look at Equation 1.1 from an unusual perspective. This problem is a simple application of the equation. You are given $S = 4$ and $\alpha = 0.9$, and you must then solve for k:

$$4 = 1/(1 - 0.9) + 0.9/k$$

$$0.4 + 3.6/k = 1.0$$

$$k = 6.0$$

Part I

Program Structure and Execution

Our exploration of computer systems starts by studying the computer itself, comprising a processor and a memory subsystem. At the core, we require ways to represent basic data types, such as approximations to integer and real arithmetic. From there, we can consider how machine-level instructions manipulate data and how a compiler translates C programs into these instructions. Next, we study several methods of implementing a processor to gain a better understanding of how hardware resources are used to execute instructions. Once we understand compilers and machine-level code, we can examine how to maximize program performance by writing C programs that, when compiled, achieve the maximum possible performance. We conclude with the design of the memory subsystem, one of the most complex components of a modern computer system.

This part of the book will give you a deep understanding of how application programs are represented and executed. You will gain skills that help you write programs that are secure, reliable, and make the best use of the computing resources.

2

Representing and Manipulating Information

2.1 Information Storage 70

2.2 Integer Representations 95

2.3 Integer Arithmetic 120

2.4 Floating Point 144

2.5 Summary 162

Bibliographic Notes 163

Homework Problems 164

Solutions to Practice Problems 179

Modern computers store and process information represented as two-valued signals. These lowly binary digits, or bits, form the basis of the digital revolution. The familiar decimal, or base-10, representation has been in use for over 1,000 years, having been developed in India, improved by Arab mathematicians in the 12th century, and brought to the West in the 13th century by the Italian mathematician Leonardo Pisano (ca. 1170 to ca. 1250), better known as Fibonacci. Using decimal notation is natural for 10-fingered humans, but binary values work better when building machines that store and process information. Two-valued signals can readily be represented, stored, and transmitted—for example, as the presence or absence of a hole in a punched card, as a high or low voltage on a wire, or as a magnetic domain oriented clockwise or counterclockwise. The electronic circuitry for storing and performing computations on two-valued signals is very simple and reliable, enabling manufacturers to integrate millions, or even billions, of such circuits on a single silicon chip.

In isolation, a single bit is not very useful. When we group bits together and apply some interpretation that gives meaning to the different possible bit patterns, however, we can represent the elements of any finite set. For example, using a binary number system, we can use groups of bits to encode nonnegative numbers. By using a standard character code, we can encode the letters and symbols in a document. We cover both of these encodings in this chapter, as well as encodings to represent negative numbers and to approximate real numbers.

We consider the three most important representations of numbers. Unsigned encodings are based on traditional binary notation, representing numbers greater than or equal to 0.

The computer might not generate the expected result, but at least it is consistent!

Floating-point arithmetic has altogether different mathematical properties. The product of a set of positive numbers will always be positive, although overflow will yield the special value $+\infty$. Floating-point arithmetic is not associative due to the finite precision of the representation. For example, the C expression `(3.14+1e20)-1e20` will evaluate to 0.0 on most machines, while `3.14+(1e20-1e20)` will evaluate to 3.14. The different mathematical properties of integer versus floating-point arithmetic stem from the difference in how they handle the finiteness of their representations—integer representations can encode a comparatively small range of values, but do so precisely, while floating-point representations can encode a wide range of values, but only approximately.

By studying the actual number representations, we can understand the ranges of values that can be represented and the properties of the different arithmetic operations. This understanding is critical to writing programs that work correctly over the full range of numeric values and that are portable across different combinations of machine, operating system, and compiler. As we will describe, a number of computer security vulnerabilities have arisen due to some of the subtleties of computer arithmetic. Whereas in an earlier era program bugs would only inconvenience people when they happened to be triggered, there are now legions of hackers who try to exploit any bug they can find to obtain unauthorized access to other people's systems. This puts a higher level of obligation on programmers to understand how their programs work and how they can be made to behave in undesirable ways.

Computers use several different binary representations to encode numeric values. You will need to be familiar with these representations as you progress into machine-level programming in Chapter 3. We describe these encodings in this chapter and show you how to reason about number representations.

We derive several ways to perform arithmetic operations by directly manipulating the bit-level representations of numbers. Understanding these techniques will be important for understanding the machine-level code generated by compilers in their attempt to optimize the performance of arithmetic expression evaluation.

Our treatment of this material is based on a core set of mathematical principles. We start with the basic definitions of the encodings and then derive such properties as the range of representable numbers, their bit-level representations, and the properties of the arithmetic operations. We believe it is important for you to examine the material from this abstract viewpoint, because programmers need to have a clear understanding of how computer arithmetic relates to the more familiar integer and real arithmetic.

The C++ programming language is built upon C, using the exact same numeric representations and operations. Everything said in this chapter about C also holds for C++. The Java language definition, on the other hand, created a new set of standards for numeric representations and operations. Whereas the C standards are designed to allow a wide range of implementations, the Java standard is quite specific on the formats and encodings of data. We highlight the representations and operations supported by Java at several places in the chapter.

Aside How to read this book

In this chapter, we examine the fundamental properties of how numbers and other forms of data are represented on a computer and the properties of the operations that computers perform on these data. This requires us to delve into the language of mathematics, writing formulas and equations and showing derivations of important properties.

To help you navigate this exposition, we have structured the presentation to first state a property as a principle in mathematical notation. We then illustrate this principle with examples and an informal discussion. We recommend that you go back and forth between the statement of the principle and the examples and discussion until you have a solid intuition for what is being said and what is important about the property. For more complex properties, we also provide a derivation, structured much like a mathematical proof. You should try to understand these derivations eventually, but you could skip over them on first reading.

We also encourage you to work on the practice problems as you proceed through the presentation. The practice problems engage you in active learning, helping you put thoughts into action. With these as background, you will find it much easier to go back and follow the derivations. Be assured, as well, that the mathematical skills required to understand this material are within reach of someone with a good grasp of high school algebra.

2.1 Information Storage

Rather than accessing individual bits in memory, most computers use blocks of 8 bits, or bytes, as the smallest addressable unit of memory. A machine-level program views memory as a very large array of bytes, referred to as virtual memory. Every byte of memory is identified by a unique number, known as its address, and the set of all possible addresses is known as the virtual address space. As indicated by its name, this virtual address space is just a conceptual image presented to the machine-level program. The actual implementation (presented in Chapter 9) uses a combination of dynamic random access memory (DRAM), flash memory, disk storage, special hardware, and operating system software to provide the program with what appears to be a monolithic byte array.

In subsequent chapters, we will cover how the compiler and run-time system partitions this memory space into more manageable units to store the different program objects, that is, program data, instructions, and control information. Various mechanisms are used to allocate and manage the storage for different parts of the program. This management is all performed within the virtual address space. For example, the value of a pointer in C—whether it points to an integer, a structure, or some other program object—is the virtual address of the first byte of some block of storage. The C compiler also associates type information with each pointer, so that it can generate different machine-level code to access the value stored at the location designated by the pointer depending on the type of that value. Although the C compiler maintains this type information, the actual machine-level program it generates has no information about data types. It simply treats each program object as a block of bytes and the program itself as a sequence of bytes.

New to C? Tutorial on Pointers

Pointers are a central feature of C. They provide the mechanism for referencing elements of data structures, including arrays. Just like a variable, a pointer has two aspects: its value and its type. The

The hex values B, D, and E can be translated to decimal by computing their values relative to the first three.

For example, suppose you are given the number 0x173A4C. You can convert this to binary format by expanding each hexadecimal digit, as follows:

Hexadecimal	1	7	3	A	4	C
Binary	0001	0111	0011	1010	0100	1100

This gives the binary representation 000101110011101001001100.

Conversely, given a binary number 111100101010110110110011, you convert it to hexadecimal by first splitting it into groups of 4 bits each. Note, however, that if the total number of bits is not a multiple of 4, you should make the leftmost group be the one with fewer than 4 bits, effectively padding the number with leading zeros. Then you translate each group of bits into the corresponding hexadecimal digit:

Binary	11	1100	1010	1101	1011	0011
Hexadecimal	3	C	A	D	B	3

Practice Problem 2.1 (solution page 179)

Perform the following number conversions:

A. 0x25B9D2 to binary

B. binary 1010111001001001 to hexadecimal

C. 0xA8B3D to binary

D. binary 11001000101101100101 10 to hexadecimal

When a value x is a power of 2, that is, $x = 2^n$ for some nonnegative integer n, we can readily write x in hexadecimal form by remembering that the binary representation of x is simply 1 followed by n zeros. The hexadecimal digit 0 represents 4 binary zeros. So, for n written in the form $i + 4j$, where $0 \le i \le 3$, we can write x with a leading hex digit of 1 ($i = 0$), 2 ($i = 1$), 4 ($i = 2$), or 8 ($i = 3$), followed by j hexadecimal 0s. As an example, for $x = 2{,}048 = 2^{11}$, we have $n = 11 = 3 + 4 \cdot 2$, giving hexadecimal representation 0x800.

Practice Problem 2.2 (solution page 179)

Fill in the blank entries in the following table, giving the decimal and hexadecimal representations of different powers of 2:

n	2^n (decimal)	2^n (hexadecimal)
5	32	0x20
23		
	32,768	
		0x2000
12		
	64	
		0x100

Converting between decimal and hexadecimal representations requires using multiplication or division to handle the general case. To convert a decimal number x to hexadecimal, we can repeatedly divide x by 16, giving a quotient q and a remainder r, such that $x = q \cdot 16 + r$. We then use the hexadecimal digit representing r as the least significant digit and generate the remaining digits by repeating the process on q. As an example, consider the conversion of decimal 314,156:

$$\begin{aligned} 314{,}156 &= 19{,}634 \cdot 16 + 12 && \text{(C)} \\ 19{,}634 &= 1{,}227 \cdot 16 + 2 && \text{(2)} \\ 1{,}227 &= 76 \cdot 16 + 11 && \text{(B)} \\ 76 &= 4 \cdot 16 + 12 && \text{(C)} \\ 4 &= 0 \cdot 16 + 4 && \text{(4)} \end{aligned}$$

From this we can read off the hexadecimal representation as 0x4CB2C.

Conversely, to convert a hexadecimal number to decimal, we can multiply each of the hexadecimal digits by the appropriate power of 16. For example, given the number 0x7AF, we compute its decimal equivalent as $7 \cdot 16^2 + 10 \cdot 16 + 15 = 7 \cdot 256 + 10 \cdot 16 + 15 = 1{,}792 + 160 + 15 = 1{,}967$.

Practice Problem 2.3 (solution page 180)

A single byte can be represented by 2 hexadecimal digits. Fill in the missing entries in the following table, giving the decimal, binary, and hexadecimal values of different byte patterns:

Decimal	Binary	Hexadecimal
0	0000 0000	0x00
158		
76		
145		
	1010 1110	
	0011 1100	
	1111 0001	

Aside Computing with decimal and hexadecimal

For converting larger values between decimal and hexadecimal, it is best to let a computer or calculator do the work. There are numerous tools that can do this. One simple way is to use any of the standard search engines, with queries such as

Convert 0xabcd to decimal

or

123 in hex

Decimal	Binary	Hexadecimal
_____	_____	0x75
_____	_____	0xBD
_____	_____	0xF5

Practice Problem 2.4 (solution page 180)

Without converting the numbers to decimal or binary, try to solve the following arithmetic problems, giving the answers in hexadecimal. Hint: Just modify the methods you use for performing decimal addition and subtraction to use base 16.

A. `0x605c + 0x5 =` _____

B. `0x605c − 0x20 =` _____

C. `0x605c + 32 =` _____

D. `0x60fa − 0x605c =` _____

2.1.2 Data Sizes

Every computer has a word size, indicating the nominal size of pointer data. Since a virtual address is encoded by such a word, the most important system parameter determined by the word size is the maximum size of the virtual address space. That is, for a machine with a w-bit word size, the virtual addresses can range from 0 to $2^w - 1$, giving the program access to at most 2^w bytes.

In recent years, there has been a widespread shift from machines with 32-bit word sizes to those with word sizes of 64 bits. This occurred first for high-end machines designed for large-scale scientific and database applications, followed by desktop and laptop machines, and most recently for the processors found in smartphones. A 32-bit word size limits the virtual address space to 4 gigabytes (written 4 GB), that is, just over 4×10^9 bytes. Scaling up to a 64-bit word size leads to a virtual address space of 16 exabytes, or around 1.84×10^{19} bytes.

Most 64-bit machines can also run programs compiled for use on 32-bit machines, a form of backward compatibility. So, for example, when a program `prog.c` is compiled with the directive

```
linux> gcc -m32 prog.c
```

then this program will run correctly on either a 32-bit or a 64-bit machine. On the other hand, a program compiled with the directive

```
linux> gcc -m64 prog.c
```

will only run on a 64-bit machine. We will therefore refer to programs as being either "32-bit programs" or "64-bit programs," since the distinction lies in how a program is compiled, rather than the type of machine on which it runs.

Computers and compilers support multiple data formats using different ways to encode data, such as integers and floating point, as well as different lengths. For example, many machines have instructions for manipulating single bytes, as well as integers represented as 2-, 4-, and 8-byte quantities. They also support floating-point numbers represented as 4- and 8-byte quantities.

The C language supports multiple data formats for both integer and floating-point data. Figure 2.3 shows the number of bytes typically allocated for different C data types. (We discuss the relation between what is guaranteed by the C standard versus what is typical in Section 2.2.) The exact numbers of bytes for some data types depends on how the program is compiled. We show sizes for typical 32-bit and 64-bit programs. Integer data can be either signed, able to represent negative, zero, and positive values, or unsigned, only allowing nonnegative values. Data type `char` represents a single byte. Although the name `char` derives from the fact that it is used to store a single character in a text string, it can also be used to store integer values. Data types `short`, `int`, and `long` are intended to provide a range of

C declaration		Bytes	
Signed	Unsigned	32-bit	64-bit
[signed] char	unsigned char	1	1
short	unsigned short	2	2
int	unsigned	4	4
long	unsigned long	4	8
int32_t	uint32_t	4	4
int64_t	uint64_t	8	8
char *		4	8
float		4	4
double		8	8

Figure 2.3 Typical sizes (in bytes) of basic C data types. T r o t all o at ari it o t r o ra i o i . T i art o t al t ial o 32- it a 64- it r o ra .

New to C? Declarations

For any data type `T`, the declaration

```
T *p;
```

indicates that `p` is a pointer variable, pointing to an object of type `T`. For example,

```
char *p;
```

is the declaration of a pointer to an object of type `char`.

sizes. Even when compiled for 64-bit systems, data type `int` is usually just 4 bytes. Data type `long` commonly has 4 bytes in 32-bit programs and 8 bytes in 64-bit programs.

To avoid the vagaries of relying on "typical" sizes and different compiler settings, ISO C99 introduced a class of data types where the data sizes are fixed regardless of compiler and machine settings. Among these are data types `int32_t` and `int64_t`, having exactly 4 and 8 bytes, respectively. Using fixed-size integer types is the best way for programmers to have close control over data representations.

Most of the data types encode signed values, unless prefixed by the keyword `unsigned` or using the specific unsigned declaration for fixed-size data types. The exception to this is data type `char`. Although most compilers and machines treat these as signed data, the C standard does not guarantee this. Instead, as indicated by the square brackets, the programmer should use the declaration `signed char` to guarantee a 1-byte signed value. In many contexts, however, the program's behavior is insensitive to whether data type `char` is signed or unsigned.

The C language allows a variety of ways to order the keywords and to include or omit optional keywords. As examples, all of the following declarations have identical meaning:

```
unsigned long
unsigned long int
long unsigned
long unsigned int
```

We will consistently use the forms found in Figure 2.3.

Figure 2.3 also shows that a pointer (e.g., a variable declared as being of type `char *`) uses the full word size of the program. Most machines also support two different floating-point formats: single precision, declared in C as `float`, and double precision, declared in C as `double`. These formats use 4 and 8 bytes, respectively.

Programmers should strive to make their programs portable across different machines and compilers. One aspect of portability is to make the program insensitive to the exact sizes of the different data types. The C standards set lower bounds

on the numeric ranges of the different data types, as will be covered later, but there are no upper bounds (except with the fixed-size types). With 32-bit machines and 32-bit programs being the dominant combination from around 1980 until around 2010, many programs have been written assuming the allocations listed for 32-bit programs in Figure 2.3. With the transition to 64-bit machines, many hidden word size dependencies have arisen as bugs in migrating these programs to new machines. For example, many programmers historically assumed that an object declared as type `int` could be used to store a pointer. This works fine for most 32-bit programs, but it leads to problems for 64-bit programs.

2.1.3 Addressing and Byte Ordering

For program objects that span multiple bytes, we must establish two conventions: what the address of the object will be, and how we will order the bytes in memory. In virtually all machines, a multi-byte object is stored as a contiguous sequence of bytes, with the address of the object given by the smallest address of the bytes used. For example, suppose a variable `x` of type `int` has address `0x100`; that is, the value of the address expression `&x` is `0x100`. Then (assuming data type `int` has a 32-bit representation) the 4 bytes of `x` would be stored in memory locations `0x100`, `0x101`, `0x102`, and `0x103`.

For ordering the bytes representing an object, there are two common conventions. Consider a `w`-bit integer having a bit representation $[x_{w-1}, x_{w-2}, \ldots, x_1, x_0]$, where x_{w-1} is the most significant bit and x_0 is the least. Assuming `w` is a multiple of 8, these bits can be grouped as bytes, with the most significant byte having bits $[x_{w-1}, x_{w-2}, \ldots, x_{w-8}]$, the least significant byte having bits $[x_7, x_6, \ldots, x_0]$, and the other bytes having bits from the middle. Some machines choose to store the object in memory ordered from least significant byte to most, while other machines store them from most to least. The former convention—where the least significant byte comes first—is referred to as little endian. The latter convention—where the most significant byte comes first—is referred to as big endian.

Suppose the variable `x` of type `int` and at address `0x100` has a hexadecimal value of `0x01234567`. The ordering of the bytes within the address range `0x100` through `0x103` depends on the type of machine:

Big endian

	0x100	0x101	0x102	0x103	
...	01	23	45	67	...

Little endian

	0x100	0x101	0x102	0x103	
...	67	45	23	01	...

Note that in the word `0x01234567` the high-order byte has hexadecimal value `0x01`, while the low-order byte has value `0x67`.

Most Intel-compatible machines operate exclusively in little-endian mode. On the other hand, most machines from IBM and Oracle (arising from their acquisi-

Aside Ori i , a

Here is how Jonathan Swift, writing in 1726, described the history of the controversy between big and little endians:

> . . . Lilliput and Blefuscu . . . have, as I was going to tell you, been engaged in a most obstinate war for six-and-thirty moons past. It began upon the following occasion. It is allowed on all hands, that the primitive way of breaking eggs, before we eat them, was upon the larger end; but his present majesty's grandfather, while he was a boy, going to eat an egg, and breaking it according to the ancient practice, happened to cut one of his fingers. Whereupon the emperor his father published an edict, commanding all his subjects, upon great penalties, to break the smaller end of their eggs. The people so highly resented this law, that our histories tell us, there have been six rebellions raised on that account; wherein one emperor lost his life, and another his crown. These civil commotions were constantly fomented by the monarchs of Blefuscu; and when they were quelled, the exiles always fled for refuge to that empire. It is computed that eleven thousand persons have at several times suffered death, rather than submit to break their eggs at the smaller end. Many hundred large volumes have been published upon this controversy: but the books of the Big-endians have been long forbidden, and the whole party rendered incapable by law of holding employments. (Jonathan Swift. Gulliver's Travels, Benjamin Motte, 1726.)

In his day, Swift was satirizing the continued conflicts between England (Lilliput) and France (Blefuscu). Danny Cohen, an early pioneer in networking protocols, first applied these terms to refer to byte ordering [24], and the terminology has been widely adopted.

tion of Sun Microsystems in 2010) operate in big-endian mode. Note that we said "most." The conventions do not split precisely along corporate boundaries. For example, both IBM and Oracle manufacture machines that use Intel-compatible processors and hence are little endian. Many recent microprocessor chips are bi-endian, meaning that they can be configured to operate as either little- or big-endian machines. In practice, however, byte ordering becomes fixed once a particular operating system is chosen. For example, ARM microprocessors, used

binary data are communicated over a network between different machines. A common problem is for data produced by a little-endian machine to be sent to a big-endian machine, or vice versa, leading to the bytes within the words being in reverse order for the receiving program. To avoid such problems, code written for networking applications must follow established conventions for byte ordering to make sure the sending machine converts its internal representation to the network standard, while the receiving machine converts the network standard to its internal representation. We will see examples of these conversions in Chapter 11.

A second case where byte ordering becomes important is when looking at the byte sequences representing integer data. This occurs often when inspecting machine-level programs. As an example, the following line occurs in a file that gives a text representation of the machine-level code for an Intel x86-64 processor:

```
4004d3:  01 05 43 0b 20 00       add    %eax,0x200b43(%rip)
```

This line was generated by a disassembler, a tool that determines the instruction sequence represented by an executable program file. We will learn more about disassemblers and how to interpret lines such as this in Chapter 3. For now, we simply note that this line states that the hexadecimal byte sequence `01 05 43 0b 20 00` is the byte-level representation of an instruction that adds a word of data to the value stored at an address computed by adding `0x200b43` to the current value of the program counter, the address of the next instruction to be executed. If we take the final 4 bytes of the sequence `43 0b 20 00` and write them in reverse order, we have `00 20 0b 43`. Dropping the leading 0, we have the value `0x200b43`, the numeric value written on the right. Having bytes appear in reverse order is a common occurrence when reading machine-level program representations generated for little-endian machines such as this one. The natural way to write a byte sequence is to have the lowest-numbered byte on the left and the highest on the right, but this is contrary to the normal way of writing numbers with the most significant digit on the left and the least on the right.

```
1   #include <stdio.h>
2
3   typedef unsigned char *byte_pointer;
4
5   void show_bytes(byte_pointer start, size_t len) {
6       int i;
7       for (i = 0; i < len; i++)
8           printf(" %.2x", start[i]);
9       printf("\n");
10  }
11
12  void show_int(int x) {
13      show_bytes((byte_pointer) &x, sizeof(int));
14  }
15
16  void show_float(float x) {
17      show_bytes((byte_pointer) &x, sizeof(float));
18  }
19
20  void show_pointer(void *x) {
21      show_bytes((byte_pointer) &x, sizeof(void *));
22  }
```

Figure 2.4 Code to print the byte representation of program objects. T i o
a ti t o ir tt t t . Si ilar ti o ar a l fi r o o t r
ata t .

Procedures `show_int`, `show_float`, and `show_pointer` demonstrate how to use procedure `show_bytes` to print the byte representations of C program objects of type `int`, `float`, and `void *`, respectively. Observe that they simply pass `show_bytes` a pointer `&x` to their argument `x`, casting the pointer to be of type `unsigned char *`. This cast indicates to the compiler that the program should consider the pointer to be to a sequence of bytes rather than to an object of the original data type. This pointer will then be to the lowest byte address occupied by the object.

These procedures use the C `sizeof` operator to determine the number of bytes

code/data/show-bytes.c

```
1  void test_show_bytes(int val) {
2      int ival = val;
3      float fval = (float) ival;
4      int *pval = &ival;
5      show_int(ival);
6      show_float(fval);
7      show_pointer(pval);
8  }
```

code/data/show-bytes.c

Figure 2.5 Byte representation examples. T i o' ri t t t r r tati o o a l ' ata o t .

Machine	Value	Type	Bytes (hex)
Linux 32	12,345	int	39 30 00 00
Windows	12,345	int	39 30 00 00
Sun	12,345	int	00 00 30 39
Linux 64	12,345	int	39 30 00 00
Linux 32	12,345.0	float	00 e4 40 46
Windows	12,345.0	float	00 e4 40 46
Sun	12,345.0	float	46 40 e4 00
Linux 64	12,345.0	float	00 e4 40 46
Linux 32	&ival	int *	e4 f9 ff bf
Windows	&ival	int *	b4 cc 22 00
Sun	&ival	int *	ef ff fa 0c
Linux 64	&ival	int *	b8 11 e5 ff ff 7f 00 00

Figure 2.6 Byte representations of different data values. R lt o r int a ' float ar ' ti al, t o r t o r' ri . P o i t r al ar a i ' ' t.

Our argument 12,345 has hexadecimal representation `0x00003039`. For the `int` data, we get identical results for all machines, except for the byte ordering. In particular, we can see that the least significant byte value of `0x39` is printed first for Linux 32, Windows, and Linux 64, indicating little-endian machines, and last for Sun, indicating a big-endian machine. Similarly, the bytes of the `float` data are identical, except for the byte ordering. On the other hand, the pointer values are completely different. The different machine/operating system configurations use different conventions for storage allocation. One feature to note is that the Linux 32, Windows, and Sun machines use 4-byte addresses, while the Linux 64 machine uses 8-byte addresses.

New to C? Naming data types with typedef

The `typedef` declaration in C provides a way of giving a name to a data type. This can be a great help in improving code readability, since deeply nested type declarations can be difficult to decipher.

The syntax for `typedef` is exactly like that of declaring a variable, except that it uses a type name rather than a variable name. Thus, the declaration of `byte_pointer` in Figure 2.4 has the same form as the declaration of a variable of type `unsigned char *`.

For example, the declaration

```
typedef int *int_pointer;
int_pointer ip;
```

defines type `int_pointer` to be a pointer to an `int`, and declares a variable `ip` of this type. Alternatively, we could declare this variable directly as

```
int *ip;
```

New to C? For more information about printf

The `printf` function (along with its cousins `fprintf` and `sprintf`) provides a way to print information with considerable control over the formatting details. The first argument is a format string, while any remaining arguments are values to be printed. Within the format string, each character sequence starting with '%' indicates how to format the next argument. Typical examples include `%d` to print a decimal integer, `%f` to print a floating-point number, and `%c` to print a character having the character code given by the argument.

Specifying the formatting of fixed-size data types, such as `int_32t`, is a bit more involved, as is described in the aside on page 103.

Observe that although the floating-point and the integer data both encode the numeric value 12,345, they have very different byte patterns: `0x00003039` for the integer and `0x4640E400` for floating point. In general, these two formats use different encoding schemes. If we expand these hexadecimal patterns into binary form and shift them appropriately, we find a sequence of 13 matching bits, indicated by a sequence of asterisks, as follows:

```
    0   0   0   0   3   0   3   9
00000000000000000011000000111001
                ************
            4   6   4   0   E   4   0   0
01000110010000001110010000000000
```

This is not coincidental. We will return to this example when we study floating-point formats.

New to C? Pointers and arrays

In function `show_bytes` (Figure 2.4), we see the close connection between pointers and arrays, as will be discussed in detail in Section 3.8. We see that this function has an argument `start` of type `byte_pointer` (which has been defined to be a pointer to `unsigned char`), but we see the array reference `start[i]` on line 8. In C, we can dereference a pointer with array notation, and we can reference array elements with pointer notation. In this example, the reference `start[i]` indicates that we want to read the byte that is `i` positions beyond the location pointed to by `start`.

New to C? Pointer creation and dereferencing

In lines 13, 17, and 21 of Figure 2.4 we see uses of two operations that give C (and therefore C++) its distinctive character. The C "address of" operator '`&`' creates a pointer. On all three lines, the expression `&x` creates a pointer to the location holding the object indicated by variable `x`. The type of this pointer depends on the type of `x`, and hence these three pointers are of type `int *`, `float *`, and `void **`, respectively. (Data type `void *` is a special kind of pointer with no associated type information.)

The cast operator converts from one data type to another. Thus, the cast `(byte_pointer) &x` indicates that whatever type the pointer `&x` had before, the program will now reference a pointer to data of type `unsigned char`. The casts shown here do not change the actual pointer; they simply direct the compiler to refer to the data being pointed to according to the new data type.

Aside Generating ASCII tables

You can display a table showing the ASCII character code by executing the command `man ascii`.

Practice Problem 2.5 (solution page 180)

Consider the following three calls to `show_bytes`:

```
int a = 0x12345678;
byte_pointer ap = (byte_pointer) &a;
show_bytes(ap, 1); /* A. */
show_bytes(ap, 2); /* B. */
show_bytes(ap, 3); /* C. */
```

Indicate the values that will be printed by each call on a little-endian machine and on a big-endian machine:

A. Little endian: _____ Big endian: _____

B. Little endian: _____ Big endian: _____

C. Little endian: _____ Big endian: _____

Practice Problem 2.6 (solution page 181)

Using `show_int` and `show_float`, we determine that the integer 2607352 has hexadecimal representation `0x0027C8F8`, while the floating-point number 3510593.0 has hexadecimal representation `0x4A1F23E0`.

A. Write the binary representations of these two hexadecimal values.

B. Shift these two strings relative to one another to maximize the number of matching bits. How many bits match?

C. What parts of the strings do not match?

2.1.4 Representing Strings

A string in C is encoded by an array of characters terminated by the null (having value 0) character. Each character is represented by some standard encoding, with the most common being the ASCII character code. Thus, if we run our routine `show_bytes` with arguments `"12345"` and `6` (to include the terminating character), we get the result `31 32 33 34 35 00`. Observe that the ASCII code for decimal digit `x` happens to be `0x3x`, and that the terminating byte has the hex representation `0x00`. This same result would be obtained on any system using ASCII as its character code, independent of the byte ordering and word size conventions. As a consequence, text data are more platform independent than binary data.

Practice Problem 2.7 (solution page 181)

What would be printed as a result of the following call to `show_bytes`?

```
const char *m = "mnopqr";
show_bytes((byte_pointer) m, strlen(m));
```

Note that letters 'a' through 'z' have ASCII codes `0x61` through `0x7A`.

2.1.5 Representing Code

Consider the following C function:

```
1  int sum(int x, int y) {
2      return x + y;
3  }
```

When compiled on our sample machines, we generate machine code having the following byte representations:

Linux 32	`55 89 e5 8b 45 0c 03 45 08 c9 c3`
Windows	`55 89 e5 8b 45 0c 03 45 08 5d c3`
Sun	`81 c3 e0 08 90 02 00 09`
Linux 64	`55 48 89 e5 89 7d fc 89 75 f8 03 45 fc c9 c3`

Aside Text codes: ASCII and Unicode

The ASCII character set is suitable for encoding English-language documents, but it does not have much in the way of special characters, such as the French 'ç'. It is wholly unsuited for encoding documents in languages such as Greek, Russian, and Chinese. Over the years, a variety of methods have been developed to encode text for different languages. The Unicode Consortium has devised the most comprehensive and widely accepted standard for encoding text. The current Unicode standard (version 7.0) has a repertoire of over 100,000 characters supporting a wide range of languages, including the ancient languages of Egypt and Babylon. To their credit, the Unicode Technical Committee rejected a proposal to include a standard writing for Klingon, a fictional civilization from the television series Star Trek.

The base encoding, known as the "Universal Character Set" of Unicode, uses a 32-bit representation of characters. This would seem to require every string of text to consist of 4 bytes per character. However, alternative codings are possible where common characters require just 1 or 2 bytes, while less common ones require more. In particular, the UTF-8 representation encodes each character as a sequence of bytes, such that the standard ASCII characters use the same single-byte encodings as they have in ASCII, implying that all ASCII byte sequences have the same meaning in UTF-8 as they do in ASCII.

The Java programming language uses Unicode in its representations of strings. Program libraries are also available for C to support Unicode.

Here we find that the instruction codings are different. Different machine types use different and incompatible instructions and encodings. Even identical processors running different operating systems have differences in their coding conventions and hence are not binary compatible. Binary code is seldom portable across different combinations of machine and operating system.

A fundamental concept of computer systems is that a program, from the perspective of the machine, is simply a sequence of bytes. The machine has no information about the original source program, except perhaps some auxiliary tables maintained to aid in debugging. We will see this more clearly when we study machine-level programming in Chapter 3.

2.1.6 Introduction to Boolean Algebra

Since binary values are at the core of how computers encode, store, and manipulate information, a rich body of mathematical knowledge has evolved around the study of the values 0 and 1. This started with the work of George Boole (1815–1864) around 1850 and thus is known as Boolean algebra. Boole observed that by encoding logic values TRUE and FALSE as binary values 1 and 0, he could formulate an algebra that captures the basic principles of logical reasoning.

The simplest Boolean algebra is defined over the two-element set $\{0, 1\}$. Figure 2.7 defines several operations in this algebra. Our symbols for representing these operations are chosen to match those used by the C bit-level operations,

~	
0	1
1	0

&	0	1
0	0	0
1	0	1

|	0	1
0	0	1
1	1	1

^	0	1
0	0	1
1	1	0

Figure 2.7 Operations of Boolean algebra. Binary value 1 corresponds to true and 0 to false, while operations ~, &, |, and ^ correspond to not, and, or, and exclusive-or, respectively.

as will be discussed later. The Boolean operation ~ corresponds to the logical operation NOT, denoted by the symbol ¬. That is, we say that ¬P is true when P is not true, and vice versa. Correspondingly, ~p equals 1 when p equals 0, and vice versa. Boolean operation & corresponds to the logical operation AND, denoted by the symbol ∧. We say that P ∧ Q holds when both P is true and Q is true. Correspondingly, p & q equals 1 only when p = 1 and q = 1. Boolean operation | corresponds to the logical operation OR, denoted by the symbol ∨. We say that P ∨ Q holds when either P is true or Q is true. Correspondingly, p | q equals 1 when either p = 1 or q = 1. Boolean operation ^ corresponds to the logical operation EXCLUSIVE-OR, denoted by the symbol ⊕. We say that P ⊕ Q holds when either P is true or Q is true, but not both. Correspondingly, p ^ q equals 1 when either p = 1 and q = 0, or p = 0 and q = 1.

Claude Shannon (1916–2001), who later founded the field of information theory, first made the connection between Boolean algebra and digital logic. In his 1937 master's thesis, he showed that Boolean algebra could be applied to the design and analysis of networks of electromechanical relays. Although computer technology has advanced considerably since, Boolean algebra still plays a central role in the design and analysis of digital systems.

We can extend the four Boolean operations to also operate on bit vectors, strings of zeros and ones of some fixed length w. We define the operations over bit vectors according to their applications to the matching elements of the arguments. Let a and b denote the bit vectors $[a_{w-1}, a_{w-2}, \ldots, a_0]$ and $[b_{w-1}, b_{w-2}, \ldots, b_0]$, respectively. We define a & b to also be a bit vector of length w, where the ith element equals a_i & b_i, for $0 \le i < w$. The operations |, ^, and ~ are extended to bit vectors in a similar fashion.

As examples, consider the case where w = 4, and with arguments a = [0110] and b = [1100]. Then the four operations a & b, a | b, a ^ b, and ~b yield

	0110		0110		0110		
&	1100	|	1100	^	1100	~	1100
	0100		1110		1010		0011

Practice Problem 2.8 (solution page 181)

Fill in the following table showing the results of evaluating Boolean operations on bit vectors.

Web Aside DATA:BOOL More on Boolean Algebra and Boolean Rings

The Boolean operations |, &, and ~ operating on bit vectors of length w form a Boolean algebra, for any integer w > 0. The simplest is the case where w = 1 and there are just two elements, but for the more general case there are 2^w bit vectors of length w. Boolean algebra has many of the same properties as arithmetic over integers. For example, just as multiplication distributes over addition, written a · (b + c) = (a · b) + (a · c), Boolean operation & distributes over |, written a & (b | c) = (a & b) | (a & c). In addition, however. Boolean operation | distributes over &, and so we can write a | (b & c) = (a | b) & (a | c), whereas we cannot say that a + (b · c) = (a + b) · (a + c) holds for all integers.

When we consider operations ^, &, and ~ operating on bit vectors of length w, we get a different mathematical form, known as a Boolean ring. Boolean rings have many properties in common with integer arithmetic. For example, one property of integer arithmetic is that every value x has an additive inverse −x, such that x + −x = 0. A similar property holds for Boolean rings, where ^ is the "addition" operation, but in this case each element is its own additive inverse. That is, a ^ a = 0 for any value a, where we use 0 here to represent a bit vector of all zeros. We can see this holds for single bits, since 0 ^ 0 = 1 ^ 1 = 0, and it extends to bit vectors as well. This property holds even when we rearrange terms and combine them in a different order, and so (a ^ b) ^ a = b. This property leads to some interesting results and clever tricks, as we will explore in Problem 2.10.

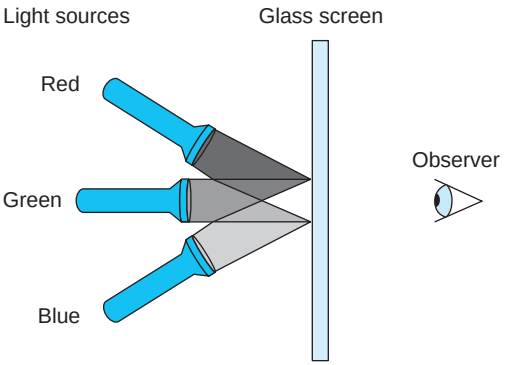
Operation	Result
a	[01001110]
b	[11100001]
~a	_____
~b	_____
a & b	_____
a | b	_____
a ^ b	_____

One useful application of bit vectors is to represent finite sets. We can encode any subset A $\subseteq$ {0, 1, . . . , w − 1} with a bit vector $[a_{w-1}, \ldots, a_1, a_0]$, where $a_i = 1$ if and only if $i \in A$. For example, recalling that we write a_{w-1} on the left and a_0 on the right, bit vector a = [01101001] encodes the set A = {0, 3, 5, 6}, while bit vector b = [01010101] encodes the set B = {0, 2, 4, 6}. With this way of encoding sets, Boolean operations | and & correspond to set union and intersection, respectively, and ~ corresponds to set complement. Continuing our earlier example, the operation a & b yields bit vector [01000001], while $A \cap B = \{0, 6\}$.

We will see the encoding of sets by bit vectors in a number of practical applications. For example, in Chapter 8, we will see that there are a number of different signals that can interrupt the execution of a program. We can selectively enable or disable different signals by specifying a bit-vector mask, where a 1 in bit position i indicates that signal i is enabled and a 0 indicates that it is disabled. Thus, the mask represents the set of enabled signals.

Practice Problem 2.9 (solution page 182)

Computers generate color pictures on a video screen or liquid crystal display by mixing three different colors of light: red, green, and blue. Imagine a simple scheme, with three different lights, each of which can be turned on or off, projecting onto a glass screen:



We can then create eight different colors based on the absence (0) or presence (1) of light sources R, G, and B:

R	G	B	Color
0	0	0	Black
0	0	1	Blue
0	1	0	Green
0	1	1	Cyan
1	0	0	Red
1	0	1	Magenta
1	1	0	Yellow
1	1	1	White

Each of these colors can be represented as a bit vector of length 3, and we can apply Boolean operations to them.

A. The complement of a color is formed by turning off the lights that are on and turning on the lights that are off. What would be the complement of each of the eight colors listed above?

B. Describe the effect of applying Boolean operations on the following colors:

Blue | Green = _____
Yellow & Cyan = _____
Red ^ Magenta = _____

2.1.7 Bit-Level Operations in C

One useful feature of C is that it supports bitwise Boolean operations. In fact, the symbols we have used for the Boolean operations are exactly those used by C: `|` for OR, `&` for AND, `~` for NOT, and `^` for EXCLUSIVE-OR. These can be applied to any "integral" data type, including all of those listed in Figure 2.3. Here are some examples of expression evaluation for data type `char`:

C expression	Binary expression	Binary result	Hexadecimal result
`~0x41`	~[0100 0001]	[1011 1110]	`0xBE`
`~0x00`	~[0000 0000]	[1111 1111]	`0xFF`
`0x69 & 0x55`	[0110 1001] & [0101 0101]	[0100 0001]	`0x41`
`0x69 | 0x55`	[0110 1001] | [0101 0101]	[0111 1101]	`0x7D`

As our examples show, the best way to determine the effect of a bit-level expression is to expand the hexadecimal arguments to their binary representations, perform the operations in binary, and then convert back to hexadecimal.

Practice Problem 2.10 (solution page 182)

As an application of the property that `a ^ a = 0` for any bit vector `a`, consider the following program:

```
1  void inplace_swap(int *x, int *y) {
2      *y = *x ^ *y;  /* Step 1 */
3      *x = *x ^ *y;  /* Step 2 */
4      *y = *x ^ *y;  /* Step 3 */
5  }
```

As the name implies, we claim that the effect of this procedure is to swap the values stored at the locations denoted by pointer variables `x` and `y`. Note that unlike the usual technique for swapping two values, we do not need a third location to temporarily store one value while we are moving the other. There is no performance advantage to this way of swapping; it is merely an intellectual amusement.

Starting with values `a` and `b` in the locations pointed to by `x` and `y`, respectively, fill in the table that follows, giving the values stored at the two locations after each step of the procedure. Use the properties of `^` to show that the desired effect is achieved. Recall that every element is its own additive inverse (that is, `a ^ a = 0`).

Step	`*x`	`*y`
Initially	`a`	`b`
Step 1	_____	_____
Step 2	_____	_____
Step 3	_____	_____

Practice Problem 2.11 (solution page 182)

Armed with the function `inplace_swap` from Problem 2.10, you decide to write code that will reverse the elements of an array by swapping elements from opposite ends of the array, working toward the middle.

You arrive at the following function:

```
1  void reverse_array(int a[], int cnt) {
2      int first, last;
3      for (first = 0, last = cnt-1;
4           first <= last;
5           first++,last--)
6           inplace_swap(&a[first], &a[last]);
7  }
```

When you apply your function to an array containing elements 1, 2, 3, and 4, you find the array now has, as expected, elements 4, 3, 2, and 1. When you try it on an array with elements 1, 2, 3, 4, and 5, however, you are surprised to see that the array now has elements 5, 4, 0, 2, and 1. In fact, you discover that the code always works correctly on arrays of even length, but it sets the middle element to 0 whenever the array has odd length.

A. For an array of odd length `cnt` = 2`k` + 1, what are the values of variables `first` and `last` in the final iteration of function `reverse_array`?

B. Why does this call to function `inplace_swap` set the array element to 0?

C. What simple modification to the code for `reverse_array` would eliminate this problem?

One common use of bit-level operations is to implement *masking* operations, where a mask is a bit pattern that indicates a selected set of bits within a word. As an example, the mask `0xFF` (having ones for the least significant 8 bits) indicates the low-order byte of a word. The bit-level operation `x & 0xFF` yields a value consisting of the least significant byte of `x`, but with all other bytes set to 0. For example, with `x = 0x89ABCDEF`, the expression would yield `0x000000EF`. The expression `~0` will yield a mask of all ones, regardless of the size of the data representation. The same mask can be written `0xFFFFFFFF` when data type `int` is 32 bits, but it would not be as portable.

Practice Problem 2.12 (solution page 182)

Write C expressions, in terms of variable `x`, for the following values. Your code should work for any word size `w` ≥ 8. For reference, we show the result of evaluating the expressions for `x = 0x87654321`, with `w = 32`.

A. The least significant byte of `x`, with all other bits set to 0. [`0x00000021`]

B. All but the least significant byte of `x` complemented, with the least significant byte left unchanged. [`0x789ABC21`]

C. The least significant byte set to all ones, and all other bytes of `x` left unchanged. [`0x876543FF`]

Practice Problem 2.13 (solution page 183)

The Digital Equipment VAX computer was a very popular machine from the late 1970s until the late 1980s. Rather than instructions for Boolean operations AND and OR, it had instructions `bis` (bit set) and `bic` (bit clear). Both instructions take a data word `x` and a mask word `m`. They generate a result `z` consisting of the bits of `x` modified according to the bits of `m`. With `bis`, the modification involves setting `z` to 1 at each bit position where `m` is 1. With `bic`, the modification involves setting `z` to 0 at each bit position where `m` is 1.

To see how these operations relate to the C bit-level operations, assume we have functions `bis` and `bic` implementing the bit set and bit clear operations, and that we want to use these to implement functions computing bitwise operations `|` and `^`, without using any other C operations. Fill in the missing code below. Hint: Write C expressions for the operations `bis` and `bic`.

```
/* Declarations of functions implementing operations bis and bic */
int bis(int x, int m);
int bic(int x, int m);

/* Compute x|y using only calls to functions bis and bic */
int bool_or(int x, int y) {
    int result = _________;
    return result;
}

/* Compute x^y using only calls to functions bis and bic */
int bool_xor(int x, int y) {
    int result = _________;
    return result;
}
```

2.1.8 Logical Operations in C

C also provides a set of logical operators `||`, `&&`, and `!`, which correspond to the OR, AND, and NOT operations of logic. These can easily be confused with the bit-level operations, but their behavior is quite different. The logical operations treat any nonzero argument as representing TRUE and argument 0 as representing FALSE. They return either 1 or 0, indicating a result of either TRUE or FALSE, respectively. Here are some examples of expression evaluation:

Expression	Result
`!0x41`	`0x00`
`!0x00`	`0x01`
`!!0x41`	`0x01`
`0x69 && 0x55`	`0x01`
`0x69 || 0x55`	`0x01`

Observe that a bitwise operation will have behavior matching that of its logical counterpart only in the special case in which the arguments are restricted to 0 or 1.

A second important distinction between the logical operators '`&&`' and '`||`' versus their bit-level counterparts '`&`' and '`|`' is that the logical operators do not evaluate their second argument if the result of the expression can be determined by evaluating the first argument. Thus, for example, the expression `a && 5/a` will never cause a division by zero, and the expression `p && *p++` will never cause the dereferencing of a null pointer.

Practice Problem 2.14 (solution page 183)

Suppose that `a` and `b` have byte values `0x55` and `0x46`, respectively. Fill in the following table indicating the byte values of the different C expressions:

Expression	Value	Expression	Value
`a & b`	_____	`a && b`	_____
`a | b`	_____	`a || b`	_____
`~a | ~b`	_____	`!a || !b`	_____
`a & !b`	_____	`a && ~b`	_____

Practice Problem 2.15 (solution page 184)

Using only bit-level and logical operations, write a C expression that is equivalent to `x == y`. In other words, it will return 1 when `x` and `y` are equal and 0 otherwise.

2.1.9 Shift Operations in C

C also provides a set of shift operations for shifting bit patterns to the left and to the right. For an operand `x` having bit representation $[x_{w-1}, x_{w-2}, \ldots, x_0]$, the C expression `x << k` yields a value with bit representation $[x_{w-k-1}, x_{w-k-2}, \ldots, x_0, 0, \ldots, 0]$. That is, `x` is shifted `k` bits to the left, dropping off the `k` most significant bits and filling the right end with `k` zeros. The shift amount should be a value between 0 and `w − 1`. Shift operations associate from left to right, so `x << j << k` is equivalent to `(x << j) << k`.

There is a corresponding right shift operation, written in C as `x >> k`, but it has a slightly subtle behavior. Generally, machines support two forms of right shift:

Logical. A logical right shift fills the left end with k zeros, giving a result $[0, \ldots, 0, x_{w-1}, x_{w-2}, \ldots x_k]$.

Arithmetic. An arithmetic right shift fills the left end with k repetitions of the most significant bit, giving a result $[x_{w-1}, \ldots, x_{w-1}, x_{w-1}, x_{w-2}, \ldots x_k]$. This convention might seem peculiar, but as we will see, it is useful for operating on signed integer data.

As examples, the following table shows the effect of applying the different shift operations to two different values of an 8-bit argument x:

Operation	Value 1	Value 2
Argument x	[01100011]	[10010101]
x << 4	[00110000]	[01010000]
x >> 4 (logical)	[00000110]	[00001001]
x >> 4 (arithmetic)	[00000110]	[11111001]

The italicized digits indicate the values that fill the right (left shift) or left (right shift) ends. Observe that all but one entry involves filling with zeros. The exception is the case of shifting [10010101] right arithmetically. Since its most significant bit is 1, this will be used as the fill value.

The C standards do not precisely define which type of right shift should be used with signed numbers—either arithmetic or logical shifts may be used. This unfortunately means that any code assuming one form or the other will potentially encounter portability problems. In practice, however, almost all compiler/machine combinations use arithmetic right shifts for signed data, and many programmers assume this to be the case. For unsigned data, on the other hand, right shifts must be logical.

In contrast to C, Java has a precise definition of how right shifts should be performed. The expression x >> k shifts x arithmetically by k positions, while x >>> k shifts it logically.

Practice Problem 2.16 (solution page 184)

Fill in the table below showing the effects of the different shift operations on single-byte quantities. The best way to think about shift operations is to work with binary representations. Convert the initial values to binary, perform the shifts, and then convert back to hexadecimal. Each of the answers should be 8 binary digits or 2 hexadecimal digits.

a		a << 2		Logical a >> 3		Arithmetic a >> 3	
Hex	Binary	Binary	Hex	Binary	Hex	Binary	Hex
0xD4							
0x64							
0x72							
0x44							

Aside Shifting by k, for large values of k

For a data type consisting of w bits, what should be the effect of shifting by some value $k \geq w$? For example, what should be the effect of computing the following expressions, assuming data type `int` has $w = 32$:

```
int      lval = 0xFEDCBA98  << 32;
int      aval = 0xFEDCBA98  >> 36;
unsigned uval = 0xFEDCBA98u >> 40;
```

The C standards carefully avoid stating what should be done in such a case. On many machines, the shift instructions consider only the lower $\log_2 w$ bits of the shift amount when shifting a w-bit value, and so the shift amount is computed as k mod w. For example, with $w = 32$, the above three shifts would be computed as if they were by amounts 0, 4, and 8, respectively, giving results

```
lval    0xFEDCBA98
aval    0xFFEDCBA9
uval    0x00FEDCBA
```

This behavior is not guaranteed for C programs, however, and so shift amounts should be kept less than the word size.

Java, on the other hand, specifically requires that shift amounts should be computed in the modular fashion we have shown.

Aside Operator precedence and bit-level operations

It might be tempting to write the expression `1<<2 + 3<<4`, intending it to mean `(1<<2) + (3<<4)`. However, in C the former expression is equivalent to `1 << (2+3) << 4`, since addition (and subtraction) have higher precedence than shifts. The left-to-right associativity rule then causes this to be parenthesized as `(1 << (2+3)) << 4`, giving value 512, rather than the intended 52.

Getting the precedence wrong in C expressions is a common source of program errors, and often these are difficult to spot by inspection. When in doubt, put in parentheses!

2.2 Integer Representations

In this section, we describe two different ways bits can be used to encode integers—one that can only represent nonnegative numbers, and one that can represent negative, zero, and positive numbers. We will see later that they are strongly related both in their mathematical properties and their machine-level implementations. We also investigate the effect of expanding or shrinking an encoded integer to fit a representation with a different length.

Figure 2.8 lists the mathematical terminology we introduce to precisely define and characterize how computers encode and operate on integer data. This

Symbol	Type	Meaning	Page
$B2T_w$	Function	Binary to two's complement	100
$B2U_w$	Function	Binary to unsigned	98
$U2B_w$	Function	Unsigned to binary	100
$U2T_w$	Function	Unsigned to two's complement	107
$T2B_w$	Function	Two's complement to binary	101
$T2U_w$	Function	Two's complement to unsigned	107
$TMin_w$	Constant	Minimum two's-complement value	101
$TMax_w$	Constant	Maximum two's-complement value	101
$UMax_w$	Constant	Maximum unsigned value	99
$+^t_w$	Operation	Two's-complement addition	126
$+^u_w$	Operation	Unsigned addition	121
$*^t_w$	Operation	Two's-complement multiplication	133
$*^u_w$	Operation	Unsigned multiplication	132
$-^t_w$	Operation	Two's-complement negation	131
$-^u_w$	Operation	Unsigned negation	125

Figure 2.8 Terminology for integer data and arithmetic operations. T ri t w t t r it i t ata r r tati . T , Pa l i i at t a i t t r i fi .

terminology will be introduced over the course of the presentation. The figure is included here as a reference.

2.2.1 Integral Data Types

C supports a variety of integral data types—ones that represent finite ranges of integers. These are shown in Figures 2.9 and 2.10, along with the ranges of values they can have for "typical" 32- and 64-bit programs. Each type can specify a size with keyword `char`, `short`, `long`, as well as an indication of whether the represented numbers are all nonnegative (declared as `unsigned`), or possibly negative (the default.) As we saw in Figure 2.3, the number of bytes allocated for the different sizes varies according to whether the program is compiled for 32 or 64 bits. Based on the byte allocations, the different sizes allow different ranges of values to be represented. The only machine-dependent range indicated is for size designator `long`. Most 64-bit programs use an 8-byte representation, giving a much wider range of values than the 4-byte representation used with 32-bit programs.

One important feature to note in Figures 2.9 and 2.10 is that the ranges are not symmetric—the range of negative numbers extends one further than the range of positive numbers. We will see why this happens when we consider how negative numbers are represented.

C data type	Minimum	Maximum
[signed] char	−128	127
unsigned char	0	255
short	−32,768	32,767
unsigned short	0	65,535
int	−2,147,483,648	2,147,483,647
unsigned	0	4,294,967,295
long	−2,147,483,648	2,147,483,647
unsigned long	0	4,294,967,295
int32_t	−2,147,483,648	2,147,483,647
uint32_t	0	4,294,967,295
int64_t	−9,223,372,036,854,775,808	9,223,372,036,854,775,807
uint64_t	0	18,446,744,073,709,551,615

Figure 2.9 Typical ranges for C integral data types for 32-bit programs.

C data type	Minimum	Maximum
[signed] char	−128	127
unsigned char	0	255
short	−32,768	32,767
unsigned short	0	65,535
int	−2,147,483,648	2,147,483,647
unsigned	0	4,294,967,295
long	−9,223,372,036,854,775,808	9,223,372,036,854,775,807
unsigned long	0	18,446,744,073,709,551,615
int32_t	−2,147,483,648	2,147,483,647
uint32_t	0	4,294,967,295
int64_t	−9,223,372,036,854,775,808	9,223,372,036,854,775,807
uint64_t	0	18,446,744,073,709,551,615

Figure 2.10 Typical ranges for C integral data types for 64-bit programs.

The C standards define minimum ranges of values that each data type must be able to represent. As shown in Figure 2.11, their ranges are the same or smaller than the typical implementations shown in Figures 2.9 and 2.10. In particular, with the exception of the fixed-size data types, we see that they require only a

New to C? Signed and unsigned numbers in C, C++, and Java

Both C and C++ support signed (the default) and unsigned numbers. Java supports only signed numbers.

C data type	Minimum	Maximum
`[signed] char`	−127	127
`unsigned char`	0	255
`short`	−32,767	32,767
`unsigned short`	0	65,535
`int`	−32,767	32,767
`unsigned`	0	65,535
`long`	−2,147,483,647	2,147,483,647
`unsigned long`	0	4,294,967,295
`int32_t`	−2,147,483,648	2,147,483,647
`uint32_t`	0	4,294,967,295
`int64_t`	−9,223,372,036,854,775,808	9,223,372,036,854,775,807
`uint64_t`	0	18,446,744,073,709,551,615

Figure 2.11 Guaranteed ranges for C integral data types. These data ranges are typical for 32-bit and 64-bit programs.

symmetric range of positive and negative numbers. We also see that data type `int` could be implemented with 2-byte numbers, although this is mostly a throwback to the days of 16-bit machines. We also see that size `long` can be implemented with 4-byte numbers, and it typically is for 32-bit programs. The fixed-size data types guarantee that the ranges of values will be exactly those given by the typical numbers of Figure 2.9, including the asymmetry between negative and positive.

2.2.2 Unsigned Encodings

Let us consider an integer data type of w bits. We write a bit vector as either $\mathbf{x}$, to denote the entire vector, or as $[x_{w-1}, x_{w-2}, \ldots, x_0]$ to denote the individual bits within the vector. Treating $\mathbf{x}$ as a number written in binary notation, we obtain the unsigned interpretation of $\mathbf{x}$. In this encoding, each bit x_i has value 0 or 1, with the latter case indicating that value 2^i should be included as part of the numeric value. We can express this interpretation as a function $B2U_w$ (for "binary to unsigned," length w):

there is only one representation of decimal value 11 as an unsigned 4-bit number—namely, [1011]. We highlight this as a mathematical principle, which we first state and then explain.

principle: Uniqueness of unsigned encoding

Function $B2U_w$ is a bijection. ■

The mathematical term bijection refers to a function f that goes two ways: it maps a value x to a value y where $y = f(x)$, but it can also operate in reverse, since for every y, there is a unique value x such that $f(x) = y$. This is given by the inverse function f^{-1}, where, for our example, $x = f^{-1}(y)$. The function $B2U_w$ maps each bit vector of length w to a unique number between 0 and $2^w - 1$, and it has an inverse, which we call $U2B_w$ (for "unsigned to binary"), that maps each number in the range 0 to $2^w - 1$ to a unique pattern of w bits.

2.2.3 Two's-Complement Encodings

For many applications, we wish to represent negative values as well. The most common computer representation of signed numbers is known as two's-complement form. This is defined by interpreting the most significant bit of the word to have negative weight. We express this interpretation as a function $B2T_w$ (for "binary to two's complement" length w):

principle: Definition of two's-complement encoding

For vector $x = [x_{w-1}, x_{w-2}, \ldots, x_0]$:

$$B2T_w(x) \doteq -x_{w-1}2^{w-1} + \sum_{i=0}^{w-2} x_i 2^i \tag{2.3}$$

■

The most significant bit x_{w-1} is also called the sign bit. Its "weight" is -2^{w-1}, the negation of its weight in an unsigned representation. When the sign bit is set to 1, the represented value is negative, and when set to 0, the value is nonnegative. As examples, Figure 2.13 shows the mapping, given by $B2T$, from bit vectors to integers for the following cases:

$$\begin{aligned}
B2T_4([0001]) &= -0\cdot 2^3 + 0\cdot 2^2 + 0\cdot 2^1 + 1\cdot 2^0 = 0 + 0 + 0 + 1 = 1\\
B2T_4([0101]) &= -0\cdot 2^3 + 1\cdot 2^2 + 0\cdot 2^1 + 1\cdot 2^0 = 0 + 4 + 0 + 1 = 5\\
B2T_4([1011]) &= -1\cdot 2^3 + 0\cdot 2^2 + 1\cdot 2^1 + 1\cdot 2^0 = -8 + 0 + 2 + 1 = -5\\
B2T_4([1111]) &= -1\cdot 2^3 + 1\cdot 2^2 + 1\cdot 2^1 + 1\cdot 2^0 = -8 + 4 + 2 + 1 = -1
\end{aligned} \tag{2.4}$$

In the figure, we indicate that the sign bit has negative weight by showing it as a leftward-pointing gray bar. The numeric value associated with a bit vector is then given by the combination of the possible leftward-pointing gray bar and the rightward-pointing blue bars.

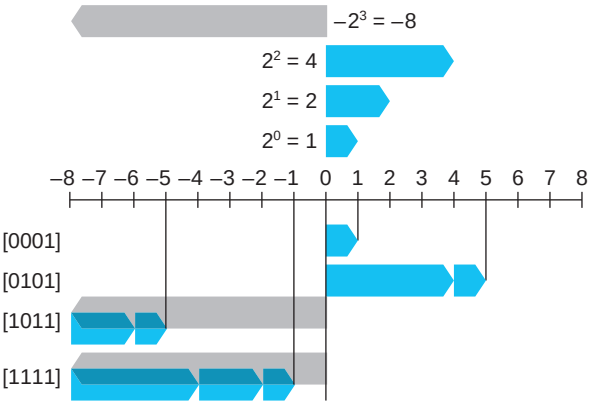


Figure 2.13 Two's-complement number examples for $w = 4$. Bit 3 has a weight of -8; the other bits have weight 1, 2, and 4, respectively. The bit vector [1011] has value $-2^3 + 2^1 + 2^0 = -8 + 2 + 1 = -5$. The bit vector [0101] has value $2^2 + 2^0 = 4 + 1 = 5$. Adding the two bit vectors together (ignoring any overflow) gives a result of zero.

We see that the bit patterns are identical for Figures 2.12 and 2.13 (as well as for Equations 2.2 and 2.4), but the values differ when the most significant bit is 1, since in one case it has weight $+8$, and in the other case it has weight -8.

Let us consider the range of values that can be represented as a w-bit two's-complement number. The least representable value is given by bit vector $[10\cdots0]$ (set the bit with negative weight but clear all others), having integer value $\mathrm{TMin}_w \doteq -2^{w-1}$. The greatest value is given by bit vector $[01\cdots1]$ (clear the bit with negative weight but set all others), having integer value $\mathrm{TMax}_w \doteq \sum_{i=0}^{w-2} 2^i = 2^{w-1} - 1$. Using the 4-bit case as an example, we have $\mathrm{TMin}_4 = \mathrm{B2T}_4([1000]) = -2^3 = -8$ and $\mathrm{TMax}_4 = \mathrm{B2T}_4([0111]) = 2^2 + 2^1 + 2^0 = 4 + 2 + 1 = 7$.

We can see that $\mathrm{B2T}_w$ is a mapping of bit patterns of length w to numbers between TMin_w and TMax_w, written as $\mathrm{B2T}_w: \{0, 1\}^w \rightarrow \{\mathrm{TMin}_w, \ldots, \mathrm{TMax}_w\}$. As we saw with the unsigned representation, every number within the representable range has a unique encoding as a w-bit two's-complement number. This leads to a principle for two's-complement numbers similar to that for unsigned numbers:

principle: Uniqueness of two's-complement encoding

Function $\mathrm{B2T}_w$ is a bijection. ■

We define function $\mathrm{T2B}_w$ (for "two's complement to binary") to be the inverse of $\mathrm{B2T}_w$. That is, for a number x, such that $\mathrm{TMin}_w \le x \le \mathrm{TMax}_w$, $\mathrm{T2B}_w(x)$ is the (unique) w-bit pattern that encodes x.

Practice Problem 2.17 (solution page 184)

Assuming $w = 4$, we can assign a numeric value to each possible hexadecimal digit, assuming either an unsigned or a two's-complement interpretation. Fill in the following table according to these interpretations by writing out the nonzero powers of 2 in the summations shown in Equations 2.1 and 2.3:

x			
Hexadecimal	Binary	$B2U_4(x)$	$B2T_4(x)$
0xA	[1010]	$2^3 + 2^1 = 10$	$-2^3 + 2^1 = -6$
0x1			
0xB			
0x2			
0x7			
0xC			

Figure 2.14 shows the bit patterns and numeric values for several important numbers for different word sizes. The first three give the ranges of representable integers in terms of the values of $UMax_w$, $TMin_w$, and $TMax_w$. We will refer to these three special values often in the ensuing discussion. We will drop the subscript w and refer to the values UMax, TMin, and TMax when w can be inferred from context or is not central to the discussion.

A few points are worth highlighting about these numbers. First, as observed in Figures 2.9 and 2.10, the two's-complement range is asymmetric: |TMin| = |TMax| + 1; that is, there is no positive counterpart to TMin. As we shall see, this leads to some peculiar properties of two's-complement arithmetic and can be the source of subtle program bugs. This asymmetry arises because half the bit patterns (those with the sign bit set to 1) represent negative numbers, while half (those with the sign bit set to 0) represent nonnegative numbers. Since 0 is nonnegative, this means that it can represent one less positive number than negative. Second, the maximum unsigned value is just over twice the maximum two's-complement value: UMax = 2TMax + 1. All of the bit patterns that denote negative numbers in two's-complement notation become positive values in an unsigned representation.

	Word size w			
Value	8	16	32	64
$UMax_w$	0xFF	0xFFFF	0xFFFFFFFF	0xFFFFFFFFFFFFFFFF
	255	65,535	4,294,967,295	18,446,744,073,709,551,615
$TMin_w$	0x80	0x8000	0x80000000	0x8000000000000000
	−128	−32,768	−2,147,483,648	−9,223,372,036,854,775,808
$TMax_w$	0x7F	0x7FFF	0x7FFFFFFF	0x7FFFFFFFFFFFFFFF
	127	32,767	2,147,483,647	9,223,372,036,854,775,807
−1	0xFF	0xFFFF	0xFFFFFFFF	0xFFFFFFFFFFFFFFFF
0	0x00	0x0000	0x00000000	0x0000000000000000

Figure 2.14 Important numbers. B0t ri al a'0 a0 i al r r ta- ti0 ar 0 .

Aside More on fixed-size integer types

For some programs, it is essential that data types be encoded using representations with specific sizes. For example, when writing programs to enable a machine to communicate over the Internet according to a standard protocol, it is important to have data types compatible with those specified by the protocol. We have seen that some C data types, especially `long`, have different ranges on different machines, and in fact the C standards only specify the minimum ranges for any data type, not the exact ranges. Although we can choose data types that will be compatible with standard representations on most machines, there is no guarantee of portability.

We have already encountered the 32- and 64-bit versions of fixed-size integer types (Figure 2.3); they are part of a larger class of data types. The ISO C99 standard introduces this class of integer types in the file `stdint.h`. This file defines a set of data types with declarations of the form `intN_t` and `uintN_t`, specifying `N`-bit signed and unsigned integers, for different values of `N`. The exact values of `N` are implementation dependent, but most compilers allow values of 8, 16, 32, and 64. Thus, we can unambiguously declare an unsigned 16-bit variable by giving it type `uint16_t`, and a signed variable of 32 bits as `int32_t`.

Along with these data types are a set of macros defining the minimum and maximum values for each value of `N`. These have names of the form `INTN_MIN`, `INTN_MAX`, and `UINTN_MAX`.

Formatted printing with fixed-width types requires use of macros that expand into format strings in a system-dependent manner. So, for example, the values of variables `x` and `y` of type `int32_t` and `uint64_t` can be printed by the following call to `printf`:

```
printf("x = %" PRId32 ", y = %" PRIu64 "\n", x, y);
```

When compiled as a 64-bit program, macro `PRId32` expands to the string `"d"`, while `PRIu64` expands to the pair of strings `"l" "u"`. When the C preprocessor encounters a sequence of string constants separated only by spaces (or other whitespace characters), it concatenates them together. Thus, the above call to `printf` becomes

```
printf("x = %d, y = %lu\n", x, y);
```

Using the macros ensures that a correct format string will be generated regardless of how the code is compiled.

Figure 2.14 also shows the representations of constants −1 and 0. Note that −1 has the same bit representation as UMax—a string of all ones. Numeric value 0 is represented as a string of all zeros in both representations.

The C standards do not require signed integers to be represented in two's-complement form, but nearly all machines do so. Programmers who are concerned with maximizing portability across all possible machines should not assume any particular range of representable values, beyond the ranges indicated in Figure 2.11, nor should they assume any particular representation of signed numbers. On the other hand, many programs are written assuming a two's-complement representation of signed numbers, and the "typical" ranges shown in Figures 2.9 and 2.10, and these programs are portable across a broad range of machines and compilers. The file `<limits.h>` in the C library defines a set of constants

Aside Alt r ati r r tati o o i r

There are two other standard representations for signed numbers:

Ones' complement. This is the same as two's complement, except that the most significant bit has weight $-(2^{w-1} - 1)$ rather than -2^{w-1}:

$$B2O_w(x) \doteq -x_{w-1}(2^{w-1} - 1) + \sum_{i=0}^{w-2} x_i 2^i$$

Sign magnitude. The most significant bit is a sign bit that determines whether the remaining bits should be given negative or positive weight:

$$B2S_w(x) \doteq (-1)^{x_{w-1}} \cdot \sum_{i=0}^{w-2} x_i 2^i$$

Both of these representations have the curious property that there are two different encodings of the number 0. For both representations, $[00\cdots0]$ is interpreted as $+0$. The value -0 can be represented in sign-magnitude form as $[10\cdots0]$ and in ones' complement as $[11\cdots1]$. Although machines based on ones'-complement representations were built in the past, almost all modern machines use two's complement. We will see that sign-magnitude encoding is used with floating-point numbers.

Note the different position of apostrophes: two's complement versus ones' complement. The term "two's complement" arises from the fact that for nonnegative x we compute a w-bit representation of $-x$ as $2^w - x$ (a single two.) The term "ones' complement" comes from the property that we can compute $-x$ in this notation as $[111\cdots1] - x$ (multiple ones).

delimiting the ranges of the different integer data types for the particular machine on which the compiler is running. For example, it defines constants `INT_MAX`, `INT_MIN`, and `UINT_MAX` describing the ranges of signed and unsigned integers. For a two's-complement machine in which data type `int` has w bits, these constants correspond to the values of $TMax_w$, $TMin_w$, and $UMax_w$.

The Java standard is quite specific about integer data type ranges and representations. It requires a two's-complement representation with the exact ranges shown for the 64-bit case (Figure 2.10). In Java, the single-byte data type is called `byte` instead of `char`. These detailed requirements are intended to enable Java programs to behave identically regardless of the machines or operating systems running them.

To get a better understanding of the two's-complement representation, consider the following code example:

```
1       short x = 12345;
2       short mx = -x;
3
4       show_bytes((byte_pointer) &x, sizeof(short));
5       show_bytes((byte_pointer) &mx, sizeof(short));
```

```
4004d0:  48 81 ec e0 02 00 00    sub    $0x2e0,%rsp              A.
4004d7:  48 8b 44 24 a8          mov    -0x58(%rsp),%rax         B.
4004dc:  48 03 47 28             add    0x28(%rdi),%rax          C.
4004e0:  48 89 44 24 d0          mov    %rax,-0x30(%rsp)         D.
4004e5:  48 8b 44 24 78          mov    0x78(%rsp),%rax          E.
4004ea:  48 89 87 88 00 00 00    mov    %rax,0x88(%rdi)          F.
4004f1:  48 8b 84 24 f8 01 00    mov    0x1f8(%rsp),%rax         G.
4004f8:  00
4004f9:  48 03 44 24 08          add    0x8(%rsp),%rax
4004fe:  48 89 84 24 c0 00 00    mov    %rax,0xc0(%rsp)          H.
400505:  00
400506:  48 8b 44 d4 b8          mov    -0x48(%rsp,%rdx,8),%rax  I.
```

2.2.4 Conversions between Signed and Unsigned

C allows casting between different numeric data types. For example, suppose variable `x` is declared as `int` and `u` as `unsigned`. The expression `(unsigned) x` converts the value of `x` to an unsigned value, and `(int) u` converts the value of `u` to a signed integer. What should be the effect of casting signed value to unsigned, or vice versa? From a mathematical perspective, one can imagine several different conventions. Clearly, we want to preserve any value that can be represented in both forms. On the other hand, converting a negative value to unsigned might yield zero. Converting an unsigned value that is too large to be represented in two's-complement form might yield `TMax`. For most implementations of C, however, the answer to this question is based on a bit-level perspective, rather than on a numeric one.

For example, consider the following code:

```
1       short    int    v  = -12345;
2       unsigned short uv = (unsigned short) v;
3       printf("v = %d, uv = %u\n", v, uv);
```

When run on a two's-complement machine, it generates the following output:

```
v = -12345, uv = 53191
```

What we see here is that the effect of casting is to keep the bit values identical but change how these bits are interpreted. We saw in Figure 2.15 that the 16-bit two's-complement representation of −12,345 is identical to the 16-bit unsigned representation of 53,191. Casting from `short` to `unsigned short` changed the numeric value, but not the bit representation.

Similarly, consider the following code:

```
1       unsigned u = 4294967295u;   /* UMax */
2       int     tu = (int) u;
```

```
3       printf("u = %u, tu = %d\n", u, tu);
```

When run on a two's-complement machine, it generates the following output:

```
u = 4294967295, tu = -1
```

We can see from Figure 2.14 that, for a 32-bit word size, the bit patterns representing 4,294,967,295 ($UMax_{32}$) in unsigned form and -1 in two's-complement form are identical. In casting from `unsigned` to `int`, the underlying bit representation stays the same.

This is a general rule for how most C implementations handle conversions between signed and unsigned numbers with the same word size—the numeric values might change, but the bit patterns do not. Let us capture this idea in a more mathematical form. We defined functions $U2B_w$ and $T2B_w$ that map numbers to their bit representations in either unsigned or two's-complement form. That is, given an integer x in the range $0 \le x < UMax_w$, the function $U2B_w(x)$ gives the unique w-bit unsigned representation of x. Similarly, when x is in the range $TMin_w \le x \le TMax_w$, the function $T2B_w(x)$ gives the unique w-bit two's-complement representation of x.

Now define the function $T2U_w$ as $T2U_w(x) \doteq B2U_w(T2B_w(x))$. This function takes a number between $TMin_w$ and $TMax_w$ and yields a number between 0 and $UMax_w$, where the two numbers have identical bit representations, except that the argument has a two's-complement representation while the result is unsigned. Similarly, for x between 0 and $UMax_w$, the function $U2T_w$, defined as $U2T_w(x) \doteq B2T_w(U2B_w(x))$, yields the number having the same two's-complement representation as the unsigned representation of x.

Pursuing our earlier examples, we see from Figure 2.15 that $T2U_{16}(-12{,}345) = 53{,}191$, and that $U2T_{16}(53{,}191) = -12{,}345$. That is, the 16-bit pattern written in hexadecimal as `0xCFC7` is both the two's-complement representation of $-12{,}345$ and the unsigned representation of 53,191. Note also that $12{,}345 + 53{,}191 = 65{,}536 = 2^{16}$. This property generalizes to a relationship between the two numeric values (two's complement and unsigned) represented by a given bit pattern. Similarly, from Figure 2.14, we see that $T2U_{32}(-1) = 4{,}294{,}967{,}295$, and $U2T_{32}(4{,}294{,}967{,}295) = -1$. That is, $UMax$ has the same bit representation in unsigned form as does -1 in two's-complement form. We can also see the relationship between these two numbers: $1 + UMax_w = 2^w$.

We see, then, that function $T2U$ describes the conversion of a two's-complement number to its unsigned counterpart, while $U2T$ converts in the opposite direction. These describe the effect of casting between these data types in most C implementations.

Practice Problem 2.19 (solution page 185)

Using the table you filled in when solving Problem 2.17, fill in the following table describing the function $T2U_4$:

x	$T2U_4(x)$
−1	
−5	
−6	
−4	
1	
8	

The relationship we have seen, via several examples, between the two's-complement and unsigned values for a given bit pattern can be expressed as a property of the function T2U:

principle: Conversion from two's complement to unsigned

For x such that $TMin_w \le x \le TMax_w$:

$$T2U_w(x) = \begin{cases} x + 2^w, & x < 0 \\ x, & x \ge 0 \end{cases} \tag{2.5}$$

For example, we saw that $T2U_{16}(-12{,}345) = -12{,}345 + 2^{16} = 53{,}191$, and also that $T2U_w(-1) = -1 + 2^w = UMax_w$.

This property can be derived by comparing Equations 2.1 and 2.3.

derivation: Conversion from two's complement to unsigned

Comparing Equations 2.1 and 2.3, we can see that for bit pattern x, if we compute the difference $B2U_w(x) - B2T_w(x)$, the weighted sums for bits from 0 to w − 2 will cancel each other, leaving a value $B2U_w(x) - B2T_w(x) = x_{w-1}(2^{w-1} - -2^{w-1}) = x_{w-1}2^w$. This gives a relationship $B2U_w(x) = B2T_w(x) + x_{w-1}2^w$. We therefore have

$$B2U_w(T2B_w(x)) = T2U_w(x) = x + x_{w-1}2^w \tag{2.6}$$

In a two's-complement representation of x, bit x_{w-1} determines whether or not x is negative, giving the two cases of Equation 2.5.

As examples, Figure 2.16 compares how functions B2U and B2T assign values to bit patterns for w = 4. For the two's-complement case, the most significant bit serves as the sign bit, which we diagram as a leftward-pointing gray bar. For the unsigned case, this bit has positive weight, which we show as a rightward-pointing black bar. In going from two's complement to unsigned, the most significant bit changes its weight from −8 to +8. As a consequence, the values that are negative in a two's-complement representation increase by $2^4 = 16$ with an unsigned representation. Thus, −5 becomes +11, and −1 becomes +15.

Figure 2.16
Comparing unsigned and two's-complement representations for w = 4.

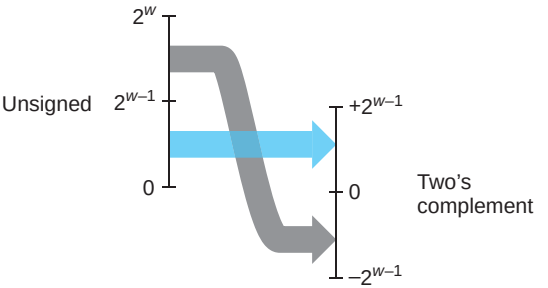


Figure 2.18 Conversion from unsigned to two's complement. Function U2T converts a w-bit unsigned number to the corresponding two's-complement value. For arguments greater than $2^{w-1} - 1$, this involves subtracting 2^w.

This principle can be justified as follows:

derivation: Unsigned to two's-complement conversion

Let $u = \mathrm{U2B}_w(u)$. This bit vector will also be the two's-complement representation of $\mathrm{U2T}_w(u)$. Equations 2.1 and 2.3 can be combined to give

$$\mathrm{U2T}_w(u) = -u_{w-1}2^w + u \tag{2.8}$$

In the unsigned representation of u, bit u_{w-1} determines whether or not u is greater than $\mathrm{TMax}_w = 2^{w-1} - 1$, giving the two cases of Equation 2.7. ■

The behavior of function U2T is illustrated in Figure 2.18. For small ($\leq \mathrm{TMax}_w$) numbers, the conversion from unsigned to signed preserves the numeric value. Large ($> \mathrm{TMax}_w$) numbers are converted to negative values.

To summarize, we considered the effects of converting in both directions between unsigned and two's-complement representations. For values x in the range $0 \leq x \leq \mathrm{TMax}_w$, we have $\mathrm{T2U}_w(x) = x$ and $\mathrm{U2T}_w(x) = x$. That is, numbers in this range have identical unsigned and two's-complement representations. For values outside of this range, the conversions either add or subtract 2^w. For example, we have $\mathrm{T2U}_w(-1) = -1 + 2^w = \mathrm{UMax}_w$—the negative number closest to zero maps to the largest unsigned number. At the other extreme, one can see that $\mathrm{T2U}_w(\mathrm{TMin}_w) = -2^{w-1} + 2^w = 2^{w-1} = \mathrm{TMax}_w + 1$—the most negative number maps to an unsigned number just outside the range of positive two's-complement numbers. Using the example of Figure 2.15, we can see that $\mathrm{T2U}_{16}(-12{,}345) = 65{,}536 + -12{,}345 = 53{,}191$.

2.2.5 Signed versus Unsigned in C

As indicated in Figures 2.9 and 2.10, C supports both signed and unsigned arithmetic for all of its integer data types. Although the C standard does not specify a particular representation of signed numbers, almost all machines use two's complement. Generally, most numbers are signed by default. For example, when declaring a constant such as `12345` or `0x1A2B`, the value is considered signed. Adding character '`U`' or '`u`' as a suffix creates an unsigned constant; for example, `12345U` or `0x1A2Bu`.

C allows conversion between unsigned and signed. Although the C standard does not specify precisely how this conversion should be made, most systems follow the rule that the underlying bit representation does not change. This rule has the effect of applying the function $\mathsf{U2T}_w$ when converting from unsigned to signed, and $\mathsf{T2U}_w$ when converting from signed to unsigned, where w is the number of bits for the data type.

Conversions can happen due to explicit casting, such as in the following code:

```
1       int tx, ty;
2       unsigned ux, uy;
3
4       tx = (int) ux;
5       uy = (unsigned) ty;
```

Alternatively, they can happen implicitly when an expression of one type is assigned to a variable of another, as in the following code:

```
1       int tx, ty;
2       unsigned ux, uy;
3
4       tx = ux; /* Cast to signed */
5       uy = ty; /* Cast to unsigned */
```

When printing numeric values with `printf`, the directives `%d`, `%u`, and `%x` are used to print a number as a signed decimal, an unsigned decimal, and in hexadecimal format, respectively. Note that `printf` does not make use of any type information, and so it is possible to print a value of type `int` with directive `%u` and a value of type `unsigned` with directive `%d`. For example, consider the following code:

```
1       int x = -1;
2       unsigned u = 2147483648; /* 2 to the 31st */
3
4       printf("x = %u = %d\n", x, x);
5       printf("u = %u = %d\n", u, u);
```

When compiled as a 32-bit program, it prints the following:

```
x = 4294967295 = -1
u = 2147483648 = -2147483648
```

In both cases, `printf` prints the word first as if it represented an unsigned number and second as if it represented a signed number. We can see the conversion routines in action: $\mathsf{T2U}_{32}(-1) = \mathsf{UMax}_{32} = 2^{32} - 1$ and $\mathsf{U2T}_{32}(2^{31}) = 2^{31} - 2^{32} = -2^{31} = \mathsf{TMin}_{32}$.

Some possibly nonintuitive behavior arises due to C's handling of expressions containing combinations of signed and unsigned quantities. When an operation is performed where one operand is signed and the other is unsigned, C implicitly casts the signed argument to unsigned and performs the operations

Expression			Type	Evaluation
`0`	`==`	`0U`	Unsigned	1
`-1`	`<`	`0`	Signed	1
`-1`	`<`	`0U`	Unsigned	0 *
`2147483647`	`>`	`-2147483647-1`	Signed	1
`2147483647U`	`>`	`-2147483647-1`	Unsigned	0 *
`2147483647`	`>`	`(int) 2147483648U`	Signed	1 *
`-1`	`>`	`-2`	Signed	1
`(unsigned) -1`	`>`	`-2`	Unsigned	1

Figure 2.19 Effects of C promotion rules. N[illegible] i t [illegible]i a ar ar [illegible] '*'. W [illegible] r ra [illegible] a ari [illegible] i [illegible], t [illegible] t r [illegible] ra [illegible] i i li [illegible] a t t [illegible] i [illegible]. S W A [illegible] data:tmin [illegible] r [illegible] rit $TMin_{32}$ a -2,147,483,647-1.

assuming the numbers are nonnegative. As we will see, this convention makes little difference for standard arithmetic operations, but it leads to nonintuitive results for relational operators such as < and >. Figure 2.19 shows some sample relational expressions and their resulting evaluations, when data type `int` has a 32-bit two's-complement representation. Consider the comparison `-1 < 0U`. Since the second operand is unsigned, the first one is implicitly cast to unsigned, and hence the expression is equivalent to the comparison `4294967295U < 0U` (recall that $T2U_w(-1) = UMax_w$), which of course is false. The other cases can be understood by similar analyses.

Practice Problem 2.21 (solution page 185)

Assuming the expressions are evaluated when executing a 32-bit program on a machine that uses two's-complement arithmetic, fill in the following table describing the effect of casting and relational operations, in the style of Figure 2.19:

Expression	Type	Evaluation
`-2147483647-1 == 2147483648U`	_____	_____
`-2147483647-1 < 2147483647`	_____	_____
`-2147483647-1U < 2147483647`	_____	_____
`-2147483647-1 < -2147483647`	_____	_____
`-2147483647-1U < -2147483647`	_____	_____

2.2.6 Expanding the Bit Representation of a Number

One common operation is to convert between integers having different word sizes while retaining the same numeric value. Of course, this may not be possible when the destination data type is too small to represent the desired value. Converting from a smaller to a larger data type, however, should always be possible.

Web Aside DATA:TMIN Writing TMin in C

In Figure 2.19 and in Problem 2.21, we carefully wrote the value of $TMin_{32}$ as `-2,147,483,647-1`. Why not simply write it as either `-2,147,483,648` or `0x80000000`? Looking at the C header file `limits.h`, we see that they use a similar method as we have to write $TMin_{32}$ and $TMax_{32}$:

```
/* Minimum and maximum values a 'signed int' can hold.  */
#define INT_MAX   2147483647
#define INT_MIN   (-INT_MAX - 1)
```

Unfortunately, a curious interaction between the asymmetry of the two's-complement representation and the conversion rules of C forces us to write $TMin_{32}$ in this unusual way. Although understanding this issue requires us to delve into one of the murkier corners of the C language standards, it will help us appreciate some of the subtleties of integer data types and representations.

To convert an unsigned number to a larger data type, we can simply add leading zeros to the representation; this operation is known as zero extension, expressed by the following principle:

principle: Expansion of an unsigned number by zero extension

Define bit vectors $u = [u_{w-1}, u_{w-2}, \ldots, u_0]$ of width w and $u' = [0, \ldots, 0, u_{w-1}, u_{w-2}, \ldots, u_0]$ of width w', where $w' > w$. Then $B2U_{w'}(u') = B2U_w(u)$. ■

This principle can be seen to follow directly from the definition of the unsigned encoding, given by Equation 2.1.

For converting a two's-complement number to a larger data type, the rule is to perform a sign extension, adding copies of the most significant bit to the representation, expressed by the following principle. We show the sign bit x_{w-1} in blue to highlight its role in sign extension.

principle: Expansion of a two's-complement number by sign extension

Define bit vectors $x = [x_{w-1}, x_{w-2}, \ldots, x_0]$ of width w and $x' = [x_{w-1}, \ldots, x_{w-1}, x_{w-1}, x_{w-2}, \ldots, x_0]$ of width w', where $w' > w$. Then $B2T_{w'}(x') = B2T_w(x)$. ■

As an example, consider the following code:

```
1    short sx = -12345;        /* -12345 */
2    unsigned short usx = sx;  /*  53191 */
3    int x = sx;               /* -12345 */
4    unsigned ux = usx;        /*  53191 */
5
6    printf("sx  = %d:\t", sx);
7    show_bytes((byte_pointer) &sx, sizeof(short));
8    printf("usx = %u:\t", usx);
9    show_bytes((byte_pointer) &usx, sizeof(unsigned short));
10   printf("x   = %d:\t", x);
```

```
11    show_bytes((byte_pointer) &x, sizeof(int));
12    printf("ux  = %u:\t", ux);
13    show_bytes((byte_pointer) &ux, sizeof(unsigned));
```

When run as a 32-bit program on a big-endian machine that uses a two's-complement representation, this code prints the output

```
sx  = -12345:  cf c7
usx = 53191:   cf c7
x   = -12345:  ff ff cf c7
ux  = 53191:   00 00 cf c7
```

We see that, although the two's-complement representation of −12,345 and the unsigned representation of 53,191 are identical for a 16-bit word size, they differ for a 32-bit word size. In particular, −12,345 has hexadecimal representation `0xFFFFCFC7`, while 53,191 has hexadecimal representation `0x0000CFC7`. The former has been sign extended—16 copies of the most significant bit 1, having hexadecimal representation `0xFFFF`, have been added as leading bits. The latter has been extended with 16 leading zeros, having hexadecimal representation `0x0000`.

As an illustration, Figure 2.20 shows the result of expanding from word size w = 3 to w = 4 by sign extension. Bit vector [101] represents the value −4 + 1 = −3. Applying sign extension gives bit vector [1101] representing the value −8 + 4 + 1 = −3. We can see that, for w = 4, the combined value of the two most significant bits, −8 + 4 = −4, matches the value of the sign bit for w = 3. Similarly, bit vectors [111] and [1111] both represent the value −1.

With this as intuition, we can now show that sign extension preserves the value of a two's-complement number.

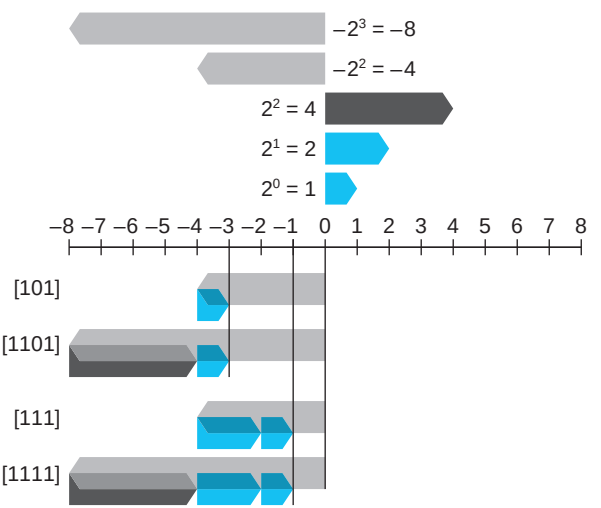


Figure 2.20
Examples of sign extension from w = 3 to w = 4. For w = 4, the two most significant bits have weights −8 and 4, so that the bit vector [1101] has numeric value −8 + 4 + 1 = −3, matching the value of the bit vector for w = 3.

derivation: Expansion of a two's-complement number by sign extension

Let $w = w + k$. What we want to prove is that

$$B2T_{w+k}([\underbrace{x_{w-1}, \ldots, x_{w-1}}_{k \text{ times}}, x_{w-1}, x_{w-2}, \ldots, x_0]) = B2T_w([x_{w-1}, x_{w-2}, \ldots, x_0])$$

The proof follows by induction on k. That is, if we can prove that sign extending by 1 bit preserves the numeric value, then this property will hold when sign extending by an arbitrary number of bits. Thus, the task reduces to proving that

$$B2T_{w+1}([x_{w-1}, x_{w-1}, x_{w-2}, \ldots, x_0]) = B2T_w([x_{w-1}, x_{w-2}, \ldots, x_0])$$

Expanding the left-hand expression with Equation 2.3 gives the following:

$$\begin{aligned}
B2T_{w+1}([x_{w-1}, x_{w-1}, x_{w-2}, \ldots, x_0]) &= -x_{w-1}2^w + \sum_{i=0}^{w-1} x_i 2^i \\
&= -x_{w-1}2^w + x_{w-1}2^{w-1} + \sum_{i=0}^{w-2} x_i 2^i \\
&= -x_{w-1}\left(2^w - 2^{w-1}\right) + \sum_{i=0}^{w-2} x_i 2^i \\
&= -x_{w-1}2^{w-1} + \sum_{i=0}^{w-2} x_i 2^i \\
&= B2T_w([x_{w-1}, x_{w-2}, \ldots, x_0])
\end{aligned}$$

The key property we exploit is that $2^w - 2^{w-1} = 2^{w-1}$. Thus, the combined effect of adding a bit of weight -2^w and of converting the bit having weight -2^{w-1} to be one with weight 2^{w-1} is to preserve the original numeric value. ■

Practice Problem 2.22 (solution page 186)

Show that each of the following bit vectors is a two's-complement representation of −4 by applying Equation 2.3:

A. [1100]

B. [11100]

C. [111100]

Observe that the second and third bit vectors can be derived from the first by sign extension.

One point worth making is that the relative order of conversion from one data size to another and between unsigned and signed can affect the behavior of a program. Consider the following code:

```
1    short sx = -12345;       /* -12345   */
2    unsigned uy = sx;        /* Mystery! */
3
4    printf("uy  = %u:\t", uy);
5    show_bytes((byte_pointer) &uy, sizeof(unsigned));
```

When run on a big-endian machine, this code causes the following output to be printed:

```
uy = 4294954951:  ff ff cf c7
```

This shows that, when converting from `short` to `unsigned`, the program first changes the size and then the type. That is, `(unsigned) sx` is equivalent to `(unsigned) (int) sx`, evaluating to 4,294,954,951, not `(unsigned) (unsigned short) sx`, which evaluates to 53,191. Indeed, this convention is required by the C standards.

Practice Problem 2.23 (solution page 186)

Consider the following C functions:

```
int fun1(unsigned word) {
    return (int) ((word << 24) >> 24);
}

int fun2(unsigned word) {
    return ((int) word << 24) >> 24;
}
```

Assume these are executed as a 32-bit program on a machine that uses two's-complement arithmetic. Assume also that right shifts of signed values are performed arithmetically, while right shifts of unsigned values are performed logically.

A. Fill in the following table showing the effect of these functions for several example arguments. You will find it more convenient to work with a hexadecimal representation. Just remember that hex digits 8 through F have their most significant bits equal to 1.

w	fun1(w)	fun2(w)
0x00000076	_____	_____
0x87654321	_____	_____
0x000000C9	_____	_____
0xEDCBA987	_____	_____

B. Describe in words the useful computation each of these functions performs.

2.2.7 Truncating Numbers

principle: Truncation of a two's-complement number

Let x be the bit vector $[x_{w-1}, x_{w-2}, \ldots, x_0]$, and let x' be the result of truncating it to k bits: $x' = [x_{k-1}, x_{k-2}, \ldots, x_0]$. Let $x = B2T_w(x)$ and $x' = B2T_k(x')$. Then $x' = U2T_k(x \bmod 2^k)$. ■

In this formulation, $x \bmod 2^k$ will be a number between 0 and $2^k - 1$. Applying function $U2T_k$ to it will have the effect of converting the most significant bit x_{k-1} from having weight 2^{k-1} to having weight -2^{k-1}. We can see this with the example of converting value $x = 53{,}191$ from `int` to `short`. Since $2^{16} = 65{,}536 \geq x$, we have $x \bmod 2^{16} = x$. But when we convert this number to a 16-bit two's-complement number, we get $x' = 53{,}191 - 65{,}536 = -12{,}345$.

derivation: Truncation of a two's-complement number

Using a similar argument to the one we used for truncation of an unsigned number shows that

$$B2T_w([x_{w-1}, x_{w-2}, \ldots, x_0]) \bmod 2^k = B2U_k([x_{k-1}, x_{k-2}, \ldots, x_0])$$

That is, $x \bmod 2^k$ can be represented by an unsigned number having bit-level representation $[x_{k-1}, x_{k-2}, \ldots, x_0]$. Converting this to a two's-complement number gives $x' = U2T_k(x \bmod 2^k)$. ■

Summarizing, the effect of truncation for unsigned numbers is

$$B2U_k([x_{k-1}, x_{k-2}, \ldots, x_0]) = B2U_w([x_{w-1}, x_{w-2}, \ldots, x_0]) \bmod 2^k \quad (2.9)$$

while the effect for two's-complement numbers is

$$B2T_k([x_{k-1}, x_{k-2}, \ldots, x_0]) = U2T_k(B2U_w([x_{w-1}, x_{w-2}, \ldots, x_0]) \bmod 2^k) \quad (2.10)$$

Practice Problem 2.24 (solution page 186)

Suppose we truncate a 4-bit value (represented by hex digits `0` through `F`) to a 3-bit value (represented as hex digits `0` through `7`.) Fill in the table below showing the effect of this truncation for some cases, in terms of the unsigned and two's-complement interpretations of those bit patterns.

Hex	Unsigned	Two's complement

2.2.8 Advice on Signed versus Unsigned

As we have seen, the implicit casting of signed to unsigned leads to some nonintuitive behavior. Nonintuitive features often lead to program bugs, and ones involving the nuances of implicit casting can be especially difficult to see. Since the casting takes place without any clear indication in the code, programmers often overlook its effects.

The following two practice problems illustrate some of the subtle errors that can arise due to implicit casting and the unsigned data type.

Practice Problem 2.25 (solution page 187)

Consider the following code that attempts to sum the elements of an array `a`, where the number of elements is given by parameter `length`:

```
1   /* WARNING: This is buggy code */
2   float sum_elements(float a[], unsigned length) {
3       int i;
4       float result = 0;
5
6       for (i = 0; i <= length-1; i++)
7           result += a[i];
8       return result;
9   }
```

When run with argument `length` equal to 0, this code should return 0.0. Instead, it encounters a memory error. Explain why this happens. Show how this code can be corrected.

Practice Problem 2.26 (solution page 187)

You are given the assignment of writing a function that determines whether one string is longer than another. You decide to make use of the string library function `strlen` having the following declaration:

```
/* Prototype for library function strlen */
size_t strlen(const char *s);
```

Here is your first attempt at the function:

```
/* Determine whether string s is longer than string t */
/* WARNING: This function is buggy */
int strlonger(char *s, char *t) {
    return strlen(s) - strlen(t) > 0;
}
```

When you test this on some sample data, things do not seem to work quite right. You investigate further and determine that, when compiled as a 32-bit

program, data type `size_t` is defined (via `typedef`) in header file `stdio.h` to be `unsigned`.

A. For what cases will this function produce an incorrect result?

B. Explain how this incorrect result comes about.

C. Show how to fix the code so that it will work reliably.

We have seen multiple ways in which the subtle features of unsigned arithmetic, and especially the implicit conversion of signed to unsigned, can lead to errors or vulnerabilities. One way to avoid such bugs is to never use unsigned numbers. In fact, few languages other than C support unsigned integers. Apparently, these other language designers viewed them as more trouble than they are worth. For example, Java supports only signed integers, and it requires that they be implemented with two's-complement arithmetic. The normal right shift operator `>>` is guaranteed to perform an arithmetic shift. The special operator `>>>` is defined to perform a logical right shift.

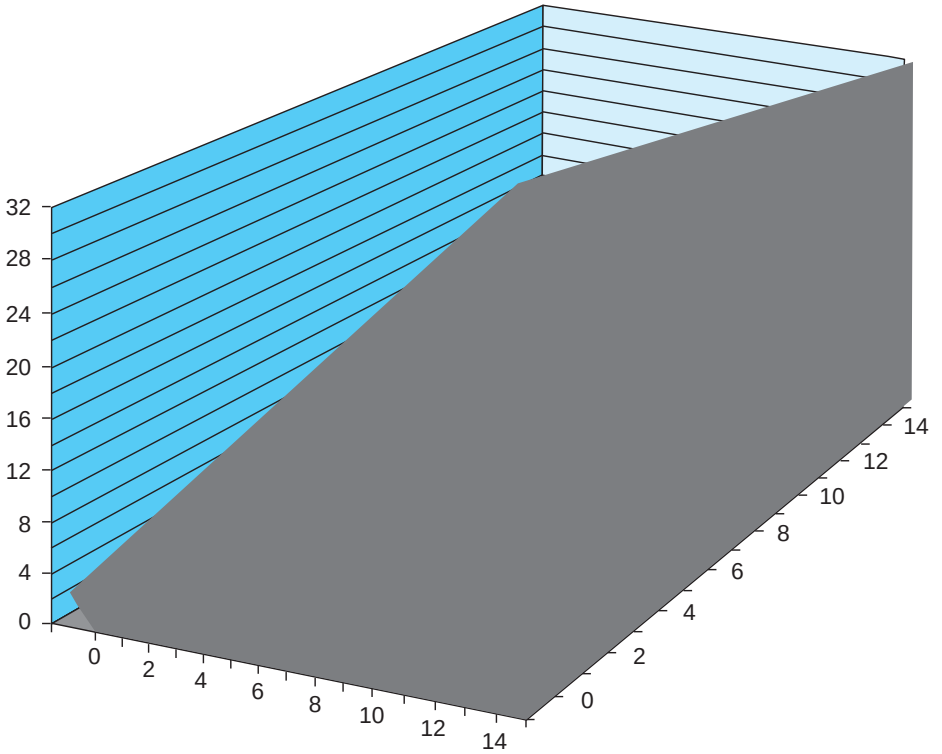
Unsigned values are very useful when we want to think of words as just collections of bits with no numeric interpretation. This occurs, for example, when packing a word with flags describing various Boolean conditions. Addresses are naturally unsigned, so systems programmers find unsigned types to be helpful. Unsigned values are also useful when implementing mathematical packages for modular arithmetic and for multiprecision arithmetic, in which numbers are represented by arrays of words.

2.3 Integer Arithmetic

Many beginning programmers are surprised to find that adding two positive numbers can yield a negative result, and that the comparison `x < y` can yield a different result than the comparison `x-y < 0`. These properties are artifacts of the finite nature of computer arithmetic. Understanding the nuances of computer arithmetic can help programmers write more reliable code.

2.3.1 Unsigned Addition

Consider two nonnegative integers `x` and `y`, such that $0 \le x, y < 2^w$. Each of these values can be represented by a `w`-bit unsigned number. If we compute their sum, however, we have a possible range $0 \le x + y \le 2^{w+1} - 2$. Representing this sum could require `w + 1` bits. For example, Figure 2.21 shows a plot of the function `x + y` when `x` and `y` have 4-bit representations. The arguments (shown on the horizontal axes) range from 0 to 15, but the sum ranges from 0 to 30. The shape of the function is a sloping plane (the function is linear in both dimensions). If we were to maintain the sum as a `(w + 1)`-bit number and add it to another value, we may require `w + 2` bits, and so on. This continued "word size


32
28
24
20
16
12
8
4
0
0
2
4
6
8
10
12
14
0
2
4
6
8
10
12
14

Aside Security vulnerability in getpeername

In 2002, programmers involved in the FreeBSD open-source operating systems project realized that their implementation of the `getpeername` library function had a security vulnerability. A simplified version of their code went something like this:

```
1   /*
2    * Illustration of code vulnerability similar to that found in
3    * FreeBSD's implementation of getpeername()
4    */
5
6   /* Declaration of library function memcpy */
7   void *memcpy(void *dest, void *src, size_t n);
8
9   /* Kernel memory region holding user-accessible data */
10  #define KSIZE 1024
11  char kbuf[KSIZE];
12
13  /* Copy at most maxlen bytes from kernel region to user buffer */
14  int copy_from_kernel(void *user_dest, int maxlen) {
15      /* Byte count len is minimum of buffer size and maxlen */
16      int len = KSIZE < maxlen ? KSIZE : maxlen;
17      memcpy(user_dest, kbuf, len);
18      return len;
19  }
```

In this code, we show the prototype for library function `memcpy` on line 7, which is designed to copy a specified number of bytes `n` from one region of memory to another.

The function `copy_from_kernel`, starting at line 14, is designed to copy some of the data maintained by the operating system kernel to a designated region of memory accessible to the user. Most of the data structures maintained by the kernel should not be readable by a user, since they may contain sensitive information about other users and about other jobs running on the system, but the region shown as `kbuf` was intended to be one that the user could read. The parameter `maxlen` is intended to be the length of the buffer allocated by the user and indicated by argument `user_dest`. The computation at line 16 then makes sure that no more bytes are copied than are available in either the source or the destination buffer.

Suppose, however, that some malicious programmer writes code that calls `copy_from_kernel` with a negative value of `maxlen`. Then the minimum computation on line 16 will compute this value for `len`, which will then be passed as the parameter `n` to `memcpy`. Note, however, that parameter `n` is declared as having data type `size_t`. This data type is declared (via `typedef`) in the library file `stdio.h`. Typically, it is defined to be `unsigned` for 32-bit programs and `unsigned long` for 64-bit programs. Since argument `n` is unsigned, `memcpy` will treat it as a very large positive number and attempt to copy that many bytes from the kernel region to the user's buffer. Copying that many bytes (at least 2^{31}) will not actually work, because the program will encounter invalid addresses in the process, but the program could read regions of the kernel memory for which it is not authorized.

Aside Security vulnerability in the Internet worm

We can see that this problem arises due to the mismatch between data types: in one place the length parameter is signed; in another place it is unsigned. Such mismatches can be a source of bugs and, as this example shows, can even lead to security vulnerabilities. Fortunately, there were no reported cases where a programmer had exploited the vulnerability in FreeBSD. They issued a security advisory "FreeBSD-SA-02:38.signed-error" advising system administrators on how to apply a patch that would remove the vulnerability. The bug can be fixed by declaring parameter `maxlen` to `copy_from_kernel` to be of type `size_t`, to be consistent with parameter `n` of `memcpy`. We should also declare local variable `len` and the return value to be of type `size_t`.

We can characterize operation $+^u_w$ as follows:

principle: Unsigned addition

For x and y such that $0 \le x, y < 2^w$:

$$x +^u_w y = \begin{cases} x + y, & x + y < 2^w \quad \text{Normal} \\ x + y - 2^w, & 2^w \le x + y < 2^{w+1} \quad \text{Overflow} \end{cases} \tag{2.11}$$

The two cases of Equation 2.11 are illustrated in Figure 2.22, showing the sum $x + y$ on the left mapping to the unsigned w-bit sum $x +^u_w y$ on the right. The normal case preserves the value of $x + y$, while the overflow case has the effect of decrementing this sum by 2^w.

derivation: Unsigned addition

In general, we can see that if $x + y < 2^w$, the leading bit in the $(w + 1)$-bit representation of the sum will equal 0, and hence discarding it will not change the numeric value. On the other hand, if $2^w \le x + y < 2^{w+1}$, the leading bit in the $(w + 1)$-bit representation of the sum will equal 1, and hence discarding it is equivalent to subtracting 2^w from the sum.

An arithmetic operation is said to overflow when the full integer result cannot fit within the word size limits of the data type. As Equation 2.11 indicates, overflow

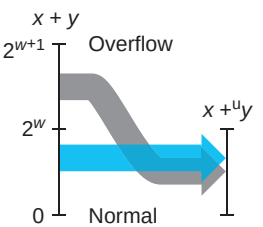
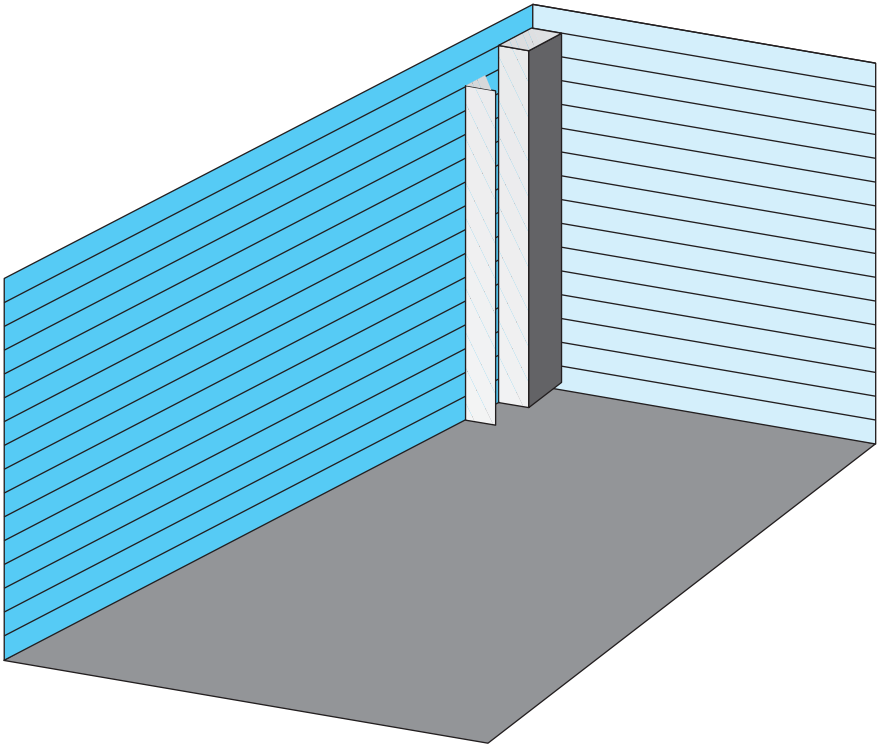


Figure 2.22 Relation between integer addition and unsigned addition. W x + y i r at r t a $2^w - 1$, t o r o .



derivation: Detecting overflow of unsigned addition

Observe that $x + y \geq x$, and hence if s did not overflow, we will surely have $s \geq x$. On the other hand, if s did overflow, we have $s = x + y - 2^w$. Given that $y < 2^w$, we have $y - 2^w < 0$, and hence $s = x + (y - 2^w) < x$. ■

Practice Problem 2.27 (solution page 188)

Write a function with the following prototype:

```
/* Determine whether arguments can be added without overflow */
int uadd_ok(unsigned x, unsigned y);
```

This function should return 1 if arguments x and y can be added without causing overflow.

Modular addition forms a mathematical structure known as an abelian group, named after the Norwegian mathematician Niels Henrik Abel (1802–1829). That is, it is commutative (that's where the "abelian" part comes in) and associative; it has an identity element 0, and every element has an additive inverse. Let us consider the set of w-bit unsigned numbers with addition operation $+^u_w$. For every value x, there must be some value $-^u_w x$ such that $-^u_w x +^u_w x = 0$. This additive inverse operation can be characterized as follows:

principle: Unsigned negation

For any number x such that $0 \leq x < 2^w$, its w-bit unsigned negation $-^u_w x$ is given by the following:

$$-^u_w x = \begin{cases} x, & x = 0 \\ 2^w - x, & x > 0 \end{cases} \qquad (2.12)$$

■

This result can readily be derived by case analysis:

derivation: Unsigned negation

When $x = 0$, the additive inverse is clearly 0. For $x > 0$, consider the value $2^w - x$. Observe that this number is in the range $0 < 2^w - x < 2^w$. We can also see that $(x + 2^w - x) \bmod 2^w = 2^w \bmod 2^w = 0$. Hence it is the inverse of x under $+^u_w$. ■

Practice Problem 2.28 (solution page 188)

We can represent a bit pattern of length $w = 4$ with a single hex digit. For an unsigned interpretation of these digits, use Equation 2.12 to fill in the following table giving the values and the bit representations (in hex) of the unsigned additive inverses of the digits shown.

x		$-_4^u x$	
Hex	Decimal	Decimal	Hex
1			
4			
7			
A			
E			

2.3.2 Two's-Complement Addition

With two's-complement addition, we must decide what to do when the result is either too large (positive) or too small (negative) to represent. Given integer values x and y in the range $-2^{w-1} \le x, y \le 2^{w-1} - 1$, their sum is in the range $-2^w \le x + y \le 2^w - 2$, potentially requiring $w + 1$ bits to represent exactly. As before, we avoid ever-expanding data sizes by truncating the representation to w bits. The result is not as familiar mathematically as modular addition, however. Let us define $x +_w^t y$ to be the result of truncating the integer sum $x + y$ to be w bits long and then viewing the result as a two's-complement number.

principle: Two's-complement addition

For integer values x and y in the range $-2^{w-1} \le x, y \le 2^{w-1} - 1$:

$$x +_w^t y = \begin{cases} x + y - 2^w, & 2^{w-1} \le x + y \quad \text{Positive overflow} \\ x + y, & -2^{w-1} \le x + y < 2^{w-1} \quad \text{Normal} \\ x + y + 2^w, & x + y < -2^{w-1} \quad \text{Negative overflow} \end{cases} \tag{2.13}$$

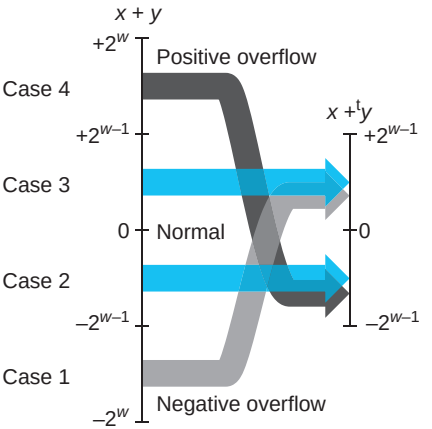
This principle is illustrated in Figure 2.24, where the sum $x + y$ is shown on the left, having a value in the range $-2^w \le x + y \le 2^w - 2$, and the result of truncating the sum to a w-bit two's-complement number is shown on the right. (The labels "Case 1" to "Case 4" in this figure are for the case analysis of the formal derivation of the principle.) When the sum $x + y$ exceeds TMax_w (case 4), we say that positive overflow has occurred. In this case, the effect of truncation is to subtract 2^w from the sum. When the sum $x + y$ is less than TMin_w (case 1), we say that negative overflow has occurred. In this case, the effect of truncation is to add 2^w to the sum.

The w-bit two's-complement sum of two numbers has the exact same bit-level representation as the unsigned sum. In fact, most computers use the same machine instruction to perform either unsigned or signed addition.

derivation: Two's-complement addition

Since two's-complement addition has the exact same bit-level representation as unsigned addition, we can characterize the operation $+_w^t$ as one of converting its arguments to unsigned, performing unsigned addition, and then converting back to two's complement:

Figure 2.24
Relation between integer and two's-complement addition. W x + y i l t a -2^{w-1}, t r i a ati o r o . W it i r at r t a o r al t o 2^{w-1}, t r i a o it i o r o .



$$x +_w^t y = U2T_w(T2U_w(x) +_w^u T2U_w(y)) \tag{2.14}$$

By Equation 2.6, we can write $T2U_w(x)$ as $x_{w-1}2^w + x$ and $T2U_w(y)$ as $y_{w-1}2^w + y$. Using the property that $+_w^u$ is simply addition modulo 2^w, along with the properties of modular addition, we then have

$$\begin{aligned} x +_w^t y &= U2T_w(T2U_w(x) +_w^u T2U_w(y)) \\ &= U2T_w[(x_{w-1}2^w + x + y_{w-1}2^w + y) \bmod 2^w] \\ &= U2T_w[(x + y) \bmod 2^w] \end{aligned}$$

The terms $x_{w-1}2^w$ and $y_{w-1}2^w$ drop out since they equal 0 modulo 2^w.

To better understand this quantity, let us define z as the integer sum $z \doteq x + y$, z as $z \doteq z \bmod 2^w$, and z as $z \doteq U2T_w(z)$. The value z is equal to $x +_w^t y$. We can divide the analysis into four cases as illustrated in Figure 2.24:

1. $-2^w \le z < -2^{w-1}$. Then we will have $z = z + 2^w$. This gives $0 \le z < -2^{w-1} + 2^w = 2^{w-1}$. Examining Equation 2.7, we see that z is in the range such that $z = z$. This is the case of negative overflow. We have added two negative numbers x and y (that's the only way we can have $z < -2^{w-1}$) and obtained a nonnegative result $z = x + y + 2^w$.
2. $-2^{w-1} \le z < 0$. Then we will again have $z = z + 2^w$, giving $-2^{w-1} + 2^w = 2^{w-1} \le z < 2^w$. Examining Equation 2.7, we see that z is in such a range that $z = z - 2^w$, and therefore $z = z - 2^w = z + 2^w - 2^w = z$. That is, our two's-complement sum z equals the integer sum $x + y$.
3. $0 \le z < 2^{w-1}$. Then we will have $z = z$, giving $0 \le z < 2^{w-1}$, and hence $z = z = z$. Again, the two's-complement sum z equals the integer sum $x + y$.
4. $2^{w-1} \le z < 2^w$. We will again have $z = z$, giving $2^{w-1} \le z < 2^w$. But in this range we have $z = z - 2^w$, giving $z = x + y - 2^w$. This is the case of positive overflow. We have added two positive numbers x and y (that's the only way we can have $z \ge 2^{w-1}$) and obtained a negative result $z = x + y - 2^w$. ■

x	y	x + y	$x +^t_4 y$	Case
−8	−5	−13	3	1
[1000]	[1011]	[10011]	[0011]	
−8	−8	−16	0	1
[1000]	[1000]	[10000]	[0000]	
−8	5	−3	−3	2
[1000]	[0101]	[11101]	[1101]	
2	5	7	7	3
[0010]	[0101]	[00111]	[0111]	
5	5	10	−6	4
[0101]	[0101]	[01010]	[1010]	

Figure 2.25 Two's-complement addition examples. T it-l l r r tati o o t 4- itt ' - o l t a o tai r r i i ar a i ti o o t o ra a tr ati t r lt t o 4 it .

As illustrations of two's-complement addition, Figure 2.25 shows some examples when w = 4. Each example is labeled by the case to which it corresponds in the derivation of Equation 2.13. Note that $2^4 = 16$, and hence negative overflow yields a result 16 more than the integer sum, and positive overflow yields a result 16 less. We include bit-level representations of the operands and the result. Observe that the result can be obtained by performing binary addition of the operands and truncating the result to 4 bits.

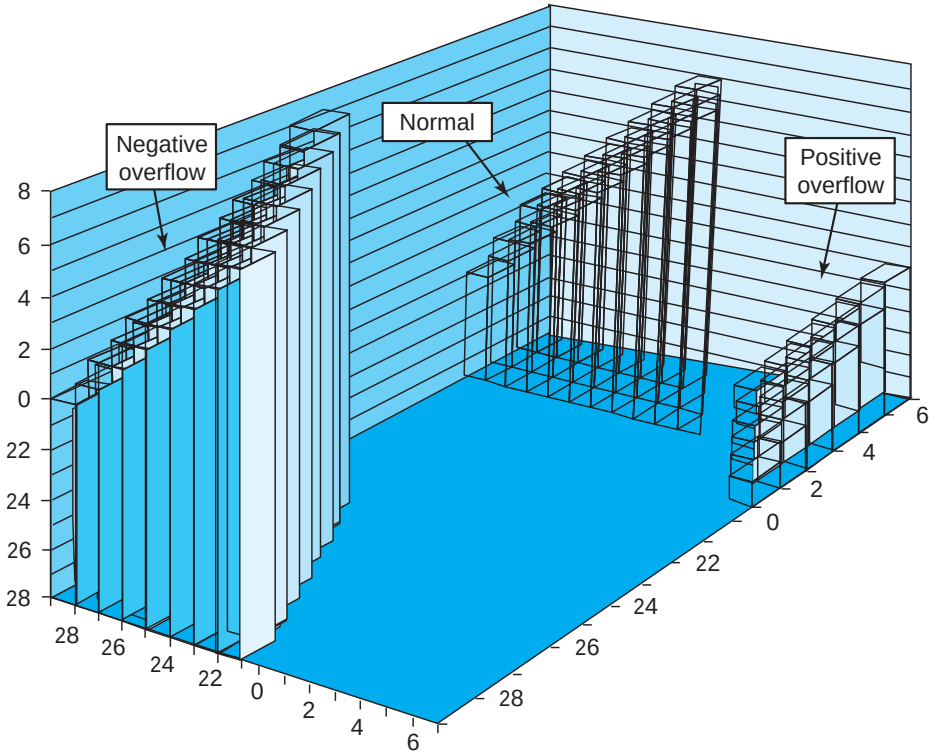
Figure 2.26 illustrates two's-complement addition for word size w = 4. The operands range between −8 and 7. When $x + y < -8$, two's-complement addition has a negative overflow, causing the sum to be incremented by 16. When $-8 \le x + y < 8$, the addition yields $x + y$. When $x + y \ge 8$, the addition has a positive overflow, causing the sum to be decremented by 16. Each of these three ranges forms a sloping plane in the figure.

Equation 2.13 also lets us identify the cases where overflow has occurred:

principle: Detecting overflow in two's-complement addition

For x and y in the range $TMin_w \le x, y \le TMax_w$, let $s \doteq x +^t_w y$. Then the computation of s has had positive overflow if and only if $x > 0$ and $y > 0$ but $s \le 0$. The computation has had negative overflow if and only if $x < 0$ and $y < 0$ but $s \ge 0$. ■

Figure 2.25 shows several illustrations of this principle for w = 4. The first entry shows a case of negative overflow, where two negative numbers sum to a positive one. The final entry shows a case of positive overflow, where two positive numbers sum to a negative one.


Negative overflow
Normal
Positive overflow
8
6
4
2
0
22
24
26
28
28
26
24
22
0
2
4
6
6
4
2
0
22
24
26
28

x	y	x + y	$x +_5^t y$	Case
[11000]	[11000]			
[10111]	[01000]			
[00010]	[00101]			
[01100]	[00100]			

Practice Problem 2.30 (solution page 189)

Write a function with the following prototype:

```
/* Determine whether arguments can be added without overflow */
int tadd_ok(int x, int y);
```

This function should return 1 if arguments `x` and `y` can be added without causing overflow.

Practice Problem 2.31 (solution page 189)

Your coworker gets impatient with your analysis of the overflow conditions for two's-complement addition and presents you with the following implementation of `tadd_ok`:

```
/* Determine whether arguments can be added without overflow */
/* WARNING: This code is buggy. */
int tadd_ok(int x, int y) {
    int sum = x+y;
    return (sum-x == y) && (sum-y == x);
}
```

You look at the code and laugh. Explain why.

Practice Problem 2.32 (solution page 189)

You are assigned the task of writing code for a function `tsub_ok`, with arguments `x` and `y`, that will return 1 if computing `x-y` does not cause overflow. Having just written the code for Problem 2.30, you write the following:

```
/* Determine whether arguments can be subtracted without overflow */
/* WARNING: This code is buggy. */
int tsub_ok(int x, int y) {
```

```
    return tadd_ok(x, -y);
}
```

For what values of x and y will this function give incorrect results? Writing a correct version of this function is left as an exercise (Problem 2.74).

2.3.3 Two's-Complement Negation

We can see that every number x in the range $TMin_w \le x \le TMax_w$ has an additive inverse under $+^t_w$, which we denote $-^t_w x$ as follows:

principle: Two's-complement negation

For x in the range $TMin_w \le x \le TMax_w$, its two's-complement negation $-^t_w x$ is given by the formula

$$-^t_w x = \begin{cases} TMin_w, & x = TMin_w \\ -x, & x > TMin_w \end{cases} \quad (2.15)$$

That is, for w-bit two's-complement addition, $TMin_w$ is its own additive inverse, while any other value x has $-x$ as its additive inverse.

derivation: Two's-complement negation

Observe that $TMin_w + TMin_w = -2^{w-1} + -2^{w-1} = -2^w$. This would cause negative overflow, and hence $TMin_w +^t_w TMin_w = -2^w + 2^w = 0$. For values of x such that $x > TMin_w$, the value $-x$ can also be represented as a w-bit two's-complement number, and their sum will be $-x + x = 0$.

Practice Problem 2.33 (solution page 189)

We can represent a bit pattern of length w = 4 with a single hex digit. For a two's-complement interpretation of these digits, fill in the following table to determine the additive inverses of the digits shown:

x		$-^t_4 x$	
Hex	Decimal	Decimal	Hex
2			
3			
9			
B			
C			

What do you observe about the bit patterns generated by two's-complement and unsigned (Problem 2.28) negation?

Web Aside DATA:TNEG Bit-level representation of two's-complement negation

There are several clever ways to determine the two's-complement negation of a value represented at the bit level. The following two techniques are both useful, such as when one encounters the value `0xfffffffa` when debugging a program, and they lend insight into the nature of the two's-complement representation.

One technique for performing two's-complement negation at the bit level is to complement the bits and then increment the result. In C, we can state that for any integer value `x`, computing the expressions `-x` and `~x + 1` will give identical results.

Here are some examples with a 4-bit word size:

x		~x		incr(~x)	
[0101]	5	[1010]	−6	[1011]	−5
[0111]	7	[1000]	−8	[1001]	−7
[1100]	−4	[0011]	3	[0100]	4
[0000]	0	[1111]	−1	[0000]	0
[1000]	−8	[0111]	7	[1000]	−8

For our earlier example, we know that the complement of `0xf` is `0x0` and the complement of `0xa` is `0x5`, and so `0xfffffffa` is the two's-complement representation of −6.

A second way to perform two's-complement negation of a number `x` is based on splitting the bit vector into two parts. Let `k` be the position of the rightmost 1, so the bit-level representation of `x` has the form $[x_{w-1}, x_{w-2}, \ldots, x_{k+1}, 1, 0, \ldots 0]$. (This is possible as long as $x = 0$.) The negation is then written in binary form as $[\sim x_{w-1}, \sim x_{w-2}, \ldots \sim x_{k+1}, 1, 0, \ldots, 0]$. That is, we complement each bit to the left of bit position `k`.

We illustrate this idea with some 4-bit numbers, where we highlight the rightmost pattern $1, 0, \ldots, 0$ in italics:

x		−x	
[1100]	−4	[0100]	4
[1000]	−8	[1000]	−8
[0101]	5	[1011]	−5
[0111]	7	[1001]	−7

2.3.4 Unsigned Multiplication

Integers `x` and `y` in the range $0 \le x, y \le 2^w - 1$ can be represented as `w`-bit unsigned numbers, but their product $x \cdot y$ can range between 0 and $(2^w - 1)^2 = 2^{2w} - 2^{w+1} + 1$. This could require as many as `2w` bits to represent. Instead, unsigned multiplication in C is defined to yield the `w`-bit value given by the low-order `w` bits of the `2w`-bit integer product. Let us denote this value as $x *^u_w y$.

Truncating an unsigned number to `w` bits is equivalent to computing its value modulo 2^w, giving the following:

principle: Unsigned multiplication

For x and y such that $0 \le x, y \le \mathrm{UMax}_w$:

$$x *^u_w y = (x \cdot y) \bmod 2^w \tag{2.16}$$

2.3.5 Two's-Complement Multiplication

Integers x and y in the range $-2^{w-1} \le x, y \le 2^{w-1} - 1$ can be represented as w-bit two's-complement numbers, but their product $x \cdot y$ can range between $-2^{w-1} \cdot (2^{w-1} - 1) = -2^{2w-2} + 2^{w-1}$ and $-2^{w-1} \cdot -2^{w-1} = 2^{2w-2}$. This could require as many as $2w$ bits to represent in two's-complement form. Instead, signed multiplication in C generally is performed by truncating the $2w$-bit product to w bits. We denote this value as $x *^t_w y$. Truncating a two's-complement number to w bits is equivalent to first computing its value modulo 2^w and then converting from unsigned to two's complement, giving the following:

principle: Two's-complement multiplication

For x and y such that $\mathrm{TMin}_w \le x, y \le \mathrm{TMax}_w$:

$$x *^t_w y = \mathrm{U2T}_w((x \cdot y) \bmod 2^w) \tag{2.17}$$

We claim that the bit-level representation of the product operation is identical for both unsigned and two's-complement multiplication, as stated by the following principle:

principle: Bit-level equivalence of unsigned and two's-complement multiplication

Let x and y be bit vectors of length w. Define integers x and y as the values represented by these bits in two's-complement form: $x = \mathrm{B2T}_w(x)$ and $y = \mathrm{B2T}_w(y)$. Define nonnegative integers x' and y' as the values represented by these bits in unsigned form: $x' = \mathrm{B2U}_w(x)$ and $y' = \mathrm{B2U}_w(y)$. Then

$$\mathrm{T2B}_w(x *^t_w y) = \mathrm{U2B}_w(x' *^u_w y')$$

As illustrations, Figure 2.27 shows the results of multiplying different 3-bit numbers. For each pair of bit-level operands, we perform both unsigned and two's-complement multiplication, yielding 6-bit products, and then truncate these to 3 bits. The unsigned truncated product always equals $x \cdot y \bmod 8$. The bit-level representations of both truncated products are identical for both unsigned and two's-complement multiplication, even though the full 6-bit representations differ.

Mode	x		y		x · y		Truncated x · y	
Unsigned	5	[101]	3	[011]	15	[001111]	7	[111]
Two's complement	−3	[101]	3	[011]	−9	[110111]	−1	[111]
Unsigned	4	[100]	7	[111]	28	[011100]	4	[100]
Two's complement	−4	[100]	−1	[111]	4	[000100]	−4	[100]
Unsigned	3	[011]	3	[011]	9	[001001]	1	[001]
Two's complement	3	[011]	3	[011]	9	[001001]	1	[001]

Figure 2.27 Three-bit unsigned and two's-complement multiplication examples. Although the bit-level representations of the full products differ, the bit patterns of the truncated products are identical.

derivation: Bit-level equivalence of unsigned and two's-complement multiplication

From Equation 2.6, we have $x = x + x_{w-1}2^w$ and $y = y + y_{w-1}2^w$. Computing the product of these values modulo 2^w gives the following:

$$\begin{aligned}(x \cdot y) \bmod 2^w &= [(x + x_{w-1}2^w) \cdot (y + y_{w-1}2^w)] \bmod 2^w \\ &= [x \cdot y + (x_{w-1}y + y_{w-1}x)2^w + x_{w-1}y_{w-1}2^{2w}] \bmod 2^w \\ &= (x \cdot y) \bmod 2^w\end{aligned} \tag{2.18}$$

The terms with weight 2^w and 2^{2w} drop out due to the modulus operator. By Equation 2.17, we have $x *^t_w y = U2T_w((x \cdot y) \bmod 2^w)$. We can apply the operation $T2U_w$ to both sides to get

$$T2U_w(x *^t_w y) = T2U_w(U2T_w((x \cdot y) \bmod 2^w)) = (x \cdot y) \bmod 2^w$$

Combining this result with Equations 2.16 and 2.18 shows that $T2U_w(x *^t_w y) = (x \cdot y) \bmod 2^w = x *^u_w y$. We can then apply $U2B_w$ to both sides to get

$$U2B_w(T2U_w(x *^t_w y)) = T2B_w(x *^t_w y) = U2B_w(x *^u_w y)$$

■

Practice Problem 2.34 (solution page 189)

Fill in the following table showing the results of multiplying different 3-bit numbers, in the style of Figure 2.27:

Mode	x		y		x · y		Truncated x · y	
Unsigned	_____	[100]	_____	[101]	_____	_____	_____	_____
Two's complement	_____	[100]	_____	[101]	_____	_____	_____	_____
Unsigned	_____	[010]	_____	[111]	_____	_____	_____	_____
Two's complement	_____	[010]	_____	[111]	_____	_____	_____	_____

Mode	x		y		x · y		Truncated x · y	
Unsigned	_____	[110]	_____	[110]	_____	_____	_____	_____
Two's complement	_____	[110]	_____	[110]	_____	_____	_____	_____

Practice Problem 2.35 (solution page 190)

You are given the assignment to develop code for a function `tmult_ok` that will determine whether two arguments can be multiplied without causing overflow. Here is your solution:

```
/* Determine whether arguments can be multiplied without overflow */
int tmult_ok(int x, int y) {
    int p = x*y;
    /* Either x is zero, or dividing p by x gives y */
    return !x || p/x == y;
}
```

You test this code for a number of values of x and y, and it seems to work properly. Your coworker challenges you, saying, "If I can't use subtraction to test whether addition has overflowed (see Problem 2.31), then how can you use division to test whether multiplication has overflowed?"

Devise a mathematical justification of your approach, along the following lines. First, argue that the case $x = 0$ is handled correctly. Otherwise, consider w-bit numbers x ($x = 0$), y, p, and q, where p is the result of performing two's-complement multiplication on x and y, and q is the result of dividing p by x.

1. Show that $x \cdot y$, the integer product of x and y, can be written in the form $x \cdot y = p + t2^w$, where $t = 0$ if and only if the computation of p overflows.
2. Show that p can be written in the form $p = x \cdot q + r$, where $|r| < |x|$.
3. Show that $q = y$ if and only if $r = t = 0$.

Practice Problem 2.36 (solution page 190)

For the case where data type `int` has 32 bits, devise a version of `tmult_ok` (Problem 2.35) that uses the 64-bit precision of data type `int64_t`, without using division.

Practice Problem 2.37 (solution page 191)

You are given the task of patching the vulnerability in the XDR code shown in the aside on page 136 for the case where both data types `int` and `size_t` are 32 bits. You decide to eliminate the possibility of the multiplication overflowing by computing the number of bytes to allocate using data type `uint64_t`. You replace

Aside Security vulnerability in the XDR library

In 2002, it was discovered that code supplied by Sun Microsystems to implement the XDR library, a widely used facility for sharing data structures between programs, had a security vulnerability arising from the fact that multiplication can overflow without any notice being given to the program.

Code similar to that containing the vulnerability is shown below:

```
1  /* Illustration of code vulnerability similar to that found in
2   * Sun's XDR library.
3   */
4  void* copy_elements(void *ele_src[], int ele_cnt, size_t ele_size) {
5      /*
6       * Allocate buffer for ele_cnt objects, each of ele_size bytes
7       * and copy from locations designated by ele_src
8       */
9      void *result = malloc(ele_cnt * ele_size);
10     if (result == NULL)
11         /* malloc failed */
12         return NULL;
13     void *next = result;
14     int i;
15     for (i = 0; i < ele_cnt; i++) {
16         /* Copy object i to destination */
17         memcpy(next, ele_src[i], ele_size);
18         /* Move pointer to next memory region */
19         next += ele_size;
20     }
21     return result;
22 }
```

The function `copy_elements` is designed to copy `ele_cnt` data structures, each consisting of `ele_size` bytes into a buffer allocated by the function on line 9. The number of bytes required is computed as `ele_cnt * ele_size`.

Imagine, however, that a malicious programmer calls this function with `ele_cnt` being 1,048,577 ($2^{20} + 1$) and `ele_size` being 4,096 (2^{12}) with the program compiled for 32 bits. Then the multiplication on line 9 will overflow, causing only 4,096 bytes to be allocated, rather than the 4,294,971,392 bytes required to hold that much data. The loop starting at line 15 will attempt to copy all of those bytes, overrunning the end of the allocated buffer, and therefore corrupting other data structures. This could cause the program to crash or otherwise misbehave.

The Sun code was used by almost every operating system and in such widely used programs as Internet Explorer and the Kerberos authentication system. The Computer Emergency Response Team (CERT), an organization run by the Carnegie Mellon Software Engineering Institute to track security vulnerabilities and breaches, issued advisory "CA-2002-25," and many companies rushed to patch their code. Fortunately, there were no reported security breaches caused by this vulnerability.

A similar vulnerability existed in many implementations of the library function `calloc`. These have since been patched. Unfortunately, many programmers call allocation functions, such as `malloc`, using arithmetic expressions as arguments, without checking these expressions for overflow. Writing a reliable version of `calloc` is left as an exercise (Problem 2.76).

the original call to `malloc` (line 9) as follows:

```
uint64_t asize =
    ele_cnt * (uint64_t) ele_size;
void *result = malloc(asize);
```

Recall that the argument to `malloc` has type `size_t`.

A. Does your code provide any improvement over the original?

B. How would you change the code to eliminate the vulnerability?

2.3.6 Multiplying by Constants

Historically, the integer multiply instruction on many machines was fairly slow, requiring 10 or more clock cycles, whereas other integer operations—such as addition, subtraction, bit-level operations, and shifting—required only 1 clock cycle. Even on the Intel Core i7 Haswell we use as our reference machine, integer multiply requires 3 clock cycles. As a consequence, one important optimization used by compilers is to attempt to replace multiplications by constant factors with combinations of shift and addition operations. We will first consider the case of multiplying by a power of 2, and then we will generalize this to arbitrary constants.

principle: Multiplication by a power of 2

Let x be the unsigned integer represented by bit pattern $[x_{w-1}, x_{w-2}, \ldots, x_0]$. Then for any $k \geq 0$, the $w + k$-bit unsigned representation of $x2^k$ is given by $[x_{w-1}, x_{w-2}, \ldots, x_0, 0, \ldots, 0]$, where k zeros have been added to the right. ■

So, for example, 11 can be represented for $w = 4$ as [1011]. Shifting this left by $k = 2$ yields the 6-bit vector [101100], which encodes the unsigned number $11 \cdot 4 = 44$.

derivation: Multiplication by a power of 2

This property can be derived using Equation 2.1:

$$\begin{aligned} B2U_{w+k}([x_{w-1}, x_{w-2}, \ldots, x_0, 0, \ldots, 0]) &= \sum_{i=0}^{w-1} x_i 2^{i+k} \\ &= \left(\sum_{i=0}^{w-1} x_i 2^i\right) \cdot 2^k \\ &= x2^k \end{aligned}$$

■

When shifting left by k for a fixed word size, the high-order k bits are discarded, yielding

$$[x_{w-k-1}, x_{w-k-2}, \ldots, x_0, 0, \ldots, 0]$$

but this is also the case when performing multiplication on fixed-size words. We can therefore see that shifting a value left is equivalent to performing unsigned multiplication by a power of 2:

principle: Unsigned multiplication by a power of 2

For C variables x and k with unsigned values x and k, such that $0 \le k < w$, the C expression x << k yields the value $x *^u_w 2^k$. ■

Since the bit-level operation of fixed-size two's-complement arithmetic is equivalent to that for unsigned arithmetic, we can make a similar statement about the relationship between left shifts and multiplication by a power of 2 for two's-complement arithmetic:

principle: Two's-complement multiplication by a power of 2

For C variables x and k with two's-complement value x and unsigned value k, such that $0 \le k < w$, the C expression x << k yields the value $x *^t_w 2^k$. ■

Note that multiplying by a power of 2 can cause overflow with either unsigned or two's-complement arithmetic. Our result shows that even then we will get the same effect by shifting. Returning to our earlier example, we shifted the 4-bit pattern [1011] (numeric value 11) left by two positions to get [101100] (numeric value 44). Truncating this to 4 bits gives [1100] (numeric value 12 = 44 mod 16).

Given that integer multiplication is more costly than shifting and adding, many C compilers try to remove many cases where an integer is being multiplied by a constant with combinations of shifting, adding, and subtracting. For example, suppose a program contains the expression x*14. Recognizing that $14 = 2^3 + 2^2 + 2^1$, the compiler can rewrite the multiplication as (x<<3) + (x<<2) + (x<<1), replacing one multiplication with three shifts and two additions. The two computations will yield the same result, regardless of whether x is unsigned or two's complement, and even if the multiplication would cause an overflow. Even better, the compiler can also use the property $14 = 2^4 - 2^1$ to rewrite the multiplication as (x<<4) - (x<<1), requiring only two shifts and a subtraction.

Practice Problem 2.38 (solution page 191)

As we will see in Chapter 3, the LEA instruction can perform computations of the form (a<<k) + b, where k is either 0, 1, 2, or 3, and b is either 0 or some program value. The compiler often uses this instruction to perform multiplications by constant factors. For example, we can compute 3*a as (a<<1) + a.

Considering cases where b is either 0 or equal to a, and all possible values of k, what multiples of a can be computed with a single LEA instruction?

Generalizing from our example, consider the task of generating code for the expression x * K, for some constant K. The compiler can express the binary representation of K as an alternating sequence of zeros and ones:

$$[(0\ldots0)\,(1\ldots1)\,(0\ldots0)\cdots(1\ldots1)]$$

For example, 14 can be written as [(0 . . . 0)(111)(0)]. Consider a run of ones from bit position n down to bit position m ($n \geq m$). (For the case of 14, we have $n = 3$ and $m = 1$.) We can compute the effect of these bits on the product using either of two different forms:

Form A: `(x<<n) + (x<<(n – 1)) + · · · + (x<<m)`

Form B: `(x<<(n + 1)) - (x<<m)`

By adding together the results for each run, we are able to compute `x * K` without any multiplications. Of course, the trade-off between using combinations of shifting, adding, and subtracting versus a single multiplication instruction depends on the relative speeds of these instructions, and these can be highly machine dependent. Most compilers only perform this optimization when a small number of shifts, adds, and subtractions suffice.

Practice Problem 2.39 (solution page 192)

How could we modify the expression for form B for the case where bit position n is the most significant bit?

Practice Problem 2.40 (solution page 192)

For each of the following values of K, find ways to express `x * K` using only the specified number of operations, where we consider both additions and subtractions to have comparable cost. You may need to use some tricks beyond the simple form A and B rules we have considered so far.

K	Shifts	Add/Subs	Expression
7	1	1	
30	4	3	
28	2	1	
55	2	2	

Practice Problem 2.41 (solution page 192)

For a run of ones starting at bit position n down to bit position m ($n \geq m$), we saw that we can generate two forms of code, A and B. How should the compiler decide which form to use?

2.3.7 Dividing by Powers of 2

Integer division on most machines is even slower than integer multiplication—requiring 30 or more clock cycles. Dividing by a power of 2 can also be performed

k	>> k (binary)	Decimal	$12{,}340/2^k$
0	0011000000110100	12,340	12,340.0
1	0001100000011010	6,170	6,170.0
4	0000001100000011	771	771.25
8	0000000000110000	48	48.203125

Figure 2.28 Dividing unsigned numbers by powers of 2. T a l ill trat r o r i a l o ial ri t it k a t a ta \ i i 2^k a \ t r o \ i t o ar r o.

using shift operations, but we use a right shift rather than a left shift. The two different right shifts—logical and arithmetic—serve this purpose for unsigned and two's-complement numbers, respectively.

Integer division always rounds toward zero. To define this precisely, let us introduce some notation. For any real number a, define $\lfloor a \rfloor$ to be the unique integer a such that $a \leq a < a + 1$. As examples, $\lfloor 3.14 \rfloor = 3$, $\lfloor -3.14 \rfloor = -4$, and $\lfloor 3 \rfloor = 3$. Similarly, define $\lceil a \rceil$ to be the unique integer a such that $a - 1 < a \leq a$. As examples, $\lceil 3.14 \rceil = 4$, $\lceil -3.14 \rceil = -3$, and $\lceil 3 \rceil = 3$. For $x \geq 0$ and $y > 0$, integer division should yield $\lfloor x/y \rfloor$, while for $x < 0$ and $y > 0$, it should yield $\lceil x/y \rceil$. That is, it should round down a positive result but round up a negative one.

The case for using shifts with unsigned arithmetic is straightforward, in part because right shifting is guaranteed to be performed logically for unsigned values.

principle: Unsigned division by a power of 2

For C variables x and k with unsigned values x and k, such that $0 \leq k < w$, the C expression $x >> k$ yields the value $\lfloor x/2^k \rfloor$. ■

As examples, Figure 2.28 shows the effects of performing logical right shifts on a 16-bit representation of 12,340 to perform division by 1, 2, 16, and 256. The zeros shifted in from the left are shown in italics. We also show the result we would obtain if we did these divisions with real arithmetic. These examples show that the result of shifting consistently rounds toward zero, as is the convention for integer division.

derivation: Unsigned division by a power of 2

Let x be the unsigned integer represented by bit pattern $[x_{w-1}, x_{w-2}, \ldots, x_0]$, and let k be in the range $0 \leq k < w$. Let x' be the unsigned number with $w - k$-bit representation $[x_{w-1}, x_{w-2}, \ldots, x_k]$, and let x'' be the unsigned number with k-bit representation $[x_{k-1}, \ldots, x_0]$. We can therefore see that $x = 2^k x' + x''$, and that $0 \leq x'' < 2^k$. It therefore follows that $\lfloor x/2^k \rfloor = x'$.

Performing a logical right shift of bit vector $[x_{w-1}, x_{w-2}, \ldots, x_0]$ by k yields the bit vector

$$[0, \ldots, 0, x_{w-1}, x_{w-2}, \ldots, x_k]$$

k	>> k (binary)	Decimal	$-12{,}340/2^k$
0	1100111111001100	−12,340	−12,340.0
1	1110011111100110	−6,170	−6,170.0
4	1111110011111100	−772	−771.25
8	1111111111001111	−49	−48.203125

Figure 2.29 Applying arithmetic right shift. T a l ill trat t at ari t ti ri t it i i ilar t i i a r 2, tt at it r ~ ~ o rat rt a t ari r.

This bit vector has numeric value x , which we have seen is the value that would result by computing the expression x >> k. ■

The case for dividing by a power of 2 with two's-complement arithmetic is slightly more complex. First, the shifting should be performed using an arithmetic right shift, to ensure that negative values remain negative. Let us investigate what value such a right shift would produce.

principle: Two's-complement division by a power of 2, rounding down

Let C variables x and k have two's-complement value x and unsigned value k, respectively, such that $0 \le k < w$. The C expression x >> k, when the shift is performed arithmetically, yields the value $x/2^k$. ■

For $x \ge 0$, variable x has 0 as the most significant bit, and so the effect of an arithmetic shift is the same as for a logical right shift. Thus, an arithmetic right shift by k is the same as division by 2^k for a nonnegative number. As an example of a negative number, Figure 2.29 shows the effect of applying arithmetic right shift to a 16-bit representation of −12,340 for different shift amounts. For the case when no rounding is required ($k = 1$), the result will be $x/2^k$. When rounding is required, shifting causes the result to be rounded downward. For example, the shifting right by four has the effect of rounding −771.25 down to −772. We will need to adjust our strategy to handle division for negative values of x.

derivation: Two's-complement division by a power of 2, rounding down

Let x be the two's-complement integer represented by bit pattern $[x_{w-1}, x_{w-2}, \ldots, x_0]$, and let k be in the range $0 \le k < w$. Let x' be the two's-complement number represented by the $w - k$ bits $[x_{w-1}, x_{w-2}, \ldots, x_k]$, and let x'' be the unsigned number represented by the low-order k bits $[x_{k-1}, \ldots, x_0]$. By a similar analysis as the unsigned case, we have $x = 2^k x' + x''$ and $0 \le x'' < 2^k$, giving $x' = x/2^k$. Furthermore, observe that shifting bit vector $[x_{w-1}, x_{w-2}, \ldots, x_0]$ right arithmetically by k yields the bit vector

$$[x_{w-1}, \ldots, x_{w-1}, x_{w-1}, x_{w-2}, \ldots, x_k]$$

which is the sign extension from $w - k$ bits to w bits of $[x_{w-1}, x_{w-2}, \ldots, x_k]$. Thus, this shifted bit vector is the two's-complement representation of $x/2^k$. ■

k	Bias	−12,340 + bias (binary)	>> k (binary)	Decimal	$-12{,}340/2^k$
0	0	1100111111001100	1100111111001100	−12,340	−12,340.0
1	1	1100111111001101	1110011111100110	−6,170	−6,170.0
4	15	1100111111011011	1111110011111101	−771	−771.25
8	255	1101000011001011	1111111111010000	−48	−48.203125

Figure 2.30 Dividing two's-complement numbers by powers of 2. Bias is added before right shift, causing proper rounding.

We can correct for the improper rounding that occurs when a negative number is shifted right by "biasing" the value before shifting.

principle: Two's-complement division by a power of 2, rounding up

Let C variables x and k have two's-complement value x and unsigned value k, respectively, such that $0 \le k < w$. The C expression `(x + (1 << k) - 1) >> k`, when the shift is performed arithmetically, yields the value $x/2^k$. ■

Figure 2.30 demonstrates how adding the appropriate bias before performing the arithmetic right shift causes the result to be correctly rounded. In the third column, we show the result of adding the bias value to −12,340, with the lower k bits (those that will be shifted off to the right) shown in italics. We can see that the bits to the left of these may or may not be incremented. For the case where no rounding is required (k = 1), adding the bias only affects bits that are shifted off. For the cases where rounding is required, adding the bias causes the upper bits to be incremented, so that the result will be rounded toward zero.

The biasing technique exploits the property that $x/y = (x + y - 1)/y$ for integers x and y such that $y > 0$. As examples, when $x = -30$ and $y = 4$, we have $x + y - 1 = -27$ and $-30/4 = -7 = -27/4$. When $x = -32$ and $y = 4$, we have $x + y - 1 = -29$ and $-32/4 = -8 = -29/4$.

derivation: Two's-complement division by a power of 2, rounding up

To see that $x/y = (x + y - 1)/y$, suppose that $x = qy + r$, where $0 \le r < y$, giving $(x + y - 1)/y = q + (r + y - 1)/y$, and so $(x + y - 1)/y = q + (r + y - 1)/y$. The latter term will equal 0 when $r = 0$ and 1 when $r > 0$. That is, by adding a bias of $y - 1$ to x and then rounding the division downward, we will get q when y divides x and $q + 1$ otherwise.

Returning to the case where $y = 2^k$, the C expression `x + (1 << k) - 1` yields the value $x + 2^k - 1$. Shifting this right arithmetically by k therefore yields $x/2^k$. ■

These analyses show that for a two's-complement machine using arithmetic right shifts, the C expression

```
(x<0 ? x+(1<<k)-1 : x) >> k
```

will compute the value $x/2^k$.

Practice Problem 2.42 (solution page 192)

Write a function `div16` that returns the value `x/16` for integer argument `x`. Your function should not use division, modulus, multiplication, any conditionals (`if` or `?:`), any comparison operators (e.g., `<`, `>`, or `==`), or any loops. You may assume that data type `int`

limits the range of possible values, and the resulting operations can overflow. We have also seen that the two's-complement representation provides a clever way to represent both negative and positive values, while using the same bit-level implementations as are used to perform unsigned arithmetic—operations such as addition, subtraction, multiplication, and even division have either identical or very similar bit-level behaviors, whether the operands are in unsigned or two's-complement form.

We have seen that some of the conventions in the C language can yield some surprising results, and these can be sources of bugs that are hard to recognize or understand. We have especially seen that the `unsigned` data type, while conceptually straightforward, can lead to behaviors that even experienced programmers do not expect. We have also seen that this data type can arise in unexpected ways—for example, when writing integer constants and when invoking library routines.

Practice Problem 2.44 (solution page 193)

Assume data type `int` is 32 bits long and uses a two's-complement representation for signed values. Right shifts are performed arithmetically for signed values and logically for unsigned values. The variables are declared and initialized as follows:

```
int x = foo();   /* Arbitrary value */
int y = bar();   /* Arbitrary value */

unsigned ux = x;
unsigned uy = y;
```

For each of the following C expressions, either (1) argue that it is true (evaluates to 1) for all values of `x` and `y`, or (2) give values of `x` and `y` for which it is false (evaluates to 0):

A. `(x > 0) || (x-1 < 0)`

B. `(x & 7) != 7 || (x<<29 < 0)`

C. `(x * x) >= 0`

D. `x < 0 || -x <= 0`

E. `x > 0 || -x >= 0`

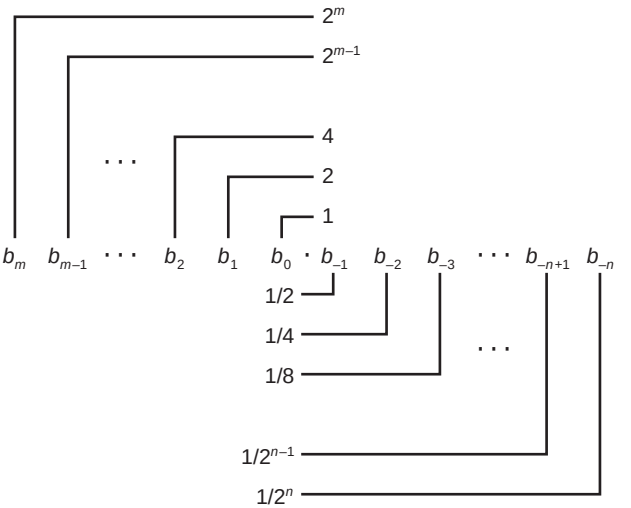
F. `x+y == uy+ux`

G. `x*~y + uy*ux == -x`

2.4 Floating Point

A floating-point representation encodes rational numbers of the form $V = x \times 2^y$. It is useful for performing computations involving very large numbers ($|V|$ 0),

Figure 2.31
Fractional binary representation. Digit to the left of the point are weighted by nonnegative powers of 2, while those to the right are weighted by negative powers of 2.



where each decimal digit d_i ranges between 0 and 9. This notation represents a value d defined as

$$d = \sum_{i=-n}^{m} 10^i \times d_i$$

The weighting of the digits is defined relative to the decimal point symbol ('.'), meaning that digits to the left are weighted by nonnegative powers of 10, giving integral values, while digits to the right are weighted by negative powers of 10, giving fractional values. For example, 12.34_{10} represents the number $1 \times 10^1 + 2 \times 10^0 + 3 \times 10^{-1} + 4 \times 10^{-2} = 12\frac{34}{100}$.

By analogy, consider a notation of the form

$$b_m\, b_{m-1} \cdots b_1\, b_0\, .\, b_{-1}\, b_{-2} \cdots b_{-n+1}\, b_{-n}$$

where each binary digit, or bit, b_i ranges between 0 and 1, as is illustrated in Figure 2.31. This notation represents a number b defined as

$$b = \sum_{i=-n}^{m} 2^i \times b_i \tag{2.19}$$

The symbol '.' now becomes a binary point, with bits on the left being weighted by nonnegative powers of 2, and those on the right being weighted by negative powers of 2. For example, 101.11_2 represents the number $1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 + 1 \times 2^{-1} + 1 \times 2^{-2} = 4 + 0 + 1 + \frac{1}{2} + \frac{1}{4} = 5\frac{3}{4}$.

One can readily see from Equation 2.19 that shifting the binary point one position to the left has the effect of dividing the number by 2. For example, while 101.11_2 represents the number $5\frac{3}{4}$, 10.111_2 represents the number $2 + 0 + \frac{1}{2} +$

$\frac{1}{4} + \frac{1}{8} = 2\frac{7}{8}$. Similarly, shifting the binary point one position to the right has the effect of multiplying the number by 2. For example, 1011.1_2 represents the number $8 + 0 + 2 + 1 + \frac{1}{2} = 11\frac{1}{2}$.

Note that numbers of the form $0.11\cdots1_2$ represent numbers just below 1. For example, 0.111111_2 represents $\frac{63}{64}$. We will use the shorthand notation $1.0 -$ to represent such values.

Assuming we consider only finite-length encodings, decimal notation cannot represent numbers such as $\frac{1}{3}$ and $\frac{5}{7}$ exactly. Similarly, fractional binary notation can only represent numbers that can be written $x \times 2^y$. Other values can only be approximated. For example, the number $\frac{1}{5}$ can be represented exactly as the fractional decimal number 0.20. As a fractional binary number, however, we cannot represent it exactly and instead must approximate it with increasing accuracy by lengthening the binary representation:

Representation	Value	Decimal
0.0_2	$\frac{0}{2}$	0.0_{10}
0.01_2	$\frac{1}{4}$	0.25_{10}
0.010_2	$\frac{2}{8}$	0.25_{10}
0.0011_2	$\frac{3}{16}$	0.1875_{10}
0.00110_2	$\frac{6}{32}$	0.1875_{10}
0.001101_2	$\frac{13}{64}$	0.203125_{10}
0.0011010_2	$\frac{26}{128}$	0.203125_{10}
0.00110011_2	$\frac{51}{256}$	0.19921875_{10}

Practice Problem 2.45 (solution page 193)

Fill in the missing information in the following table:

Fractional value	Binary representation	Decimal representation
$\frac{1}{8}$	0.001	0.125
$\frac{3}{4}$	_____	_____
$\frac{5}{16}$	_____	_____
_____	10.1011	_____
_____	1.001	_____
_____	_____	5.875
_____	_____	3.1875

Practice Problem 2.46 (solution page 194)

The imprecision of floating-point arithmetic can have disastrous effects. On February 25, 1991, during the first Gulf War, an American Patriot Missile battery in Dharan, Saudi Arabia, failed to intercept an incoming Iraqi Scud missile. The Scud struck an American Army barracks and killed 28 soldiers. The US General

Accounting Office (GAO) conducted a detailed analysis of the failure [76] and determined that the underlying cause was an imprecision in a numeric calculation. In this exercise, you will reproduce part of the GAO's analysis.

The Patriot system contains an internal clock, implemented as a counter that is incremented every 0.1 seconds. To determine the time in seconds, the program would multiply the value of this counter by a 24-bit quantity that was a fractional binary approximation to $\frac{1}{10}$. In particular, the binary representation of $\frac{1}{10}$ is the nonterminating sequence $0.000110011[0011]\cdots_2$, where the portion in brackets is repeated indefinitely. The program approximated 0.1, as a value x, by considering just the first 23 bits of the sequence to the right of the binary point: x = 0.00011001100110011001100. (See Problem 2.51 for a discussion of how they could have approximated 0.1 more precisely.)

A. What is the binary representation of 0.1 − x?

B. What is the approximate decimal value of 0.1 − x?

C. The clock starts at 0 when the system is first powered up and keeps counting up from there. In this case, the system had been running for around 100 hours. What was the difference between the actual time and the time computed by the software?

D. The system predicts where an incoming missile will appear based on its velocity and the time of the last radar detection. Given that a Scud travels at around 2,000 meters per second, how far off was its prediction?

Normally, a slight error in the absolute time reported by a clock reading would not affect a tracking computation. Instead, it should depend on the relative time between two successive readings. The problem was that the Patriot software had been upgraded to use a more accurate function for reading time, but not all of the function calls had been replaced by the new code. As a result, the tracking software used the accurate time for one reading and the inaccurate time for the other [103].

2.4.2 IEEE Floating-Point Representation

Positional notation such as considered in the previous section would not be efficient for representing very large numbers. For example, the representation of 5×2^{100} would consist of the bit pattern 101 followed by 100 zeros. Instead, we would like to represent numbers in a form $x \times 2^y$ by giving the values of x and y.

The IEEE floating-point standard represents a number in a form $V = (-1)^s \times M \times 2^E$:

- The sign s determines whether the number is negative (s = 1) or positive (s = 0), where the interpretation of the sign bit for numeric value 0 is handled as a special case.
- The significand M is a fractional binary number that ranges either between 1 and 2 − or between 0 and 1 − .
- The exponent E weights the value by a (possibly negative) power of 2.

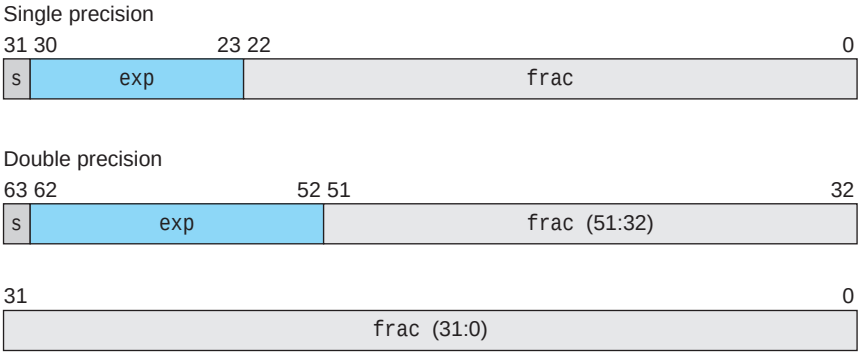


Figure 2.32 Standard floating-point formats. Floating-point numbers are represented using three fields. For the single-precision format, they are encoded as one 32-bit word, while for the double-precision format, they are encoded as one 64-bit word.

The bit representation of a floating-point number is divided into three fields to encode these values:

- The single sign bit s directly encodes the sign s.
- The k-bit exponent field $\mathtt{exp} = e_{k-1} \cdots e_1 e_0$ encodes the exponent E.
- The n-bit fraction field $\mathtt{frac} = f_{n-1} \cdots f_1 f_0$ encodes the significand M, but the value encoded also depends on whether or not the exponent field equals 0.

Figure 2.32 shows the packing of these three fields into words for the two most common formats. In the single-precision floating-point format (a `float` in C), fields `s`, `exp`, and `frac` are 1, k = 8, and n = 23 bits each, yielding a 32-bit representation. In the double-precision floating-point format (a `double` in C), fields `s`, `exp`, and `frac` are 1, k = 11, and n = 52 bits each, yielding a 64-bit representation.

The value encoded by a given bit representation can be divided into three different cases (the latter having two variants), depending on the value of `exp`. These are illustrated in Figure 2.33 for the single-precision format.

Case 1: Normalized Values

This is the most common case. It occurs when the bit pattern of `exp` is neither all zeros (numeric value 0) nor all ones (numeric value 255 for single precision, 2047 for double). In this case, the exponent field is interpreted as representing a signed integer in biased form. That is, the exponent value is E = e − Bias, where e is the unsigned number having bit representation $e_{k-1} \cdots e_1 e_0$ and Bias is a bias value equal to $2^{k-1} - 1$ (127 for single precision and 1023 for double). This yields exponent ranges from −126 to +127 for single precision and −1022 to +1023 for double precision.

The fraction field `frac` is interpreted as representing the fractional value f, where $0 \le f < 1$, having binary representation $0.f_{n-1} \cdots f_1 f_0$, that is, with the

Aside W tt ia t i a r\ r ali\ al ?

Having the exponent value be 1 − Bias rather than simply −Bias might seem counterintuitive. We will see shortly that it provides for smooth transition from denormalized to normalized values.

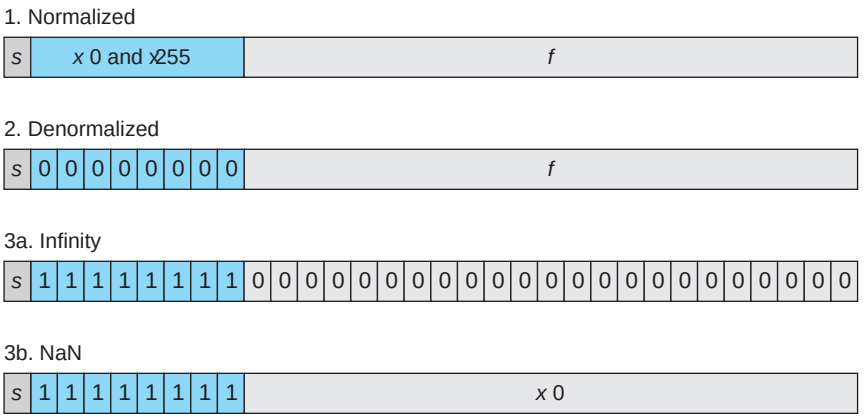


Figure 2.33 Categories of single-precision floating-point values. T al t t\ t r i t rt r i (1) r ali\, (2)\ r ali\, r (3) a al al .

binary point to the left of the most significant bit. The significand is defined to be $M = 1 + f$. This is sometimes called an implied leading 1 representation, because we can view M to be the number with binary representation $1.f_{n-1}f_{n-2}\cdots f_0$. This representation is a trick for getting an additional bit of precision for free, since we can always adjust the exponent E so that significand M is in the range $1 \le M < 2$ (assuming there is no overflow). We therefore do not need to explicitly represent the leading bit, since it always equals 1.

Ca 2: D r ali\ Val

When the exponent field is all zeros, the represented number is in denormalized form. In this case, the exponent value is $E = 1 - \text{Bias}$, and the significand value is $M = f$, that is, the value of the fraction field without an implied leading 1.

Denormalized numbers serve two purposes. First, they provide a way to represent numeric value 0, since with a normalized number we must always have $M \ge 1$, and hence we cannot represent 0. In fact, the floating-point representation of +0.0 has a bit pattern of all zeros: the sign bit is 0, the exponent field is all zeros (indicating a denormalized value), and the fraction field is all zeros, giving $M = f = 0$. Curiously, when the sign bit is 1, but the other fields are all zeros, we get the value −0.0. With IEEE floating-point format, the values −0.0 and +0.0 are considered different in some ways and the same in others.

A second function of denormalized numbers is to represent numbers that are very close to 0.0. They provide a property known as gradual underflow in which possible numeric values are spaced evenly near 0.0.

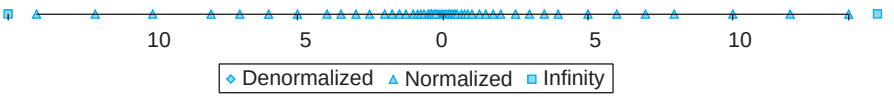
Ca 3: S ial Val

A final category of values occurs when the exponent field is all ones. When the fraction field is all zeros, the resulting values represent infinity, either $+\infty$ when $s = 0$ or $-\infty$ when $s = 1$. Infinity can represent results that overflow, as when we multiply two very large numbers, or when we divide by zero. When the fraction field is nonzero, the resulting value is called a NaN, short for "not a number." Such values are returned as the result of an operation where the result cannot be given as a real number or as infinity, as when computing $\sqrt{-1}$ or $\infty - \infty$. They can also be useful in some applications for representing uninitialized data.

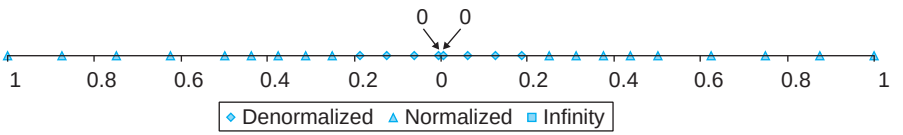
2.4.3 Example Numbers

Figure 2.34 shows the set of values that can be represented in a hypothetical 6-bit format having $k = 3$ exponent bits and $n = 2$ fraction bits. The bias is $2^{3-1} - 1 = 3$. Part (a) of the figure shows all representable values (other than NaN). The two infinities are at the extreme ends. The normalized numbers with maximum magnitude are −14. The denormalized numbers are clustered around 0. These can be seen more clearly in part (b) of the figure, where we show just the numbers between −1.0 and +1.0. The two zeros are special cases of denormalized numbers. Observe that the representable numbers are not uniformly distributed—they are denser nearer the origin.

Figure 2.35 shows some examples for a hypothetical 8-bit floating-point format having $k = 4$ exponent bits and $n = 3$ fraction bits. The bias is $2^{4-1} - 1 = 7$. The figure is divided into three regions representing the three classes of numbers. The different columns show how the exponent field encodes the exponent E, while the fraction field encodes the significand M, and together they form the



(a) Complete range



(b) Values between 1.0 and 1.0

Figure 2.34 Representable values for 6-bit floating-point format. T r ar k = 3 t it a n = 2 ra ti it . T ia i 3.

		Exponent			Fraction		Value		
Description	Bit representation	e	E	2^E	f	M	$2^E \times M$	V	Decimal
Zero	0 0000 000	0	−6	$\frac{1}{64}$	$\frac{0}{8}$	$\frac{0}{8}$	$\frac{0}{512}$	0	0.0
Smallest positive	0 0000 001	0	−6	$\frac{1}{64}$	$\frac{1}{8}$	$\frac{1}{8}$	$\frac{1}{512}$	$\frac{1}{512}$	0.001953
	0 0000 010	0	−6	$\frac{1}{64}$	$\frac{2}{8}$	$\frac{2}{8}$	$\frac{2}{512}$	$\frac{1}{256}$	0.003906
	0 0000 011	0	−6	$\frac{1}{64}$	$\frac{3}{8}$	$\frac{3}{8}$	$\frac{3}{512}$	$\frac{3}{512}$	0.005859
	⋮								
Largest denormalized	0 0000 111	0	−6	$\frac{1}{64}$	$\frac{7}{8}$	$\frac{7}{8}$	$\frac{7}{512}$	$\frac{7}{512}$	0.013672
Smallest normalized	0 0001 000	1	−6	$\frac{1}{64}$	$\frac{0}{8}$	$\frac{8}{8}$	$\frac{8}{512}$	$\frac{1}{64}$	0.015625
	0 0001 001	1	−6	$\frac{1}{64}$	$\frac{1}{8}$	$\frac{9}{8}$	$\frac{9}{512}$	$\frac{9}{512}$	0.017578
	⋮								
	0 0110 110	6	−1	$\frac{1}{2}$	$\frac{6}{8}$	$\frac{14}{8}$	$\frac{14}{16}$	$\frac{7}{8}$	0.875
	0 0110 111	6	−1	$\frac{1}{2}$	$\frac{7}{8}$	$\frac{15}{8}$	$\frac{15}{16}$	$\frac{15}{16}$	0.9375
One	0 0111 000	7	0	1	$\frac{0}{8}$	$\frac{8}{8}$	$\frac{8}{8}$	1	1.0
	0 0111 001	7	0	1	$\frac{1}{8}$	$\frac{9}{8}$	$\frac{9}{8}$	$\frac{9}{8}$	1.125
	0 0111 010	7	0	1	$\frac{2}{8}$	$\frac{10}{8}$	$\frac{10}{8}$	$\frac{5}{4}$	1.25
	⋮								
	0 1110 110	14	7	128	$\frac{6}{8}$	$\frac{14}{8}$	$\frac{1792}{8}$	224	224.0
Largest normalized	0 1110 111	14	7	128	$\frac{7}{8}$	$\frac{15}{8}$	$\frac{1920}{8}$	240	240.0
Infinity	0 1111 000	—	—	—	—	—	—	∞	—

Figure 2.35 Example nonnegative values for 8-bit floating-point format. T r ar k = 4 o t it a n = 3 ra ti it . T ia i 7.

represented value $V = 2^E \times M$. Closest to 0 are the denormalized numbers, starting with 0 itself. Denormalized numbers in this format have $E = 1 - 7 = -6$, giving a weight $2^E = \frac{1}{64}$. The fractions f and significands M range over the values $0, \frac{1}{8}, \ldots, \frac{7}{8}$, giving numbers V in the range 0 to $\frac{1}{64} \times \frac{7}{8} = \frac{7}{512}$.

The smallest normalized numbers in this format also have $E = 1 - 7 = -6$, and the fractions also range over the values $0, \frac{1}{8}, \ldots \frac{7}{8}$. However, the significands then range from $1 + 0 = 1$ to $1 + \frac{7}{8} = \frac{15}{8}$, giving numbers V in the range $\frac{8}{512} = \frac{1}{64}$ to $\frac{15}{512}$.

Observe the smooth transition between the largest denormalized number $\frac{7}{512}$ and the smallest normalized number $\frac{8}{512}$. This smoothness is due to our definition of E for denormalized values. By making it $1 - \text{Bias}$ rather than $-\text{Bias}$, we compensate for the fact that the significand of a denormalized number does not have an implied leading 1.

As we increase the exponent, we get successively larger normalized values, passing through 1.0 and then to the largest normalized number. This number has exponent $E = 7$, giving a weight $2^E = 128$. The fraction equals $\frac{7}{8}$, giving a significand $M = \frac{15}{8}$. Thus, the numeric value is $V = 240$. Going beyond this overflows to $+\infty$.

One interesting property of this representation is that if we interpret the bit representations of the values in Figure 2.35 as unsigned integers, they occur in ascending order, as do the values they represent as floating-point numbers. This is no accident—the IEEE format was designed so that floating-point numbers could be sorted using an integer sorting routine. A minor difficulty occurs when dealing with negative numbers, since they have a leading 1 and occur in descending order, but this can be overcome without requiring floating-point operations to perform comparisons (see Problem 2.84).

Practice Problem 2.47 (solution page 194)

Consider a 5-bit floating-point representation based on the IEEE floating-point format, with one sign bit, two exponent bits ($k = 2$), and two fraction bits ($n = 2$). The exponent bias is $2^{2-1} - 1 = 1$.

The table that follows enumerates the entire nonnegative range for this 5-bit floating-point representation. Fill in the blank table entries using the following directions:

- e: The value represented by considering the exponent field to be an unsigned integer
- E: The value of the exponent after biasing
- 2^E: The numeric weight of the exponent
- f: The value of the fraction
- M: The value of the significand
- $2^E \times M$: The (unreduced) fractional value of the number
- V: The reduced fractional value of the number
- Decimal: The decimal representation of the number

Express the values of 2^E, f, M, $2^E \times M$, and V either as integers (when possible) or as fractions of the form $\frac{x}{y}$, where y is a power of 2. You need not fill in entries marked —.

Bits	e	E	2^E	f	M	$2^E \times M$	V	Decimal
0 00 00								
0 00 01								
0 00 10								
0 00 11								
0 01 00								
0 01 01	1	0	1	$\frac{1}{4}$	$\frac{5}{4}$	$\frac{5}{4}$	$\frac{5}{4}$	1.25

Bits	e	E	2^E	f	M	$2^E \times M$	V	Decimal
0 01 10								
0 01 11								
0 10 00								
0 10 01								
0 10 10								
0 10 11								
0 11 00	—	—	—	—	—	—		—
0 11 01	—	—	—	—	—	—		—
0 11 10	—	—	—	—	—	—		—
0 11 11	—	—	—	—	—	—		—

Figure 2.36 shows the representations and numeric values of some important single- and double-precision floating-point numbers. As with the 8-bit format shown in Figure 2.35, we can see some general properties for a floating-point representation with a k-bit exponent and an n-bit fraction:

- The value +0.0 always has a bit representation of all zeros.
- The smallest positive denormalized value has a bit representation consisting of a 1 in the least significant bit position and otherwise all zeros. It has a fraction (and significand) value $M = f = 2^{-n}$ and an exponent value $E = -2^{k-1} + 2$. The numeric value is therefore $V = 2^{-n-2^{k-1}+2}$.
- The largest denormalized value has a bit representation consisting of an exponent field of all zeros and a fraction field of all ones. It has a fraction (and significand) value $M = f = 1 - 2^{-n}$ (which we have written $1 - $) and an exponent value $E = -2^{k-1} + 2$. The numeric value is therefore $V = (1 - 2^{-n}) \times 2^{-2^{k-1}+2}$, which is just slightly smaller than the smallest normalized value.
- The smallest positive normalized value has a bit representation with a 1 in the least significant bit of the exponent field and otherwise all zeros. It has a

			Single precision		Double precision	
Description	exp	frac	Value	Decimal	Value	Decimal
Zero	$00\cdots00$	$0\cdots00$	0	0.0	0	0.0
Smallest denormalized	$00\cdots00$	$0\cdots01$	$2^{-23} \times 2^{-126}$	1.4×10^{-45}	$2^{-52} \times 2^{-1022}$	4.9×10^{-324}
Largest denormalized	$00\cdots00$	$1\cdots11$	$(1 -) \times 2^{-126}$	1.2×10^{-38}	$(1 -) \times 2^{-1022}$	2.2×10^{-308}
Smallest normalized	$00\cdots01$	$0\cdots00$	1×2^{-126}	1.2×10^{-38}	1×2^{-1022}	2.2×10^{-308}
One	$01\cdots11$	$0\cdots00$	1×2^0	1.0	1×2^0	1.0
Largest normalized	$11\cdots10$	$1\cdots11$	$(2 -) \times 2^{127}$	3.4×10^{38}	$(2 -) \times 2^{1023}$	1.8×10^{308}

Figure 2.36 Examples of nonnegative floating-point numbers.

significand value $M = 1$ and an exponent value $E = -2^{k-1} + 2$. The numeric value is therefore $V = 2^{-2^{k-1}+2}$.

- The value 1.0 has a bit representation with all but the most significant bit of the exponent field equal to 1 and all other bits equal to 0. Its significand value is $M = 1$ and its exponent value is $E = 0$.
- The largest normalized value has a bit representation with a sign bit of 0, the least significant bit of the exponent equal to 0, and all other bits equal to 1. It has a fraction value of $f = 1 - 2^{-n}$, giving a significand $M = 2 - 2^{-n}$ (which we have written $2 -$.) It has an exponent value $E = 2^{k-1} - 1$, giving a numeric value $V = (2 - 2^{-n}) \times 2^{2^{k-1}-1} = (1 - 2^{-n-1}) \times 2^{2^{k-1}}$.

One useful exercise for understanding floating-point representations is to convert sample integer values into floating-point form. For example, we saw in Figure 2.15 that 12,345 has binary representation [11000000111001]. We create a normalized representation of this by shifting 13 positions to the right of a binary point, giving $12{,}345 = 1.1000000111001_2 \times 2^{13}$. To encode this in IEEE single-precision format, we construct the fraction field by dropping the leading 1 and adding 10 zeros to the end, giving binary representation [10000001110010000000000]. To construct the exponent field, we add bias 127 to 13, giving 140, which has binary representation [10001100]. We combine this with a sign bit of 0 to get the floating-point representation in binary of [01000110010000001110010000000000]. Recall from Section 2.1.3 that we observed the following correlation in the bit-level representations of the integer value `12345` (`0x3039`) and the single-precision floating-point value `12345.0` (`0x4640E400`):

```
   0   0   0   0   3   0   3   9
00000000000000000011000000111001
                 *************
             4   6   4   0   E   4   0   0
01000110010000001110010000000000
```

We can now see that the region of correlation corresponds to the low-order bits of the integer, stopping just before the most significant bit equal to 1 (this bit forms the implied leading 1), matching the high-order bits in the fraction part of the floating-point representation.

Practice Problem 2.48 (solution page 195)

As mentioned in Problem 2.6, the integer 3,510,593 has hexadecimal representation `0x00359141`, while the single-precision floating-point number 3,510,593.0 has hexadecimal representation `0x4A564504`. Derive this floating-point representation and explain the correlation between the bits of the integer and floating-point representations.

Practice Problem 2.49 (solution page 195)

A. For a floating-point format with an n-bit fraction, give a formula for the smallest positive integer that cannot be represented exactly (because it would require an $(n + 1)$-bit fraction to be exact). Assume the exponent field size k is large enough that the range of representable exponents does not provide a limitation for this problem.

B. What is the numeric value of this integer for single-precision format ($n = 23$)?

2.4.4 Rounding

Floating-point arithmetic can only approximate real arithmetic, since the representation has limited range and precision. Thus, for a value x, we generally want a systematic method of finding the "closest" matching value x that can be represented in the desired floating-point format. This is the task of the rounding operation. One key problem is to define the direction to round a value that is halfway between two possibilities. For example, if I have \$1.50 and want to round it to the nearest dollar, should the result be \$1 or \$2? An alternative approach is to maintain a lower and an upper bound on the actual number. For example, we could determine representable values x^- and x^+ such that the value x is guaranteed to lie between them: $x^- \leq x \leq x^+$. The IEEE floating-point format defines four different rounding modes. The default method finds a closest match, while the other three can be used for computing upper and lower bounds.

Figure 2.37 illustrates the four rounding modes applied to the problem of rounding a monetary amount to the nearest whole dollar. Round-to-even (also called round-to-nearest) is the default mode. It attempts to find a closest match. Thus, it rounds \$1.40 to \$1 and \$1.60 to \$2, since these are the closest whole dollar values. The only design decision is to determine the effect of rounding values that are halfway between two possible results. Round-to-even mode adopts the convention that it rounds the number either upward or downward such that the least significant digit of the result is even. Thus, it rounds both \$1.50 and \$2.50 to \$2.

The other three modes produce guaranteed bounds on the actual value. These can be useful in some numerical applications. Round-toward-zero mode rounds positive numbers downward and negative numbers upward, giving a value $\hat{x}$ such

Mode	\$1.40	\$1.60	\$1.50	\$2.50	\$–1.50
Round-to-even	\$1	\$2	\$2	\$2	\$–2
Round-toward-zero	\$1	\$1	\$1	\$2	\$–1
Round-down	\$1	\$1	\$1	\$2	\$–2
Round-up	\$2	\$2	\$2	\$3	\$–1

Figure 2.37 Illustration of rounding modes for dollar rounding. T fir t r o ' t o a ar t al , i t o t rt r o ' t r lt a o o r l o .

that $|\hat{x}| \leq |x|$. Round-down mode rounds both positive and negative numbers downward, giving a value x^- such that $x^- \leq x$. Round-up mode rounds both positive and negative numbers upward, giving a value x^+ such that $x \leq x^+$.

Round-to-even at first seems like it has a rather arbitrary goal—why is there any reason to prefer even numbers? Why not consistently round values halfway between two representable values upward? The problem with such a convention is that one can easily imagine scenarios in which rounding a set of data values would then introduce a statistical bias into the computation of an average of the values. The average of a set of numbers that we rounded by this means would be slightly higher than the average of the numbers themselves. Conversely, if we always rounded numbers halfway between downward, the average of a set of rounded numbers would be slightly lower than the average of the numbers themselves. Rounding toward even numbers avoids this statistical bias in most real-life situations. It will round upward about 50% of the time and round downward about 50% of the time.

Round-to-even rounding can be applied even when we are not rounding to a whole number. We simply consider whether the least significant digit is even or odd. For example, suppose we want to round decimal numbers to the nearest hundredth. We would round 1.2349999 to 1.23 and 1.2350001 to 1.24, regardless of rounding mode, since they are not halfway between 1.23 and 1.24. On the other hand, we would round both 1.2350000 and 1.2450000 to 1.24, since 4 is even.

Similarly, round-to-even rounding can be applied to binary fractional numbers. We consider least significant bit value 0 to be even and 1 to be odd. In general, the rounding mode is only significant when we have a bit pattern of the form $XX\cdots X.YY\cdots Y100\cdots$, where X and Y denote arbitrary bit values with the rightmost Y being the position to which we wish to round. Only bit patterns of this form denote values that are halfway between two possible results. As examples, consider the problem of rounding values to the nearest quarter (i.e., 2 bits to the right of the binary point.) We would round 10.00011_2 ($2\frac{3}{32}$) down to 10.00_2 (2), and 10.00110_2 ($2\frac{3}{16}$) up to 10.01_2 ($2\frac{1}{4}$), because these values are not halfway between two possible values. We would round 10.11100_2 ($2\frac{7}{8}$) up to 11.00_2 (3) and 10.10100_2 ($2\frac{5}{8}$) down to 10.10_2 ($2\frac{1}{2}$), since these values are halfway between two possible results, and we prefer to have the least significant bit equal to zero.

Practice Problem 2.50 (solution page 195)

Show how the following binary fractional values would be rounded to the nearest half (1 bit to the right of the binary point), according to the round-to-even rule. In each case, show the numeric values, both before and after rounding.

A. 10.111_2

B. 11.010_2

C. 11.000_2

D. 10.110_2

Practice Problem 2.51 (solution page 195)

We saw in Problem 2.46 that the Patriot missile software approximated 0.1 as $x = 0.0001100110011001100110_2$. Suppose instead that they had used IEEE round-to-even mode to determine an approximation x to 0.1 with 23 bits to the right of the binary point.

A. What is the binary representation of x?

B. What is the approximate decimal value of $x - 0.1$?

C. How far off would the computed clock have been after 100 hours of operation?

D. How far off would the program's prediction of the position of the Scud missile have been?

Practice Problem 2.52 (solution page 196)

Consider the following two 7-bit floating-point representations based on the IEEE floating-point format. Neither has a sign bit—they can only represent nonnegative numbers.

1. Format A
   - There are $k = 3$ exponent bits. The exponent bias is 3.
   - There are $n = 4$ fraction bits.
2. Format B
   - There are $k = 4$ exponent bits. The exponent bias is 7.
   - There are $n = 3$ fraction bits.

Below, you are given some bit patterns in format A, and your task is to convert them to the closest value in format B. If necessary, you should apply the round-to-even rounding rule. In addition, give the values of numbers given by the format A and format B bit patterns. Give these as whole numbers (e.g., 17) or as fractions (e.g., 17/64).

Format A		Format B	
Bits	Value	Bits	Value
011 0000	1	0111 000	1
101 1110			
010 1001			
110 1111			
000 0001			

2.4.5 Floating-Point Operations

The IEEE standard specifies a simple rule for determining the result of an arithmetic operation such as addition or multiplication. Viewing floating-point values x

and y as real numbers, and some operation defined over real numbers, the computation should yield Round(x y), the result of applying rounding to the exact result of the real operation. In practice, there are clever tricks floating-point unit designers use to avoid performing this exact computation, since the computation need only be sufficiently precise to guarantee a correctly rounded result. When one of the arguments is a special value, such as -0, ∞, or NaN, the standard specifies conventions that attempt to be reasonable. For example, $1/-0$ is defined to yield $-\infty$, while $1/+0$ is defined to yield $+\infty$.

One strength of the IEEE standard's method of specifying the behavior of floating-point operations is that it is independent of any particular hardware or software realization. Thus, we can examine its abstract mathematical properties without considering how it is actually implemented.

We saw earlier that integer addition, both unsigned and two's complement, forms an abelian group. Addition over real numbers also forms an abelian group, but we must consider what effect rounding has on these properties. Let us define $x +^f y$ to be Round(x + y). This operation is defined for all values of x and y, although it may yield infinity even when both x and y are real numbers due to overflow. The operation is commutative, with $x +^f y = y +^f x$ for all values of x and y. On the other hand, the operation is not associative. For example, with single-precision floating point the expression `(3.14+1e10)-1e10` evaluates to `0.0`—the value 3.14 is lost due to rounding. On the other hand, the expression `3.14+(1e10-1e10)` evaluates to `3.14`. As with an abelian group, most values have inverses under floating-point addition, that is, $x +^f -x = 0$. The exceptions are infinities (since $+\infty - \infty =$ NaN), and NaNs, since NaN $+^f x =$ NaN for any x.

The lack of associativity in floating-point addition is the most important group property that is lacking. It has important implications for scientific programmers and compiler writers. For example, suppose a compiler is given the following code fragment:

```
x = a + b + c;
y = b + c + d;
```

The compiler might be tempted to save one floating-point addition by generating the following code:

```
t = b + c;
x = a + t;
y = t + d;
```

However, this computation might yield a different value for x than would the original, since it uses a different association of the addition operations. In most applications, the difference would be so small as to be inconsequential. Unfortunately, compilers have no way of knowing what trade-offs the user is willing to make between efficiency and faithfulness to the exact behavior of the original program. As a result, they tend to be very conservative, avoiding any optimizations that could have even the slightest effect on functionality.

On the other hand, floating-point addition satisfies the following monotonicity property: if $a \geq b$, then $x +^f a \geq x +^f b$ for any values of a, b, and x other than NaN. This property of real (and integer) addition is not obeyed by unsigned or two's-complement addition.

Floating-point multiplication also obeys many of the properties one normally associates with multiplication. Let us define $x *^f y$ to be $\text{Round}(x \times y)$. This operation is closed under multiplication (although possibly yielding infinity or NaN), it is commutative, and it has 1.0 as a multiplicative identity. On the other hand, it is not associative, due to the possibility of overflow or the loss of precision due to rounding. For example, with single-precision floating point, the expression `(1e20*1e20)*1e-20` evaluates to $+\infty$, while `1e20*(1e20*1e-20)` evaluates to `1e20`. In addition, floating-point multiplication does not distribute over addition. For example, with single-precision floating point, the expression `1e20*(1e20-1e20)` evaluates to `0.0`, while `1e20*1e20-1e20*1e20` evaluates to NaN.

On the other hand, floating-point multiplication satisfies the following monotonicity properties for any values of a, b, and c other than NaN:

$$a \geq b \quad \text{and} \quad c \geq 0 \qquad a *^f c \geq b *^f c$$

$$a \geq b \quad \text{and} \quad c \leq 0 \qquad a *^f c \leq b *^f c$$

In addition, we are also guaranteed that $a *^f a \geq 0$, as long as $a = \text{NaN}$. As we saw earlier, none of these monotonicity properties hold for unsigned or two's-complement multiplication.

This lack of associativity and distributivity is of serious concern to scientific programmers and to compiler writers. Even such a seemingly simple task as writing code to determine whether two lines intersect in three-dimensional space can be a major challenge.

2.4.6 Floating Point in C

All versions of C provide two different floating-point data types: `float` and `double`. On machines that support IEEE floating point, these data types correspond to single- and double-precision floating point. In addition, the machines use the round-to-even rounding mode. Unfortunately, since the C standards do not require the machine to use IEEE floating point, there are no standard methods to change the rounding mode or to get special values such as -0, $+\infty$, $-\infty$, or NaN. Most systems provide a combination of include (`.h`) files and procedure libraries to provide access to these features, but the details vary from one system to another. For example, the GNU compiler GCC defines program constants `INFINITY` (for $+\infty$) and `NAN` (for NaN) when the following sequence occurs in the program file:

```
#define _GNU_SOURCE 1
#include <math.h>
```

Practice Problem 2.53 (solution page 196)

Fill in the following macro definitions to generate the double-precision values $+\infty$, $-\infty$, and -0:

```
#define POS_INFINITY
#define NEG_INFINITY
#define NEG_ZERO
```

You cannot use any include files (such as `math.h`), but you can make use of the fact that the largest finite number that can be represented with double precision is around 1.8×10^{308}.

When casting values between `int`, `float`, and `double` formats, the program changes the numeric values and the bit representations as follows (assuming data type `int` is 32 bits):

- From `int` to `float`, the number cannot overflow, but it may be rounded.
- From `int` or `float` to `double`, the exact numeric value can be preserved because `double` has both greater range (i.e., the range of representable values), as well as greater precision (i.e., the number of significant bits).
- From `double` to `float`, the value can overflow to $+\infty$ or $-\infty$, since the range is smaller. Otherwise, it may be rounded, because the precision is smaller.
- From `float` or `double` to `int`, the value will be rounded toward zero. For example, 1.999 will be converted to 1, while -1.999 will be converted to -1. Furthermore, the value may overflow. The C standards do not specify a fixed result for this case. Intel-compatible microprocessors designate the bit pattern $[10\cdots00]$ (TMin_w for word size w) as an integer indefinite value. Any conversion from floating point to integer that cannot assign a reasonable integer approximation yields this value. Thus, the expression `(int) +1e10` yields `-21483648`, generating a negative value from a positive one.

Practice Problem 2.54 (solution page 196)

Assume variables `x`, `f`, and `d` are of type `int`, `float`, and `double`, respectively. Their values are arbitrary, except that neither `f` nor `d` equals $+\infty$, $-\infty$, or `NaN`. For each of the following C expressions, either argue that it will always be true (i.e., evaluate to 1) or give a value for the variables such that it is not true (i.e., evaluates to 0).

A. `x == (int)(double) x`

B. `x == (int)(float) x`

C. `d == (double)(float) d`

D. `f == (float)(double) f`

E. `f == -(-f)`

F. `1.0/2 == 1/2.0`

G. `d*d >= 0.0`

H. `(f+d)-f == d`

2.5 Summary

Computers encode information as bits, generally organized as sequences of bytes. Different encodings are used for representing integers, real numbers, and character strings. Different models of computers use different conventions for encoding numbers and for ordering the bytes within multi-byte data.

The C language is designed to accommodate a wide range of different implementations in terms of word sizes and numeric encodings. Machines with 64-bit word sizes have become increasingly common, replacing the 32-bit machines that dominated the market for around 30 years. Because 64-bit machines can also run programs compiled for 32-bit machines, we have focused on the distinction between 32- and 64-bit programs, rather than machines. The advantage of 64-bit programs is that they can go beyond the 4 GB address limitation of 32-bit programs.

Most machines encode signed numbers using a two's-complement representation and encode floating-point numbers using IEEE Standard 754. Understanding these encodings at the bit level, as well as understanding the mathematical characteristics of the arithmetic operations, is important for writing programs that operate correctly over the full range of numeric values.

When casting between signed and unsigned integers of the same size, most C implementations follow the convention that the underlying bit pattern does not change. On a two's-complement machine, this behavior is characterized by functions $\mathsf{T2U}_w$ and $\mathsf{U2T}_w$, for a `w`-bit value. The implicit casting of C gives results that many programmers do not anticipate, often leading to program bugs.

Due to the finite lengths of the encodings, computer arithmetic has properties quite different from conventional integer and real arithmetic. The finite length can cause numbers to overflow, when they exceed the range of the representation. Floating-point values can also underflow, when they are so close to 0.0 that they are changed to zero.

The finite integer arithmetic implemented by C, as well as most other programming languages, has some peculiar properties compared to true integer arithmetic. For example, the expression `x*x` can evaluate to a negative number due to overflow. Nonetheless, both unsigned and two's-complement arithmetic satisfy many of the other properties of integer arithmetic, including associativity, commutativity, and distributivity. This allows compilers to do many optimizations. For example, in replacing the expression `7*x` by `(x<<3)-x`, we make use of the associative, commutative, and distributive properties, along with the relationship between shifting and multiplying by powers of 2.

We have seen several clever ways to exploit combinations of bit-level operations and arithmetic operations. For example, we saw that with two's-complement arithmetic, `~x+1` is equivalent to `-x`. As another example, suppose we want a bit

Aside Ariane 5: Trouble with floating-point to integer conversion

Converting large floating-point numbers to integers is a common source of programming errors. Such an error had disastrous consequences for the maiden voyage of the Ariane 5 rocket, on June 4, 1996. Just 37 seconds after liftoff, the rocket veered off its flight path, broke up, and exploded. Communication satellites valued at $500 million were on board the rocket.

A later investigation [73, 33] showed that the computer controlling the inertial navigation system had sent invalid data to the computer controlling the engine nozzles. Instead of sending flight control information, it had sent a diagnostic bit pattern indicating that an overflow had occurred during the conversion of a 64-bit floating-point number to a 16-bit signed integer.

The value that overflowed measured the horizontal velocity of the rocket, which could be more than five times higher than that achieved by the earlier Ariane 4 rocket. In the design of the Ariane 4 software, they had carefully analyzed the numeric values and determined that the horizontal velocity would never overflow a 16-bit number. Unfortunately, they simply reused this part of the software in the Ariane 5 without checking the assumptions on which it had been based.

pattern of the form $[0, \ldots, 0, 1, \ldots, 1]$, consisting of $w - k$ zeros followed by k ones. Such bit patterns are useful for masking operations. This pattern can be generated by the C expression `(1<<k)-1`, exploiting the property that the desired bit pattern has numeric value $2^k - 1$. For example, the expression `(1<<8)-1` will generate the bit pattern `0xFF`.

Floating-point representations approximate real numbers by encoding numbers of the form $x \times 2^y$. IEEE Standard 754 provides for several different precisions, with the most common being single (32 bits) and double (64 bits). IEEE floating point also has representations for special values representing plus and minus infinity, as well as not-a-number.

Floating-point arithmetic must be used very carefully, because it has only limited range and precision, and because it does not obey common mathematical properties such as associativity.

Bibliographic Notes

Reference books on C [45, 61] discuss properties of the different data types and operations. Of these two, only Steele and Harbison [45] cover the newer features found in ISO C99. There do not yet seem to be any books that cover the features found in ISO C11. The C standards do not specify details such as precise word sizes or numeric encodings. Such details are intentionally omitted to make it possible to implement C on a wide range of different machines. Several books have been written giving advice to C programmers [59, 74] that warn about problems with overflow, implicit casting to unsigned, and some of the other pitfalls we have covered in this chapter. These books also provide helpful advice on variable naming, coding styles, and code testing. Seacord's book on security issues in C and C++ programs [97] combines information about C programs, how they are compiled and executed, and how vulnerabilities may arise. Books on Java (we

recommend the one coauthored by James Gosling, the creator of the language [5]) describe the data formats and arithmetic operations supported by Java.

Most books on logic design [58, 116] have a section on encodings and arithmetic operations. Such books describe different ways of implementing arithmetic circuits. Overton's book on IEEE floating point [82] provides a detailed description of the format as well as the properties from the perspective of a numerical applications programmer.

Homework Problems

2.55

Compile and run the sample code that uses `show_bytes` (file `show-bytes.c`) on different machines to which you have access. Determine the byte orderings used by these machines.

2.56

Try running the code for `show_bytes` for different sample values.

2.57

Write procedures `show_short`, `show_long`, and `show_double` that print the byte representations of C objects of types `short`, `long`, and `double`, respectively. Try these out on several machines.

2.58

Write a procedure `is_little_endian` that will return 1 when compiled and run on a little-endian machine, and will return 0 when compiled and run on a big-endian machine. This program should run on any machine, regardless of its word size.

2.59

Write a C expression that will yield a word consisting of the least significant byte of `x` and the remaining bytes of `y`. For operands `x = 0x89ABCDEF` and `y = 0x76543210`, this would give `0x765432EF`.

2.60

Suppose we number the bytes in a `w`-bit word from 0 (least significant) to `w/8 − 1` (most significant). Write code for the following C function, which will return an unsigned value in which byte `i` of argument `x` has been replaced by byte `b`:

```
unsigned replace_byte (unsigned x, int i, unsigned char b);
```

Here are some examples showing how the function should work:

```
replace_byte(0x12345678, 2, 0xAB) --> 0x12AB5678
replace_byte(0x12345678, 0, 0xAB) --> 0x123456AB
```

Bit-Level Integer Coding Rules

In several of the following problems, we will artificially restrict what programming constructs you can use to help you gain a better understanding of the bit-level,

logic, and arithmetic operations of C. In answering these problems, your code must follow these rules:

- Assumptions
  - Integers are represented in two's-complement form.
  - Right shifts of signed data are performed arithmetically.
  - Data type `int` is `w` bits long. For some of the problems, you will be given a specific value for `w`, but otherwise your code should work as long as `w` is a multiple of 8. You can use the expression `sizeof(int)<<3` to compute `w`.
- Forbidden
  - Conditionals (`if` or `?:`), loops, switch statements, function calls, and

C. Any bit in the least significant byte of `x` equals 1.

D. Any bit in the most significant byte of `x` equals 0.

Your code should follow the bit-level integer coding rules (page 164), with the additional restriction that you may not use equality (`==`) or inequality (`!=`) tests.

2.62

Write a function `int_shifts_are_arithmetic()` that yields 1 when run on a machine that uses arithmetic right shifts for data type `int` and yields 0 otherwise.

2.65

Write code to implement the following function:

```
/* Return 1 when x contains an odd number of 1s; 0 otherwise.
   Assume w=32 */
int odd_ones(unsigned x);
```

Your function should follow the bit-level integer coding rules (page 164), except that you may assume that data type `int` has `w = 32` bits.

Your code should contain a total of at most 12 arithmetic, bitwise, and logical operations.

2.66

Write code to implement the following function:

```
/*
 * Generate mask indicating leftmost 1 in x.  Assume w=32.
 * For example, 0xFF00 -> 0x8000, and 0x6600 --> 0x4000.
 * If x = 0, then return 0.
 */
int leftmost_one(unsigned x);
```

Your function should follow the bit-level integer coding rules (page 164), except that you may assume that data type `int` has `w = 32` bits.

Your code should contain a total of at most 15 arithmetic, bitwise, and logical operations.

Hint: First transform x into a bit vector of the form [0...0]1[1...1].

A. In what way does our code fail to comply with the C standard?

B. Modify the code to run properly on any machine for which data type `int` is at least 32 bits.

C. Modify the code to run properly on any machine for which data type `int` is at least 16 bits.

2.68
Write code for a function with the following prototype:

```
/*
 * Mask with least signficant n bits set to 1
 * Examples: n = 6 --> 0x3F, n = 17 --> 0x1FFFF
 * Assume 1 <= n <= w
 */
int lower_one_mask(int n);
```

Your function should follow the bit-level integer coding rules (page 164). Be careful of the case `n = w`.

2.69
Write code for a function with the following prototype:

```
/*
 * Do rotating left shift.  Assume 0 <= n < w
 * Examples when x = 0x12345678 and w = 32:
 *    n=4 -> 0x23456781, n=20 -> 0x67812345
 */
unsigned rotate_left(unsigned x, int n);
```

Your function should follow the bit-level integer coding rules (page 164). Be careful of the case `n = 0`.

2.70
Write code for the function with the following prototype:

```
/*
 * Return 1 when x can be represented as an n-bit, 2's-complement
 * number; 0 otherwise
 * Assume 1 <= n <= w
 */
int fits_bits(int x, int n);
```

Your function should follow the bit-level integer coding rules (page 164).

2.71
You just started working for a company that is implementing a set of procedures to operate on a data structure where 4 signed bytes are packed into a 32-bit `unsigned`. Bytes within the word are numbered from 0 (least significant) to 3

(most significant). You have been assigned the task of implementing a function for a machine using two's-complement arithmetic and arithmetic right shifts with the following prototype:

```
/* Declaration of data type where 4 bytes are packed
   into an unsigned */
typedef unsigned packed_t;

/* Extract byte from word.  Return as signed integer */
int xbyte(packed_t word, int bytenum);
```

That is, the function will extract the designated byte and sign extend it to be a 32-bit `int`.

Your predecessor (who was fired for incompetence) wrote the following code:

```
/* Failed attempt at xbyte */
int xbyte(packed_t word, int bytenum)
{
    return (word >> (bytenum << 3)) & 0xFF;
}
```

A. What is wrong with this code?

B. Give a correct implementation of the function that uses only left and right shifts, along with one subtraction.

2.72

You are given the task of writing a function that will copy an integer `val` into a buffer `buf`, but it should do so only if enough space is available in the buffer.

Here is the code you write:

```
/* Copy integer into buffer if space is available */
/* WARNING: The fy025(The)-25(space)-525(buggy25(xbyte)-525(*/)]voidy025(Thepy_er)d,)-52
```

2.73
Write code for a function with the following prototype:

```
/* Addition that saturates to TMin or TMax */
int saturating_add(int x, int y);
```

Instead of overflowing the way normal two's-complement addition does, saturating addition returns TMax when there would be positive overflow, and TMin when there would be negative overflow. Saturating arithmetic is commonly used in programs that perform digital signal processing.

Your function should follow the bit-level integer coding rules (page 164).

2.74
Write a function with the following prototype:

```
/* Determine whether arguments can be subtracted without overflow */
int tsub_ok(int x, int y);
```

This function should return 1 if the computation x-y does not overflow.

2.75
Suppose we want to compute the complete 2w-bit representation of $x \cdot y$, where both x and y are unsigned, on a machine for which data type unsigned is w bits. The low-order w bits of the product can be computed with the expression x*y, so we only require a procedure with prototype

```
unsigned unsigned_high_prod(unsigned x, unsigned y);
```

that computes the high-order w bits of $x \cdot y$ for unsigned variables.

We have access to a library function with prototype

```
int signed_high_prod(int x, int y);
```

that computes the high-order w bits of $x \cdot y$ for the case where x and y are in two's-complement form. Write code calling this procedure to implement the function for unsigned arguments. Justify the correctness of your solution.

Hint: Look at the relationship between the signed product $x \cdot y$ and the unsigned product $x \cdot y$ in the derivation of Equation 2.18.

2.76
The library function calloc has the following declaration:

```
void *calloc(size_t nmemb, size_t size);
```

According to the library documentation, "The calloc function allocates memory for an array of nmemb elements of size bytes each. The memory is set to zero. If nmemb or size is zero, then calloc returns NULL."

Write an implementation of calloc that performs the allocation by a call to malloc and sets the memory to zero via memset. Your code should not have any vulnerabilities due to arithmetic overflow, and it should work correctly regardless of the number of bits used to represent data of type size_t.

As a reference, functions malloc and memset have the following declarations:

```
void *malloc(size_t size);
void *memset(void *s, int c, size_t n);
```

2.77
Suppose we are given the task of generating code to multiply integer variable x by various different constant factors K. To be efficient, we want to use only the operations +, -, and <<. For the following values of K, write C expressions to perform the multiplication using at most three operations per expression.

A. K = 17

B. K = −7

C. K = 60

D. K = −112

2.78
Write code for a function with the following prototype:

```
/* Divide by power of 2. Assume 0 <= k < w-1 */
int divide_power2(int x, int k);
```

The function should compute $x/2^k$ with correct rounding, and it should follow the bit-level integer coding rules (page 164).

2.79
Write code for a function mul3div4 that, for integer argument x, computes 3 x/4 but follows the bit-level integer coding rules (page 164). Your code should replicate the fact that the computation 3*x can cause overflow.

2.80
Write code for a function threefourths that, for integer argument x, computes the value of $\frac{3}{4}$x, rounded toward zero. It should not overflow. Your function should follow the bit-level integer coding rules (page 164).

2.81
Write C expressions to generate the bit patterns that follow, where a^k represents k repetitions of symbol a. Assume a w-bit data type. Your code may contain references to parameters j and k, representing the values of j and k, but not a parameter representing w.

A. $1^{w-k}0^k$

B. $0^{w-k-j}1^k0^j$

2.82
We are running programs where values of type int are 32 bits. They are represented in two's complement, and they are right shifted arithmetically. Values of type unsigned are also 32 bits.

We generate arbitrary values x and y, and convert them to unsigned values as follows:

```
/* Create some arbitrary values */
int x = random();
int y = random();
/* Convert to unsigned */
unsigned ux = (unsigned) x;
unsigned uy = (unsigned) y;
```

For each of the following C expressions, you are to indicate whether or not the expression always yields 1. If it always yields 1, describe the underlying mathematical principles. Otherwise, give an example of arguments that make it yield 0.

A. `(x<y) == (-x>-y)`

B. `((x+y)<<4) + y-x == 17*y+15*x`

C. `~x+~y+1 == ~(x+y)`

D. `(ux-uy) == -(unsigned)(y-x)`

E. `((x >> 2) << 2) <= x`

2.83

Consider numbers having a binary representation consisting of an infinite string of the form $0.\mathtt{y}\,\mathtt{y}\,\mathtt{y}\,\mathtt{y}\,\mathtt{y}\,\mathtt{y}\cdots$, where y is a k-bit sequence. For example, the binary representation of $\frac{1}{3}$ is $0.01010101\cdots$ (y = 01), while the representation of $\frac{1}{5}$ is $0.001100110011\cdots$ (y = 0011).

A. Let Y = $\mathrm{B2U}_k$(y), that is, the number having binary representation y. Give a formula in terms of Y and k for the value represented by the infinite string. Hint: Consider the effect of shifting the binary point k positions to the right.

B. What is the numeric value of the string for the following values of y?
   (a) 101
   (b) 0110
   (c) 010011

2.84

Fill in the return value for the following procedure, which tests whether its first argument is less than or equal to its second. Assume the function f2u returns an unsigned 32-bit number having the same bit representation as its floating-point argument. You can assume that neither argument is NaN. The two flavors of zero, +0 and −0, are considered equal.

```
int float_le(float x, float y) {
    unsigned ux = f2u(x);
    unsigned uy = f2u(y);
```

Hex: The four hexadecimal digits describing the encoded form.

M: The value of the significand. This should be a number of the form x or $\frac{x}{y}$, where x is an integer and y is an integral power of 2. Examples include 0, $\frac{67}{64}$, and $\frac{1}{256}$.

E: The integer value of the exponent.

V: The numeric value represented. Use the notation x or $x \times 2^z$, where x and z are integers.

D: The (possibly approximate) numerical value, as is printed using the `%f` formatting specification of `printf`.

As an example, to represent the number $\frac{7}{8}$, we would have s = 0, M = $\frac{7}{4}$, and E = −1. Our number would therefore have an exponent field of 01110_2 (decimal value 15 − 1 = 14) and a significand field of 1100000000_2, giving a hex representation `3B00`. The numerical value is 0.875.

You need not fill in entries marked —.

Description	Hex	M	E	V	D
−0	_____	_____	_____	−0	−0.0
Smallest value > 2	_____	_____	_____	_____	_____
512		_____	_____	_____512	512.0
Largest denormalized	_____	_____	_____	_____	_____
− ∞	_____	—	—	− ∞	− ∞
Number with hex representation `3BB0`	`3BB0`	_____	_____	_____	_____

2.88

Consider the following two 9-bit floating-point representations based on the IEEE floating-point format.

1. Format A
   - There is 1 sign bit.
   - There are k = 5 exponent bits. The exponent bias is 15.
   - There are n = 3 fraction bits.
2. Format B
   - There is 1 sign bit.
   - There are k = 4 exponent bits. The exponent bias is 7.
   - There are n = 4 fraction bits.

In the following table, you are given some bit patterns in format A, and your task is to convert them to the closest value in format B. If rounding is necessary you should round toward + ∞. In addition, give the values of numbers given by the format A and format B bit patterns. Give these as whole numbers (e.g., 17) or as fractions (e.g., 17/64 or $17/2^6$).

Format A		Format B	
Bits	Value	Bits	Value
`1 01111 001`	$\frac{-9}{8}$	`1 0111 0010`	$\frac{-9}{8}$
`0 10110 011`			

function u2f returns a floating-point value having an identical bit representation as its unsigned argument.

```
float fpwr2(int x)
{
    /* Result exponent and fraction */
    unsigned exp, frac;
    unsigned u;

    if (x < _________) {
        /* Too small.  Return 0.0 */
        exp = _________;
        frac = _________;
    } else if (x < _________) {
        /* Denormalized result */
        exp = _________;
        frac = _________;
    } else if (x < _________) {
        /* Normalized result. */
        exp = _________;
        frac = _________;
    } else {
        /* Too big.  Return +oo */
        exp = _________;
        frac = _________;
    }

    /* Pack exp and frac into 32 bits */
    u = exp << 23 | frac;
    /* Return as float */
    return u2f(u);
}
```

2.91

Around 250 B.C., the Greek mathematician Archimedes proved that $\frac{223}{71} < \pi < \frac{22}{7}$. Had he had access to a computer and the standard library `<math.h>`, he would have been able to determine that the single-precision floating-point approximation of π has the hexadecimal representation `0x40490FDB`. Of course, all of these are just approximations, since π is not rational.

A. What is the fractional binary number denoted by this floating-point value?

B. What is the fractional binary representation of $\frac{22}{7}$? Hint: See Problem 2.83.

C. At what bit position (relative to the binary point) do these two approximations to π diverge?

2.93

Following the bit-level floating-point coding rules, implement the function with the following prototype:

```
/* Compute |f|.  If f is NaN, then return f. */
float_bits float_absval(float_bits f);
```

For floating-point number `f`, this function computes `|f|`. If `f` is NaN, your function should simply return `f`.

Test your function by evaluating it for all 2^{32} values of argument `f` and comparing the result to what would be obtained using your machine's floating-point operations.

2.94

Following the bit-level floating-point coding rules, implement the function with the following prototype:

```
/* Compute 2*f.  If f is NaN, then return f. */
float_bits float_twice(float_bits f);
```

For floating-point number `f`, this function computes $2.0 \cdot$ `f`. If `f` is NaN, your function should simply return `f`.

Test your function by evaluating it for all 2^{32} values of argument `f` and comparing the result to what would be obtained using your machine's floating-point operations.

2.95

Following the bit-level floating-point coding rules, implement the function with the following prototype:

```
/* Compute 0.5*f.  If f is NaN, then return f. */
float_bits float_half(float_bits f);
```

For floating-point number `f`, this function computes $0.5 \cdot$ `f`. If `f` is NaN, your function should simply return `f`.

Test your function by evaluating it for all 2^{32} values of argument `f` and comparing the result to what would be obtained using your machine's floating-point operations.

2.96

Following the bit-level floating-point coding rules, implement the function with the following prototype:

```
/*
 * Compute (int) f.
 * If conversion causes overflow or f is NaN, return 0x80000000
 */
int float_f2i(float_bits f);
```

For floating-point number `f`, this function computes `(int) f`. Your function should round toward zero. If `f` cannot be represented as an integer (e.g., it is out of range, or it is NaN), then the function should return `0x80000000`.

Test your function by evaluating it for all 2^{32} values of argument `f` and comparing the result to what would be obtained using your machine's floating-point operations.

2.97

Following the bit-level floating-point coding rules, implement the function with the following prototype:

```
/* Compute (float) i */
float_bits float_i2f(int i);
```

For argument `i`, this function computes the bit-level representation of `(float) i`.

Test your function by evaluating it for all 2^{32} values of argument `f` and comparing the result to what would be obtained using your machine's floating-point operations.

Solutions to Practice Problems

Solution to Problem 2.1 (page 73)

Understanding the relation between hexadecimal and binary formats will be important once we start looking at machine-level programs. The method for doing these conversions is in the text, but it takes a little practice to become familiar.

A. `0x25B9D2` to binary:

Hexadecimal	2	5	B	9	D	2
Binary	0010	0101	1101	1001	1101	0010

B. Binary 1100 1001 0111 1011 to hexadecimal:

Binary	1100	1001	0111	1011
Hexadecimal	C	9	7	B

C. `0xA8B3D` to binary:

Hexadecimal	A	8	B	3	D
Binary	1010	1000	1011	0011	1101

D. Binary 11 0010 0010 1101 1001 0110 to hexadecimal:

Binary	11	0010	0010	1101	1001	0110
Hexadecimal	3	2	2	D	9	6

Solution to Problem 2.2 (page 73)

This problem gives you a chance to think about powers of 2 and their hexadecimal representations.

n	2^n (decimal)	2^n (hexadecimal)
5	32	0x20
23	8,388,608	0x800000
15	32,768	0x8000
13	8,192	0x2000
12	4,096	0x1000
6	64	0x40
8	256	0x100

Solution to Problem 2.3 (page 74)

This problem gives you a chance to try out conversions between hexadecimal and decimal representations for some smaller numbers. For larger ones, it becomes much more convenient and reliable to use a calculator or conversion program.

Decimal	Binary	Hexadecimal
0	0000 0000	0x00
158 = 16 · 9 + 14	1001 1110	0x9E
76 = 16 · 4 + 12	0100 1100	0x4C
145 = 16 · 9 + 1	1001 0001	0x91
16 · 10 + 14 = 174	1010 1110	0xAE
16 · 3 + 12 = 60	0011 1100	0x3C
16 · 15 + 1 = 241	1111 0001	0xF1
16 · 7 + 5 = 117	0111 0101	0x75
16 · 11 + 13 = 189	1011 1101	0xBD
16 · 15 + 5 = 245	1111 0101	0xF5

Solution to Problem 2.4 (page 75)

When you begin debugging machine-level programs, you will find many cases where some simple hexadecimal arithmetic would be useful. You can always convert numbers to decimal, perform the arithmetic, and convert them back, but being able to work directly in hexadecimal is more efficient and informative.

A. `0x605C + 0x5 = 0x6061`. Adding 5 to hex C gives 1 with a carry of 1.

B. `0x605C − 0x20 = 0x603C`. Subtracting 2 from 5 in the second digit position requires no borrow from the third. This gives 3.

C. `0x605C` + 32 = `0x607C`. Decimal 32 (2^5) equals hexadecimal `0x20`.

D. `0x60FA − 0x605C = 0x9E`. To subtract hex C (decimal 12) from hex A (decimal 10), we borrow 16 from the second digit, giving hex F (decimal 15). In the second digit, we now subtract 5 from hex E (decimal 14), giving decimal 9.

Solution to Problem 2.5 (page 84)

This problem tests your understanding of the byte representation of data and the two different byte orderings.

A.	Little endian: 78	Big endian: 12
B.	Little endian: 78 56	Big endian: 12 34

C. Little endian: 78 56 34 Big endian: 12 34 56

Recall that `show_bytes` enumerates a series of bytes starting from the one with lowest address and working toward the one with highest address. On a little-endian machine, it will list the bytes from least significant to most. On a big-endian machine, it will list bytes from the most significant byte to the least.

Solution to Problem 2.6 (page 85)

This problem is another chance to practice hexadecimal to binary conversion. It also gets you thinking about integer and floating-point representations. We will explore these representations in more detail later in this chapter.

A. Using the notation of the example in the text, we write the two strings as follows:

```
0   0   2   7   C   8   F   8
00000000001001111100100011111000
**********************
4    A    1    F    2      3     E     0
01001010000111111001000111110000
```

B. With the second word shifted two positions to the right relative to the first, we find a sequence with 21 matching bits.

C. We find all bits of the integer embedded in the floating-point number, except for the most significant bit having value 0. Such is the case for the example in the text as well. In addition, the floating-point number has some nonzero high-order bits that do not match those of the integer.

Solution to Problem 2.7 (page 85)

It prints `6D 6E 6F 70 71 72`. Recall also that the library routine `strlen` does not count the terminating null character, and so `show_bytes` printed only through the character '`r`'.

Solution to Problem 2.8 (page 87)

This problem is a drill to help you become more familiar with Boolean operations.

Operation	Result
`a`	[01001110]
`b`	[11100001]
`~a`	[10110001]
`~b`	[00011110]
`a & b`	[01000000]
`a | b`	[11101111]
`a ^ b`	[10101111]

Solution to Problem 2.9 (page 89)

This problem illustrates how Boolean algebra can be used to describe and reason about real-world systems. We can see that this color algebra is identical to the Boolean algebra over bit vectors of length 3.

A. Colors are complemented by complementing the values of R, G, and B. From this, we can see that white is the complement of black, yellow is the complement of blue, magenta is the complement of green, and cyan is the complement of red.

B. We perform Boolean operations based on a bit-vector representation of the colors. From this we get the following:

Blue (001)	|	Green (010)	=	Cyan (011)
Yellow (110)	&	Cyan (011)	=	Green (010)
Red (100)	^	Magenta (101)	=	Blue (001)

Solution to Problem 2.10 (page 90)

This procedure relies on the fact that EXCLUSIVE-OR is commutative and associative, and that a ^ a = 0 for any a.

Step	*x	*y
Initially	a	b
Step 1	a	a ^ b
Step 2	a ^ (a ^ b) = (a ^ a) ^ b = b	a ^ b
Step 3	b	b ^ (a ^ b) = (b ^ b) ^ a = a

See Problem 2.11 for a case where this function will fail.

Solution to Problem 2.11 (page 91)

This problem illustrates a subtle and interesting feature of our inplace swap routine.

A. Both `first` and `last` have value k, so we are attempting to swap the middle element with itself.

B. In this case, arguments x and y to `inplace_swap` both point to the same location. When we compute *x ^ *y, we get 0. We then store 0 as the middle element of the array, and the subsequent steps keep setting this element to 0. We can see that our reasoning in Problem 2.10 implicitly assumed that x and y denote different locations.

C. Simply replace the test in line 4 of `reverse_array` to be `first < last`, since there is no need to swap the middle element with itself.

Solution to Problem 2.12 (page 91)

Here are the expressions:

A. `x & 0xFF`

B. `x ^ ~0xFF`

C. `x | 0xFF`

These expressions are typical of the kind commonly found in performing low-level bit operations. The expression `~0xFF` creates a mask where the 8 least-significant bits equal 0 and the rest equal 1. Observe that such a mask will be generated regardless of the word size. By contrast, the expression `0xFFFFFF00` would only work when data type `int` is 32 bits.

Solution to Problem 2.13 (page 92)

These problems help you think about the relation between Boolean operations and typical ways that programmers apply masking operations. Here is the code:

```
/* Declarations of functions implementing operations bis and bic */
int bis(int x, int m);
int bic(int x, int m);

/* Compute x|y using only calls to functions bis and bic */
int bool_or(int x, int y) {
  int result = bis(x,y);
  return result;
}

/* Compute x^y using only calls to functions bis and bic */
int bool_xor(int x, int y) {
  int result = bis(bic(x,y), bic(y,x));
  return result;
}
```

The `bis` operation is equivalent to Boolean OR—a bit is set in `z` if either this bit is set in `x` or it is set in `m`. On the other hand, `bic(x, m)` is equivalent to `x & ~m`; we want the result to equal 1 only when the corresponding bit of `x` is 1 and of `m` is 0.

Given that, we can implement `|` with a single call to `bis`. To implement `^`, we take advantage of the property

$$x \wedge y = (x \& \sim y) \mid (\sim x \& y)$$

Solution to Problem 2.14 (page 93)

This problem highlights the relation between bit-level Boolean operations and logical operations in C. A common programming error is to use a bit-level operation when a logical one is intended, or vice versa.

Expression	Value	Expression	Value
`a & b`	`0x44`	`a && b`	`0x01`
`a | b`	`0x57`	`a || b`	`0x01`
`~a | ~b`	`0xBB`	`!a || !b`	`0x00`
`a & !b`	`0x00`	`a && ~b`	`0x01`

Solution to Problem 2.15 (page 93)

The expression is `!(x ^ y)`.

That is, `x^y` will be zero if and only if every bit of `x` matches the corresponding bit of `y`. We then exploit the ability of `!` to determine whether a word contains any nonzero bit.

There is no real reason to use this expression rather than simply writing `x == y`, but it demonstrates some of the nuances of bit-level and logical operations.

Solution to Problem 2.16 (page 94)

This problem is a drill to help you understand the different shift operations.

x		a << 2		Logical a >> 3		Arithmetic a >> 3	
Hex	Binary	Binary	Hex	Binary	Hex	Binary	Hex
`0xD4`	[11010100]	[01010000]	`0x50`	[00011010]	`0x1A`	[11111010]	`0xFA`
`0x64`	[01100100]	[10010000]	`0x90`	[00001100]	`0x0C`	[11101100]	`0xEC`
`0x72`	[01110010]	[11001000]	`0xC8`	[00001110]	`0x0E`	[00001110]	`0x0E`
`0x44`	[01000100]	[00010000]	`0x10`	[00001000]	`0x08`	[11101000]	`0xE9`

Solution to Problem 2.17 (page 101)

In general, working through examples for very small word sizes is a very good way to understand computer arithmetic.

The unsigned values correspond to those in Figure 2.2. For the two's-complement values, hex digits `0` through `7` have a most significant bit of 0, yielding nonnegative values, while hex digits `8` through `F` have a most significant bit of 1, yielding a negative value.

Hexadecimal	Binary		
x		$B2U_4(x)$	$B2T_4(x)$
`0xA`	[1010]	$2^3 + 2^1 = 10$	$-2^3 + 2^2 = -6$
`0x1`	[0001]	$2^0 = 1$	$2^0 = 1$
`0xB`	[1011]	$2^3 + 2^1 + 2^0 = 11$	$-2^3 + 2^1 + 2^0 = -5$
`0x2`	[0010]	$2^1 = 2$	$2^1 = 2$
`0x7`	[0111]	$2^2 + 2^1 + 2^0 = 7$	$2^2 + 2^1 + 2^0 = 7$
`0xC`	[1100]	$2^3 + 2^2 = 12$	$-2^3 + 2^2 = -4$

Solution to Problem 2.18 (page 105)

For a 32-bit word, any value consisting of 8 hexadecimal digits beginning with one of the digits 8 through f represents a negative number. It is quite common to see numbers beginning with a string of f's, since the leading bits of a negative number are all ones. You must look carefully, though. For example, the number 0x8048337 has only 7 digits. Filling this out with a leading zero gives 0x08048337, a positive number.

```
4004d0:  48 81 ec e0 02 00 00    sub   $0x2e0,%rsp              A. 736
4004d7:  48 8b 44 24 a8          mov   -0x58(%rsp),%rax         B. -88
4004dc:  48 03 47 28             add   0x28(%rdi),%rax          C.  40
4004e0:  48 89 44 24 d0          mov   %rax,-0x30(%rsp)         D. -48
4004e5:  48 8b 44 24 78          mov   0x78(%rsp),%rax          E. 120
4004ea:  48 89 87 88 00 00 00    mov   %rax,0x88(%rdi)          F. 136
4004f1:  48 8b 84 24 f8 01 00    mov   0x1f8(%rsp),%rax         G. 504
4004f8:  00
4004f9:  48 03 44 24 08          add   0x8(%rsp),%rax
4004fe:  48 89 84 24 c0 00 00    mov   %rax,0xc0(%rsp)          H. 192
400505:  00
400506:  48 8b 44 d4 b8          mov   -0x48(%rsp,%rdx,8),%rax  I. -72
```

Solution to Problem 2.19 (page 107)

The functions T2U and U2T are very peculiar from a mathematical perspective. It is important to understand how they behave.

We solve this problem by reordering the rows in the solution of Problem 2.17 according to the two's-complement value and then listing the unsigned value as the result of the function application. We show the hexadecimal values to make this process more concrete.

x (hex)	x	$T2U_4$(vecx)
0xF	−1	15
0xB	−5	11
0xA	−6	10
0xC	−4	12
0x1	1	1
0x8	8	8

Solution to Problem 2.20 (page 109)

This exercise tests your understanding of Equation 2.5.

For the first four entries, the values of x are negative and $T2U_4(x) = x + 2^4$. For the remaining two entries, the values of x are nonnegative and $T2U_4(x) = x$.

Solution to Problem 2.21 (page 112)

This problem reinforces your understanding of the relation between two's-complement and unsigned representations, as well as the effects of the C promotion rules. Recall that $TMin_{32}$ is −2,147,483,648, and that when cast to unsigned it

becomes 2,147,483,648. In addition, if either operand is unsigned, then the other operand will be cast to unsigned before comparing.

Expression	Type	Evaluation
`-2147483647-1 == 2147483648U`	Unsigned	1
`-2147483647-1 < 2147483647`	Signed	1
`-2147483647-1U < 2147483647`	Unsigned	0
`-2147483647-1 < -2147483647`	Signed	1
`-2147483647-1U < -2147483647`	Unsigned	1

Solution to Problem 2.22 (page 115)

This exercise provides a concrete demonstration of how sign extension preserves the numeric value of a two's-complement representation.

A.	[1100]	$-2^3 + 2^2$	=	$-8 + 4$	=	-4
B.	[11100]	$-2^4 + 2^3 + 2^2$	=	$-16 + 8 + 4$	=	-4
C.	[111100]	$-2^5 + 2^4 + 2^3 + 2^2$	=	$-32 + 16 + 8 + 4$	=	-4

Solution to Problem 2.23 (page 116)

The expressions in these functions are common program "idioms" for extracting values from a word in which multiple bit fields have been packed. They exploit the zero-filling and sign-extending properties of the different shift operations. Note carefully the ordering of the cast and shift operations. In `fun1`, the shifts are performed on unsigned variable `word` and hence are logical. In `fun2`, shifts are performed after casting `word` to `int` and hence are arithmetic.

A.

`w`	`fun1(w)`	`fun2(w)`
`0x00000076`	`0x00000076`	`0x00000076`
`0x87654321`	`0x00000021`	`0x00000021`
`0x000000C9`	`0x000000C9`	`0xFFFFFFC9`
`0xEDCBA987`	`0x00000087`	`0xFFFFFF87`

B. Function `fun1` extracts a value from the low-order 8 bits of the argument, giving an integer ranging between 0 and 255. Function `fun2` extracts a value from the low-order 8 bits of the argument, but it also performs sign extension. The result will be a number between -128 and 127.

Solution to Problem 2.24 (page 118)

The effect of truncation is fairly intuitive for unsigned numbers, but not for two's-complement numbers. This exercise lets you explore its properties using very small word sizes.

Hex		Unsigned		Two's complement	
Original	Truncated	Original	Truncated	Original	Truncated
1	1	1	1	1	1
3	3	3	3	3	3
5	5	5	5	5	5
C	4	12	4	−4	4
E	6	14	6	−2	6

As Equation 2.9 states, the effect of this truncation on unsigned values is to simply find their residue, modulo 8. The effect of the truncation on signed values is a bit more complex. According to Equation 2.10, we first compute the modulo 8 residue of the argument. This will give values 0 through 7 for arguments 0 through 7, and also for arguments -8 through -1. Then we apply function $\mathrm{U2T}_3$ to these residues, giving two repetitions of the sequences 0 through 3 and -4 through -1.

Solution to Problem 2.25 (page 119)

This problem is designed to demonstrate how easily bugs can arise due to the implicit casting from signed to unsigned. It seems quite natural to pass parameter `length` as an unsigned, since one would never want to use a negative length. The stopping criterion `i <= length-1` also seems quite natural. But combining these two yields an unexpected outcome!

Since parameter `length` is unsigned, the computation $0 - 1$ is performed using unsigned arithmetic, which is equivalent to modular addition. The result is then UMax. The $\leq$ comparison is also performed using an unsigned comparison, and since any number is less than or equal to UMax, the comparison always holds! Thus, the code attempts to access invalid elements of array `a`.

The code can be fixed either by declaring `length` to be an `int` or by changing the test of the `for` loop to be `i < length`.

Solution to Problem 2.26 (page 119)

This example demonstrates a subtle feature of unsigned arithmetic, and also the property that we sometimes perform unsigned arithmetic without realizing it. This can lead to very tricky bugs.

A. For what cases will this function produce an incorrect result? The function will incorrectly return 1 when `s` is shorter than `t`.

B. Explain how this incorrect result comes about. Since `strlen` is defined to yield an unsigned result, the difference and the comparison are both computed using unsigned arithmetic. When `s` is shorter than

Solution to Problem 2.27 (page 125)

Solution to Problem 2.30 (page 130)

This function is a direct implementation of the rules given to determine whether or not a two's-complement addition overflows.

```
/* Determine whether arguments can be added without overflow */
int tadd_ok(int x, int y) {
    int sum = x+y;
    int neg_over = x <  0 && y <  0 && sum >= 0;
    int pos_over = x >= 0 && y >= 0 && sum <  0;
    return !neg_over && !pos_over;
}
```

Solution to Problem 2.31 (page 130)

Your coworker could have learned, by studying Section 2.3.2, that two's-complement addition forms an abelian group, and so the expression `(x+y)-x` will evaluate to `y` regardless of whether or not the addition overflows, and that `(x+y)-y` will always evaluate to `x`.

Solution to Problem 2.32 (page 130)

This function will give correct values, except when `y` is `TMin`. In this case, we will have `-y` also equal to `TMin`, and so the call to function `tadd_ok` will indicate overflow when `x` is negative and no overflow when `x` is nonnegative. In fact, the opposite is true: `tsub_ok(x, TMin)` should yield 0 when `x` is negative and 1 when it is nonnegative.

One lesson to be learned from this exercise is that `TMin` should be included as one of the cases in any test procedure for a function.

Solution to Problem 2.33 (page 131)

This problem helps you understand two's-complement negation using a very small word size.

For $w = 4$, we have $\mathrm{TMin}_4 = -8$. So -8 is its own additive inverse, while other values are negated by integer negation.

x		$-_4^t$ x	
Hex	Decimal	Decimal	Hex
2	2	−2	E
3	3	−3	D
9	−9	−9	7
B	−5	5	5
C	−4	4	4

The bit patterns are the same as for unsigned negation.

Solution to Problem 2.34 (page 134)

This problem is an exercise to make sure you understand two's-complement multiplication.

Mode	x		y		x · y		Truncated x · y	
Unsigned	4	[100]	5	[101]	20	[010100]	4	[100]
Two's complement	−4	[100]	−3	[101]	12	[001100]	−4	[100]
Unsigned	2	[010]	7	[111]	14	[001110]	6	[110]
Two's complement	2	[010]	−1	[111]	−2	[111110]	−2	[110]
Unsigned	6	[110]	6	[110]	36	[100100]	4	[100]
Two's complement	−2	[110]	−2	[110]	4	[000100]	−4	[100]

Solution to Problem 2.35 (page 135)

It is not realistic to test this function for all possible values of x and y. Even if you could run 10 billion tests per second, it would require over 58 years to test all combinations when data type `int` is 32 bits. On the other hand, it is feasible to test your code by writing the function with data type `short` or `char` and then testing it exhaustively.

Here's a more principled approach, following the proposed set of arguments:

1. We know that $x \cdot y$ can be written as a $2w$-bit two's-complement number. Let u denote the unsigned number represented by the lower w bits, and v denote the two's-complement number represented by the upper w bits. Then, based on Equation 2.3, we can see that $x \cdot y = v2^w + u$.

   We also know that $u = T2U_w(p)$, since they are unsigned and two's-complement numbers arising from the same bit pattern, and so by Equation 2.6, we can write $u = p + p_{w-1}2^w$, where p_{w-1} is the most significant bit of p. Letting $t = v + p_{w-1}$, we have $x \cdot y = p + t2^w$.

   When $t = 0$, we have $x \cdot y = p$; the multiplication does not overflow. When $t = 0$, we have $x \cdot y = p$; the multiplication does overflow.

2. By definition of integer division, dividing p by nonzero x gives a quotient q and a remainder r such that $p = x \cdot q + r$, and $|r| < |x|$. (We use absolute values here, because the signs of x and r may differ. For example, dividing -7 by 2 gives quotient -3 and remainder -1.)

3. Suppose $q = y$. Then we have $x \cdot y = x \cdot y + r + t2^w$. From this, we can see that $r + t2^w = 0$. But $|r| < |x| \leq 2^w$, and so this identity can hold only if $t = 0$, in which case $r = 0$.

   Suppose $r = t = 0$. Then we will have $x \cdot y = x \cdot q$, implying that $y = q$.

When x equals 0, multiplication does not overflow, and so we see that our code provides a reliable way to test whether or not two's-complement multiplication causes overflow.

Solution to Problem 2.36 (page 135)

With 64 bits, we can perform the multiplication without overflowing. We then test whether casting the product to 32 bits changes the value:

```
1  /* Determine whether the arguments can be multiplied
2     without overflow */
3  int tmult_ok(int x, int y) {
4      /* Compute product without overflow */
5      int64_t pll = (int64_t) x*y;
6      /* See if casting to int preserves value */
7      return pll == (int) pll;
8  }
```

Note that the casting on the right-hand side of line 5 is critical. If we instead wrote the line as

```
int64_t pll = x*y;
```

the product would be computed as a 32-bit value (possibly overflowing) and then sign extended to 64 bits.

Solution to Problem 2.37 (page 135)

A. This change does not help at all. Even though the computation of `asize` will be accurate, the call to `malloc` will cause this value to be converted to a 32-bit unsigned number, and so the same overflow conditions will occur.

B. With `malloc` having a 32-bit unsigned number as its argument, it cannot possibly allocate a block of more than 2^{32} bytes, and so there is no point attempting to allocate or copy this much memory. Instead, the function should abort and return `NULL`, as illustrated by the following replacement to the original call to `malloc` (line 9):

```
uint64_t required_size = ele_cnt * (uint64_t) ele_size;
size_t request_size = (size_t) required_size;
if (required_size != request_size)
    /* Overflow must have occurred. Abort operation */
    return NULL;
void *result = malloc(request_size);
if (result == NULL)
    /* malloc failed */
    return NULL;
```

Solution to Problem 2.38 (page 138)

In Chapter 3, we will see many examples of the LEA instruction in action. The instruction is provided to support pointer arithmetic, but the C compiler often uses it as a way to perform multiplication by small constants.

For each value of k, we can compute two multiples: 2^k (when b is 0) and $2^k + 1$ (when b is a). Thus, we can compute multiples 1, 2, 3, 4, 5, 8, and 9.

Solution to Problem 2.39 (page 139)

The expression simply becomes `-(x<<m)`. To see this, let the word size be `w` so that `n = w − 1`. Form B states that we should compute `(x<<w) - (x<<m)`, but shifting `x` to the left by `w` will yield the value `0`.

Solution to Problem 2.40 (page 139)

This problem requires you to try out the optimizations already described and also to supply a bit of your own ingenuity.

K	Shifts	Add/Subs	Expression
7	1	1	`(x<<3) - x`
30	4	3	`(x<<4) + (x<<3) + (x<<2) + (x<<1)`
28	2	1	`(x<<5) - (x<<2)`
55	2	2	`(x<<6) - (x<<3) - x`

Observe that the fourth case uses a modified version of form B. We can view the bit pattern [11011] as having a run of 6 ones with a zero in the middle, and so we apply the rule for form B, but then we subtract the term corresponding to the middle zero bit.

Solution to Problem 2.41 (page 139)

Assuming that addition and subtraction have the same performance, the rule is to choose form A when `n = m`, either form when `n = m + 1`, and form B when `n > m + 1`.

The justification for this rule is as follows. Assume first that `m > 0`. When `n = m`, form A requires only a single shift, while form B requires two shifts and a subtraction. When `n = m + 1`, both forms require two shifts and either an addition or a subtraction. When `n > m + 1`, form B requires only two shifts and one subtraction, while form A requires `n − m + 1 > 2` shifts and `n − m > 1` additions. For the case of `m = 0`, we get one fewer shift for both forms A and B, and so the same rules apply for choosing between the two.

Solution to Problem 2.42 (page 143)

The only challenge here is to compute the bias without any testing or conditional operations. We use the trick that the expression `x >> 31` generates a word with all ones if `x` is negative, and all zeros otherwise. By masking off the appropriate bits, we get the desired bias value.

```
int div16(int x) {
    /* Compute bias to be either 0 (x >= 0) or 15 (x < 0) */
    int bias = (x >> 31) & 0xF;
    return (x + bias) >> 4;
}
```

Solution to Problem 2.43 (page 143)

We have found that people have difficulty with this exercise when working directly with assembly code. It becomes more clear when put in the form shown in `optarith`.

We can see that `M` is 31; `x*M` is computed as `(x<<5)-x`.

We can see that `N` is 8; a bias value of 7 is added when `y` is negative, and the right shift is by 3.

Solution to Problem 2.44 (page 144)

These "C puzzle" problems provide a clear demonstration that programmers must understand the properties of computer arithmetic:

A. `(x > 0) || (x-1 < 0)`
False. Let `x` be −2,147,483,648 ($\mathtt{TMin}_{32}$). We will then have `x-1` equal to 2,147,483,647 ($\mathtt{TMax}_{32}$).

B. `(x & 7) != 7 || (x<<29 < 0)`
True. If `(x & 7) != 7` evaluates to 0, then we must have bit x_2 equal to 1. When shifted left by 29, this will become the sign bit.

C. `(x * x) >= 0`
False. When `x` is 65,535 (`0xFFFF`), `x*x` is −131,071 (`0xFFFE0001`).

D. `x < 0 || -x <= 0`
True. If `x` is nonnegative, then `-x` is nonpositive.

E. `x > 0 || -x >= 0`
False. Let `x` be −2,147,483,648 ($\mathtt{TMin}_{32}$). Then both `x` and `-x` are negative.

F. `x+y == uy+ux`
True. Two's-complement and unsigned addition have the same bit-level behavior, and they are commutative.

G. `x*~y + uy*ux == -x`
True. `~y` equals `-y-1`. `uy*ux` equals `x*y`. Thus, the left-hand side is equivalent to `x*-y-x+x*y`.

Solution to Problem 2.45 (page 147)

Understanding fractional binary representations is an important step to understanding floating-point encodings. This exercise lets you try out some simple examples.

$\frac{1}{8}$	0.001	0.125
$\frac{3}{4}$	0.11	0.75
$\frac{25}{16}$	1.1001	1.5625
$\frac{43}{16}$	10.1011	2.6875
$\frac{9}{8}$	1.001	1.125
$\frac{47}{8}$	101.111	5.875
$\frac{51}{16}$	11.0011	3.1875

One simple way to think about fractional binary representations is to represent a number as a fraction of the form $\frac{x}{2^k}$. We can write this in binary using the binary representation of x, with the binary point inserted k positions from the right. As an example, for $\frac{25}{16}$, we have $25_{10} = 11001_2$. We then put the binary point four positions from the right to get 1.1001_2.

Solution to Problem 2.46 (page 147)

In most cases, the limited precision of floating-point numbers is not a major problem, because the relative error of the computation is still fairly low. In this example, however, the system was sensitive to the absolute error.

A. We can see that $0.1 - x$ has the binary representation

$$0.000000000000000000000001100[1100]\cdots_2$$

B. Comparing this to the binary representation of $\frac{1}{10}$, we can see that it is simply $2^{-20} \times \frac{1}{10}$, which is around 9.54×10^{-8}.

C. $9.54 \times 10^{-8} \times 100 \times 60 \times 60 \times 10 \approx 0.343$ seconds.

D. $0.343 \times 2{,}000 \approx 687$ meters.

Solution to Problem 2.47 (page 153)

Working through floating-point representations for very small word sizes helps clarify how IEEE floating point works. Note especially the transition between denormalized and normalized values.

Bits	e	E	2^E	f	M	$2^E \times M$	V	Decimal
0 00 00	0	0	1	$\frac{0}{4}$	$\frac{0}{4}$	$\frac{0}{4}$	0	0.0
0 00 01	0	0	1	$\frac{1}{4}$	$\frac{1}{4}$	$\frac{1}{4}$	$\frac{1}{4}$	0.25
0 00 10	0	0	1	$\frac{2}{4}$	$\frac{2}{4}$	$\frac{2}{4}$	$\frac{1}{2}$	0.5
0 00 11	0	0	1	$\frac{3}{4}$	$\frac{3}{4}$	$\frac{3}{4}$	$\frac{3}{4}$	0.75
0 01 00	1	0	1	$\frac{0}{4}$	$\frac{4}{4}$	$\frac{4}{4}$	1	1.0
0 01 01	1	0	1	$\frac{1}{4}$	$\frac{5}{4}$	$\frac{5}{4}$	$\frac{5}{4}$	1.25
0 01 10	1	0	1	$\frac{2}{4}$	$\frac{6}{4}$	$\frac{6}{4}$	$\frac{3}{2}$	1.5
0 01 11	1	0	1	$\frac{3}{4}$	$\frac{7}{4}$	$\frac{7}{4}$	$\frac{7}{4}$	1.75
0 10 00	2	1	2	$\frac{0}{4}$	$\frac{4}{4}$	$\frac{8}{4}$	2	2.0
0 10 01	2	1	2	$\frac{1}{4}$	$\frac{5}{4}$	$\frac{10}{4}$	$\frac{5}{2}$	2.5
0 10 10	2	1	2	$\frac{2}{4}$	$\frac{6}{4}$	$\frac{12}{4}$	3	3.0
0 10 11	2	1	2	$\frac{3}{4}$	$\frac{7}{4}$	$\frac{14}{4}$	$\frac{7}{2}$	3.5
0 11 00	—	—	—	—	—	—	∞	—
0 11 01	—	—	—	—	—	—	NaN	—
0 11 10	—	—	—	—	—	—	NaN	—
0 11 11	—	—	—	—	—	—	NaN	—

Solution to Problem 2.48 (page 155)

Hexadecimal `0x359141` is equivalent to binary [11010110001000101000001]. Shifting this right 21 places gives $1.10101100100010100000 1_2 \times 2^{21}$. We form the fraction field by dropping the leading 1 and adding two zeros, giving

[10101100100010100000100]

The exponent is formed by adding bias 127 to 21, giving 148 (binary [10010100]). We combine this with a sign field of 0 to give a binary representation

[01001010010101100100010100000100]

We see that the matching bits in the two representations correspond to the low-order bits of the integer, up to the most significant bit equal to 1 matching the high-order 21 bits of the fraction:

```
    0   0   3   5   9   1   4   1
00000000001101011001000101000001
           ********************
     4   A   5   6   4   5   0   4
01001010010101100100010100000100
```

Solution to Problem 2.49 (page 156)

This exercise helps you think about what numbers cannot be represented exactly in floating point.

A. The number has binary representation 1, followed by n zeros, followed by 1, giving value $2^{n+1} + 1$.

B. When n = 23, the value is $2^{24} + 1 = 16{,}777{,}217$.

Solution to Problem 2.50 (page 157)

Performing rounding by hand helps reinforce the idea of round-to-even with binary numbers.

Original		Rounded	
10.111_2	$2\frac{7}{8}$	11.0	3
11.010_2	$3\frac{1}{4}$	11.0	3
11.000_2	3	11.0	3
10.110_2	$2\frac{3}{4}$	11.0	3

Solution to Problem 2.51 (page 158)

A. Looking at the nonterminating sequence for $\frac{1}{10}$, we see that the 2 bits to the right of the rounding position are 1, so a better approximation to $\frac{1}{10}$ would be obtained by incrementing x to get $x = 0.0001100110011001100110 1_2$, which is larger than 0.1.

B. We can see that $x - 0.1$ has binary representation

0.00000000000000000000000[1100]

Comparing this to the binary representation of $\frac{1}{10}$, we can see that it is $2^{-22} \times \frac{1}{10}$, which is around 2.38×10^{-8}.

C. $2.38 \times 10^{-8} \times 100 \times 60 \times 60 \times 10 \approx 0.086$ seconds, a factor of 4 less than the error in the Patriot system.

D. $0.086 \times 2{,}000 \approx 171$ meters.

Solution to Problem 2.52 (page 158)

This problem tests a lot of concepts about floating-point representations, including the encoding of normalized and denormalized values, as well as rounding.

Format A		Format B		
Bits	Value	Bits	Value	Comments
`011 0000`	1	`0111 000`	1	
`101 1110`	$\frac{15}{2}$	`1001 111`	$\frac{15}{2}$	
`010 1001`	$\frac{25}{32}$	`0110 100`	$\frac{3}{4}$	Round down
`110 1111`	$\frac{31}{2}$	`1011 000`	16	Round up
`000 0001`	$\frac{1}{64}$	`0001 000`	$\frac{1}{64}$	Denorm → norm

Solution to Problem 2.53 (page 161)

In general, it is better to use a library macro rather than inventing your own code. This code seems to work on a variety of machines, however.

We assume that the value `1e400` overflows to infinity.

```
#define POS_INFINITY 1e400
#define NEG_INFINITY (-POS_INFINITY)
#define NEG_ZERO (-1.0/POS_INFINITY)
```

Solution to Problem 2.54 (page 161)

Exercises such as this one help you develop your ability to reason about floating-point operations from a programmer's perspective. Make sure you understand each of the answers.

A. `x == (int)(double) x`
Yes, since `double` has greater precision and range than `int`.

B. `x == (int)(float) x`
No. For example, when `x` is `TMax`.

C. `d == (double)(float) d`
No. For example, when `d` is `1e40`, we will get `+` ∞ on the right.

D. `f == (float)(double) f`
Yes, since `double` has greater precision and range than `float`.

E. `f == -(-f)`
Yes, since a floating-point number is negated by simply inverting its sign bit.

F. `1.0/2 == 1/2.0`
Yes, the numerators and denominators will both be converted to floating-point representations before the division is performed.

G. `d*d >= 0.0`
Yes, although it may overflow to $+\infty$.

H. `(f+d)-f == d`
No. For example, when `f` is `1.0e20` and `d` is 1.0, the expression `f+d` will be rounded to `1.0e20`, and so the expression on the left-hand side will evaluate to 0.0, while the right-hand side will be 1.0.

3

Machine-Level Representation of Programs

3.1 A Historical Perspective 202

3.2 Program Encoding 205

3.3 Data Formats 213

3.4 Accessing Information 215

3.5 Arithmetic and Logical Operations 227

3.6 Control 236

3.7 Procedures 274

3.8 Array Allocation and Access 291

3.9 Heterogeneous Data Structures 301

3.10 Combining Control and Data in Machine-Level Programs 312

3.11 Floating-Point Code 329

3.12 Summary 345

Bibliographic Notes 346

Homework Problems 347

Solutions to Practice Problems 361

Computers execute machine code, sequences of bytes encoding the low-level operations that manipulate data, manage memory, read and write data on storage devices, and communicate over networks. A compiler generates machine code through a series of stages, based on the rules of the programming language, the instruction set of the target machine, and the conventions followed by the operating system. The GCC C compiler generates its output in the form of assembly code, a textual representation of the machine code giving the individual instructions in the program. GCC then invokes both an assembler and a linker to generate the executable machine code from the assembly code. In this chapter, we will take a close look at machine code and its human-readable representation as assembly code.

When programming in a high-level language such as C, and even more so in Java, we are shielded from the detailed machine-level implementation of our program. In contrast, when writing programs in assembly code (as was done in the early days of computing) a programmer must specify the low-level instructions the program uses to carry out a computation. Most of the time, it is much more productive and reliable to work at the higher level of abstraction provided by a high-level language. The type checking provided by a compiler helps detect many program errors and makes sure we reference and manipulate data in consistent ways. With modern optimizing compilers, the generated code is usually at least as efficient as what a skilled assembly-language programmer would write by hand. Best of all, a program written in a high-level language can be compiled and executed on a number of different machines, whereas assembly code is highly machine specific.

So why should we spend our time learning machine code? Even though compilers do most of the work in generating assembly code, being able to read and understand it is an important skill for serious programmers. By invoking the compiler with appropriate command-line parameters, the compiler will generate a file showing its output in assembly-code form. By reading this code, we can understand the optimization capabilities of the compiler and analyze the underlying inefficiencies in the code. As we will experience in Chapter 5, programmers seeking to maximize the performance of a critical section of code often try different variations of the source code, each time compiling and examining the generated assembly code to get a sense of how efficiently the program will run. Furthermore, there are times when the layer of abstraction provided by a high-level language hides information about the run-time behavior of a program that we need to understand. For example, when writing concurrent programs using a thread package, as covered in Chapter 12, it is important to understand how program data are shared or kept private by the different threads and precisely how and where shared data are accessed. Such information is visible at the machine-code level. As another example, many of the ways programs can be attacked, allowing malware to infest a system, involve nuances of the way programs store their run-time control information. Many attacks involve exploiting weaknesses in system programs to overwrite information and thereby take control of the system. Understanding how these vulnerabilities arise and how to guard against them requires a knowledge of the machine-level representation of programs. The need for programmers to learn

machine code has shifted over the years from one of being able to write programs directly in assembly code to one of being able to read and understand the code generated by compilers.

In this chapter, we will learn the details of one particular assembly language and see how C programs get compiled into this form of machine code. Reading the assembly code generated by a compiler involves a different set of skills than writing assembly code by hand. We must understand the transformations typical compilers make in converting the constructs of C into machine code. Relative to the computations expressed in the C code, optimizing compilers can rearrange execution order, eliminate unneeded computations, replace slow operations with faster ones, and even change recursive computations into iterative ones. Understanding the relation between source code and the generated assembly can often be a challenge—it's much like putting together a puzzle having a slightly different design than the picture on the box. It is a form of reverse engineering—trying to understand the process by which a system was created by studying the system and working backward. In this case, the system is a machine-generated assembly-language program, rather than something designed by a human. This simplifies the task of reverse engineering because the generated code follows fairly regular patterns and we can run experiments, having the compiler generate code for many different programs. In our presentation, we give many examples and provide a number of exercises illustrating different aspects of assembly language and compilers. This is a subject where mastering the details is a prerequisite to understanding the deeper and more fundamental concepts. Those who say "I understand the general principles, I don't want to bother learning the details" are deluding themselves. It is critical for you to spend time studying the examples, working through the exercises, and checking your solutions with those provided.

Our presentation is based on x86-64, the machine language for most of the processors found in today's laptop and desktop machines, as well as those that power very large data centers and supercomputers. This language has evolved over a long history, starting with Intel Corporation's first 16-bit processor in 1978, through to the expansion to 32 bits, and most recently to 64 bits. Along the way, features have been added to make better use of the available semiconductor technology, and to satisfy the demands of the marketplace. Much of the development has been driven by Intel, but its rival Advanced Micro Devices (AMD) has also made important contributions. The result is a rather peculiar design with features that make sense only when viewed from a historical perspective. It is also laden with features providing backward compatibility that are not used by modern compilers and operating systems. We will focus on the subset of the features used by GCC and Linux. This allows us to avoid much of the complexity and many of the arcane features of x86-64.

Our technical presentation starts with a quick tour to show the relation between C, assembly code, and machine code. We then proceed to the details of x86-64, starting with the representation and manipulation of data and the implementation of control. We see how control constructs in C, such as `if`, `while`, and `switch` statements, are implemented. We then cover the implementation of procedures, including how the program maintains a run-time stack to support the

Web Aside ASM:IA32 IA32 r_0 ra i

IA32, the 32-bit predecessor to x86-64, was introduced by Intel in 1985. It served as the machine language of choice for several decades. Most x86 microprocessors sold today, and most operating systems installed on these machines, are designed to run x86-64. However, they can also execute IA32 programs in a backward compatibility mode. As a result, many application programs are still based on IA32. In addition, many existing systems cannot execute x86-64, due to limitations of their hardware or system software. IA32 continues to be an important machine language. You will find that having a background in x86-64 will enable you to learn the IA32 machine language quite readily.

passing of data and control between procedures, as well as storage for local variables. Next, we consider how data structures such as arrays, structures, and unions are implemented at the machine level. With this background in machine-level programming, we can examine the problems of out-of-bounds memory references and the vulnerability of systems to buffer overflow attacks. We finish this part of the presentation with some tips on using the GDB debugger for examining the run-time behavior of a machine-level program. The chapter concludes with a presentation on machine-program representations of code involving floating-point data and operations.

The computer industry has recently made the transition from 32-bit to 64-bit machines. A 32-bit machine can only make use of around 4 gigabytes (2^{32} bytes) of random access memory, With memory prices dropping at dramatic rates, and our computational demands and data sizes increasing, it has become both economically feasible and technically desirable to go beyond this limitation. Current 64-bit machines can use up to 256 terabytes (2^{48} bytes) of memory, and could readily be extended to use up to 16 exabytes (2^{64} bytes). Although it is hard to imagine having a machine with that much memory, keep in mind that 4 gigabytes seemed like an extreme amount of memory when 32-bit machines became commonplace in the 1970s and 1980s.

Our presentation focuses on the types of machine-level programs generated when compiling C and similar programming languages targeting modern operating systems. As a consequence, we make no attempt to describe many of the features of x86-64 that arise out of its legacy support for the styles of programs written in the early days of microprocessors, when much of the code was written manually and where programmers had to struggle with the limited range of addresses allowed by 16-bit machines.

3.1 A Historical Perspective

The Intel processor line, colloquially referred to as x86, has followed a long evolutionary development. It started with one of the first single-chip 16-bit microprocessors, where many compromises had to be made due to the limited capabilities of integrated circuit technology at the time. Since then, it has grown to take ad-

vantage of technology improvements as well as to satisfy the demands for higher performance and for supporting more advanced operating systems.

The list that follows shows some models of Intel processors and some of their key features, especially those affecting machine-level programming. We use the number of transistors required to implement the processors as an indication of how they have evolved in complexity. In this table, "K" denotes 1,000 (10^3), "M" denotes 1,000,000 (10^6), and "G" denotes 1,000,000,000 (10^9).

8086 (1978, 29 K transistors). One of the first single-chip, 16-bit microprocessors. The 8088, a variant of the 8086 with an 8-bit external bus, formed the heart of the original IBM personal computers. IBM contracted with then-tiny Microsoft to develop the MS-DOS operating system. The original models came with 32,768 bytes of memory and two floppy drives (no hard drive). Architecturally, the machines were limited to a 655,360-byte address space—addresses were only 20 bits long (1,048,576 bytes addressable), and the operating system reserved 393,216 bytes for its own use. In 1980, Intel introduced the 8087 floating-point coprocessor (45 K transistors) to operate alongside an 8086 or 8088 processor, executing the floating-point instructions. The 8087 established the floating-point model for the x86 line, often referred to as "x87."

80286 (1982, 134 K transistors). Added more (and now obsolete) addressing modes. Formed the basis of the IBM PC-AT personal computer, the original platform for MS Windows.

i386 (1985, 275 K transistors). Expanded the architecture to 32 bits. Added the flat addressing model used by Linux and recent versions of the Windows operating system. This was the first machine in the series that could fully support a Unix operating system.

i486 (1989, 1.2 M transistors). Improved performance and integrated the floating-point unit onto the processor chip but did not significantly change the instruction set.

Pentium (1993, 3.1 M transistors). Improved performance but only added minor extensions to the instruction set.

PentiumPro (1995, 5.5 M transistors). Introduced a radically new processor design, internally known as the P6 microarchitecture. Added a class of "conditional move" instructions to the instruction set.

Pentium/MMX (1997, 4.5 M transistors). Added new class of instructions to the Pentium processor for manipulating vectors of integers. Each datum can be 1, 2, or 4 bytes long. Each vector totals 64 bits.

Pentium II (1997, 7 M transistors). Continuation of the P6 microarchitecture.

Pentium III (1999, 8.2 M transistors). Introduced SSE, a class of instructions for manipulating vectors of integer or floating-point data. Each datum can be 1, 2, or 4 bytes, packed into vectors of 128 bits. Later versions of this chip

went up to 24 M transistors, due to the incorporation of the level-2 cache on chip.

Pentium 4 (2000, 42 M transistors). Extended SSE to SSE2, adding new data types (including double-precision floating point), along with 144 new instructions for these formats. With these extensions, compilers can use SSE instructions, rather than x87 instructions, to compile floating-point code.

Pentium 4E (2004, 125 M transistors). Added hyperthreading, a method to run two programs simultaneously on a single processor, as well as EM64T, Intel's implementation of a 64-bit extension to IA32 developed by Advanced Micro Devices (AMD), which we refer to as x86-64.

Core 2 (2006, 291 M transistors). Returned to a microarchitecture similar to P6. First multi-core Intel microprocessor, where multiple processors are implemented on a single chip. Did not support hyperthreading.

Core i7, Nehalem (2008, 781 M transistors). Incorporated both hyperthreading and multi-core, with the initial version supporting two executing programs on each core and up to four cores on each chip.

Core i7, Sandy Bridge (2011, 1.17 G transistors). Introduced AVX, an extension of the SSE to support data packed into 256-bit vectors.

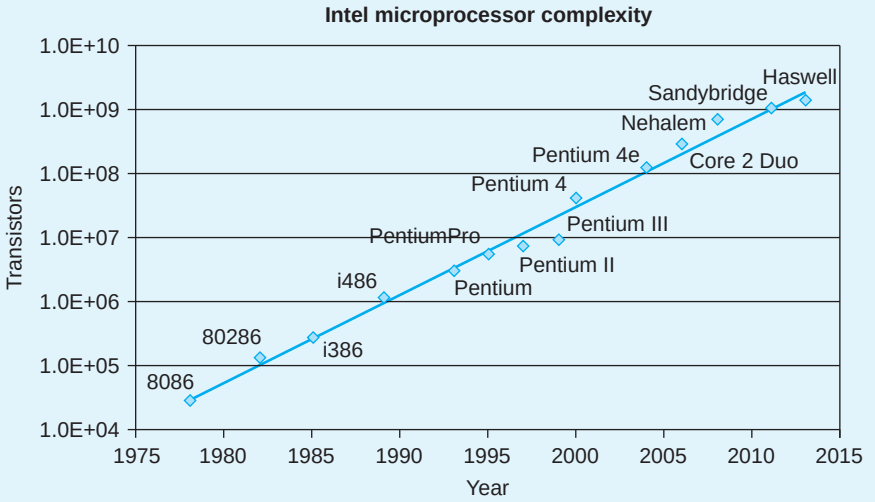
Core i7, Haswell (2013, 1.4 G transistors). Extended AVX to AVX2, adding more instructions and instruction formats.

Each successive processor has been designed to be backward compatible—able to run code compiled for any earlier version. As we will see, there are many strange artifacts in the instruction set due to this evolutionary heritage. Intel has had several names for their processor line, including IA32, for "Intel Architecture 32-bit" and most recently Intel64, the 64-bit extension to IA32, which we will refer to as x86-64. We will refer to the overall line by the commonly used colloquial name "x86," reflecting the processor naming conventions up through the i486.

Over the years, several companies have produced processors that are compatible with Intel processors, capable of running the exact same machine-level programs. Chief among these is Advanced Micro Devices (AMD). For years, AMD lagged just behind Intel in technology, forcing a marketing strategy where they produced processors that were less expensive although somewhat lower in performance. They became more competitive around 2002, being the first to break the 1-gigahertz clock-speed barrier for a commercially available microprocessor, and introducing x86-64, the widely adopted 64-bit extension to Intel's IA32. Although we will talk about Intel processors, our presentation holds just as well for the compatible processors produced by Intel's rivals.

Much of the complexity of x86 is not of concern to those interested in programs for the Linux operating system as generated by the GCC compiler. The memory model provided in the original 8086 and its extensions in the 80286 became obsolete with the i386. The original x87 floating-point instructions became obsolete

Aside Moore's Law



If we plot the number of transistors in the different Intel processors versus the year of introduction, and use a logarithmic scale for the y-axis, we can see that the growth has been phenomenal. Fitting a line through the data, we see that the number of transistors increases at an annual rate of approximately 37%, meaning that the number of transistors doubles about every 26 months. This growth has been sustained over the multiple-decade history of x86 microprocessors.

In 1965, Gordon Moore, a founder of Intel Corporation, extrapolated from the chip technology of the day (by which they could fabricate circuits with around 64 transistors on a single chip) to predict that the number of transistors per chip would double every year for the next 10 years. This prediction became known as Moore's Law. As it turns out, his prediction was just a little bit optimistic, but also too short-sighted. Over more than 50 years, the semiconductor industry has been able to double transistor counts on average every 18 months.

Similar exponential growth rates have occurred for other aspects of computer technology, including the storage capacities of magnetic disks and semiconductor memories. These remarkable growth rates have been the major driving forces of the computer revolution.

with the introduction of SSE2. Although we see vestiges of the historical evolution of x86 in x86-64 programs, many of the most arcane features of x86 do not appear.

3.2 Program Encodings

Suppose we write a C program as two files `p1.c` and `p2.c`. We can then compile this code using a Unix command line:

```
linux> gcc -Og -o p p1.c p2.c
```

The command `gcc` indicates the GCC C compiler. Since this is the default compiler on Linux, we could also invoke it as simply `cc`. The command-line option `-Og`[1] instructs the compiler to apply a level of optimization that yields machine code that follows the overall structure of the original C code. Invoking higher levels of optimization can generate code that is so heavily transformed that the relationship between the generated machine code and the original source code is difficult to understand. We will therefore use `-Og` optimization as a learning tool and then see what happens as we increase the level of optimization. In practice, higher levels of optimization (e.g., specified with the option `-O1` or `-O2`) are considered a better choice in terms of the resulting program performance.

The `gcc` command invokes an entire sequence of programs to turn the source code into executable code. First, the C preprocessor expands the source code to include any files specified with `#include` commands and to expand any macros, specified with `#define` declarations. Second, the compiler generates assembly-code versions of the two source files having names `p1.s` and `p2.s`. Next, the assembler converts the assembly code into binary object-code files `p1.o` and `p2.o`. Object code is one form of machine code—it contains binary representations of all of the instructions, but the addresses of global values are not yet filled in. Finally, the linker merges these two object-code files along with code implementing library functions (e.g., `printf`) and generates the final executable code file `p` (as specified by the command-line directive `-o p`). Executable code is the second form of machine code we will consider—it is the exact form of code that is executed by the processor. The relation between these different forms of machine code and the linking process is described in more detail in Chapter 7.

3.2.1 Machine-Level Code

As described in Section 1.9.3, computer systems employ several different forms of abstraction, hiding details of an implementation through the use of a simpler abstract model. Two of these are especially important for machine-level programming. First, the format and behavior of a machine-level program is defined by the instruction set architecture, or ISA, defining the processor state, the format of the instructions, and the effect each of these instructions will have on the state. Most ISAs, including x86-64, describe the behavior of a program as if each instruction is executed in sequence, with one instruction completing before the next one begins. The processor hardware is far more elaborate, executing many instructions concurrently, but it employs safeguards to ensure that the overall behavior matches the sequential operation dictated by the ISA. Second, the memory addresses used by a machine-level program are virtual addresses, providing a memory model that

1. This optimization level was introduced in GCC version 4.8. Earlier versions of GCC, as well as non-GNU compilers, will not recognize this option. For these, using optimization level one (specified with the command-line flag `-O1`) is probably the best choice for generating code that follows the original program structure.

appears to be a very large byte array. The actual implementation of the memory system involves a combination of multiple hardware memories and operating system software, as described in Chapter 9.

The compiler does most of the work in the overall compilation sequence, transforming programs expressed in the relatively abstract execution model provided by C into the very elementary instructions that the processor executes. The assembly-code representation is very close to machine code. Its main feature is that it is in a more readable textual format, as compared to the binary format of machine code. Being able to understand assembly code and how it relates to the original C code is a key step in understanding how computers execute programs.

The machine code for x86-64 differs greatly from the original C code. Parts of the processor state are visible that normally are hidden from the C programmer:

- The program counter (commonly referred to as the PC, and called `%rip` in x86-64) indicates the address in memory of the next instruction to be executed.
- The integer register file contains 16 named locations storing 64-bit values. These registers can hold addresses (corresponding to C pointers) or integer data. Some registers are used to keep track of critical parts of the program state, while others are used to hold temporary data, such as the arguments and local variables of a procedure, as well as the value to be returned by a function.
- The condition code registers hold status information about the most recently executed arithmetic or logical instruction. These are used to implement conditional changes in the control or data flow, such as is required to implement `if` and `while` statements.
- A set of vector registers can each hold one or more integer or floating-point values.

Whereas C provides a model in which objects of different data types can be declared and allocated in memory, machine code views the memory as simply a large byte-addressable array. Aggregate data types in C such as arrays and structures are represented in machine code as contiguous collections of bytes. Even for scalar data types, assembly code makes no distinctions between signed or unsigned integers, between different types of pointers, or even between pointers and integers.

The program memory contains the executable machine code for the program, some information required by the operating system, a run-time stack for managing procedure calls and returns, and blocks of memory allocated by the user (e.g., by using the `malloc` library function). As mentioned earlier, the program memory is addressed using virtual addresses. At any given time, only limited subranges of virtual addresses are considered valid. For example, x86-64 virtual addresses are represented by 64-bit words. In current implementations of these machines, the upper 16 bits must be set to zero, and so an address can potentially specify a byte over a range of 2^{48}, or 64 terabytes. More typical programs will only have access to a few megabytes, or perhaps several gigabytes. The operating system manages

Aside T r- a i r o rat

In our presentation, we will show the code generated by a particular version of GCC with particular settings of the command-line options. If you compile code on your own machine, chances are you will be using a different compiler or a different version of GCC and hence will generate different code. The open-source community supporting GCC keeps changing the code generator, attempting to generate more efficient code according to changing code guidelines provided by the microprocessor manufacturers.

Our goal in studying the examples shown in our presentation is to demonstrate how to examine assembly code and map it back to the constructs found in high-level programming languages. You will need to adapt these techniques to the style of code generated by your particular compiler.

this virtual address space, translating virtual addresses into the physical addresses of values in the actual processor memory.

A single machine instruction performs only a very elementary operation. For example, it might add two numbers stored in registers, transfer data between memory and a register, or conditionally branch to a new instruction address. The compiler must generate sequences of such instructions to implement program constructs such as arithmetic expression evaluation, loops, or procedure calls and returns.

3.2.2 Code Examples

Suppose we write a C code file `mstore.c` containing the following function definition:

```
long mult2(long, long);

void multstore(long x, long y, long *dest) {
    long t = mult2(x, y);
    *dest = t;
}
```

To see the assembly code generated by the C compiler, we can use the `-S` option on the command line:

```
linux> gcc -Og -S mstore.c
```

This will cause GCC to run the compiler, generating an assembly file `mstore.s`, and go no further. (Normally it would then invoke the assembler to generate an object-code file.)

The assembly-code file contains various declarations, including the following set of lines:

```
multstore:
  pushq   %rbx
```

Aside How do I view machine-level program data?

To display the binary object code for a program (say, `mstore`), we use a disassembler (described below) to determine that the code for the procedure is 14 bytes long. Then we run the GNU debugging tool GDB on file `mstore.o` and give it the command

```
(gdb) x/14xb multstore
```

telling it to display (abbreviated 'x') 14 hex-formatted (also 'x') bytes ('b') starting at the address where function `multstore` is located. You will find that GDB has many useful features for analyzing machine-level programs, as will be discussed in Section 3.10.2.

```
movq    %rdx, %rbx
call    mult2
movq    %rax, (%rbx)
popq    %rbx
ret
```

Each indented line in the code corresponds to a single machine instruction. For example, the `pushq` instruction indicates that the contents of register `%rbx` should be pushed onto the program stack. All information about local variable names or data types has been stripped away.

If we use the `-c` command-line option, GCC will both compile and assemble the code

```
linux> gcc -Og -c mstore.c
```

This will generate an object-code file `mstore.o` that is in binary format and hence cannot be viewed directly. Embedded within the 1,368 bytes of the file `mstore.o` is a 14-byte sequence with the hexadecimal representation

```
53 48 89 d3 e8 00 00 00 00 48 89 03 5b c3
```

This is the object code corresponding to the assembly instructions listed previously. A key lesson to learn from this is that the program executed by the machine is simply a sequence of bytes encoding a series of instructions. The machine has very little information about the source code from which these instructions were generated.

To inspect the contents of machine-code files, a class of programs known as disassemblers can be invaluable. These programs generate a format similar to assembly code from the machine code. With Linux systems, the program OBJDUMP (for "object dump") can serve this role given the `-d` command-line flag:

```
linux> objdump -d mstore.o
```

The result (where we have added line numbers on the left and annotations in italicized text) is as follows:

```
    Disassembly of function sum in binary file mstore.o
1   0000000000000000 <multstore>:
    Offset   Bytes                    Equivalent assembly language
2      0:    53                       push   %rbx
3      1:    48 89 d3                 mov    %rdx,%rbx
4      4:    e8 00 00 00 00           callq  9 <multstore+0x9>
5      9:    48 89 03                 mov    %rax,(%rbx)
6      c:    5b                       pop    %rbx
7      d:    c3                       retq
```

On the left we see the 14 hexadecimal byte values, listed in the byte sequence shown earlier, partitioned into groups of 1 to 5 bytes each. Each of these groups is a single instruction, with the assembly-language equivalent shown on the right.

Several features about machine code and its disassembled representation are worth noting:

- x86-64 instructions can range in length from 1 to 15 bytes. The instruction encoding is designed so that commonly used instructions and those with fewer operands require a smaller number of bytes than do less common ones or ones with more operands.
- The instruction format is designed in such a way that from a given starting position, there is a unique decoding of the bytes into machine instructions. For example, only the instruction `pushq %rbx` can start with byte value `53`.
- The disassembler determines the assembly code based purely on the byte sequences in the machine-code file. It does not require access to the source or assembly-code versions of the program.
- The disassembler uses a slightly different naming convention for the instructions than does the assembly code generated by GCC. In our example, it has omitted the suffix '`q`' from many of the instructions. These suffixes are size designators and can be omitted in most cases. Conversely, the disassembler adds the suffix '`q`' to the `call` and `ret` instructions. Again, these suffixes can safely be omitted.

Generating the actual executable code requires running a linker onaregccgcc

```
long mult2(long a, long b) {
    long s = a * b;
    return s;
}
```

Then we could generate an executable program `prog` as follows:

```
linux> gcc -Og -o prog main.c mstore.c
```

The file `prog` has grown to 8,655 bytes, since it contains not just the machine code for the procedures we provided but also code used to start and terminate the program as well as to interact with the operating system.

We can disassemble the file `prog`:

```
linux> objdump -d prog
```

The disassembler will extract various code sequences, including the following:

```
Disassembly of function sum in binary file prog
0000000000400540 <multstore>:
  400540:  53                      push   %rbx
  400541:  48 89 d3                mov    %rdx,%rbx
  400544:  e8 42 00 00 00          callq  40058b <mult2>
  400549:  48 89 03                mov    %rax,(%rbx)
  40054c:  5b                      pop    %rbx
  40054d:  c3                      retq
  40054e:  90                      nop
  40054f:  90                      nop
```

This code is almost identical to that generated by the disassembly of `mstore.c`. One important difference is that the addresses listed along the left are different—the linker has shifted the location of this code to a different range of addresses. A second difference is that the linker has filled in the address that the `callq` instruction should use in calling the function `mult2` (line 4 of the disassembly). One task for the linker is to match function calls with the locations of the executable code for those functions. A final difference is that we see two additional lines of code (lines 8–9). These instructions will have no effect on the program, since they occur after the return instruction (line 7). They have been inserted to grow the code for the function to 16 bytes, enabling a better placement of the next block of code in terms of memory system performance.

3.2.3 Notes on Formatting

The assembly code generated by GCC is difficult for a human to read. On one hand, it contains information with which we need not be concerned, while on the other hand, it does not provide any description of the program or how it works. For example, suppose we give the command

```
linux> gcc -Og -S mstore.c
```

to generate the file `mstore.s`. The full content of the file is as follows:

```
        .file   "010-mstore.c"
        .text
        .globl  multstore
        .type   multstore, @function
multstore:
        pushq   %rbx
        movq    %rdx, %rbx
        call    mult2
        movq    %rax, (%rbx)
        popq    %rbx
        ret
        .size   multstore, .-multstore
        .ident  "GCC: (Ubuntu 4.8.1-2ubuntu1~12.04) 4.8.1"
        .section        .note.GNU-stack,"",@progbits
```

All of the lines beginning with '.' are directives to guide the assembler and linker. We can generally ignore these. On the other hand, there are no explanatory remarks about what the instructions do or how they relate to the source code.

To provide a clearer presentation of assembly code, we will show it in a form that omits most of the directives, while including line numbers and explanatory annotations. For our example, an annotated version would appear as follows:

```
     void multstore(long x, long y, long *dest)
     x in %rdi, y in %rsi, dest in %rdx
1    multstore:
2      pushq   %rbx             Save %rbx
3      movq    %rdx, %rbx       Copy dest to %rbx
4      call    mult2            Call mult2(x, y)
5      movq    %rax, (%rbx)     Store result at *dest
6      popq    %rbx             Restore %rbx
7      ret                      Return
```

We typically show only the lines of code relevant to the point being discussed. Each line is numbered on the left for reference and annotated on the right by a brief description of the effect of the instruction and how it relates to the computations of the original C code. This is a stylized version of the way assembly-language programmers format their code.

We also provide Web asides to cover material intended for dedicated machine-language enthusiasts. One Web aside describes IA32 machine code. Having a background in x86-64 makes learning IA32 fairly simple. Another Web aside gives a brief presentation of ways to incorporate assembly code into C programs. For some applications, the programmer must drop down to assembly code to access low-level features of the machine. One approach is to write entire functions in assembly code and combine them with C functions during the linking stage. A

Aside ATT r ltla l-o o r at

In our presentation, we show assembly code in ATT format (named after AT&T, the company that operated Bell Laboratories for many years), the default format for GCC, OBJDUMP, and the other tools we will consider. Other programming tools, including those from Microsoft as well as the documentation from Intel, show assembly code in Intel format. The two formats differ in a number of ways. As an example, GCC can generate code in Intel format for the `sum` function using the following command line:

```
linux> gcc -Og -S -masm=intel mstore.c
```

This gives the following assembly code:

```
multstore:
  push    rbx
  mov     rbx, rdx
  call    mult2
  mov     QWORD PTR [rbx], rax
  pop     rbx
  ret
```

We see that the Intel and ATT formats differ in the following ways:

- The Intel code omits the size designation suffixes. We see instruction `push` and `mov` instead of `pushq` and `movq`.
- The Intel code omits the '%' character in front of register names, using `rbx` instead of `%rbx`.
- The Intel code has a different way of describing locations in memory—for example, `QWORD PTR [rbx]` rather than `(%rbx)`.
- Instructions with multiple operands list them in the reverse order. This can be very confusing when switching between the two formats.

Although we will not be using Intel format in our presentation, you will encounter it in documentation from Intel and Microsoft.

second is to use GCC's support for embedding assembly code directly within C programs.

3.3 Data Formats

Due to its origins as a 16-bit architecture that expanded into a 32-bit one, Intel uses the term "word" to refer to a 16-bit data type. Based on this, they refer to 32-bit quantities as "double words," and 64-bit quantities as "quad words." Figure 3.1 shows the x86-64 representations used for the primitive data types of C. Standard `int` values are stored as double words (32 bits). Pointers (shown here as `char *`) are stored as 8-byte quad words, as would be expected in a 64-bit machine. With x86-64, data type `long` is implemented with 64 bits, allowing a very wide range of values. Most of our code examples in this chapter use pointers and `long` data

Web Aside ASM:EASM C o i i a l o it C r o ra

Although a C compiler does a good job of converting the computations expressed in a program into machine code, there are some features of a machine that cannot be accessed by a C program. For example, every time an x86-64 processor executes an arithmetic or logical operation, it sets a 1-bit condition code flag, named PF (for "parity flag"), to 1 when the lower 8 bits in the resulting computation have an even number of ones and to 0 otherwise. Computing this information in C requires at least seven shifting, masking, and EXCLUSIVE-OR operations (see Problem 2.65). Even though the hardware performs this computation as part of every arithmetic or logical operation, there is no way for a C program to determine the value of the PF condition code flag. This task can readily be performed by incorporating a small number of assembly-code instructions into the program.

There are two ways to incorporate assembly code into C programs. First, we can write an entire function as a separate assembly-code file and let the assembler and linker combine this with code we have written in C. Second, we can use the inline assembly feature of GCC, where brief sections of assembly code can be incorporated into a C program using the `asm` directive. This approach has the advantage that it minimizes the amount of machine-specific code.

Of course, including assembly code in a C program makes the code specific to a particular class of machines (such as x86-64), and so it should only be used when the desired feature can only be accessed in this way.

C declaration	Intel data type	Assembly-code suffix	Size (bytes)
`char`	Byte	`b`	1
`short`	Word	`w`	2
`int`	Double word	`l`	4
`long`	Quad word	`q`	8
`char *`	Quad word	`q`	8
`float`	Single precision	`s`	4
`double`	Double precision	`l`	8

Figure 3.1 Sizes of C data types in x86-64. Wit a 64- it a i , o i t r ar 8 t l o .

types, and so they will operate on quad words. The x86-64 instruction set includes a full complement of instructions for bytes, words, and double words as well.

Floating-point numbers come in two principal formats: single-precision (4-byte) values, corresponding to C data type `float`, and double-precision (8-byte) values, corresponding to C data type `double`. Microprocessors in the x86 family historically implemented all floating-point operations with a special 80-bit (10-byte) floating-point format (see Problem 2.86). This format can be specified in C programs using the declaration `long double`. We recommend against using this format, however. It is not portable to other classes of machines, and it is typically

not implemented with the same high-performance hardware as is the case for single- and double-precision arithmetic.

As the table of Figure 3.1 indicates, most assembly-code instructions generated by GCC have a single-character suffix denoting the size of the operand. For example, the data movement instruction has four variants: `movb` (move byte), `movw` (move word), `movl` (move double word), and `movq` (move quad word). The suffix '`l`' is used for double words, since 32-bit quantities are considered to be "long words." The assembly code uses the suffix '`l`' to denote a 4-byte integer as well as an 8-byte double-precision floating-point number. This causes no ambiguity, since floating-point code involves an entirely different set of instructions and registers.

3.4 Accessing Information

An x86-64 central processing unit (CPU) contains a set of 16 general-purpose registers storing 64-bit values. These registers are used to store integer data as well as pointers. Figure 3.2 diagrams the 16 registers. Their names all begin with `%r`, but otherwise follow multiple different naming conventions, owing to the historical evolution of the instruction set. The original 8086 had eight 16-bit registers, shown in Figure 3.2 as registers `%ax` through `%bp`. Each had a specific purpose, and hence they were given names that reflected how they were to be used. With the extension to IA32, these registers were expanded to 32-bit registers, labeled `%eax` through `%ebp`. In the extension to x86-64, the original eight registers were expanded to 64 bits, labeled `%rax` through `%rbp`. In addition, eight new registers were added, and these were given labels according to a new naming convention: `%r8` through `%r15`.

As the nested boxes in Figure 3.2 indicate, instructions can operate on data of different sizes stored in the low-order bytes of the 16 registers. Byte-level operations can access the least significant byte, 16-bit operations can access the least significant 2 bytes, 32-bit operations can access the least significant 4 bytes, and 64-bit operations can access entire registers.

In later sections, we will present a number of instructions for copying and generating 1-, 2-, 4-, and 8-byte values. When these instructions have registers as destinations, two conventions arise for what happens to the remaining bytes in the register for instructions that generate less than 8 bytes: Those that generate 1- or 2-byte quantities leave the remaining bytes unchanged. Those that generate 4-byte quantities set the upper 4 bytes of the register to zero. The latter convention was adopted as part of the expansion from IA32 to x86-64.

As the annotations along the right-hand side of Figure 3.2 indicate, different registers serve different roles in typical programs. Most unique among them is the stack pointer, `%rsp`, used to indicate the end position in the run-time stack. Some instructions specifically read and write this register. The other 15 registers have more flexibility in their uses. A small number of instructions make specific use of certain registers. More importantly, a set of standard programming conventions governs how the registers are to be used for managing the stack, passing function

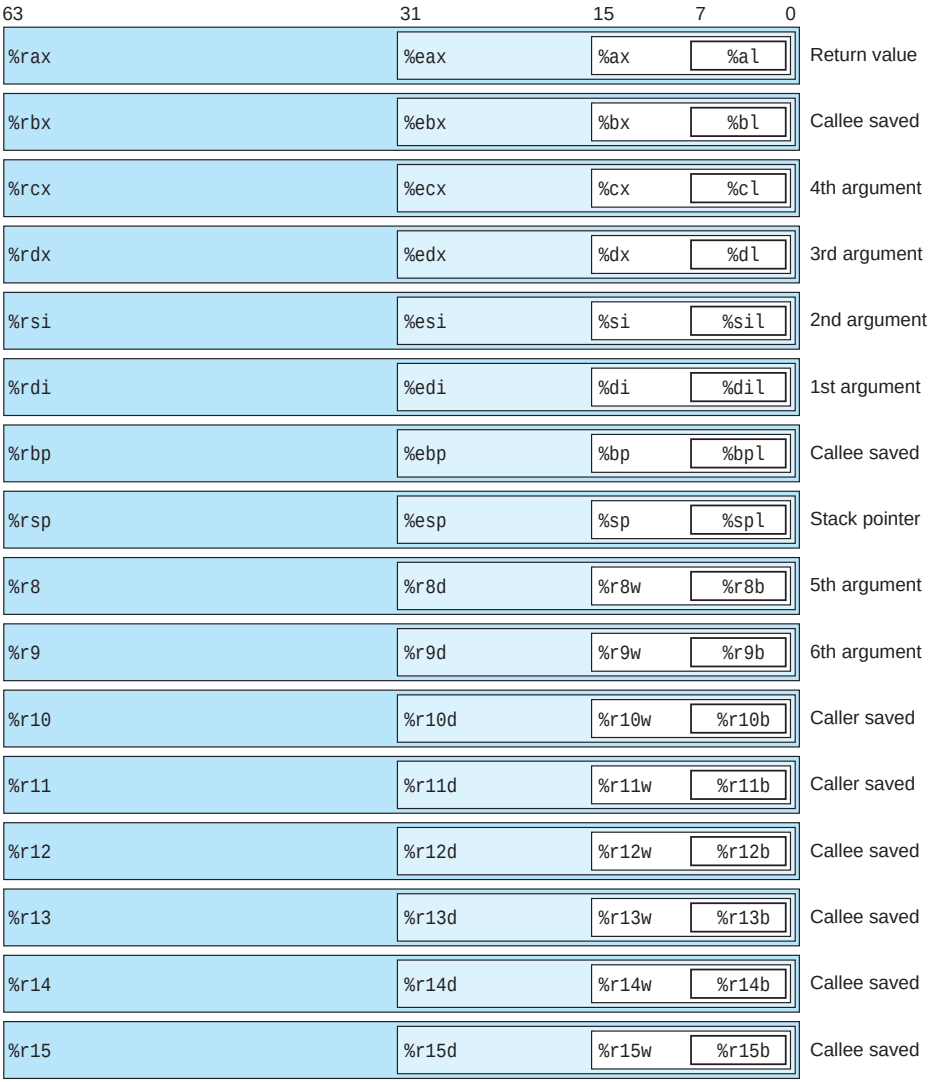


Figure 3.2 Integer registers. The low-order portion of all 16 registers can be accessed as 1 byte, 2 bytes (16-bit), 4 bytes (32-bit), or 8 bytes (64-bit) as indicated.

arguments, returning values from functions, and storing local and temporary data. We will cover these conventions in our presentation, especially in Section 3.7, where we describe the implementation of procedures.

3.4.1 Operand Specifiers

Most instructions have one or more operands specifying the source values to use in performing an operation and the destination location into which to place the

Type	Form	Operand value	Name
Immediate	$Imm	Imm	Immediate
Register	r_a	R[r_a]	Register
Memory	Imm	M[Imm]	Absolute
Memory	(r_a)	M[R[r_a]]	Indirect
Memory	Imm(r_b)	M[Imm + R[r_b]]	Base + displacement
Memory	(r_b,r_i)	M[R[r_b] + R[r_i]]	Indexed
Memory	Imm(r_b,r_i)	M[Imm + R[r_b] + R[r_i]]	Indexed
Memory	(,r_i,s)	M[R[r_i] · s]	Scaled indexed
Memory	Imm(,r_i,s)	M[Imm + R[r_i] · s]	Scaled indexed
Memory	(r_b,r_i,s)	M[R[r_b] + R[r_i] · s]	Scaled indexed
Memory	Imm(r_b,r_i,s)	M[Imm + R[r_b] + R[r_i] · s]	Scaled indexed

Figure 3.3 Operand forms. O ra a t i at (ta t) al , r i t r al , r al r r . T ali a t rs t t r 1, 2, 4, r 8.

result. x86-64 supports a number of operand forms (see Figure 3.3). Source values can be given as constants or read from registers or memory. Results can be stored in either registers or memory. Thus, the different operand possibilities can be classified into three types. The first type, immediate, is for constant values. In ATT-format assembly code, these are written with a '$' followed by an integer using standard C notation—for example, `$-577` or `$0x1F`. Different instructions allow different ranges of immediate values; the assembler will automatically select the most compact way of encoding a value. The second type, register, denotes the contents of a register, one of the sixteen 8-, 4-, 2-, or 1-byte low-order portions of the registers for operands having 64, 32, 16, or 8 bits, respectively. In Figure 3.3, we use the notation r_a to denote an arbitrary register a and indicate its value with the reference R[r_a], viewing the set of registers as an array R indexed by register identifiers.

The third type of operand is a memory reference, in which we access some memory location according to a computed address, often called the effective address. Since we view the memory as a large array of bytes, we use the notation M_b[

will see, the more complex addressing modes are useful when referencing array and structure elements.

Practice Problem 3.1 (solution page 361)

Assume the following values are stored at the indicated memory addresses and registers:

Address	Value	Register	Value
0x100	0xFF	%rax	0x100
0x104	0xAB	%rcx	0x1
0x108	0x13	%rdx	0x3
0x10C	0x11		

Fill in the following table showing the values for the indicated operands:

Operand	Value
%rax	_____
0x104	_____
$0x108	_____
(%rax)	_____
4(%rax)	_____
9(%rax,%rdx)	_____
260(%rcx,%rdx)	_____
0xFC(,%rcx,4)	_____
(%rax,%rdx,4)	_____

3.4.2 Data Movement Instructions

Among the most heavily used instructions are those that copy data from one location to another. The generality of the operand notation allows a simple data movement instruction to express a range of possibilities that in many machines would require a number of different instructions. We present a number of different data movement instructions, differing in their source and destination types, what conversions they perform, and other side effects they may have. In our presentation, we group the many different instructions into instruction classes, where the instructions in a class perform the same operation but with different operand sizes.

Figure 3.4 lists the simplest form of data movement instructions—MOV class. These instructions copy data from a source location to a destination location, without any transformation. The class consists of four instructions: `movb`, `movw`, `movl`, and `movq`. All four of these instructions have similar effects; they differ primarily in that they operate on data of different sizes: 1, 2, 4, and 8 bytes, respectively.

Instruction		Effect	Description
MOV	S, D	D ← S	Move
`movb`			Move byte
`movw`			Move word
`movl`			Move double word
`movq`			Move quad word
`movabsq`	I, R	R ← I	Move absolute quad word

Figure 3.4 Simple data movement instructions.

The source operand designates a value that is immediate, stored in a register, or stored in memory. The destination operand designates a location that is either a register or a memory address. x86-64 imposes the restriction that a move instruction cannot have both operands refer to memory locations. Copying a value from one memory location to another requires two instructions—the first to load the source value into a register, and the second to write this register value to the destination. Referring to Figure 3.2, register operands for these instructions can be the labeled portions of any of the 16 registers, where the size of the register must match the size designated by the last character of the instruction ('`b`', '`w`', '`l`', or '`q`'). For most cases, the MOV instructions will only update the specific register bytes or memory locations indicated by the destination operand. The only exception is that when `movl` has a register as the destination, it will also set the high-order 4 bytes of the register to 0. This exception arises from the convention, adopted in x86-64, that any instruction that generates a 32-bit value for a register also sets the high-order portion of the register to 0.

The following MOV instruction examples show the five possible combinations of source and destination types. Recall that the source operand comes first and the destination second.

```
1      movl $0x4050,%eax          Immediate--Register, 4 bytes
2      movw %bp,%sp               Register--Register,  2 bytes
3      movb (%rdi,%rcx),%al       Memory--Register,    1 byte
4      movb $-17,(%esp)           Immediate--Memory,   1 byte
5      movq %rax,-12(%rbp)        Register--Memory,    8 bytes
```

A final instruction documented in Figure 3.4 is for dealing with 64-bit immediate data. The regular `movq` instruction can only have immediate source operands that can be represented as 32-bit two's-complement numbers. This value is then sign extended to produce the 64-bit value for the destination. The `movabsq` instruction can have an arbitrary 64-bit immediate value as its source operand and can only have a register as a destination.

Figures 3.5 and 3.6 document two classes of data movement instructions for use when copying a smaller source value to a larger destination. All of these instructions copy data from a source, which can be either a register or stored

Aside Understanding data movement instructions

As described, there are two different conventions regarding whether and how data movement instructions modify the upper bytes of a destination register. This distinction is illustrated by the following code sequence:

```
1    movabsq $0x0011223344556677, %rax    %rax = 0011223344556677
2    movb    $-1, %al                     %rax = 00112233445566FF
3    movw    $-1, %ax                     %rax = 001122334455FFFF
4    movl    $-1, %eax                    %rax = 00000000FFFFFFFF
5    movq    $-1, %rax                    %rax = FFFFFFFFFFFFFFFF
```

In the following discussion, we use hexadecimal notation. In the example, the instruction on line 1 initializes register `%rax` to the pattern `0011223344556677`. The remaining instructions have immediate value -1 as their source values. Recall that the hexadecimal representation of -1 is of the form `FF`$\cdots$`F`, where the number of `F`'s is twice the number of bytes in the representation. The `movb` instruction (line 2) therefore sets the low-order byte of `%rax` to `FF`, while the `movw` instruction (line 3) sets the low-order 2 bytes to `FFFF`, with the remaining bytes unchanged. The `movl` instruction (line 4) sets the low-order 4 bytes to `FFFFFFFF`, but it also sets the high-order 4 bytes to `00000000`. Finally, the `movq` instruction (line 5) sets the complete register to `FFFFFFFFFFFFFFFF`.

Instruction	Effect	Description
MOVZ S, R	R ← ZeroExtend(S)	Move with zero extension
`movzbw`		Move zero-extended byte to word
`movzbl`		Move zero-extended byte to double word
`movzwl`		Move zero-extended word to double word
`movzbq`		Move zero-extended byte to quad word
`movzwq`		Move zero-extended word to quad word

Figure 3.5 Zero-extending data movement instructions. The instruction can have a register or memory location as its source operand and a register as its destination.

in memory, to a register destination. Instructions in the MOVZ class fill out the remaining bytes of the destination with zeros, while those in the MOVS class fill them out by sign extension, replicating copies of the most significant bit of the source operand. Observe that each instruction name has size designators as its final two characters—the first specifying the source size, and the second specifying the destination size. As can be seen, there are three instructions in each of these classes, covering all cases of 1- and 2-byte source sizes and 2- and 4-byte destination sizes, considering only cases where the destination is larger than the source, of course.

Instruction	Effect	Description
MOVS S, R	R ← Sign Extend(S)	Move with sign extension
movsbw		Move sign-extended byte to word
movsbl		Move sign-extended byte to double word
movswl		Move sign-extended word to double word
movsbq		Move sign-extended byte to quad word
movswq		Move sign-extended word to quad word
movslq		Move sign-extended double word to quad word
cltq	%rax ← Sign Extend(%eax)	Sign-extend %eax to %rax

Figure 3.6 Sign-extending data movement instructions. The movs instructions have arguments specifying source and destination formats. The cltq instruction is specific to the %eax and %rax registers.

Note the absence of an explicit instruction to zero-extend a 4-byte source value to an 8-byte destination in Figure 3.5. Such an instruction would logically be named `movzlq`, but this instruction does not exist. Instead, this type of data movement can be implemented using a `movl` instruction having a register as the destination. This technique takes advantage of the property that an instruction generating a 4-byte value with a register as the destination will fill the upper 4 bytes with zeros. Otherwise, for 64-bit destinations, moving with sign extension is supported for all three source types, and moving with zero extension is supported for the two smaller source types.

Figure 3.6 also documents the `cltq` instruction. This instruction has no operands—it always uses register `%eax` as its source and `%rax` as the destination for the sign-extended result. It therefore has the exact same effect as the instruction `movslq %eax, %rax`, but it has a more compact encoding.

Practice Problem 3.2 (solution page 361)

For each of the following lines of assembly language, determine the appropriate instruction suffix based on the operands. (For example, `mov` can be rewritten as `movb`, `movw`, `movl`, or `movq`.)

```
mov__   %eax, (%rsp)
mov__   (%rax), %dx
mov__   $0xFF, %bl
mov__   (%rsp,%rdx,4), %dl
mov__   (%rdx), %rax
mov__   %dx, (%rax)
```

Aside C o ari t o t i tr ti o

The following example illustrates how different data movement instructions either do or do not change the high-order bytes of the destination. Observe that the three byte-movement instructions `movb`, `movsbq`, and `movzbq` differ from each other in subtle ways. Here is an example:

```
1     movabsq $0x0011223344556677, %rax    %rax = 0011223344556677
2     movb    $0xAA, %dl                   %dl  = AA
3     movb %dl,%al                         %rax = 00112233445566AA
4     movsbq %dl,%rax                      %rax = FFFFFFFFFFFFFFAA
5     movzbq %dl,%rax                      %rax = 00000000000000AA
```

In the following discussion, we use hexadecimal notation for all of the values. The first two lines of the code initialize registers `%rax` and `%dl` to `0011223344556677` and `AA`, respectively. The remaining instructions all copy the low-order byte of `%rdx` to the low-order byte of `%rax`. The `movb` instruction (line 3) does not change the other bytes. The `movsbq` instruction (line 4) sets the other 7 bytes to either all ones or all zeros depending on the high-order bit of the source byte. Since hexadecimal `A` represents binary value `1010`, sign extension causes the higher-order bytes to each be set to `FF`. The `movzbq` instruction (line 5) always sets the other 7 bytes to zero.

Practice Problem 3.3 (solution page 362)

Each of the following lines of code generates an error message when we invoke the assembler. Explain what is wrong with each line.

```
movb $0xF, (%ebx)
movl %rax, (%rsp)
movw (%rax),4(%rsp)
movb %al,%sl
movq %rax,$0x123
movl %eax,%rdx
movb %si, 8(%rbp)
```

3.4.3 Data Movement Example

As an example of code that uses data movement instructions, consider the data exchange routine shown in Figure 3.7, both as C code and as assembly code generated by GCC.

As Figure 3.7(b) shows, function `exchange` is implemented with just three instructions: two data movements (`movq`) plus an instruction to return back to the point from which the function was called (`ret`). We will cover the details of function call and return in Section 3.7. Until then, it suffices to say that arguments are passed to functions in registers. Our annotated assembly code documents these. A function returns a value by storing it in register `%rax`, or in one of the low-order portions of this register.

(a) C

```
long exchange(long *xp, long y)
{
    long x = *xp;
    *xp = y;
    return x;
}
```

( ) A l

```
    long exchange(long *xp, long y)
    xp in %rdi, y in %rsi
1   exchange:
2     movq    (%rdi), %rax     Get x at xp. Set as return value.
3     movq    %rsi, (%rdi)     Store y at xp.
4     ret                      Return.
```

Figure 3.7 C and assembly code for exchange routine. R it r %rdi a %rsi ara t r xp a y, r ti l .

When the procedure begins execution, procedure parameters `xp` and `y` are stored in registers `%rdi` and `%rsi`, respectively. Instruction 2 then reads `x` from memory and stores the value in register `%rax`, a direct implementation of the operation `x = *xp` in the C program. Later, register `%rax` will be used to return a value from the function, and so the return value will be

New to C? Pointers

Function exchange (Figure 3.7(a)) provides a good illustration of the use of pointers in C. Argument xp is a pointer to a long integer, while y is a long integer itself. The statement

```
long x = *xp;
```

indicates that we should read the value stored in the location designated by xp and store it as a local variable named x. This read operation is known as pointer dereferencing. The C operator '*' performs pointer dereferencing.

The statement

```
*xp = y;
```

does the reverse—it writes the value of parameter y at the location designated by xp. This is also a form of pointer dereferencing (and hence the operator *), but it indicates a write operation since it is on the left-hand side of the assignment.

The following is an example of exchange in action:

```
long a = 4;
long b = exchange(&a, 3);
printf("a = %ld, b = %ld\verb@\@n", a, b);
```

This code will print

```
a = 3, b = 4
```

The C operator '&' (called the "address of" operator) creates a pointer, in this case to the location holding local variable a. Function exchange overwrites the value stored in a with 3 but returns the previous value, 4, as the function value. Observe how by passing a pointer to exchange, it could modify data held at some remote location.

Assume that the values of sp and dp are stored in registers %rdi and %rsi, respectively. For each entry in the table, show the two instructions that implement the specified data movement. The first instruction in the sequence should read from memory, do the appropriate conversion, and set the appropriate portion of register %rax. The second instruction should then write the appropriate portion of %rax to memory. In both cases, the portions may be %rax, %eax, %ax, or %al, and they may differ from one another.

Recall that when performing a cast that involves both a size change and a change of "signedness" in C, the operation should change the size first (Section 2.2.6).

src_t	dest_t	Instruction
long	long	movq (%rdi), %rax movq %rax, (%rsi)
char	int	_____ _____

`char`	`unsigned`	
`unsigned char`	`long`	
`int`	`char`	
`unsigned`	`unsigned char`	
`char`	`short`	

Practice Problem 3.5 (solution page 363)

You are given the following information. A function with prototype

```
void decode1(long *xp, long *yp, long *zp);
```

is compiled into assembly code, yielding the following:

```
  void decode1(long *xp, long *yp, long *zp)
  xp in %rdi, yp in %rsi, zp in %rdx
decode1:
  movq    (%rdi), %r8
  movq    (%rsi), %rcx
  movq    (%rdx), %rax
  movq    %r8, (%rsi)
  movq    %rcx, (%rdx)
  movq    %rax, (%rdi)
  ret
```

Parameters `xp`, `yp`, and `zp` are stored in registers `%rdi`, `%rsi`, and `%rdx`, respectively.

Write C code for `decode1` that will have an effect equivalent to the assembly code shown.

3.4.4 Pushing and Popping Stack Data

The final two data movement operations are used to push data onto and pop data from the program stack, as documented in Figure 3.8. As we will see, the stack plays a vital role in the handling of procedure calls. By way of background, a stack is a data structure where values can be added or deleted, but only according to a "last-in, first-out" discipline. We add data to a stack via a push operation and remove it via a pop operation, with the property that the value popped will always be the value that was most recently pushed and is still on the stack. A stack can be implemented as an array, where we always insert and remove elements from one

Instruction	Effect	Description
`pushq S`	R[%rsp] ← R[%rsp] − 8; M[R[%rsp]] ← S	Push quad word
`popq D`	D ← M[R[%rsp]]; R[%rsp] ← R[%rsp] + 8	Pop quad word

Figure 3.8 Push and pop instructions.

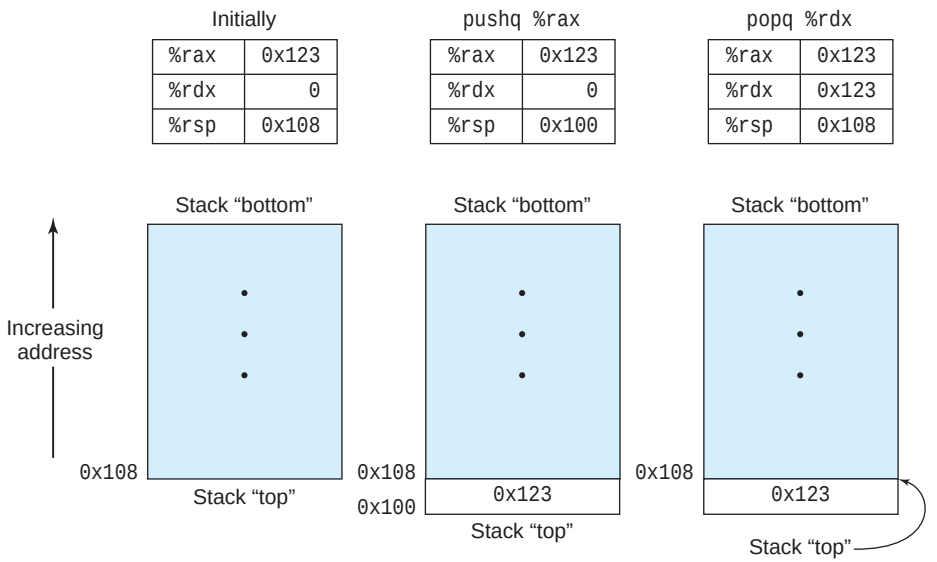


Figure 3.9 Illustration of stack operation. By convention, we draw stacks upside down, with the top of the stack at the bottom of the figure. With x86-64, the stack pointer is stored in register %rsp.

end of the array. This end is called the top of the stack. With x86-64, the program stack is stored in some region of memory. As illustrated in Figure 3.9, the stack grows downward such that the top element of the stack has the lowest address of all stack elements. (By convention, we draw stacks upside down, with the stack "top" shown at the bottom of the figure.) The stack pointer `%rsp` holds the address of the top stack element.

The `pushq` instruction provides the ability to push data onto the stack, while the `popq` instruction pops it. Each of these instructions takes a single operand—the data source for pushing and the data destination for popping.

Pushing a quad word value onto the stack involves first decrementing the stack pointer by 8 and then writing the value at the new top-of-stack address.

Therefore, the behavior of the instruction `pushq %rbp` is equivalent to that of the pair of instructions

```
subq $8,%rsp            Decrement stack pointer
movq %rbp,(%rsp)        Store %rbp on stack
```

except that the `pushq` instruction is encoded in the machine code as a single byte, whereas the pair of instructions shown above requires a total of 8 bytes. The first two columns in Figure 3.9 illustrate the effect of executing the instruction `pushq %rax` when `%rsp` is `0x108` and `%rax` is `0x123`. First `%rsp` is decremented by 8, giving `0x100`, and then `0x123` is stored at memory address `0x100`.

Popping a quad word involves reading from the top-of-stack location and then incrementing the stack pointer by 8. Therefore, the instruction `popq %rax` is equivalent to the following pair of instructions:

```
movq (%rsp),%rax        Read %rax from stack
addq $8,%rsp            Increment stack pointer
```

The third column of Figure 3.9 illustrates the effect of executing the instruction `popq %edx` immediately after executing the `pushq`. Value `0x123` is read from memory and written to register `%rdx`. Register `%rsp` is incremented back to `0x108`. As shown in the figure, the value `0x123` remains at memory location `0x104` until it is overwritten (e.g., by another push operation). However, the stack top is always considered to be the address indicated by `%rsp`.

Since the stack is contained in the same memory as the program code and other forms of program data, programs can access arbitrary positions within the stack using the standard memory addressing methods. For example, assuming the topmost element of the stack is a quad word, the instruction `movq 8(%rsp),%rdx` will copy the second quad word from the stack to register `%rdx`.

3.5 Arithmetic and Logical Operations

Figure 3.10 lists some of the x86-64 integer and logic operations. Most of the operations are given as instruction classes, as they can have different variants with different operand sizes. (Only `leaq` has no other size variants.) For example, the instruction class ADD consists of four addition instructions: `addb`, `addw`, `addl`, and `addq`, adding bytes, words, double words, and quad words, respectively. Indeed, each of the instruction classes shown has instructions for operating on these four different sizes of data. The operations are divided into four groups: load effective address, unary, binary, and shifts. Binary operations have two operands, while unary operations have one operand. These operands are specified using the same notation as described in Section 3.4.

3.5.1 Load Effective Address

The load effective address instruction `leaq` is actually a variant of the `movq` instruction. It has the form of an instruction that reads from memory to a register,

Instruction		Effect		Description
leaq	S, D	D ← &S		Load effective address
INC	D	D ← D+1		Increment
DEC	D	D ← D−1		Decrement
NEG	D	D ← -D		Negate
NOT	D	D ← ~D		Complement
ADD	S, D	D ← D + S		Add
SUB	S, D	D ← D − S		Subtract
IMUL	S, D	D ← D * S		Multiply
XOR	S, D	D ← D ^ S		Exclusive-or
OR	S, D	D ← D | S		Or
AND	S, D	D ← D & S		And
SAL	k, D	D ← D << k		Left shift
SHL	k, D	D ← D << k		Left shift (same as SAL)
SAR	k, D	D ← D >>$_A$ k		Arithmetic right shift
SHR	k, D	D ← D >>$_L$ k		Logical right shift

Figure 3.10 Integer arithmetic operations. The load effective address (leaq) instruction is commonly used to perform simple arithmetic. The remaining ones are unary or binary operations, where we distinguish between arithmetic and logical right shifts. We use the notation >>$_A$ and >>$_L$ to denote arithmetic and logical right shift, respectively. Note the peculiar syntax inherited from ATT-format data.

but it does not reference memory at all. Its first operand appears to be a memory reference, but instead of reading from the designated location, the instruction copies the effective address to the destination. We indicate this computation in Figure 3.10 using the C address operator &S. This instruction can be used to generate pointers for later memory references. In addition, it can be used to compactly describe common arithmetic operations. For example, if register %rdx contains value x, then the instruction leaq 7(%rdx,%rdx,4), %rax will set register %rax to 5x + 7. Compilers often find clever uses of leaq that have nothing to do with effective address computations. The destination operand must be a register.

Practice Problem 3.6 (solution page 363)

Suppose register %rbx holds value p and %rdx holds value q. Fill in the table below with formulas indicating the value that will be stored in register %rax for each of the given assembly-code instructions:

Instruction	Result
leaq 9(%rdx), %rax	_____
leaq (%rdx,%rbx), %rax	_____
leaq (%rdx,%rbx,3), %rax	_____
leaq 2(%rbx,%rbx,7), %rax	_____

```
leaq 0xE(,%rdx,3), %rax      _____
leaq 6(%rbx,%rdx,7), %rax    _____
```

As an illustration of the use of `leaq` in compiled code, consider the following C program:

```
long scale(long x, long y, long z) {
    long t = x + 4 * y + 12 * z;
    return t;
}
```

When compiled, the arithmetic operations of the function are implemented by a sequence of three `leaq` functions, as is documented by the comments on the right-hand side:

```
  long scale(long x, long y, long z)
  x in %rdi, y in %rsi, z in %rdx
scale:
  leaq    (%rdi,%rsi,4), %rax     x + 4*y
  leaq    (%rdx,%rdx,2), %rdx     z + 2*z = 3*z
  leaq    (%rax,%rdx,4), %rax     (x+4*y) + 4*(3*z) = x + 4*y + 12*z
  ret
```

The ability of the `leaq` instruction to perform addition and limited forms of multiplication proves useful when compiling simple arithmetic expressions such as this example.

Practice Problem 3.7 (solution page 364)

Consider the following code, in which we have omitted the expression being computed:

```
short scale3(short x, short y, short z) {
  short t = _____;
  return t;
}
```

Compiling the actual function with GCC yields the following assembly code:

```
  short scale3(short x, short y, short z)
  x in %rdi, y in %rsi, z in %rdx
scale3:
  leaq (%rsi,%rsi,9), %rbx
  leaq (%rbx,%rdx), %rbx
  leaq (%rbx,%rdi,%rsi), %rbx
  ret
```

Fill in the missing expression in the C code.

3.5.2 Unary and Binary Operations

Operations in the second group are unary operations, with the single operand serving as both source and destination. This operand can be either a register or a memory location. For example, the instruction `incq (%rsp)` causes the 8-byte element on the top of the stack to be incremented. This syntax is reminiscent of the C increment (`++`) and decrement (`--`) operators.

The third group consists of binary operations, where the second operand is used as both a source and a destination. This syntax is reminiscent of the C assignment operators, such as `x -= y`. Observe, however, that the source operand is given first and the destination second. This looks peculiar for noncommutative operations. For example, the instruction `subq %rax,%rdx` decrements register `%rdx` by the value in `%rax`. (It helps to read the instruction as "Subtract `%rax` from `%rdx`.") The first operand can be either an immediate value, a register, or a memory location. The second can be either a register or a memory location. As with the MOV instructions, the two operands cannot both be memory locations. Note that when the second operand is a memory location, the processor must read the value from memory, perform the operation, and then write the result back to memory.

Practice Problem 3.8 (solution page 364)

Assume the following values are stored at the indicated memory addresses and registers:

Address	Value	Register	Value
0x100	0xFF	%rax	0x100
0x108	0xAB	%rcx	0x1
0x110	0x13	%rdx	0x3
0x118	0x11		

Fill in the following table showing the effects of the following instructions, in terms of both the register or memory location that will be updated and the resulting value:

Instruction	Destination	Value
`addq %rcx,(%rax)`		
`subq %rdx,8(%rax)`		
`imulq $16,(%rax,%rdx,8)`		
`incq 16(%rax)`		
`decq %rcx`		
`subq %rdx,%rax`		

3.5.3 Shift Operations

The final group consists of shift operations, where the shift amount is given first and the value to shift is given second. Both arithmetic and logical right shifts are

possible. The different shift instructions can specify the shift amount either as an immediate value or with the single-byte register `%cl`. (These instructions are unusual in only allowing this specific register as the operand.) In principle, having a 1-byte shift amount would make it possible to encode shift amounts ranging up to $2^8 - 1 = 255$. With x86-64, a shift instruction operating on data values that are `w` bits long determines the shift amount from the low-order `m` bits of register `%cl`, where $2^m = w$. The higher-order bits are ignored. So, for example, when register `%cl` has hexadecimal value `0xFF`, then instruction `salb` would shift by 7, while `salw` would shift by 15, `sall` would shift by 31, and `salq` would shift by 63.

As Figure 3.10 indicates, there are two names for the left shift instruction: SAL and SHL. Both have the same effect, filling from the right with zeros. The right shift instructions differ in that SAR performs an arithmetic shift (fill with copies of the sign bit), whereas SHR performs a logical shift (fill with zeros). The destination operand of a shift operation can be either a register or a memory location. We denote the two different right shift operations in Figure 3.10 as $>>_A$ (arithmetic) and $>>_L$ (logical).

Practice Problem 3.9 (solution page 364)

Suppose we want to generate assembly code for the following C function:

```
long shift_left4_rightn(long x, long n)
{
    x <<= 4;
    x >>= n;
    return x;
}
```

The code that follows is a portion of the assembly code that performs the actual shifts and leaves the final value in register `%rax`. Two key instructions have been omitted. Parameters `x` and `n` are stored in registers `%rdi` and `%rsi`, respectively.

```
  long shift_left4_rightn(long x, long n)
  x in %rdi, n in %rsi
shift_left4_rightn:
  movq    %rdi, %rax      Get x
  _____________________   x <<= 4
  movl    %esi, %ecx      Get n (4 bytes)
  _____________________   x >>= n
```

Fill in the missing instructions, following the annotations on the right. The right shift should be performed arithmetically.

(a) C

```
long arith(long x, long y, long z)
{
    long t1 = x ^ y;
    long t2 = z * 48;
    long t3 = t1 & 0x0F0F0F0F;
    long t4 = t2 - t3;
    return t4;
}
```

( ) A l

```
    long arith(long x, long y, long z)
    x in %rdi, y in %rsi, z in %rdx
1   arith:
2     xorq    %rsi, %rdi              t1 = x ^ y
3     leaq    (%rdx,%rdx,2), %rax     3*z
4     salq    $4, %rax                t2 = 16 * (3*z) = 48*z
5     andl    $252645135, %edi        t3 = t1 & 0x0F0F0F0F
6     subq    %rdi, %rax              Return t2 - t3
7     ret
```

Figure 3.11 C and assembly code for arithmetic function.

3.5.4 Discussion

We see that most of the instructions shown in Figure 3.10 can be used for either unsigned or two's-complement arithmetic. Only right shifting requires instructions that differentiate between signed versus unsigned data. This is one of the features that makes two's-complement arithmetic the preferred way to implement signed integer arithmetic.

Figure 3.11 shows an example of a function that performs arithmetic operations and its translation into assembly code. Arguments `x`, `y`, and `z` are initially stored in registers `%rdi`, `%rsi`, and `%rdx`, respectively. The assembly-code instructions correspond closely with the lines of C source code. Line 2 computes the value of `x^y`. Lines 3 and 4 compute the expression `z*48` by a combination of `leaq` and shift instructions. Line 5 computes the AND of `t1` and `0x0F0F0F0F`. The final subtraction is computed by line 6. Since the destination of the subtraction is register `%rax`, this will be the value returned by the function.

In the assembly code of Figure 3.11, the sequence of values in register `%rax` corresponds to program values `3*z`, `z*48`, and `t4` (as the return value). In general, compilers generate code that uses individual registers for multiple program values and moves program values among the registers.

Practice Problem 3.10 (solution page 365)

Consider the following code, in which we have omitted the expression being computed:

```
short arith3(short x, short y, short z)
{
    short p1 = _________;
    short p2 = _________;
    short p3 = _________;
    short p4 = _________;
    return p4;
}
```

The portion of the generated assembly code implementing these expressions is as follows:

```
  short arith3(short x, short y, short z)
  x in %rdi, y in %rsi, z in %rdx
arith3:
  orq     %rsi, %rdx
  sarq    $9, %rdx
  notq    %rdx
  movq    %rdx, %bax
  subq    %rsi, %rbx
  ret
```

Based on this assembly code, fill in the missing portions of the C code.

Practice Problem 3.11 (solution page 365)

It is common to find assembly-code lines of the form

```
xorq %rcx,%rcx
```

in code that was generated from C where no EXCLUSIVE-OR operations were present.

A. Explain the effect of this particular EXCLUSIVE-OR instruction and what useful operation it implements.

B. What would be the more straightforward way to express this operation in assembly code?

C. Compare the number of bytes to encode any two of these three different implementations of the same operation.

3.5.5 Special Arithmetic Operations

As we saw in Section 2.3, multiplying two 64-bit signed or unsigned integers can yield a product that requires 128 bits to represent. The x86-64 instruction set provides limited support for operations involving 128-bit (16-byte) numbers. Continuing with the naming convention of word (2 bytes), double word (4 bytes), and quad word (8 bytes), Intel refers to a 16-byte quantity as an oct word. Figure 3.12

Instruction	Effect	Description
`imulq S`	R[%rdx]:R[%rax] ← S × R[%rax]	Signed full multiply
`mulq S`	R[%rdx]:R[%rax] ← S × R[%rax]	Unsigned full multiply
`cqto`	R[%rdx]:R[%rax] ← Sign Extend(R[%rax])	Convert to oct word
`idivq S`	R[%rdx] ← R[%rdx]:R[%rax] mod S; R[%rax] ← R[%rdx]:R[%rax] ÷ S	Signed divide
`divq S`	R[%rdx] ← R[%rdx]:R[%rax] mod S; R[%rax] ← R[%rdx]:R[%rax] ÷ S	Unsigned divide

Figure 3.12 Special arithmetic operations. These operations provide full 128-bit multiplication and division, both for signed and unsigned numbers. The pair of registers %rdx and %rax are viewed as forming a single 128-bit oct word.

describes instructions that support generating the full 128-bit product of two 64-bit numbers, as well as integer division.

The `imulq` instruction has two different forms One form, shown in Figure 3.10, is as a member of the IMUL instruction class. In this form, it serves as a "two-operand" multiply instruction, generating a 64-bit product from two 64-bit operands. It implements the operations $*^{u}_{64}$ and $*^{t}_{64}$ described in Sections 2.3.4 and 2.3.5. (Recall that when truncating the product to 64 bits, both unsigned multiply and two's-complement multiply have the same bit-level behavior.)

Additionally, the x86-64 instruction set includes two different "one-operand" multiply instructions to compute the full 128-bit product of two 64-bit values—one for unsigned (`mulq`) and one for two's-complement (`imulq`) multiplication. For both of these instructions, one argument must be in register `%rax`, and the other is given as the instruction source operand. The product is then stored in registers `%rdx` (high-order 64 bits) and `%rax` (low-order 64 bits). Although the name `imulq` is used for two distinct multiplication operations, the assembler can tell which one is intended by counting the number of operands.

As an example, the following C code demonstrates the generation of a 128-bit product of two unsigned 64-bit numbers `x` and `y`:

```
#include <inttypes.h>

typedef unsigned __int128 uint128_t;

void store_uprod(uint128_t *dest, uint64_t x, uint64_t y) {
    *dest = x * (uint128_t) y;
}
```

In this program, we explicitly declare `x` and `y` to be 64-bit numbers, using definitions declared in the file `inttypes.h`, as part of an extension of the C standard. Unfortunately, this standard does not make provisions for 128-bit values. Instead,

we rely on support provided by GCC for 128-bit integers, declared using the name `__int128`. Our code uses a `typedef` declaration to define data type `uint128_t`, following the naming pattern for other data types found in `inttypes.h`. The code specifies that the resulting product should be stored at the 16 bytes designated by pointer `dest`.

The assembly code generated by GCC for this function is as follows:

```
     void store_uprod(uint128_t *dest, uint64_t x, uint64_t y)
     dest in %rdi, x in %rsi, y in %rdx
1    store_uprod:
2      movq    %rsi, %rax       Copy x to multiplicand
3      mulq    %rdx             Multiply by y
4      movq    %rax, (%rdi)     Store lower 8 bytes at dest
5      movq    %rdx, 8(%rdi)    Store upper 8 bytes at dest+8
6      ret
```

Observe that storing the product requires two `movq` instructions: one for the low-order 8 bytes (line 4), and one for the high-order 8 bytes (line 5). Since the code is generated for a little-endian machine, the high-order bytes are stored at higher addresses, as indicated by the address specification `8(%rdi)`.

Our earlier table of arithmetic operations (Figure 3.10) does not list any division or modulus operations. These operations are provided by the single-operand divide instructions similar to the single-operand multiply instructions. The signed division instruction `idivl` takes as its dividend the 128-bit quantity in registers `%rdx` (high-order 64 bits) and `%rax` (low-order 64 bits). The divisor is given as the instruction operand. The instruction stores the quotient in register `%rax` and the remainder in register `%rdx`.

For most applications of 64-bit addition, the dividend is given as a 64-bit value. This value should be stored in register `%rax`. The bits of `%rdx` should then be set to either all zeros (unsigned arithmetic) or the sign bit of `%rax` (signed arithmetic). The latter operation can be performed using the instruction `cqto`.[2] This instruction takes no operands—it implicitly reads the sign bit from `%rax` and copies it across all of `%rdx`.

As an illustration of the implementation of division with x86-64, the following C function computes the quotient and remainder of two 64-bit, signed numbers:

```
void remdiv(long x, long y,
            long *qp, long *rp) {
    long q = x/y;
    long r = x%y;
    *qp = q;
    *rp = r;
}
```

2. This instruction is called `cqo` in the Intel documentation, one of the few cases where the ATT-format name for an instruction does not match the Intel name.

This compiles to the following assembly code:

```
    void remdiv(long x, long y, long *qp, long *rp)
    x in %rdi, y in %rsi, qp in %rdx, rp in %rcx
1   remdiv:
2     movq    %rdx, %r8       Copy qp
3     movq    %rdi, %rax      Move x to lower 8 bytes of dividend
4     cqto                    Sign-extend to upper 8 bytes of dividend
5     idivq   %rsi            Divide by y
6     movq    %rax, (%r8)     Store quotient at qp
7     movq    %rdx, (%rcx)    Store remainder at rp
8     ret
```

In this code, argument `rp` must first be saved in a different register (line 2), since argument register `%rdx` is required for the division operation. Lines 3–4 then prepare the dividend by copying and sign-extending `x`. Following the division, the quotient in register `%rax` gets stored at `qp` (line 6), while the remainder in register `%rdx` gets stored at `rp` (line 7).

Unsigned division makes use of the `divq` instruction. Typically, register `%rdx` is set to zero beforehand.

Practice Problem 3.12 (solution page 365)

Consider the following function for computing the quotient and remainder of two unsigned 64-bit numbers:

```
void uremdiv(unsigned long x, unsigned long y,
             unsigned long *qp, unsigned long *rp) {
    unsigned long q = x/y;
    unsigned long r = x%y;
    *qp = q;
    *rp = r;
}
```

Modify the assembly code shown for signed division to implement this function.

3.6 Control

So far, we have only considered the behavior of straight-line code, where instructions follow one another in sequence. Some constructs in C, such as conditionals, loops, and switches, require conditional execution, where the sequence of operations that get performed depends on the outcomes of tests applied to the data. Machine code provides two basic low-level mechanisms for implementing conditional behavior: it tests data values and then alters either the control flow or the data flow based on the results of these tests.

Data-dependent control flow is the more general and more common approach for implementing conditional behavior, and so we will examine this first. Normally,

both statements in C and instructions in machine code are executed sequentially, in the order they appear in the program. The execution order of a set of machine-code instructions can be altered with a jump instruction, indicating that control should pass to some other part of the program, possibly contingent on the result of some test. The compiler must generate instruction sequences that build upon this low-level mechanism to implement the control constructs of C.

In our presentation, we first cover the two ways of implementing conditional operations. We then describe methods for presenting loops and `switch` statements.

3.6.1 Condition Codes

In addition to the integer registers, the CPU maintains a set of single-bit condition code registers describing attributes of the most recent arithmetic or logical operation. These registers can then be tested to perform conditional branches. These condition codes are the most useful:

`CF`: Carry flag. The most recent operation generated a carry out of the most significant bit. Used to detect overflow for unsigned operations.

`ZF`: Zero flag. The most recent operation yielded zero.

`SF`: Sign flag. The most recent operation yielded a negative value.

`OF`: Overflow flag. The most recent operation caused a two's-complement overflow—either negative or positive.

For example, suppose we used one of the ADD instructions to perform the equivalent of the C assignment `t = a+b`, where variables `a`, `b`, and `t` are integers. Then the condition codes would be set according to the following C expressions:

`CF`	`(unsigned) t < (unsigned) a`	Unsigned overflow
`ZF`	`(t == 0)`	Zero
`SF`	`(t < 0)`	Negative
`OF`	`(a < 0 == b < 0) && (t < 0 != a < 0)`	Signed overflow

The `leaq` instruction does not alter any condition codes, since it is intended to be used in address computations. Otherwise, all of the instructions listed in Figure 3.10 cause the condition codes to be set. For the logical operations, such as XOR, the carry and overflow flags are set to zero. For the shift operations, the carry flag is set to the last bit shifted out, while the overflow flag is set to zero. For reasons that we will not delve into, the INC and DEC instructions set the overflow and zero flags, but they leave the carry flag unchanged.

In addition to the setting of condition codes by the instructions of Figure 3.10, there are two instruction classes (having 8-, 16-, 32-, and 64-bit forms) that set condition codes without altering any other registers; these are listed in Figure 3.13. The CMP instructions set the condition codes according to the differences of their two operands. They behave in the same way as the SUB instructions, except that they set the condition codes without updating their destinations. With ATT format,

Aside Tra i t ti o a je i tr ti

Let us trace the processing of the `je` instruction on line 8 of the object code shown in Figure 4.17. The condition codes were all set to zero by the `subq` instruction (line 3), and so the branch will not be taken. The instruction is located at address `0x02e` and consists of 9 bytes. The first has value `0x73`, while the remaining 8 bytes are a byte-reversed version of the number `0x0000000000000040`, the jump target. The stages would proceed as follows:

Stage	Generic jXX D t	Specific je 0x040
Fetch	i : i ← M_1[PC] alC ← M_8[PC + 1] alP ← PC + 9	i : i ← M_1[0x02e] = 7:3 alC ← M_8[0x02f] = 0x040 alP ← 0x02e + 9 = 0x037
Decode		
Execute	C ← C (CC, i)	C ← C (0, 0, 0 , 3) = 0
Memory		
Write back		
PC update	PC ← C ? alC : alP	PC ← 0 ? 0x040 : 0x037 = 0x037

As this trace shows, the instruction has the effect of incrementing the PC by 9.

Instructions `call` and `ret` bear some similarity to instructions `pushq` and `popq`, except that we push and pop program counter values. With instruction `call`, we push alP, the address of the instruction that follows the `call` instruction. During the PC update stage, we set the PC to alC, the call destination. With instruction `ret`, we assign alM, the value popped from the stack, to the PC in the PC update stage.

Practice Problem 4.18 (solution page 523)

Fill in the right-hand column of the following table to describe the processing of the `call` instruction on line 9 of the object code in Figure 4.17:

Stage	Generic call D t	Specific call 0x041
Fetch	i : i ← M_1[PC] alC ← M_8[PC + 1] alP ← PC + 9	

Aside Tra i t ti o a ret i tr ti

Let us trace the processing of the `ret` instruction on line 13 of the object code shown in Figure 4.17. The instruction address is `0x041` and is encoded by a single byte `0x90`. The previous `call` instruction set `%rsp` to 120 and stored the return address `0x040` at memory address 120. The stages would proceed as follows:

Stage	Generic `ret`	Specific `ret`
Fetch	i : i ← M_1[PC]	i : i ← M_1[0x041] = 9:0
	alP ← PC + 1	alP ← 0x041 + 1 = 0x042
Decode	alA ← R[%rsp]	alA ← R[%rsp] = 120
	alB ← R[%rsp]	alB ← R[%rsp] = 120
Execute	alE ← alB + 8	alE ← 120 + 8 = 128
Memory	alM ← M_8[alA]	alM ← M_8[120] = 0x040
Write back	R[%rsp] ← alE	R[%rsp] ← 128
PC update	PC ← alM	PC ← 0x040

As this trace shows, the instruction has the effect of setting the PC to `0x040`, the address of the `halt` instruction. It also sets `%rsp` to 128.

Stage	Generic `call` D t	Specific `call` 0x041
Decode	alB ← R[%rsp]	
Execute	alE ← alB + (−8)	
Memory	M_8[alE] ← alP	
Write back	R[%rsp] ← alE	
PC update	PC ← alC	

What effect would this instruction execution have on the registers, the PC, and the memory?

We have created a uniform framework that handles all of the different types of Y86-64 instructions. Even though the instructions have widely varying behavior, we can organize the processing into six stages. Our task now is to create a hardware design that implements the stages and connects them together.

4.3.2 SEQ Hardware Structure

The computations required to implement all of the Y86-64 instructions can be organized as a series of six basic stages: fetch, decode, execute, memory, write back, and PC update. Figure 4.22 shows an abstract view of a hardware structure that can perform these computations. The program counter is stored in a register, shown in the lower left-hand corner (labeled "PC"). Information then flows along wires (shown grouped together as a heavy gray line), first upward and then around to the right. Processing is performed by hardware units associated with the different stages. The feedback paths coming back down on the right-hand side contain the updated values to write to the register file and the updated program counter. In SEQ, all of the processing by the hardware units occurs within a single clock cycle, as is discussed in Section 4.3.3. This diagram omits some small blocks of combinational logic as well as all of the control logic needed to operate the different hardware units and to route the appropriate values to the units. We will add this detail later. Our method of drawing processors with the flow going from bottom to top is unconventional. We will explain the reason for this convention when we start designing pipelined processors.

The hardware units are associated with the different processing stages:

Fetch. Using the program counter register as an address, the instruction memory reads the bytes of an instruction. The PC incrementer computes valP, the incremented program counter.

Decode. The register file has two read ports, A and B, via which register values valA and valB are read simultaneously.

Execute. The execute stage uses the arithmetic/logic (ALU) unit for different purposes according to the instruction type. For integer operations, it performs the specified operation. For other instructions, it serves as an adder to compute an incremented or decremented stack pointer, to compute an effective address, or simply to pass one of its inputs to its outputs by adding zero.

The condition code register (CC) holds the three condition code bits. New values for the condition codes are computed by the ALU. When executing a conditional move instruction, the decision as to whether or not to update the destination register is computed based on the condition codes and move condition. Similarly, when executing a jump instruction, the branch signal Cnd is computed based on the condition codes and the jump type.

Memory. The data memory reads or writes a word of memory when executing a memory instruction. The instruction and data memories access the same memory locations, but for different purposes.

Write back. The register file has two write ports. Port E is used to write values computed by the ALU, while port M is used to write values read from the data memory.

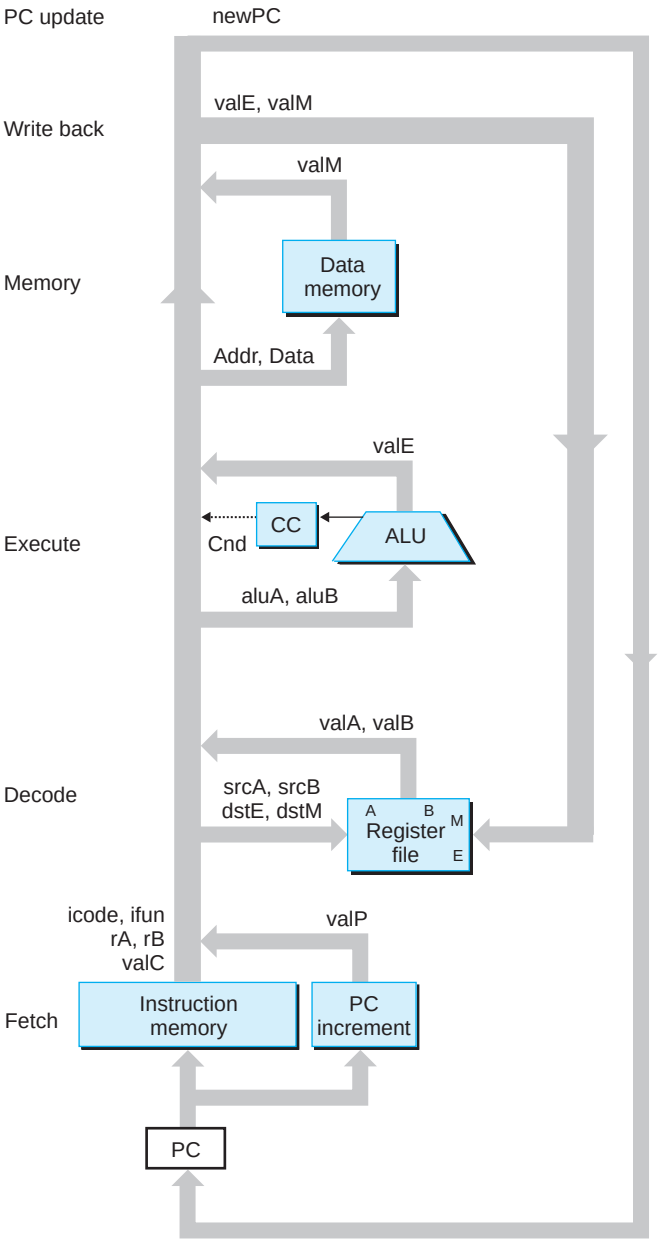


Figure 4.22 Abstract view of SEQ, a sequential implementation. T i r ati r ri ti a i tr ti ll a l i tarti it a i tr ti t i t ra t r (PC), i t l rl t- a r r t fi r .

PC update. The new value of the program counter is selected to be either valP, the address of the next instruction, valC, the destination address specified by a call or jump instruction, or valM, the return address read from memory.

Figure 4.23 gives a more detailed view of the hardware required to implement SEQ (although we will not see the complete details until we examine the individual stages). We see the same set of hardware units as earlier, but now the wires are shown explicitly. In this figure, as well as in our other hardware diagrams, we use the following drawing conventions:

- Clocked registers are shown as white rectangles. The program counter PC is the only clocked register in SEQ.
- Hardware units are shown as light blue boxes. These include the memories, the ALU, and so forth. We will use the same basic set of units for all of our processor implementations. We will treat these units as "black boxes" and not go into their detailed designs.
- Control logic blocks are drawn as gray rounded rectangles. These blocks serve to select from among a set of signal sources or to compute some Boolean function. We will examine these blocks in complete detail, including developing HCL descriptions.
- Wire names are indicated in white circles. These are simply labels on the wires, not any kind of hardware element.
- Word-wide data connections are shown as medium lines. Each of these lines actually represents a bundle of 64 wires, connected in parallel, for transferring a word from one part of the hardware to another.
- Byte and narrower data connections are shown as thin lines. Each of these lines actually represents a bundle of four or eight wires, depending on what type of values must be carried on the wires.
- Single-bit connections are shown as dotted lines. These represent control values passed between the units and blocks on the chip.

All of the computations we have shown in Figures 4.18 through 4.21 have the property that each line represents either the computation of a specific value, such as valP, or the activation of some hardware unit, such as the memory. These computations and actions are listed in the second column of Figure 4.24. In addition to the signals we have already described, this list includes four register ID signals: rA, the source of valA; rB, the source of valB; dstE, the register to which valE gets written; and dstM, the register to which valM gets written.

The two right-hand columns of this figure show the computations for the OPq and mrmovq instructions to illustrate the values being computed. To map the computations into hardware, we want to implement control logic that will transfer the data between the different hardware units and operate these units in such a way that the specified operations are performed for each of the different instruction types. That is the purpose of the control logic blocks, shown as gray rounded boxes

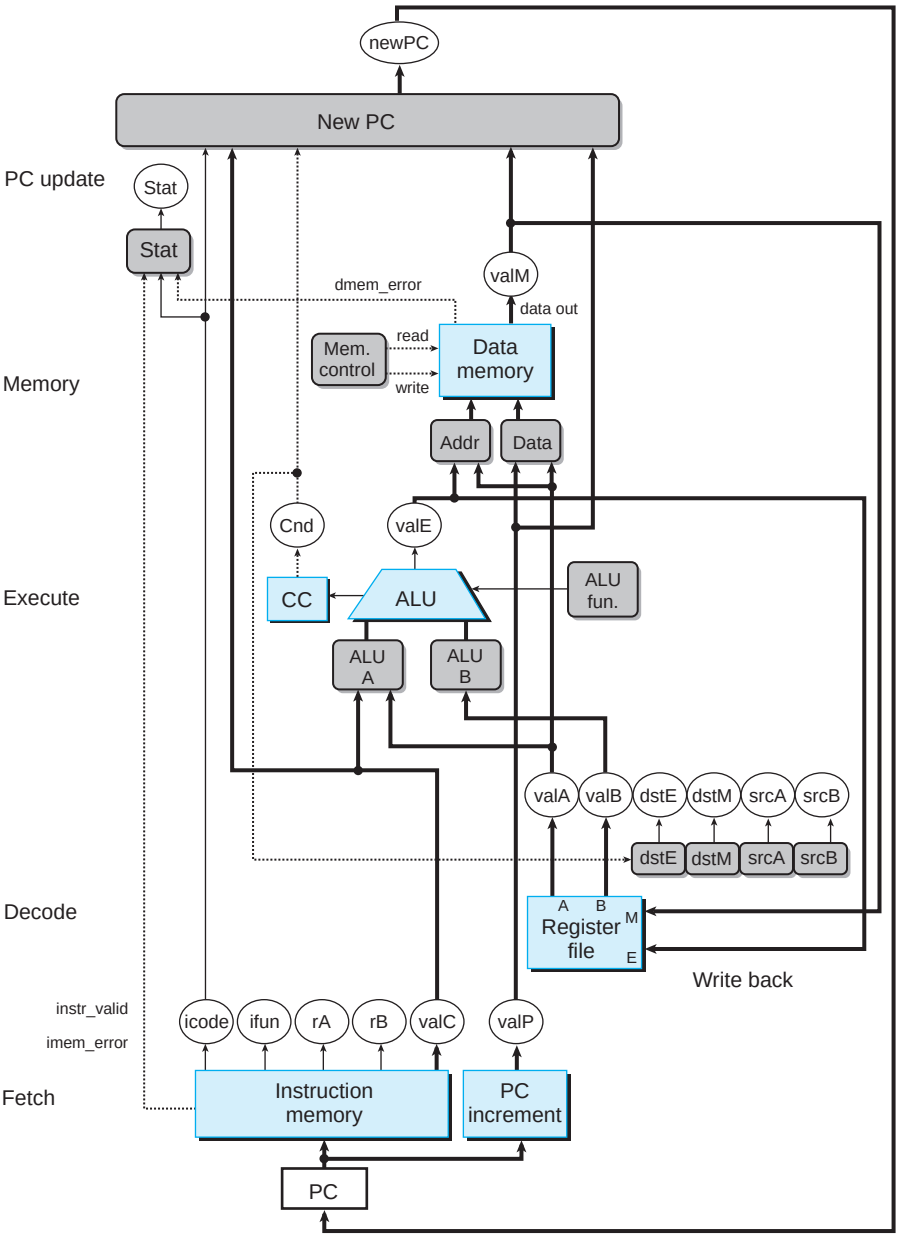


Figure 4.23 Hardware structure of SEQ, a sequential implementation. S o o t o tr o l i al, a ll a t r itra o tr o l o o ti , ar o t o .

Stage	Computation	OPq rA, rB	mrmovq D(rB), rA
Fetch	i $_0^{\vee}$, i rA, rB	i $_0^{\vee}$: i ← M_1[PC] rA:rB ← M_1[PC +]1]	i $_0^{\vee}$: i ← M_1[PC] rA:rB ← M

rA]

memories (such as the register file), and we can mimic this effect for larger circuits using special clock circuits. Since our instruction memory is only used to read instructions, we can therefore treat this unit as if it were combinational logic.

We are left with just four hardware units that require an explicit control over their sequencing—the program counter, the condition code register, the data memory, and the register file. These are controlled via a single clock signal that triggers the loading of new values into the registers and the writing of values to the random access memories. The program counter is loaded with a new instruction address every clock cycle. The condition code register is loaded only when an integer operation instruction is executed. The data memory is written only when an `rmmovq`, `pushq`, or `call` instruction is executed. The two write ports of the register file allow two program registers to be updated on every cycle, but we can use the special register ID `0xF` as a port address to indicate that no write should be performed for this port.

This clocking of the registers and memories is all that is required to control the sequencing of activities in our processor. Our hardware achieves the same effect as would a sequential execution of the assignments shown in the tables of Figures 4.18 through 4.21, even though all of the state updates actually occur simultaneously and only as the clock rises to start the next cycle. This equivalence holds because of the nature of the Y86-64 instruction set, and because we have organized the computations in such a way that our design obeys the following principle:

principle: No reading back

The processor never needs to read back the state updated by an instruction in order to complete the processing of this instruction. ■

This principle is crucial to the success of our implementation. As an illustration, suppose we implemented the `pushq` instruction by first decrementing `%rsp` by 8 and then using the updated value of `%rsp` as the address of a write operation. This approach would violate the principle stated above. It would require reading the updated stack pointer from the register file in order to perform the memory operation. Instead, our implementation (Figure 4.20) generates the decremented value of the stack pointer as the signal `aluE` and then uses this signal both as the data for the register write and the address for the memory write. As a result, it can perform the register and memory writes simultaneously as the clock rises to begin the next clock cycle.

As another illustration of this principle, we can see that some instructions (the integer operations) set the condition codes, and some instructions (the conditional move and jump instructions) read these condition codes, but no instruction must both set and then read the condition codes. Even though the condition codes are not set until the clock rises to begin the next clock cycle, they will be updated before any instruction attempts to read them.

Figure 4.25 shows how the SEQ hardware would process the instructions at lines 3 and 4 in the following code sequence, shown in assembly code with the instruction addresses listed on the left:

```
1     0x000:   irmovq $0x100,%rbx   # %rbx <-- 0x100
2     0x00a:   irmovq $0x200,%rdx   # %rdx <-- 0x200
3     0x014:   addq %rdx,%rbx       # %rbx <-- 0x300 CC <-- 000
4     0x016:   je dest              # Not taken
5     0x01f:   rmmovq %rbx,0(%rdx)  # M[0x200] <-- 0x300
6     0x029: dest: halt
```

Each of the diagrams labeled 1 through 4 shows the four state elements plus the combinational logic and the connections among the state elements. We show the combinational logic as being wrapped around the condition code register, because some of the combinational logic (such as the ALU) generates the input to the condition code register, while other parts (such as the branch computation and the PC selection logic) have the condition code register as input. We show the register file and the data memory as having separate connections for reading and writing, since the read operations propagate through these units as if they were combinational logic, while the write operations are controlled by the clock.

The color coding in Figure 4.25 indicates how the circuit signals relate to the different instructions being executed. We assume the processing starts with the condition codes, listed in the order `ZF`, `SF`, and `OF`, set to `100`. At the beginning of clock cycle 3 (point 1), the state elements hold the state as updated by the second `irmovq` instruction (line 2 of the listing), shown in light gray. The combinational logic is shown in white, indicating that it has not yet had time to react to the changed state. The clock cycle begins with address `0x014` add4

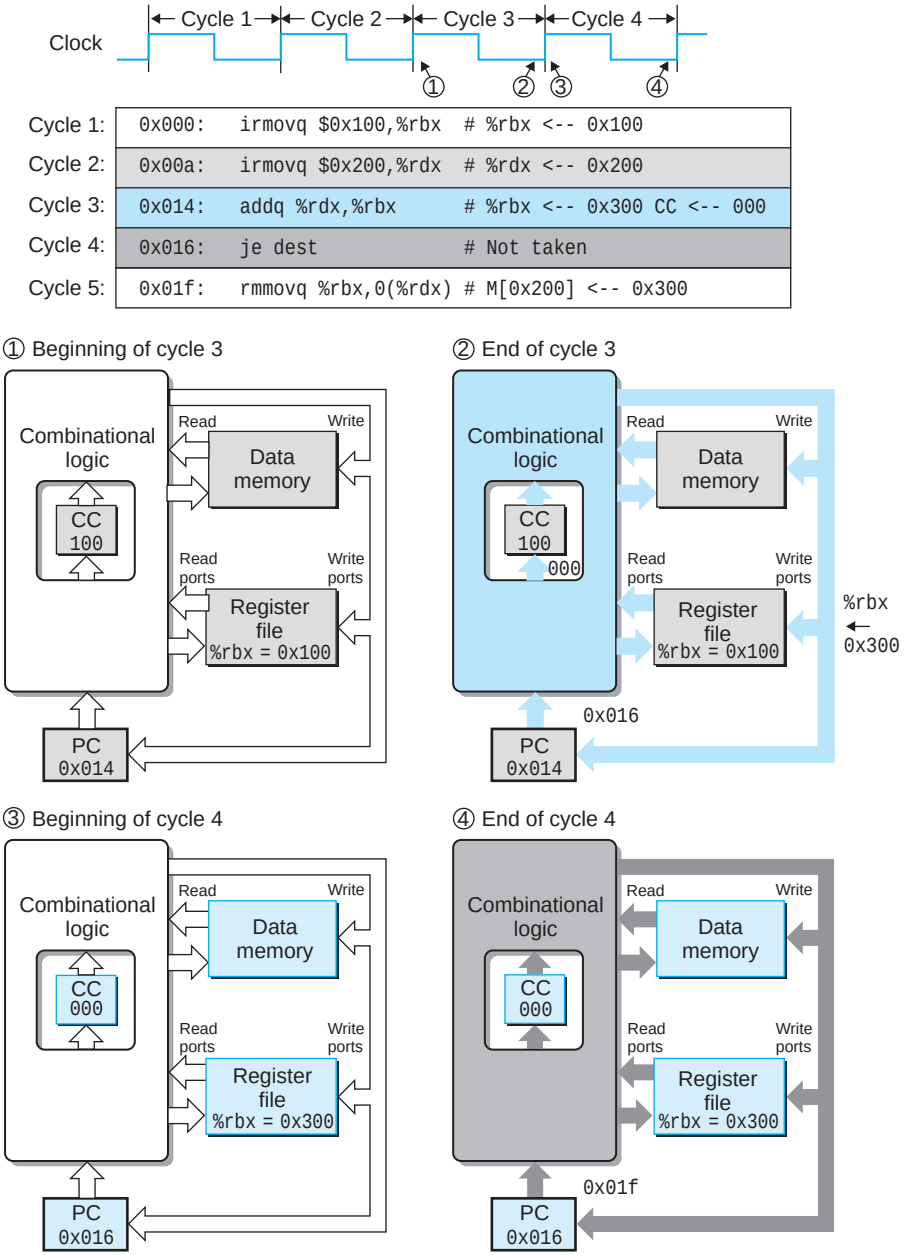


Figure 4.25 Tracing two cycles of execution by SEQ. Ea l i it t tat l t (ro ra o t r, o \ ti o r it r, r it r fil , a \ ata or) ta or i t o t r i i tr ti o . Si al r o a at t r o t i ati al lo i, r ati al ort tat l t . T al ar la \ i t o t tat l t t o tart t t l .

4.3.4 SEQ Stage Implementations

In this section, we devise HCL descriptions for the control logic blocks required to implement SEQ. A complete HCL description for SEQ is given in Web Aside ARCH:HCL on page 508. We show some example blocks here, and others are given as practice problems. We recommend that you work these problems as a way to check your understanding of how the blocks relate to the computational requirements of the different instructions.

Part of the HCL description of SEQ that we do not include here is a definition of the different integer and Boolean signals that can be used as arguments to the HCL operations. These include the names of the different hardware signals, as well as constant values for the different instruction codes, function codes, register names, ALU operations, and status codes. Only those that must be explicitly

Name	Value (hex)	Meaning
IHALT	0	Code for `halt` instruction
INOP	1	Code for `nop` instruction
IRRMOVQ	2	Code for `rrmovq` instruction
IIRMOVQ	3	Code for `irmovq` instruction
IRMMOVQ	4	Code for `rmmovq` instruction
IMRMOVQ	5	Code for `mrmovq` instruction
IOPL	6	Code for integer operation instructions
IJXX	7	Code for jump instructions
ICALL	8	Code for `call` instruction
IRET	9	Code for `ret` instruction
IPUSHQ	A	Code for `pushq` instruction
IPOPQ	B	Code for `popq` instruction
FNONE	0	Default function code
RESP	4	Register ID for `%rsp`
RNONE	F	Indicates no register file access
ALUADD	0	Function for addition operation
SAOK	1	Status code for normal operation
SADR	2	Status code for address exception
SINS	3	Status code for illegal instruction exception
SHLT	4	Status code for `halt`

Figure 4.26 Constant values used in HCL descriptions. T al r r t t i t i tr ti , ti , r it r ID , ALU rati , a tat .

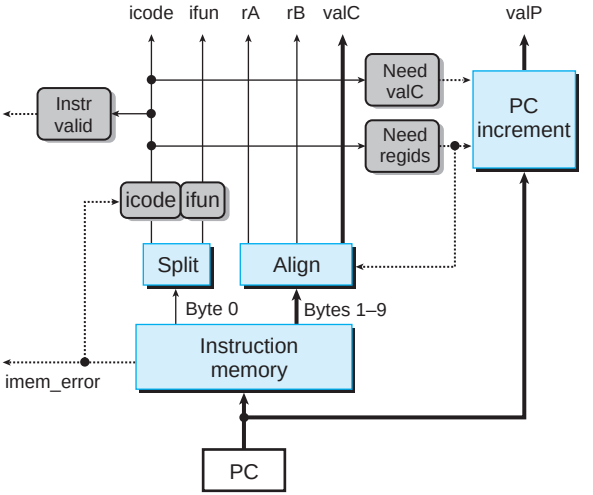


Figure 4.27
SEQ fetch stage. Si t ar r a r o t i tr ti o o r i t PC a t tarti a r . Fr o t t , rat t i r t i tr ti o fi . T PC i r t l o o t i al alP.

referenced in the control logic are shown. The constants we use are documented in Figure 4.26. By convention, we use uppercase names for constant values.

In addition to the instructions shown in Figures 4.18 to 4.21, we include the processing for the `nop` and `halt` instructions. The `nop` instruction simply flows through stages without much processing, except to increment the PC by 1. The `halt` instruction causes the processor status to be set to `HLT`, causing it to halt operation.

F t Sta

As shown in Figure 4.27, the fetch stage includes the instruction memory hardware unit. This unit reads 10 bytes from memory at a time, using the PC as the address of the first byte (byte 0). This byte is interpreted as the instruction byte and is split (by the unit labeled "Split") into two 4-bit quantities. The control logic blocks labeled "icode" and "ifun" then compute the instruction and function codes as equaling either the values read from memory or, in the event that the instruction address is not valid (as indicated by the signal i _ rr o r), the values corresponding to a nop instruction. Based on the value of i o , we can compute three 1-bit signals (shown as dashed lines):

- i tr_ al . Does this byte correspond to a legal Y86-64 instruction? This signal is used to detect an illegal instruction.
- _r . Does this instruction include a register specifier byte?
- _ alC. Does this instruction include a constant word?

The signals i tr_ al and i _ rr o r (generated when the instruction address is out of bounds) are used to generate the status code in the memory stage.

As an example, the HCL description for need_regids simply determines whether the value of icode is one of the instructions that has a register specifier byte:

```
bool need_regids =
        icode in { IRRMOVQ, IOPQ, IPUSHQ, IPOPQ,
                   IIRMOVQ, IRMMOVQ, IMRMOVQ };
```

Practice Problem 4.19 (solution page 523)

Write HCL code for the signal `need_valC` in the SEQ implementation.

As Figure 4.27 shows, the remaining 9 bytes read from the instruction memory encode some combination of the register specifier byte and the constant word. These bytes are processed by the hardware unit labeled "Align" into the register fields and the constant word. Byte 1 is split into register specifiers rA and rB when the computed signal need_regids is 1. If need_regids is 0, both register specifiers are set to 0xF (RNONE), indicating there are no registers specified by this instruction. Recall also (Figure 4.2) that for any instruction having only one register operand, the other field of the register specifier byte will be 0xF (RNONE). Thus, we can assume that the signals rA and rB either encode registers we want to access or indicate that register access is not required. The unit labeled "Align" also generates the constant word valC. This will either be bytes 1–8 or bytes 2–9, depending on the value of signal need_regids.

The PC incrementer hardware unit generates the signal valP, based on the current value of the PC, and the two signals need_regids and need_valC. For PC value p, need_regids value r, and need_valC value i, the incrementer generates the value p + 1 + r + 8i.

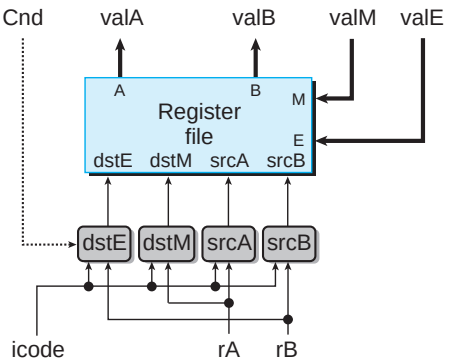
Decode and Write-Back Stages

Figure 4.28 provides a detailed view of logic that implements both the decode and write-back stages in SEQ. These two stages are combined because they both access the register file.

The register file has four ports. It supports up to two simultaneous reads (on ports A and B) and two simultaneous writes (on ports E and M). Each port has both an address connection and a data connection, where the address connection is a register ID, and the data connection is a set of 64 wires serving as either an output word (for a read port) or an input word (for a write port) of the register file. The two read ports have address inputs srcA and srcB, while the two write ports have address inputs dstE and dstM. The special identifier 0xF (RNONE) on an address port indicates that no register should be accessed.

The four blocks at the bottom of Figure 4.28 generate the four different register IDs for the register file, based on the instruction code icode, the register specifiers rA and rB, and possibly the condition signal Cnd computed in the execute stage. Register ID srcA indicates which register should be read to generate valA.

Figure 4.28
SEQ decode and write-back stage. T i tr ti fi ar t rat r it r tfi r r ra r (t r a a t rit) t r it r fil . T al r a r t r it r fil t i al alA a alB. T t rit - a al alE a alM r a t ata rt rit .



The desired value depends on the instruction type, as shown in the first row for the decode stage in Figures 4.18 to 4.21. Combining all of these entries into a single computation gives the following HCL description of r A (recall that RESP is the register ID of %rsp):

```
word srcA = [
        icode in { IRRMOVQ, IRMMOVQ, IOPQ, IPUSHQ  } : rA;
        icode in { IPOPQ, IRET } : RRSP;
        1 : RNONE; # Don't need register
];
```

Practice Problem 4.20 (solution page 524)

The register signal r B indicates which register should be read to generate the signal alB. The desired value is shown as the second step in the decode stage in Figures 4.18 to 4.21. Write HCL code for r B.

Register ID tE indicates the destination register for write port E, where the computed value alE is stored. This is shown in Figures 4.18 to 4.21 as the first step in the write-back stage. If we ignore for the moment the conditional move instructions, then we can combine the destination registers for all of the different instructions to give the following HCL description of tE:

```
# WARNING: Conditional move not implemented correctly here
word dstE = [
        icode in { IRRMOVQ } : rB;
        icode in { IIRMOVQ, IOPQ} : rB;
        icode in { IPUSHQ, IPOPQ, ICALL, IRET } : RRSP;
        1 : RNONE;  # Don't write any register
];
```

We will revisit this signal and how to implement conditional moves when we examine the execute stage.

Practice Problem 4.21 (solution page 524)

Register ID tM indicates the destination register for write port M, where alM, the value read from memory, is stored. This is shown in Figures 4.18 to 4.21 as the second step in the write-back stage. Write HCL code for tM.

Practice Problem 4.22 (solution page 524)

Only the `popq` instruction uses both register file write ports simultaneously. For the instruction `popq %rsp`, the same address will be used for both the E and M write ports, but with different data. To handle this conflict, we must establish a priority among the two write ports so that when both attempt to write the same register on the same cycle, only the write from the higher-priority port takes place. Which of the two ports should be given priority in order to implement the desired behavior, as determined in Practice Problem 4.8?

E t Sta

The execute stage includes the arithmetic/logic unit (ALU). This unit performs the operation ADD, SUBTRACT, AND, or EXCLUSIVE-OR on inputs al A and al B based on the setting of the al signal. These data and control signals are generated by three control blocks, as diagrammed in Figure 4.29. The ALU output becomes the signal alE.

In Figures 4.18 to 4.21[(Ex340(th)-212(a[/T1a27(destinata[/)-226(a[/ea)18(ha[/ Td [(i

```
        icode in { ICALL, IPUSHQ } : -8;
        icode in { IRET, IPOPQ } : 8;
        # Other instructions don't need ALU
];
```

Practice Problem 4.23 (solution page 524)

Based on the first operand of the first step of the execute stage in Figures 4.18 to 4.21, write an HCL description for the signal alB in SEQ.

Looking at the operations performed by the ALU in the execute stage, we can see that it is mostly used as an adder. For the OPq instructions, however, we want it to use the operation encoded in the i field of the instruction. We can therefore write the HCL description for the ALU control as follows:

```
word alufun = [
        icode == IOPQ : ifun;
        1 : ALUADD;
];
```

The execute stage also includes the condition code register. Our ALU generates the three signals on which the condition codes are based—zero, sign, and overflow—every time it operates. However, we only want to set the condition codes when an OPq instruction is executed. We therefore generate a signal t_ that controls whether or not the condition code register should be updated:

```
bool set_cc = icode in { IOPQ };
```

The hardware unit labeled "cond" uses a combination of the condition codes and the function code to determine whether a conditional branch or data transfer should take place (Figure 4.3). It generates the C signal used both for the setting of tE with conditional moves and in the next PC logic for conditional branches. For other instructions, the C signal may be set to either 1 or 0, depending on the instruction's function code and the setting of the condition codes, but it will be ignored by the control logic. We omit the detailed design of this unit.

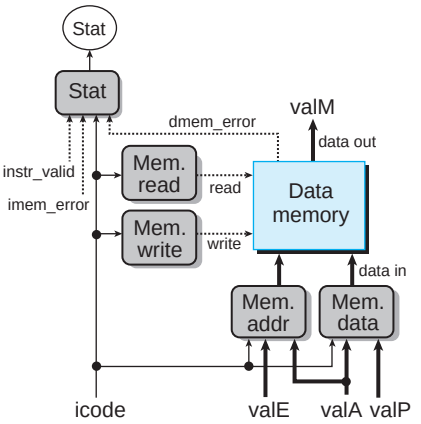
Practice Problem 4.24 (solution page 524)

The conditional move instructions, abbreviated cmovXX, have instruction code IRRMOVQ. As Figure 4.28 shows, we can implement these instructions by making use of the C signal, generated in the execute stage. Modify the HCL code for tE to implement these instructions.

M or Sta

The memory stage has the task of either reading or writing program data. As shown in Figure 4.30, two control blocks generate the values for the memory

Figure 4.30
SEQ memory stage. T
ata r a it r
rit o r r a r
al . T al r a r
o r o r t i al
alM.



address and the memory input data (for write operations). Two other blocks generate the control signals indicating whether to perform a read or a write operation. When a read operation is performed, the data memory generates the value alM.

The desired memory operation for each instruction type is shown in the memory stage of Figures 4.18 to 4.21. Observe that the address for memory reads and writes is always alE or alA. We can describe this block in HCL as follows:

```
word mem_addr = [
        icode in { IRMMOVQ, IPUSHQ, ICALL, IMRMOVQ } : valE;
        icode in { IPOPQ, IRET } : valA;
        # Other instructions don't need address
];
```

Practice Problem 4.25 (solution page 524)

Looking at the memory operations for the different instructions shown in Figures 4.18 to 4.21, we can see that the data for memory writes are always either alA or alP. Write HCL code for the signal _ ata in SEQ.

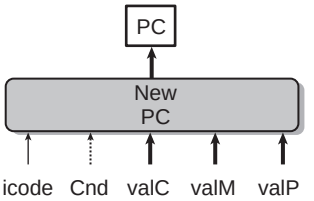
We want to set the control signal _r a only for instructions that read data from memory, as expressed by the following HCL code:

```
bool mem_read = icode in { IMRMOVQ, IPOPQ, IRET };
```

Practice Problem 4.26 (solution page 525)

We want to set the control signal _ rit only for instructions that write data to memory. Write HCL code for the signal _ rit in SEQ.

Figure 4.31
SEQ PC update stage. T t al o t PC i l t r o a t i al alC, alM, a alP, i t i tr ti o o a t ra a .



A final function for the memory stage is to compute the status code Stat resulting from the instruction execution according to the values of i , i _ rr r, and i tr_ al generated in the fetch stage and the signal _ rr r generated by the data memory.

Practice Problem 4.27 (solution page 525)

Write HCL code for Stat, generating the four status codes SAOK, SADR, SINS, and SHLT (see Figure 4.26).

PC U at Sta

The final stage in SEQ generates the new value of the program counter (see Figure 4.31). As the final steps in Figures 4.18 to 4.21 show, the new PC will be alC, alM, or alP, depending on the instruction type and whether or not a branch should be taken. This selection can be described in HCL as follows:

```
word new_pc = [
        # Call.  Use instruction constant
        icode == ICALL : valC;
        # Taken branch.  Use instruction constant
        icode == IJXX && Cnd : valC;
        # Completion of RET instruction.  Use value from stack
        icode == IRET : valM;
        # Default: Use incremented PC
        1 : valP;
];
```

S r i SEQ

We have now stepped through a complete design for a Y86-64 processor. We have seen that by organizing the steps required to execute each of the different instructions into a uniform flow, we can implement the entire processor with a small number of different hardware units and with a single clock to control the sequencing of computations. The control logic must then route the signals between these units and generate the proper control signals based on the instruction types and the branch conditions.

The only problem with SEQ is that it is too slow. The clock must run slowly enough so that signals can propagate through all of the stages within a single cycle. As an example, consider the processing of a `ret` instruction. Starting with an updated program counter at the beginning of the clock cycle, the instruction must be read from the instruction memory, the stack pointer must be read from the register file, the ALU must increment the stack pointer by 8, and the return address must be read from the memory in order to determine the next value for the program counter. All of these must be completed by the end of the clock cycle.

This style of implementation does not make very good use of our hardware units, since each unit is only active for a fraction of the total clock cycle. We will see that we can achieve much better performance by introducing pipelining.

4.4 General Principles of Pipelining

Before attempting to design a pipelined Y86-64 processor, let us consider some general properties and principles of pipelined systems. Such systems are familiar to anyone who has been through the serving line at a cafeteria or run a car through an automated car wash. In a pipelined system, the task to be performed is divided into a series of discrete stages. In a cafeteria, this involves supplying salad, a main dish, dessert, and beverage. In a car wash, this involves spraying water and soap, scrubbing, applying wax, and drying. Rather than having one customer run through the entire sequence from beginning to end before the next can begin, we allow multiple customers to proceed through the system at once. In a traditional cafeteria line, the customers maintain the same order in the pipeline and pass through all stages, even if they do not want some of the courses. In the case of the car wash, a new car is allowed to enter the spraying stage as the preceding car moves from the spraying stage to the scrubbing stage. In general, the cars must move through the system at the same rate to avoid having one car crash into the next.

A key feature of pipelining is that it increases the throughput of the system (i.e., the number of customers served per unit time), but it may also slightly increase the latency (i.e., the time required to service an individual customer). For example, a customer in a cafeteria who only wants a dessert could pass through a nonpipelined system very quickly, stopping only at the dessert stage. A customer in a pipelined system who attempts to go directly to the dessert stage risks incurring the wrath of other customers.

4.4.1 Computational Pipelines

Shifting our focus to computational pipelines, the "customers" are instructions and the stages perform some portion of the instruction execution. Figure 4.32(a) shows an example of a simple nonpipelined hardware system. It consists of some logic that performs a computation, followed by a register to hold the results of this computation. A clock signal controls the loading of the register at some regular time interval. An example of such a system is the decoder in a compact disk (CD) player. The incoming signals are the bits read from the surface of the CD, and

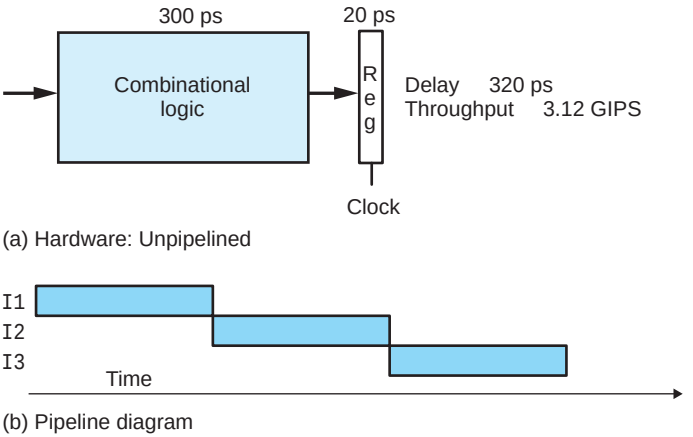


Figure 4.32
Unpipelined computation hardware. O a 320 l , t t 300 al ati a i ati al l i ti a 20 t ri t r lt i a t t r it r.

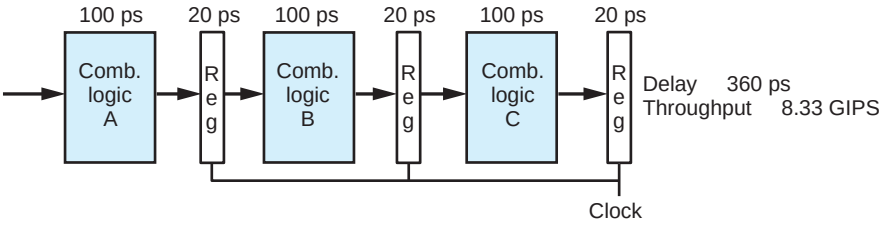
the logic decodes these to generate audio signals. The computational block in the figure is implemented as combinational logic, meaning that the signals will pass through a series of logic gates, with the outputs becoming some function of the inputs after some time delay.

In contemporary logic design, we measure circuit delays in units of picoseconds (abbreviated "ps"), or 10^{-12} seconds. In this example, we assume the combinational logic requires 300 ps, while the loading of the register requires 20 ps. Figure 4.32 shows a form of timing diagram known as a pipeline diagram. In this diagram, time flows from left to right. A series of instructions (here named `I1`, `I2`, and `I3`) are written from top to bottom. The solid rectangles indicate the times during which these instructions are executed. In this implementation, we must complete one instruction before beginning the next. Hence, the boxes do not overlap one another vertically. The following formula gives the maximum rate at which we could operate the system:

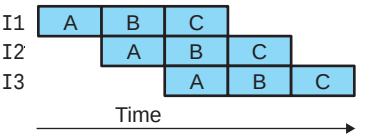
$$\text{Throughput} = \frac{1 \text{ instruction}}{(20 + 300) \text{ picoseconds}} \cdot \frac{1{,}000 \text{ picoseconds}}{1 \text{ nanosecond}} \approx 3.12 \text{ GIPS}$$

We express throughput in units of giga-instructions per second (abbreviated GIPS), or billions of instructions per second. The total time required to perform a single instruction from beginning to end is known as the latency. In this system, the latency is 320 ps, the reciprocal of the throughput.

Suppose we could divide the computation performed by our system into three stages, A, B, and C, where each requires 100 ps, as illustrated in Figure 4.33. Then we could put pipeline registers between the stages so that each instruction moves through the system in three steps, requiring three complete clock cycles from beginning to end. As the pipeline diagram in Figure 4.33 illustrates, we could allow `I2` to enter stage A as soon as `I1` moves from A to B, and so on. In steady state, all three stages would be active, with one instruction leaving and a new one entering the system every clock cycle. We can see this during the third clock cycle in the pipeline diagram where `I1` is in stage C, `I2` is in stage B, and `I3` is in stage A. In



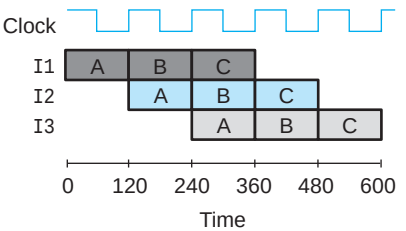
(a) Hardware: Three-stage pipeline



(b) Pipeline diagram

Figure 4.33 Three-stage pipelined computation hardware. T o tati i lit i t ta A, B, a C. O a 120 l , a i tr ti r r t r o ta .

Figure 4.34 Three-stage pipeline timing. T ri i t l i al tr l t t i tr ti r i li ta t t.



this system, we could cycle the clocks every 100 + 20 = 120 picoseconds, giving a throughput of around 8.33 GIPS. Since processing a single instruction requires 3 clock cycles, the latency of this pipeline is 3 × 120 = 360 ps. We have increased the throughput of the system by a factor of 8.33/3.12 = 2.67 at the expense of some added hardware and a slight increase in the latency (360/320 = 1.12). The increased latency is due to the time overhead of the added pipeline registers.

4.4.2 A Detailed Look at Pipeline Operation

To better understand how pipelining works, let us look in some detail at the timing and operation of pipeline computations. Figure 4.34 shows the pipeline diagram for the three-stage pipeline we have already looked at (Figure 4.33). The transfer of the instructions between pipeline stages is controlled by a clock signal, as shown above the pipeline diagram. Every 120 ps, this signal rises from 0 to 1, initiating the next set of pipeline stage evaluations.

Figure 4.35 traces the circuit activity between times 240 and 360, as instruction `I1` (shown in dark gray) propagates through stage C, `I2` (shown in blue)

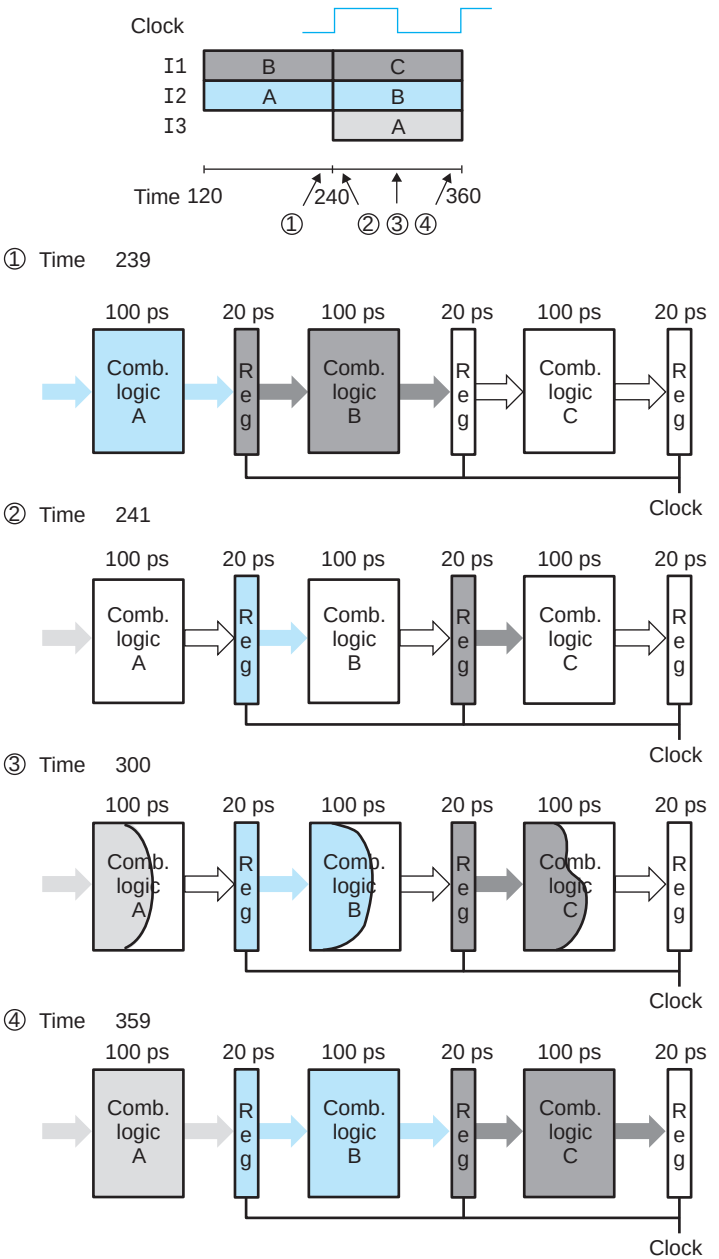
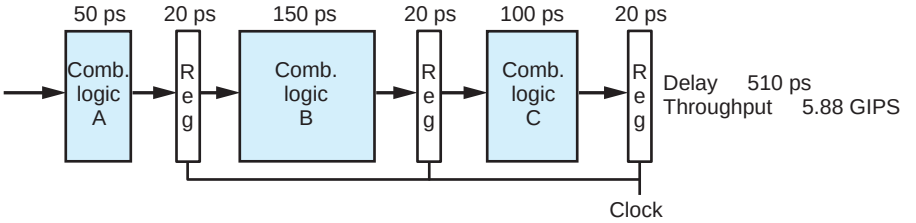
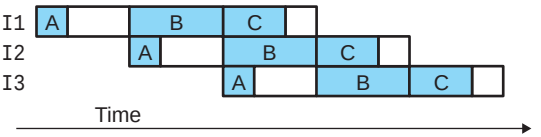


Figure 4.35 One clock cycle of pipeline operation. J t r t l o ri at ti 240 (o i t 1), i tr ti o I1 (o i ' ar ra) a ' I2 (o i l) a l t ' ta B a ' A. A t r t l o ri , t i tr ti o i r o a ati t r o ta C a ' B, i i tr ti o I3 (o i li t ra) i r o a ati t r o ta A (o i t 2 a ' 3). J t r t l o ri a ai, t r lt o rt i tr ti o a r o a at ' t o t i t o t i li r itr (o i t 4).

propagates through stage B, and I3 (shown in light gray) propagates through stage A. Just before the rising clock at time 240 (point 1), the values computed in stage A for instruction I2 have reached the input of the first pipeline register, but its state and output remain set to those computed during stage A for instruction I1. The values computed in stage B for instruction I1 have reached the input of the second pipeline register. As the clock rises, these inputs are loaded into the pipeline registers, becoming the register outputs (point 2). In addition, the input to stage A is set to initiate the computation of instruction I3. The signals then propagate through the combinational logic for the different stages (point 3). As the curved wave fronts in the diagram at point 3 suggest, signals can propagate through different sections at different rates. Before time 360, the result values reach the inputs of the pipeline registers (point 4). When the clock rises at time 360, each of the



(a) Hardware: Three-stage pipeline, nonuniform stage delays



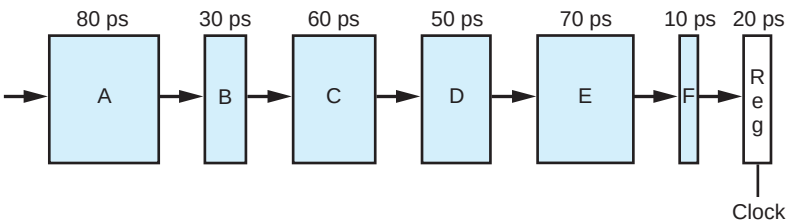
(b) Pipeline diagram

Figure 4.36 Limitations of pipelining due to nonuniform stage delays. T t t r o t i li i t\ t \ o t l o t ta .

some of the hardware units in a processor, such as the ALU and the memories, cannot be subdivided into multiple units with shorter delay. This makes it difficult to create a set of balanced stages. We will not concern ourselves with this level of detail in designing our pipelined Y86-64 processor, but it is important to appreciate the importance of timing optimization in actual system design.

Practice Problem 4.28 (solution page 525)

Suppose we analyze the combinational logic of Figure 4.32 and determine that it can be separated into a sequence of six blocks, named A to F, having delays of 80, 30, 60, 50, 70, and 10 ps, respectively, illustrated as follows:



We can create pipelined versions of this design by inserting pipeline registers between pairs of these blocks. Different combinations of pipeline depth (how many stages) and maximum throughput arise, depending on where we insert the pipeline registers. Assume that a pipeline register has a delay of 20 ps.

A. Inserting a single register gives a two-stage pipeline. Where should the register be inserted to maximize throughput? What would be the throughput and latency?

B. Where should two registers be inserted to maximize the throughput of a three-stage pipeline? What would be the throughput and latency?

C. Where should three registers be inserted to maximize the throughput of a 4-stage pipeline? What would be the throughput and latency?

D. What is the minimum number of stages that would yield a design with the maximum achievable throughput? Describe this design, its throughput, and its latency.

Diminishing Returns Due to Pipelining Overhead

Figure 4.37 illustrates another limitation of pipelining. In this example, we have divided the computation into six stages, each requiring 50 ps. Inserting a pipeline register between each pair of stages yields a six-stage pipeline. The minimum clock period for this system is 50 + 20 = 70 picoseconds, giving a throughput of 14.29 GIPS. Thus, in doubling the number of pipeline stages, we improve the performance by a factor of 14.29/8.33 = 1.71. Even though we have cut the time required for each computation block by a factor of 2, we do not get a doubling of the throughput, due to the delay through the pipeline registers. This delay becomes a limiting factor in the throughput of the pipeline. In our new design, this delay consumes 28.6% of the total clock period.

Modern processors employ very deep pipelines (15 or more stages) in an attempt to maximize the processor clock rate. The processor architects divide the instruction execution into a large number of very simple steps so that each stage can have a very small delay. The circuit designers carefully design the pipeline registers to minimize their delay. The chip designers must also carefully design the clock distribution network to ensure that the clock changes at the exact same time across the entire chip. All of these factors contribute to the challenge of designing high-speed microprocessors.

Practice Problem 4.29 (solution page 526)

Suppose we could take the system of Figure 4.32 and divide it into an arbitrary number of pipeline stages k, each having a delay of 300/k, and with each pipeline register having a delay of 20 ps.

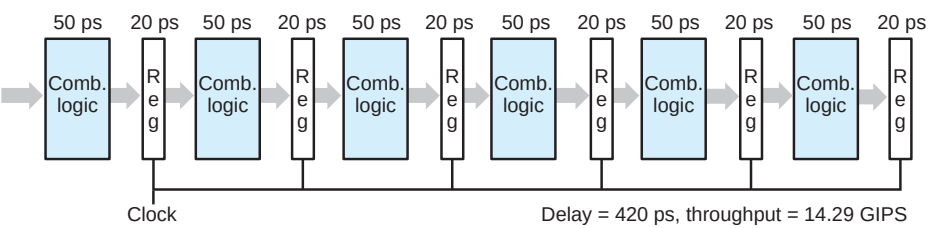


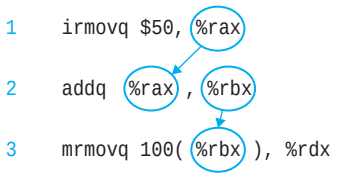
Figure 4.37 Limitations of pipelining due to overhead. Although additional pipelining can improve system performance, the incremental improvement diminishes as more pipeline registers are added.

A. What would be the latency and the throughput of the system, as functions of k?

B. What would be the ultimate limit on the throughput?

4.4.4 Pipelining a System with Feedback

Up to this point, we have considered only systems in which the objects passing through the pipeline—whether cars, people, or instructions—are completely independent of one another. For a system that executes machine programs such as x86-64 or Y86-64, however, there are potential dependencies between successive instructions. For example, consider the following Y86-64 instruction sequence:



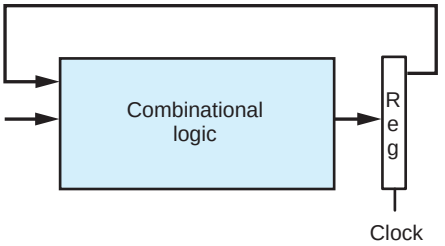
In this three-instruction sequence, there is a data dependency between each successive pair of instructions, as indicated by the circled register names and the arrows between them. The `irmovq` instruction (line 1) stores its result in `%rax`, which then must be read by the `addq` instruction (line 2); and this instruction stores its result in `%rbx`, which must then be read by the `mrmovq` instruction (line 3).

Another source of sequential dependencies occurs due to the instruction control flow. Consider the following Y86-64 instruction sequence:

```
1   loop:
2       subq %rdx,%rbx
3       jne targ
4       irmovq $10,%rdx
5       jmp loop
6   targ:
7       halt
```

The `jne` instruction (line 3) creates a control dependency since the outcome of the conditional test determines whether the next instruction to execute will be the `irmovq` instruction (line 4) or the `halt` instruction (line 7). In our design for SEQ, these dependencies were handled by the feedback paths shown on the right-hand side of Figure 4.22. This feedback brings the updated register values down to the register file and the new PC value down to the PC register.

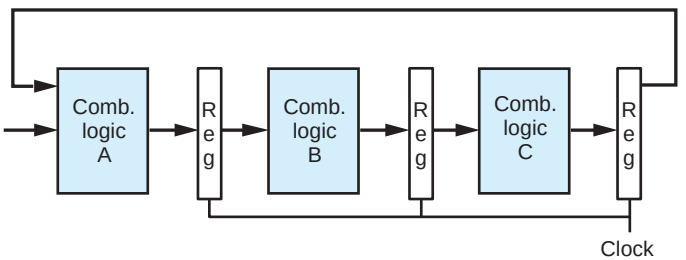
Figure 4.38 illustrates the perils of introducing pipelining into a system containing feedback paths. In the original system (Figure 4.38(a)), the result of each



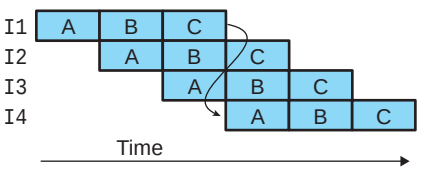
(a) Hardware: Unpipelined with feedback



(b) Pipeline diagram



(c) Hardware: Three-stage pipeline with feedback



(d) Pipeline diagram

Figure 4.38 Limitations of pipelining due to logical dependencies. I o i r o a i li \o t it \o a (a) t o a i li \o ( ), a it o tati o al a i o r, a a t t o i li \o ia ra (a \o \o).

instruction is fed back around to the next instruction. This is illustrated by the pipeline diagram (Figure 4.38(b)), where the result of I1 becomes an input to I2, and so on. If we attempt to convert this to a three-stage pipeline in the most straightforward manner (Figure 4.38(c)), we change the behavior of the system. As Figure 4.38(c) shows, the result of I1 becomes an input to I4. In attempting to speed up the system via pipelining, we have changed the system behavior.

When we introduce pipelining into a Y86-64 processor, we must deal with feedback effects properly. Clearly, it would be unacceptable to alter the system behavior as occurred in the example of Figure 4.38. Somehow we must deal with the data and control dependencies between instructions so that the resulting behavior matches the model defined by the ISA.

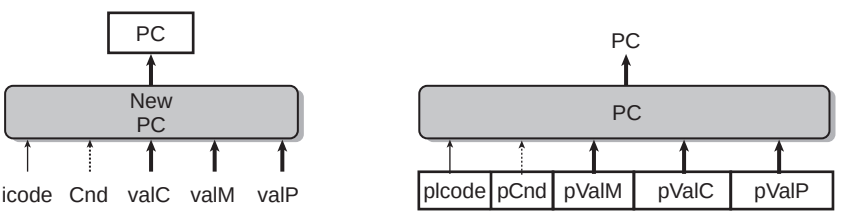
4.5 Pipelined Y86-64 Implementations

We are finally ready for the major task of this chapter—designing a pipelined Y86-64 processor. We start by making a small adaptation of the sequential processor SEQ to shift the computation of the PC into the fetch stage. We then add pipeline registers between the stages. Our first attempt at this does not handle the different data and control dependencies properly. By making some modifications, however, we achieve our goal of an efficient pipelined processor that implements the Y86-64 ISA.

4.5.1 SEQ+: Rearranging the Computation Stages

As a transitional step toward a pipelined design, we must slightly rearrange the order of the five stages in SEQ so that the PC update stage comes at the beginning of the clock cycle, rather than at the end. This transformation requires only minimal change to the overall hardware structure, and it will work better with the sequencing of activities within the pipeline stages. We refer to this modified design as SEQ+.

We can move the PC update stage so that its logic is active at the beginning of the clock cycle by making it compute the PC value for the current instruction. Figure 4.39 shows how SEQ and SEQ+ differ in their PC computation. With SEQ (Figure 4.39(a)), the PC computation takes place at the end of the clock cycle, computing the new value for the PC register based on the values of signals computed during the current clock cycle. With SEQ+ (Figure 4.39(b)), we create state registers to hold the signals computed during an instruction. Then, as a new clock cycle begins, the values propagate through the exact same logic to compute the PC for the now-current instruction. We label the registers "pIcode,"



(a) SEQ new PC computation (b) SEQ+ PC selection

Figure 4.39 Shifting the timing of the PC computation. With SEQ+, [illegible]

Aside Writ PC i SEQ+?

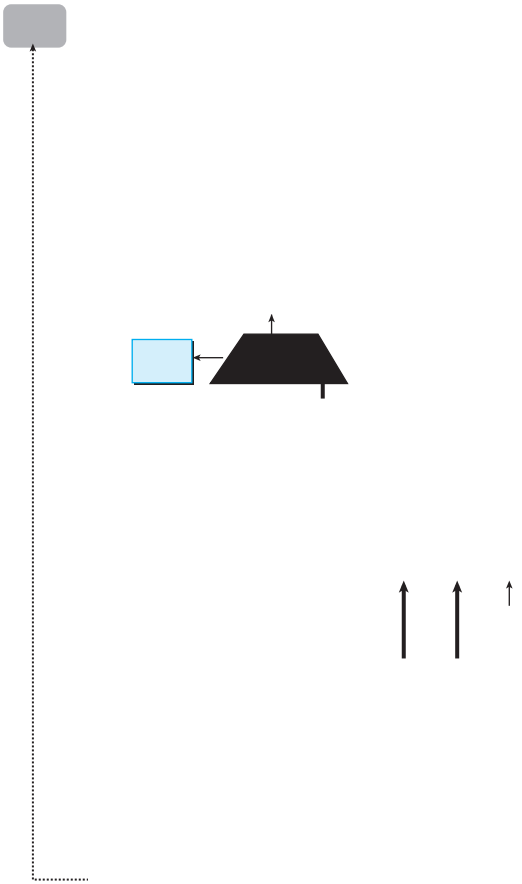
One curious feature of SEQ+ is that there is no hardware register storing the program counter. Instead, the PC is computed dynamically based on some state information stored from the previous instruction. This is a small illustration of the fact that we can implement a processor in a way that differs from the conceptual model implied by the ISA, as long as the processor correctly executes arbitrary machine-language programs. We need not encode the state in the form indicated by the programmer-visible state, as long as the processor can generate correct values for any part of the programmer-visible state (such as the program counter). We will exploit this principle even more in creating a pipelined design. Out-of-order processing techniques, as described in Section 5.7, take this idea to an extreme by executing instructions in a completely different order than they occur in the machine-level program.

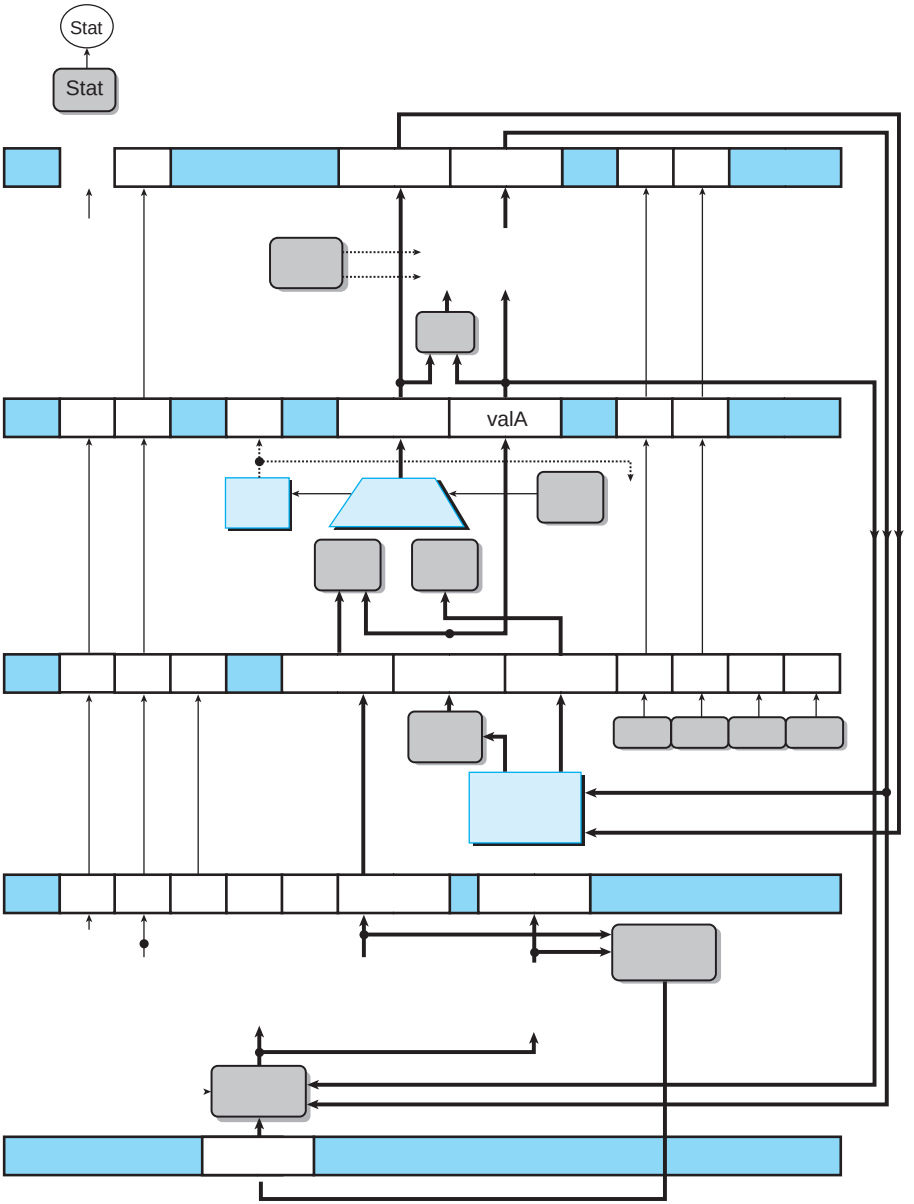
"pCnd," and so on, to indicate that on any given cycle, they hold the control signals generated during the previous cycle.

Figure 4.40 shows a more detailed view of the SEQ+ hardware. We can see that it contains the exact same hardware units and control blocks that we had in SEQ (Figure 4.23), but with the PC logic shifted from the top, where it was active at the end of the clock cycle, to the bottom, where it is active at the beginning.

The shift of state elements from SEQ to SEQ+ is an example of a general transformation known as circuit retiming [68]. Retiming changes the state representation for a system without changing its logical behavior. It is often used to balance the delays between the different stages of a pipelined system.

4.5.2 Inserting Pipeline Registers




Stat
Stat
valA

E sits between the decode and execute stages. It holds information about the most recently decoded instruction and the values read from the register file for processing by the execute stage.

M sits between the execute and memory stages. It holds the results of the most recently executed instruction for processing by the memory stage. It also holds information about branch conditions and branch targets for processing conditional jumps.

W sits between the memory stage and the feedback paths that supply the computed results to the register file for writing and the return address to the PC selection logic when completing a `ret` instruction.

Figure 4.42 shows how the following code sequence would flow through our five-stage pipeline, where the comments identify the instructions as `I1` to `I5` for reference:

```
1       irmovq  $1,%rax  # I1
2       irmovq  $2,%rbx  # I2
3       irmovq  $3,%rcx  # I3
4       irmovq  $4,%rdx  # I4
5       halt             # I5
```

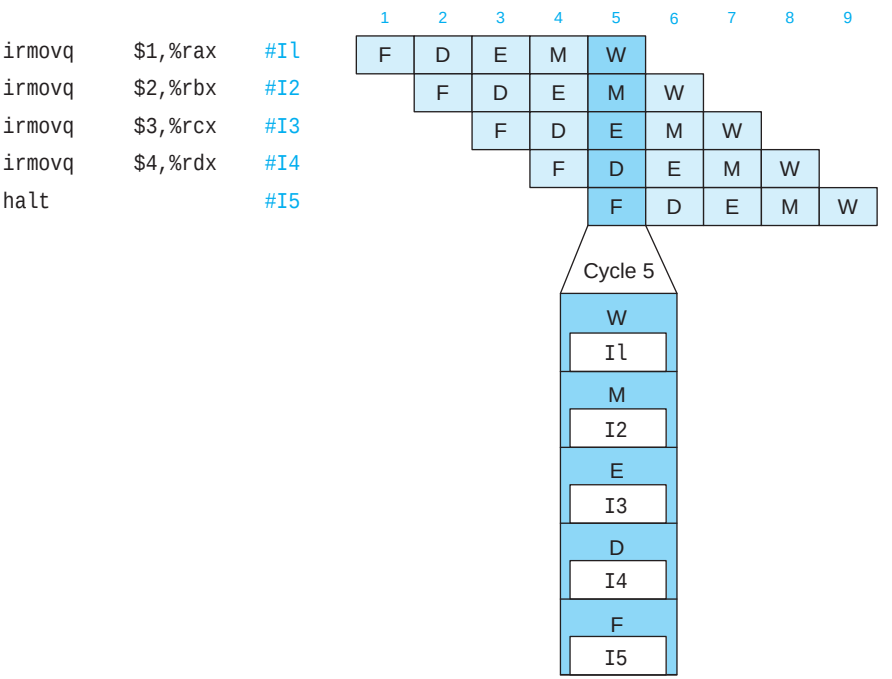


Figure 4.42 Example of instruction flow through pipeline.

The right side of the figure shows a pipeline diagram for this instruction sequence. As with the pipeline diagrams for the simple pipelined computation units of Section 4.4, this diagram shows the progression of each instruction through the pipeline stages, with time increasing from left to right. The numbers along the top identify the clock cycles at which the different stages occur. For example, in cycle 1, instruction `I1` is fetched, and it then proceeds through the pipeline stages, with its result being written to the register file after the end of cycle 5. Instruction `I2` is fetched in cycle 2, and its result is written back after the end of cycle 6, and so on. At the bottom, we show an expanded view of the pipeline for cycle 5. At this point, there is an instruction in each of the pipeline stages.

From Figure 4.42, we can also justify our convention of drawing processors so that the instructions flow from bottom to top. The expanded view for cycle 5 shows the pipeline stages with the fetch stage on the bottom and the write-back stage on the top, just as do our diagrams of the pipeline hardware (Figure 4.41). If we look at the ordering of instructions in the pipeline stages, we see that they appear in the same order as they do in the program listing. Since normal program flow goes from top to bottom of a listing, we preserve this ordering by having the pipeline flow go from bottom to top. This convention is particularly useful when working with the simulators that accompany this text.

4.5.3 Rearranging and Relabeling Signals

Our sequential implementations SEQ and SEQ+ only process one instruction at a time, and so there are unique values for signals such as alC, r A, and alE. In our pipelined design, there will be multiple versions of these values associated with the different instructions flowing through the system. For example, in the detailed structure of PIPE−, there are four white boxes labeled "Stat" that hold the status codes for four different instructions (see Figure 4.41). We need to take great care to make sure we use the proper version of a signal, or else we could have serious errors, such as storing the result computed for one instruction at the destination register specified by another instruction. We adopt a naming scheme where a signal stored in a pipeline register can be uniquely identified by prefixing its name with that of the pipe register written in uppercase. For example, the four status codes are named D_ tat, E_ tat, M_ tat, and W_ tat. We also need to refer to some signals that have just been computed within a stage. These are labeled by prefixing the signal name with the first character of the stage name, written in lowercase. Using the status codes as examples, we can see control logic blocks labeled "Stat" in the fetch and memory stages. The outputs of these blocks are therefore named _ tat and _ tat. We can also see that the actual status of the overall processor Stat is computed by a block in the write-back stage, based on the status value in pipeline register W.

The decode stages of SEQ+ and PIPE− both generate signals tE and tM indicating the destination register for values alE and alM. In SEQ+, we could connect these signals directly to the address inputs of the register file write ports. With PIPE−, these signals are carried along in the pipeline through the execute and memory stages and are directed to the register file only once they reach

Aside W at i t ' i r t i al M_ tat a ' _ tat?

With our naming system, the uppercase prefixes 'D', 'E', 'M', and 'W' refer to pipeline registers, and so M_ tat refers to the status code field of pipeline register M. The lowercase prefixes ' ', ' ', ' ', ' ', and ' ' refer to the pipeline stages, and so _ tat refers to the status signal generated in the memory stage by a control logic block.

Understanding this naming convention is critical to understanding the operation of our pipelined processors.

the write-back stage (shown in the more detailed views of the stages). We do this to make sure the write port address and data inputs hold values from the same instruction. Otherwise, the write back would be writing the values for the instruction in the write-back stage, but with register IDs from the instruction in the decode stage. As a general principle, we want to keep all of the information about a particular instruction contained within a single pipeline stage.

One block of PIPE− that is not present in SEQ+ in the exact same form is the block labeled "Select A" in the decode stage. We can see that this block generates the value alA for the pipeline register E by choosing either alP from pipeline register D or the value read from the A port of the register file. This block is included to reduce the amount of state that must be carried forward to pipeline registers E and M. Of all the different instructions, only the `call` requires alP in the memory stage. Only the jump instructions require the value of alP in the execute stage (in the event the jump is not taken). None of these instructions requires a value read from the register file. Therefore, we can reduce the amount of pipeline register state by merging these two signals and carrying them through the pipeline as a single signal alA. This eliminates the need for the block labeled "Data" in SEQ (Figure 4.23) and SEQ+ (Figure 4.40), which served a similar purpose. In hardware design, it is common to carefully identify how signals get used and then reduce the amount of register state and wiring by merging signals such as these.

As shown in Figure 4.41, our pipeline registers include a field for the status code tat, initially computed during the fetch stage and possibly modified during the memory stage. We will discuss how to implement the processing of exceptional events in Section 4.5.6, after we have covered the implementation of normal instruction execution. Suffice it to say at this point that the most systematic approach is to associate a status code with each instruction as it passes through the pipeline, as we have indicated in the figure.

4.5.4 Next PC Prediction

We have taken some measures in the design of PIPE− to properly handle control dependencies. Our goal in the pipelined design is to issue a new instruction on every clock cycle, meaning that on each clock cycle, a new instruction proceeds into the execute stage and will ultimately be completed. Achieving this goal would

Aside Other branch prediction strategies

Our design uses an always taken branch prediction strategy. Studies show this strategy has around a 60% success rate [44, 122]. Conversely, a never taken (NT) strategy has around a 40% success rate. A slightly more sophisticated strategy, known as backward taken, forward not taken (BTFNT), predicts that branches to lower addresses than the next instruction will be taken, while those to higher addresses will not be taken. This strategy has a success rate of around 65%. This improvement stems from the fact that loops are closed by backward branches and loops are generally executed multiple times. Forward branches are used for conditional operations, and these are less likely to be taken. In Problems 4.55 and 4.56, you can modify the Y86-64 pipeline processor to implement the NT and BTFNT branch prediction strategies.

As we saw in Section 3.6.6, mispredicted branches can degrade the performance of a program considerably, thus motivating the use of conditional data transfer rather than conditional control transfer when possible.

Aside Return prediction

With most programs, it is very easy to predict return addresses, since procedure calls and returns occur in matched pairs. Most of the time that a procedure is called, it returns to the instruction following the call. This property is exploited in high-performance processors by including a hardware stack within the instruction fetch unit that holds the return address generated by procedure call instructions. Every time a procedure call instruction is executed, its return address is pushed onto the stack. When a return instruction is fetched, the top value is popped from this stack and used as the predicted return address. Like branch prediction, a mechanism must be provided to recover when the prediction was incorrect, since there are times when calls and returns do not match. In general, the prediction is highly reliable. This hardware stack is not part of the programmer-visible state.

results, since the return address will be whatever word is on the top of the stack. In our design, we will not attempt to predict any value for the return address. Instead, we will simply hold off processing any more instructions until the `ret` instruction passes through the write-back stage. We will return to this part of the implementation in Section 4.5.8.

The PIPE− fetch stage, diagrammed at the bottom of Figure 4.41, is responsible for both predicting the next value of the PC and selecting the actual PC for the instruction fetch. We can see the block labeled "Predict PC" can choose either valP (as computed by the PC incrementer) or valC (from the fetched instruction). This value is stored in pipeline register F as the predicted value of the program counter. The block labeled "Select PC" is similar to the block labeled "PC" in the SEQ+ PC selection stage (Figure 4.40). It chooses one of three values to serve as the address for the instruction memory: the predicted PC, the value of valP for a not-taken branch instruction that reaches pipeline register M (stored in register M_valA), or the value of the return address when a `ret` instruction reaches pipeline register W (stored in W_valM).

4.5.5 Pipeline Hazards

Our structure PIPE− is a good start at creating a pipelined Y86-64 processor. Recall from our discussion in Section 4.4.4, however, that introducing pipelining into a system with feedback can lead to problems when there are dependencies between successive instructions. We must resolve this issue before we can complete our design. These dependencies can take two forms: (1) data dependencies, where the results computed by one instruction are used as the data for a following instruction, and (2) control dependencies, where one instruction determines the location of the following instruction, such as when executing a jump, call, or return. When such dependencies have the potential to cause an erroneous computation by the pipeline, they are called hazards. Like dependencies, hazards can be classified as either data hazards or control hazards. We first concern ourselves with data hazards and then consider control hazards.

```
# prog2
0x000: irmovq $10,%rdx
0x00a: irmovq  $3,%rax
0x014: nop
0x015: nop
0x016: addq %rdx,%rax
0x018: halt
```

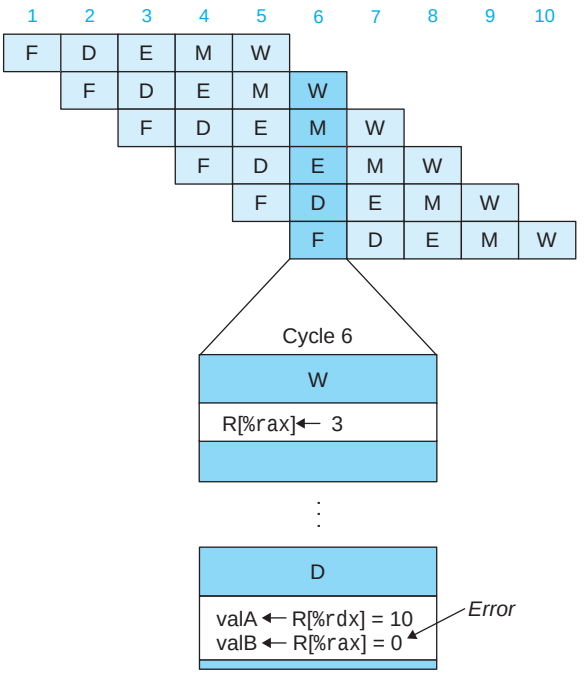


Figure 4.44 Pipelined execution of prog2 without special pipeline control. T rit to ro ra r it r %rax\ o ot o r tit tart o\ l 7, a \ o t addq i tr ti o t t i orr t al ort i r it r i t \ o\ ta .

dependencies. Let us see what happens as these nop instructions are removed. Figure 4.44 illustrates the pipeline flow of a program, named prog2, containing two nop instructions between the two irmovq instructions generating values for registers %rdx and %rax and the addq instruction having these two registers as operands. In this case, the crucial step occurs in cycle 6, when the addq instruction reads its operands from the register file. An expanded view of the pipeline activities during this cycle is shown at the bottom of the figure. The first irmovq instruction has passed through the write-back stage, and so program register %rdx has been updated in the register file. The second irmovq instruction is in the write-back stage during this cycle, and so the write to program register %rax only occurs at the start of cycle 7 as the clock rises. As a result, the incorrect value zero would be read for register %rax (recall that we assume all registers are initially zero), since the pending write for this register has not yet occurred. Clearly, we will have to adapt our pipeline to handle this hazard properly.

Figure 4.45 shows what happens when we have only one nop instruction between the irmovq instructions and the addq instruction, yielding a program prog3. Now we must examine the behavior of the pipeline during cycle 5 as the addq instruction passes through the decode stage. Unfortunately, the pending

```
# prog3
0x000: irmovq $10,%rdx
0x00a: irmovq  $3,%rax
0x014: nop
0x015: addq %rdx,%rax
0x017: halt
```

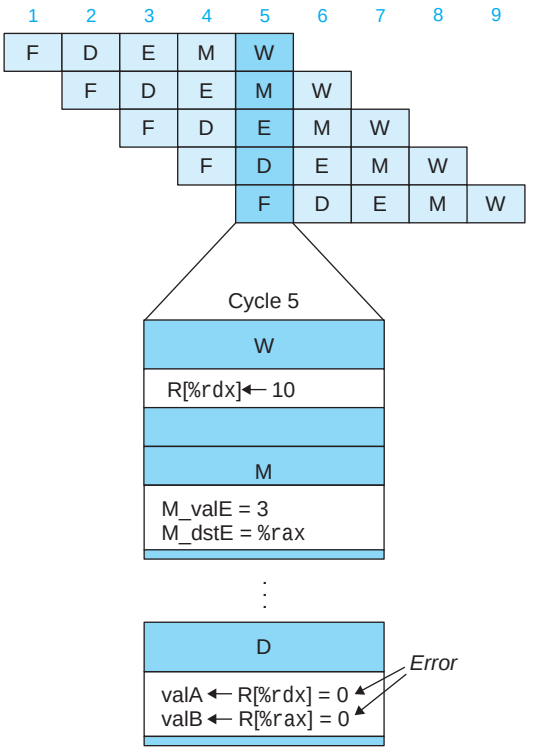


Figure 4.45 Pipelined execution of prog3 without special pipeline control. I 5, t addq i tr ti r a it r ra r t r it r fil . T i rit t r it r %rdx i till i t rit - a ta , a t i rit t r it r %rax i till i t r ta . B t ra alA a alB t i rr t al .

write to register %rdx is still in the write-back stage, and the pending write to %rax is still in the memory stage. Therefore, the addq instruction would get the incorrect values for both operands.

Figure 4.46 shows what happens when we remove all of the nop instructions between the irmovq instructions and the addq instruction, yielding a program prog4. Now we must examine the behavior of the pipeline during cycle 4 as the addq instruction passes through the decode stage. Unfortunately, the pending write to register %rdx is still in the memory stage, and the new value for %rax is just being computed in the execute stage. Therefore, the addq instruction would get the incorrect values for both operands.

These examples illustrate that a data hazard can arise for an instruction when one of its operands is updated by any of the three preceding instructions. These hazards occur because our pipelined processor reads the operands for an instruction from the register file in the decode stage but does not write the results for the instruction to the register file until three cycles later, after the instruction passes through the write-back stage.

```
# prog4
0x000: irmovq $10,%rdx
0x00a: irmovq  $3,%rax
0x014: addq %rdx,%rax
0x016: halt
```

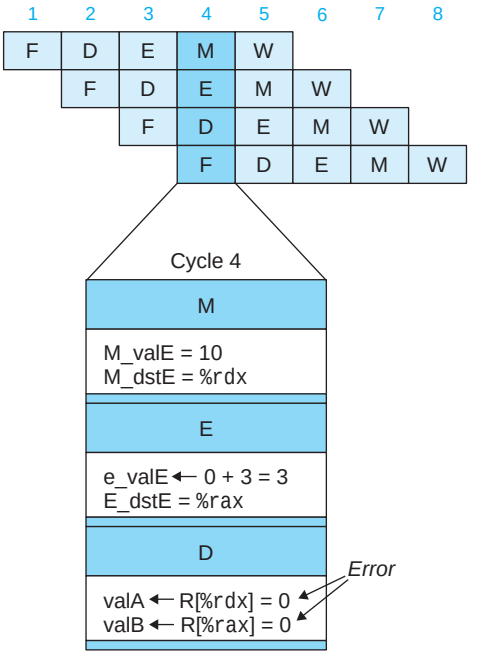
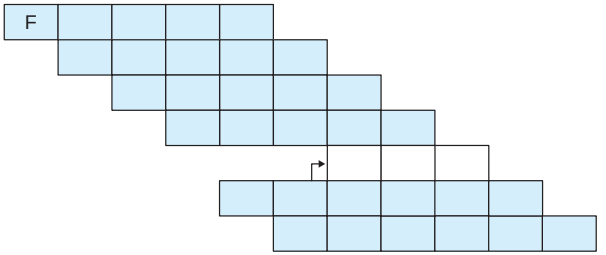


Figure 4.46 Pipelined execution of prog4 without special pipeline control. I I 4, t addq i tr ti r a it o r o ra r t r it r fil . T i rit t o r it r %rdx i till i t o r ta , a t al o r r it r %rax i t i o t i t t ta . B o t o ra alA a alB t i o rr t al .

A i Data Ha ar Stall i

One very general technique for avoiding hazards involves stalling, where the processor holds back one or more instructions in the pipeline until the hazard condition no longer holds. Our processor can avoid data hazards by holding back an instruction in the decode stage until the instructions generating its source operands have passed through the write-back stage. The details of this mechanism will be discussed in Section 4.5.8. It involves simple enhancements to the pipeline control logic. The effect of stalling is diagrammed in Figure 4.47 (prog2) and Figure 4.48 (prog4). (We omit prog3 from this discussion, since it operates similarly to the other two examples.) When the addq instruction is in the decode stage, the pipeline control logic detects that at least one of the instructions in the execute, memory, or write-back stage will update either register %rdx or register %rax. Rather than letting the addq instruction pass through the stage with the incorrect results, it stalls the instruction, holding it back in the decode stage for either one (for prog2) or three (for prog4) extra cycles. For all three programs, the addq instruction finally gets correct values for its two source operands in cycle 7 and then proceeds down the pipeline.

```
# prog2
0x000: irmovq $10,%rdx
0x00a: irmovq  $3,%rax
0x014: nop
0x015: nop
       bubble
0x016: addlq %rdx,%rax
0x018: halt
```



Aside Exploring data hazards

Hazards can potentially occur when one instruction updates part of the program state that will be read by a later instruction. For Y86-64, the program state includes the program registers, the program counter, the memory, the condition code register, and the status register. Let us look at the hazard possibilities in our proposed design for each of these forms of state.

Program registers. These are the hazards we have already identified. They arise because the register file is read in one stage and written in another, leading to possible unintended interactions between different instructions.

Program counter. Conflicts between updating and reading the program counter give rise to control hazards. No hazard arises when our fetch-stage logic correctly predicts the new value of the program counter before fetching the next instruction. Mispredicted branches and `ret` instructions require special handling, as will be discussed in Section 4.5.5.

Memory. Writes and reads of the data memory both occur in the memory stage. By the time an instruction reading memory reaches this stage, any preceding instructions writing memory will have already done so. On the other hand, there can be interference between instructions writing data in the memory stage and the reading of instructions in the fetch stage, since the instruction and data memories reference a single address space. This can only happen with programs containing self-modifying code, where instructions write to a portion of memory from which instructions are later fetched. Some systems have complex mechanisms to detect and avoid such hazards, while others simply mandate that programs should not use self-modifying code. We will assume for simplicity that programs do not modify themselves, and therefore we do not need to take special measures to update the instruction memory based on updates to the data memory during program execution.

Condition code register. These are written by integer operations in the execute stage. They are read by conditional moves in the execute stage and by conditional jumps in the memory stage. By the time a conditional move or jump reaches the execute stage, any preceding integer operation will have already completed this stage. No hazards can arise.

Status register. The program status can be affected by instructions as they flow through the pipeline. Our mechanism of associating a status code with each instruction in the pipeline enables the processor to come to an orderly halt when an exception occurs, as will be discussed in Section 4.5.6.

This analysis shows that we only need to deal with register data hazards, control hazards, and making sure exceptions are handled properly. A systematic analysis of this form is important when designing a complex system. It can identify the potential difficulties in implementing the system, and it can guide the generation of test programs to be used in checking the correctness of the system.

condition codes, or the program status. These are shown as white boxes in the pipeline diagrams of Figures 4.47 and 4.48. In these figures the arrow between the box labeled “D” for the `addq` instruction and the box labeled “E” for one of the pipeline bubbles indicates that a bubble was injected into the execute stage in place of the `addq` instruction that would normally have passed from the decode to

the execute stage. We will look at the detailed mechanisms for making the pipeline stall and for injecting bubbles in Section 4.5.8.

In using stalling to handle data hazards, we effectively execute programs `prog2` and `prog4` by dynamically generating the pipeline flow seen for `prog1` (Figure 4.43). Injecting one bubble for `prog2` and three for `prog4` has the same effect as having three `nop` instructions between the second `irmovq` instruction and the `addq` instruction. This mechanism can be implemented fairly easily (see Problem 4.53), but the resulting performance is not very good. There are numerous cases in which one instruction updates a register and a closely following instruction uses the same register. This will cause the pipeline to stall for up to three cycles, reducing the overall throughput significantly.

A i Data Ha ar F r ar i

Our design for PIPE− reads source operands from the register file in the decode stage, but there can also be a pending write to one of these source registers in the write-back stage. Rather than stalling until the write has completed, it can simply pass the value that is about to be written to pipeline register E as the source operand. Figure 4.49 shows this strategy with an expanded view of the pipeline diagram for cycle 6 of `prog2`. The decode-stage logic detects that register

```
# prog2
0x000: irmovq $10,%rdx
0x00a: irmovq  $3,%rax
0x014: nop
0x015: nop
0x016: addq %rdx,%rax
0x018: halt
```

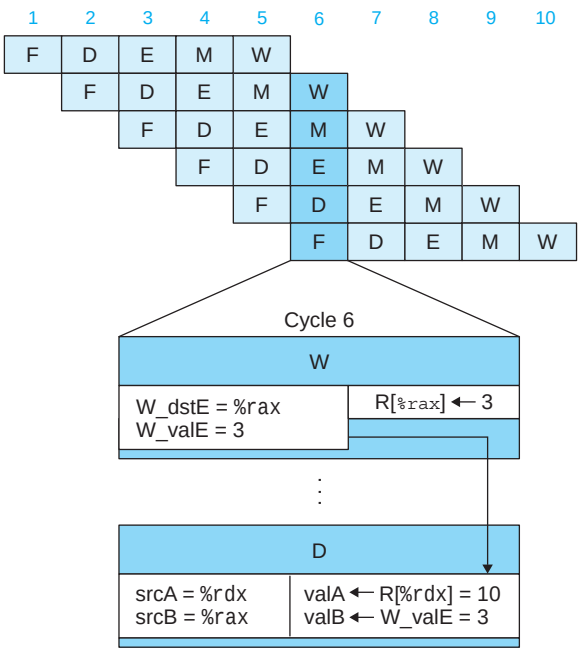


Figure 4.49 Pipelined execution of `prog2` using forwarding. I 6, t - ta i t t r a i rit t r it r

```
# prog3
0x000: irmovq $10,%rdx
0x00a: irmovq  $3,%rax
0x014: nop
0x005: addq %rdx,%rax
0x017: halt
```

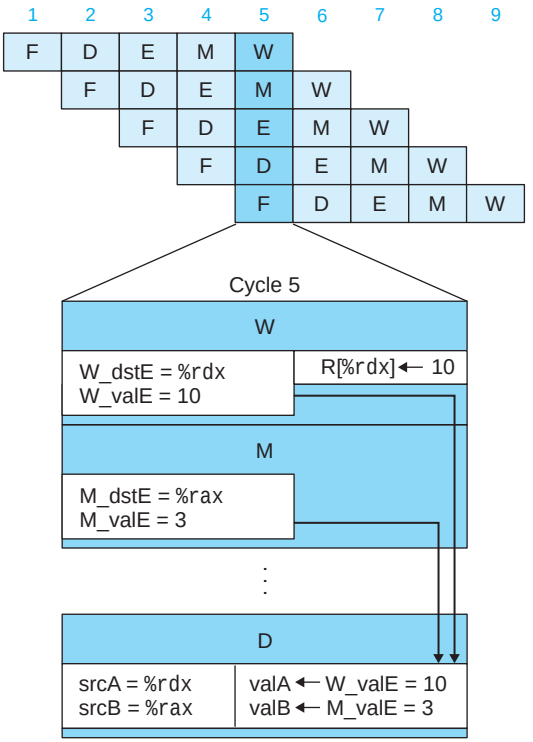


Figure 4.50 Pipelined execution of prog3 using forwarding. I l 5, t ' ' o' - ta l i ' t t a ' i rit t r it r %rdx i t rit - a ta a ' t r it r %rax i t r ta . It t a t al r alA a ' alB rat r t a t al r a' r t r it r fil .

%rax is the source register for operand alB, and that there is also a pending write to %rax on write port E. It can therefore avoid stalling by simply using the data word supplied to port E (signal W_ alE) as the value for operand alB. This technique of passing a result value directly from one pipeline stage to an earlier one is commonly known as data forwarding (or simply forwarding, and sometimes bypassing). It allows the instructions of prog2 to proceed through the pipeline without any stalling. Data forwarding requires adding additional data connections and control logic to the basic hardware structure.

As Figure 4.50 illustrates, data forwarding can also be used when there is a pending write to a register in the memory stage, avoiding the need to stall for program prog3. In cycle 5, the decode-stage logic detects a pending write to register %rdx on port E in the write-back stage, as well as a pending write to register %rax that is on its way to port E but is still in the memory stage. Rather than stalling until the writes have occurred, it can use the value in the write-back stage (signal W_ alE) for operand alA and the value in the memory stage (signal M_ alE) for operand alB.

```
# prog4
0x000: irmovq $10,%rdx
0x00a: irmovq  $3,%rax
0x014: addq %rdx,%rax
0x016: halt
```

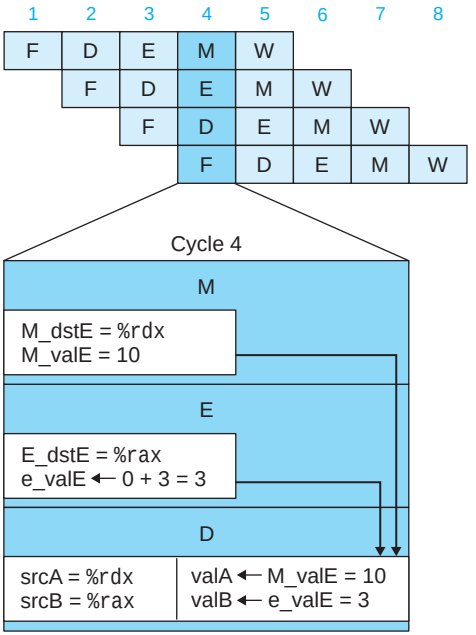


Figure 4.51 Pipelined execution of prog4 using forwarding. I l 4, t ' ' -
ta l i ' t t a ' i r t t r it r %rdx i t r ta . It al ' t t
t at a al i i t ' r r it r %rax i t t ta . It t
a t al r alA a ' alB rat r t a t al r a ' r t r it r fil .

To exploit data forwarding to its full extent, we can also pass newly computed values from the execute stage to the decode stage, avoiding the need to stall for program prog4, as illustrated in Figure 4.51. In cycle 4, the decode-stage logic detects a pending write to register %rdx in the memory stage, and also that the value being computed by the ALU in the execute stage will later be written to register %rax. It can use the value in the memory stage (signal M_ alE) for operand alA. It can also use the ALU output (signal _ alE) for operand alB. Note that using the ALU output does not introduce any timing problems. The decode stage only needs to generate signals alA and alB by the end of the clock cycle so that pipeline register E can be loaded with the results from the decode stage as the clock rises to start the next cycle. The ALU output will be valid before this point.

The uses of forwarding illustrated in programs prog2 to prog4 all involve the forwarding of values generated by the ALU and destined for write port E. Forwarding can also be used with values read from the memory and destined for write port M. From the memory stage, we can forward the value that has just been read from the data memory (signal _ alM). From the write-back stage, we can forward the pending write to port M (signal W_ alM). This gives a total of five different forwarding sources (_ alE, _ alM, M_ alE, W_ alM, and W_ alE) and two different forwarding destinations (alA and alB).

The expanded diagrams of Figures 4.49 to 4.51 also show how the decode-stage logic can determine whether to use a value from the register file or to use a forwarded value. Associated with every value that will be written back to the register file is the destination register ID. The logic can compare these IDs with the source register IDs `r A` and `r B` to detect a case for forwarding. It is possible to have multiple destination register IDs match one of the source IDs. We must establish a priority among the different forwarding sources to handle such cases. This will be discussed when we look at the detailed design of the forwarding logic.

Figure 4.52 shows the structure of PIPE, an extension of PIPE− that can handle data hazards by forwarding. Comparing this to the structure of PIPE− (Figure 4.41), we can see that the values from the five forwarding sources are fed back to the two blocks labeled "Sel+Fwd A" and "Fwd B" in the decode stage. The block labeled "Sel+Fwd A" combines the role of the block labeled "Select A" in PIPE− with the forwarding logic. It allows `alA` for pipeline register E to be either the incremented program counter `alP`, the value read from the A port of the register file, or one of the forwarded values. The block labeled "Fwd B" implements the forwarding logic for source operand `alB`.

L$_o$ā /U Data Ha aī

One class of data hazards cannot be handled purely by forwarding, because memory reads occur late in the pipeline. Figure 4.53 illustrates an example of a load/use hazard, where one instruction (the `mrmovq` at address `0x028`) reads a value from memory for register `%rax` while the next instruction (the `addq` at address `0x032`) needs this value as a source operand. Expanded views of cycles 7 and 8 are shown in the lower part of the figure, where we assume all program registers initially have

,addqanstruction Trequirs 0f Tycles-278(T7)-247(byut-2968vit-2968vi)]TJ /-7.6 -1.2 Td [(net)-207(,gen

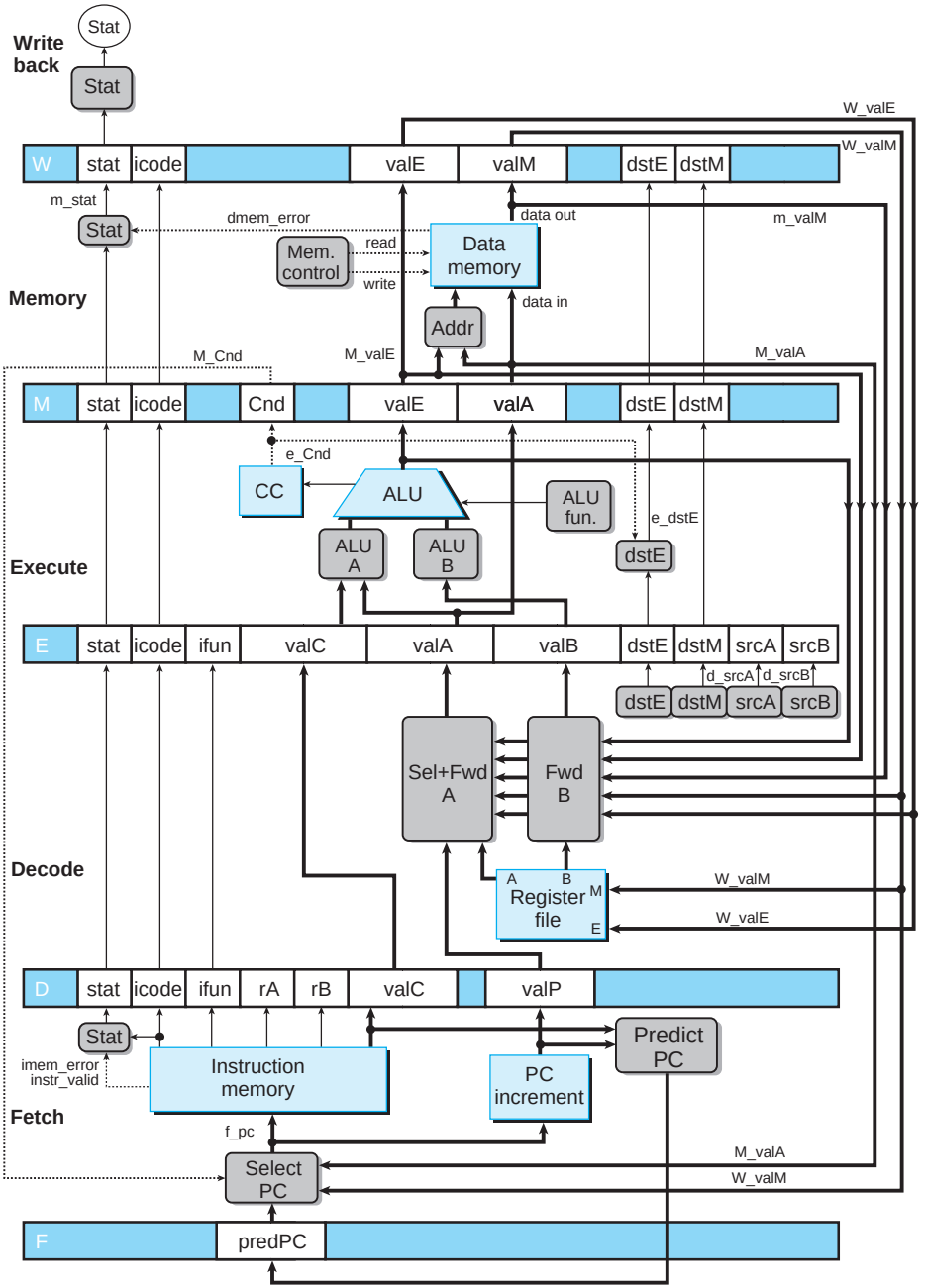


Figure 4.52 Hardware structure of PIPE, our final pipelined implementation. T a i al a i at al r ar i t r lt r t tr r i i tr ti . T i all t a l t r ata a ar it t talli t i li .

```
# prog5
0x000: irmovq $128,%rdx
0x00a: irmovq  $3,%rcx
0x014: rmmovq %rcx, 0(%rdx)
0x01e: irmovq $10,%rbx
0x028: mrmovq 0(%rdx),%rax # Load %rax
0x032: addq %ebx,%eax # Use %rax
0x034: halt
```

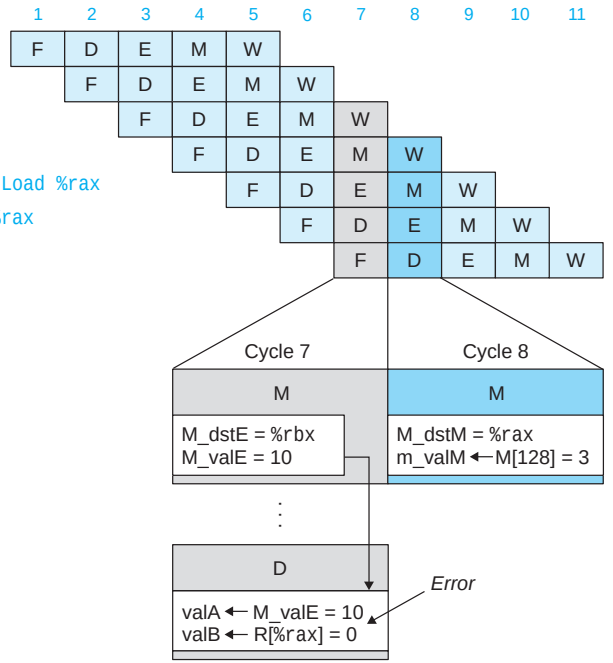


Figure 4.53 Example of load/use data hazard. The addq instruction requires the value of %rax during cycle 7. The mrmovq instruction generates this value during cycle 8, so it cannot pass from the decode stage to the execute stage until cycle 8.

This use of a stall to handle a load/use hazard is called a load interlock. Load interlocks combined with forwarding suffice to handle all possible forms of data hazards. Since only load interlocks reduce the pipeline throughput, we can nearly achieve our throughput goal of issuing one new instruction on every clock cycle.

Avoiding Control Hazards

Control hazards arise when the processor cannot reliably determine the address of the next instruction based on the current instruction in the fetch stage. As was discussed in Section 4.5.4, control hazards can only occur in our pipelined processor for `ret` and jump instructions. Moreover, the latter case only causes difficulties when the direction of a conditional jump is mispredicted. In this section, we provide a high-level view of how these hazards can be handled. The detailed implementation will be presented in Section 4.5.8 as part of a more general discussion of the pipeline control.

For the `ret` instruction, consider the following example program. This program is shown in assembly code, but with the addresses of the different instructions on the left for reference:

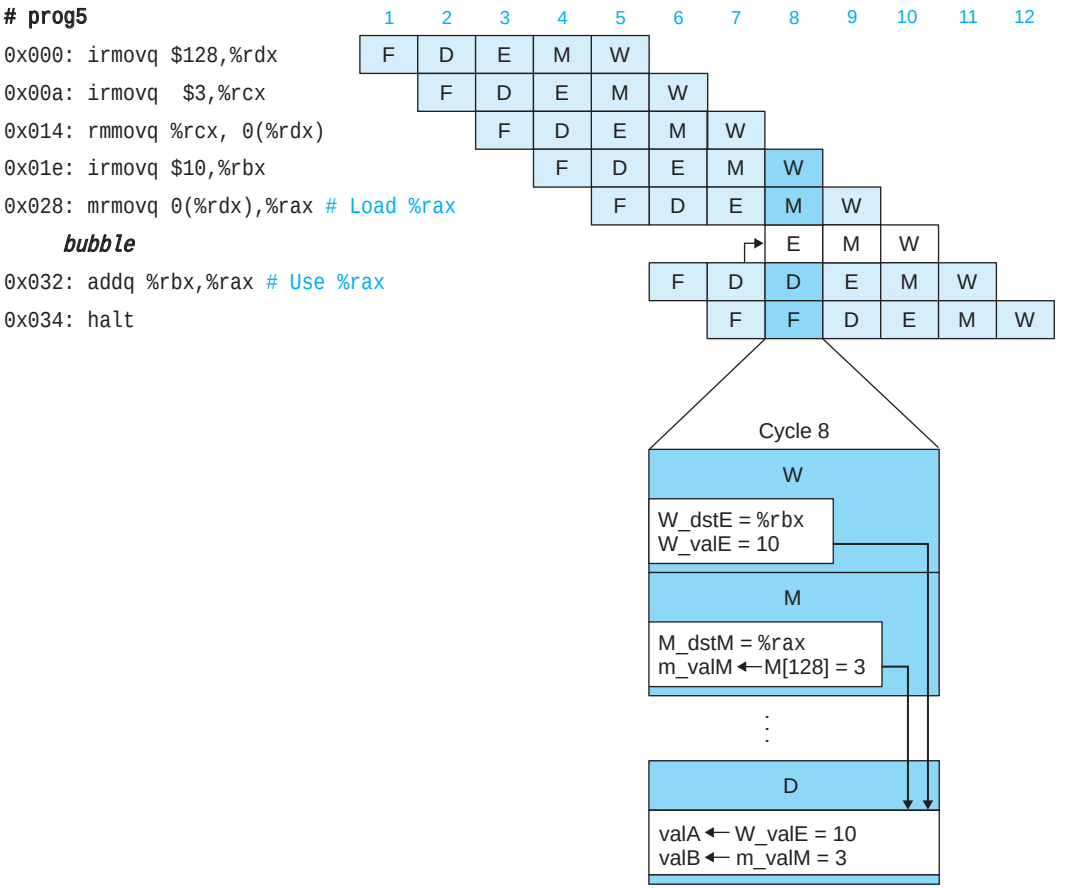


Figure 4.54 Handling a load/use hazard by stalling. B talli t addq i tr ti o r o l i t ta , t al r alB a r ar r o t mrmovq i tr ti o i t o r ta t o t addq i tr ti o i t ta .

```
0x000:    irmovq stack,%rsp  #   Initialize stack pointer
0x00a:    call proc          #   Procedure call
0x013:    irmovq $10,%rdx    #   Return point
0x01d:    halt
0x020: .pos 0x20
0x020: proc:                 # proc:
0x020:    ret                #   Return immediately
0x021:    rrmovq %rdx,%rbx   #   Not executed
0x030: .pos 0x30
0x030: stack:                # stack: Stack pointer
```

Figure 4.55 shows how we want the pipeline to process the `ret` instruction. As with our earlier pipeline diagrams, this figure shows the pipeline activity with

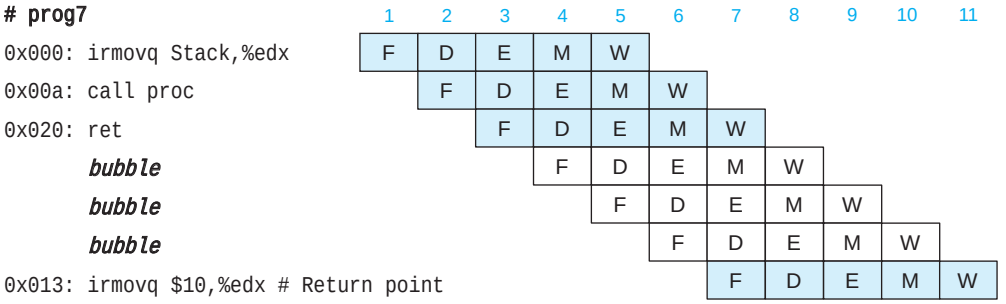


Figure 4.55 Simplified view of ret instruction processing. T i li o tall i t ret a t r t o , t a r ta , i ti t r l i t r . T PC l ti o l i ill oo t r t r a r a t i tr ti t a r o t retr a t rt - a ta (l 7).

time growing to the right. Unlike before, the instructions are not listed in the same order they occur in the program, since this program involves a control flow where instructions are not executed in a linear sequence. It is useful to look at the instruction addresses to identify the different instructions in the program.

As this diagram shows, the ret instruction is fetched during cycle 3 and proceeds down the pipeline, reaching the write-back stage in cycle 7. While it passes through the decode, execute, and memory stages, the pipeline cannot do any useful activity. Instead, we want to inject three bubbles into the pipeline. Once the ret instruction reaches the write-back stage, the PC selection logic will set the program counter to the return address, and therefore the fetch stage will fetch the irmovq instruction at the return point (address 0x013).

To handle a mispredicted branch, consider the following program, shown in assembly code but with the instruction addresses shown on the left for reference:

```
0x000:    xorq %rax,%rax
0x002:    jne  target        # Not taken
0x00b:    irmovq $1, %rax    # Fall through
0x015:    halt
0x016: target:
0x016:    irmovq $2, %rdx    # Target
0x020:    irmovq $3, %rbx    # Target+1
0x02a:    halt
```

Figure 4.56 shows how these instructions are processed. As before, the instructions are listed in the order they enter the pipeline, rather than the order they occur in the program. Since the jump instruction is predicted as being taken, the instruction at the jump target will be fetched in cycle 3, and the instruction following this one will be fetched in cycle 4. By the time the branch logic detects that the jump should not be taken during cycle 4, two instructions have been fetched that should not continue being executed. Fortunately, neither of these instructions has caused a change in the programmer-visible state. That can only occur when an instruction

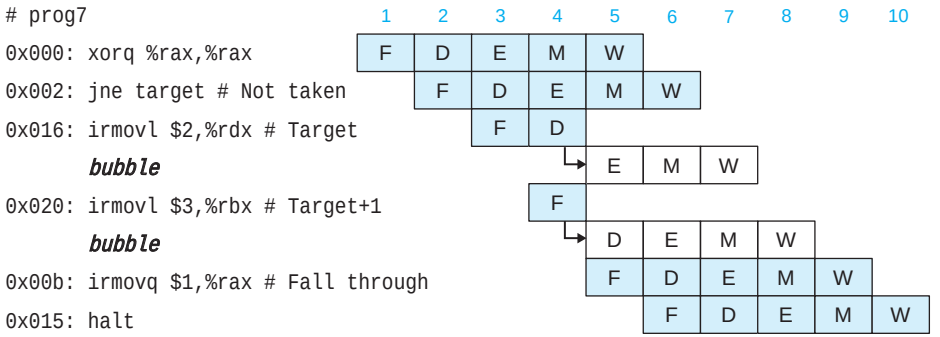


Figure 4.56 Processing mispredicted branch instructions. T i li r it ra ll ta a tart t i i tr ti at t tar t. T i tr ti ar t r t i r iti i t t i l 4 t i tr ti t r t t ta . l l 5, t i li cancels t t tar t i tr ti i ti l i t t a t ta , a it al t t i tr ti ll i t .

reaches the execute stage, where it can cause the condition codes to change. At this point, the pipeline can simply cancel (sometimes called instruction squashing) the two misfetched instructions by injecting bubbles into the decode and execute stages on the following cycle while also fetching the instruction following the jump instruction. The two misfetched instructions will then simply disappear from the pipeline and therefore not have any effect on the programmer-visible state. The only drawback is that two clock cycles' worth of instruction processing capability have been wasted.

This discussion of control hazards indicates that they can be handled by careful consideration of the pipeline control logic. Techniques such as stalling and injecting bubbles into the pipeline dynamically adjust the pipeline flow when special conditions arise. As we will discuss in Section 4.5.8, a simple extension to the basic clocked register design will enable us to stall stages and to inject bubbles into pipeline registers as part of the pipeline control logic.

4.5.6 Exception Handling

As we will discuss in Chapter 8, a variety of activities in a processor can lead to exceptional control flow, where the normal chain of program execution gets broken. Exceptions can be generated either internally, by the executing program, or externally, by some outside signal. Our instruction set architecture includes three different internally generated exceptions, caused by (1) a `halt` instruction, (2) an instruction with an invalid combination of instruction and function code, and (3) an attempt to access an invalid address, either for instruction fetch or data read or write. A more complete processor design would also handle external exceptions, such as when the processor receives a signal that the network interface has received a new packet or the user has clicked a mouse button. Handling

exceptions correctly is a challenging aspect of any microprocessor design. They can occur at unpredictable times, and they require creating a clean break in the flow of instructions through the processor pipeline. Our handling of the three internal exceptions gives just a glimpse of the true complexity of correctly detecting and handling exceptions.

Let us refer to the instruction causing the exception as the excepting instruction. In the case of an invalid instruction address, there is no actual excepting instruction, but it is useful to think of there being a sort of "virtual instruction" at the invalid address. In our simplified ISA model, we want the processor to halt when it reaches an exception and to set the appropriate status code, as listed in Figure 4.5. It should appear that all instructions up to the excepting instruction have completed, but none of the following instructions should have any effect on the programmer-visible state. In a more complete design, the processor would continue by invoking an exception handler, a procedure that is part of the operating system, but implementing this part of exception handling is beyond the scope of our presentation.

In a pipelined system, exception handling involves several subtleties. First, it is possible to have exceptions triggered by multiple instructions simultaneously. For example, during one cycle of pipeline operation, we could have a `halt` instruction in the fetch stage, and the data memory could report an out-of-bounds data address for the instruction in the memory stage. We must determine which of these exceptions the processor should report to the operating system. The basic rule is to put priority on the exception triggered by the instruction that is furthest along the pipeline. In the example above, this would be the out-of-bounds address attempted by the instruction in the memory stage. In terms of the machine-language program, the instruction in the memory stage should appear to execute before one in the fetch stage, and therefore only this exception should be reported to the operating system.

A second subtlety occurs when an instruction is first fetched and begins execution, causes an exception, and later is canceled due to a mispredicted branch. The following is an example of such a program in its object-code form:

```
0x000: 6300                 |    xorq %rax,%rax
0x002: 741600000000000000   |    jne  target      # Not taken
0x00b: 30f00100000000000000 |    irmovq $1, %rax  # Fall through
0x015: 00                   |    halt
0x016:                      | target:
0x016: ff                   |    .byte 0xFF       # Invalid instruction code
```

In this program, the pipeline will predict that the branch should be taken, and so it will fetch and attempt to use a byte with value `0xFF` as an instruction (generated in the assembly code using the `.byte` directive). The decode stage will therefore detect an invalid instruction exception. Later, the pipeline will discover that the branch should not be taken, and so the instruction at address `0x016` should never even have been fetched. The pipeline control logic will cancel this instruction, but we want to avoid raising an exception.

A third subtlety arises because a pipelined processor updates different parts of the system state in different stages. It is possible for an instruction following one causing an exception to alter some part of the state before the excepting instruction completes. For example, consider the following code sequence, in which we assume that user programs are not allowed to access addresses at the upper end of the 64-bit range:

```
1     irmovq $1,%rax
2     xorq %rsp,%rsp   # Set stack pointer to 0 and CC to 100
3     pushq %rax       # Attempt to write to 0xfffffffffffffff8
4     addq  %rax,%rax  # (Should not be executed) Would set CC to 000
```

The `pushq` instruction causes an address exception, because decrementing the stack pointer causes it to wrap around to `0xfffffffffffffff8`. This exception is detected in the memory stage. On the same cycle, the `addq` instruction is in the execute stage, and it will cause the condition codes to be set to new values. This would violate our requirement that none of the instructions following the excepting instruction should have had any effect on the system state.

In general, we can both correctly choose among the different exceptions and avoid raising exceptions for instructions that are fetched due to mispredicted branches by merging the exception-handling logic into the pipeline structure. That is the motivation for us to include a status code `stat` in each of our pipeline registers (Figures 4.41 and 4.52). If an instruction generates an exception at some stage in its processing, the status field is set to indicate the nature of the exception. The exception status propagates through the pipeline with the rest of the information for that instruction, until it reaches the write-back stage. At this point, the pipeline control logic detects the occurrence of the exception and stops execution.

To avoid having any updating of the programmer-visible state by instructions beyond the excepting instruction, the pipeline control logic must disable any updating of the condition code register or the data memory when an instruction in the memory or write-back stages has caused an exception. In the example program above, the control logic will detect that the `pushq` in the memory stage has caused an exception, and therefore the updating of the condition code register by the `addq` instruction in the execute stage will be disabled.

Let us consider how this method of handling exceptions deals with the subtleties we have mentioned. When an exception occurs in one or more stages of a pipeline, the information is simply stored in the status fields of the pipeline registers. The event has no effect on the flow of instructions in the pipeline until an excepting instruction reaches the final pipeline stage, except to disable any updating of the programmer-visible state (the condition code register and the memory) by later instructions in the pipeline. Since instructions reach the write-back stage in the same order as they would be executed in a nonpipelined processor, we are guaranteed that the first instruction encountering an exception will arrive first in the write-back stage, at which point program execution can stop and the status code in pipeline register W can be recorded as the program status. If some instruction is fetched but later canceled, any exception status information about the

instruction gets canceled as well. No instruction following one that causes an exception can alter the programmer-visible state. The simple rule of carrying the exception status together with all other information about an instruction through the pipeline provides a simple and reliable mechanism for handling exceptions.

4.5.7 PIPE Stage Implementations

We have now created an overall structure for PIPE, our pipelined Y86-64 processor with forwarding. It uses the same set of hardware units as the earlier sequential designs, with the addition of pipeline registers, some reconfigured logic blocks, and additional pipeline control logic. In this section, we go through the design of the different logic blocks, deferring the design of the pipeline control logic to the next section. Many of the logic blocks are identical to their counterparts in SEQ and SEQ+, except that we must choose proper versions of the different signals from the pipeline registers (written with the pipeline register name, written in uppercase, as a prefix) or from the stage computations (written with the first character of the stage name, written in lowercase, as a prefix).

As an example, compare the HCL code for the logic that generates the `r A` signal in SEQ to the corresponding code in PIPE:

```
    # Code from SEQ

word srcA = [
        icode in { IRRMOVQ, IRMMOVQ, IOPQ, IPUSHQ  } : rA;
        icode in { IPOPQ, IRET } : RRSP;
        1 : RNONE; # Don't need register
];

    # Code from PIPE

word d_srcA = [
        D_icode in { IRRMOVQ, IRMMOVQ, IOPQ, IPUSHQ  } : D_rA;
        D_icode in { IPOPQ, IRET } : RRSP;
        1 : RNONE; # Don't need register
];
```

They differ only in the prefixes added to the PIPE signals: `D_` for the source values, to indicate that the signals come from pipeline register D, and `d_` for the result value, to indicate that it is generated in the decode stage. To avoid repetition, we will not show the HCL code here for blocks that only differ from those in SEQ because of the prefixes on names. As a reference, the complete HCL code for PIPE is given in Web Aside ARCH:HCL on page 508.

PC S l t i a F t Sta

Figure 4.57 provides a detailed view of the PIPE fetch stage logic. As discussed earlier, this stage must also select a current value for the program counter and predict the next PC value. The hardware units for reading the instruction from

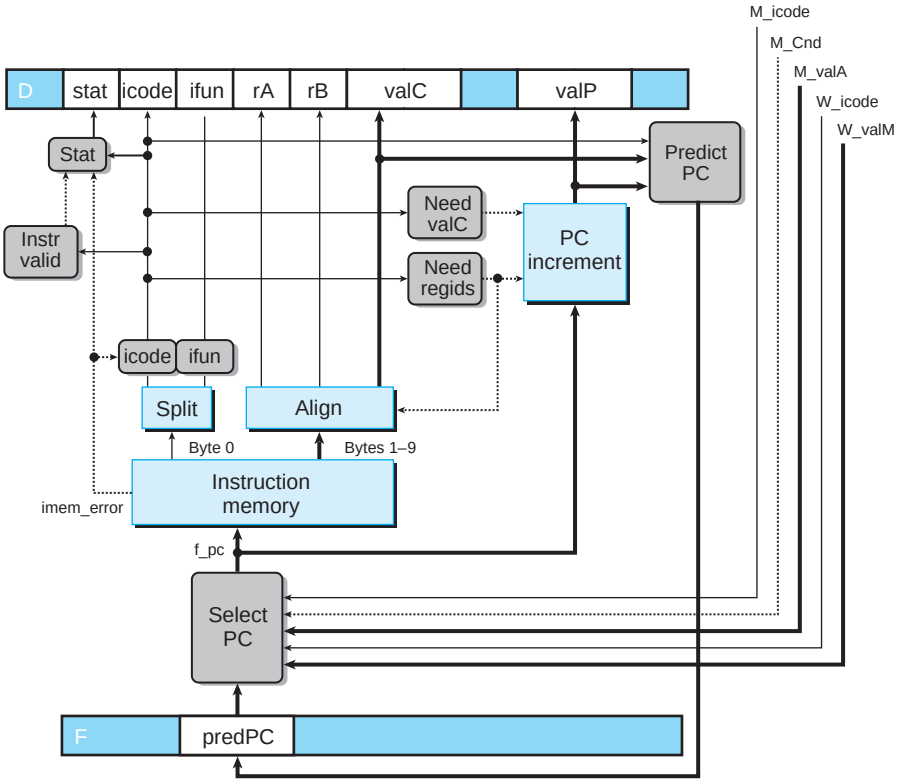


Figure 4.57 PIPE PC selection and fetch logic. Wit i t o l ti li it, t r o o r a o l r\ itt a\r o t t i tr ti o .

memory and for extracting the different instruction fields are the same as those we considered for SEQ (see the fetch stage in Section 4.3.4).

The PC selection logic chooses between three program counter sources. As a mispredicted branch enters the memory stage, the value of alP for this instruction (indicating the address of the following instruction) is read from pipeline register M (signal M_ alA). When a `ret` instruction enters the write-back stage, the return address is read from pipeline register W (signal W_ alM). All other cases use the predicted value of the PC, stored in pipeline register F (signal F_ r\ PC):

```
word f_pc = [
        # Mispredicted branch.  Fetch at incremented PC
        M_icode == IJXX && !M_Cnd : M_valA;
        # Completion of RET instruction
        W_icode == IRET : W_valM;
        # Default: Use predicted value of PC
        1 : F_predPC;
];
```

The PC prediction logic chooses valC for the fetched instruction when it is either a call or a jump, and valP otherwise:

```
word f_predPC = [
        f_icode in { IJXX, ICALL } : f_valC;
        1 : f_valP;
];
```

The logic blocks labeled "Instr valid," "Need regids," and "Need valC" are the same as for SEQ, with appropriately named source signals.

Unlike in SEQ, we must split the computation of the instruction status into two parts. In the fetch stage, we can test for a memory error due to an out-of-range instruction address, and we can detect an illegal instruction or a `halt` instruction. Detecting an invalid data address must be deferred to the memory stage.

Practice Problem 4.30 (solution page 526)

Write HCL code for the signal f_stat, providing the provisional status for the fetched instruction.

Decode and Write-Back Stage

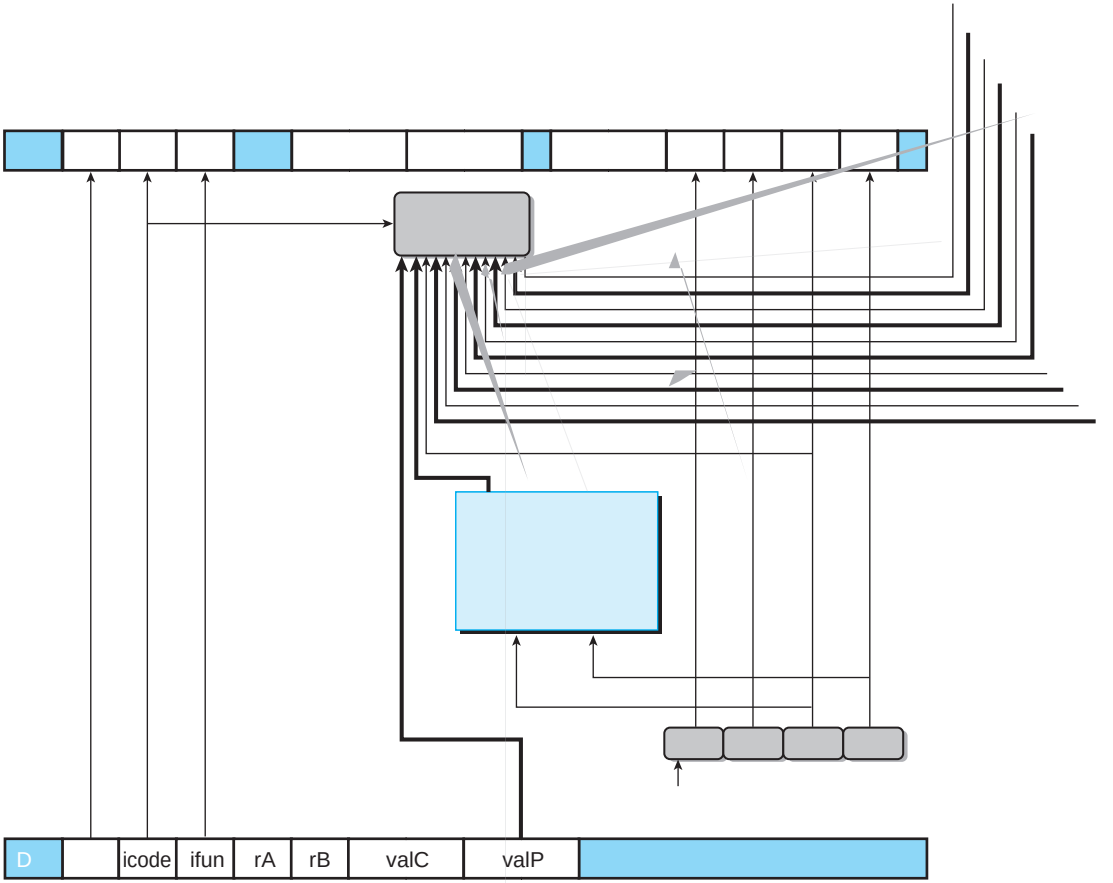
Figure 4.58 gives a detailed view of the decode and write-back logic for PIPE. The blocks labeled d_valE, d_valM, d_srcA, and d_srcB are very similar to their counterparts in the implementation of SEQ. Observe that the register IDs supplied to the write ports come from the write-back stage (signals W_dstE and W_dstM), rather than from the decode stage. This is because we want the writes to occur to the destination registers specified by the instruction in the write-back stage.

Practice Problem 4.31 (solution page 526)

The block labeled "dstE" in the decode stage generates the register ID for the E port of the register file, based on fields from the fetched instruction in pipeline register D. The resulting signal is named d_dstE in the HCL description of PIPE. Write HCL code for this signal, based on the HCL description of the SEQ signal d_dstE. (See the decode stage for SEQ in Section 4.3.4.) Do not concern yourself with the logic to implement conditional moves yet.

Most of the complexity of this stage is associated with the forwarding logic. As mentioned earlier, the block labeled "Sel+Fwd A" serves two roles. It merges the valP signal into the valA signal for later stages in order to reduce the amount of state in the pipeline register. It also implements the forwarding logic for source operand valA.

The merging of signals valA and valP exploits the fact that only the `call` and jump instructions need the value of valP in later stages, and these instructions


D
icode
ifun
rA
rB
valC
valP

Data word	Register ID	Source description
_ alE	_ tE	ALU output
_ alM	M_ tM	Memory output
M_ alE	M_ tE	Pending write to port E in memory stage
W_ alM	W_ tM	Pending write to port M in write-back stage
W_ alE	W_ tE	Pending write to port E in write-back stage

If none of the forwarding conditions hold, the block should select _r alA, the value read from register port A, as its output.

Putting all of this together, we get the following HCL description for the new value of alA for pipeline register E:

```
word d_valA = [
        D_icode in { ICALL, IJXX } : D_valP; # Use incremented PC
        d_srcA == e_dstE : e_valE;    # Forward valE from execute
        d_srcA == M_dstM : m_valM;    # Forward valM from memory
        d_srcA == M_dstE : M_valE;    # Forward valE from memory
        d_srcA == W_dstM : W_valM;    # Forward valM from write back
        d_srcA == W_dstE : W_valE;    # Forward valE from write back
        1 : d_rvalA;  # Use value read from register file
];
```

The priority given to the five forwarding sources in the above HCL code is very important. This priority is determined in the HCL code by the order in which the five destination register IDs are tested. If any order other than the one shown were chosen, the pipeline would behave incorrectly for some programs. Figure 4.59 shows an example of a program that requires a correct setting of priority among the forwarding sources in the execute and memory stages. In this program, the first two instructions write to register `%rdx`, while the third uses this register as its source operand. When the `rrmovq` instruction reaches the decode stage in cycle 4, the forwarding logic must choose between two values destined for its source register. Which one should it choose? To set the priority, we must consider the behavior of the machine-language program when it is executed one instruction at a time. The first `irmovq` instruction would set register `%rdx` to 10, the second would set the register to 3, and then the `rrmovq` instruction would read 3 from `%rdx`. To imitate this behavior, our pipelined implementation should always give priority to the forwarding source in the earliest pipeline stage, since it holds the latest instruction in the program sequence setting the register. Thus, the logic in the HCL code above first tests the forwarding source in the execute stage, then those in the memory stage, and finally the sources in the write-back stage. The forwarding priority between the two sources in either the memory or the write-back stages is only a concern for the instruction `popq %rsp`, since only this instruction can attempt two simultaneous writes to the same register.

```
# prog8
0x000: irmovq $10,%rdx
0x00a: irmovq  $3,%rdx
0x014: rrmovq %rdx,%rax
0x016: halt
```

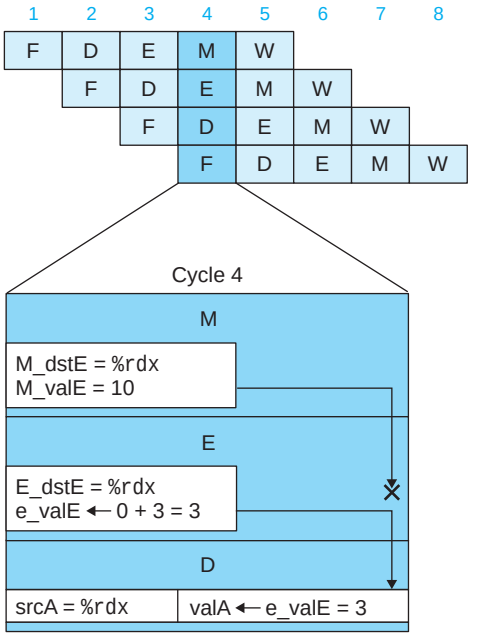


Figure 4.59 Demonstration of forwarding priority. I l 4, al r %rdx ar a ala l r t t t a r ta . T r ar i l i t i t t ta , i itr r t t tr tl rat al r t i r it r.

Practice Problem 4.32 (solution page 526)

Suppose the order of the third and fourth cases (the two forwarding sources from the memory stage) in the HCL code for _ alA were reversed. Describe the resulting behavior of the `rrmovq` instruction (line 5) for the following program:

```
1        irmovq $5, %rdx
2        irmovq $0x100,%rsp
3        rmmovq %rdx,0(%rsp)
4        popq %rsp
5        rrmovq %rsp,%rax
```

Practice Problem 4.33 (solution page 527)

Suppose the order of the fifth and sixth cases (the two forwarding sources from the write-back stage) in the HCL code for _ alA were reversed. Write a Y86-64 program that would be executed incorrectly. Describe how the error would occur and its effect on the program behavior.

Practice Problem 4.34 (solution page 527)

Write HCL code for the signal d_valB, giving the value for source operand valB supplied to pipeline register E.

One small part of the write-back stage remains. As shown in Figure 4.52, the overall processor status Stat is computed by a block based on the status value in pipeline register W. Recall from Section 4.1.1 that the code should indicate either normal operation (AOK) or one of the three exception conditions. Since pipeline register W holds the state of the most recently completed instruction, it is natural to use this value as an indication of the overall processor status. The only special case to consider is when there is a bubble in the write-back stage. This is part of normal operation, and so we want the status code to be AOK for this case as well:

```
word Stat = [
        W_stat == SBUB : SAOK;
        1 : W_stat;
];
```

E t Sta

Figure 4.60 shows the execute stage logic for PIPE. The hardware units and the logic blocks are identical to those in SEQ, with an appropriate renaming of signals. We can see the signals _ alE and d tE directed toward the decode stage as one of the forwarding sources. One difference is that the logic labeled "Set CC," which determines whether or not to update the condition codes, has signals _ tat and W_ tat as inputs. These signals are used to detect cases where an instruction

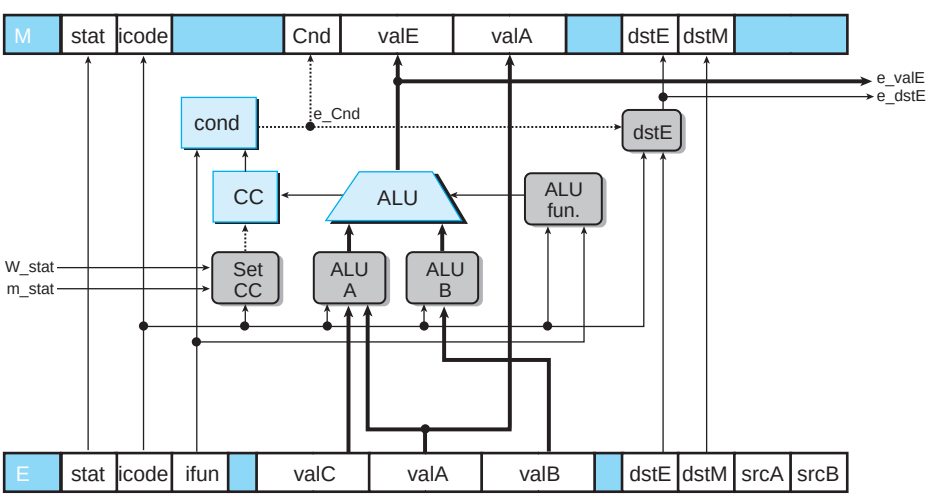
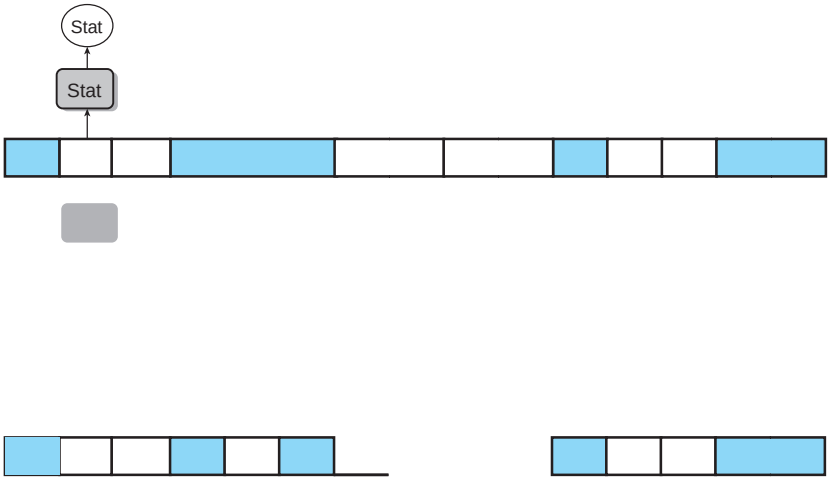


Figure 4.60 PIPE execute stage logic. T i art o t d i i r i iar t o t l o i i t SEQ i l tati o .


Stat
Stat

Practice Problem 4.36 (solution page 528)

In this stage, we can complete the computation of the status code Stat by detecting the case of an invalid address for the data memory. Write HCL code for the signal _ tat.

4.5.8 Pipeline Control Logic

We are now ready to complete our design for PIPE by creating the pipeline control logic. This logic must handle the following four control cases for which other mechanisms, such as data forwarding and branch prediction, do not suffice:

Load/use hazards. The pipeline must stall for one cycle between an instruction that reads a value from memory and an instruction that uses this value.

Processing `ret`. The pipeline must stall until the `ret` instruction reaches the write-back stage.

Mispredicted branches. By the time the branch logic detects that a jump should not have been taken, several instructions at the branch target will have started down the pipeline. These instructions must be canceled, and fetching should begin at the instruction following the jump instruction.

Exceptions. When an instruction causes an exception, we want to disable the updating of the programmer-visible state by later instructions and halt execution once the excepting instruction reaches the write-back stage.

We will go through the desired actions for each of these cases and then develop control logic to handle all of them.

D ir Ha li S al C tr l Ca

For a load/use hazard, we have described the desired pipeline operation in Section 4.5.5, as illustrated by the example of Figure 4.54. Only the `mrmovq` and `popq` instructions read data from memory. When (1) either of these is in the execute stage and (2) an instruction requiring the destination register is in the decode stage, we want to hold back the second instruction in the decode stage and inject a bubble into the execute stage on the next cycle. After this, the forwarding logic will resolve the data hazard. The pipeline can hold back an instruction in the decode stage by keeping pipeline register D in a fixed state. In doing so, it should also keep pipeline register F in a fixed state, so that the next instruction will be fetched a second time. In summary, implementing this pipeline flow requires detecting the hazard condition, keeping pipeline registers F and D fixed, and injecting a bubble into the execute stage.

For the processing of a `ret` instruction, we have described the desired pipeline operation in Section 4.5.5. The pipeline should stall for three cycles until the return address is read as the `ret` instruction passes through the memory stage.

This was illustrated by a simplified pipeline diagram in Figure 4.55 for processing the following program:

```
0x000:    irmovq stack,%rsp  #   Initialize stack pointer
0x00a:    call proc          #   Procedure call
0x013:    irmovq $10,%rdx    #   Return point
0x01d:    halt
0x020: .pos 0x20
0x020: proc:                 # proc:
0x020:    ret                #   Return immediately
0x021:    rrmovq %rdx,%rbx   #   Not executed
0x030: .pos 0x30
0x030: stack:                # stack: Stack pointer
```

Figure 4.62 provides a detailed view of the processing of the `ret` instruction for the example program. The key observation here is that there is no way to inject a bubble into the fetch stage of our pipeline. On every cycle, the fetch stage reads some instruction from the instruction memory. Looking at the HCL code for implementing the PC prediction logic in Section 4.5.7, we can see that for the `ret` instruction, the new value of the PC is predicted to be valP, the address of the following instruction. In our example program, this would be `0x021`, the address of the `rrmovq` instruction following the `ret`. This prediction is not correct for this example, nor would it be for most cases, but we are not attempting to predict return addresses correctly in our design. For three clock cycles, the fetch stage stalls, causing the `rrmovq` instruction to be fetched but then replaced by a bubble in the decode stage. This process is illustrated in Figure 4.62 by the three fetches, with an arrow leading down to the bubbles passing through the remaining pipeline stages. Finally, the `irmovq` instruction is fetched on cycle 7. Comparing Figure 4.62 with

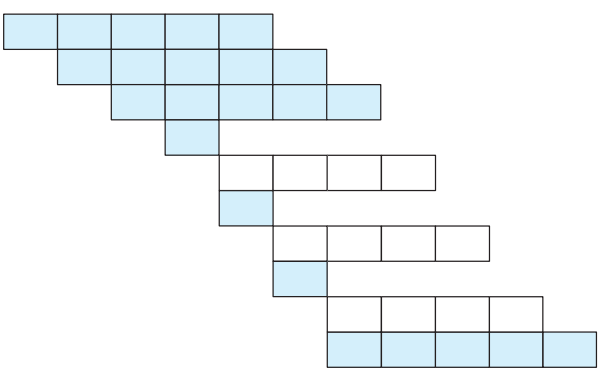


Figure 4.55, we see that our implementation achieves the desired effect, but with a slightly peculiar fetching of an incorrect instruction for three consecutive cycles.

When a mispredicted branch occurs, we have described the desired pipeline operation in Section 4.5.5 and illustrated it in Figure 4.56. The misprediction will be detected as the jump instruction reaches the execute stage. The control logic then injects bubbles into the decode and execute stages on the next cycle, causing the two incorrectly fetched instructions to be canceled. On the same cycle, the pipeline reads the correct instruction into the fetch stage.

For an instruction that causes an exception, we must make the pipelined implementation match the desired ISA behavior, with all prior instructions completing and with none of the following instructions having any effect on the program state. Achieving these effects is complicated by the facts that (1) exceptions are detected during two different stages (fetch and memory) of program execution, and (2) the program state is updated in three different stages (execute, memory, and write-back).

Our stage designs include a status code stat in each pipeline register to track the status of each instruction as it passes through the pipeline stages. When an exception occurs, we record that information as part of the instruction's status and continue fetching, decoding, and executing instructions as if nothing were amiss. As the excepting instruction reaches the memory stage, we take steps to prevent later instructions from modifying the programmer-visible state by (1) disabling the setting of condition codes by instructions in the execute stage, (2) injecting bubbles into the memory stage to disable any writing to the data memory, and (3) stalling the write-back stage when it has an excepting instruction, thus bringing the pipeline to a halt.

The pipeline diagram in Figure 4.63 illustrates how our pipeline control handles the situation where an instruction causing an exception is followed by one that
reache change the condition codes. On cycle 6, the
of

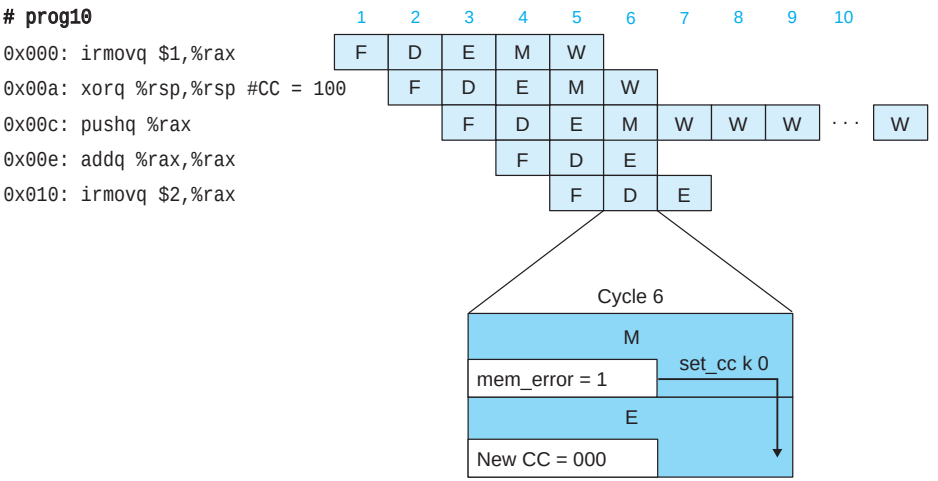


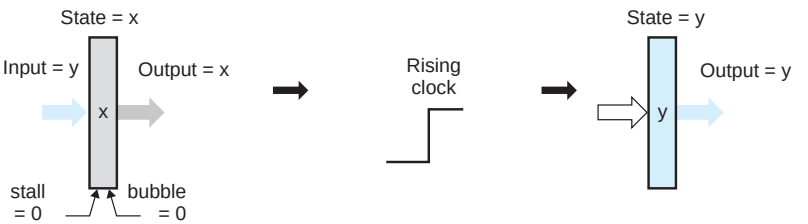
Figure 4.63 Processing invalid memory reference exception. O l 6, t i al
r r r t pushq i tr ti a t ati o t ti
t ia l . T i li tart i ti l it t r ta a talli
t ti i tr ti i t rit - a ta .

Condition	Trigger
Processing `ret`	IRET {D

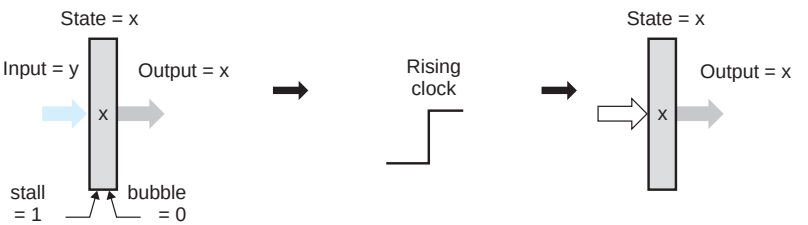
instruction is in the execute stage, so that it can set up the conditions required to recover from the misprediction as the instruction enters the memory stage. When a jump instruction is in the execute stage, the signal _C ♈ indicates whether or not the jump should be taken. We detect an excepting instruction by examining the instruction status values in the memory and write-back stages. For the memory stage, we use the signal _ tat, computed within the stage, rather than M_ tat from the pipeline register. This internal signal incorporates the possibility of a data memory address error.

Pi li C₀ tr₀l M a i

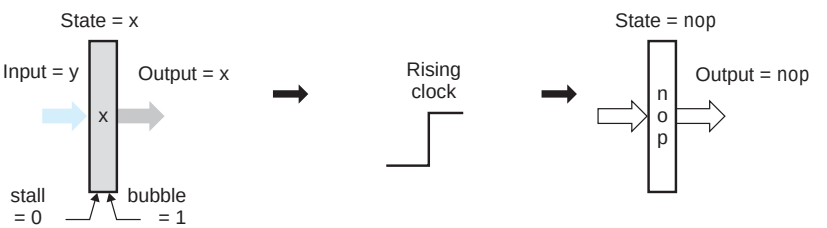
Figure 4.65 shows low-level mechanisms that allow the pipeline control logic to hold back an instruction in a pipeline register or to inject a bubble into the pipeline. These mechanisms involve small extensions to the basic clocked register described



(a) Normal



(b) Stall



(c) Bubble

Figure 4.65 Additional pipeline register operations. (a) U ♈ r or al o ♈ ti o , t tat a ♈ o t t o t r it rar tt o t al att t it t l o ri . ( ) W rat ♈ i stall o ♈ , t tat i ♈ fi ♈ at it r i al . ( ) W o rat ♈ i bubble o ♈ , t tat i o r ritt it t at o a nop o rati o .

Condition	Pipeline register				
	F	D	E	M	W
Processing `ret`	stall	bubble	normal	normal	normal
Load/use hazard	stall	stall	bubble	normal	normal
Mispredicted branch	normal	bubble	bubble	normal	normal

Figure 4.66 Actions for pipeline control logic. T i r t o i r alt ri t i li o t r tall i t i li o r a li art all t i tr ti o .

in Section 4.2.5. Suppose that each pipeline register has two control inputs tall and l . The settings of these signals determine how the pipeline register is updated as the clock rises. Under normal operation (Figure 4.65(a)), both of these inputs are set to 0, causing the register to load its input as its new state. When the tall signal is set to 1 (Figure 4.65(b)), the updating of the state is disabled. Instead, the register will remain in its previous state. This makes it possible to hold back an instruction in some pipeline stage. When the l signal is set to 1 (Figure 4.65(c)), the state of the register will be set to some fixed reset configuration, giving a state equivalent to that of a `nop` instruction. The particular pattern of ones and zeros for a pipeline register's reset configuration depends on the set of fields in the pipeline register. For example, to inject a bubble into pipeline register D, we want the i o field to be set to the constant value `INOP` (Figure 4.26). To inject a bubble into pipeline register E, we want the i o field to be set to `INOP` and the tE, tM, r A, and r B fields to be set to the constant `RNONE`. Determining the reset configuration is one of the tasks for the hardware designer in designing a pipeline register. We will not concern ourselves with the details here. We will consider it an error to set both the bubble and the stall signals to 1.

The table in Figure 4.66 shows the actions the different pipeline stages should take for each of the three special conditions. Each involves some combination of normal, stall, and bubble operations for the pipeline registers. In terms of timing, the stall and bubble control signals for the pipeline registers are generated by blocks of combinational logic. These values must be valid as the clock rises, causing each of the pipeline registers to either load, stall, or bubble as the next clock cycle begins. With this small extension to the pipeline register designs, we can implement a complete pipeline, including all of its control, using the basic building blocks of combinational logic, clocked registers, and random access memories.

C o i at i o o C o tr o l C o i t i o

In our discussion of the special pipeline control conditions so far, we assumed that at most one special case could arise during any single clock cycle. A common bug in designing a system is to fail to handle instances where multiple special conditions arise simultaneously. Let us analyze such possibilities. We need not worry about combinations involving program exceptions, since we have carefully designed our exception-handling mechanism to consider other instructions in the pipeline. Figure 4.67 diagrams the pipeline states that cause the other three special control

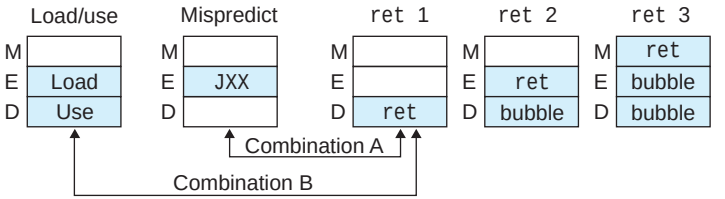


Figure 4.67
Pipeline states for special control conditions. T t a ar i i at a ari i lta l .

conditions. These diagrams show blocks for the decode, execute, and memory stages. The shaded boxes represent particular constraints that must be satisfied for the condition to arise. A load/use hazard requires that the instruction in the execute stage reads a value from memory into a register, and that the instruction in the decode stage has this register as a source operand. A mispredicted branch requires the instruction in the execute stage to have a jump instruction. There are three possible cases for `ret`—the instruction can be in either the decode, execute, or memory stage. As the `ret` instruction moves through the pipeline, the earlier pipeline stages will have bubbles.

We can see by these diagrams that most of the control conditions are mutually exclusive. For example, it is not possible to have a load/use hazard and a mispredicted branch simultaneously, since one requires a load instruction (`mrmovq` or `popq`) in the execute stage, while the other requires a jump. Similarly, the second and third `ret` combinations cannot occur at the same time as a load/use hazard or a mispredicted branch. Only the two combinations indicated by arrows can arise simultaneously.

Combination A involves a not-taken jump instruction in the execute stage and a `ret` instruction in the decode stage. Setting up this combination requires the `ret` to be at the target of a not-taken branch. The pipeline control logic should detect that the branch was mispredicted and therefore cancel the `ret` instruction.

Practice Problem 4.37 (solution page 528)

Write a Y86-64 assembly-language program that causes combination A to arise and determines whether the control logic handles it correctly.

Combining the control actions for the combination A conditions (Figure 4.66), we get the following pipeline control actions (assuming that either a bubble or a stall overrides the normal case):

	Pipeline register				
Condition	F	D	E	M	W

That is, it would be handled like a mispredicted branch, but with a stall in the fetch stage. Fortunately, on the next cycle, the PC selection logic will choose the address of the instruction following the jump, rather than the predicted program counter, and so it does not matter what happens with the pipeline register F. We conclude that the pipeline will correctly handle this combination.

Combination B involves a load/use hazard, where the loading instruction sets register `%rsp` and the `ret` instruction then uses this register as a source operand, since it must pop the return address from the stack. The pipeline control logic should hold back the `ret` instruction in the decode stage.

Practice Problem 4.38 (solution page 528)

Write a Y86-64 assembly-language program that causes combination B to arise and completes with a `halt` instruction if the pipeline operates correctly.

Combining the control actions for the combination B conditions (Figure 4.66), we get the following pipeline control actions:

	Pipeline register				
Condition	F	D	E	M	W
Processing `ret`	stall	bubble	normal	normal	normal
Load/use hazard	stall	stall	bubble	normal	normal
Combination	stall	bubble+stall	bubble	normal	normal
Desired	stall	stall	bubble	normal	normal

If both sets of actions were triggered, the control logic would try to stall the `ret` instruction to avoid the load/use hazard but also inject a bubble into the decode stage due to the `ret` instruction. Clearly, we do not want the pipeline to perform both sets of actions. Instead, we want it to just take the actions for the load/use hazard. The actions for processing the `ret` instruction should be delayed for one cycle.

This analysis shows that combination B requires special handling. In fact, our original implementation of the PIPE control logic did not handle this combination correctly. Even though the design had passed many simulation tests, it had a subtle bug that was uncovered only by the analysis we have just shown. When a program having combination B was executed, the control logic would set both the bubble and the stall signals for pipeline register D to 1. This example shows the importance of systematic analysis. It would be unlikely to uncover this bug by just running normal programs. If left undetected, the pipeline would not faithfully implement the ISA behavior.

C0 tr0 l L0 i l l tati0

Figure 4.68 shows the overall structure of the pipeline control logic. Based on signals from the pipeline registers and pipeline stages, the control logic generates

Aside Testing pipeline hardware

As we have seen, there are many ways to introduce bugs into a design, even for a simple microprocessor. With pipelining, there are many subtle interactions between the instructions at different pipeline stages. We have seen that many of the design challenges involve unusual instructions (such as popping to the stack pointer) or unusual instruction combinations (such as a not-taken jump followed by a `ret`). We also see that exception handling adds an entirely new dimension to the possible pipeline behaviors. How, then, can we be sure that our design is correct? For hardware manufacturers, this is a dominant concern, since they cannot simply report an error and have users download code patches over the Internet. Even a simple logic design error can have serious consequences, especially as microprocessors are increasingly used to operate systems that are critical to our lives and health, such as automotive antilock braking systems, heart pacemakers, and aircraft control systems.

Simply simulating a design while running a number of "typical" programs is not a sufficient means of testing a system. Instead, thorough testing requires devising ways of systematically generating many tests that will exercise as many different instructions and instruction combinations as possible. In creating our Y86-64 processor designs, we also devised a number of testing scripts, each of which generates many different tests, runs simulations of the processor, and compares the resulting register and memory values to those produced by our YIS instruction set simulator. Here is a brief description of the scripts:

optest. Runs 49 tests of different Y86-64 instructions with different source and destination registers

jtest. Runs 64 tests of the different jump and call instructions, with different combinations of whether or not the branches are taken

cmtest. Runs 28 tests of the different conditional move instructions, with different control combinations

htest. Runs 600 tests of different data hazard possibilities, with different combinations of source and destination instructions, and with different numbers of `nop` instructions between the instruction pairs

ctest. Tests 22 different control combinations, based on an analysis similar to what we did in Section 4.5.8

etest. Tests 12 different combinations where an instruction causes an exception and the instructions following it could alter the programmer-visible state

The key idea of this testing method is that we want to be as systematic as possible, generating tests that create the different conditions that are likely to cause pipeline errors.

processor on some benchmark program and observe the operation of the execute stage. On each cycle, the execute stage either (1) processes an instruction and this instruction continues through the remaining stages to completion, or (2) processes a bubble injected due to one of the three special cases. If the stage processes a total of C_i instructions and C_b bubbles, then the processor has required around $C_i + C_b$ total clock cycles to execute C_i instructions. We say "around" because we ignore

Aside For all ri i or i

Even when a design passes an extensive set of tests, we cannot be certain that it will operate correctly for all possible programs. The number of possible programs we could test is unimaginably large, even if we only consider tests consisting of short code segments. Newer methods of formal verification, however, hold the promise that we can have tools that rigorously consider all possible behaviors of a system and determine whether or not there are any design errors.

We were able to apply formal verification to an earlier version of our Y86-64 processors [13]. We set up a framework to compare the behavior of the pipelined design PIPE to the unpipelined version SEQ. That is, it was able to prove that for an arbitrary machine-language program, the two processors would have identical effects on the programmer-visible state. Of course, our verifier cannot actually run all possible programs, since there are an infinite number of them. Instead, it uses a form of proof by induction, showing a consistency between the two processors on a cycle-by-cycle basis. Carrying out this analysis requires reasoning about the hardware using symbolic methods in which we consider all program values to be arbitrary integers, and we abstract the ALU as a sort of "black box," computing some unspecified function over its arguments. We assume only that the ALUs for SEQ and PIPE compute identical functions.

We used the HCL descriptions of the control logic to generate the control logic for our symbolic processor models, and so we could catch any bugs in the HCL code. Being able to show that SEQ and PIPE are identical does not guarantee that either of them faithfully implements the instruction set architecture. However, it would uncover any bug due to an incorrect pipeline design, and this is the major source of design errors.

In our experiments, we verified not only a version of PIPE similar to the one we have presented in this chapter but also several variants that we give as homework problems, in which we add more instructions, modify the hardware capabilities, or use different branch prediction strategies. Interestingly, we found only one bug in all of our designs, involving control combination B (described in Section 4.5.8) for our solution to the variant described in Problem 4.58. This exposed a weakness in our testing regime that caused us to add additional cases to the ctest testing script.

Formal verification is still in an early stage of development. The tools are often difficult to use, and they do not have the capacity to verify large-scale designs. We were able to verify our processors in part because of their relative simplicity. Even then, it required several weeks of effort and multiple runs of the tools, each requiring up to 8 hours of computer time. This is an active area of research, with some tools becoming commercially available and some in use at companies such as Intel, AMD, and IBM.

the cycles required to start the instructions flowing through the pipeline. We can then compute the CPI for this benchmark as follows:

$$\text{CPI} = \frac{C_i + C_b}{C_i} = 1.0 + \frac{C_b}{C_i}$$

That is, the CPI equals 1.0 plus a penalty term C_b / C_i indicating the average number of bubbles injected per instruction executed. Since only three different instruction types can cause a bubble to be injected, we can break this penalty term into three components:

Web Aside ARCH:VLOG Verilog implementations of Y86-64 processor

As we have mentioned, modern logic design involves writing textual representations of hardware designs in a hardware description language. The design can then be tested by both simulation and a variety of formal verification tools. Once we have confidence in the design, we can use logic synthesis tools to translate the design into actual logic circuits.

We have developed models of our Y86-64 processor designs in the Verilog hardware description language. These designs combine modules implementing the basic building blocks of the processor, along with control logic generated directly from the HCL descriptions. We have been able to synthesize some of these designs, download the logic circuit descriptions onto field-programmable gate array (FPGA) hardware, and run the processors on actual programs.

$$\text{CPI} = 1.0 + lp + mp + rp$$

where lp (for "load penalty") is the average frequency with which bubbles are injected while stalling for load/use hazards, mp

The sum of the three penalties is 0.27, giving a CPI of 1.27.

Our goal was to design a pipeline that can issue one instruction per cycle, giving a CPI of 1.0. We did not quite meet this goal, but the overall performance is still quite good. We can also see that any effort to reduce the CPI further should focus on mispredicted branches. They account for 0.16 of our total penalty of 0.27, because conditional branches are common, our prediction strategy often fails, and we cancel two instructions for every misprediction.

Practice Problem 4.43 (solution page 530)

Suppose we use a branch prediction strategy that achieves a success rate of 65%, such as backward taken, forward not taken (BTFNT), as described in Section 4.5.4. What would be the impact on CPI, assuming all of the other frequencies are not affected?

Practice Problem 4.44 (solution page 530)

Let us analyze the relative performance of using conditional data transfers versus conditional control transfers for the programs you wrote for Problems 4.5 and 4.6. Assume that we are using these programs to compute the sum of the absolute values of a very long array, and so the overall performance is determined largely by the number of cycles required by the inner loop. Assume that our jump instructions are predicted as being taken, and that around 50% of the array values are positive.

A. On average, how many instructions are executed in the inner loops of the two programs?

B. On average, how many bubbles would be injected into the inner loops of the two programs?

C. What is the average number of clock cycles required per array element for the two programs?

4.5.10 Unfinished Business

We have created a structure for the PIPE pipelined microprocessor, designed the control logic blocks, and implemented pipeline control logic to handle special cases where normal pipeline flow does not suffice. Still, PIPE lacks several key features that would be required in an actual microprocessor design. We highlight a few of these and discuss what would be required to add them.

Multiple Instructions

All of the instructions in the Y86-64 instruction set involve simple operations such as adding numbers. These can be processed in a single clock cycle within the execute stage. In a more complete instruction set, we would also need to implement instructions requiring more complex operations such as integer multiplication and

division and floating-point operations. In a medium-performance processor such as PIPE, typical execution times for these operations range from 3 or 4 cycles for floating-point addition up to 64 cycles for integer division. To implement these instructions, we require both additional hardware to perform the computations and a mechanism to coordinate the processing of these instructions with the rest of the pipeline.

One simple approach to implementing multicycle instructions is to simply expand the capabilities of the execute stage logic with integer and floating-point arithmetic units. An instruction remains in the execute stage for as many clock cycles as it requires, causing the fetch and decode stages to stall. This approach is simple to implement, but the resulting performance is not very good.

Better performance can be achieved by handling the more complex operations with special hardware functional units that operate independently of the main pipeline. Typically, there is one functional unit for performing integer multiplication and division, and another for performing floating-point operations. As an instruction enters the decode stage, it can be issued to the special unit. While the unit performs the operation, the pipeline continues processing other instructions. Typically, the floating-point unit is itself pipelined, and thus multiple operations can execute concurrently in the main pipeline and in the different units.

The operations of the different units must be synchronized to avoid incorrect behavior. For example, if there are data dependencies between the different operations being handled by different units, the control logic may need to stall one part of the system until the results from an operation handled by some other part of the system have been completed. Often, different forms of forwarding are used to convey results from one part of the system to other parts, just as we saw between the different stages of PIPE. The overall design becomes more complex than we have seen with PIPE, but the same techniques of stalling, forwarding, and pipeline control can be used to make the overall behavior match the sequential ISA model.

Interfacing to Memory

In our presentation of PIPE, we assumed that both the instruction fetch unit and the data memory could read or write any memory location in one clock cycle. We also ignored the possible hazards caused by self-modifying code where one instruction writes to the region of memory from which later instructions are fetched. Furthermore, we reference memory locations according to their virtual addresses, and these require a translation into physical addresses before the actual read or write operation can be performed. Clearly, it is unrealistic to do all of this processing in a single clock cycle. Even worse, the memory values being accessed may reside on disk, requiring millions of clock cycles to read into the processor memory.

As will be discussed in Chapters 6 and 9, the memory system of a processor uses a combination of multiple hardware memories and operating system software to manage the virtual memory system. The memory system is organized as a hierarchy, with faster but smaller memories holding a subset of the memory being

backed up by slower and larger memories. At the level closest to the processor, the cache memories provide fast access to the most heavily referenced memory locations. A typical processor has two first-level caches—one for reading instructions and one for reading and writing data. Another type of cache memory, known as a translation look-aside buffer, or TLB, provides a fast translation from virtual to physical addresses. Using a combination of TLBs and caches, it is indeed possible to read instructions and read or write data in a single clock cycle most of the time. Thus, our simplified view of memory referencing by our processors is actually quite reasonable.

Although the caches hold the most heavily referenced memory locations, there will be times when a cache miss occurs, where some reference is made to a location that is not held in the cache. In the best case, the missing data can be retrieved from a higher-level cache or from the main memory of the processor, requiring 3 to 20 clock cycles. Meanwhile, the pipeline simply stalls, holding the instruction in the fetch or memory stage until the cache can perform the read or write operation. In terms of our pipeline design, this can be implemented by adding more stall conditions to the pipeline control logic. A cache miss and the consequent synchronization with the pipeline is handled completely by hardware, keeping the time required down to a small number of clock cycles.

In some cases, the memory location being referenced is actually stored in the disk or nonvolatile memory. When this occurs, the hardware signals a page fault exception. Like other exceptions, this will cause the processor to invoke the operating system's exception handler code. This code will then set up a transfer from the disk to the main memory. Once this completes, the operating system will return to the original program, where the instruction causing the page fault will be re-executed. This time, the memory reference will succeed, although it might cause a cache miss. Having the hardware invoke an operating system routine, which then returns control back to the hardware, allows the hardware and system software to cooperate in the handling of page faults. Since accessing a disk can require millions of clock cycles, the several thousand cycles of processing performed by the OS page fault handler has little impact on performance.

From the perspective of the processor, the combination of stalling to handle short-duration cache misses and exception handling to handle long-duration page faults takes care of any unpredictability in memory access times due to the structure of the memory hierarchy.

4.6 Summary

We have seen that the instruction set architecture, or ISA, provides a layer of abstraction between the behavior of a processor—in terms of the set of instructions and their encodings—and how the processor is implemented. The ISA provides a very sequential view of program execution, with one instruction executed to completion before the next one begins.

Aside Stat -o -t -art i r o r o or\o i

A five-stage pipeline, such as we have shown with the PIPE processor, represented the state of the art in processor design in the mid-1980s. The prototype RISC processor developed by Patterson's research group at Berkeley formed the basis for the first SPARC processor, developed by Sun Microsystems in 1987. The processor developed by Hennessy's research group at Stanford was commercialized by MIPS Technologies (a company founded by Hennessy) in 1986. Both of these used five-stage pipelines. The Intel i486 processor also uses a five-stage pipeline, although with a different partitioning of responsibilities among the stages, with two decode stages and a combined execute/memory stage [27].

These pipelined designs are limited to a throughput of at most one instruction per clock cycle. The CPI (for "cycles per instruction") measure described in Section 4.5.9 can never be less than 1.0. The different stages can only process one instruction at a time. More recent processors support superscalar operation, meaning that they can achieve a CPI less than 1.0 by fetching, decoding, and executing multiple instructions in parallel. As superscalar processors have become widespread, the accepted performance measure has shifted from CPI to its reciprocal—the average number of instructions executed per cycle, or IPC. It can exceed 1.0 for superscalar processors. The most advanced designs use a technique known as out-of-order execution to execute multiple instructions in parallel, possibly in a totally different order than they occur in the program, while preserving the overall behavior implied by the sequential ISA model. This form of execution is described in Chapter 5 as part of our discussion of program optimization.

Pipelined processors are not just historical artifacts, however. The majority of processors sold are used in embedded systems, controlling automotive functions, consumer products, and other devices where the processor is not directly visible to the system user. In these applications, the simplicity of a pipelined processor, such as the one we have explored in this chapter, reduces its cost and power requirements compared to higher-performance models.

More recently, as multicore processors have gained a following, some have argued that we could get more overall computing power by integrating many simple processors on a single chip rather than a smaller number of more complex ones. This strategy is sometimes referred to as "many-core" processors [10].

We defined the Y86-64 instruction set by starting with the x86-64 instructions and simplifying the data types, address modes, and instruction encoding considerably. The resulting ISA has attributes of both RISC and CISC instruction sets. We then organized the processing required for the different instructions into a series of five stages, where the operations at each stage vary according to the instruction being executed. From this, we constructed the SEQ processor, in which an entire instruction is executed every clock cycle by having it flow through all five stages.

Pipelining improves the throughput performance of a system by letting the different stages operate concurrently. At any given time, multiple operations are being processed by the different stages. In introducing this concurrency, we must be careful to provide the same program-level behavior as would a sequential execution of the program. We introduced pipelining by reordering parts of SEQ to get SEQ+ and then adding pipeline registers to create the PIPE− pipeline.

Web Aside ARCH:HCL HCL Descriptions for Y86-64 Processors

In this chapter, we have looked at portions of the HCL code for several simple logic designs and for the control logic for Y86-64 processors SEQ and PIPE. For reference, we provide documentation of the HCL language and complete HCL descriptions for the control logic of the two processors. Each of these descriptions requires only five to seven pages of HCL code, and it is worthwhile to study them in their entirety.

We enhanced the pipeline performance by adding forwarding logic to speed the sending of a result from one instruction to another. Several special cases require additional pipeline control logic to stall or cancel some of the pipeline stages.

Our design included rudimentary mechanisms to handle exceptions, where we make sure that only instructions up to the excepting instruction affect the programmer-visible state. Implementing a complete handling of exceptions would be significantly more challenging. Properly handling exceptions gets even more complex in systems that employ greater degrees of pipelining and parallelism.

In this chapter, we have learned several important lessons about processor design:

- **Managing complexity is a top priority.** We want to make optimum use of the hardware resources to get maximum performance at minimum cost. We did this by creating a very simple and uniform framework for processing all of the different instruction types. With this framework, we could share the hardware units among the logic for processing the different instruction types.
- **We do not need to implement the ISA directly.** A direct implementation of the ISA would imply a very sequential design. To achieve higher performance, we want to exploit the ability in hardware to perform many operations simultaneously. This led to the use of a pipelined design. By careful design and analysis, we can handle the various pipeline hazards, so that the overall effect of running a program exactly matches what would be obtained with the ISA model.
- **Hardware designers must be meticulous.** Once a chip has been fabricated, it is nearly impossible to correct any errors. It is very important to get the design right on the first try. This means carefully analyzing different instruction types and combinations, even ones that do not seem to make sense, such as popping to the stack pointer. Designs must be thoroughly tested with systematic simulation test programs. In developing the control logic for PIPE, our design had a subtle bug that was uncovered only after a careful and systematic analysis of control combinations.

4.6.1 Y86-64 Simulators

The lab materials for this chapter include simulators for the SEQ and PIPE processors. Each simulator has two versions:

- The GUI (graphic user interface) version displays the memory, program code, and processor state in graphic windows. This provides a way to readily see how the instructions flow through the processors. The control panel also allows you to reset, single-step, or run the simulator interactively.
- The text version runs the same simulator, but it only displays information by printing to the terminal. This version is not as useful for debugging, but it allows automated testing of the processor.

The control logic for the simulators is generated by translating the HCL declarations of the logic blocks into C code. This code is then compiled and linked with the rest of the simulation code. This combination makes it possible for you to test out variants of the original designs using the simulators. Testing scripts are also available that thoroughly exercise the different instructions and the different hazard possibilities.

Bibliographic Notes

For those interested in learning more about logic design, the Katz and Borriello logic design textbook [58] is a standard introductory text, emphasizing the use of hardware description languages. Hennessy and Patterson's computer architecture textbook [46] provides extensive coverage of processor design, including both simple pipelines, such as the one we have presented here, and advanced processors that execute more instructions in parallel. Shriver and Smith [101] give a very thorough presentation of an Intel-compatible x86-64 processor manufactured by AMD.

Homework Problems

4.45

In Section 3.4.2, the x86-64 `pushq` instruction was described as decrementing the stack pointer and then storing the register at the stack pointer location. So, if we had an instruction of the form `pushq REG`, for some register `REG`, it would be equivalent to the code sequence

```
subq $8,%rsp            Decrement stack pointer
movq REG, (%rsp)        Store REG on stack
```

A. In light of analysis done in Practice Problem 4.7, does this code sequence correctly describe the behavior of the instruction `pushq %rsp`? Explain.

B. How could you rewrite the code sequence so that it correctly describes both the cases where `REG` is `%rsp` as well as any other register?

4.46

In Section 3.4.2, the x86-64 `popq` instruction was described as copying the result from the top of the stack to the destination register and then incrementing the stack pointer. So, if we had an instruction of the form `popq REG`, it would be equivalent to the code sequence

```
movq (%rsp), REG        Read REG from stack
addq $8,%rsp            Increment stack pointer
```

A. In light of analysis done in Practice Problem 4.8, does this code sequence correctly describe the behavior of the instruction `popq %rsp`? Explain.

B. How could you rewrite the code sequence so that it correctly describes both the cases where `REG` is `%rsp` as well as any other register?

4.47

Your assignment will be to write a Y86-64 program to perform bubblesort. For reference, the following C function implements bubblesort using array referencing:

```
1  /* Bubble sort: Array version */
2  void bubble_a(long *data, long count) {
3      long i, last;
4      for (last = count-1; last > 0; last--) {
5          for (i = 0; i < last; i++)
6              if (data[i+1] < data[i]) {
7                  /* Swap adjacent elements */
8                  long t = data[i+1];
9                  data[i+1] = data[i];
10                 data[i] = t;
11             }
12     }
13 }
```

A. Write and test a C version that references the array elements with pointers, rather than using array indexing.

B. Write and test a Y86-64 program consisting of the function and test code. You may find it useful to pattern your implementation after x86-64 code generated by compiling your C code. Although pointer comparisons are normally done using unsigned arithmetic, you can use signed arithmetic for this exercise.

4.48

Modify the code you wrote for Problem 4.47 to implement the test and swap in the bubblesort function (lines 6–11) using no jumps and at most three conditional moves.

4.49

Modify the code you wrote for Problem 4.47 to implement the test and swap in the bubblesort function (lines 6–11) using no jumps and just one conditional move.

4.50

In Section 3.6.8, we saw that a common way to implement `switch` statements is to create a set of code blocks and then index those blocks using a jump table. Consider

```
#include <stdio.h>
/* Example use of switch statement */

long switchv(long idx) {
    long result = 0;
    switch(idx) {
    case 0:
        result = 0xaaa;
        break;
    case 2:
    case 5:
        result = 0xbbb;
        break;
    case 3:
        result = 0xccc;
        break;
    default:
        result = 0xddd;

    }
    return result;
 }

/* Testing Code */
#define CNT 8
#define MINVAL -1

int main() {
    long vals[CNT];
    long i;
    for (i = 0; i < CNT; i++) {
        vals[i] = switchv(i + MINVAL);
        printf("idx = %ld, val = 0x%lx\n", i + MINVAL, vals[i]);
    }
    return 0;
}
```

Figure 4.69 Switch statements can be translated into Y86-64 code. T i r ir i l tati o o a ta l .

the C code shown in Figure 4.69 for a function `switchv`, along with associated test code.

Implement `switchv` in Y86-64 using a jump table. Although the Y86-64 instruction set does not include an indirect jump instruction, you can get the same effect by pushing a computed address onto the stack and then executing the `ret`

instruction. Implement test code similar to what is shown in C to demonstrate that your implementation of `switchv` will handle both the cases handled explicitly as well as those that trigger the `default` case.

4.51

Practice Problem 4.3 introduced the `iaddq` instruction to add immediate data to a register. Describe the computations performed to implement this instruction. Use the computations for `irmovq` and `OPq` (Figure 4.18) as a guide.

4.52

The file `seq-full.hcl` contains the HCL description for SEQ, along with the declaration of a constant `IIADDQ` having hexadecimal value `C`, the instruction code for `iaddq`. Modify the HCL descriptions of the control logic blocks to implement the `iaddq` instruction, as described in Practice Problem 4.3 and Problem 4.51. See the lab material for directions on how to generate a simulator for your solution and how to test it.

4.53

Suppose we wanted to create a lower-cost pipelined processor based on the structure we devised for PIPE− (Figure 4.41), without any bypassing. This design would handle all data dependencies by stalling until the instruction generating a needed value has passed through the write-back stage.

The file `pipe-stall.hcl` contains a modified version of the HCL code for PIPE in which the bypassing logic has been disabled. That is, the signals `_ alA` and `_ alB` are simply declared as follows:

```
##  DO NOT MODIFY THE FOLLOWING CODE.
## No forwarding.  valA is either valP or value from register file
word d_valA = [
        D_icode in { ICALL, IJXX } : D_valP; # Use incremented PC
        1 : d_rvalA;  # Use value read from register file
];

## No forwarding.  valB is value from register file
word d_valB = d_rvalB;
```

Modify the pipeline control logic at the end of this file so that it correctly handles all possible control and data hazards. As part of your design effort, you should analyze the different combinations of control cases, as we did in the design of the pipeline control logic for PIPE. You will find that many different combinations can occur, since many more conditions require the pipeline to stall. Make sure your control logic handles each combination correctly. See the lab material for directions on how to generate a simulator for your solution and how to test it.

4.54

The file `pipe-full.hcl` contains a copy of the PIPE HCL description, along with a declaration of the constant value `IIADDQ`. Modify this file to implement the `iaddq` instruction, as described in Practice Problem 4.3 and Problem 4.51. See the lab

material for directions on how to generate a simulator for your solution and how to test it.

4.55

The file `pipe-nt.hcl` contains a copy of the HCL code for PIPE, plus a declaration of the constant `J_YES` with value `0`, the function code for an unconditional jump instruction. Modify the branch prediction logic so that it predicts conditional jumps as being not taken while continuing to predict unconditional jumps and `call` as being taken. You will need to devise a way to get valC, the jump target address, to pipeline register M to recover from mispredicted branches. See the lab material for directions on how to generate a simulator for your solution and how to test it.

4.56

The file `pipe-btfnt.hcl` contains a copy of the HCL code for PIPE, plus a declaration of the constant `J_YES` with value `0`, the function code for an unconditional jump instruction. Modify the branch prediction logic so that it predicts conditional jumps as being taken when valC < valP (backward branch) and as being not taken when valC ≥ valP (forward branch). (Since Y86-64 does not support unsigned arithmetic, you should implement this test using a signed comparison.) Continue to predict unconditional jumps and `call` as being taken. You will need to devise a way to get both valC and valP to pipeline register M to recover from mispredicted branches. See the lab material for directions on how to generate a simulator for your solution and how to test it.

4.57

In our design of PIPE, we generate a stall whenever one instruction performs a load, reading a value from memory into a register, and the next instruction has this register as a source operand. When the source gets used in the execute stage, this stalling is the only way to avoid a hazard. For cases where the second instruction stores the source operand to memory, such as with an `rmmovq` or `pushq` instruction, this stalling is not necessary. Consider the following code examples:

```
1       mrmovq 0(%rcx),%rdx   # Load  1
2       pushq  %rdx           # Store 1
3       nop
4       popq %rdx             # Load  2
5       rmmovq %rax,0(%rdx)   # Store 2
```

In lines 1 and 2, the `mrmovq` instruction reads a value from memory into `%rdx`, and the `pushq` instruction then pushes this value onto the stack. Our design for PIPE would stall the `pushq` instruction to avoid a load/use hazard. Observe, however, that the value of `%rdx` is not required by the `pushq` instruction until it reaches the memory stage. We can add an additional bypass path, as diagrammed in Figure 4.70, to forward the memory output (signal m_valM) to the valA field in pipeline register M. On the next clock cycle, this forwarded value can then be written to memory. This technique is known as load forwarding.

Note that the second example (lines 4 and 5) in the code sequence above cannot make use of load forwarding. The value loaded by the `popq` instruction is

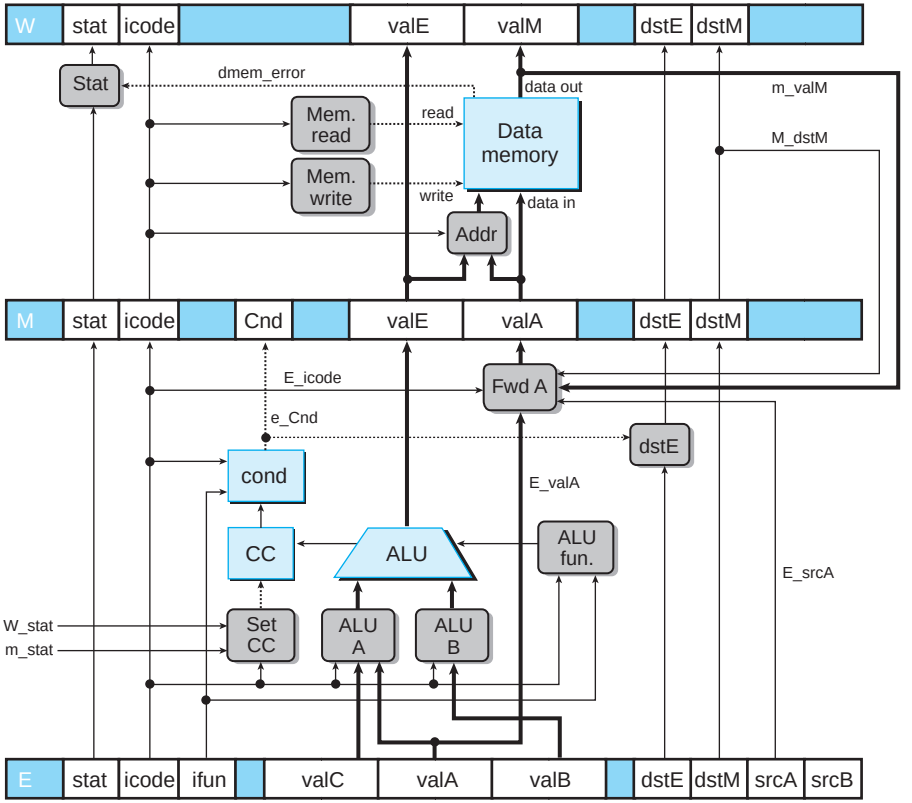


Figure 4.70 Execute and memory stages capable of load forwarding. B a i a a at r t r t tt t r alA i i li r it r M, a r a i rat r t a tall i r r r l a / a a . T i i t t Pr l 4.57.

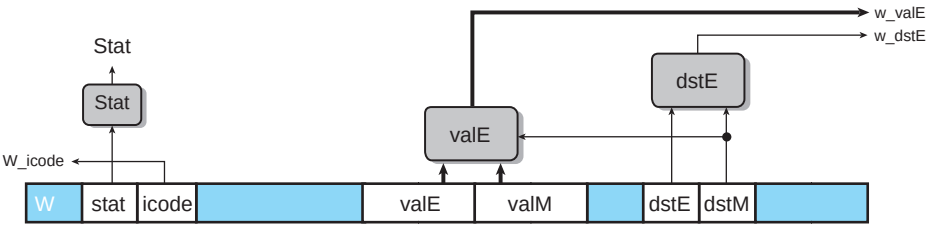
used as part of the address computation by the next instruction, and this value is required in the execute stage rather than the memory stage.

A. Write a logic formula describing the detection condition for a load/use hazard, similar to the one given in Figure 4.64, except that it will not cause a stall in cases where load forwarding can be used.

B. The file `pipe-lf.hcl` contains a modified version of the control logic for PIPE. It contains the definition of a signal _ alA to implement the block labeled "Fwd A" in Figure 4.70. It also has the conditions for a load/use hazard in the pipeline control logic set to zero, and so the pipeline control logic will not detect any forms of load/use hazards. Modify this HCL description to implement load forwarding. See the lab material for directions on how to generate a simulator for your solution and how to test it.

4.58

Our pipelined design is a bit unrealistic in that we have two write ports for the register file, but only the popq instruction requires two simultaneous writes to the register file. The other instructions could therefore use a single write port, sharing this for writing alE and alM. The following figure shows a modified version of the write-back logic, in which we merge the write-back register IDs (W_ tE and W_ tM) into a single signal _ tE and the write-back values (W_ alE and W_ alM) into a single signal _ alE:



The logic for performing the merges is written in HCL as follows:

```
## Set E port register ID
word w_dstE = [
        ## writing from valM
        W_dstM != RNONE : W_dstM;
        1: W_dstE;
];

## Set E port value
word w_valE = [
        W_dstM != RNONE : W_valM;
        1: W_valE;
];
```

The control for these multiplexors is determined by tE—when it indicates there is some register, then it selects the value for port E, and otherwise it selects the value for port M.

In the simulation model, we can then disable register port M, as shown by the following HCL code:

```
## Disable register port M
## Set M port register ID
word w_dstM = RNONE;

## Set M port value
word w_valM = 0;
```

The challenge then becomes to devise a way to handle popq. One method is to use the control logic to dynamically process the instruction popq rA so that it has the same effect as the two-instruction sequence

```
iaddq  $8, %rsp
mrmovq -8(%rsp), rA
```

(See Practice Problem 4.3 for a description of the `iaddq` instruction.) Note the ordering of the two instructions to make sure `popq %rsp` works properly. You can do this by having the logic in the decode stage treat `popq` the same as it would the `iaddq` listed above, except that it predicts the next PC to be equal to the current PC. On the next cycle, the `popq` instruction is refetched, but the instruction code is converted to a special value `IPOP2`. This is treated as a special instruction that has the same behavior as the `mrmovq` instruction listed above.

The file `pipe-1w.hcl` contains the modified write port logic described above. It contains a declaration of the constant `IPOP2` having hexadecimal value `E`. It also contains the definition of a signal `_i` that generates the `i` field for pipeline register D. This definition can be modified to insert the instruction code `IPOP2` the second time the `popq` instruction is fetched. The HCL file also contains a declaration of the signal `_`, the value of the program counter generated in the fetch stage by the block labeled "Select PC" (Figure 4.57).

Modify the control logic in this file to process `popq` instructions in the manner we have described. See the lab material for directions on how to generate a simulator for your solution and how to test it.

4.59

Compare the performance of the three versions of bubblesort (Problems 4.47, 4.48, and 4.49). Explain why one version performs better than the other.

Solutions to Practice Problems

Solution to Problem 4.1 (page 396)

Encoding instructions by hand is rather tedious, but it will solidify your understanding of the idea that assembly code gets turned into byte sequences by the assembler. In the following output from our Y86-64 assembler, each line shows an address and a byte sequence that starts at that address:

```
1   0x100:                      | .pos 0x100  # Start code at address
0x100
2   0x100: 30f30f00000000000000 |     irmovq $15,%rbx
3   0x10a: 2031                 |     rrmovq %rbx,%rcx
4   0x10c:                      | loop:
5   0x10c: 4013fdffffffffffffff |     rmmovq %rcx,-3(%rbx)
6   0x116: 6031                 |     addq   %rbx,%rcx
7   0x118: 700c01000000000000   |     jmp loop
```

Several features of this encoding are worth noting:

- Decimal 15 (line 2) has hex representation `0x000000000000000f`. Writing the bytes in reverse order gives `0f 00 00 00 00 00 00 00`.

- Decimal −3 (line 5) has hex representation `0xfffffffffffffffd`. Writing the bytes in reverse order gives `fd ff ff ff ff ff ff ff`.
- The code starts at address `0x100`. The first instruction requires 10 bytes, while the second requires 2. Thus, the loop target will be `0x0000010c`. Writing these bytes in reverse order gives `0c 01 00 00 00 00 00 00`.

Solution to Problem 4.2 (page 396)

Decoding a byte sequence by hand helps you understand the task faced by a processor. It must read byte sequences and determine what instructions are to be executed. In the following, we show the assembly code used to generate each of the byte sequences. To the left of the assembly code, you can see the address and byte sequence for each instruction.

A. Some operations with immediate data and address displacements:

```
0x100: 30f3fcffffffffffff |     irmovq $-4,%rbx
0x10a: 4063000800000000000 |     rmmovq %rsi,0x800(%rbx)
0x114: 00                  |     halt
```

B. Code including a function call:

```
0x200: a06f                |     pushq %rsi
0x202: 800c02000000000000  |     call proc
0x20b: 00                  |     halt
0x20c:                     | proc:
0x20c: 30f30a00000000000000 |     irmovq $10,%rbx
0x216: 90                  |     ret
```

C. Code containing illegal instruction specifier byte `0xf0`:

```
0x300: 5054070000000000000 |     mrmovq 7(%rsp),%rbp
0x30a: 10                  |     nop
0x30b: f0                  | .byte 0xf0  # Invalid instruction code
0x30c: b01f                |     popq %rcx
```

D. Code containing a jump operation:

```
0x400:                     | loop:
0x400: 6113                |     subq %rcx, %rbx
0x402: 730004000000000000  |     je loop
0x40b: 00                  |     halt
```

E. Code containing an invalid second byte in a `pushq` instruction:

```
0x500: 6362                |     xorq %rsi,%rdx
0x502: a0                  |     .byte 0xa0  # pushq instruction
code
0x503: f0                  |     .byte 0xf0  # Invalid register
specifier byte
```

Solution to Problem 4.3 (page 405)

Using the `iaddq` instruction, we can rewrite the `sum` function as

```
# long sum(long *start, long count)
# start in %rdi, count in %rsi
sum:
        xorq %rax,%rax          # sum = 0
        andq %rsi,%rsi          # Set condition codes
        jmp    test
loop:
        mrmovq (%rdi),%r10      # Get *start
        addq %r10,%rax          # Add to sum
        iaddq $8,%rdi           # start++
        iaddq $-1,%rsi          # count--
test:
        jne    loop             # Stop when 0
        ret
```

Solution to Problem 4.4 (page 406)

Gcc, running on an x86-64 machine, produces the following code for `rproduct`:

```
  long rproduct(long *start, long count)
  start in %rdi, count in %rsi
rproduct:
  movl    $1, %eax
  testq   %rsi, %rsi
  jle     .L9
  pushq   %rbx
  movq    (%rdi), %rbx
  subq    $1, %rsi
  addq    $8, %rdi
  call    rproduct
  imulq   %rbx, %rax
  popq    %rbx
.L9:
  rep; ret
```

This can easily be adapted to produce Y86-64 code:

```
# long rproduct(long *start, long count)
# start in %rdi, count in %rsi
rproduct:
        xorq   %rax,%rax        # Set return value to 1
        andq   %rsi,%rsi        # Set condition codes
        je     return           # If count <= 0, return 1
        pushq  %rbx             # Save callee-saved register
```

```
        mrmovq (%rdi),%rbx      # Get *start
        irmovq $-1,%r10
        addq   %r10,%rsi        # count--
        irmovq $8,%r10
        addq   %r10,%rdi        # start++
        call   rproduct
        imulq  %rbx,%rax        # Multiply *start to product
        popq   %rbx             # Restore callee-saved register
return:
        ret
```

Solution to Problem 4.5 (page 406)

This problem gives you a chance to try your hand at writing assembly code.

```
1   # long absSum(long *start, long count)
2   # start in %rdi, count in %rsi
3   absSum:
4           irmovq $8,%r8           # Constant 8
5           irmovq $1,%r9           # Constant 1
6           xorq %rax,%rax          # sum = 0
7           andq %rsi,%rsi          # Set condition codes
8           jmp  test
9   loop:
10          mrmovq (%rdi),%r10      # x = *start
11          xorq %r11,%r11          # Constant 0
12          subq %r10,%r11          # -x
13          jle pos                 # Skip if -x <= 0
14          rrmovq %r11,%r10        # x = -x
15  pos:
16          addq %r10,%rax          # Add to sum
17          addq %r8,%rdi           # start++
18          subq %r9,%rsi           # count--
19  test:
20          jne    loop             # Stop when 0
21          ret
```

Solution to Problem 4.6 (page 406)

This problem gives you a chance to try your hand at writing assembly code with conditional moves. We show only the code for the loop. The rest is the same as for Problem 4.5:

```
9   loop:
10          mrmovq (%rdi),%r10      # x = *start
11          xorq %r11,%r11          # Constant 0
12          subq %r10,%r11          # -x
13          cmovg %r11,%r10         # If -x > 0 then x = -x
```

```
14          addq %r10,%rax          # Add to sum
15          addq %r8,%rdi           # start++
16          subq %r9,%rsi           # count--
17  test:
18          jne    loop             # Stop when 0
```

Solution to Problem 4.7 (page 406)

Although it is hard to imagine any practical use for this particular instruction, it is important when designing a system to avoid any ambiguities in the specification. We want to determine a reasonable convention for the instruction's behavior and to make sure each of our implementations adheres to this convention.

The `subq` instruction in this test compares the starting value of `%rsp` to the value pushed onto the stack. The fact that the result of this subtraction is zero implies that the old value of `%rsp` gets pushed.

Solution to Problem 4.8 (page 407)

It is even more difficult to imagine why anyone would want to pop to the stack pointer. Still, we should decide on a convention and stick with it. This code sequence pushes `0xabcd` onto the stack, pops to `%rsp`, and returns the popped value. Since the result equals `0xabcd`, we can deduce that `popq %rsp` sets the stack pointer to the value read from memory. It is therefore equivalent to the instruction `mrmovq (%rsp),%rsp`.

Solution to Problem 4.9 (page 410)

The EXCLUSIVE-OR function requires that the 2 bits have opposite values:

```
bool xor = (!a && b) || (a && !b);
```

In general, the signals `eq` and `xor` will be complements of each other. That is, one will equal 1 whenever the other is 0.

Solution to Problem 4.10 (page 413)

The outputs of the EXCLUSIVE-OR circuits will be the complements of the bit equality values. Using DeMorgan's laws (Web Aside DATA:BOOL on page 88), we can implement AND using OR and NOT, yielding the circuit shown in Figure 4.71.

Solution to Problem 4.11 (page 415)

We can see that the second part of the case expression can be written as

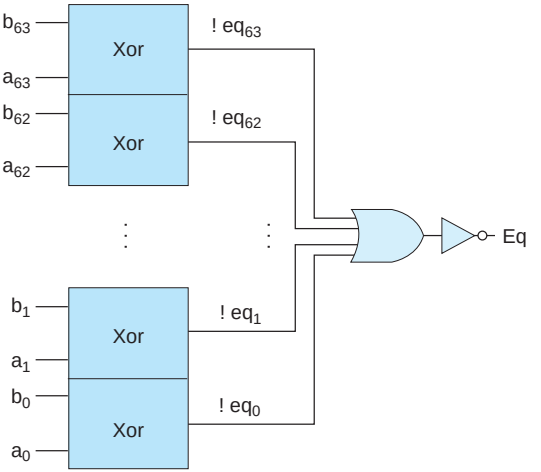
```
        B <= C          : B;
```

Since the first line will detect the case where `A` is the minimum element, the second line need only determine whether `B` or `C` is minimum.

Solution to Problem 4.12 (page 416)

This design is a variant of the one to find the minimum of the three inputs:

Figure 4.71
Solution for Problem 4.10.




```
word Med3 = [
        A <= B && B <= C : B;
        C <= B && B <= A : B;
        B <= A && A <= C : A;
        C <= A && A <= B : A;
        1                : C;
];
```

Solution to Problem 4.13 (page 423)

These exercises help make the stage computations more concrete. We can see from the object code that this instruction is located at address `0x016`. It consists of 10 bytes, with the first two being `0x30` and `0xf4`. The last 8 bytes are a byte-reversed version of `0x0000000000000080` (decimal 128).

Stage	Generic `irmovq V, rB`	Specific `irmovq $128, %rsp`
Fetch	i$_0^7$: i ← M_1[PC] rA : rB ← M_1[PC + 1] alC ← M_8[PC + 2] alP ← PC + 10	i$_0^7$: i ← M_1[0x016] = 3:0 rA : rB ← M_1[0x017] = f:4 alC ← M_8[0x018] = 128 alP ← 0x016 + 10 = 0x020
Decode		
Execute	alE ← 0 + alC	alE ← 0 + 128 = 128
Memory		
Write back	R[rB] ← alE	R[%rsp] ← alE = 128
PC update	PC ← alP	PC ← alP = 0x020

This instruction sets register `%rsp` to 128 and increments the PC by 10.

Solution to Problem 4.14 (page 426)

We can see that the instruction is located at address `0x02c` and consists of 2 bytes with values `0xb0` and `0x00f`. Register `%rsp` was set to 120 by the `pushq` instruction (line 6), which also stored 9 at this memory location.

Stage	Generic `popq rA`	Specific `popq %rax`
Fetch	i$_0^7$: i ← M_1[PC] rA : rB ← M_1[PC + 1] valP ← PC + 2	i$_0^7$: i ← M_1[0x02c] = b:0 rA : rB ← M_1[0x02d] = 0:f valP ← 0x02c + 2 = 0x02e
Decode	valA ← R[%rsp] valB ← R[%rsp]	valA ← R[%rsp] = 120 valB ← R[%rsp] = 120
Execute	valE ← valB + 8	valE ← 120 + 8 = 128
Memory	valM ← M_8[valA]	valM ← M_8[120] = 9
Write back	R[%rsp] ← valE R[rA] ← valM	R[%rsp] ← 128 R[%rax] ← 9
PC update	PC ← valP	PC ← 0x02e

The instruction sets `%rax` to 9, sets `%rsp` to 128, and increments the PC by 2.

Solution to Problem 4.15 (page 427)

Tracing the steps listed in Figure 4.20 with rA equal to `%rsp`, we can see that in the memory stage the instruction will store valA, the original value of the stack pointer, to memory, just as we found for x86-64.

Solution to Problem 4.16 (page 428)

Tracing the steps listed in Figure 4.20 with rA equal to `%rsp`, we can see that both of the write-back operations will update `%rsp`. Since the one writing valM would occur last, the net effect of the instruction will be to write the value read from memory to `%rsp`, just as we saw for x86-64.

Solution to Problem 4.17 (page 429)

Implementing conditional moves requires only minor changes from register-to-register moves. We simply condition the write-back step on the outcome of the conditional test:

Stage	`cmovXX rA, rB`
Fetch	i$_0^7$: i ← M_1[PC] rA : rB ← M_1[PC + 1] valP ← PC + 2
Decode	valA ← R[rA]

Stage	cmovXX rA, rB
Execute	aluE ← 0 + aluA Cond ← C_0(CC, i)
Memory	
Write back	if (Cond) R[rB] ← aluE
PC update	PC ← aluP

Solution to Problem 4.18 (page 430)

We can see that this instruction is located at address 0x037 and is 9 bytes long. The first byte has value 0x80, while the last 8 bytes are a byte-reversed version of 0x0000000000000041, the call target. The stack pointer was set to 128 by the popq instruction (line 7).

Stage	Generic call D t	Specific call 0x041
Fetch	ifun : icode ← M_1[PC]	ifun : icode ← M_1[0x037] = 8:0
	aluC ← M_8[PC + 1]	aluC ← M_8[0x038] = 0x041
	aluP ← PC + 9	aluP ← 0x037 + 9 = 0x040
Decode		
	aluB ← R[%rsp]	aluB ← R[%rsp] = 128
Execute	aluE ← aluB + −8	aluE ← 128 + −8 = 120
Memory	M_8[aluE] ← aluP	M_8[120] ← 0x040
Write back	R[%rsp] ← aluE	R[%rsp] ← 120
PC update	PC ← aluC	PC ← 0x041

The effect of this instruction is to set %rsp to 120, to store 0x040 (the return address) at this memory address, and to set the PC to 0x041 (the call target).

Solution to Problem 4.19 (page 442)

All of the HCL code in this and other practice problems is straightforward, but trying to generate it yourself will help you think about the different instructions and how they are processed. For this problem, we can simply look at the set of Y86-64 instructions (Figure 4.2) and determine which have a constant field.

```
bool need_valC =
        icode in { IIRMOVQ, IRMMOVQ, IMRMOVQ, IJXX, ICALL };
```

Solution to Problem 4.20 (page 443)

This code is similar to the code for `r A`.

```
word srcB = [
        icode in { IOPQ, IRMMOVQ, IMRMOVQ  } : rB;
        icode in { IPUSHQ, IPOPQ, ICALL, IRET } : RRSP;
        1 : RNONE;  # Don't need register
];
```

Solution to Problem 4.21 (page 444)

This code is similar to the code for `tE`.

```
word dstM = [
        icode in { IMRMOVQ, IPOPQ } : rA;
        1 : RNONE;  # Don't write any register
];
```

Solution to Problem 4.22 (page 444)

As we found in Practice Problem 4.16, we want the write via the M port to take priority over the write via the E port in order to store the value read from memory into `%rsp`.

Solution to Problem 4.23 (page 445)

This code is similar to the code for `al A`.

```
word aluB = [
        icode in { IRMMOVQ, IMRMOVQ, IOPQ, ICALL,
                      IPUSHQ, IRET, IPOPQ } : valB;
        icode in { IRRMOVQ, IIRMOVQ } : 0;
        # Other instructions don't need ALU
];
```

Solution to Problem 4.24 (page 445)

Implementing conditional moves is surprisingly simple: we disable writing to the register file by setting the destination register to `RNONE` when the condition does not hold.

```
word dstE = [
        icode in { IRRMOVQ } && Cnd : rB;
        icode in { IIRMOVQ, IOPQ} : rB;
        icode in { IPUSHQ, IPOPQ, ICALL, IRET } : RRSP;
        1 : RNONE;  # Don't write any register
];
```

Solution to Problem 4.25 (page 446)

This code is similar to the code for `_a r`.

```
word mem_data = [
        # Value from register
        icode in { IRMMOVQ, IPUSHQ } : valA;
        # Return PC
        icode == ICALL : valP;
        # Default: Don't write anything
];
```

Solution to Problem 4.26 (page 446)

This code is similar to the code for _r á.

```
bool mem_write = icode in { IRMMOVQ, IPUSHQ, ICALL };
```

Solution to Problem 4.27 (page 447)

Computing the Stat field requires collecting status information from several stages:

```
## Determine instruction status
word Stat = [
        imem_error || dmem_error : SADR;
        !instr_valid: SINS;
        icode == IHALT : SHLT;
        1 : SAOK;
];
```

Solution to Problem 4.28 (page 453)

This problem is an interesting exercise in trying to find the optimal balance among a set of partitions. It provides a number of opportunities to compute throughputs and latencies in pipelines.

A. For a two-stage pipeline, the best partition would be to have blocks A, B, and C in the first stage and D, E, and F in the second. The first stage has a delay of 170 ps, giving a total cycle time of 170 + 20 = 190 ps. We therefore have a throughput of 5.26 GIPS and a latency of 380 ps.

B. For a three-stage pipeline, we should have blocks A and B in the first stage, blocks C and D in the second, and blocks E and F in the third. The first two stages have a delay of 110 ps, giving a total cycle time of 130 ps and a throughput of 7.69 GIPS. The latency is 390 ps.

C. For a four-stage pipeline, we should have block A in the first stage, blocks B and C in the second, block D in the third, and blocks E and F in the fourth. The second stage requires 90 ps, giving a total cycle time of 110 ps and a throughput of 9.09 GIPS. The latency is 440 ps.

D. The optimal design would be a five-stage pipeline, with each block in its own stage, except that the fifth stage has blocks E and F. The cycle time is 80 + 20 = 100 ps, for a throughput of around 10.00 GIPS and a latency of

500 ps. Adding more stages would not help, since we cannot run the pipeline any faster than one cycle every 100 ps.

Solution to Problem 4.29 (page 454)

Each stage would have combinational logic requiring 300/k ps and a pipeline register requiring 20 ps.

A. The total latency would be 300 + 20k ps, while the throughput (in GIPS) would be

$$\frac{1{,}000}{\frac{300}{k} + 20} = \frac{1{,}000k}{300 + 20k}$$

B. As we let k go to infinity, the throughput becomes 1,000/20 = 50 GIPS. Of course, the latency would approach infinity as well.

This exercise quantifies the diminishing returns of deep pipelining. As we try to subdivide the logic into many stages, the latency of the pipeline registers becomes a limiting factor.

Solution to Problem 4.30 (page 485)

This code is very similar to the corresponding code for SEQ, except that we cannot yet determine whether the data memory will generate an error signal for this instruction.

```
# Determine status code for fetched instruction
word f_stat = [
        imem_error: SADR;
        !instr_valid : SINS;
        f_icode == IHALT : SHLT;
        1 : SAOK;
];
```

Solution to Problem 4.31 (page 485)

This code simply involves prefixing the signal names in the code for SEQ with d_ and D_.

```
word d_dstE = [
        D_icode in { IRRMOVQ, IIRMOVQ, IOPQ} : D_rB;
        D_icode in { IPUSHQ, IPOPQ, ICALL, IRET } : RRSP;
        1 : RNONE;  # Don't write any register
];
```

Solution to Problem 4.32 (page 488)

The rrmovq instruction (line 5) would stall for one cycle due to a load/use hazard caused by the popq instruction (line 4). As it enters the decode stage, the popq instruction would be in the memory stage, giving both M_valE and M_valM equal to %rsp. If the two cases were reversed, then the write back from M_valE would take priority, causing the incremented stack pointer to be passed as the argument

to the rrmovq instruction. This would not be consistent with the convention for handling popq %rsp determined in Practice Problem 4.8.

Solution to Problem 4.33 (page 488)

This problem lets you experience one of the important tasks in processor design—devising test programs for a new processor. In general, we should have test programs that will exercise all of the different hazard possibilities and will generate incorrect results if some dependency is not handled properly.

For this example, we can use a slightly modified version of the program shown in Practice Problem 4.32:

```
1       irmovq $5, %rdx
2       irmovq $0x100,%rsp
3       rmmovq %rdx,0(%rsp)
4       popq %rsp
5       nop
6       nop
7       rrmovq %rsp,%rax
```

The two nop instructions will cause the popq instruction to be in the write-back stage when the rrmovq instruction is in the decode stage. If the two forwarding sources in the write-back stage are given the wrong priority, then register %rax will be set to the incremented program counter rather than the value read from memory.

Solution to Problem 4.34 (page 489)

This logic only needs to check the five forwarding sources:

```
word d_valB = [
        d_srcB == e_dstE : e_valE;    # Forward valE from execute
        d_srcB == M_dstM : m_valM;    # Forward valM from memory
        d_srcB == M_dstE : M_valE;    # Forward valE from memory
        d_srcB == W_dstM : W_valM;    # Forward valM from write back
        d_srcB == W_dstE : W_valE;    # Forward valE from write back
        1 : d_rvalB;  # Use value read from register file
];
```

Solution to Problem 4.35 (page 490)

This change would not handle the case where a conditional move fails to satisfy the condition, and therefore sets the tE value to RNONE. The resulting value could get forwarded to the next instruction, even though the conditional transfer does not occur.

```
1           irmovq $0x123,%rax
2           irmovq $0x321,%rdx
3           xorq %rcx,%rcx          # CC = 100
4           cmovne  %rax,%rdx       # Not transferred
5           addq %rdx,%rdx          # Should be 0x642
6           halt
```

This code initializes register %rdx to 0x321. The conditional data transfer does not take place, and so the final addq instruction should double the value in %rdx to 0x642. With the altered design, however, the conditional move source value 0x321 gets forwarded into ALU input aluA, while input aluB correctly gets operand value 0x123. These inputs get added to produce result 0x444.

Solution to Problem 4.36 (page 491)

This code completes the computation of the status code for this instruction.

```
## Update the status
word m_stat = [
        dmem_error : SADR;
        1 : M_stat;
];
```

Solution to Problem 4.37 (page 497)

The following test program is designed to set up control combination A (Figure 4.67) and detect whether something goes wrong:

```
1   # Code to generate a combination of not-taken branch and ret
2           irmovq Stack, %rsp
3           irmovq rtnp,%rax
4           pushq %rax       # Set up return pointer
5           xorq %rax,%rax   # Set Z condition code
6           jne target       # Not taken (First part of combination)
7           irmovq $1,%rax   # Should execute this
8           halt
9   target: ret              # Second part of combination
10          irmovq $2,%rbx   # Should not execute this
11          halt
12  rtnp:   irmovq $3,%rdx   # Should not execute this
13          halt
14  .pos 0x40
15  Stack:
```

This program is designed so that if something goes wrong (for example, if the ret instruction is actually executed), then the program will execute one of the extra irmovq instructions and then halt. Thus, an error in the pipeline would cause some register to be updated incorrectly. This code illustrates the care required to implement a test program. It must set up a potential error condition and then detect whether or not an error occurs.

Solution to Problem 4.38 (page 498)

The following test program is designed to set up control combination B (Figure 4.67). The simulator will detect a case where the bubble and stall control signals for a pipeline register are both set to zero, and so our test program need only set up the combination for it to be detected. The biggest challenge is to make the program do something sensible when handled correctly.

```
1   # Test instruction that modifies %esp followed by ret
2           irmovq mem,%rbx
3           mrmovq  0(%rbx),%rsp # Sets %rsp to point to return point
4           ret                  # Returns to return point
5           halt                 #
6   rtnpt:  irmovq $5,%rsi       # Return point
7           halt
8   .pos 0x40
9   mem:    .quad stack          # Holds desired stack pointer
10  .pos 0x50
11  stack:  .quad rtnpt          # Top of stack: Holds return point
```

This program uses two initialized words in memory. The first word (`mem`) holds the address of the second (`stack`—the desired stack pointer). The second word holds the address of the desired return point for the `ret` instruction. The program loads the stack pointer into `%rsp` and executes the `ret` instruction.

Solution to Problem 4.39 (page 499)

From Figure 4.66, we can see that pipeline register D must be stalled for a load/use hazard:

```
bool D_stall =
        # Conditions for a load/use hazard
        E_icode in { IMRMOVQ, IPOPQ } &&
         E_dstM in { d_srcA, d_srcB };
```

Solution to Problem 4.40 (page 500)

From Figure 4.66, we can see that pipeline register E must be set to bubble for a load/use hazard or for a mispredicted branch:

```
bool E_bubble =
        # Mispredicted branch
        (E_icode == IJXX && !e_Cnd) ||
        # Conditions for a load/use hazard
        E_icode in { IMRMOVQ, IPOPQ } &&
         E_dstM in { d_srcA, d_srcB};
```

Solution to Problem 4.41 (page 500)

This control requires examining the code of the executing instruction and checking for exceptions further down the pipeline.

```
## Should the condition codes be updated?
bool set_cc = E_icode == IOPQ &&
       # State changes only during normal operation
       !m_stat in { SADR, SINS, SHLT } && !W_stat in { SADR, SINS, SHLT };
```

Solution to Problem 4.42 (page 500)

Injecting a bubble into the memory stage on the next cycle involves checking for an exception in either the memory or the write-back stage during the current cycle.

```
# Start injecting bubbles as soon as exception passes through memory stage
bool M_bubble = m_stat in { SADR, SINS, SHLT } || W_stat in { SADR, SINS, SHLT };
```

For stalling the write-back stage, we check only the status of the instruction in this stage. If we also stalled when an excepting instruction was in the memory stage, then this instruction would not be able to enter the write-back stage.

```
bool W_stall = W_stat in { SADR, SINS, SHLT };
```

Solution to Problem 4.43 (page 504)

We would then have a misprediction frequency of 0.35, giving mp = 0.20 × 0.35 × 2 = 0.14, giving an overall CPI of 1.25. This seems like a fairly marginal gain, but it would be worthwhile if the cost of implementing the new branch prediction strategy were not too high.

Solution to Problem 4.44 (page 504)

This simplified analysis, where we focus on the inner loop, is a useful way to estimate program performance. As long as the array is sufficiently large, the time spent in other parts of the code will be negligible.

A. The inner loop of the code using the conditional jump has 11 instructions, all of which are executed when the array element is zero or negative, and 10 of which are executed when the array element is positive. The average is 10.5. The inner loop of the code using the conditional move has 10 instructions, all of which are executed every time.

B. The loop-closing jump will be predicted correctly, except when the loop terminates. For a very long array, this one misprediction will have a negligible effect on the performance. The only other source of bubbles for the jump-based code is the conditional jump, depending on whether or not the array element is positive. This will cause two bubbles, but it only occurs 50% of the time, so the average is 1.0. There are no bubbles in the conditional move code.

C. Our conditional jump code requires an average of 10.5 + 1.0 = 11.5 cycles per array element (11 cycles in the best case and 12 cycles in the worst), while our conditional move code requires 10.0 cycles in all cases.

Our pipeline has a branch misprediction penalty of only two cycles—far better than those for the deep pipelines of higher-performance processors. As a result, using conditional moves does not affect program performance very much.

5

Optimizing Program Performance

5.1 Capabilities and Limitations of Optimizing Compilers 534
5.2 Expressing Program Performance 538
5.3 Program Example 540
5.4 Eliminating Loop Inefficiencies 544
5.5 Reducing Procedure Calls 548
5.6 Eliminating Unneeded Memory References 550
5.7 Understanding Modern Processors 553
5.8 Loop Unrolling 567
5.9 Enhancing Parallelism 572
5.10 Summary of Results for Optimizing Combining Code 583
5.11 Some Limiting Factors 584
5.12 Understanding Memory Performance 589
5.13 Life in the Real World: Performance Tuning Tools 597
5.14 Identifying and Eliminating Performance Bottlenecks 598
5.15 Summary 604
Bibliographic Notes 605
Homework Problems 606
Solutions to Practice Problems 609

The primary objective in writing a program must be to make it work correctly under all possible conditions. A program that runs fast but gives incorrect results serves no useful purpose. Programmers must write clear and concise code, not only so that they can make sense of it, but also so that others can read and understand the code during code reviews and when modifications are required later.

On the other hand, there are many occasions when making a program run fast is also an important consideration. If a program must process video frames or network packets in real time, then a slow-running program will not provide the needed functionality. When a computational task is so demanding that it requires days or weeks to execute, then making it run just 20% faster can have significant impact. In this chapter, we will explore how to make programs run faster via several different types of program optimization.

Writing an efficient program requires several types of activities. First, we must select an appropriate set of algorithms and data structures. Second, we must write source code that the compiler can effectively optimize to turn into efficient executable code. For this second part, it is important to understand the capabilities and limitations of optimizing compilers. Seemingly minor changes in how a program is written can make large differences in how well a compiler can optimize it. Some programming languages are more easily optimized than others. Some features of C, such as the ability to perform pointer arithmetic and casting, make it challenging for a compiler to optimize. Programmers can often write their programs in ways that make it easier for compilers to generate efficient code. A third technique for dealing with especially demanding computations is to divide a task into portions that can be computed in parallel, on some combination of multiple cores and multiple processors. We will defer this aspect of performance enhancement to Chapter 12. Even when exploiting parallelism, it is important that each parallel thread execute with maximum performance, and so the material of this chapter remains relevant in any case.

In approaching program development and optimization, we must consider how the code will be used and what critical factors affect it. In general, programmers must make a trade-off between how easy a program is to implement and maintain, and how fast it runs. At an algorithmic level, a simple insertion sort can be programmed in a matter of minutes, whereas a highly efficient sort routine may take a day or more to implement and optimize. At the coding level, many low-level optimizations tend to reduce code readability and modularity, making the programs more susceptible to bugs and more difficult to modify or extend. For code that will be executed repeatedly in a performance-critical environment, extensive optimization may be appropriate. One challenge is to maintain some degree of elegance and readability in the code despite extensive transformations.

We describe a number of techniques for improving code performance. Ideally, a compiler would be able to take whatever code we write and generate the most efficient possible machine-level program having the specified behavior. Modern compilers employ sophisticated forms of analysis and optimization, and they keep getting better. Even the best compilers, however, can be thwarted by optimization blockers—aspects of the program's behavior that depend strongly on the execu-

tion environment. Programmers must assist the compiler by writing code that can be optimized readily.

The first step in optimizing a program is to eliminate unnecessary work, making the code perform its intended task as efficiently as possible. This includes eliminating unnecessary function calls, conditional tests, and memory references. These optimizations do not depend on any specific properties of the target machine.

To maximize the performance of a program, both the programmer and the compiler require a model of the target machine, specifying how instructions are processed and the timing characteristics of the different operations. For example, the compiler must know timing information to be able to decide whether it should use a multiply instruction or some combination of shifts and adds. Modern computers use sophisticated techniques to process a machine-level program, executing many instructions in parallel and possibly in a different order than they appear in the program. Programmers must understand how these processors work to be able to tune their programs for maximum speed. We present a high-level model of such a machine based on recent designs of Intel and AMD processors. We also devise a graphical data-flow notation to visualize the execution of instructions by the processor, with which we can predict program performance.

With this understanding of processor operation, we can take a second step in program optimization, exploiting the capability of processors to provide instruction-level parallelism, executing multiple instructions simultaneously. We cover several program transformations that reduce the data dependencies between different parts of a computation, increasing the degree of parallelism with which they can be executed.

We conclude the chapter by discussing issues related to optimizing large programs. We describe the use of code profilers—tools that measure the performance of different parts of a program. This analysis can help find inefficiencies in the code and identify the parts of the program on which we should focus our optimization efforts.

In this presentation, we make code optimization look like a simple linear process of applying a series of transformations to the code in a particular order. In fact, the task is not nearly so straightforward. A fair amount of trial-and-error experimentation is required. This is especially true as we approach the later optimization stages, where seemingly small changes can cause major changes in performance and some very promising techniques prove ineffective. As we will see in the examples that follow, it can be difficult to explain exactly why a particular code sequence has a particular execution time. Performance can depend on many detailed features of the processor design for which we have relatively little documentation or understanding. This is another reason to try a number of different variations and combinations of techniques.

Studying the assembly-code representation of a program is one of the most effective means for gaining an understanding of the compiler and how the generated code will run. A good strategy is to start by looking carefully at the code for the inner loops, identifying performance-reducing attributes such as excessive memory references and poor use of registers. Starting with the assembly code, we

can also predict what operations will be performed in parallel and how well they will use the processor resources. As we will see, we can often determine the time (or at least a lower bound on the time) required to execute a loop by identifying critical paths, chains of data dependencies that form during repeated executions of a loop. We can then go back and modify the source code to try to steer the compiler toward more efficient implementations.

Most major compilers, including GCC, are continually being updated and improved, especially in terms of their optimization abilities. One useful strategy is to do only as much rewriting of a program as is required to get it to the point where the compiler can then generate efficient code. By this means, we avoid compromising the readability, modularity, and portability of the code as much as if we had to work with a compiler of only minimal capabilities. Again, it helps to iteratively modify the code and analyze its performance both through measurements and by examining the generated assembly code.

To novice programmers, it might seem strange to keep modifying the source code in an attempt to coax the compiler into generating efficient code, but this is indeed how many high-performance programs are written. Compared to the alternative of writing code in assembly language, this indirect approach has the advantage that the resulting code will still run on other machines, although perhaps not with peak performance.

5.1 Capabilities and Limitations of Optimizing Compilers

Modern compilers employ sophisticated algorithms to determine what values are computed in a program and how they are used. They can then exploit opportunities to simplify expressions, to use a single computation in several different places, and to reduce the number of times a given computation must be performed. Most compilers, including GCC, provide users with some control over which optimizations they apply. As discussed in Chapter 3, the simplest control is to specify the optimization level. For example, invoking GCC with the command-line option `-Og` specifies that it should apply a basic set of optimizations.

Invoking GCC with option `-O1` or higher (e.g., `-O2` or `-O3`) will cause it to apply more extensive optimizations. These can further improve program performance, but they may expand the program size and they may make the program more difficult to debug using standard debugging tools. For our presentation, we will mostly consider code compiled with optimization level `-O1`, even though level `-O2` has become the accepted standard for most software projects that use GCC. We purposely limit the level of optimization to demonstrate how different ways of writing a function in C can affect the efficiency of the code generated by a compiler. We will find that we can write C code that, when compiled just with option `-O1`, vastly outperforms a more naive version compiled with the highest possible optimization levels.

Compilers must be careful to apply only safe optimizations to a program, meaning that the resulting program will have the exact same behavior as would an unoptimized version for all possible cases the program may encounter, up to the limits of the guarantees provided by the C language standards. Constraining

the compiler to perform only safe optimizations eliminates possible sources of undesired run-time behavior, but it also means that the programmer must make more of an effort to write programs in a way that the compiler can then transform into efficient machine-level code. To appreciate the challenges of deciding which program transformations are safe or not, consider the following two procedures:

```
1  void twiddle1(long *xp, long *yp)
2  {
3      *xp += *yp;
4      *xp += *yp;
5  }
6
7
```

The value computed for `t1` depends on whether or not pointers `p` and `q` are aliased—if not, it will equal 3,000, but if so it will equal 1,000. This leads to one of the major optimization blockers, aspects of programs that can severely limit the opportunities for a compiler to generate optimized code. If a compiler cannot determine whether or not two pointers may be aliased, it must assume that either case is possible, limiting the set of possible optimizations.

Practice Problem 5.1 (solution page 609)

The following problem illustrates the way memory aliasing can cause unexpected program behavior. Consider the following procedure to swap two values:

```
1  /* Swap value x at xp with value y at yp */
2  void swap(long *xp, long *yp)
3  {
4      *xp = *xp + *yp; /* x+y       */
5      *yp = *xp - *yp; /* x+y-y = x */
6      *xp = *xp - *yp; /* x+y-x = y */
7  }
```

If this procedure is called with `xp` equal to `yp`, what effect will it have?

A second optimization blocker is due to function calls. As an example, consider the following two procedures:

```
1  long f();
2
3  long func1() {
4      return f() + f() + f() + f();
5  }
6
7  long func2() {
8      return 4*f();
9  }
```

It might seem at first that both compute the same result, but with `func2` calling `f` only once, whereas `func1` calls it four times. It is tempting to generate code in the style of `func2` when given `func1` as the source.

Consider, however, the following code for `f`:

```
1  long counter = 0;
2
3  long f() {
4      return counter++;
5  }
```

This function has a side effect—it modifies some part of the global program state. Changing the number of times it gets called changes the program behavior. In

Aside Optimization blocker: Inlining

Code involving function calls can be optimized by a process known as inline substitution (or simply "inlining"), where the function call is replaced by the code for the body of the function. For example, we can expand the code for `func1` by substituting four instantiations of function `f`:

```
1  /* Result of inlining f in func1 */
2  long func1in() {
3      long t = counter++; /* +0 */
4      t += counter++;     /* +1 */
5      t += counter++;     /* +2 */
6      t += counter++;     /* +3 */
7      return t;
8  }
```

This transformation both reduces the overhead of the function calls and allows further optimization of the expanded code. For example, the compiler can consolidate the updates of global variable `counter` in `func1in` to generate an optimized version of the function:

```
1  /* Optimization of inlined code */
2  long func1opt() {
3      long t = 4 * counter + 6;
4      counter += 4;
5      return t;
6  }
```

This code faithfully reproduces the behavior of `func1` for this particular definition of function `f`.

Recent versions of GCC attempt this form of optimization, either when directed to with the command-line option `-finline` or for optimization level `-O1` and higher. Unfortunately, GCC only attempts inlining for functions defined within a single file. That means it will not be applied in the common case where a set of library functions is defined in one file but invoked by functions in other files.

There are times when it is best to prevent a compiler from performing inline substitution. One is when the code will be evaluated using a symbolic debugger, such as GDB, as described in Section 3.10.2. If a function call has been optimized away via inline substitution, then any attempt to trace or set a breakpoint for that call will fail. The second is when evaluating the performance of a program by profiling, as is discussed in Section 5.14.1. Calls to functions that have been eliminated by inline substitution will not be profiled correctly.

particular, a call to `func1` would return $0 + 1 + 2 + 3 = 6$, whereas a call to `func2` would return $4 \cdot 0 = 0$, assuming both started with global variable `counter` set to zero.

Most compilers do not try to determine whether a function is free of side effects and hence is a candidate for optimizations such as those attempted in `func2`. Instead, the compiler assumes the worst case and leaves function calls intact.

Among compilers, GCC is considered adequate, but not exceptional, in terms of its optimization capabilities. It performs basic optimizations, but it does not perform the radical transformations on programs that more "aggressive" compilers do. As a consequence, programmers using GCC must put more effort into writing programs in a way that simplifies the compiler's task of generating efficient code.

5.2 Expressing Program Performance

We introduce the metric cycles per element, abbreviated CPE, to express program performance in a way that can guide us in improving the code. CPE measurements help us understand the loop performance of an iterative program at a detailed level. It is appropriate for programs that perform a repetitive computation, such as processing the pixels in an image or computing the elements in a matrix product.

The sequencing of activities by a processor is controlled by a clock providing a regular signal of some frequency, usually expressed in gigahertz (GHz), billions of cycles per second. For example, when product literature characterizes a system as a "4 GHz" processor, it means that the processor clock runs at 4.0×10^9 cycles per second. The time required for each clock cycle is given by the reciprocal of the clock frequency. These typically are expressed in nanoseconds (1 nanosecond is 10^{-9} seconds) or picoseconds (1 picosecond is 10^{-12} seconds). For example, the period of a 4 GHz clock can be expressed as either 0.25 nanoseconds or 250 picoseconds. From a programmer's perspective, it is more instructive to express measurements in clock cycles rather than nanoseconds or picoseconds. That way, the measurements express how many instructions are being executed rather than how fast the clock runs.

Many procedures contain a loop that iterates over a set of elements. For example, functions `psum1` and `psum2` in Figure 5.1 both compute the prefix sum of a vector of length n. For a vector $a = a_0, a_1, \ldots, a_{n-1}$, the prefix sum $p = p_0, p_1, \ldots, p_{n-1}$ is defined as

$$\begin{aligned} p_0 &= a_0 \\ p_i &= p_{i-1} + a_i, \quad 1 \le i < n \end{aligned} \tag{5.1}$$

Function `psum1` computes one element of the result vector per iteration. Function `psum2` uses a technique known as loop unrolling to compute two elements per iteration. We will explore the benefits of loop unrolling later in this chapter. (See Problems 5.11, 5.12, and 5.19 for more about analyzing and optimizing the prefix-sum computation.)

The time required by such a procedure can be characterized as a constant plus a factor proportional to the number of elements processed. For example, Figure 5.2 shows a plot of the number of clock cycles required by the two functions for a range of values of n. Using a least squares fit, we find that the run times (in clock cycles) for `psum1` and `psum2` can be approximated by the equations $368 + 9.0n$ and $368 + 6.0n$, respectively. These equations indicate an overhead of 368 cycles due to the timing code and to initiate the procedure, set up the loop, and complete the

```
1   /* Compute prefix sum of vector a */
2   void psum1(float a[], float p[], long n)
3   {
4       long i;
5       p[0] = a[0];
6       for (i = 1; i < n; i++)
7           p[i] = p[i-1] + a[i];
8   }
9
10  void psum2(float a[], float p[], long n)
11  {
12      long i;
13      p[0] = a[0];
14      for (i = 1; i < n-1; i+=2) {
15          float mid_val = p[i-1] + a[i];
16          p[i]    = mid_val;
17          p[i+1]  = mid_val + a[i+1];
18      }
19      /* For even n, finish remaining element */
20      if (i < n)
21          p[i] = p[i-1] + a[i];
22  }
```

Figure 5.1 Prefix-sum functions. T ti r i a l r r ra r r a .

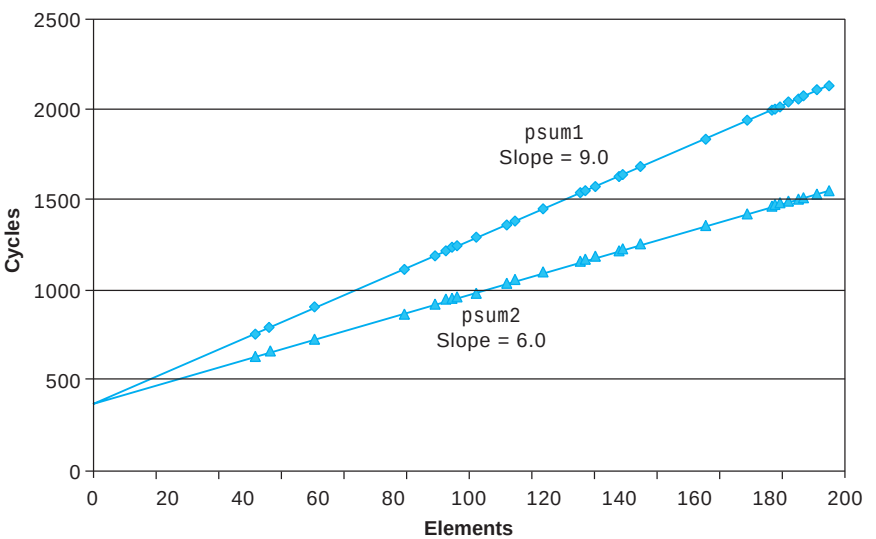


Figure 5.2 Performance of prefix-sum functions. T l t li i i at t r l l r l t (CPE).

Aside What is a least squares fit?

For a set of data points $(x_1, y_1), \ldots (x_n, y_n)$, we often try to draw a line that best approximates the X–Y trend represented by these data. With a least squares fit, we look for a line of the form $y = mx + b$ that minimizes the following error measure:

$$E(m, b) = \sum_{i=1,n} (mx_i + b - y_i)^2$$

An algorithm for computing m and b can be derived by finding the derivatives of $E(m, b)$ with respect to m and b and setting them to 0.

procedure, plus a linear factor of 6.0 or 9.0 cycles per element. For large values of n (say, greater than 200), the run times will be dominated by the linear factors. We refer to the coefficients in these terms as the effective number of cycles per element. We prefer measuring the number of cycles per element rather than the number of cycles per iteration, because techniques such as loop unrolling allow us to use fewer iterations to complete the computation, but our ultimate concern is how fast the procedure will run for a given vector length. We focus our efforts on minimizing the CPE for our computations. By this measure, `psum2`, with a CPE of 6.0, is superior to `psum1`, with a CPE of 9.0.

Practice Problem 5.2 (solution page 609)

Later in this chapter we will start with a single function and generate many different variants that preserve the function's behavior, but with different performance characteristics. For three of these variants, we found that the run times (in clock cycles) can be approximated by the following functions:

Version 1: $60 + 35n$

Version 2: $136 + 4n$

Version 3: $157 + 1.25n$

For what values of n would each version be the fastest of the three? Remember that n will always be an integer.

5.3 Program Example

To demonstrate how an abstract program can be systematically transformed into more efficient code, we will use a running example based on the vector data structure shown in Figure 5.3. A vector is represented with two blocks of memory: the header and the data array. The header is a structure declared as follows:

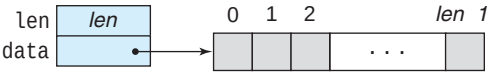


Figure 5.3 Vector abstract data type. A vector is represented by a header and a data array.

code/opt/vec.h

```
1  /* Create abstract data type for vector */
2  typedef struct {
3      long len;
4      data_t *data;
5  } vec_rec, *vec_ptr;
```

code/opt/vec.h

The declaration uses `data_t` to designate the data type of the underlying elements. In our evaluation, we measured the performance of our code for integer (C `int` and `long`), and floating-point (C `float` and `double`) data. We do this by compiling and running the program separately for different type declarations, such as the following for data type `long`:

```
typedef long data_t;
```

We allocate the data array block to store the vector elements as an array of `len` objects of type `data_t`.

Figure 5.4 shows some basic procedures for generating vectors, accessing vector elements, and determining the length of a vector. An important feature to note is that `get_vec_element`, the vector access routine, performs bounds checking for every vector reference. This code is similar to the array representations used in many other languages, including Java. Bounds checking reduces the chances of program error, but it can also slow down program execution.

As an optimization example, consider the code shown in Figure 5.5, which combines all of the elements in a vector into a single value according to some operation. By using different definitions of compile-time constants `IDENT` and `OP`, the code can be recompiled to perform different operations on the data. In particular, using the declarations

```
#define IDENT 0
#define OP  +
```

it sums the elements of the vector. Using the declarations

```
#define IDENT 1
#define OP  *
```

it computes the product of the vector elements.

code/opt/vec.c

```
1  /* Create vector of specified length */
2  vec_ptr new_vec(long len)
3  {
4      /* Allocate header structure */
5      vec_ptr result = (vec_ptr) malloc(sizeof(vec_rec));
6      data_t *data = NULL;
7      if (!result)
8          return NULL;  /* Couldn't allocate storage */
9      result->len = len;
10     /* Allocate array */
11     if (len > 0) {
12         data = (data_t *)calloc(len, sizeof(data_t));
13         if (!data) {
14             free((void *) result);
15             return NULL; /* Couldn't allocate storage */
16         }
17     }
18     /* Data will either be NULL or allocated array */
19     result->data = data;
20     return result;
21 }
22
23 /*
24  * Retrieve vector element and store at dest.
25  * Return 0 (out of bounds) or 1 (successful)
26  */
27 int get_vec_element(vec_ptr v, long index, data_t *dest)
28 {
29     if (index < 0 || index >= v->len)
30         return 0;
31     *dest = v->data[index];
32     return 1;
33 }
34
35 /* Return length of vector */
36 long vec_length(vec_ptr v)
37 {
38     return v->len;
39 }
```

code/opt/vec.c

Figure 5.4 Implementation of vector abstract data type. I t a t al r o ra , ata t data_t i lar t o int, long, float, r double.

```
1   /* Implementation with maximum use of data abstraction */
2   void combine1(vec_ptr v, data_t *dest)
3   {
4       long i;
5
6       *dest = IDENT;
7       for (i = 0; i < vec_length(v); i++) {
8           data_t val;
9           get_vec_element(v, i, &val);
10          *dest = *dest OP val;
11      }
12  }
```

Figure 5.5 Initial implementation of combining operation. Using abstract declarations of data type and operation, this code can be compiled to work for any data type and operation.

we measured the CPE performance of the functions on a machine with an Intel Core i7 Haswell processor, which we refer to as our reference machine. Some characteristics of this processor were given in Section 3.1. These measurements characterize performance in terms of how the programs run on just one particular machine, and so there is no guarantee of comparable performance on other combinations of machine and compiler. However, we have compared the results with those for a number of different compiler/processor combinations, and we have found them generally consistent with those presented here.

As we proceed through a set of transformations, we will find that many lead to only minimal performance gains, while others have more dramatic effects. Determining which combinations of transformations to apply is indeed part of the "black art" of writing fast code. Some combinations that do not provide measurable benefits are indeed ineffective, while others are important as ways to enable further optimizations by the compiler. In our experience, the best approach involves a combination of experimentation and analysis: repeatedly attempting different approaches, performing measurements, and examining the assembly-code representations to identify underlying performance bottlenecks.

As a starting point, the following table shows CPE measurements for `combine1` running on our reference machine, with different combinations of operation (addition or multiplication) and data type (long integer and double-precision floating point). Our experiments with many different programs showed that operations on 32-bit and 64-bit integers have identical performance, with the exception of code involving division operations. Similarly, we found identical performance for programs operating on single- or double-precision floating-point data. In our tables, we will therefore show only separate results for integer data and for floating-point data.

Function	Page	Method	Integer +	Integer *	Floating point +	Floating point *
`combine1`	543	Abstract unoptimized	22.68	20.02	19.98	20.18
`combine1`	543	Abstract `-O1`	10.12	10.12	10.17	11.14

We can see that our measurements are somewhat imprecise. The more likely CPE number for integer sum is 23.00, rather than 22.68, while the number for integer product is likely 20.0 instead of 20.02. Rather than "fudging" our numbers to make them look good, we will present the measurements we actually obtained. There are many factors that complicate the task of reliably measuring the precise number of clock cycles required by some code sequence. It helps when examining these numbers to mentally round the results up or down by a few hundredths of a clock cycle.

The unoptimized code provides a direct translation of the C code into machine code, often with obvious inefficiencies. By simply giving the command-line option `-O1`, we enable a basic set of optimizations. As can be seen, this significantly improves the program performance—more than a factor of 2—with no effort on behalf of the programmer. In general, it is good to get into the habit of enabling some level of optimization. (Similar performance results were obtained with optimization level `-Og`.) For the remainder of our measurements, we use optimization levels `-O1` and `-O2` when generating and measuring our programs.

5.4 Eliminating Loop Inefficiencies

Observe that procedure `combine1`, as shown in Figure 5.5, calls function `vec_length` as the test condition of the `for` loop. Recall from our discussion of how to translate code containing loops into machine-level programs (Section 3.6.7) that the test condition must be evaluated on every iteration of the loop. On the other hand, the length of the vector does not change as the loop proceeds. We could therefore compute the vector length only once and use this value in our test condition.

Figure 5.6 shows a modified version called `combine2`. It calls `vec_length` at the beginning and assigns the result to a local variable `length`. This transformation has noticeable effect on the overall performance for some data types and operations, and minimal or even none for others. In any case, this transformation is required to eliminate inefficiencies that would become bottlenecks as we attempt further optimizations.

Function	Page	Method	Integer +	Integer *	Floating point +	Floating point *
`combine1`	543	Abstract `-O1`	10.12	10.12	10.17	11.14
`combine2`	545	Move `vec_length`	7.02	9.03	9.02	11.03

This optimization is an instance of a general class of optimizations known as code motion. They involve identifying a computation that is performed multiple

```
1  /* Move call to vec_length out of loop */
2  void combine2(vec_ptr v, data_t *dest)
3  {
4      long i;
5      long length = vec_length(v);
6
7      *dest = IDENT;
8      for (i = 0; i < length; i++) {
9          data_t val;
10         get_vec_element(v, i, &val);
11         *dest = *dest OP val;
12     }
13 }
```

Figure 5.6 Improving the efficiency of the loop test. By moving the call to vec_length outside the loop test, we only call the function once, rather than *n* + 1 times.

times, (e.g., within a loop), but such that the result of the computation will not change. We can therefore move the computation to an earlier section of the code that does not get evaluated as often. In this case, we moved the call to `vec_length` from within the loop to just before the loop.

Optimizing compilers attempt to perform code motion. Unfortunately, as discussed previously, they are typically very cautious about making transformations that change where or how many times a procedure is called. They cannot reliably detect whether or not a function will have side effects, and so they assume that it might. For example, if `vec_length` had some side effect, then `combine1` and `combine2` could have different behaviors. To improve the code, the programmer must often help the compiler by explicitly performing code motion.

As an extreme example of the loop inefficiency seen in `combine1`, consider the procedure `lower1` shown in Figure 5.7. This procedure is styled after routines submitted by several students as part of a network programming project. Its purpose is to convert all of the uppercase letters in a string to lowercase. The procedure steps through the string, converting each uppercase character to lowercase. The case conversion involves shifting characters in the range 'A' to 'Z' to the range 'a' to 'z'.

The library function `strlen` is called as part of the loop test of `lower1`. Although `strlen` is typically implemented with special x86 string-processing instructions, its overall execution is similar to the simple version that is also shown in Figure 5.7. Since strings in C are null-terminated character sequences, `strlen` can only determine the length of a string by stepping through the sequence until it hits a null character. For a string of length `n`, `strlen` takes time proportional to `n`. Since `strlen` is called in each of the `n` iterations of `lower1`, the overall run time of `lower1` is quadratic in the string length, proportional to n^2.

```
/* Convert string to lowercase: slow */
void lower1(char *s)
{
    long i;

    for (i = 0; i < strlen(s); i++)
        if (s[i] >= 'A' && s[i] <= 'Z')
            s[i] -= ('A' - 'a');
}

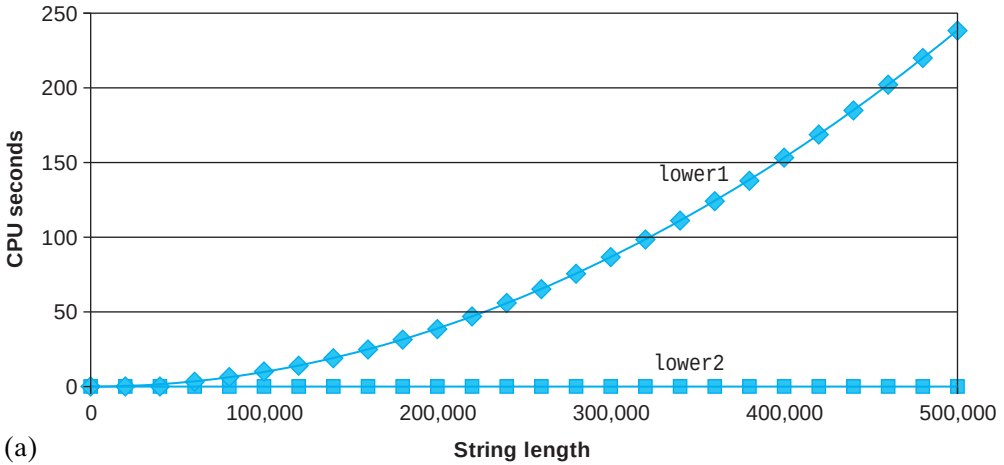
/* Convert string to lowercase: faster */
void lower2(char *s)
{
    long i;
    long len = strlen(s);

    for (i = 0; i < len; i++)
        if (s[i] >= 'A' && s[i] <= 'Z')
            s[i] -= ('A' - 'a');
}

/* Sample implementation of library function strlen */
/* Compute length of string */
size_t strlen(const char *s)
{
    long length = 0;
    while (*s != '\0') {
        s++;
        length++;
    }
    return length;
}
```

Figure 5.7 Lowercase conversion routines. T t o r o r a ra i all i r t r o r a .

This analysis is confirmed by actual measurements of the functions for different length strings, as shown in Figure 5.8 (and using the library version of `strlen`). The graph of the run time for `lower1` rises steeply as the string length increases (Figure 5.8(a)). Figure 5.8(b) shows the run times for seven different lengths (not the same as shown in the graph), each of which is a power of 2. Observe that for `lower1` each doubling of the string length causes a quadrupling of the run time. This is a clear indicator of a quadratic run time. For a string of length 1,048,576, `lower1` requires over 17 minutes of CPU time.



(a)

	String length						
Function	16,384	32,768	65,536	131,072	262,144	524,288	1,048,576
`lower1`	0.26	1.03	4.10	16.41	65.62	262.48	1,049.89
`lower2`	0.0000	0.0001	0.0001	0.0003	0.0005	0.0010	0.0020

(b)

Figure 5.8 Comparative performance of lowercase conversion routines. T ri i al lower1 a a rati r ti t a i fi i tl tr t r . T fi lower2 a ali arr ti .

Function `lower2` shown in Figure 5.7 is identical to that of `lower1`, except that we have moved the call to `strlen` out of the loop. The performance improves dramatically. For a string length of 1,048,576, the function requires just 2.0 milliseconds—over 500,000 times faster than `lower1`. Each doubling of the string length causes a doubling of the run time—a clear indicator of linear run time. For longer strings, the run-time improvement will be even greater.

In an ideal world, a compiler would recognize that each call to `strlen` in the loop test will return the same result, and thus the call could be moved out of the loop. This would require a very sophisticated analysis, since `strlen` checks the elements of the string and these values are changing as `lower1` proceeds. The compiler would need to detect that even though the characters within the string are changing, none are being set from nonzero to zero, or vice versa. Such an analysis is well beyond the ability of even the most sophisticated compilers, even if they employ inlining, and so programmers must do such transformations themselves.

This example illustrates a common problem in writing programs, in which a seemingly trivial piece of code has a hidden asymptotic inefficiency. One would not expect a lowercase conversion routine to be a limiting factor in a program's performance. Typically, programs are tested and analyzed on small data sets, for which the performance of `lower1` is adequate. When the program is ultimately

deployed, however, it is entirely possible that the procedure could be applied to strings of over one million characters. All of a sudden this benign piece of code has become a major performance bottleneck. By contrast, the performance of `lower2` will be adequate for strings of arbitrary length. Stories abound of major programming projects in which problems of this sort occur. Part of the job of a competent programmer is to avoid ever introducing such asymptotic inefficiency.

Practice Problem 5.3 (solution page 609)

Consider the following functions:

```
long min(long x, long y) { return x < y ? x : y; }
long max(long x, long y) { return x < y ? y : x; }
void incr(long *xp, long v) { *xp += v; }
long square(long x) { return x*x; }
```

The following three code fragments call these functions:

A.
```
for (i = min(x, y); i < max(x, y); incr(&i, 1))
    t += square(i);
```

B.
```
for (i = max(x, y) - 1; i >= min(x, y); incr(&i, -1))
    t += square(i);
```

C.
```
long low = min(x, y);
long high = max(x, y);

for (i = low; i < high; incr(&i, 1))
    t += square(i);
```

Assume `x` equals 10 and `y` equals 100. Fill in the following table indicating the number of times each of the four functions is called in code fragments A–C:

Code	`min`	`max`	`incr`	`square`
A.				
B.				
C.				

5.5 Reducing Procedure Calls

As we have seen, procedure calls can incur overhead and also block most forms of program optimization. We can see in the code for `combine2` (Figure 5.6) that `get_vec_element` is called on every loop iteration to retrieve the next vector element. This function checks the vector index `i` against the loop bounds with every vector reference, a clear source of inefficiency. Bounds checking might be a useful feature when dealing with arbitrary array accesses, but a simple analysis of the code for `combine2` shows that all references will be valid.

code/opt/vec.c

```
1  data_t *get_vec_start(vec_ptr v)
2  {
3      return v->data;
4  }
```

code/opt/vec.c

```
1  /* Direct access to vector data */
2  void combine3(vec_ptr v, data_t *dest)
3  {
4      long i;
5      long length = vec_length(v);
6      data_t *data = get_vec_start(v);
7
8      *dest = IDENT;
9      for (i = 0; i < length; i++) {
10         *dest = *dest OP data[i];
11     }
12 }
```

Figure 5.9 Eliminating function calls within the loop. T r lti o t o a r r a ai, t lt a l a lti al o ti iati .

Suppose instead that we add a function `get_vec_start` to our abstract data type. This function returns the starting address of the data array, as shown in Figure 5.9. We could then write the procedure shown as `combine3` in this figure, having no function calls in the inner loop. Rather than making a function call to retrieve each vector element, it accesses the array directly. A purist might say that this transformation seriously impairs the program modularity. In principle, the user of the vector abstract data type should not even need to know that the vector contents are stored as an array, rather than as some other data structure such as a linked list. A more pragmatic programmer would argue that this transformation is a necessary step toward achieving high-performance results.

			Integer		Floating point	
Function	Page	Method	+	*	+	*
`combine2`	545	Move `vec_length`	7.02	9.03	9.02	11.03
`combine3`	549	Direct data access	7.17	9.02	9.02	11.03

Surprisingly, there is no apparent performance improvement. Indeed, the performance for integer sum has gotten slightly worse. Evidently, other operations in the inner loop are forming a bottleneck that limits the performance more than the call to `get_vec_element`. We will return to this function later (Section 5.11.2) and see why the repeated bounds checking by `combine2` does not incur a performance penalty. For now, we can view this transformation as one of a series of steps that will ultimately lead to greatly improved performance.

The code for `combine3` accumulates the value being computed by the combining operation at the location designated by the pointer `dest`. This attribute can be seen by examining the assembly code generated for the inner loop of the compiled code. We show here the x86-64 code generated for data type `double` and with multiplication as the combining operation:

```
    Inner loop of combine3.  data_t = double, OP = *
    dest in %rbx, data+i in %rdx, data+length in %rax
1   .L17:                                   loop:
2     vmovsd  (%rbx), %xmm0                   Read product from dest
3     vmulsd  (%rdx), %xmm0, %xmm0            Multiply product by data[i]
4     vmovsd  %xmm0, (%rbx)                   Store product at dest
5     addq    $8, %rdx                        Increment data+i
6     cmpq    %rax, %rdx                      Compare to data+length
7     jne     .L17                            If !=, goto loop
```

We see in this loop code that the address corresponding to pointer `dest` is held in register `%rbx`. It has also transformed the code to maintain a pointer to the `i`th data element in register `%rdx`, shown in the annotations as `data+i`. This pointer is incremented by 8 on every iteration. The loop termination is detected by comparing this pointer to one stored in register `%rax`. We can see that the accumulated value is read from and written to memory on each iteration. This reading and writing is wasteful, since the value read from `dest` at the beginning of each iteration should simply be the value written at the end of the previous iteration.

We can eliminate this needless reading and writing of memory by rewriting the code in the style of `combine4`

```
1   /* Accumulate result in local variable */
2   void combine4(vec_ptr v, data_t *dest)
3   {
4       long i;
5       long length = vec_length(v);
6       data_t *data = get_vec_start(v);
7       data_t acc = IDENT;
8
9       for (i = 0; i < length; i++) {
10          acc = acc OP data[i];
11      }
12      *dest = acc;
13  }
```

Figure 5.10 Accumulating result in temporary. Holding the value in local variable acc (short for accumulator) allows it to be stored in a register during the execution of the loop.

			Integer		Floating point	
Function	Page	Method	+	*	+	*
combine3	549	Direct data access	7.17	9.02	9.02	11.03
combine4	551	Accumulate in temporary	1.27	3.01	3.01	5.01

All of our times improve by factors ranging from 2.2× to 5.7×, with the integer addition case dropping to just 1.27 clock cycles per element.

Again, one might think that a compiler should be able to automatically transform the combine3 code shown in Figure 5.9 to accumulate the value in a register, as it does with the code for combine4 shown in Figure 5.10. In fact, however, the two functions can have different behaviors due to memory aliasing. Consider, for example, the case of integer data with multiplication as the operation and 1 as the identity element. Let v = [2, 3, 5] be a vector of three elements and consider the following two function calls:

```
combine3(v, get_vec_start(v) + 2);
combine4(v, get_vec_start(v) + 2);
```

That is, we create an alias between the last element of the vector and the destination for storing the result. The two functions would then execute as follows:

Function	Initial	Before loop	i = 0	i = 1	i = 2	Final
combine3	[2, 3, 5]	[2, 3, 1]	[2, 3, 2]	[2, 3, 6]	[2, 3, 36]	[2, 3, 36]
combine4	[2, 3, 5]	[2, 3, 5]	[2, 3, 5]	[2, 3, 5]	[2, 3, 5]	[2, 3, 30]

As shown previously, `combine3` accumulates its result at the destination, which in this case is the final vector element. This value is therefore set first to 1, then to $2 \cdot 1 = 2$, and then to $3 \cdot 2 = 6$. On the last iteration, this value is then multiplied by itself to yield a final value of 36. For the case of `combine4`, the vector remains unchanged until the end, when the final element is set to the computed result $1 \cdot 2 \cdot 3 \cdot 5 = 30$.

Of course, our example showing the distinction between `combine3` and `combine4` is highly contrived. One could argue that the behavior of `combine4` more closely matches the intention of the function description. Unfortunately, a compiler cannot make a judgment about the conditions under which a function might be used and what the programmer's intentions might be. Instead, when given `combine3` to compile, the conservative approach is to keep reading and writing memory, even though this is less efficient.

Practice Problem 5.4 (solution page 610)

When we use GCC to compile `combine3` with command-line option `-O2`, we get code with substantially better CPE performance than with `-O1`:

			Integer		Floating point	
Function	Page	Method	+	*	+	*
`combine3`	549	Compiled `-O1`	7.17	9.02	9.02	11.03
`combine3`	549	Compiled `-O2`	1.60	3.01	3.01	5.01
`combine4`	551	Accumulate in temporary	1.27	3.01	3.01	5.01

We achieve performance comparable to that for `combine4`, except for the case of integer sum, but even it improves significantly. On examining the assembly code generated by the compiler, we find an interesting variant for the inner loop:

```
    Inner loop of combine3.  data_t = double, OP = *.  Compiled -O2
    dest in %rbx, data+i in %rdx, data+length in %rax
    Accumulated product in %xmm0
1   .L22:                                   loop:
2     vmulsd  (%rdx), %xmm0, %xmm0            Multiply product by data[i]
3     addq    $8, %rdx                        Increment data+i
4     cmpq    %rax, %rdx                      Compare to data+length
5     vmovsd  %xmm0, (%rbx)                   Store product at dest
6     jne     .L22                            If !=, goto loop
```

We can compare this to the version created with optimization level 1:

```
    Inner loop of combine3.  data_t = double, OP = *.  Compiled -O1
    dest in %rbx, data+i in %rdx, data+length in %rax
1   .L17:                                   loop:
2     vmovsd  (%rbx), %xmm0                   Read product from dest
3     vmulsd  (%rdx), %xmm0, %xmm0            Multiply product by data[i]
4     vmovsd  %xmm0, (%rbx)                   Store product at dest
```

```
5       addq    $8, %rdx                Increment data+i
6       cmpq    %rax, %rdx              Compare to data+length
7       jne     .L17                    If !=, goto loop
```

We see that, besides some reordering of instructions, the only difference is that the more optimized version does not contain the `vmovsd` implementing the read from the location designated by `dest` (line 2).

A. How does the role of register `%xmm0` differ in these two loops?

B. Will the more optimized version faithfully implement the C code of `combine3`, including when there is memory aliasing between `dest` and the vector data?

C. Either explain why this optimization preserves the desired behavior, or give an example where it would produce different results than the less optimized code.

With this final transformation, we reached a point where we require just 1.25–5 clock cycles for each element to be computed. This is a considerable improvement over the original 9–11 cycles when we first enabled optimization. We would now like to see just what factors are constraining the performance of our code and how we can improve things even further.

5.7 Understanding Modern Processors

Up to this point, we have applied optimizations that did not rely on any features of the target machine. They simply reduced the overhead of procedure calls and eliminated some of the critical "optimization blockers" that cause difficulties for optimizing compilers. As we seek to push the performance further, we must consider optimizations that exploit the microarchitecture of the processor—that is, the underlying system design by which a processor executes instructions. Getting every last bit of performance requires a detailed analysis of the program as well as code generation tuned for the target processor. Nonetheless, we can apply some basic optimizations that will yield an overall performance improvement on a large class of processors. The detailed performance results we report here may not hold for other machines, but the general principles of operation and optimization apply to a wide variety of machines.

To understand ways to improve performance, we require a basic understanding of the microarchitectures of modern processors. Due to the large number of transistors that can be integrated onto a single chip, modern microprocessors employ complex hardware that attempts to maximize program performance. One result is that their actual operation is far different from the view that is perceived by looking at machine-level programs. At the code level, it appears as if instructions are executed one at a time, where each instruction involves fetching values from registers or memory, performing an operation, and storing results back to a register or memory location. In the actual processor, a number of instructions

are evaluated simultaneously, a phenomenon referred to as instruction-level parallelism. In some designs, there can be 100 or more instructions "in flight." Elaborate mechanisms are employed to make sure the behavior of this parallel execution exactly captures the sequential semantic model required by the machine-level program. This is one of the remarkable feats of modern microprocessors: they employ complex and exotic microarchitectures, in which multiple instructions can be executed in parallel, while presenting an operational view of simple sequential instruction execution.

Although the detailed design of a modern microprocessor is well beyond the scope of this book, having a general idea of the principles by which they operate suffices to understand how they achieve instruction-level parallelism. We will find that two different lower bounds characterize the maximum performance of a program. The latency bound is encountered when a series of operations must be performed in strict sequence, because the result of one operation is required before the next one can begin. This bound can limit program performance when the data dependencies in the code limit the ability of the processor to exploit instruction-level parallelism. The throughput bound characterizes the raw computing capacity of the processor's functional units. This bound becomes the ultimate limit on program performance.

5.7.1 Overall Operation

Figure 5.11 shows a very simplified view of a modern microprocessor. Our hypothetical processor design is based loosely on the structure of recent Intel processors. These processors are described in the industry as being superscalar, which means they can perform multiple operations on every clock cycle and out of order, meaning that the order in which instructions execute need not correspond to their ordering in the machine-level program. The overall design has two main parts: the instruction control unit (ICU), which is responsible for reading a sequence of instructions from memory and generating from these a set of primitive operations to perform on program data, and the execution unit (EU), which then executes these operations. Compared to the simple in-order pipeline we studied in Chapter 4, out-of-order processors require far greater and more complex hardware, but they are better at achieving higher degrees of instruction-level parallelism.

The ICU reads the instructions from an instruction cache—a special high-speed memory containing the most recently accessed instructions. In general, the ICU fetches well ahead of the currently executing instructions, so that it has enough time to decode these and send operations down to the EU. One problem, however, is that when a program hits a branch,[1] there are two possible directions the program might go. The branch can be taken, with control passing to the branch target. Alternatively, the branch can be not taken, with control passing to the next

1. We use the term "branch" specifically to refer to conditional jump instructions. Other instructions that can transfer control to multiple destinations, such as procedure return and indirect jumps, provide similar challenges for the processor.

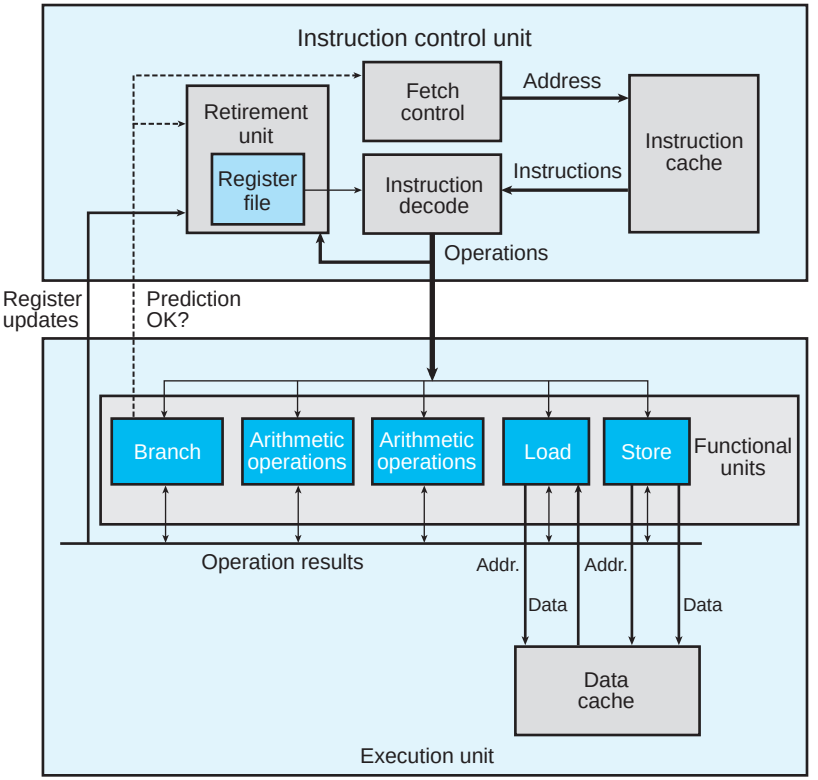


Figure 5.11 Block diagram of an out-of-order processor. T i tr ti o tr l it ir o il rra i i tr ti r o r a rati a ri iti o rati . T ti it r r t o rati a i iat t rt ra r rr tl r it .

instruction in the instruction sequence. Modern processors employ a technique known as branch prediction, in which they guess whether or not a branch will be taken and also predict the target address for the branch. Using a technique known as speculative execution, the processor begins fetching and decoding instructions at where it predicts the branch will go, and even begins executing these operations before it has been determined whether or not the branch prediction was correct. If it later determines that the branch was predicted incorrectly, it resets the state to that at the branch point and begins fetching and executing instructions in the other direction. The block labeled "Fetch control" incorporates branch prediction to perform the task of determining which instructions to fetch.

The instruction decoding logic takes the actual program instructions and converts them into a set of primitive operations (sometimes referred to as micro-operations). Each of these operations performs some simple computational task such as adding two numbers, reading data from memory, or writing data to memory. For machines with complex instructions, such as x86 processors, an instruction

can be decoded into multiple operations. The details of how instructions are decoded into sequences of operations varies between machines, and this information is considered highly proprietary. Fortunately, we can optimize our programs without knowing the low-level details of a particular machine implementation.

In a typical x86 implementation, an instruction that only operates on registers, such as

```
addq %rax,%rdx
```

is converted into a single operation. On the other hand, an instruction involving one or more memory references, such as

```
addq %rax,8(%rdx)
```

yields multiple operations, separating the memory references from the arithmetic operations. This particular instruction would be decoded as three operations: one to load a value from memory into the processor, one to add the loaded value to the value in register `%eax`, and one to store the result back to memory. The decoding splits instructions to allow a division of labor among a set of dedicated hardware units. These units can then execute the different parts of multiple instructions in parallel.

The EU receives operations from the instruction fetch unit. Typically, it can receive a number of them on each clock cycle. These operations are dispatched to a set of functional units that perform the actual operations. These functional units are specialized to handle different types of operations.

Reading and writing memory is implemented by the load and store units. The load unit handles operations that read data from the memory into the processor. This unit has an adder to perform address computations. Similarly, the store unit handles operations that write data from the processor to the memory. It also has an adder to perform address computations. As shown in the figure, the load and store units access memory via a data cache, a high-speed memory containing the most recently accessed data values.

With speculative execution, the operations are evaluated, but the final results are not stored in the program registers or data memory until the processor can be certain that these instructions should actually have been executed. Branch operations are sent to the EU, not to determine where the branch should go, but rather to determine whether or not they were predicted correctly. If the prediction was incorrect, the EU will discard the uestatuesdiscard can

microprocessor chip has grown over time, successive models of microprocessors have increased the total number of functional units, the combinations of operations each unit can perform, and the performance of each of these units. The arithmetic units are intentionally designed to be able to perform a variety of different operations, since the required operations vary widely across different programs. For example, some programs might involve many integer operations, while others require many floating-point operations. If one functional unit were specialized to perform integer operations while another could only perform floating-point operations, then none of these programs would get the full benefit of having multiple functional units.

For example, our Intel Core i7 Haswell reference machine has eight functional units, numbered 0–7. Here is a partial list of each one's capabilities:

0. Integer arithmetic, floating-point multiplication, integer and floating-point division, branches
1. Integer arithmetic, floating-point addition, integer multiplication, floating-point multiplication
2. Load, address computation
3. Load, address computation
4. Store
5. Integer arithmetic
6. Integer arithmetic, branches
7. Store address computation

In the above list, "integer arithmetic" refers to basic operations, such as addition, bitwise operations, and shifting. Multiplication and division require more specialized resources. We see that a store operation requires two functional units—one to compute the store address and one to actually store the data. We will discuss the mechanics of store (and load) operations in Section 5.12.

We can see that this combination of functional units has the potential to perform multiple operations of the same type simultaneously. It has four units capable of performing integer operations, two that can perform load operations, and two that can perform floating-point multiplication. We will later see the impact these resources have on the maximum performance our programs can achieve.

Within the ICU, the retirement unit keeps track of the ongoing processing and makes sure that it obeys the sequential semantics of the machine-level program. Our figure shows a register file containing the integer, floating-point, and, more recently, SSE and AVX registers as part of the retirement unit, because this unit controls the updating of these registers. As an instruction is decoded, information about it is placed into a first-in, first-out queue. This information remains in the queue until one of two outcomes occurs. First, once the operations for the instruction have completed and any branch points leading to this instruction are confirmed as having been correctly predicted, the instruction can be retired, with any updates to the program registers being made. If some branch point leading to this instruction was mispredicted, on the other hand, the instruction will be

Aside T it r o o t- o - r r o i

Out-of-order processing was first implemented in the Control Data Corporation 6600 processor in 1964. Instructions were processed by 10 different functional units, each of which could be operated independently. In its day, this machine, with a clock rate of 10 MHz, was considered the premium machine for scientific computing.

IBM first implemented out-of-order processing with the IBM 360/91 processor in 1966, but just to execute the floating-point instructions. For around 25 years, out-of-order processing was considered an exotic technology, found only in machines striving for the highest possible performance, until IBM reintroduced it in the RS/6000 line of workstations in 1990. This design became the basis for the IBM/Motorola PowerPC line, with the model 601, introduced in 1993, becoming the first single-chip microprocessor to use out-of-order processing. Intel introduced out-of-order processing with its PentiumPro model in 1995, with an underlying microarchitecture similar to that of our reference machine.

flushed, discarding any results that may have been computed. By this means, mispredictions will not alter the program state.

As we have described, any updates to the program registers occur only as instructions are being retired, and this takes place only after the processor can be certain that any branches leading to this instruction have been correctly predicted. To expedite the communication of results from one instruction to another, much of this information is exchanged among the execution units, shown in the figure as "Operation results." As the arrows in the figure show, the execution units can send results directly to each other. This is a more elaborate form of the data-forwarding techniques we incorporated into our simple processor design in Section 4.5.5.

The most common mechanism for controlling the communication of operands among the execution units is called register renaming. When an instruction that updates register r is decoded, a tag t is generated giving a unique identifier to the result of the operation. An entry (r, t) is added to a table maintaining the association between program register r and tag t for an operation that will update this register. When a subsequent instruction using register r as an operand is decoded, the operation sent to the execution unit will contain t as the source for the operand value. When some execution unit completes the first operation, it generates a result (v, t), indicating that the operation with tag t produced value v. Any operation waiting for t as a source will then use v as the source value, a form of data forwarding. By this mechanism, values can be forwarded directly from one operation to another, rather than being written to and read from the register file, enabling the second operation to begin as soon as the first has completed. The renaming table only contains entries for registers having pending write operations. When a decoded instruction requires a register r, and there is no tag associated with this register, the operand is retrieved directly from the register file. With register renaming, an entire sequence of operations can be performed speculatively, even though the registers are updated only after the processor is certain of the branch outcomes.

Operation	Integer			Floating point		
	Latency	Issue	Capacity	Latency	Issue	Capacity
Addition	1	1	4	3	1	1
Multiplication	3	1	1	5	1	2
Division	3–30	3–30	1	3–15	3–15	1

Figure 5.12 Latency, issue time, and capacity characteristics of reference machine operations. Lat i \v i at t t tal r o l o l r i \v t o r o r t a t al rati , i i ti i \v i at t i i r o l t t i \v o \v o t o rati o . T a a it i \v i at a t o rati a i \v i lta o l . T ti r \v i i i \v o \v o t \v ata al .

5.7.2 Functional Unit Performance

Figure 5.12 documents the performance of some of the arithmetic operations for our Intel Core i7 Haswell reference machine, determined by both measurements and by reference to Intel literature [49]. These timings are typical for other processors as well. Each operation is characterized by its latency, meaning the total time required to perform the operation, the issue time, meaning the minimum number of clock cycles between two independent operations of the same type, and the capacity, indicating the number of functional units capable of performing that operation.

We see that the latencies increase in going from integer to floating-point operations. We see also that the addition and multiplication operations all have issue times of 1, meaning that on each clock cycle, the processor can start a new one of these operations. This short issue time is achieved through the use of pipelining. A pipelined function unit is implemented as a series of stages, each of which performs part of the operation. For example, a typical floating-point adder contains three stages (and hence the three-cycle latency): one to process the exponent values, one to add the fractions, and one to round the result. The arithmetic operations can proceed through the stages in close succession rather than waiting for one operation to complete before the next begins. This capability can be exploited only if there are successive, logically independent operations to be performed. Functional units with issue times of 1 cycle are said to be fully pipelined: they can start a new operation every clock cycle. Operations with capacity greater than 1 arise due to the capabilities of the multiple functional units, as was described earlier for the reference machine.

We see also that the divider (used for integer and floating-point division, as well as floating-point square root) is not pipelined—its issue time equals its latency. What this means is that the divider must perform a complete division before it can begin a new one. We also see that the latencies and issue times for division are given as ranges, because some combinations of dividend and divisor require more steps than others. The long latency and issue times of division make it a comparatively costly operation.

A more common way of expressing issue time is to specify the maximum throughput of the unit, defined as the reciprocal of the issue time. A fully pipelined functional unit has a maximum throughput of 1 operation per clock cycle, while units with higher issue times have lower maximum throughput. Having multiple functional units can increase throughput even further. For an operation with capacity C and issue time I, the processor can potentially achieve a throughput of C/I operations per clock cycle. For example, our reference machine is capable of performing floating-point multiplication operations at a rate of 2 per clock cycle. We will see how this capability can be exploited to increase program performance.

Circuit designers can create functional units with wide ranges of performance characteristics. Creating a unit with short latency or with pipelining requires more hardware, especially for more complex functions such as multiplication and floating-point operations. Since there is only a limited amount of space for these units on the microprocessor chip, CPU designers must carefully balance the number of functional units and their individual performance to achieve optimal overall performance. They evaluate many different benchmark programs and dedicate the most resources to the most critical operations. As Figure 5.12 indicates, integer multiplication and floating-point multiplication and addition were considered important operations in the design of the Core i7 Haswell processor, even though a significant amount of hardware is required to achieve the low latencies and high degree of pipelining shown. On the other hand, division is relatively infrequent and difficult to implement with either short latency or full pipelining.

The latencies, issue times, and capacities of these arithmetic operations can affect the performance of our combining functions. We can express these effects in terms of two fundamental bounds on the CPE values:

	Integer		Floating point	
Bound	+	*	+	*
Latency	1.00	3.00	3.00	5.00
Throughput	0.50	1.00	1.00	0.50

The latency bound gives a minimum value for the CPE for any function that must perform the combining operation in a strict sequence. The throughput bound gives a minimum bound for the CPE based on the maximum rate at which the functional units can produce results. For example, since there is only one integer multiplier, and it has an issue time of 1 clock cycle, the processor cannot possibly sustain a rate of more than 1 multiplication per clock cycle. On the other hand, with four functional units capable of performing integer addition, the processor can potentially sustain a rate of 4 operations per cycle. Unfortunately, the need to read elements from memory creates an additional throughput bound. The two load units limit the processor to reading at most 2 data values per clock cycle, yielding a throughput bound of 0.50. We will demonstrate the effect of both the latency and throughput bounds with different versions of the combining functions.

5.7.3 An Abstract Model of Processor Operation

As a tool for analyzing the performance of a machine-level program executing on a modern processor, we will use a data-flow representation of programs, a graphical notation showing how the data dependencies between the different operations constrain the order in which they are executed. These constraints then lead to critical paths in the graph, putting a lower bound on the number of clock cycles required to execute a set of machine instructions.

Before proceeding with the technical details, it is instructive to examine the CPE measurements obtained for function `combine4`, our fastest code up to this point:

			Integer		Floating point	
Function	Page	Method	+	*	+	*
`combine4`	551	Accumulate in temporary	1.27	3.01	3.01	5.01
Latency bound			1.00	3.00	3.00	5.00
Throughput bound			0.50	1.00	1.00	0.50

We can see that these measurements match the latency bound for the processor, except for the case of integer addition. This is not a coincidence—it indicates that the performance of these functions is dictated by the latency of the sum or product computation being performed. Computing the product or sum of n elements requires around $L \cdot n + K$ clock cycles, where L is the latency of the combining operation and K represents the overhead of calling the function and initiating and terminating the loop. The CPE is therefore equal to the latency bound L.

Fr$_0$ Ma i -L l C$_0$ t$_0$ Data-Fl$_0$ Gra

Our data-flow representation of programs is informal. We use it as a way to visualize how the data dependencies in a program dictate its performance. We present the data-flow notation by working with `combine4` (Figure 5.10) as an example. We focus just on the computation performed by the loop, since this is the dominating factor in performance for large vectors. We consider the case of data type `double` with multiplication as the combining operation. Other combinations of data type and operation yield similar code. The compiled code for this loop consists of four instructions, with registers `%rdx` holding a pointer to the `i`th element of array `data`, `%rax` holding a pointer to the end of the array, and `%xmm0` holding the accumulated value `acc`.

```
    Inner loop of combine4.  data_t = double, OP = *
    acc in %xmm0, data+i in %rdx, data+length in %rax
1   .L25:                                  loop:
2     vmulsd  (%rdx), %xmm0, %xmm0           Multiply acc by data[i]
3     addq    $8, %rdx                       Increment data+i
4     cmpq    %rax, %rdx                     Compare to data+length
5     jne     .L25                           If !=, goto loop
```

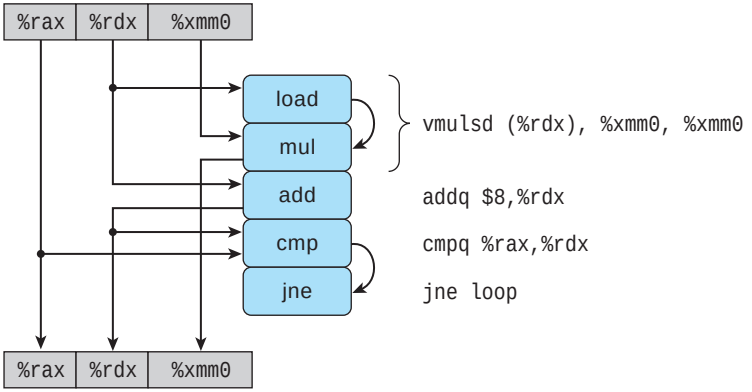


Figure 5.13 Graphical representation of inner-loop code for combine4. I tr ti ar a i all tra lat i t rt rati , a i r i al r t r rati r r r it r a r al rt r rati a r r it r. W t tar t t fi al i tr ti a t la l loop. It t t fir t i tr ti .

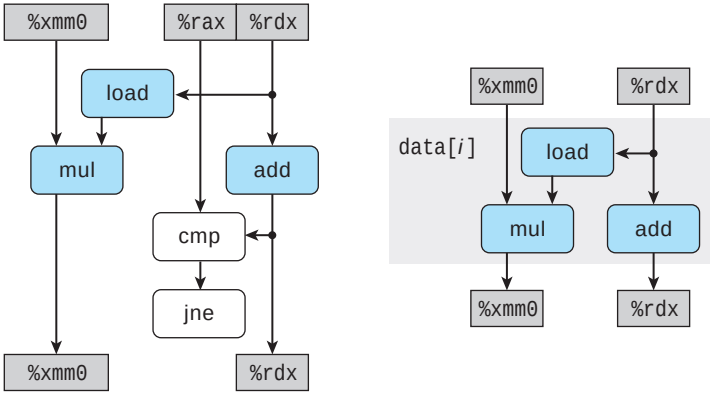
As Figure 5.13 indicates, with our hypothetical processor design, the four instructions are expanded by the instruction decoder into a series of five operations, with the initial multiplication instruction being expanded into a l a operation to read the source operand from memory, and a l operation to perform the multiplication.

As a step toward generating a data-flow graph representation of the program, the boxes and lines along the left-hand side of Figure 5.13 show how the registers are used and updated by the different operations, with the boxes along the top representing the register values at the beginning of the loop, and those along the bottom representing the values at the end. For example, register %rax is only used as a source value by the operation, and so the register has the same value at the end of the loop as at the beginning. Register %rdx, on the other hand, is both used and updated within the loop. Its initial value is used by the l a and a operations; its new value is generated by the a operation, which is then used by the operation. Register %xmm0 is also updated within the loop by the l operation, which first uses the initial value as a source value.

Some of the operations in Figure 5.13 produce values that do not correspond to registers. We show these as arcs between operations on the right-hand side. The l a operation reads a value from memory and passes it directly to the l operation. Since these two operations arise from decoding a single vmulsd instruction, there is no register associated with the intermediate value passing between them. The operation updates the condition codes, and these are then tested by the operation.

For a code segment forming a loop, we can classify the registers that are accessed into four categories:

Figure 5.14
Abstracting combine4 operations as a data-flow graph. W r arra t o rat r o Fi r 5.13 t r l arl o t ata i (a), a t rt r o o l t rati t at al r o it rati t r al r t ( ).



Read-only. These are used as source values, either as data or to compute memory addresses, but they are not modified within the loop. The only read-only register for the loop in `combine4` is `%rax`.

Write-only. These are used as the destinations of data-movement operations. There are no such registers in this loop.

Local. These are updated and used within the loop, but there is no dependency from one iteration to another. The condition code registers are examples for this loop: they are updated by the operation and used by the operation, but this dependency is contained within individual iterations.

Loop. These are used both as source values and as destinations for the loop, with the value generated in one iteration being used in another. We can see that `%rdx` and `%xmm0` are loop registers for `combine4`, corresponding to program values `data+i` and `acc`.

As we will see, the chains of operations between loop registers determine the performance-limiting data dependencies.

Figure 5.14 shows further refinements of the graphical representation of Figure 5.13, with a goal of showing only those operations and data dependencies that affect the program execution time. We see in Figure 5.14(a) that we rearranged the operators to show more clearly the flow of data from the source registers at the top (both read-only and loop registers) and to the destination registers at the bottom (both write-only and loop registers).

In Figure 5.14(a), we also color operators white if they are not part of some chain of dependencies between loop registers. For this example, the comparison ( ) and branch ( ) operations do not directly affect the flow of data in the program. We assume that the instruction control unit predicts that branch will be taken, and hence the program will continue looping. The purpose of the compare

not taken. We assume this checking can be done quickly enough that it does not slow down the processor.

In Figure 5.14(b), we have eliminated the operators that were colored white on the left, and we have retained only the loop registers. What we have left is an abstract template showing the data dependencies that form among loop registers due to one iteration of the loop. We can see in this diagram that there are two data dependencies from one iteration to the next. Along one side, we see the dependencies between successive values of program value `acc`, stored in register `%xmm0`. The loop computes a new value for `acc` by multiplying the old value by a data element, generated by the load operation. Along the other side, we see the dependencies between successive values of the pointer to the `i`th data element. On each iteration, the old value is used as the address for the load operation, and it is also incremented by the add operation to compute its new value.

Figure 5.15 shows the data-flow representation of `n` iterations by the inner loop of function `combine4`. This graph was obtained by simply replicating the template shown in Figure 5.14(b) `n` times. We can see that the program has two chains of data

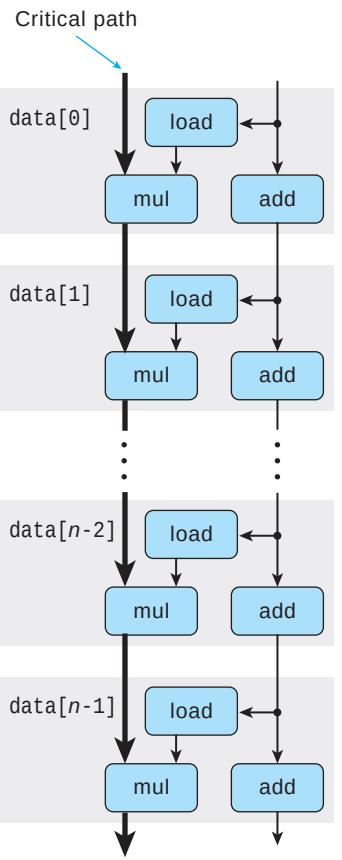


Figure 5.15
Data-flow representation of computation by n iterations of the inner loop of combine4. T lti li ati o rati o r a rti i al o at t at li it r o ra r o r a .

dependencies, corresponding to the updating of program values `acc` and `data+i` with operations `l` and `a`, respectively. Given that floating-point multiplication has a latency of 5 cycles, while integer addition has a latency of 1 cycle, we can see that the chain on the left will form a critical path, requiring 5`n` cycles to execute. The chain on the right would require only `n` cycles to execute, and so it does not limit the program performance.

Figure 5.15 demonstrates why we achieved a CPE equal to the latency bound of 5 cycles for `combine4`, when performing floating-point multiplication. When executing the function, the floating-point multiplier becomes the limiting resource. The other operations required during the loop—manipulating and testing pointer value `data+i` and reading data from memory—proceed in parallel with the multiplication. As each successive value of `acc` is computed, it is fed back around to compute the next value, but this will not occur until 5 cycles later.

The flow for other combinations of data type and operation are identical to those shown in Figure 5.15, but with a different data operation forming the chain of data dependencies shown on the left. For all of the cases where the operation has a latency `L` greater than 1, we see that the measured CPE is simply `L`, indicating that this chain forms the performance-limiting critical path.

Ot r P r$_0$r a Fa t$_0$r

For the case of integer addition, on the other hand, our measurements of `combine4` show a CPE of 1.27, slower than the CPE of 1.00 we would predict based on the chains of dependencies formed along either the left- or the right-hand side of the graph of Figure 5.15. This illustrates the principle that the -241(the)-23(theuis)-241(ns)-241as dat-e r(pfisw enacyicl eb0267 prvside

Practice Problem 5.5 (solution page 611)

Suppose we wish to write a function to evaluate a polynomial, where a polynomial of degree n is defined to have a set of coefficients $a_0, a_1, a_2, \ldots, a_n$. For a value x, we evaluate the polynomial by computing

$$a_0 + a_1x + a_2x^2 + \cdots + a_nx^n \tag{5.2}$$

This evaluation can be implemented by the following function, having as arguments an array of coefficients a, a value x, and the polynomial degree degree (the value n in Equation 5.2). In this function, we compute both the successive terms of the equation and the successive powers of x within a single loop:

```
1  double poly(double a[], double x, long degree)
2  {
3      long i;
4      double result = a[0];
5      double xpwr = x;  /* Equals x^i at start of loop */
6      for (i = 1; i <= degree; i++) {
7          result += a[i] * xpwr;
8          xpwr = x * xpwr;
9      }
10     return result;
11 }
```

A. For degree n, how many additions and how many multiplications does this code perform?

B. On our reference machine, with arithmetic operations having the latencies shown in Figure 5.12, we measure the CPE for this function to be 5.00. Explain how this CPE arises based on the data dependencies formed between iterations due to the operations implementing lines 7–8 of the function.

Practice Problem 5.6 (solution page 611)

Let us continue exploring ways to evaluate polynomials, as described in Practice Problem 5.5. We can reduce the number of multiplications in evaluating a polynomial by applying Horner's method, named after British mathematician William G. Horner (1786–1837). The idea is to repeatedly factor out the powers of x to get the following evaluation:

$$a_0 + x(a_1 + x(a_2 + \cdots + x(a_{n-1} + xa_n)\cdots)) \tag{5.3}$$

Using Horner's method, we can implement polynomial evaluation using the following code:

```
1  /* Apply Horner's method */
2  double polyh(double a[], double x, long degree)
3  {
```

```
4       long i;
5       double result = a[degree];
6       for (i = degree-1; i >= 0; i--)
7           result = a[i] + x*result;
8       return result;
9   }
```

A. For degree n, how many additions and how many multiplications does this code perform?

B. On our reference machine, with the arithmetic operations having the latencies shown in Figure 5.12, we measure the CPE for this function to be 8.00. Explain how this CPE arises based on the data dependencies formed between iterations due to the operations implementing line 7 of the function.

C. Explain how the function shown in Practice Problem 5.5 can run faster, even though it requires more operations.

5.8 Loop Unrolling

Loop unrolling is a program transformation that reduces the number of iterations for a loop by increasing the number of elements computed on each iteration. We saw an example of this with the function psum2 (Figure 5.1), where each iteration computes two elements of the prefix sum, thereby halving the total number of iterations required. Loop unrolling can improve performance in two ways. First, it reduces the number of operations that do not contribute directly to the program result, such as loop indexing and conditional branching. Second, it exposes ways in which we can further transform the code to reduce the number of operations in the critical paths of the overall computation. In this section, we will examine simple loop unrolling, without any further transformations.

Figure 5.16 shows a version of our combining code using what we will refer to as "2 × 1 loop unrolling." The first loop steps through the array two elements at a time. That is, the loop index i is incremented by 2 on each iteration, and the combining operation is applied to array elements i and i + 1 in a single iteration.

In general, the vector length will not be a multiple of 2. We want our code to work correctly for arbitrary vector lengths. We account for this requirement in two ways. First, we make sure the first loop does not overrun the array bounds. For a vector of length n, we set the loop limit to be n − 1. We are then assured that the loop will only be executed when the loop index i satisfies i < n − 1, and hence the maximum array index i + 1 will satisfy i + 1 < (n − 1) + 1 = n.

We can generalize this idea to unroll a loop by any factor k, yielding k × 1 loop unrolling. To do so, we set the upper limit to be n − k + 1 and within the loop apply the combining operation to elements i through i + k − 1. Loop index i is incremented by k in each iteration. The maximum array index i + k − 1 will then be less than n. We include the second loop to step through the final few elements of the vector one at a time. The body of this loop will be executed between 0 and k − 1 times. For k = 2, we could use a simple conditional statement

```
/* 2 x 1 loop unrolling */
void combine5(vec_ptr v, data_t *dest)
{
    long i;
    long length = vec_length(v);
    long limit = length-1;
    data_t *data = get_vec_start(v);
    data_t acc = IDENT;

    /* Combine 2 elements at a time */
    for (i = 0; i < limit; i+=2) {
        acc = (acc OP data[i]) OP data[i+1];
    }

    /* Finish any remaining elements */
    for (; i < length; i++) {
        acc = acc OP data[i];
    }
    *dest = acc;
}
```

Figure 5.16 Applying 2 × 1 loop unrolling. T i tra r ati a r t t o l oo o r a .

to optionally add a final iteration, as we did with the function psum2 (Figure 5.1). For k > 2, the finishing cases are better expressed with a loop, and so we adopt this programming convention for k = 2 as well. We refer to this transformation as "k × 1 loop unrolling," since we unroll by a factor of k but accumulate values in a single variable acc.

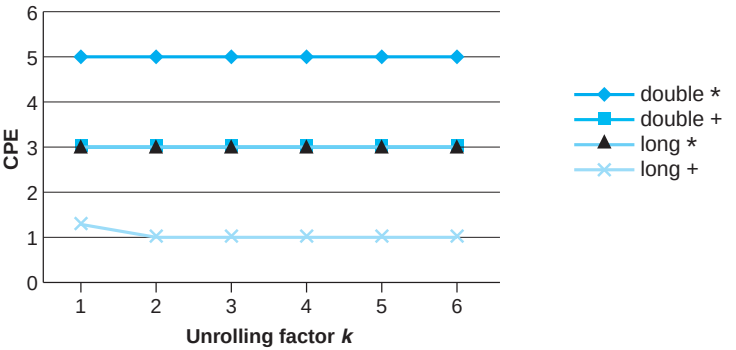
Practice Problem 5.7 (solution page 611)

Modify the code for combine5 to unroll the loop by a factor k = 5.

When we measure the performance of unrolled code for unrolling factors k = 2 (combine5) and k = 3, we get the following results:

			Integer		Floating point	
Function	Page	Method	+	*	+	*
combine4	551	No unrolling	1.27	3.01	3.01	5.01
combine5	568	2 × 1 unrolling	1.01	3.01	3.01	5.01
		3 × 1 unrolling	1.01	3.01	3.01	5.01
Latency bound			1.00	3.00	3.00	5.00
Throughput bound			0.50	1.00	1.00	0.50

Figure 5.17
CPE performance for different degrees of k × 1 loop unrolling. Only integer addition improves. Integer multiplication and floating-point operations are limited by their latency bounds.



We see that the CPE for integer addition improves, achieving the latency bound of 1.00. This result can be attributed to the benefits of reducing loop overhead operations. By reducing the number of overhead operations relative to the number of additions required to compute the vector sum, we can reach the point where the 1-cycle latency of integer addition becomes the performance-limiting factor. On the other hand, none of the other cases improve—they are already at their latency bounds. Figure 5.17 shows CPE measurements when unrolling the loop by up to a factor of 10. We see that the trends we observed for unrolling by 2 and 3 continue—none go below their latency bounds.

To understand why k × 1 unrolling cannot improve performance beyond the latency bound, let us examine the machine-level code for the inner loop of `combine5`, having k = 2. The following code gets generated when type `data_t` is `double`

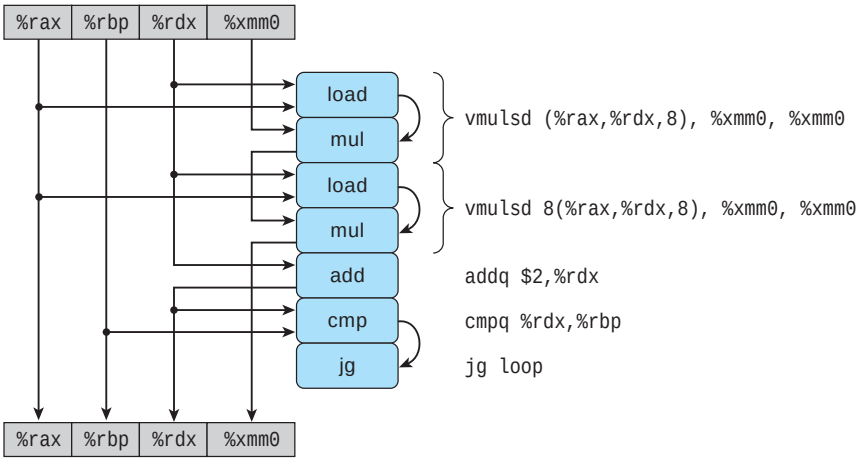
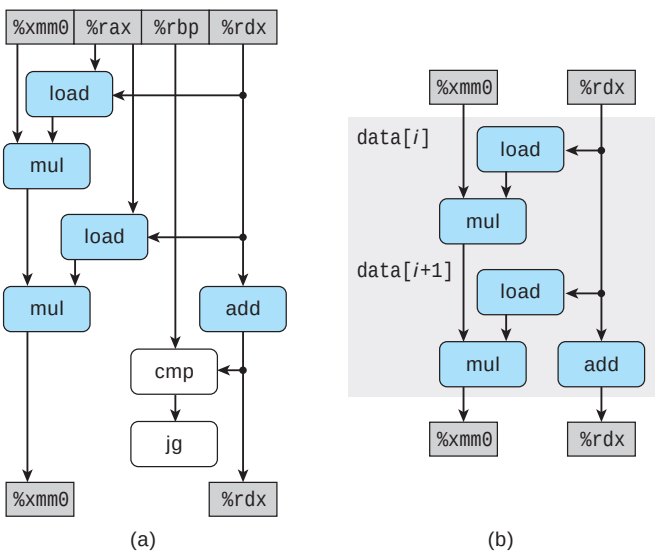


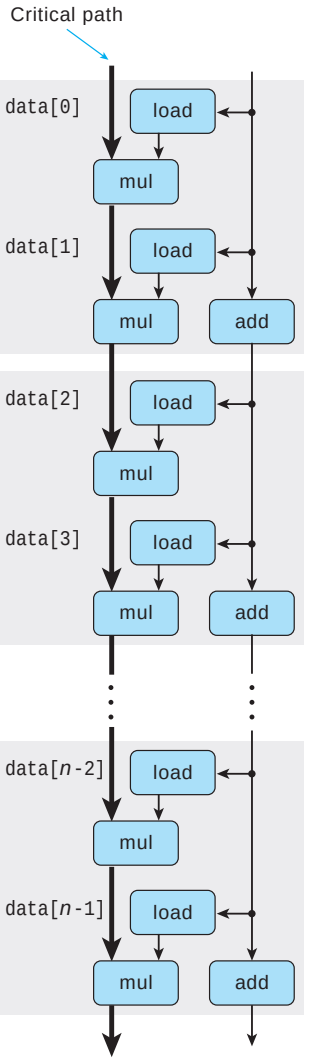
Figure 5.18 Graphical representation of inner-loop code for combine5. Ea it rati a t vmulsd i tr ti , a i i tra lat i t a l a a a l rati .

Figure 5.19
Abstracting combine5 operations as a data-flow graph. W r arra , i li , a a tra t t r r tati o o Fi r 5.18 t t ata i t i it rati (a). W t at a it rati t r r t ltili ati i ( ).



the second to add data[i+1] to acc. Figure 5.18 shows a graphical representation of this code. The vmulsd instructions each get translated into two operations: one to load an array element from memory and one to multiply this value by the accumulated value. We see here that register %xmm0 gets read and written twice in each execution of the loop. We can rearrange, simplify, and abstract this graph, following the process shown in Figure 5.19(a), to obtain the template shown in Figure 5.19(b). We then replicate this template n/2 times to show the computation for a vector of length n, obtaining the data-flow representation

Figure 5.20
Data-flow representation of combine5 operating on a vector of length n. E t o t l oo a r ll \ o a a t r o o 2, t r ar tll n l o rati o al o t rtial at .



shown in Figure 5.20. We see here that there is still a critical path of n l operations in this graph—there are half as many iterations, but each iteration has two multiplication operations in sequence. Since the critical path was the limiting factor for the performance of the code without loop unrolling, it remains so with k × 1 loop unrolling.

Aside G tti t o i rt o r o ll l oo

Loop unrolling can easily be performed by a compiler. Many compilers do this as part of their collection of optimizations. GCC will perform some forms of loop unrolling when invoked with optimization level 3 or higher.

5.9

```
/* 2 x 2 loop unrolling */
void combine6(vec_ptr v, data_t *dest)
{
    long i;
    long length = vec_length(v);
    long limit = length-1;
    data_t *data = get_vec_start(v);
    data_t acc0 = IDENT;
    data_t acc1 = IDENT;

    /* Combine 2 elements at a time */
    for (i = 0; i < limit; i+=2) {
        acc0 = acc0 OP data[i];
        acc1 = acc1 OP data[i+1];
    }

    /* Finish any remaining elements */
    for (; i < length; i++) {
        acc0 = acc0 OP data[i];
    }
    *dest = acc0 OP acc1;
}
```

Figure 5.21 Applying 2 × 2 loop unrolling. B ai tai i lti l a lat r, t i a r a a tt r t lti l ti al t a t r i li i a a lti .

			Integer		Floating point	
Function	Page	Method	+	*	+	*
`combine4`	551	Accumulate in temporary	1.27	3.01	3.01	5.01
`combine5`	568	2 × 1 unrolling	1.01	3.01	3.01	5.01
`combine6`	573	2 × 2 unrolling	0.81	1.51	1.51	2.51
Latency bound			1.00	3.00	3.00	5.00
Throughput bound			0.50	1.00	1.00	0.50

We see that we have improved the performance for all cases, with integer product, floating-point addition, and floating-point multiplication improving by a factor of around 2, and integer addition improving somewhat as well. Most significantly, we have broken through the barrier imposed by the latency bound. The processor no longer needs to delay the start of one sum or product operation until the previous one has completed.

To understand the performance of `combine6`, we start with the code and operation sequence shown in Figure 5.22. We can derive a template showing the

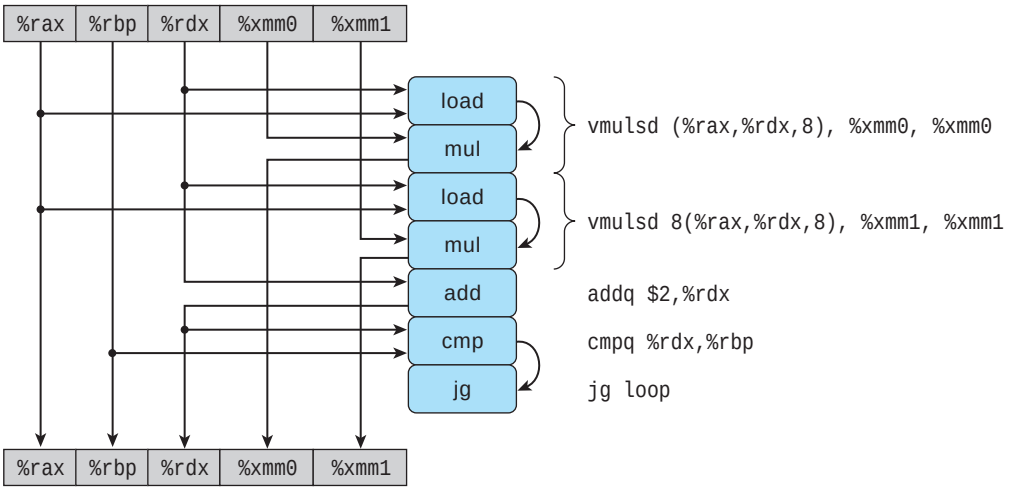


Figure 5.22 Graphical representation of inner-loop code for `combine6`. Each iteration of the `vmulsd` instructions, as translated into load and arithmetic operations.

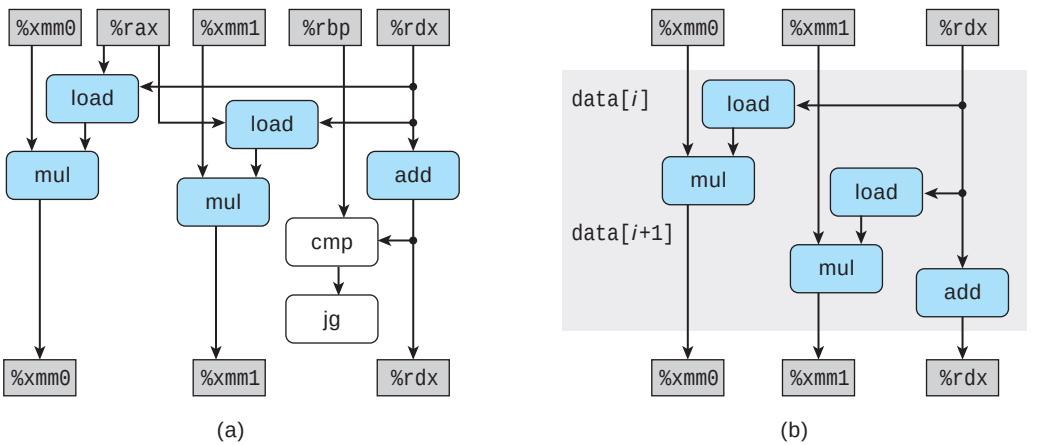
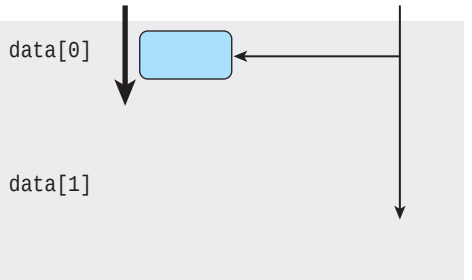


Figure 5.23 Abstracting `combine6` operations as a data-flow graph. We organize the data-flow representation of Figure 5.22 to show the data flow for one iteration (a). We then show how the operations for two iterations (b).

data dependencies between iterations through the process shown in Figure 5.23. As with `combine5`, the inner loop contains two `vmulsd` operations, but these instructions translate into load operations that read and write separate registers, with no data dependency between them (Figure 5.23(b)). We then replicate this template `n/2` times (Figure 5.24), modeling the execution of the function on a vector of length `n`. We see that we now have two critical paths, one corresponding to computing the product of even-numbered elements (program value `acc0`) and

Figure 5.24

Data-flow representation of `combine6` operating on a vector of length n. W o a t o rti al at , a o tai i n/2 o rati o .



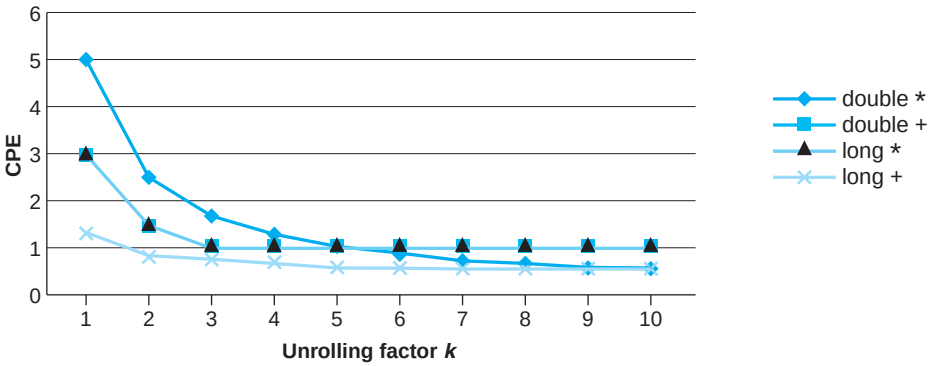


Figure 5.25 CPE performance of k × k loop unrolling. All t CPE i r t t i tra r ati , a i i ar r at t ir t r t .

achieve nearly the throughput bounds for all cases. Integer addition achieves a CPE of 0.54 with k = 7, close to the throughput bound of 0.50 caused by the two load units. Integer multiplication and floating-point addition achieve CPEs of 1.01 when k ≥ 3, approaching the throughput bound of 1.00 set by their functional units. Floating-point multiplication achieves a CPE of 0.51 for k ≥ 10, approaching the throughput bound of 0.50 set by the two floating-point multipliers and the two load units. It is worth noting that our code is able to achieve nearly twice the throughput with floating-point multiplication as it can with floating-point addition, even though multiplication is a more complex operation.

In general, a program can achieve the throughput bound for an operation only when it can keep the pipelines filled for all of the functional units capable of performing that operation. For an operation with latency L and capacity C, this requires an unrolling factor k ≥ C · L. For example, floating-point multiplication has C = 2 and L = 5, necessitating an unrolling factor of k ≥ 10. Floating-point addition has C = 1 and L = 3, achieving maximum throughput with k ≥ 3.

In performing the k × k unrolling transformation, we must consider whether it preserves the functionality of the original function. We have seen in Chapter 2 that two's-complement arithmetic is commutative and associative, even when overflow occurs. Hence, for an integer data type, the result computed by `combine6` will be identical to that computed by `combine5` under all possible conditions. Thus, an optimizing compiler could potentially convert the code shown in `combine4` first to a two-way unrolled variant of `combine5` by loop unrolling, and then to that of `combine6` by introducing parallelism. Some compilers do either this or similar transformations to improve performance for integer data.

On the other hand, floating-point multiplication and addition are not associative. Thus, `combine5` and `combine6` could produce different results due to rounding or overflow. Imagine, for example, a product computation in which all of the elements with even indices are numbers with very large absolute values, while those with odd indices are very close to 0.0. In such a case, product PE_n might overflow, or PO_n might underflow, even though computing product P_n pro-

ceeds normally. In most real-life applications, however, such patterns are unlikely. Since most physical phenomena are continuous, numerical data tend to be reasonably smooth and well behaved. Even when there are discontinuities, they do not generally cause periodic patterns that lead to a condition such as that sketched earlier. It is unlikely that multiplying the elements in strict order gives fundamentally better accuracy than does multiplying two groups independently and then multiplying those products together. For most applications, achieving a performance gain of 2× outweighs the risk of generating different results for strange data patterns. Nevertheless, a program developer should check with potential users to see if there are particular conditions that may cause the revised algorithm to be unacceptable. Most compilers do not attempt such transformations with floating-point code, since they have no way to judge the risks of introducing transformations that can change the program behavior, no matter how small.

5.9.2 Reassociation Transformation

We now explore another way to break the sequential dependencies and thereby improve performance beyond the latency bound. We saw that the k × 1 loop unrolling of `combine5` did not change the set of operations performed in combining the vector elements to form their sum or product. By a very small change in the code, however, we can fundamentally change the way the combining is performed, and also greatly increase the program performance.

Figure 5.26 shows a function `combine7` that differs from the unrolled code of `combine5` (Figure 5.16) only in the way the elements are combined in the inner loop. In `combine5`, the combining is performed by the statement

```
12      acc = (acc OP data[i]) OP data[i+1];
```

while in `combine7` it is performed by the statement

```
12      acc = acc OP (data[i] OP data[i+1]);
```

differing only in how two parentheses are placed. We call this a reassociation transformation, because the parentheses shift the order in which the vector elements are combined with the accumulated value `acc`, yielding a form of loop unrolling we refer to as "2 × 1a."

To an untrained eye, the two statements may seem essentially the same, but when we measure the CPE, we get a surprising result:

			Integer		Floating point	
Function	Page	Method	+	*	+	*
`combine4`	551	Accumulate in temporary	1.27	3.01	3.01	5.01
`combine5`	568	2 × 1 unrolling	1.01	3.01	3.01	5.01
`combine6`	573	2 × 2 unrolling	0.81	1.51	1.51	2.51
`combine7`	578	2 × 1a unrolling	1.01	1.51	1.51	2.51
Latency bound			1.00	3.00	3.00	5.00
Throughput bound			0.50	1.00	1.00	0.50

```
/* 2 x 1a loop unrolling */
void combine7(vec_ptr v, data_t *dest)
{
    long i;
    long length = vec_length(v);
    long limit = length-1;
    data_t *data = get_vec_start(v);
    data_t acc = IDENT;

    /* Combine 2 elements at a time */
    for (i = 0; i < limit; i+=2) {
        acc = acc OP (data[i] OP data[i+1]);
    }

    /* Finish any remaining elements */
    for (; i < length; i++) {
        acc = acc OP data[i];
    }
    *dest = acc;
}
```

Figure 5.26 Applying 2 × 1a unrolling. B r a ati t art t ti, t i a r a i r a t r o o rati t at a r r \ i arall l.

The integer addition case matches the performance of k × 1 unrolling (combine5), while the other three cases match the performance of the versions with parallel accumulators (combine6), doubling the performance relative to k × 1 unrolling. These cases have broken through the barrier imposed by the latency bound.

Figure 5.27 illustrates how the code for the inner loop of combine7 (for the case of multiplication as the combining operation and double as data type) gets decoded into operations and the resulting data dependencies. We see that the l a operations resulting from the vmovsd and the first vmulsd instructions load vector elements i and i + 1 from memory, and the first l operation multiplies them together. The second l operation then multiples this result by the accumulated value acc. Figure 5.28(a) shows how we rearrange, refine, and abstract the operations of Figure 5.27 to get a template representing the data dependencies for one iteration (Figure 5.28(b)). As with the templates for combine5 and combine7, we have two l a and two l operations, but only one of the l operations forms a data-dependency chain between loop registers. When we then replicate this template n/2 times to show the computations performed in multiplying n vector elements (Figure 5.29), we see that we only have n/2 operations along the critical path. The first multiplication within each iteration can be performed without waiting for the accumulated value from the previous iteration. Thus, we reduce the minimum possible CPE by a factor of around 2.

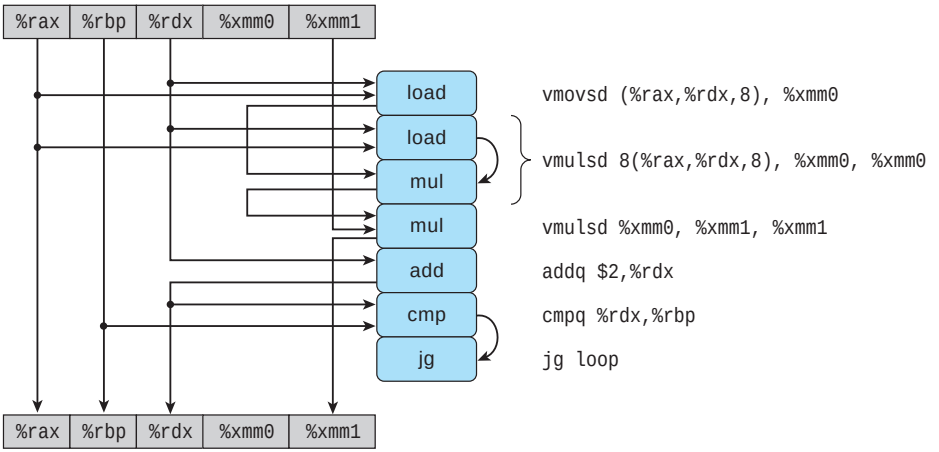


Figure 5.27 Graphical representation of inner-loop code for combine7. Ea it rati t o i t i lar rati a r combine5 r combine6, t it i r t ata i .

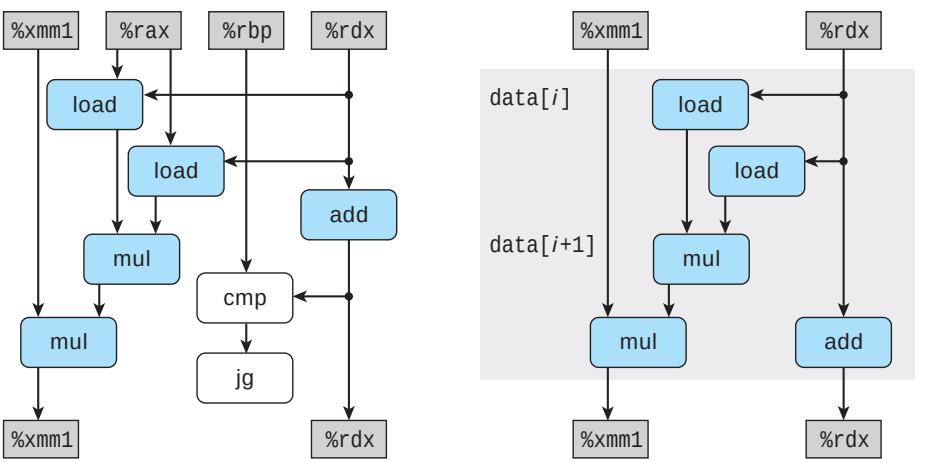


Figure 5.28 Abstracting combine7 operations as a data-flow graph. W r arra , i li , a a tra t t r r tati o o Fi r 5.27 t o t ata i t i it rati . T r l rati lti li t 2- t r l t it a t r, i t l r lti li t r lt l aria l acc.

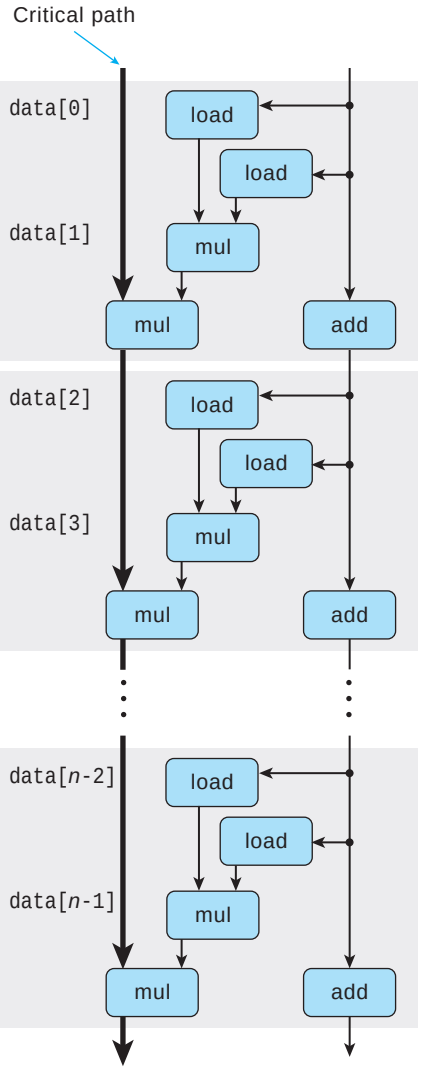


Figure 5.29
Data-flow representation of combine7 operating on a vector of length n. W a a i l rti al at , t it o tai o l n/2 o rati o .

Figure 5.30 demonstrates the effect of applying the reassociation transformation to achieve what we refer to as k × 1a loop unrolling for values up to k = 10. We can see that this transformation yields performance results similar to what is achieved by maintaining k separate accumulators with k × k unrolling. In all cases, we come close to the throughput bounds imposed by the functional units.

In performing the reassociation transformation, we once again change the order in which the vector elements will be combined together. For integer addition and multiplication, the fact that these operations are associative implies that this reordering will have no effect on the result. For the floating-point cases, we must once again assess whether this reassociation is likely to significantly affect

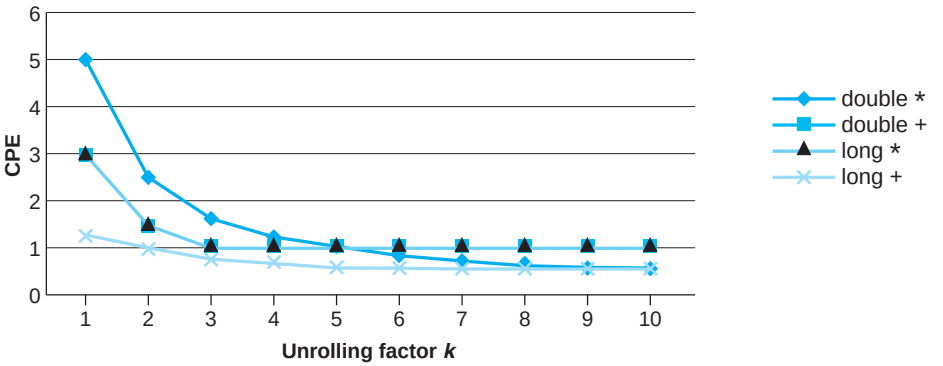


Figure 5.30 CPE performance for k × 1a loop unrolling. All t CPE i r t t i tra r ati , arl a r a i t irt r t \ .

the outcome. We would argue that the difference would be immaterial for most applications.

In summary, a reassociation transformation can reduce the number of operations along the critical path in a computation, resulting in better performance by better utilizing the multiple functional units and their pipelining capabilities. Most compilers will not attempt any reassociations of floating-point operations, since these operations are not guaranteed to be associative. Current versions of GCC do perform reassociations of integer operations, but not always with good effects. In general, we have found that unrolling a loop and accumulating multiple values in parallel is a more reliable way to achieve improved program performance.

Practice Problem 5.8 (solution page 612)

Consider the following function for computing the product of an array of n double-precision numbers. We have unrolled the loop by a factor of 3.

```
double aprod(double a[], long n)
{
    long i;
    double x, y, z;
    double r = 1;
    for (i = 0; i < n-2; i+= 3) {
        x = a[i]; y = a[i+1]; z = a[i+2];
        r = r * x * y * z; /* Product computation */
    }
    for (; i < n; i++)
        r *= a[i];
    return r;
}
```

For the line labeled "`Product computation`," we can use parentheses to create five different associations of the computation, as follows:

```
r = ((r * x) * y) * z; /* A1 */
r = (r * (x * y)) * z; /* A2 */
r = r * ((x * y) * z); /* A3 */
r = r * (x * (y * z)); /* A4 */
r = (r * x) * (y * z); /* A5 */
```

Assume we run these functions on a machine where floating-point multiplication has a latency of 5 clock cycles. Determine the lower bound on the CPE set by the data dependencies of the multiplication. (Hint: It helps to draw a data-flow representation of how `r` is computed on every iteration.)

Web Aside OPT:SIMD Aiir atr arall lit tr ti

As described in Section 3.1, Intel introduced the SSE instructions in 1999, where SSE is the acronym for "streaming SIMD extensions" and, in turn, SIMD (pronounced "sim-dee") is the acronym for "single instruction, multiple data." The SSE capability has gone through multiple generations, with more recent versions being named advanced vector extensions, or AVX. The SIMD execution model involves operating on entire vectors of data within single instructions. These vectors are held in a special set of vector registers, named `%ymm0`–`%ymm15`. Current AVX vector registers are 32 bytes long, and therefore each can hold eight 32-bit numbers or four 64-bit numbers, where the numbers can be either integer or floating-point values. AVX instructions can then perform vector operations on these registers, such as adding or multiplying eight or four sets of values in parallel. For example, if YMM register `%ymm0` contains eight single-precision floating-point numbers, which we denote $a_0, \ldots, a_7$, and `%rcx` contains the memory address of a sequence of eight single-precision floating-point numbers, which we denote $b_0, \ldots, b_7$, then the instruction

```
vmulps  (%rcs), %ymm0, %ymm1
```

will read the eight values from memory and perform eight multiplications in parallel, computing $a_i \leftarrow a_i \cdot b_i$, for $0 \le i \le 7$ and storing the resulting eight products in vector register `%ymm1`. We see that a single instruction is able to generate a computation over multiple data values, hence the term "SIMD."

GCC supports extensions to the C language that let programmers express a program in terms of vector operations that can be compiled into the vector instructions of AVX (as well as code based on the earlier SSE instructions). This coding style is preferable to writing code directly in assembly language, since GCC can also generate code for the vector instructions found on other processors.

Using a combination of GCC instructions, loop unrolling, and multiple accumulators, we are able to achieve the following performance for our combining functions:

Web Aside OPT:SIMD Arithmetic with parallel instructions (continued)

		Integer				Floating point			
		`int`		`long`		`int`		`long`	
Method		+	*	+	*	+	*	+	*
Scalar 10 × 10	×	0.54	1.01	0.55	1.00	1.01	0.51		

Tf 1.0.m 0m11.47 55 -0 9 0 Tf

By using multiple optimizations, we have been able to achieve CPEs close to the throughput bounds of 0.50 and 1.00, limited only by the capacities of the functional units. These represent 10–20× improvements on the original code. This has all been done using ordinary C code and a standard compiler. Rewriting the code to take advantage of the newer SIMD instructions yields additional performance gains of nearly 4× or 8×. For example, for single-precision multiplication, the CPE drops from the original value of 11.14 down to 0.06, an overall performance gain of over 180×. This example demonstrates that modern processors have considerable amounts of computing power, but we may need to coax this power out of them by writing our programs in very stylized ways.

5.11 Some Limiting Factors

We have seen that the critical path in a data-flow graph representation of a program indicates a fundamental lower bound on the time required to execute a program. That is, if there is some chain of data dependencies in a program where the sum of all of the latencies along that chain equals T, then the program will require at least T cycles to execute.

We have also seen that the throughput bounds of the functional units also impose a lower bound on the execution time for a program. That is, assume that a program requires a total of N computations of some operation, that the microprocessor has C functional units capable of performing that operation, and that these units have an issue time of I. Then the program will require at least $N \cdot I / C$ cycles to execute.

In this section, we will consider some other factors that limit the performance of programs on actual machines.

5.11.1 Register Spilling

The benefits of loop parallelism are limited by the ability to express the computation in assembly code. If a program has a degree of parallelism P that exceeds the number of available registers, then the compiler will resort to *spilling*, storing some of the temporary values in memory, typically by allocating space on the run-time stack. As an example, the following measurements compare the result of extending the multiple accumulator scheme of `combine6` to the cases of `k = 10` and `k = 20`:

			Integer		Floating point	
Function	Page	Method	+	*	+	*
`combine6`	573					
		10 × 10 unrolling	0.55	1.00	1.01	0.52
		20 × 20 unrolling	0.83	1.03	1.02	0.68
Throughput bound			0.50	1.00	1.00	0.50

We can see that none of the CPEs improve with this increased unrolling, and some even get worse. Modern x86-64 processors have 16 integer registers and can make use of the 16 YMM registers to store floating-point data. Once the number of loop variables exceeds the number of available registers, the program must allocate some on the stack.

As an example, the following snippet of code shows how accumulator `acc0` is updated in the inner loop of the code with 10 × 10 unrolling:

```
Updating of accumulator acc0 in 10 x 10 urolling
vmulsd  (%rdx), %xmm0, %xmm0     acc0 *= data[i]
```

We can see that the accumulator is kept in register `%xmm0`, and so the program can simply read `data[i]` from memory and multiply it by this register.

The comparable part of the code for 20 × 20 unrolling has a much different form:

```
Updating of accumulator acc0 in 20 x 20 unrolling
vmovsd  40(%rsp), %xmm0
vmulsd  (%rdx), %xmm0, %xmm0
vmovsd  %xmm0, 40(%rsp)
```

The accumulator is kept as a local variable on the stack, at offset 40 from the stack pointer. The program must read both its value and the value of `data[i]` from memory, multiply them, and store the result back to memory.

Once a compiler must resort to register spilling, any advantage of maintaining multiple accumulators will most likely be lost. Fortunately, x86-64 has enough registers that most loops will become throughput limited before this occurs.

5.11.2 Branch Prediction and Misprediction Penalties

We demonstrated via experiments in Section 3.6.6 that a conditional branch can incur a significant misprediction penalty when the branch prediction logic does not correctly anticipate whether or not a branch will be taken. Now that we have learned something about how processors operate, we can understand where this penalty arises.

Modern processors work well ahead of the currently executing instructions, reading new instructions from memory and decoding them to determine what operations to perform on what operands. This instruction pipelining works well as long as the instructions follow in a simple sequence. When a branch is encountered, the processor must guess which way the branch will go. For the case of a conditional jump, this means predicting whether or not the branch will be taken. For an instruction such as an indirect jump (as we saw in the code to jump to an address specified by a jump table entry) or a procedure return, this means predicting the target address. In this discussion, we focus on conditional branches.

In a processor that employs speculative execution, the processor begins executing the instructions at the predicted branch target. It does this in a way that avoids modifying any actual register or memory locations until the actual outcome has been determined. If the prediction is correct, the processor can then

"commit" the results of the speculatively executed instructions by storing them in registers or memory. If the prediction is incorrect, the processor must discard all of the speculatively executed results and restart the instruction fetch process at the correct location. The misprediction penalty is incurred in doing this, because the instruction pipeline must be refilled before useful results are generated.

We saw in Section 3.6.6 that recent versions of x86 processors, including all processors capable of executing x86-64 programs, have conditional move instructions. GCC can generate code that uses these instructions when compiling conditional statements and expressions, rather than the more traditional realizations based on conditional transfers of control. The basic idea for translating into conditional moves is to compute the values along both branches of a conditional expression or statement and then use conditional moves to select the desired value. We saw in Section 4.5.7 that conditional move instructions can be implemented as part of the pipelined processing of ordinary instructions. There is no need to guess whether or not the condition will hold, and hence no penalty for guessing incorrectly.

How, then, can a C programmer make sure that branch misprediction penalties do not hamper a program's efficiency? Given the 19-cycle misprediction penalty we measured for the reference machine, the stakes are very high. There is no simple answer to this question, but the following general principles apply.

D N t B O rl C r a t Pr i ta l Bra

We have seen that the effect of a mispredicted branch can be very high, but that does not mean that all program branches will slow a program down. In fact, the branch prediction logic found in modern processors is very good at discerning regular patterns and long-term trends for the different branch instructions. For example, the loop-closing branches in our combining routines would typically be predicted as being taken, and hence would only incur a misprediction penalty on the last time around.

As another example, consider the results we observed when shifting from `combine2` to `combine3`, when we took the function `get_vec_element` out of the inner loop of the function, as is reproduced below:

			Integer		Floating point	
Function	Page	Method	+	*	+	*
`combine2`	545	Move `vec_length`	7.02	9.03	9.02	11.03
`combine3`	549	Direct data access	7.17	9.02	9.02	11.03

The CPE did not improve, even though the transformation eliminated two conditionals on each iteration that check whether the vector index is within bounds. For this function, the checks always succeed, and hence they are highly predictable.

As a way to measure the performance impact of bounds checking, consider the following combining code, where we have modified the inner loop of `combine4` by replacing the access to the data element with the result of performing an inline substitution of the code for `get_vec_element`. We will call this new version

combine4b. This code performs bounds checking and also references the vector elements through the vector data structure.

```
1   /* Include bounds check in loop */
2   void combine4b(vec_ptr v, data_t *dest)
3   {
4       long i;
5       long length = vec_length(v);
6       data_t acc = IDENT;
7
8       for (i = 0; i < length; i++) {
9           if (i >= 0 && i < v->len) {
10              acc = acc OP v->data[i];
11          }
12      }
13      *dest = acc;
14  }
```

We can then directly compare the CPE for the functions with and without bounds checking:

			Integer		Floating point	
Function	Page	Method	+	*	+	*
combine4	551	No bounds checking	1.27	3.01	3.01	5.01
combine4b	551	Bounds checking	2.02	3.01	3.01	5.01

The version with bounds checking is slightly slower for the case of integer addition, but it achieves the same performance for the other three cases. The performance of these cases is limited by the latencies of their respective combining operations. The additional computation required to perform bounds checking can take place in parallel with the combining operations. The processor is able to predict the outcomes of these branches, and so none of this evaluation has much effect on the fetching and processing of the instructions that form the critical path in the program execution.

Wrt C S ta l rl l tat t C ti al M

Branch prediction is only reliable for regular patterns. Many tests in a program are completely unpredictable, dependent on arbitrary features of the data, such as whether a number is negative or positive. For these, the branch prediction logic will do very poorly. For inherently unpredictable cases, program performance can be greatly enhanced if the compiler is able to generate code using conditional data transfers rather than conditional control transfers. This cannot be controlled directly by the C programmer, but some ways of expressing conditional behavior can be more directly translated into conditional moves than others.

We have found that GCC is able to generate conditional moves for code written in a more "functional" style, where we use conditional operations to compute

values and then update the program state with these values, as opposed to a more "imperative" style, where we use conditionals to selectively update program state.

There are no strict rules for these two styles, and so we illustrate with an example. Suppose we are given two arrays of integers `a` and `b`, and at each position `i`, we want to set `a[i]` to the minimum of `a[i]` and `b[i]`, and `b[i]` to the maximum.

An imperative style of implementing this function is to check at each position `i` and swap the two elements if they are out of order:

```
1   /* Rearrange two vectors so that for each i, b[i] >= a[i] */
2   void minmax1(long a[], long b[], long n) {
3       long i;
4       for (i = 0; i < n; i++) {
5           if (a[i] > b[i]) {
6               long t = a[i];
7               a[i] = b[i];
8               b[i] = t;
9           }
10      }
11  }
```

Our measurements for this function show a CPE of around 13.5 for random data and 2.5–3.5 for predictable data, an indication of a misprediction penalty of around 20 cycles.

A functional style of implementing this function is to compute the minimum and maximum values at each position `i` and then assign these values to `a[i]` and `b[i]`, respectively:

```
1   /* Rearrange two vectors so that for each i, b[i] >= a[i] */
2   void minmax2(long a[], long b[], long n) {
3       long i;
4       for (i = 0; i < n; i++) {
5           long min = a[i] < b[i] ? a[i] : b[i];
6           long max = a[i] < b[i] ? b[i] : a[i];
7           a[i] = min;
8           b[i] = max;
9       }
10  }
```

Our measurements for this function show a CPE of around 4.0 regardless of whether the data are arbitrary or predictable. (We also examined the generated assembly code to make sure that it indeed uses conditional moves.)

As discussed in Section 3.6.6, not all conditional behavior can be implemented with conditional data transfers, and so there are inevitably cases where programmers cannot avoid writing code that will lead to conditional branches for which the processor will do poorly with its branch prediction. But, as we have shown, a little cleverness on the part of the programmer can sometimes make code more amenable to translation into conditional data transfers. This requires some amount

of experimentation, writing different versions of the function and then examining the generated assembly code and measuring performance.

Practice Problem 5.9 (solution page 612)

The traditional implementation of the merge step of mergesort requires three loops [98]:

```
1   void merge(long src1[], long src2[], long dest[], long n) {
2       long i1 = 0;
3       long i2 = 0;
4       long id = 0;
5       while (i1 < n && i2 < n) {
6           if (src1[i1] < src2[i2])
7               dest[id++] = src1[i1++];
8           else
9               dest[id++] = src2[i2++];
10      }
11      while (i1 < n)
12          dest[id++] = src1[i1++];
13      while (i2 < n)
14          dest[id++] = src2[i2++];
15  }
```

The branches caused by comparing variables `i1` and `i2` to `n` have good prediction performance—the only mispredictions occur when they first become false. The comparison between values `src1[i1]` and `src2[i2]` (line 6), on the other hand, is highly unpredictable for typical data. This comparison controls a conditional branch, yielding a CPE (where the number of elements is 2`n`) of around 15.0 when run on random data.

Rewrite the code so that the effect of the conditional statement in the first loop (lines 6–9) can be implemented with a conditional move.

5.12 Understanding Memory Performance

All of the code we have written thus far, and all the tests we have run, access relatively small amounts of memory. For example, the combining routines were measured over vectors of length less than 1,000 elements, requiring no more than 8,000 bytes of data. All modern processors contain one or more *cache* memories to provide fast access to such small amounts of memory. In this section, we will further investigate the performance of programs that involve load (reading from memory into registers) and store (writing from registers to memory) operations, considering only the cases where all data are held in cache. In Chapter 6, we go into much more detail about how caches work, their performance characteristics, and how to write code that makes best use of caches.

As Figure 5.11 shows, modern processors have dedicated functional units to perform load and store operations, and these units have internal buffers to hold sets of outstanding requests for memory operations. For example, our reference machine has two load units, each of which can hold up to 72 pending read requests. It has a single store unit with a store buffer containing up to 42 write requests. Each of these units can initiate 1 operation every clock cycle.

5.12.1 Load Performance

The performance of a program containing load operations depends on both the pipelining capability and the latency of the load unit. In our experiments with combining operations using our reference machine, we saw that the CPE never got below 0.50 for any combination of data type and combining operation, except when using SIMD operations. One factor limiting the CPE for our examples is that they all require reading one value from memory for each element computed. With two load units, each able to initiate at most 1 load operation every clock cycle, the CPE cannot be less than 0.50. For applications where we must load `k` values for every element computed, we can never achieve a CPE lower than `k`/2 (see, for example, Problem 5.15).

In our examples so far, we have not seen any performance effects due to the latency of load operations. The addresses for our load operations depended only on the loop index `i`, and so the load operations did not form part of a performance-limiting critical path.

To determine the latency of the load operation on a machine, we can set up a computation with a sequence of load operations, where the outcome of one determines the address for the next. As an example, consider the function `list_len` in Figure 5.31, which computes the length of a linked list. In the loop of this function, each successive value of variable `ls` depends on the value read by the pointer reference `ls->next`. Our measurements show that function `list_len` has

```
1   typedef struct ELE {
2       struct ELE *next;
3       long data;
4   } list_ele, *list_ptr;
5
6   long list_len(list_ptr ls) {
7       long len = 0;
8       while (ls) {
9         len++;
10        ls = ls->next;
11      }
12      return len;
13  }
```

Figure 5.31 Linked list function. It [illegible] latency bound and load operation [illegible].

a CPE of 4.00, which we claim is a direct indication of the latency of the load operation. To see this, consider the assembly code for the loop:

```
    Inner loop of list_len
    ls in %rdi, len in %rax
1   .L3:                        loop:
2     addq    $1, %rax            Increment len
3     movq    (%rdi), %rdi        ls = ls->next
4     testq   %rdi, %rdi          Test ls
5     jne     .L3                 If nonnull, goto loop
```

The `movq` instruction on line 3 forms the critical bottleneck in this loop. Each successive value of register `%rdi` depends on the result of a load operation having the value in `%rdi` as its address. Thus, the load operation for one iteration cannot begin until the one for the previous iteration has completed. The CPE of 4.00 for this function is determined by the latency of the load operation. Indeed, this measurement matches the documented access time of 4 cycles for the reference machine's L1 cache, as is discussed in Section 6.4.

5.12.2 Store Performance

In all of our examples thus far, we analyzed only functions that reference memory mostly with load operations, reading from a memory location into a register. Its counterpart, the store operation, writes a register value to memory. The performance of this operation, particularly in relation to its interactions with load operations, involves several subtle issues.

As with the load operation, in most cases, the store operation can operate in a fully pipelined mode, beginning a new store on every cycle. For example, consider the function shown in Figure 5.32 that sets the elements of an array `dest` of length `n` to zero. Our measurements show a CPE of 1.0. This is the best we can achieve on a machine with a single store functional unit.

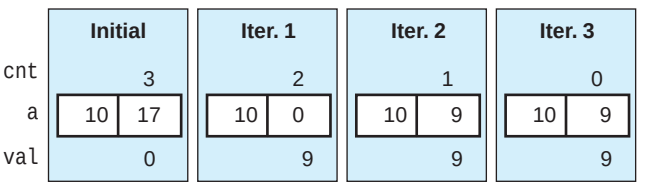
Unlike the other operations we have considered so far, the store operation does not affect any register values. Thus, by their very nature, a series of store operations cannot create a data dependency. Only a load operation is affected by the result of a store operation, since only a load can read back the memory value that has been written by the store. The function `write_read` shown in Figure 5.33

```
1   /* Set elements of array to 0 */
2   void clear_array(long *dest, long n) {
3       long i;
4       for (i = 0; i < n; i++)
5           dest[i] = 0;
6   }
```

Figure 5.32 Function to set array elements to 0. T i a i a CPE 1.0.

```
1   /* Write to dest, read from src */
2   void write_read(long *src, long *dst, long n)
3   {
4       long cnt = n;
5       long val = 0;
6
7       while (cnt) {
8           *dst = val;
9           val = (*src)+1;
10          cnt--;
11      }
12  }
```

Example A: `write_read(&a[0],&a[1],3)`



Example B: `write_read(&a[0],&a[0],3)`

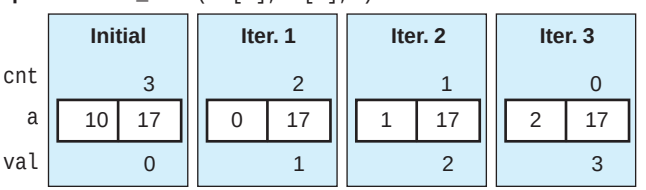


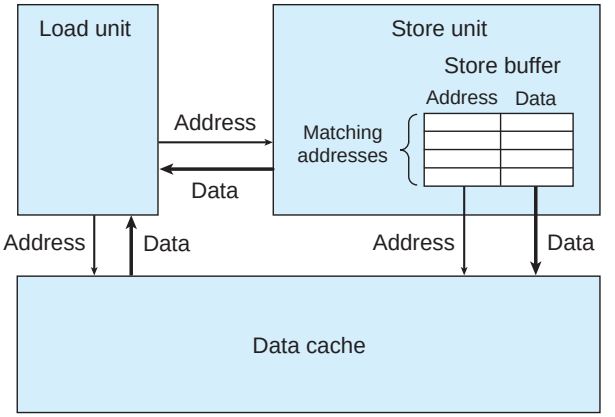
Figure 5.33 Code to write and read memory locations, along with illustrative executions. T i ti i li t t i t ra ti t t r a l a ar t src a dest ar al.

illustrates the potential interactions between loads and stores. This figure also shows two example executions of this function, when it is called for a two-element array `a`, with initial contents −10 and 17, and with argument `cnt` equal to 3. These executions illustrate some subtleties of the load and store operations.

In Example A of Figure 5.33, argument `src` is a pointer to array element `a[0]`, while `dest` is a pointer to array element `a[1]`. In this case, each load by the pointer reference `*src` will yield the value −10. Hence, after two iterations, the array elements will remain fixed at −10 and −9, respectively. The result of the read from `src` is not affected by the write to `dest`. Measuring this example over a larger number of iterations gives a CPE of 1.3.

In Example B of Figure 5.33, both arguments `src` and `dest` are pointers to array element `a[0]`. In this case, each load by the pointer reference `*src` will yield the value stored by the previous execution of the pointer reference `*dest`.

Figure 5.34
Detail of load and store units. T t r it ai tai a r o \ i rit . T l a \ it t it a \ r it t i t t r it t \ t ta rit /r a \ \ .



As a consequence, a series of ascending values will be stored in this location. In general, if function `write_read` is called with arguments `src` and `dest` pointing to the same memory location, and with argument `cnt` having some value `n` > 0, the net effect is to set the location to `n` − 1. This example illustrates a phenomenon we will call a write/read dependency—the outcome of a memory read depends on a recent memory write. Our performance measurements show that Example B has a CPE of 7.3. The write/read dependency causes a slowdown in the processing of around 6 clock cycles.

To see how the processor can distinguish between these two cases and why one runs slower than the other, we must take a more detailed look at the load and store execution units, as shown in Figure 5.34. The store unit includes a store buffer containing the addresses and data of the store operations that have been issued to the store unit, but have not yet been completed, where completion involves updating the data cache. This buffer is provided so that a series of store operations can be executed without having to wait for each one to update the cache. When a load operation occurs, it must check the entries in the store buffer for matching addresses. If it finds a match (meaning that any of the bytes being written have the same address as any of the bytes being read), it retrieves the corresponding data entry as the result of the load operation.

GCC generates the following code for the inner loop of `write_read`:

```
  Inner loop of write_read
  src in %rdi, dst in %rsi, val in %rax
.L3:                          loop:
  movq    %rax, (%rsi)          Write val to dst
  movq    (%rdi), %rax          t = *src
  addq    $1, %rax              val = t+1
  subq    $1, %rdx              cnt--
  jne     .L3                   If != 0, goto loop
```

Figure 5.35
Graphical representation of inner-loop code for write_read. T fir t movl i tr ti i i t arat rati t t t ata t r .

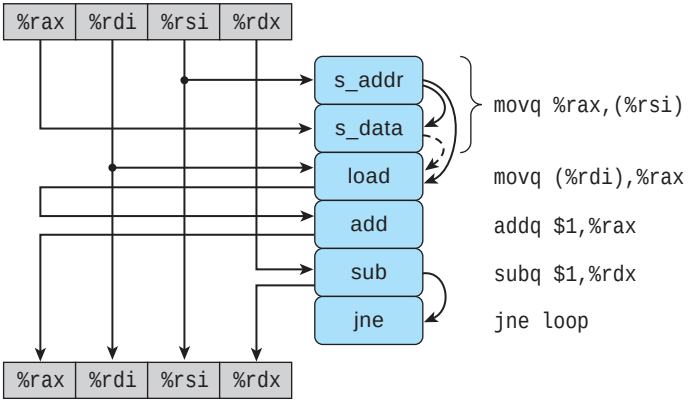


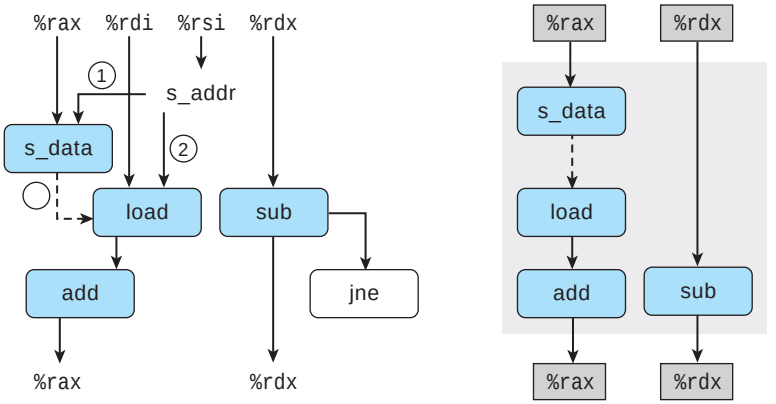
Figure 5.35 shows a data-flow representation of this loop code. The instruction `movq %rax,(%rsi)` is translated into two operations: The _a r instruction computes the address for the store operation, creates an entry in the store buffer, and sets the address field for that entry. The _ ata operation sets the data field for the entry. As we will see, the fact that these two computations are performed independently can be important to program performance. This motivates the separate functional units for these operations in the reference machine.

In addition to the data dependencies between the operations caused by the writing and reading of registers, the arcs on the right of the operators denote a set of implicit dependencies for these operations. In particular, the address computation of the _a r operation must clearly precede the _ ata operation. In addition, the l a operation generated by decoding the instruction `movq (%rdi), %rax` must check the addresses of any pending store operations, creating a data dependency between it and the _a r operation. The figure shows a dashed arc between the _ ata and l a operations. This dependency is conditional: if the two addresses match, the l a operation must wait until the _ ata has deposited its result into the store buffer, but if the two addresses differ, the two operations can proceed independently.

Figure 5.36 illustrates the data dependencies between the operations for the inner loop of `write_read`. In Figure 5.36(a), we have rearranged the operations to allow the dependencies to be seen more clearly. We have labeled the three dependencies involving the load and store operations for special attention. The arc labeled "1" represents the requirement that the store address must be computed before the data can be stored. The arc labeled "2" represents the need for the l a operation to compare its address with that for any pending store operations. Finally, the dashed arc labeled "3" represents the conditional data dependency that arises when the load and store addresses match.

Figure 5.36(b) illustrates what happens when we take away those operations that do not directly affect the flow of data from one iteration to the next. The data-flow graph shows just two chains of dependencies: the one on the left, with data values being stored, loaded, and incremented (only for the case of matching addresses); and the one on the right, decrementing variable `cnt`.

Figure 5.36
Abstracting the operations for write_read. W fir tr arra t rat r Fi r 5.35(a) a t l t rati t at al it rati t al r t ( ).



We can now understand the performance characteristics of function write_read. Figure 5.37 illustrates the data dependencies formed by multiple iterations of its inner loop. For the case of Example A in Figure 5.33, with differing source and destination addresses, the load and store operations can proceed independently, and hence the only critical path is formed by the decrementing of variable cnt, resulting in a CPE bound of 1.0. For the case of Example B with matching source and destination addresses, the data dependency between the ata and l a instructions causes a critical path to form involving data being stored, loaded, and incremented. We found that these three operations in sequence require a total of around 7 clock cycles.

As these two examples show, the implementation of memory operations involves many subtleties. With operations on registers, the processor can determine which instructions will affect which others as they are being decoded into operations. With memory operations, on the other hand, the processor cannot predict which will affect which others until the load and store addresses have been computed. Efficient handling of memory operations is critical to the performance of many programs. The memory subsystem makes use of many optimizations, such as the potential parallelism when operations can proceed independently.

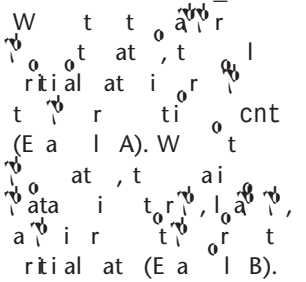
Practice Problem 5.10 (solution page 613)

As another example of code with potential load-store interactions, consider the following function to copy the contents of one array to another:

```
1  void copy_array(long *src, long *dest, long n)
2  {
3      long i;
4      for (i = 0; i < n; i++)
5          dest[i] = src[i];
6  }
```

Figure 5.37

Data-flow representation of function write_read.



Practice Problem 5.11 (solution page 613)

We saw that our measurements of the prefix-sum function `psum1` (Figure 5.1) yield a CPE of 9.00 on a machine where the basic operation to be performed, floating-point addition, has a latency of just 3 clock cycles. Let us try to understand why our function performs so poorly.

The following is the assembly code for the inner loop of the function:

```
    Inner loop of psum1
    a in %rdi, i in %rax, cnt in %rdx
1   .L5:                                          loop:
2     vmovss  -4(%rsi,%rax,4), %xmm0                Get p[i-1]
3     vaddss  (%rdi,%rax,4), %xmm0, %xmm0           Add a[i]
4     vmovss  %xmm0, (%rsi,%rax,4)                  Store at p[i]
5     addq    $1, %rax                              Increment i
6     cmpq    %rdx, %rax                            Compare i:cnt
7     jne     .L5                                   If !=, goto loop
```

Perform an analysis similar to those shown for `combine3` (Figure 5.14) and for `write_read` (Figure 5.36) to diagram the data dependencies created by this loop, and hence the critical path that forms as the computation proceeds. Explain why the CPE is so high.

Practice Problem 5.12 (solution page 613)

Rewrite the code for `psum1` (Figure 5.1) so that it does not need to repeatedly retrieve the value of `p[i]` from memory. You do not need to use loop unrolling. We measured the resulting code to have a CPE of 3.00, limited by the latency of floating-point addition.

5.13 Life in the Real World: Performance Improvement Techniques

Although we have only considered a limited set of applications, we can draw important lessons on how to write efficient code. We have described a number of basic strategies for optimizing program performance:

High-level design. Choose appropriate algorithms and data structures for the problem at hand. Be especially vigilant to avoid algorithms or coding techniques that yield asymptotically poor performance.

Basic coding principles. Avoid optimization blockers so that a compiler can generate efficient code.

- Eliminate excessive function calls. Move computations out of loops when possible. Consider selective compromises of program modularity to gain greater efficiency.

- Eliminate unnecessary memory references. Introduce temporary variables to hold intermediate results. Store a result in an array or global variable only when the final value has been computed.

Low-level optimizations. Structure code to take advantage of the hardware capabilities.

- Unroll loops to reduce overhead and to enable further optimizations.
- Find ways to increase instruction-level parallelism by techniques such as multiple accumulators and reassociation.
- Rewrite conditional operations in a functional style to enable compilation via conditional data transfers.

A final word of advice to the reader is to be vigilant to avoid introducing errors as you rewrite programs in the interest of efficiency. It is very easy to make mistakes when introducing new variables, changing loop bounds, and making the code more complex overall. One useful technique is to use checking code to test each version of a function as it is being optimized, to ensure no bugs are introduced during this process. Checking code applies a series of tests to the new versions of a function and makes sure they yield the same results as the original. The set of test cases must become more extensive with highly optimized code, since there are more cases to consider. For example, checking code that uses loop unrolling requires testing for many different loop bounds to make sure it handles all of the different possible numbers of single-step iterations required at the end.

5.14 Identifying and Eliminating Performance Bottlenecks

Up to this point, we have only considered optimizing small programs, where there is some clear place in the program that limits its performance and therefore should be the focus of our optimization efforts. When working with large programs, even knowing where to focus our optimization efforts can be difficult. In this section,

eanreunnng
eUnix-3242(sytepms-3242(rogvie)-262.che)-2242(rog036clng)]1242(rogram)-TJ TT1_05g

the relative importance of the different functions in determining the overall run time. The calling information allows us to understand the dynamic behavior of the program.

Profiling with GPROF requires three steps, as shown for a C program `prog.c`, which runs with command-line argument `file.txt`:

1. The program must be compiled and linked for profiling. With GCC (and other C compilers), this involves simply including the run-time flag `-pg` on the command line. It is important to ensure that the compiler does not attempt to perform any optimizations via inline substitution, or else the calls to functions may not be tabulated accurately. We use optimization flag `-Og`, guaranteeing that function calls will be tracked properly.

   ```
   linux> gcc -Og -pg prog.c -o prog
   ```

2. The program is then executed as usual:

   ```
   linux> ./prog file.txt
   ```

   It runs slightly (around a factor of 2) slower than normal, but otherwise the only difference is that it generates a file `gmon.out`.

3. GPROF is invoked to analyze the data in `gmon.out`:

   ```
   linux> gprof prog
   ```

The first part of the profile report lists the times spent executing the different functions, sorted in descending order. As an example, the following listing shows this part of the report for the three most time-consuming functions in a program:

```
  %   cumulative   self              self     total
 time   seconds   seconds    calls   s/call   s/call  name
 97.58    203.66   203.66        1   203.66   203.66  sort_words
  2.32    208.50     4.85   965027     0.00     0.00  find_ele_rec
  0.14    208.81     0.30 12511031     0.00     0.00  Strlen
```

Each row represents the time spent for all calls to some function. The first column indicates the percentage of the overall time spent on the function. The second shows the cumulative time spent by the functions up to and including the one on this row. The third shows the time spent on this particular function, and the fourth shows how many times it was called (not counting recursive calls). In our example, the function `sort_words` was called only once, but this single call required 203.66 seconds, while the function `find_ele_rec` was called 965,027 times (not including recursive calls), requiring a total of 4.85 seconds. Function `Strlen` computes the length of a string by calling the library function `strlen`. Library function calls are normally not shown in the results by GPROF. Their times are usually reported as part of the function calling them. By creating the "wrapper function" `Strlen`, we can reliably track the calls to `strlen`, showing that it was called 12,511,031 times but only requiring a total of 0.30 seconds.

The second part of the profile report shows the calling history of the functions. The following is the history for a recursive function `find_ele_rec`:

```
                                158655725             find_ele_rec [5]
                4.85    0.10  965027/965027      insert_string [4]
[5]      2.4    4.85    0.10  965027+158655725 find_ele_rec [5]
                0.08    0.01  363039/363039      save_string [8]
                0.00    0.01  363039/363039      new_ele [12]
                                158655725             find_ele_rec [5]
```

This history shows both the functions that called `find_ele_rec`, as well as the functions that it called. The first two lines show the calls to the function: 158,655,725 calls by itself recursively, and 965,027 calls by function `insert_string` (which is itself called 965,027 times). Function `find_ele_rec`, in turn, called two other functions, `save_string` and `new_ele`, each a total of 363,039 times.

From these call data, we can often infer useful information about the program behavior. For example, the function `find_ele_rec` is a recursive procedure that scans the linked list for a hash bucket looking for a particular string. For this function, comparing the number of recursive calls with the number of top-level calls provides statistical information about the lengths of the traversals through these lists. Given that their ratio is 164.4:1, we can infer that the program scanned an average of around 164 elements each time.

Some properties of GPROF are worth noting:

- The timing is not very precise. It is based on a simple interval counting scheme in which the compiled program maintains a counter for each function recording the time spent executing that function. The operating system causes the program to be interrupted at some regular time interval δ. Typical values of δ range between 1.0 and 10.0 milliseconds. It then determines what function the program was executing when the interrupt occurred and increments the counter for that function by δ. Of course, it may happen that this function just started executing and will shortly be completed, but it is assigned the full cost of the execution since the previous interrupt. Some other function may run between two interrupts and therefore not be charged any time at all.

  Over a long duration, this scheme works reasonably well. Statistically, every function should be charged according to the relative time spent executing it. For programs that run for less than around 1 second, however, the numbers should be viewed as only rough estimates.

- The calling information is quite reliable, assuming no inline substitutions have been performed. The compiled program maintains a counter for each combination of caller and callee. The appropriate counter is incremented every time a procedure is called.

- By default, the timings for library functions are not shown. Instead, these times are incorporated into the times for the calling functions.

5.14.2 Using a Profiler to Guide Optimization

As an example of using a profiler to guide program optimization, we created an application that involves several different tasks and data structures. This application analyzes the n-gram statistics of a text document, where an n-gram is a sequence of n words occurring in a document. For n = 1, we collect statistics on individual words, for n = 2 on pairs of words, and so on. For a given value of n, our program reads a text file, creates a table of unique n-grams and how many times each one occurs, then sorts the n-grams in descending order of occurrence.

As a benchmark, we ran it on a file consisting of the complete works of William Shakespeare, totaling 965,028 words, of which 23,706 are unique. We found that for n = 1, even a poorly written analysis program can readily process the entire file in under 1 second, and so we set n = 2 to make things more challenging. For the case of n = 2, n-grams are referred to as bigrams (pronounced "bye-grams"). We determined that Shakespeare's works contain 363,039 unique bigrams. The most common is "I am," occurring 1,892 times. Perhaps his most famous bigram, "to be," occurs 1,020 times. Fully 266,018 of the bigrams occur only once.

Our program consists of the following parts. We created multiple versions, starting with simple algorithms for the different parts and then replacing them with more sophisticated ones:

1. Each word is read from the file and converted to lowercase. Our initial version used the function `lower1` (Figure 5.7), which we know to have quadratic run time due to repeated calls to `strlen`.
2. A hash function is applied to the string to create a number between 0 and s − 1, for a hash table with s buckets. Our initial function simply summed the ASCII codes for the characters modulo s.
3. Each hash bucket is organized as a linked list. The program scans down this list looking for a matching entry. If one is found, the frequency for this n-gram is incremented. Otherwise, a new list element is created. Our initial version performed this operation recursively, inserting new elements at the end of the list.
4. Once the table has been generated, we sort all of the elements according to the frequencies. Our initial version used insertion sort.

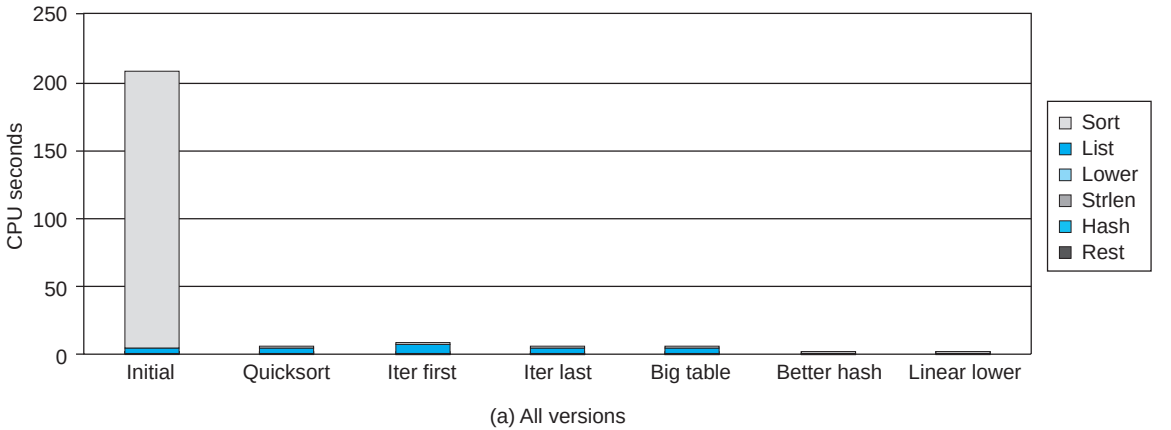
Figure 5.38 shows the profile results for six different versions of our n-gram-frequency analysis program. For each version, we divide the time into the following categories:

Sort. Sorting n-grams by frequency

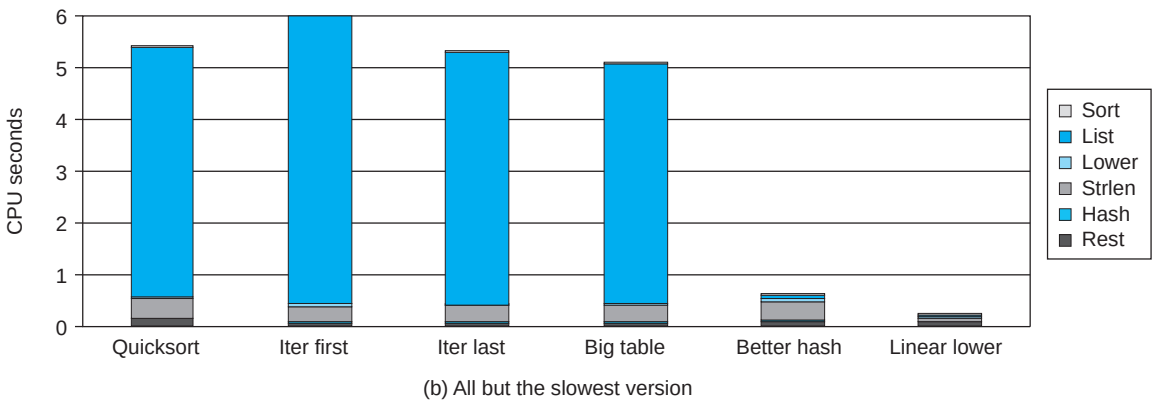
List. Scanning the linked list for a matching n-gram, inserting a new element if necessary

Lower. Converting strings to lowercase

Strlen. Computing string lengths



(a) All versions



(b) All but the slowest version

Figure 5.38 Profile results for different versions of bigram-frequency counting program. Ti i i t a r t a r rat i t r ra .

Hash. Computing the hash function

Rest. The sum of all other functions

As part (a) of the figure shows, our initial version required 3.5 minutes, with most of the time spent sorting. This is not surprising, since insertion sort has quadratic run time and the program sorted 363,039 values.

In our next version, we performed sorting using the library function `qsort`, which is based on the quicksort algorithm [98]. It has an expected run time of $O(n \log n)$. This version is labeled "Quicksort" in the figure. The more efficient sorting algorithm reduces the time spent sorting to become negligible, and the overall run time to around 5.4 seconds. Part (b) of the figure shows the times for the remaining version on a scale where we can see them more clearly.

With improved sorting, we now find that list scanning becomes the bottleneck. Thinking that the inefficiency is due to the recursive structure of the function, we replaced it by an iterative one, shown as "Iter first." Surprisingly, the run time increases to around 7.5 seconds. On closer study, we find a subtle difference between the two list functions. The recursive version inserted new elements at the end of the list, while the iterative one inserted them at the front. To maximize performance, we want the most frequent n-grams to occur near the beginning of the lists. That way, the function will quickly locate the common cases. Assuming that n-grams are spread uniformly throughout the document, we would expect the first occurrence of a frequent one to come before that of a less frequent one. By inserting new n-grams at the end, the first function tended to order n-grams in descending order of frequency, while the second function tended to do just the opposite. We therefore created a third list-scanning function that uses iteration but inserts new elements at the end of this list. With this version, shown as "Iter last," the time dropped to around 5.3 seconds, slightly better than with the recursive version. These measurements demonstrate the importance of running experiments on a program as part of an optimization effort. We initially assumed that converting recursive code to iterative code would improve its performance and did not consider the distinction between adding to the end or to the beginning of a list.

Next, we consider the hash table structure. The initial version had only 1,021 buckets (typically, the number of buckets is chosen to be a prime number to enhance the ability of the hash function to distribute keys uniformly among the buckets). For a table with 363,039 entries, this would imply an average load of 363,039/1,021 = 355.6. That explains why so much of the time is spent performing list operations—the searches involve testing a significant number of candidate n-grams. It also explains why the performance is so sensitive to the list ordering.

Finally, we have reduced the run time to the point where most of the time is spent in `strlen`, and most of the calls to `strlen` occur as part of the lowercase conversion. We have already seen that function `lower1` has quadratic performance, especially for long strings. The words in this document are short enough to avoid the disastrous consequences of quadratic performance; the longest bigram is just 32 characters. Still, switching to `lower2`, shown as "Linear lower," yields a significant improvement, with the overall time dropping to around 0.2 seconds.

With this exercise, we have shown that code profiling can help drop the time required for a simple application from 3.5 minutes down to 0.2 seconds, yielding a performance gain of around 1,000×. The profiler helps us focus our attention on the most time-consuming parts of the program and also provides useful information about the procedure call structure. Some of the bottlenecks in our code, such as using a quadratic sort routine, are easy to anticipate, while others, such as whether to append to the beginning or end of a list, emerge only through a careful analysis.

We can see that profiling is a useful tool to have in the toolbox, but it should not be the only one. The timing measurements are imperfect, especially for shorter (less than 1 second) run times. More significantly, the results apply only to the particular data tested. For example, if we had run the original function on data consisting of a smaller number of longer strings, we would have found that the lowercase conversion routine was the major performance bottleneck. Even worse, if it only profiled documents with short words, we might never detect hidden bottlenecks such as the quadratic performance of `lower1`. In general, profiling can help us optimize for typical cases, assuming we run the program on representative data, but we should also make sure the program will have respectable performance for all possible cases. This mainly involves avoiding algorithms (such as insertion sort) and bad programming practices (such as `lower1`) that yield poor asymptotic performance.

Amdahl's law, described in Section 1.9.1, provides some additional insights into the performance gains that can be obtained by targeted optimizations. For our n-gram code, we saw the total execution time drop from 209.0 to 5.4 seconds when we replaced insertion sort by quicksort. The initial version spent 203.7 of its 209.0 seconds performing insertion sort, giving $\alpha = 0.974$, the fraction of time subject to speedup. With quicksort, the time spent sorting becomes negligible, giving a predicted speedup of $209/\alpha = 39.0$, close to the measured speedup of 38.5. We were able to gain a large speedup because sorting constituted a very large fraction of the overall execution time. However, when one bottleneck is eliminated, a new one arises, and so gaining additional speedup required focusing on other parts of the program.

5.15 Summary

Although most presentations on code optimization describe how compilers can generate efficient code, much can be done by an application programmer to assist the compiler in this task. No compiler can replace an inefficient algorithm or data

structure by a good one, and so these aspects of program design should remain a primary concern for programmers. We also have seen that optimization blockers, such as memory aliasing and procedure calls, seriously restrict the ability of compilers to perform extensive optimizations. Again, the programmer must take primary responsibility for eliminating these. These should simply be considered parts of good programming practice, since they serve to eliminate unneeded work.

Tuning performance beyond a basic level requires some understanding of the processor's microarchitecture, describing the underlying mechanisms by which the processor implements its instruction set architecture. For the case of out-of-order processors, just knowing something about the operations, capabilities, latencies, and issue times of the functional units establishes a baseline for predicting program performance.

We have studied a series of techniques—including loop unrolling, creating multiple accumulators, and reassociation—that can exploit the instruction-level parallelism provided by modern processors. As we get deeper into the optimization, it becomes important to study the generated assembly code and to try to understand how the computation is being performed by the machine. Much can be gained by identifying the critical paths determined by the data dependencies in the program, especially between the different iterations of a loop. We can also compute a throughput bound for a computation, based on the number of operations that must be computed and the number and issue times of the units that perform those operations.

Programs that involve conditional branches or complex interactions with the memory system are more difficult to analyze and optimize than the simple loop programs we first considered. The basic strategy is to try to make branches more predictable or make them amenable to implementation using conditional data transfers. We must also watch out for the interactions between store and load operations. Keeping values in local variables, allowing them to be stored in registers, can often be helpful.

When working with large programs, it becomes important to focus our optimization efforts on the parts that consume the most time. Code profilers and related tools can help us systematically evaluate and improve program performance. We described GPROF, a standard Unix profiling tool. More sophisticated profilers are available, such as the VTUNE program development system from Intel, and VALGRIND, commonly available on Linux systems. These tools can break down the execution time below the procedure level to estimate the performance of each basic block of the program. (A basic block is a sequence of instructions that has no transfers of control out of its middle, and so the block is always executed in its entirety.)

Bibliographic Notes

Our focus has been to describe code optimization from the programmer's perspective, demonstrating how to write code that will make it easier for compilers to generate efficient code. An extended paper by Chellappa, Franchetti, and Püschel [19]

takes a similar approach but goes into more detail with respect to the processor's characteristics.

Many publications describe code optimization from a compiler's perspective, formulating ways that compilers can generate more efficient code. Muchnick's book is considered the most comprehensive [80]. Wadleigh and Crawford's book on software optimization [115] covers some of the material we have presented, but it also describes the process of getting high performance on parallel machines. An early paper by Mahlke et al. [75] describes how several techniques developed for compilers that map programs onto parallel machines can be adapted to exploit the instruction-level parallelism of modern processors. This paper covers the code transformations we presented, including loop unrolling, multiple accumulators (which they refer to as accumulator variable expansion), and reassociation (which they refer to as tree height reduction).

Our presentation of the operation of an out-of-order processor is fairly brief and abstract. More complete descriptions of the general principles can be found in advanced computer architecture textbooks, such as the one by Hennessy and Patterson [46, Ch. 2–3]. Shen and Lipasti's book [100] provides an in-depth treatment of modern processor design.

Homework Problems

5.13

Suppose we wish to write a procedure that computes the inner product of two vectors u and v. An abstract version of the function has a CPE of 14–18 with x86-64 for different types of integer and floating-point data. By doing the same sort of transformations we did to transform the abstract program `combine1` into the more efficient `combine4`, we geT1_5 1016gett50e

```
    Inner loop of inner4.  data_t = double, OP = *
    udata in %rbp, vdata in %rax, sum in %xmm0
    i in %rcx, limit in %rbx
1   .L15:                                       loop:
2     vmovsd  0(%rbp,%rcx,8), %xmm1               Get udata[i]
3     vmulsd  (%rax,%rcx,8), %xmm1, %xmm1         Multiply by vdata[i]
4     vaddsd  %xmm1, %xmm0, %xmm0                 Add to sum
5     addq    $1, %rcx                            Increment i
6     cmpq    %rbx, %rcx                          Compare i:limit
7     jne     .L15                                If !=, goto loop
```

Assume that the functional units have the characteristics listed in Figure 5.12.

A. Diagram how this instruction sequence would be decoded into operations and show how the data dependencies between them would create a critical path of operations, in the style of Figures 5.13 and 5.14.

B. For data type `double`, what lower bound on the CPE is determined by the critical path?

C. Assuming similar instruction sequences for the integer code as well, what lower bound on the CPE is determined by the critical path for integer data?

D. Explain how the floating-point versions can have CPEs of 3.00, even though the multiplication operation requires 5 clock cycles.

5.14

Write a version of the inner product procedure described in Problem 5.13 that uses 6 × 1 loop unrolling. For x86-64, our measurements of the unrolled version give a CPE of 1.07 for integer data but still 3.01 for both floating-point data.

A. Explain why any (scalar) version of an inner product procedure running on an Intel Core i7 Haswell processor cannot achieve a CPE less than 1.00.

B. Explain why the performance for floating-point data did not improve with loop unrolling.

5.15

Write a version of the inner product procedure described in Problem 5.13 that uses 6 × 6 loop unrolling. Our measurements for this function with x86-64 give a CPE of 1.06 for integer data and 1.01 for floating-point data.

What factor limits the performance to a CPE of 1.00?

5.16

Write a version of the inner product procedure described in Problem 5.13 that uses 6 × 1a loop unrolling to enable greater parallelism. Our measurements for this function give a CPE of 1.10 for integer data and 1.05 for floating-point data.

5.17

The library function `memset` has the following prototype:

```
void *memset(void *s, int c, size_t n);
```

faster versions of the function using the optimization techniques we have explored,

Code	min	max	incr	square
A.	1	91	90	90
B.	91	1	90	90
C.	1	1	90	90

Solution to Problem 5.4 (page 552)

This assembly code demonstrates a clever optimization opportunity detected by GCC. It is worth studying this code carefully to better understand the subtleties of code optimization.

A. In the less optimized code, register `%xmm0` is simply used as a temporary value, both set and used on each loop iteration. In the more optimized code, it is used more in the manner of variable `acc` in `combine4`, accumulating the product of the vector elements. The difference with `combine4`, however, is that location `dest` is updated on each iteration by the second `vmovsd` instruction.

We can see that this optimized version operates much like the following C code:

```
1   /* Make sure dest updated on each iteration */
2   void combine3w(vec_ptr v, data_t *dest)
3   {
4       long i;
5       long length = vec_length(v);
6       data_t *data = get_vec_start(v);
7       data_t acc = IDENT;
8
9       /* Initialize in event length <= 0 */
10      *dest = acc;
11
12      for (i = 0; i < length; i++) {
13          acc = acc OP data[i];
14          *dest = acc;
15      }
16  }
```

B. The two versions of `combine3` will have identical functionality, even with memory aliasing.

C. This transformation can be made without changing the program behavior, because, with the exception of the first iteration, the value read from `dest` at the beginning of each iteration will be the same value written to this register

at the end of the previous iteration. Therefore, the combining instruction can simply use the value already in %xmm0 at the beginning of the loop.

Solution to Problem 5.5 (page 566)

Polynomial evaluation is a core technique for solving many problems. For example, polynomial functions are commonly used to approximate trigonometric functions in math libraries.

A. The function performs 2n multiplications and n additions.

B. We can see that the performance-limiting computation here is the repeated computation of the expression xpwr = x * xpwr. This requires a floating-point multiplication (5 clock cycles), and the computation for one iteration cannot begin until the one for the previous iteration has completed. The updating of result only requires a floating-point addition (3 clock cycles) between successive iterations.

Solution to Problem 5.6 (page 566)

This problem demonstrates that minimizing the number of operations in a computation may not improve its performance.

A. The function performs n multiplications and n additions, half the number of multiplications as the original function poly.

B. We can see that the performance-limiting computation here is the repeated computation of the expression result = a[i] + x*result. Starting from the value of result from the previous iteration, we must first multiply it by x (5 clock cycles) and then add it to a[i] (3 cycles) before we have the value for this iteration. Thus, each iteration imposes a minimum latency of 8 cycles, exactly our measured CPE.

C. Although each iteration in function poly requires two multiplications rather than one, only a single multiplication occurs along the critical path per iteration.

Solution to Problem 5.7 (page 568)

The following code directly follows the rules we have stated for unrolling a loop by some factor k:

```
1  void unroll5(vec_ptr v, data_t *dest)
2  {
3      long i;
4      long length = vec_length(v);
5      long limit = length-4;
6      data_t *data = get_vec_start(v);
7      data_t acc = IDENT;
8
```

```
9       /* Combine 5 elements at a time */
10      for (i = 0; i < limit; i+=5) {
11          acc = acc OP data[i]   OP data[i+1];
12          acc = acc OP data[i+2] OP data[i+3];
13          acc = acc OP data[i+4];
14      }
15
16      /* Finish any remaining elements */
17      for (; i < length; i++) {
18          acc = acc OP data[i];
19      }
20      *dest = acc;
21  }
```

Solution to Problem 5.8 (page 581)

This problem demonstrates how small changes in a program can yield dramatic performance differences, especially on a machine with out-of-order execution. Figure 5.39 diagrams the three multiplication operations for a single iteration of the function. In this figure, the operations shown as blue boxes are along the critical path—they need to be computed in sequence to compute a new value for loop variable r. The operations shown as light boxes can be computed in parallel with the critical path operations. For a loop with P operations along the critical path, each iteration will require a minimum of 5P clock cycles and will compute the product for three elements, giving a lower bound on the CPE of 5P /3. This implies lower bounds of 5.00 for A1, 3.33 for A2 and A5, and 1.67 for A3 and A4. We ran these functions on an Intel Core i7 Haswell processor and found that it could achieve these CPE values.

Solution to Problem 5.9 (page 589)

This is another demonstration that a slight change in coding style can make it much easier for the compiler to detect opportunities to use conditional moves:

```
while (i1 < n && i2 < n) {
    long v1 = src1[i1];
```

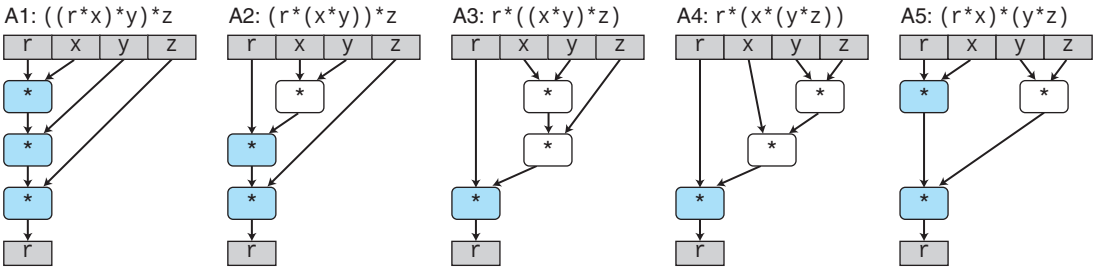


Figure 5.39 Data dependencies among multiplication operations for cases in Problem 5.8. T o rati o o a l o o r t rti al at o rt i rati o .

```
        long v2 = src2[i2];
        long take1 = v1 < v2;
        dest[id++] = take1 ? v1 : v2;
        i1 += take1;
        i2 += (1-take1);
    }
```

We measured a CPE of around 12.0 for this version of the code, a modest improvement over the original CPE of 15.0.

Solution to Problem 5.10 (page 595)

This problem requires you to analyze the potential load-store interactions in a program.

A. It will set each element `a[i]` to `i + 1`, for $0 \le i \le 998$.

B. It will set each element `a[i]` to 0, for $1 \le i \le 999$.

C. In the second case, the load of one iteration depends on the result of the store from the previous iteration. Thus, there is a write/read dependency between successive iterations.

D. It will give a CPE of 1.2, the same as for Example A, since there are no dependencies between stores and subsequent loads.

Solution to Problem 5.11 (page 597)

We can see that this function has a write/read dependency between successive iterations—the destination value `p[i]` on one iteration matches the source value `p[i-1]` on the next. A critical path is therefore formed for each iteration consisting of a store (from the previous iteration), a load, and a floating-point addition. The CPE measurement of 9.0 is consistent with our measurement of 7.3 for the CPE of `write_read` when there is a data dependency, since `write_read` involves an integer addition (1 clock-cycle latency), while `psum1` involves a floating-point addition (3 clock-cycle latency).

Solution to Problem 5.12 (page 597)

Here is a revised version of the function:

```
1   void psum1a(float a[], float p[], long n)
2   {
3       long i;
4       /* last_val holds p[i-1]; val holds p[i] */
5       float last_val, val;
6       last_val = p[0] = a[0];
7       for (i = 1; i < n; i++) {
8           val  = last_val + a[i];
9           p[i] = val;
10          last_val = val;
11      }
12  }
```

We introduce a local variable `last_val`. At the start of iteration `i`, it holds the value of `p[i-1]`. We then compute `val` to be the value of `p[i]` and to be the new value for `last_val`.

This version compiles to the following assembly code:

```
Inner loop of psum1a
a in %rdi, i in %rax, cnt in %rdx, last_val in %xmm0
.L16:                                          loop:
  vaddss  (%rdi,%rax,4), %xmm0, %xmm0            last_val = val = last_val + a[i]
  vmovss  %xmm0, (%rsi,%rax,4)                   Store val in p[i]
  addq    $1, %rax                               Increment i
  cmpq    %rdx, %rax                             Compare i:cnt
  jne     .L16                                   If !=, goto loop
```

This code holds `last_val` in `%xmm0`, avoiding the need to read `p[i-1]` from memory and thus eliminating the write/read dependency seen in `psum1`.

6

The Memory Hierarchy

6.1 Storage Technologies 617

6.2 Locality 640

6.3 The Memory Hierarchy 645

6.4 Caching in the Memory Hierarchy 650

6.5 Writing Cache-Friendly Code 669

6.6 Putting It Together: The Impact of Caches on Program Performance 675

6.7 Summary 684

Bibliographic Notes 684

Homework Problems 685

Solutions to Practice Problems 696

To this point in our study of systems, we have relied on a simple model of a computer system as a CPU that executes instructions and a memory system that holds instructions and data for the CPU. In our simple model, the memory system is a linear array of bytes, and the CPU can access each memory location in a constant amount of time. While this is an effective model up to a point, it does not reflect the way that modern systems really work.

In practice, a memory system is a hierarchy of storage devices with different capacities, costs, and access times. CPU registers hold the most frequently used data. Small, fast cache memories nearby the CPU act as staging areas for a subset of the data and instructions stored in the relatively slow main memory. The main memory stages data stored on large, slow disks, which in turn often serve as staging areas for data stored on the disks or tapes of other machines connected by networks.

Memory hierarchies work because well-written programs tend to access the storage at any particular level more frequently than they access the storage at the next lower level. So the storage at the next level can be slower, and thus larger

6.1 Storage Technologies

Much of the success of computer technology stems from the tremendous progress in storage technology. Early computers had a few kilobytes of random access memory. The earliest IBM PCs didn't even have a hard disk. That changed with the introduction of the IBM PC-XT in 1982, with its 10-megabyte disk. By the year 2015, typical machines had 300,000 times as much disk storage, and the amount of storage was increasing by a factor of 2 every couple of years.

6.1.1 Random Access Memory

Random access memory (RAM) comes in two varieties—static and dynamic. Static RAM (SRAM) is faster and significantly more expensive than dynamic RAM (DRAM). SRAM is used for cache memories, both on and off the CPU chip. DRAM is used for the main memory plus the frame buffer of a graphics system. Typically, a desktop system will have no more than a few tens of megabytes of SRAM, but hundreds or thousands of megabytes of DRAM.

Static RAM

SRAM stores each bit in a bistable memory cell. Each cell is implemented with a six-transistor circuit. This circuit has the property that it can stay indefinitely in either of two different voltage configurations, or states. Any other state will be unstable—starting from there, the circuit will quickly move toward one of the stable states. Such a memory cell is analogous to the inverted pendulum illustrated in Figure 6.1.

The pendulum is stable when it is tilted either all the way to the left or all the way to the right. From any other position, the pendulum will fall to one side or the other. In principle, the pendulum could also remain balanced in a vertical position indefinitely, but this state is metastable—the smallest disturbance would make it start to fall, and once it fell it would never return to the vertical position.

Due to its bistable nature, an SRAM memory cell will retain its value indefinitely, as long as it is kept powered. Even when a disturbance, such as electrical noise, perturbs the voltages, the circuit will return to the stable value when the disturbance is removed.

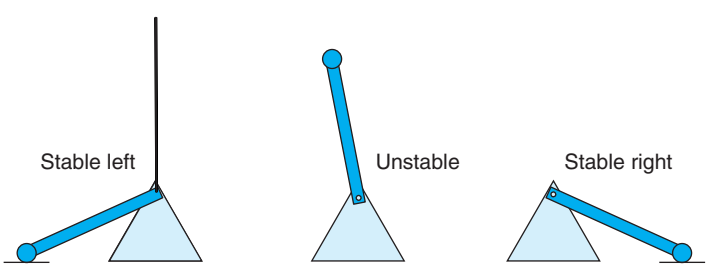


Figure 6.1
Inverted pendulum. Like a SRAM cell, it has two stable states.

	Transistors per bit	Relative access time	Persistent?	Sensitive?	Relative cost	Applications
SRAM	6	1×	Yes	No	1,000×	Cache memory
DRAM	1	10×	No	Yes	1×	Main memory, frame buffers

Figure 6.2 Characteristics of DRAM and SRAM memory.

Dynamic RAM

DRAM stores each bit as charge on a capacitor. This capacitor is very small—typically around 30 femtofarads—that is, 30×10^{-15} farads. Recall, however, that a farad is a very large unit of measure. DRAM storage can be made very dense—each cell consists of a capacitor and a single access transistor. Unlike SRAM, however, a DRAM memory cell is very sensitive to any disturbance. When the capacitor voltage is disturbed, it will never recover. Exposure to light rays will cause the capacitor voltages to change. In fact, the sensors in digital cameras and camcorders are essentially arrays of DRAM cells.

Various sources of leakage current cause a DRAM cell to lose its charge within a time period of around 10 to 100 milliseconds. Fortunately, for computers operating with clock cycle times measured in nanoseconds, this retention time is quite long. The memory system must periodically refresh every bit of memory by reading it out and then rewriting it. Some systems also use error-correcting codes, where the computer words are encoded using a few more bits (e.g., a 64-bit word might be encoded using 72 bits), such that circuitry can detect and correct any single erroneous bit within a word.

Figure 6.2 summarizes the characteristics of SRAM and DRAM memory. SRAM is persistent as long as power is applied. Unlike DRAM, no refresh is necessary. SRAM can be accessed faster than DRAM. SRAM is not sensitive to disturbances such as light and electrical noise. The trade-off is that SRAM cells use more transistors than DRAM cells and thus have lower densities, are more expensive, and consume more power.

Conventional DRAM

The cells (bits) in a DRAM chip are partitioned into d supercells, each consisting of w DRAM cells. A $d \times w$ DRAM stores a total of dw bits of information. The supercells are organized as a rectangular array with r rows and c columns, where $rc = d$. Each supercell has an address of the form (i, j), where i denotes the row and j denotes the column.

For example, Figure 6.3 shows the organization of a 16 × 8 DRAM chip with $d = 16$ supercells, $w = 8$ bits per supercell, $r = 4$ rows, and $c = 4$ columns. The shaded box denotes the supercell at address (2, 1). Information flows in and out of the chip via external connectors called pins. Each pin carries a 1-bit signal. Figure 6.3 shows two of these sets of pins: eight data pins that can transfer 1 byte

Aside A brief history

The storage community has never settled on a standard name for a DRAM array element. Computer architects tend to refer to it as a "cell," overloading the term with the DRAM storage cell. Circuit designers tend to refer to it as a "word," overloading the term with a word of main memory. To avoid confusion, we have adopted the unambiguous term "supercell."

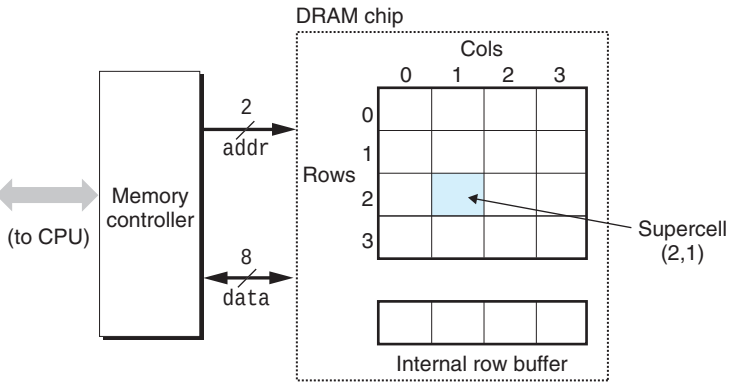


Figure 6.3
High-level view of a 128-bit 16 × 8 DRAM chip.

in or out of the chip, and two addr pins that carry two-bit row and column supercell addresses. Other pins that carry control information are not shown.

Each DRAM chip is connected to some circuitry, known as the memory controller, that can transfer w bits at a time to and from each DRAM chip. To read the contents of supercell (i, j), the memory controller sends the row address i to the DRAM, followed by the column address j. The DRAM responds by sending the contents of supercell (i, j) back to the controller. The row address i is called a RAS (row access strobe) request. The column address j is called a CAS (column access strobe) request. Notice that the RAS and CAS requests share the same DRAM address pins.

For example, to read supercell (2, 1) from the 16 × 8 DRAM in Figure 6.3, the memory controller sends row address 2, as shown in Figure 6.4(a). The DRAM responds by copying the entire contents of row 2 into an internal row buffer. Next, the memory controller sends column address 1, as shown in Figure 6.4(b). The DRAM responds by copying the 8 bits in supercell (2, 1) from the row buffer and sending them to the memory controller.

One reason circuit designers organize DRAMs as two-dimensional arrays instead of linear arrays is to reduce the number of address pins on the chip. For example, if our example 128-bit DRAM were organized as a linear array of 16 supercells with addresses 0 to 15, then the chip would need four address pins instead of two. The disadvantage of the two-dimensional array organization is that addresses must be sent in two distinct steps, which increases the access time.

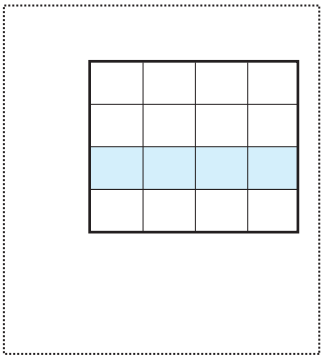
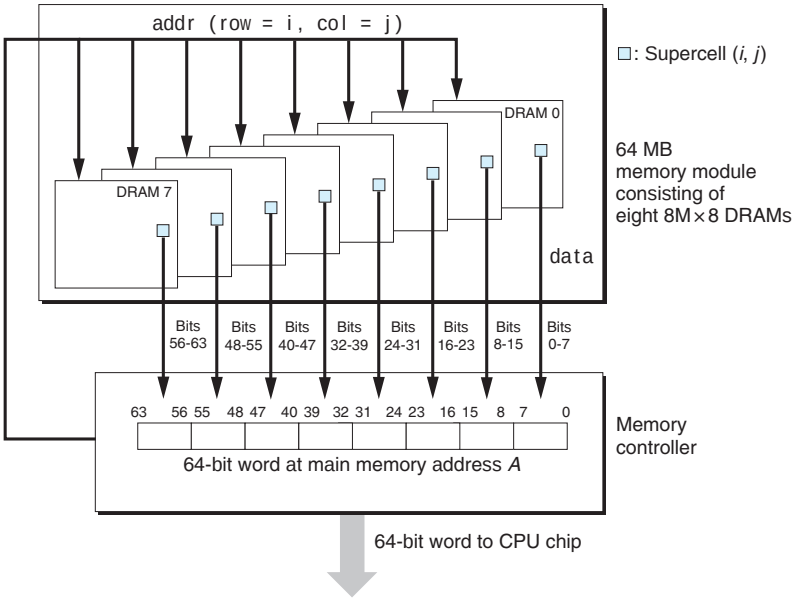


Figure 6.5
Reading the contents of a memory module.



Organization	r	c	b_r	b_c	$\max(b_r, b_c)$
16 × 1					
16 × 4					
128 × 8					
512 × 4					
1,024 × 4					

E a DRAM

There are many kinds of DRAM memories, and new kinds appear on the market with regularity as manufacturers attempt to keep up with rapidly increasing processor speeds. Each is based on the conventional DRAM cell, with optimizations that improve the speed with which the basic DRAM cells can be accessed.

Fast page mode DRAM (FPM DRAM). A conventional DRAM copies an entire row of supercells into its internal row buffer, uses one, and then discards the rest. FPM DRAM improves on this by allowing consecutive accesses to the same row to be served directly from the row buffer. For example, to read four supercells from row i of a conventional DRAM, the memory controller must send four RAS/CAS requests, even though the row address i is identical in each case. To read supercells from the same row of an FPM DRAM, the memory controller sends an initial RAS/CAS request, followed by three CAS requests. The initial RAS/CAS request copies row i into the row buffer and returns the supercell addressed by the

CAS. The next three supercells are served directly from the row buffer, and thus are returned more quickly than the initial supercell.

Extended data out DRAM (EDO DRAM). An enhanced form of FPM DRAM that allows the individual CAS signals to be spaced closer together in time.

Synchronous DRAM (SDRAM). Conventional, FPM, and EDO DRAMs are asynchronous in the sense that they communicate with the memory controller using a set of explicit control signals. SDRAM replaces many of these control signals with the rising edges of the same external clock signal that drives the memory controller. Without going into detail, the net effect is that an SDRAM can output the contents of its supercells at a faster rate than its asynchronous counterparts.

Double Data-Rate Synchronous DRAM (DDR SDRAM). DDR SDRAM is an enhancement of SDRAM that doubles the speed of the DRAM by using both clock edges as control signals. Different types of DDR SDRAMs are characterized by the size of a small prefetch buffer that increases the effective bandwidth: DDR (2 bits), DDR2 (4 bits), and DDR3 (8 bits).

Video RAM (VRAM). Used in the frame buffers of graphics systems. VRAM is similar in spirit to FPM DRAM. Two major differences are that (1) VRAM output is produced by shifting the entire contents of the internal buffer in sequence and (2) VRAM allows concurrent reads and writes to the memory. Thus, the system can be painting the screen with the pixels in the frame buffer (reads) while concurrently writing new values for the next update (writes).

Nonvolatile Memory

DRAMs and SRAMs are volatile in the sense that they lose their information if the supply voltage is turned off. Nonvolatile memories, on the other hand, retain their information even when they are powered off. There are a variety of nonvolatile memories. For historical reasons, they are referred to collectively as read-only memories (ROMs), even though some types of ROMs can be written to as well as read. ROMs are distinguished by the number of times they can be reprogrammed (written to) and by the mechanism for reprogramming them.

Aside **Historical perspective of DRAM evolution**

Until 1995, most PCs were built with FPM DRAMs. From 1996 to 1999, EDO DRAMs dominated the market, while FPM DRAMs all but disappeared. SDRAMs first appeared in 1995 in high-end systems, and by 2002 most PCs were built with SDRAMs and DDR SDRAMs. By 2010, most server and desktop systems were built with DDR3 SDRAMs. In fact, the Intel Core i7 supports only DDR3 SDRAM.

A programmable ROM (PROM) can be programmed exactly once. PROMs include a sort of fuse with each memory cell that can be blown once by zapping it with a high current.

An erasable programmable ROM (EPROM) has a transparent quartz window that permits light to reach the storage cells. The EPROM cells are cleared to zeros by shining ultraviolet light through the window. Programming an EPROM is done by using a special device to write ones into the EPROM. An EPROM can be erased and reprogrammed on the order of 1,000 times. An electrically erasable PROM (EEPROM) is akin to an EPROM, but it does not require a physically separate programming device, and thus can be reprogrammed in-place on printed circuit cards. An EEPROM can be reprogrammed on the order of 10^5 times before it wears out.

Flash memory is a type of nonvolatile memory, based on EEPROMs, that has become an important storage technology. Flash memories are everywhere, providing fast and durable nonvolatile storage for a slew of electronic devices, including digital cameras, cell phones, and music players, as well as laptop, desktop, and server computer systems. In Section 6.1.3, we will look in detail at a new form of flash-based disk drive, known as a solid state disk (SSD), that provides a faster, sturdier, and less power-hungry alternative to conventional rotating disks.

Programs stored in ROM devices are often referred to as firmware. When a computer system is powered up, it runs firmware stored in a ROM. Some systems provide a small set of primitive input and output functions in firmware—for example, a PC's BIOS (basic input/output system) routines. Complicated devices such as graphics cards and disk drive controllers also rely on firmware to translate I/O (input/output) requests from the CPU.

A i Mai M or

Data flows back and forth between the processor and the DRAM main memory over shared electrical conduits called buses. Each transfer of data between the CPU and memory is accomplished with a series of steps called a bus transaction. A read transaction transfers data from the main memory to the CPU. A write transaction transfers data from the CPU to the main memory.

A bus is a collection of parallel wires that carry address, data, and control signals. Depending on the particular bus design, data and address signals can share the same set of wires or can use different sets. Also, more than two devices can share the same bus. The control wires carry signals that synchronize the transaction and identify what kind of transaction is currently being performed. For example, is this transaction of interest to the main memory, or to some other I/O device such as a disk controller? Is the transaction a read or a write? Is the information on the bus an address or a data item?

Figure 6.6 shows the configuration of an example computer system. The main components are the CPU chip, a chipset that we will call an I/O bridge (which includes the memory controller), and the DRAM memory modules that make up main memory. These components are connected by a pair of buses: a system bus that connects the CPU to the I/O bridge, and a memory bus that connects the I/O

Aside A t i

Bus design is a complex and rapidly changing aspect of computer systems. Different vendors develop different bus architectures as a way to differentiate their products. For example, some Intel systems use chipsets known as the northbridge and the southbridge to connect the CPU to memory and I/O devices, respectively. In older Pentium and Core 2 systems, a front side bus (FSB) connects the CPU to the northbridge. Systems from AMD replace the FSB with the HyperTransport interconnect, while newer Intel Core i7 systems use the QuickPath interconnect. The details of these different bus architectures are beyond the scope of this text. Instead, we will use the high-level bus architecture from Figure 6.6 as a running example throughout. It is a simple but useful abstraction that allows us to be concrete. It captures the main ideas without being tied too closely to the detail of any proprietary designs.

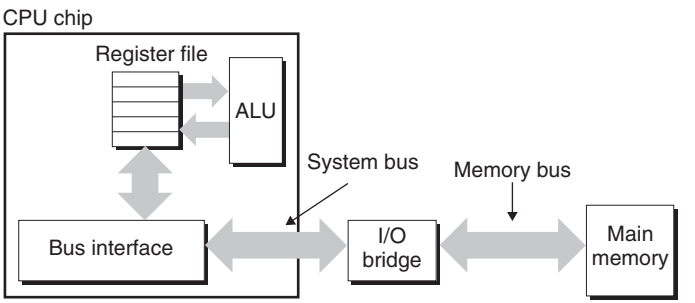


Figure 6.6
Example bus structure that connects the CPU and main memory.

bridge to the main memory. The I/O bridge translates the electrical signals of the system bus into the electrical signals of the memory bus. As we will see, the I/O bridge also connects the system bus and memory bus to an I/O bus that is shared by I/O devices such as disks and graphics cards. For now, though, we will focus on the memory bus.

Consider what happens when the CPU performs a load operation such as

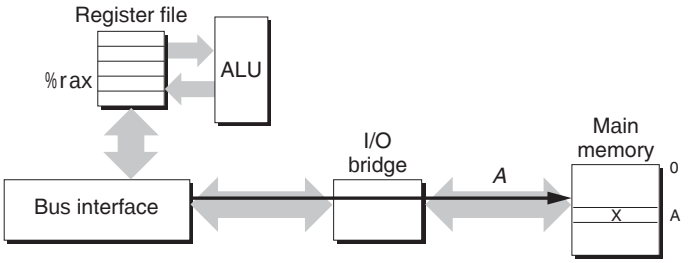
```
movq A,%rax
```

where the contents of address A are loaded into register %rax. Circuitry on the CPU chip called the bus interface initiates a read transaction on the bus. The read transaction consists of three steps. First, the CPU places the address A on the system bus. The I/O bridge passes the signal along to the memory bus (Figure 6.7(a)). Next, the main memory senses the address signal on the memory bus, reads the address from the memory bus, fetches the data from the DRAM, and writes the data to the memory bus. The I/O bridge translates the memory bus signal into a system bus signal and passes it along to the system bus (Figure 6.7(b)). Finally, the CPU senses the data on the system bus, reads the data from the bus, and copies the data to register %rax (Figure 6.7(c)).

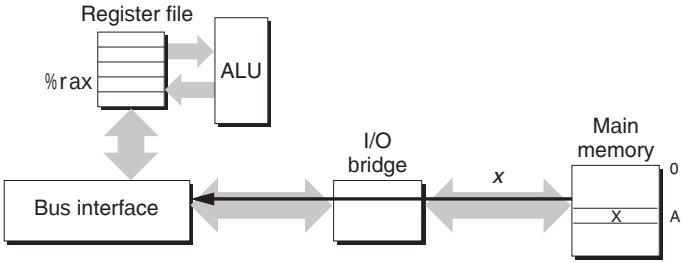
Conversely, when the CPU performs a store operation such as

```
movq %rax,A
```

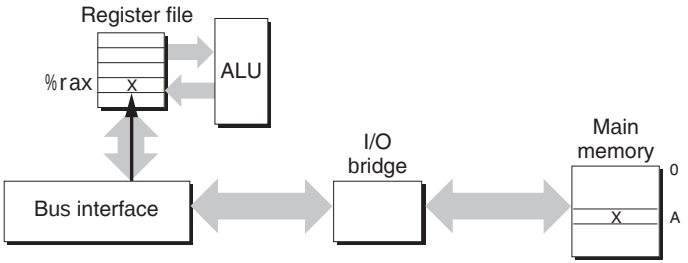
Figure 6.7
Memory read transaction for a load operation: `movq A,%rax`.



(a) CPU places address *A* on the memory bus.



(b) Main memory reads *A* from the bus, retrieves word *x*, and places it on the bus.



(c) CPU reads word *x* from the bus, and copies it into register `%rax`.

where the contents of register `%rax` are written to address A, the CPU initiates a write transaction. Again, there are three basic steps. First, the CPU places the address on the system bus. The memory reads the address from the memory bus and waits for the data to arrive (Figure 6.8(a)). Next, the CPU copies the data in `%rax` to the system bus (Figure 6.8(b)). Finally, the main memory reads the data from the memory bus and stores the bits in the DRAM (Figure 6.8(c)).

6.1.2 Disk Storage

Disks are workhorse storage devices that hold enormous amounts of data, on the order of hundreds to thousands of gigabytes, as opposed to the hundreds or thousands of megabytes in a RAM-based memory. However, it takes on the order of milliseconds to read information from a disk, a hundred thousand times longer than from DRAM and a million times longer than from SRAM.

code/ecf/waitforsignal.c

```
1  #include "csapp.h"
2
3  volatile sig_atomic_t pid;
4
5  void sigchld_handler(int s)
6  {
7      int olderrno = errno;
8      pid = waitpid(-1, NULL, 0);
9      errno = olderrno;
10 }
11
12 void sigint_handler(int s)
13 {
14 }
15
16 int main(int argc, char **argv)
17 {
18     sigset_t mask, prev;
19
20     Signal(SIGCHLD, sigchld_handler);
21     Signal(SIGINT, sigint_handler);
22     Sigemptyset(&mask);
23     Sigaddset(&mask, SIGCHLD);
24
25     while (1) {
26         Sigprocmask(SIG_BLOCK, &mask, &prev); /* Block SIGCHLD */
27         if (Fork() == 0) /* Child */
28             exit(0);
29
30         /* Parent */
31         pid = 0;
32         Sigprocmask(SIG_SETMASK, &prev, NULL); /* Unblock SIGCHLD */
33
34         /* Wait for SIGCHLD to be received (wasteful) */
35         while (!pid)
36             ;
37
38         /* Do some work after receiving SIGCHLD */
39         printf(".");
40     }
41     exit(0);
42 }
```

code/ecf/waitforsignal.c

Figure 8.41 Waiting for a signal with a spin loop. T i o i orr t, t t i l oo i a t l.

The proper solution is to use `sigsuspend`.

```
#include <signal.h>

int sigsuspend(const sigset_t *mask);
                                                    Returns: −1
```

The `sigsuspend` function temporarily replaces the current blocked set with `mask` and then suspends the process until the receipt of a signal whose action is either to run a handler or to terminate the process. If the action is to terminate, then the process terminates without returning from `sigsuspend`. If the action is to run a handler, then `sigsuspend` returns after the handler returns, restoring the blocked set to its state when `sigsuspend` was called.

The `sigsuspend` function is equivalent to an atomic (uninterruptible) version of the following:

```
1       sigprocmask(SIG_BLOCK, &mask, &prev);
2       pause();
3       sigprocmask(SIG_SETMASK, &prev, NULL);
```

The atomic property guarantees that the calls to `sigprocmask` (line 1) and `pause` (line 2) occur together, without being interrupted. This eliminates the potential race where a signal is received after the call to `sigprocmask` and before the call to `pause`.

Figure 8.42 shows how we would use `sigsuspend` to replace the spin loop in Figure 8.41. Before each call to `sigsuspend`, SIGCHLD is blocked. The `sigsuspend` temporarily unblocks SIGCHLD, and then sleeps until the parent catches a signal. Before returning, it restores the original blocked set, which blocks SIGCHLD again. If the parent caught a SIGINT, then the loop test succeeds and the next iteration calls `sigsuspend` again. If the parent caught a SIGCHLD, then the loop test fails and we exit the loop. At this point, SIGCHLD is blocked, and so we can optionally unblock SIGCHLD. This might be useful in a real shell with background jobs that need to be reaped.

The `sigsuspend` version is less wasteful than the original spin loop, avoids the race introduced by `pause`, and is more efficient than `sleep`.

8.6 Nonlocal Jumps

C provides a form of user-level exceptional control flow, called a nonlocal jump, that transfers control directly from one function to another currently executing function without having to go through the normal call-and-return sequence. Nonlocal jumps are provided by the `setjmp` and `longjmp` functions.

code/ecf/sigsuspend.c

```
1  #include "csapp.h"
2
3  volatile sig_atomic_t pid;
4
5  void sigchld_handler(int s)  ;-1_ a NULL_ a 0)  i _t  ;9581.8942
```

```
#include <setjmp.h>

int setjmp(jmp_buf env);
int sigsetjmp(sigjmp_buf env, int savesigs);
                                    Returns: 0 from setjmp, nonzero from longjmps
```

The setjmp function saves the current calling environment in the env buffer, for later use by longjmp, and returns 0. The calling environment includes the program counter, stack pointer, and general-purpose registers. For subtle reasons beyond our scope, the value that setjmp returns should not be assigned to a variable:

```
rc = setjmp(env);  /* Wrong! */
```

However, it can be safely used as a test in a switch or conditional statement [62].

```
#include <setjmp.h>

void longjmp(jmp_buf env, int retval);
void siglongjmp(sigjmp_buf env, int retval);
                                                              Never returns
```

The longjmp function restores the calling environment from the env buffer and then triggers a return from the most recent setjmp call that initialized env. The setjmp then returns with the nonzero return value retval.

The interactions between setjmp and longjmp can be confusing at first glance. The setjmp function is called once but returns multiple times: once when the setjmp is first called and the calling environment is stored in the env buffer, and once for each corresponding longjmp call. On the other hand, the longjmp function is called once but never returns.

An important application of nonlocal jumps is to permit an immediate return from a deeply nested function call, usually as a result of detecting some error condition. If an error condition is detected deep in a nested function call, we can use a nonlocal jump to return directly to a common localized error handler instead of laboriously unwinding the call stack.

Figure 8.43 shows an example of how this might work. The main routine first calls setjmp to save the current calling environment, and then calls function foo, which in turn calls function bar. If foo or bar encounter an error, they return immediately from the setjmp via a longjmp call. The nonzero return value of the setjmp indicates the error type, which can then be decoded and handled in one place in the code.

The feature of longjmp

code/ecf/setjmp.c

```
1  #include "csapp.h"
2
3  jmp_buf buf;
4
5  int error1 = 0;
6  int error2 = 1;
7
8  void foo(void), bar(void);
9
10 int main()
11 {
12     switch(setjmp(buf)) {
13     case 0:
14         foo();
15         break;
16     case 1:
17         printf("Detected an error1 condition in foo\n");
18         break;
19     case 2:
20         printf("Detected an error2 condition in foo\n");
21         break;
22     default:
23         printf("Unknown error condition in foo\n");
24     }
25     exit(0);
26 }
27
28 /* Deeply nested function foo */
29 void foo(void)
30 {
31     if (error1)
32         longjmp(buf, 1);
33     bar();
34 }
35
36 void bar(void)
37 {
38     if (error2)
39         longjmp(buf, 2);
40 }
```

code/ecf/setjmp.c

Figure 8.43 Nonlocal jump example. T i a l o t ra o r i o l o al t r o r r o rr o r o \ t i o i \ o l t \ t i o t o t a i t o i \ t t r ta .

code/ecf/restart.c

```
1   #include "csapp.h"
2
3   sigjmp_buf buf;
4
5   void handler(int sig)
6   {
7       siglongjmp(buf, 1);
8   }
9
10  int main()
11  {
12      if (!sigsetjmp(buf, 1)) {
13          Signal(SIGINT, handler);
14          Sio_puts("starting\n");
15      }
16      else
17          Sio_puts("restarting\n");
18
19      while(1) {
20          Sleep(1);
21          Sio_puts("processing...\n");
22      }
23      exit(0); /* Control never reaches here */
24  }
```

code/ecf/restart.c

Figure 8.44 A program that uses nonlocal jumps to restart itself when the user types Ctrl+C.

Another important application of nonlocal jumps is to branch out of a signal handler to a specific code location, rather than returning to the instruction that was interrupted by the arrival of the signal. Figure 8.44 shows a simple program that illustrates this basic technique. The program uses signals and nonlocal jumps to do a soft restart whenever the user types Ctrl+C at the keyboard. The `sigsetjmp` and `siglongjmp` functions are versions of `setjmp` and `longjmp` that can be used by signal handlers.

The initial call to the `sigsetjmp` function saves the calling environment and signal context (including the pending and blocked signal vectors) when the program first starts. The main routine then enters an infinite processing loop. When the user types Ctrl+C, the kernel sends a SIGINT signal to the process, which catches it. Instead of returning from the signal handler, which would pass control back to the interrupted processing loop, the handler performs a nonlocal jump back to the beginning of the `main` program. When we run the program on our system, we get the following output:

Aside Software exceptions in C++ and Java

The exception mechanisms provided by C++ and Java are higher-level, more structured versions of the C `setjmp` and `longjmp` functions. You can think of a `catch` clause inside a `try` statement as being akin to a `setjmp` function. Similarly, a `throw` statement is similar to a `longjmp` function.

```
linux> ./restart
starting
processing...
processing...
Ctrl+C
restarting
processing...
Ctrl+C
restarting
processing...
```

There a couple of interesting things about this program. First, To avoid a race, we must install the handler after we call `sigsetjmp`. If not, we would run the risk of the handler running before the initial call to `sigsetjmp` sets up the calling environment for `siglongjmp`. Second, you might have noticed that the `sigsetjmp` and `siglongjmp` functions are not on the list of async-signal-safe functions in Figure 8.33. The reason is that in general `siglongjmp` can jump into arbitrary code, so we must be careful to call only safe functions in any code reachable from a `siglongjmp`. In our example, we call the safe `sio_puts` and `sleep` functions. The unsafe `exit` function is unreachable.

8.7 Tools for Manipulating Processes

Linux systems provide a number of useful tools for monitoring and manipulating processes:

STRACE. Prints a trace of each system call invoked by a running program and its children. It is a fascinating tool for the curious student. Compile your program with `-static` to get a cleaner trace without a lot of output related to shared libraries.

PS. Lists processes (including zombies) currently in the system.

TOP. Prints information about the resource usage of current processes.

PMAP. Displays the memory map of a process.

`/proc`. A virtual filesystem that exports the contents of numerous kernel data structures in an ASCII text form that can be read by user programs. For example, type `cat /proc/loadavg` to see the current load average on your Linux system.

8.8 Summary

Exceptional control flow (ECF) occurs at all levels of a computer system and is a basic mechanism for providing concurrency in a computer system.

At the hardware level, exceptions are abrupt changes in the control flow that are triggered by events in the processor. The control flow passes to a software handler, which does some processing and then returns control to the interrupted control flow.

There are four different types of exceptions: interrupts, faults, aborts, and traps. Interrupts occur asynchronously (with respect to any instructions) when an external I/O device such as a timer chip or a disk controller sets the interrupt pin on the processor chip. Control returns to the instruction following the faulting instruction. Faults and aborts occur synchronously as the result of the execution of an instruction. Fault handlers restart the faulting instruction, while abort handlers never return control to the interrupted flow. Finally, traps are like function calls that are used to implement the system calls that provide applications with controlled entry points into the operating system code.

At the operating system level, the kernel uses ECF to provide the fundamental notion of a process. A process provides applications with two important abstractions: (1) logical control flows that give each program the illusion that it has exclusive use of the processor, and (2) private address spaces that provide the illusion that each program has exclusive use of the main memory.

At the interface between the operating system and applications, applications can create child processes, wait for their child processes to stop or terminate, run new programs, and catch signals from other processes. The semantics of signal handling is subtle and can vary from system to system. However, mechanisms exist on Posix-compliant systems that allow programs to clearly specify the expected signal-handling semantics.

Finally, at the application level, C programs can use nonlocal jumps to bypass the normal call/return stack discipline and branch directly from one function to another.

Bibliographic Notes

Kerrisk is the essential reference for all aspects of programming in the Linux environment [62]. The Intel ISA specification contains a detailed discussion of exceptions and interrupts on Intel processors [50]. Operating systems texts [102, 106, 113] contain additional information on exceptions, processes, and signals. The classic work by W. Richard Stevens [111] is a valuable and highly readable description of how to work with processes and signals from application programs. Bovet and Cesati [11] give a wonderfully clear description of the Linux kernel, including details of the process and signal implementations.

Homework Problems

8.9

Consider four processes with the following starting and ending times:

Process	Start time	End time
A	6	8
B	3	5
C	4	7
D	2	9

For each pair of processes, indicate whether they run concurrently (Y) or not (N):

Process pair	Concurrent?
AB	_____
AC	_____
AD	_____
BC	_____
BD	_____
CD	_____

8.10

In this chapter, we have introduced some functions with unusual call and return behaviors: `getenv`, `setenv`, `unsetenv`, and `execve`. Match each function with one of the following behaviors:

A. Called once, returns only if there is an error

B. Called once, returns nothing

C. Called once, returns either a pointer or `NULL`

8.11

How many "Example" output lines does this program print?

code/ecf/global-forkprob1.c

```
1   #include "csapp.h"
2
3   int main()
4   {
5       int i;
6
7       for (i = 3; i > 0; i--)
8           Fork();
9       printf("Example\n");
10      exit(0);
11  }
```

code/ecf/global-forkprob1.c

8.12
How many “Example” output lines does this program print?

code/ecf/global-forkprob4.c

```
#include "csapp.h"

void try()
{
    Fork();
    printf("Example\n");
    Fork();
    return;
}

int main()
{
    try(); Fork();
    printf("Example\n");
    exit(0);
}
```

code/ecf/global-forkprob4.c

8.13
What is one possible output of the following program?

code/ecf/global-forkprob3.c

```
#include "csapp.h"

int main()
{
    int a = 5;

    if (Fork() != 0)
        printf("a=%d\n", --a);

    printf("a=%d\n", ++a);
    exit(0);
}
```

code/ecf/global-forkprob3.c

8.14
How many “Example” output lines does this program print?

code/ecf/global-forkprob5.c

```
#include "csapp.h"

void try()
{
```

```
5       if (Fork() != 0) {
6           Fork();
7           printf("Example\n");
8           exit(0);
9       }
10      return;
11  }
12
13  int main()
14  {
15      try();
16      fork(); printf("Example\n");
17      exit(0);
18  }
```

code/ecf/global-forkprob5.c

8.15
How many "Example" lines does this program print?

code/ecf/global-forkprob6.c

```
1   #include "csapp.h"
2
3   void try()
4   {
5       if (Fork() == 0) {
6           Fork(); Fork();
7           printf("Example\n");
8           return;
9       }
10      return;
11  }
12
13  int main()
14  {
15      try();
16      printf("Example\n");
17      exit(0);
18  }
```

code/ecf/global-forkprob6.c

8.16
What is the output of the following program?

code/ecf/global-forkprob7.c

```
1   #include "csapp.h"
2   int counter = 1;
3
```

```
int main()
{
    if (fork() == 0) {
        counter++;
        exit(0);
    }
    else {
        Wait(NULL);
        counter++; printf("counter = %d\n", counter);
    }
    exit(0);
}
```

code/ecf/global-forkprob7.c

8.17

Enumerate all of the possible outputs of the program in Practice Problem 8.4.

8.18

Consider the following program:

code/ecf/forkprob2.c

```
#include "csapp.h"

void end(void)
{
    printf("2"); fflush(stdout);
}

int main()
{
    if (Fork() == 0)
        atexit(end);
    if (Fork() == 0) {
        printf("0"); fflush(stdout);
    }
    else {
        printf("1"); fflush(stdout);
    }
    exit(0);
}
```

code/ecf/forkprob2.c

Determine which of the following outputs are possible. Note: The `atexit` function takes a pointer to a function and adds it to a list of functions (initially empty) that will be called when the `exit` function is called.

A. 112002

B. 211020

C. 102120

D. 122001

E. 100212

8.19

How many lines of output does the following function print if the value of `n` entered by the user is 6?

code/ecf/global-forkprob8.c

```
1  void foo(int n)
2
```

```
4       if (fork() != 0) {
5           printf("q"); fflush(stdout);
6           return 0;
7       }
8       else {
9           printf("r"); fflush(stdout);
10          waitpid(-1, NULL, 0);
11      }
12      return 0;
13  }
```

code/ecf/global-waitprob3.c

8.22

Write your own version of the Unix `system` function

```
int mysystem(char *command);
```

The `mysystem` function executes `command` by invoking `/bin/sh -c command`, and then returns after `command` has completed. If `command` exits normally (by calling the `exit` function or executing a `return` statement), then `mysystem` returns the `command` exit status. For example, if `command` terminates by calling `exit(8)`, then `mysystem` returns the value 8. Otherwise, if `command` terminates abnormally, then `mysystem` returns the status returned by the shell.

8.23

One of your colleagues is thinking of using signals to allow a parent process to count events that occur in a child process. The idea is to notify the parent each time an event occurs by sending it a signal and letting the parent's signal handler increment a global `counter` variable, which the parent can then inspect after the child has terminated. However, when he runs the test program in Figure 8.45 on his system, he discovers that when the parent calls `printf`, `counter` always has a value of 2, even though the child has sent five signals to the parent. Perplexed, he comes to you for help. Can you explain the bug?

8.24

Modify the program in Figure 8.18 so that the following two conditions are met:

1. Each child terminates abnormally after attempting to write to a location in the read-only text segment.
2. The parent prints output that is identical (except for the PIDs) to the following:

```
child 12255 terminated by signal 11: Segmentation fault
child 12254 terminated by signal 11: Segmentation fault
```

Hint: Read the `man` page for `psignal(3)`.

code/ecf/counterprob.c

```
1   #include "csapp.h"
2
3   int counter = 0;
4
5   void handler(int sig)
6   {
7       counter++;
8       sleep(1); /* Do some work in the handler */
9       return;
10  }
11
12  int main()
13  {
14      int i;
15
16      Signal(SIGUSR2, handler);
17
18      if (Fork() == 0) {  /* Child */
19          for (i = 0; i < 5; i++) {
20              Kill(getppid(), SIGUSR2);
21              printf("sent SIGUSR2 to parent\n");
22          }
23          exit(0);
24      }
25
26      Wait(NULL);
27      printf("counter=%d\n", counter);
28      exit(0);
29  }
```

code/ecf/counterprob.c

Figure 8.45 Counter program referenced in Problem 8.23.

8.25

Write a version of the `fgets` function, called `tfgets`, that times out after 5 seconds. The `tfgets` function accepts the same inputs as `fgets`. If the user doesn't type an input line within 5 seconds, `tfgets` returns NULL. Otherwise, it returns a pointer to the input line.

8.26

Using the example in Figure 8.23 as a starting point, write a shell program that supports job control. Your shell should have the following features:

- The command line typed by the user consists of a `name` and zero or more arguments, all separated by one or more spaces. If `name` is a built-in command, the

shell handles it immediately and waits for the next command line. Otherwise, the shell assumes that `name` is an executable file, which it loads and runs in the context of an initial child process (job). The process group ID for the job is identical to the PID of the child.

- Each job is identified by either a process ID (PID) or a job ID (JID), which is a small arbitrary positive integer assigned by the shell. JIDs are denoted on the command line by the prefix '%'. For example, '%5' denotes JID 5, and '5' denotes PID 5.
- If the command line ends with an ampersand, then the shell runs the job in the background. Otherwise, the shell runs the job in the foreground.
- Typing Ctrl+C (Ctrl+Z) causes the kernel to send a SIGINT (SIGTSTP) signal to your shell, which then forwards it to every process in the foreground process group.[2]
- The `jobs` built-in command lists all background jobs.
- The `bg job` built-in command restarts `job` by sending it a SIGCONT signal and then runs it in the background. The `job` argument can be either a PID or a JID.
- The `fg job` built-in command restarts `job` by sending it a SIGCONT signal and then runs it in the foreground.
- The shell reaps all of its zombie children. If any job terminates because it receives a signal that was not caught, then the shell prints a message to the terminal with the job's PID and a description of the offending signal.

Figure 8.46 shows an example shell session.

Solutions to Practice Problems

Solution to Problem 8.1 (page 770)

Processes A and B are concurrent with respect to each other, as are B and C, because their respective executions overlap—that is, one process starts before the other finishes. Processes A and C are not concurrent because their executions do not overlap; A finishes before C begins.

Solution to Problem 8.2 (page 779)

In our example program in Figure 8.15, the parent and child execute disjoint sets of instructions. However, in this program, the parent and child execute nondisjoint sets of instructions, which is possible because the parent and child have identical code segments. This can be a difficult conceptual hurdle, so be sure you understand the solution to this problem. Figure 8.47 shows the process graph.

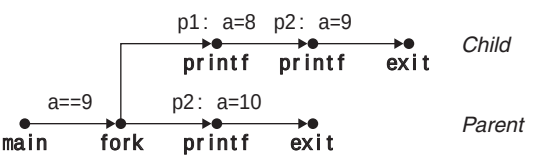
---

2. Note that this is a simplification of the way that real shells work. With real shells, the kernel responds to Ctrl+C (Ctrl+Z) by sending SIGINT (SIGTSTP) directly to each process in the terminal foreground process group. The shell manages the membership of this group using the `tcsetpgrp` function, and manages the attributes of the terminal using the `tcsetattr` function, both of which are outside the scope of this book. See [62] for details.

```
linux> ./shell                                  Run your shell program
>bogus
bogus: Command not found.                       Execve can't find executable
>foo 10
Job 5035 terminated by signal: Interrupt        User types Ctrl+C
>foo 100 &
[1] 5036 foo 100 &
>foo 200 &
[2] 5037 foo 200 &
>jobs
[1] 5036 Running    foo 100 &
[2] 5037 Running    foo 200 &
>fg %1
Job [1] 5036 stopped by signal: Stopped         User types Ctrl+Z
>jobs
[1] 5036 Stopped    foo 100 &
[2] 5037 Running    foo 200 &
>bg 5035
5035: No such process
>bg 5036
[1] 5036 foo 100 &
>/bin/kill 5036
Job 5036 terminated by signal: Terminated
> fg %2                                         Wait for fg job to finish
>quit
linux>                                          Back to the Unix shell
```

Figure 8.46 Sample shell session for Problem 8.26.

Figure 8.47
Process graph for Practice Problem 8.2.



A. The key idea here is that the child executes both `printf` statements. After the `fork` returns, it executes the `printf` in line 6. Then it falls out of the `if` statement and executes the `printf` in line 7. Here is the output produced by the child:

```
p1: a=8
p2: a=9
```

B. The parent executes only the `printf` in line 7:

```
p1: a=10
```

Figure 8.48
Process graph for Practice Problem 8.3.

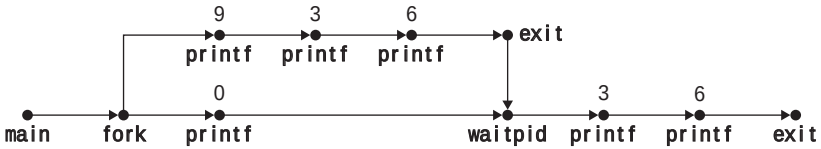
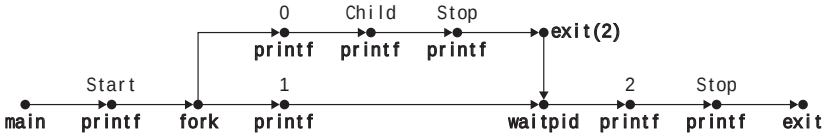


Figure 8.49
Process graph for Practice Problem 8.4.



Solution to Problem 8.3 (page 781)

We know that the sequences 936036, 903636, and 093636 are possible because they correspond to topological sorts of the process graph (Figure 8.48). However, sequences such as 036936 and 360369 do not correspond to any topological sort and thus are not feasible.

Solution to Problem 8.4 (page 784)

A. We can determine the number of lines of output by simply counting the number of `printf`

```
8       for (i=0; argv[i] != NULL; i++)
9           printf("    argv[%2d]: %s\n", i, argv[i]);
10
11      printf("\n");
12      printf("Environment variables:\n");
13      for (i=0; envp[i] != NULL; i++)
14          printf("    envp[%2d]: %s\n", i, envp[i]);
15
16      exit(0);
17  }
```

code/ecf/myecho.c

Solution to Problem 8.7 (page 800)

The `sleep` function returns prematurely whenever the sleeping process receives a signal that is not ignored. But since the default action upon receipt of a SIGINT is to terminate the process (Figure 8.26), we must install a SIGINT handler to allow the `sleep` function to return. The handler simply catches the SIGNAL and returns control to the `sleep` function, which returns immediately.

code/ecf/snooze.c

```
1   #include "csapp.h"
2
3   /* SIGINT handler */
4   void handler(int sig)
5   {
6       return; /* Catch the signal and return */
7   }
8
9   unsigned int snooze(unsigned int secs) {
10      unsigned int rc = sleep(secs);
11
12      printf("Slept for %d of %d secs.\n", secs-rc, secs);
13      return rc;
14  }
15
16  int main(int argc, char **argv) {
17
18      if (argc != 2) {
19          fprintf(stderr, "usage: %s <secs>\n", argv[0]);
20          exit(0);
21      }
22
23      if (signal(SIGINT, handler) == SIG_ERR) /* Install SIGINT */
24          unix_error("signal error\n");       /* handler        */
25      (void)snooze(atoi(argv[1]));
```

```
26        exit(0);
27    }
```

code/ecf/snooze.c

Solution to Problem 8.8 (page 809)

This program prints the string 213, which is the shorthand name of the CS:APP course at Carnegie Mellon. The parent starts by printing '2', then forks the child, which spins in an infinite loop. The parent then sends a signal to the child and waits for it to terminate. The child catches the signal (interrupting the infinite loop), decrements the counter (from an initial value of 2), prints '1', and then terminates. After the parent reaps the child, it increments the counter (from an initial value of 2), prints '3', and terminates.

9

Virtual Memory

9.1 P ial a Virt al Ar i 839

9.2 Ar S a 840

9.3 VM a a T ol or Ca i 841

9.4 VM a a T ol or M or Ma a t 847

9.5 VM a a T ol or M or Pr t ti 848

9.6 Ar Tra lat i 849

9.7 Ca St : T I t l C r /Li M or S t 861

9.8 M or Ma i 869

9.9 D a i M or All at i 875

9.10 Gar a C ll ti 901

9.11 C o M or -R lat B i C Pr ra 906

9.12 S ar 911

Bi li ra i N t 912

H or Pr l 912

S l ti t Pra ti Pr l 916

Processes in a system share the CPU and main memory with other processes. However, sharing the main memory poses some special challenges. As demand on the CPU increases, processes slow down in some reasonably smooth way. But if too many processes need too much memory, then some of them will simply not be able to run. When a program is out of space, it is out of luck. Memory is also vulnerable to corruption. If some process inadvertently writes to the memory used by another process, that process might fail in some bewildering fashion totally unrelated to the program logic.

In order to manage memory more efficiently and with fewer errors, modern systems provide an abstraction of main memory known as virtual memory (VM). Virtual memory is an elegant interaction of hardware exceptions, hardware address translation, main memory, disk files, and kernel software that provides each process with a large, uniform, and private address space. With one clean mechanism, virtual memory provides three important capabilities: (1) It uses main memory efficiently by treating it as a cache for an address space stored on disk, keeping only the active areas in main memory and transferring data back and forth between disk and memory as needed. (2) It simplifies memory management by providing each process with a uniform address space. (3) It protects the address space of each process from corruption by other processes.

Virtual memory is one of the great ideas in computer systems. A major reason for its success is that it works silently and automatically, without any intervention from the application programmer. Since virtual memory works so well behind the scenes, why would a programmer need to understand it? There are several reasons.

- Virtual memory is central. Virtual memory pervades all levels of computer systems, playing key roles in the design of hardware exceptions, assemblers, linkers, loaders, shared objects, files, and processes. Understanding virtual memory will help you better understand how systems work in general.
- Virtual memory is powerful. Virtual memory gives applications powerful capabilities to create and destroy chunks of memory, map chunks of memory to portions of disk files, and share memory with other processes. For example, did you know that you can read or modify the contents of a disk file by reading and writing memory locations? Or that you can load the contents of a file into memory without doing any explicit copying? Understanding virtual memory will help you harness its powerful capabilities in your applications.
- Virtual memory is dangerous. Applications interact with virtual memory every time they reference a variable, dereference a pointer, or make a call to a dynamic allocation package such as `malloc`. If virtual memory is used improperly, applications can suffer from perplexing and insidious memory-related bugs. For example, a program with a bad pointer can crash immediately with a "segmentation fault" or a "protection fault," run silently for hours before crashing, or scariest of all, run to completion with incorrect results. Understanding virtual memory, and the allocation packages such as `malloc` that manage it, can help you avoid these errors.

This chapter looks at virtual memory from two angles. The first half of the chapter describes how virtual memory works. The second half describes how virtual memory is used and managed by applications. There is no avoiding the fact that VM is complicated, and the discussion reflects this in places. The good news is that if you work through the details, you will be able to simulate the virtual memory mechanism of a small system by hand, and the virtual memory idea will be forever demystified.

The second half builds on this understanding, showing you how to use and manage virtual memory in your programs. You will learn how to manage virtual memory via explicit memory mapping and calls to dynamic storage allocators such as the `malloc` package. You will also learn about a host of common memory-related errors in C programs and how to avoid them.

9.1 Physical and Virtual Addressing

The main memory of a computer system is organized as an array of M contiguous byte-size cells. Each byte has a unique physical address (PA). The first byte has an address of 0, the next byte an address of 1, the next byte an address of 2, and so on. Given this simple organization, the most natural way for a CPU to access memory would be to use physical addresses. We call this approach physical addressing. Figure 9.1 shows an example of physical addressing in the context of a load instruction that reads the 4-byte word starting at physical address 4. When the CPU executes the load instruction, it generates an effective physical address and passes it to main memory over the memory bus. The main memory fetches the 4-byte word starting at physical address 4 and returns it to the CPU, which stores it in a register.

Early PCs used physical addressing, and systems such as digital signal processors, embedded microcontrollers, and Cray supercomputers continue to do so. However, modern processors use a form of addressing known as virtual addressing, as shown in Figure 9.2.

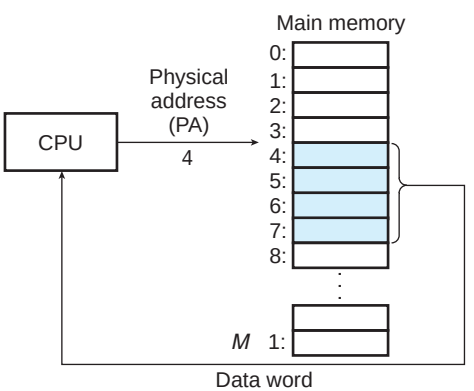
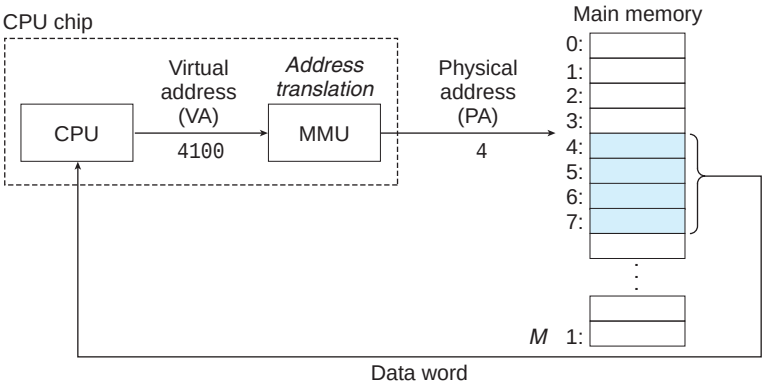


Figure 9.1
A system that uses physical addressing.

Figure 9.2
A system that uses virtual addressing.



With virtual addressing, the CPU accesses main memory by generating a virtual address (VA), which is converted to the appropriate physical address before being sent to main memory. The task of converting a virtual address to a physical one is known as address translation. Like exception handling, address translation requires close cooperation between the CPU hardware and the operating system. Dedicated hardware on the CPU chip called the memory management unit (MMU) translates virtual addresses on the fly, using a lookup table stored in main memory whose contents are managed by the operating system.

9.2 Address Spaces

An address space is an ordered set of nonnegative integer addresses

$$\{0, 1, 2, \ldots\}$$

If the integers in the address space are consecutive, then we say that it is a linear address space. To simplify our discussion, we will always assume linear address spaces. In a system with virtual memory, the CPU generates virtual addresses from an address space of $N = 2^n$ addresses called the virtual address space:

$$\{0, 1, 2, \ldots, N - 1\}$$

The size of an address space is characterized by the number of bits that are needed to represent the largest address. For example, a virtual address space with $N = 2^n$ addresses is called an n-bit address space. Modern systems typically support either 32-bit or 64-bit virtual address spaces.

A system also has a physical address space that corresponds to the M bytes of physical memory in the system:

$$\{0, 1, 2, \ldots, M - 1\}$$

M is not required to be a power of 2, but to simplify the discussion, we will assume that $M = 2^m$.

The concept of an address space is important because it makes a clean distinction between data objects (bytes) and their attributes (addresses). Once we recognize this distinction, then we can generalize and allow each data object to have multiple independent addresses, each chosen from a different address space. This is the basic idea of virtual memory. Each byte of main memory has a virtual address chosen from the virtual address space, and a physical address chosen from the physical address space.

Practice Problem 9.1 (solution page 916)

Complete the following table, filling in the missing entries and replacing each question mark with the appropriate integer. Use the following units: K = 2^{10} (kilo), M = 2^{20} (mega), G = 2^{30} (giga), T = 2^{40} (tera), P = 2^{50} (peta), or E = 2^{60} (exa).

Number of virtual address bits (n)	Number of virtual addresses (N)	Largest possible virtual address
4	_____	_____
_____	$2^{?}$ = 16 K	_____
_____	_____	$2^{24} - 1 = ?$ M − 1
_____	$2^{?}$ = 64 T	_____
54	_____	_____

9.3 VM as a Tool for Caching

Conceptually, a virtual memory is organized as an array of N contiguous byte-size cells stored on disk. Each byte has a unique virtual address that serves as an index into the array. The contents of the array on disk are cached in main memory. As with any other cache in the memory hierarchy, the data on disk (the lower level) is partitioned into blocks that serve as the transfer units between the disk and the main memory (the upper level). VM systems handle this by partitioning the virtual memory into fixed-size blocks called virtual pages (VPs). Each virtual page is P = 2^p bytes in size. Similarly, physical memory is partitioned into physical pages (PPs), also P bytes in size. (Physical pages are also referred to as page frames.)

At any point in time, the set of virtual pages is partitioned into three disjoint subsets:

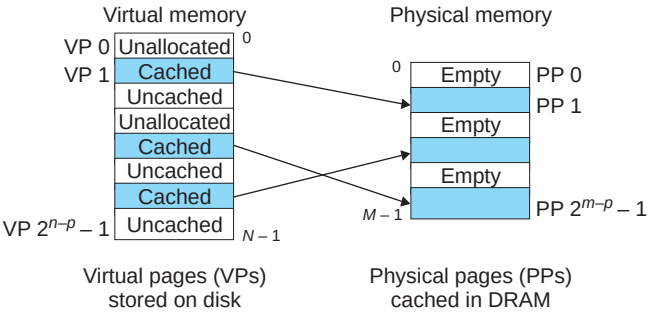
Unallocated. Pages that have not yet been allocated (or created) by the VM system. Unallocated blocks do not have any data associated with them, and thus do not occupy any space on disk.

Cached. Allocated pages that are currently cached in physical memory.

Uncached. Allocated pages that are not cached in physical memory.

The example in Figure 9.3 shows a small virtual memory with eight virtual pages. Virtual pages 0 and 3 have not been allocated yet, and thus do not yet exist

Figure 9.3
How a VM system uses main memory as a cache.



on disk. Virtual pages 1, 4, and 6 are cached in physical memory. Pages 2, 5, and 7 are allocated but are not currently cached in physical memory.

9.3.1 DRAM Cache Organization

To help us keep the different caches in the memory hierarchy straight, we will use the term SRAM cache to denote the L1, L2, and L3 cache memories between the CPU and main memory, and the term DRAM cache to denote the VM system's cache that caches virtual pages in main memory.

The position of the DRAM cache in the memory hierarchy has a big impact on the way that it is organized. Recall that a DRAM is at least 10 times slower than an SRAM and that disk is about 100,000 times slower than a DRAM. Thus, misses in DRAM caches are very expensive compared to misses in SRAM caches because DRAM cache misses are served from disk, while SRAM cache misses are usually served from DRAM-based main memory. Further, the cost of reading the first byte from a disk sector is about 100,000 times slower than reading successive bytes in the sector. The bottom line is that the organization of the DRAM cache is driven entirely by the enormous cost of misses.

Because of the large miss penalty and the expense of accessing the first byte, virtual pages tend to be large—typically 4 KB to 2 MB. Due to the large miss penalty, DRAM caches are fully associative; that is, any virtual page can be placed in any physical page. The replacement policy on misses also assumes greater importance, because the penalty associated with replacing the wrong virtual page is so high. Thus, operating systems use much more sophisticated replacement algorithms for DRAM caches than the hardware does for SRAM caches. (These replacement algorithms are beyond our scope here.) Finally, because of the large access time of disk, DRAM caches always use write-back instead of write-through.

9.3.2 Page Tables

As with any cache, the VM system must have some way to determine if a virtual page is cached somewhere in DRAM. If so, the system must determine which physical page it is cached in. If there is a miss, the system must determine

Figure 9.4
Page table.

n	P = 2^p	Number of PTEs
12	1 K	_____
16	16 K	_____
24	2 M	_____
36	1 G	_____

9.3.3 Page Hits

Consider what happens when the CPU reads a word of virtual memory contained in VP 2, which is cached in DRAM (Figure 9.5). Using a technique we will describe in detail in Section 9.6, the address translation hardware uses the virtual address as an index to locate PTE 2 and read it from memory. Since the valid bit is set, the address translation hardware knows that VP 2 is cached in memory. So it uses the physical memory address in the PTE (which points to the start of the cached page in PP 1) to construct the physical address of the word.

9.3.4 Page Faults

In virtual memory parlance, a DRAM cache miss is known as a page fault. Figure 9.6 shows the state of our example page table before the fault. The CPU has referenced a word in VP 3, which is not cached in DRAM. The address translation hardware reads PTE 3 from memory, infers from the valid bit that VP 3 is not cached, and triggers a page fault exception. The page fault exception invokes a page fault exception handler in the kernel, which selects a victim page—in this case, VP 4 stored in PP 3. If VP 4 has been modified, then the kernel copies it back to disk. In either case, the kernel modifies the page table entry for VP 4 to reflect the fact that VP 4 is no longer cached in main memory.

Figure 9.5
VM page hit. T r r t o a o r i VP 2 i a t.

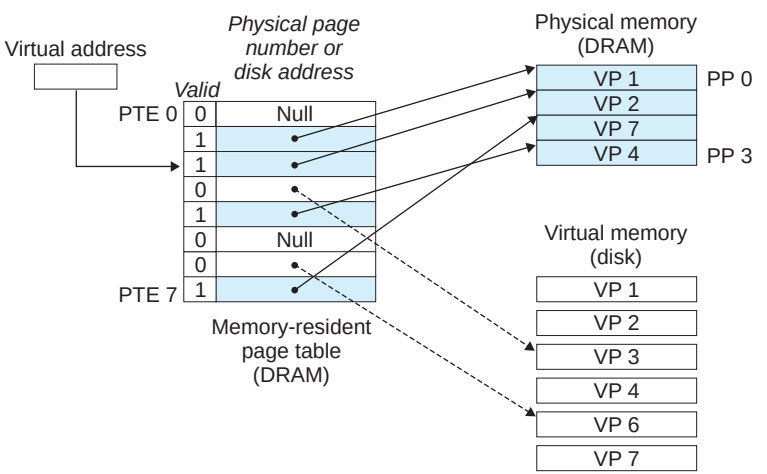


Figure 9.6
VM page fault (before). T r r t a o r i VP 3 i a i a \ tri r a a a lt.

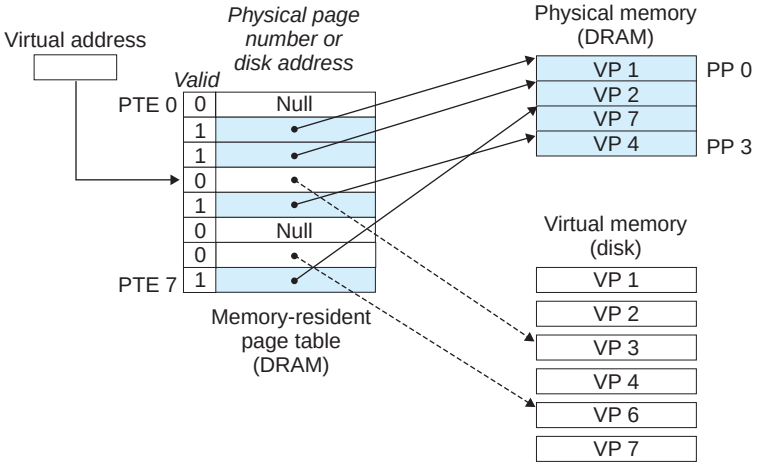
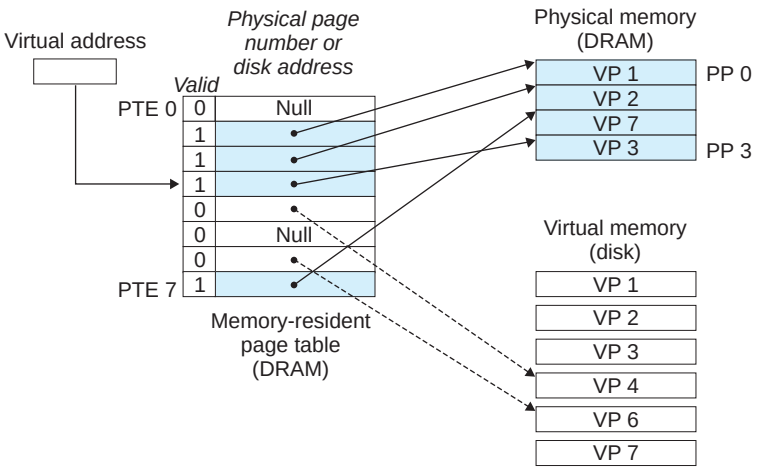


Figure 9.7
VM page fault (after). T a a lt a \ l r l t VP 4 a t iti a \ r la it it a o VP 3 r o \ i . A t r t a a lt a \ l r r tart t a lti i tr ti , it ll r a \ t o r o r o r all , it o t rati a ti o .



Next, the kernel copies VP 3 from disk to PP 3 in memory, updates PTE 3, and then returns. When the handler returns, it restarts the faulting instruction, which resends the faulting virtual address to the address translation hardware. But now, VP 3 is cached in main memory, and the page hit is handled normally by the address translation hardware. Figure 9.7 shows the state of our example page table after the page fault.

Virtual memory was invented in the early 1960s, long before the widening CPU-memory gap spawned SRAM caches. As a result, virtual memory systems use a different terminology from SRAM caches, even though many of the ideas are similar. In virtual memory parlance, blocks are known as pages. The activity of transferring a page between disk and memory is known as swapping or paging. Pages are swapped in (paged in) from disk to DRAM, and swapped out (paged out) from DRAM to disk. The strategy of waiting until the last moment to swap

Figure 9.8
Allocating a new virtual page. T r l all at VP 5 i a i t PTE 5 t t i l ati .

Aside Counting page faults

You can monitor the number of page faults (and lots of other information) with the Linux `getrusage` function.

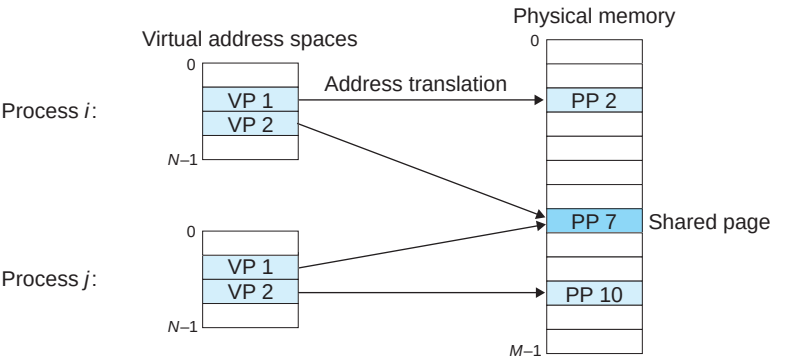


Figure 9.9 How VM provides processes with separate address spaces. The operating system maintains a separate page table for each process in this example.

9.4 VM as a Tool for Memory Management

In the last section, we saw how virtual memory provides a mechanism for using the DRAM to cache pages from a typically larger virtual address space. Interestingly, some early systems such as the DEC PDP-11/70 supported a virtual address space that was smaller than the available physical memory. Yet virtual memory was still a useful mechanism because it greatly simplified memory management and provided a natural way to protect memory.

Thus far, we have assumed a single page table that maps a single virtual address space to the physical address space. In fact, operating systems provide a separate page table, and thus a separate virtual address space, for each process. Figure 9.9 shows the basic idea. In the example, the page table for process i maps VP 1 to PP 2 and VP 2 to PP 7. Similarly, the page table for process j maps VP 1 to PP 7 and VP 2 to PP 10. Notice that multiple virtual pages can be mapped to the same shared physical page.

The combination of demand paging and separate virtual address spaces has a profound impact on the way that memory is used and managed in a system. In particular, VM simplifies linking and loading, the sharing of code and data, and allocating memory to applications.

- Simplifying linking. A separate address space allows each process to use the same basic format for its memory image, regardless of where the code and data actually reside in physical memory. For example, as we saw in Figure 8.13, every process on a given Linux system has a similar memory format. For 64-bit address spaces, the code segment always starts at virtual address `0x400000`. The data segment follows the code segment after a suitable alignment gap. The stack occupies the highest portion of the user process address space and

grows downward. Such uniformity greatly simplifies the design and implementation of linkers, allowing them to produce fully linked executables that are independent of the ultimate location of the code and data in physical memory.

- Simplifying loading. Virtual memory also makes it easy to load executable and shared object files into memory. To load the `.text` and `.data` sections of an object file into a newly created process, the Linux loader allocates virtual pages for the code and data segments, marks them as invalid (i.e., not cached), and points their page table entries to the appropriate locations in the object file. The interesting point is that the loader never actually copies any data from disk into memory. The data are paged in automatically and on demand by the virtual memory system the first time each page is referenced, either by the CPU when it fetches an instruction or by an executing instruction when it references a memory location.

  This notion of mapping a set of contiguous virtual pages to an arbitrary location in an arbitrary file is known as memory mapping. Linux provides a system call called `mmap` that allows application programs to do their own memory mapping. We will describe application-level memory mapping in more detail in Section 9.8.

- Simplifying sharing. Separate address spaces provide the operating system with a consistent mechanism for managing sharing between user processes and the operating system itself. In general, each process has its own private code, data, heap, and stack areas that are not shared with any other process. In this case, the operating system creates page tables that map the corresponding virtual pages to disjoint physical pages.

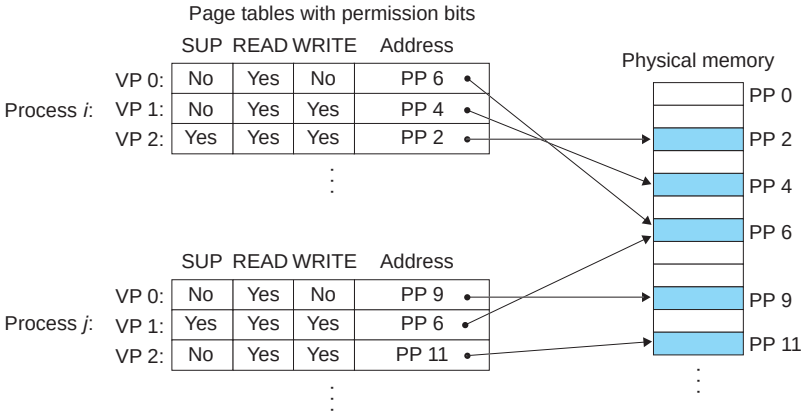
  However, in some instances it is desirable for processes to share code and data. For example, every process must call the same operating system kernel code, and every C program makes calls to routines in the standard C library such as `printf`. Rather than including separate copies of the kernel and standard C library in each process, the operating system can arrange for multiple processes to share a single copy of this code by mapping the appropriate virtual pages in different processes to the same physical pages, as we saw in Figure 9.9.

- Simplifying memory allocation. Virtual memory provides a simple mechanism for allocating additional memory to user processes. When a program running in a user process requests additional heap space (e.g., as a result of calling `malloc`), the operating system allocates an appropriate number, say, `k`, of contiguous virtual memory pages, and maps them to `k` arbitrary physical pages located anywhere in physical memory. Because of the way page tables work, there is no need for the operating system to locate `k` contiguous pages of physical memory. The pages can be scattered randomly in physical memory.

9.5 VM as a Tool for Memory Protection

Any modern computer system must provide the means for the operating system to control access to the memory system. A user process should not be allowed

Figure 9.10
Using VM to provide page-level memory protection.



to modify its read-only code section. Nor should it be allowed to read or modify any of the code and data structures in the kernel. It should not be allowed to read or write the private memory of other processes, and it should not be allowed to modify any virtual pages that are shared with other processes, unless all parties explicitly allow it (via calls to explicit interprocess communication system calls).

As we have seen, providing separate virtual address spaces makes it easy to isolate the private memories of different processes. But the address translation mechanism can be extended in a natural way to provide even finer access control. Since the address translation hardware reads a PTE each time the CPU generates an address, it is straightforward to control access to the contents of a virtual page by adding some additional permission bits to the PTE. Figure 9.10 shows the general idea.

In this example, we have added three permission bits to each PTE. The SUP bit indicates whether processes must be running in kernel (supervisor) mode to access the page. Processes running in kernel mode can access any page, but processes running in user mode are only allowed to access pages for which SUP is 0. The READ and WRITE bits control read and write access to the page. For example, if process i is running in user mode, then it has permission to read VP 0 and to read or write VP 1. However, it is not allowed to access VP 2.

If an instruction violates these permissions, then the CPU triggers a general protection fault that transfers control to an exception handler in the kernel, which sends a SIGSEGV signal to the offending process. Linux shells typically report this exception as a "segmentation fault."

9.6 Address Translation

This section covers the basics of address translation. Our aim is to give you an appreciation of the hardware's role in supporting virtual memory, with enough detail so that you can work through some concrete examples by hand. However, keep in mind that we are omitting a number of details, especially related to timing,

Symbol	Description
Basic parameters	
$N = 2^n$	Number of addresses in virtual address space
$M = 2^m$	Number of addresses in physical address space
$P = 2^p$	Page size (bytes)
Components of a virtual address (VA)	
VPO	Virtual page offset (bytes)
VPN	Virtual page number
TLBI	TLB index
TLBT	TLB tag
Components of a physical address (PA)	
PPO	Physical page offset (bytes)
PPN	Physical page number
CO	Byte offset within cache block
CI	Cache index
CT	Cache tag

Figure 9.11 Summary of address translation symbols.

that are important to hardware designers but are beyond our scope. For your reference, Figure 9.11 summarizes the symbols that we will be using throughout this section.

Formally, address translation is a mapping between the elements of an N-element virtual address space (VAS) and an M-element physical address space (PAS),

$$\text{MAP: VAS} \rightarrow \text{PAS}$$

where

$$\text{MAP(A)} = \begin{cases} \text{A} & \text{if data at virtual addr. A are present at physical addr. A in PAS} \\ & \text{if data at virtual addr. A are not present in physical memory} \end{cases}$$

Figure 9.12 shows how the MMU uses the page table to perform this mapping. A control register in the CPU, the page table base register (PTBR) points to the current page table. The n-bit virtual address has two components: a p-bit virtual page offset (VPO) and an $(n - p)$-bit virtual page number (VPN). The MMU uses the VPN to select the appropriate PTE. For example, VPN 0 selects PTE 0, VPN 1 selects PTE 1, and so on. The corresponding physical address is the concatenation of the physical page number (PPN) from the page table entry and the VPO from the virtual address. Notice that since the physical and virtual pages are both P bytes, the physical page offset (PPO) is identical to the VPO.

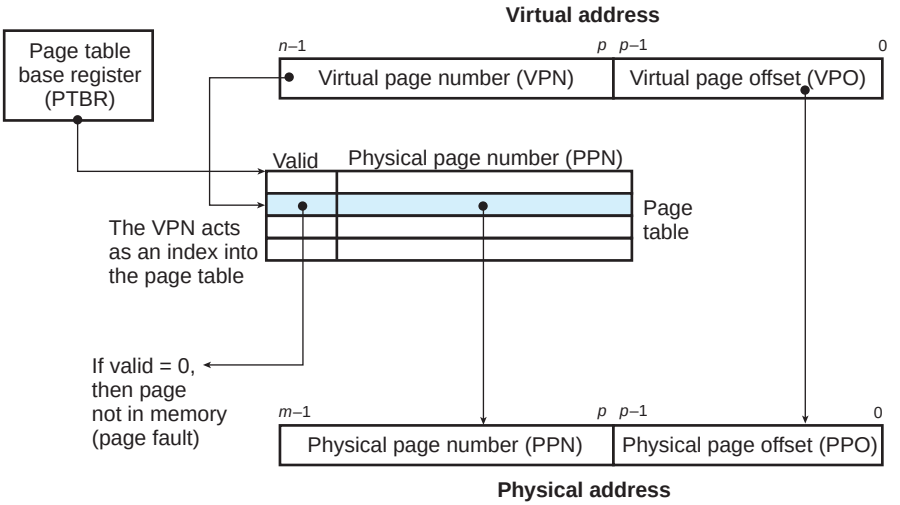


Figure 9.12 Address translation with a page table.

Figure 9.13(a) shows the steps that the CPU hardware performs when there is a page hit.

Step 1. The processor generates a virtual address and sends it to the MMU.

Step 2. The MMU generates the PTE address and requests it from the cache/main memory.

Step 3. The cache/main memory returns the PTE to the MMU.

Step 4. The MMU constructs the physical address and sends it to the cache/main memory.

Step 5. The cache/main memory returns the requested data word to the processor.

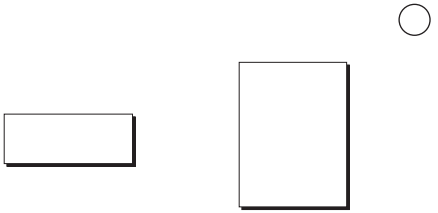
Unlike a page hit, which is handled entirely by hardware, handling a page fault requires cooperation between hardware and the operating system kernel (Figure 9.13(b)).

Steps 1 to 3. The same as steps 1 to 3 in Figure 9.13(a).

Step 4. The valid bit in the PTE is zero, so the MMU triggers an exception, which transfers control in the CPU to a page fault exception handler in the operating system kernel.

Step 5. The fault handler identifies a victim page in physical memory, and if that page has been modified, pages it out to disk.

Step 6. The fault handler pages in the new page and updates the PTE in memory.



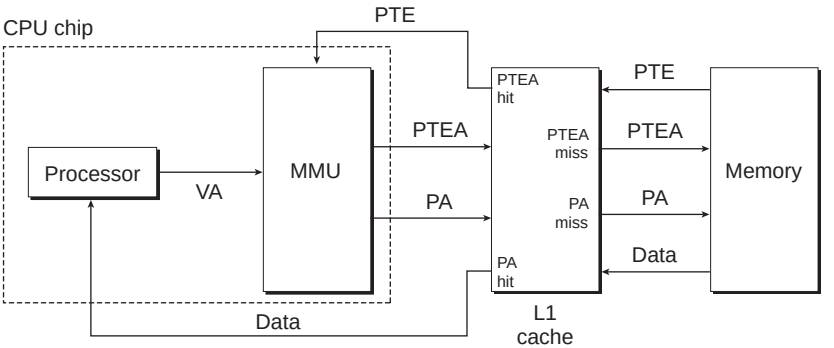


Figure 9.14 Integrating VM with a physically addressed cache. VA: virtual address. PTEA: page table entry address. PTE: page table entry. PA: physical address.

9.6.1 Integrating Caches and VM

In any system that uses both virtual memory and SRAM caches, there is the issue of whether to use virtual or physical addresses to access the SRAM cache. Although a detailed discussion of the trade-offs is beyond our scope here, most systems opt for physical addressing. With physical addressing, it is straightforward for multiple processes to have blocks in the cache at the same time and to share blocks from the same virtual pages. Further, the cache does not have to deal with protection issues, because access rights are checked as part of the address translation process.

Figure 9.14 shows how a physically addressed cache might be integrated with virtual memory. The main idea is that the address translation occurs before the cache lookup. Notice that page table entries can be cached, just like any other data words.

9.6.2 Speeding Up Address Translation with a TLB

As we have seen, every time the CPU generates a virtual address, the MMU must refer to a PTE in order to translate the virtual address into a physical address. In the worst case, this requires an additional fetch from memory, at a cost of tens to hundreds of cycles. If the PTE happens to be cached in L1, then the cost goes down to a handful of cycles. However, many systems try to eliminate even this cost by including a small cache of PTEs in the MMU called a translation lookaside buffer (TLB).

A TLB is a small, virtually addressed cache where each line holds a block consisting of a single PTE. A TLB usually has a high degree of associativity. As shown in Figure 9.15, the index and tag fields that are used for set selection and line matching are extracted from the virtual page number in the virtual address. If the TLB has $T = 2^t$ sets, then the TLB index (TLBI) consists of the t least significant bits of the VPN, and the TLB tag (TLBT) consists of the remaining bits in the VPN.

Figure 9.15
Components of a virtual address that are used to access the TLB.

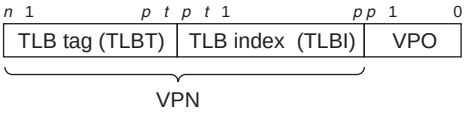
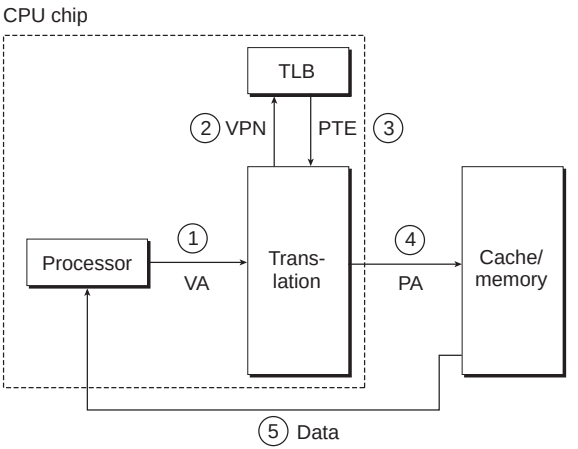
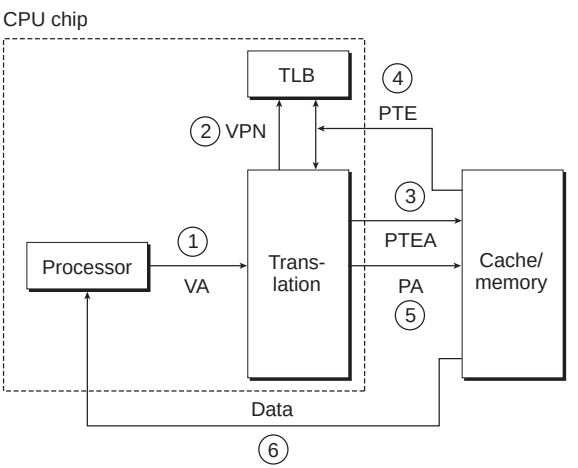


Figure 9.16
Operational view of a TLB hit and miss.



(a) TLB hit



(b) TLB miss

Figure 9.16(a) shows the steps involved when there is a TLB hit (the usual case). The key point here is that all of the address translation steps are performed inside the on-chip MMU and thus are fast.

Step 1. The CPU generates a virtual address.

Steps 2 and 3. The MMU fetches the appropriate PTE from the TLB.

Step 4. The MMU translates the virtual address to a physical address and sends it to the cache/main memory.

Step 5. The cache/main memory returns the requested data word to the CPU.

When there is a TLB miss, then the MMU must fetch the PTE from the L1 cache, as shown in Figure 9.16(b). The newly fetched PTE is stored in the TLB, possibly overwriting an existing entry.

9.6.3 Multi-Level Page Tables

Thus far, we have assumed that the system uses a single page table to do address translation. But if we had a 32-bit address space, 4 KB pages, and a 4-byte PTE, then we would need a 4 MB page table resident in memory at all times, even if the application referenced only a small chunk of the virtual address space. The problem is compounded for systems with 64-bit address spaces.

The common approach for compacting the page table is to use a hierarchy of page tables instead. The idea is easiest to understand with a concrete example. Consider a 32-bit virtual address space partitioned into 4 KB pages, with page table entries that are 4 bytes each. Suppose also that at this point in time the virtual address space has the following form: The first 2 K pages of memory are allocated for code and data, the next 6 K pages are unallocated, the next 1,023 pages are also unallocated, and the next page is allocated for the user stack. Figure 9.17 shows how we might construct a two-level page table hierarchy for this virtual address space.

Each PTE in the level 1 table is responsible for mapping a 4 MB chunk of the virtual address space, where each chunk consists of 1,024 contiguous pages. For example, PTE 0 maps the first chunk, PTE 1 the next chunk, and so on. Given that the address space is 4 GB, 1,024 PTEs are sufficient to cover the entire space.

If every page in chunk i is unallocated, then level 1 PTE i is null. For example, in Figure 9.17, chunks 2–7 are unallocated. However, if at least one page in chunk i is allocated, then level 1 PTE i points to the base of a level 2 page table. For example, in Figure 9.17, all or portions of chunks 0, 1, and 8 are allocated, so their level 1 PTEs point to level 2 page tables.

Each PTE in a level 2 page table is responsible for mapping a 4-KB page of virtual memory, just as before when we looked at single-level page tables. Notice that with 4-byte PTEs, each level 1 and level 2 page table is 4 kilobytes, which conveniently is the same size as a page.

This scheme reduces memory requirements in two ways. First, if a PTE in the level 1 table is null, then the corresponding level 2 page table does not even have to exist. This represents a significant potential savings, since most of the 4 GB virtual address space for a typical program is unallocated. Second, only the level 1 table needs to be in main memory at all times. The level 2 page tables can be created and paged in and out by the VM system as they are needed, which reduces pressure on main memory. Only the most heavily used level 2 page tables need to be cached in main memory.

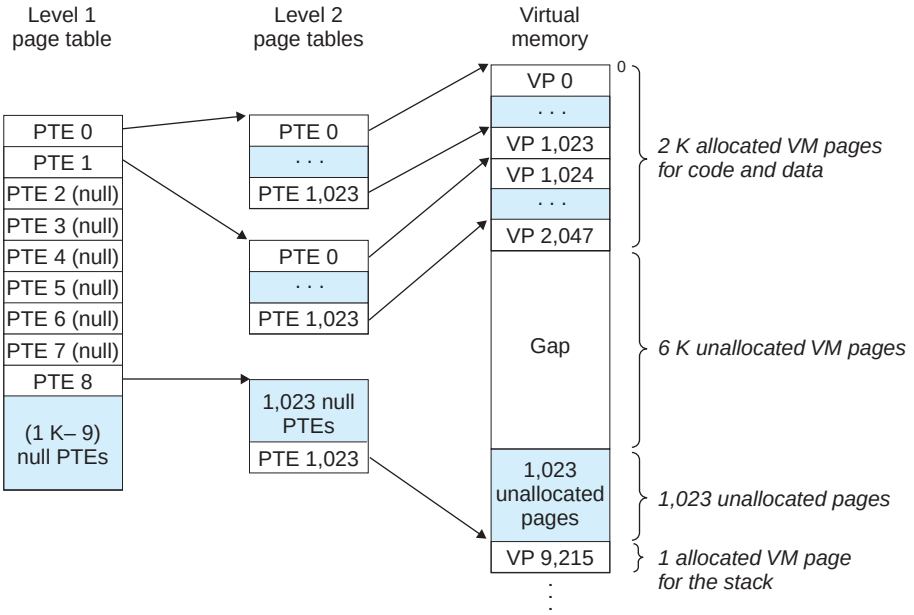


Figure 9.17 A two-level page table hierarchy. N[illegible]

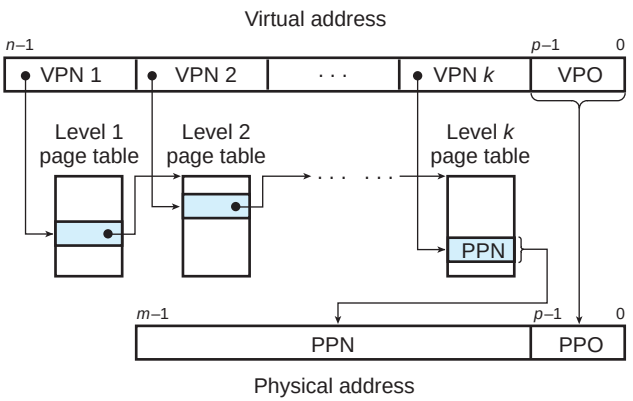


Figure 9.18 Address translation with a k-level page table.

Figure 9.18 summarizes address translation with a k-level page table hierarchy. The virtual address is partitioned into k VPNs and a VPO. Each VPN i, $1 \le i \le k$, is an index into a page table at level i. Each PTE in a level j table, $1 \le j \le k - 1$, points to the base of some page table at level $j + 1$. Each PTE in a level k table contains either the PPN of some physical page or the address of a disk block. To construct the physical address, the MMU must access k PTEs before it can

determine the PPN. As with a single-level hierarchy, the PPO is identical to the VPO.

Accessing k PTEs may seem expensive and impractical at first glance. However, the TLB comes to the rescue here by caching PTEs from the page tables at the different levels. In practice, address translation with multi-level page tables is not significantly slower than with single-level page tables.

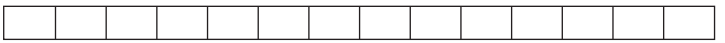
9.6.4 Putting It Together: End-to-End Address Translation

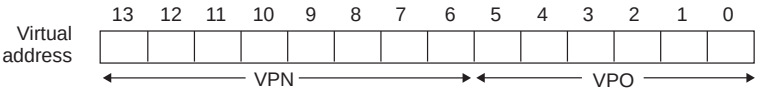
In this section, we put it all together with a concrete example of end-to-end address translation on a small system with a TLB and L1 d-cache. To keep things manageable, we make the following assumptions:

- The memory is byte addressable.
- Memory accesses are to 1-byte words (not 4-byte words).
- Virtual addresses are 14 bits wide ($n = 14$).
- Physical addresses are 12 bits wide ($m = 12$).
- The page size is 64 bytes ($P = 64$).
- The TLB is 4-way set associative with 16 total entries.
- The L1 d-cache is physically addressed and direct mapped, with a 4-byte line size and 16 total sets.

Figure 9.19 shows the formats of the virtual and physical addresses. Since each page is $2^6 = 64$ bytes, the low-order 6 bits of the virtual and physical addresses serve as the VPO and PPO, respectively. The high-order 8 bits of the virtual address serve as the VPN. The high-order 6 bits of the physical address serve as the PPN.

Figure 9.20 shows a snapshot of our little memory system, including the TLB (Figure 9.20(a)), a portion of the page table (Figure 9.20(b)), and the L1 cache (Figure 9.20(c)). Above the figures of the TLB and cache, we have also shown how the bits of the virtual and physical addresses are partitioned by the hardware as it accesses these devices.





	Tag											
0	03	–	0	09	0D	1	00	–	0	07	02	1
1	03	2D	1	02	–	0	04	–	0	0A	–	
2	02	–	0	08	–	0	06	–	0	03	–	
3	07	–	0	03	0D	1	0A	34	1	02	–	

(a) TLB: 4 sets, 16 entries, 4-way set associative

TLB. The TLB is virtually addressed using the bits of the VPN. Since the TLB has four sets, the 2 low-order bits of the VPN serve as the set index (TLBI). The remaining 6 high-order bits serve as the tag (TLBT) that distinguishes the different VPNs that might map to the same TLB set.

Page table. The page table is a single-level design with a total of 2wit6249(2)]C74_2 1 Tf 9.9626 0 0 9

	CT						CI				CO	
	0x0d						0x05				0x0	
Bit position	11	10	9	8	7	6	5	4	3	2	1	0
PA = 0x354	0	0	1	1	0	1	0	1	0	1	0	0
	PPN						PPO					
	0x0d						0x14					

Since the tag in set 0x5 matches CT, the cache detects a hit, reads out the data byte (0x36) at offset CO, and returns it to the MMU, which then passes it back to the CPU.

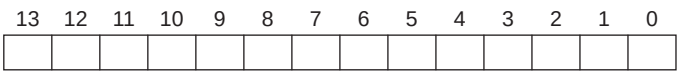
Other paths through the translation process are also possible. For example, if the TLB misses, then the MMU must fetch the PPN from a PTE in the page table. If the resulting PTE is invalid, then there is a page fault and the kernel must page in the appropriate page and rerun the load instruction. Another possibility is that the PTE is valid, but the necessary memory block misses in the cache.

Practice Problem 9.4 (solution page 917)

Show how the example memory system in Section 9.6.4 translates a virtual address into a physical address and accesses the cache. For the given virtual address, indicate the TLB entry accessed, physical address, and cache byte value returned. Indicate whether the TLB misses, whether a page fault occurs, and whether a cache miss occurs. If there is a cache miss, enter "—" for "Cache byte returned." If there is a page fault, enter "—" for "PPN" and leave parts C and D blank.

Virtual address: 0x03d7

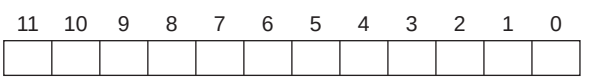
A. Virtual address format



B. Address translation

Parameter	Value
VPN	_____
TLB index	_____
TLB tag	_____
TLB hit? (Y/N)	_____
Page fault? (Y/N)	_____
PPN	_____

C. Physical address format



D. Physical memory reference

Parameter	Value
Byte offset	
Cache index	
Cache tag	
Cache hit? (Y/N)	
Cache byte returned	

9.7 Case Study: The Intel Core i7/Linux Memory System

We conclude our discussion of virtual memory mechanisms with a case study of a real system: an Intel Core i7 running Linux. Although the underlying Haswell microarchitecture allows for full 64-bit virtual and physical address spaces, the current Core i7 implementations (and those for the foreseeable future) support a 48-bit (256 TB) virtual address space and a 52-bit (4 PB) physical address space, along with a compatibility mode that supports 32-bit (4 GB) virtual and physical address spaces.

Figure 9.21 gives the highlights of the Core i7 memory system. The processor package (chip) includes four cores, a large L3 cache shared by all of the cores, and

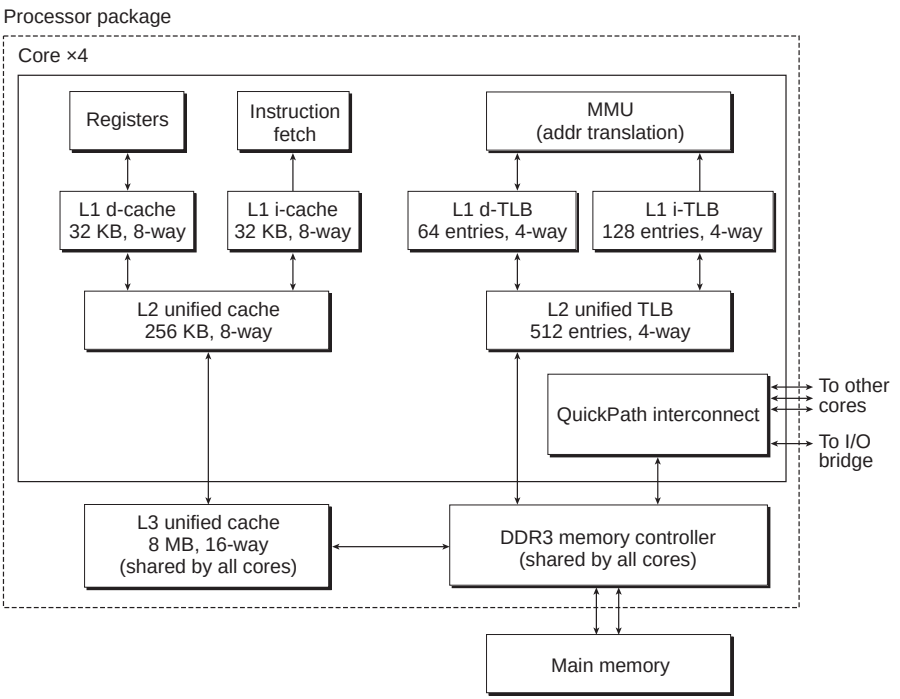


Figure 9.21 The Core i7 memory system.

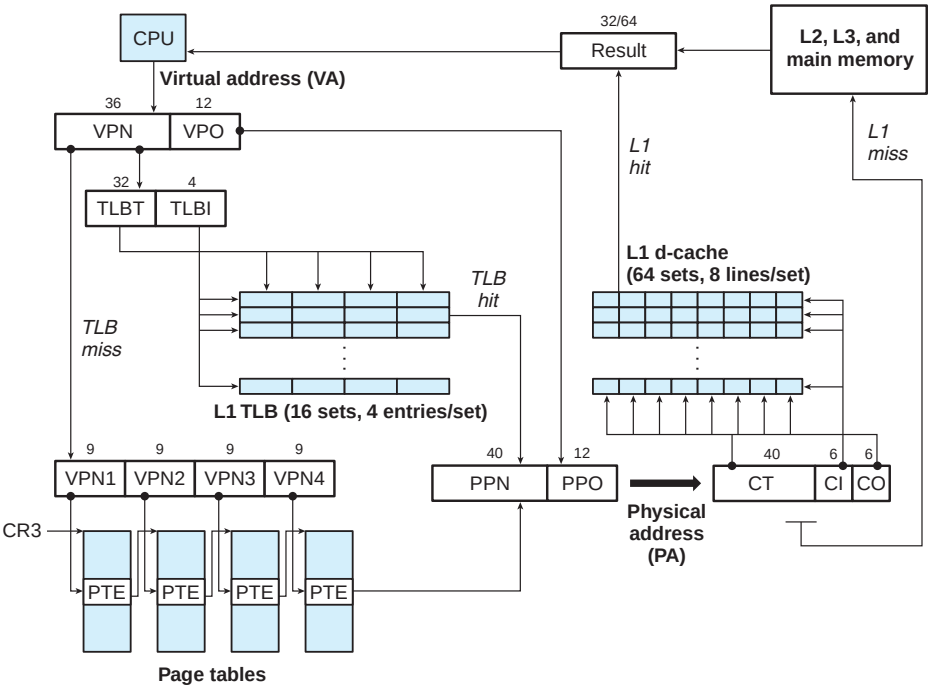
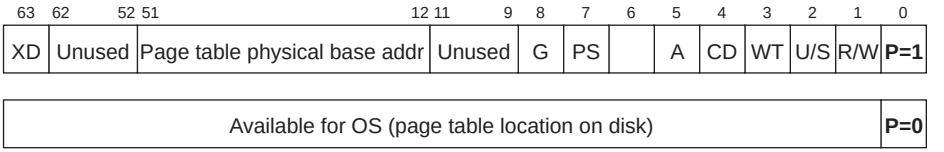


Figure 9.22 Summary of Core i7 address translation. For instructions, the instruction TLB, a unified L2 unified TLB are omitted.

a DDR3 memory controller. Each core contains a hierarchy of TLBs, a hierarchy of data and instruction caches, and a set of fast point-to-point links, based on the QuickPath technology, for communicating directly with the other cores and the external I/O bridge. The TLBs are virtually addressed, and 4-way set associative. The L1, L2, and L3 caches are physically addressed, with a block size of 64 bytes. L1 and L2 are 8-way set associative, and L3 is 16-way set associative. The page size can be configured at start-up time as either 4 KB or 4 MB. Linux uses 4 KB pages.

9.7.1 Core i7 Address Translation

Figure 9.22 summarizes the entire Core i7 address translation process, from the time the CPU generates a virtual address until a data word arrives from memory. The Core i7 uses a four-level page table hierarchy. Each process has its own private page table hierarchy. When a Linux process is running, the page tables associated with allocated pages are all memory-resident, although the Core i7 architecture allows these page tables to be swapped in and out. The CR3 control register contains the physical address of the beginning of the level 1 (L1) page table. The value of CR3 is part of each process context, and is restored during each context switch.



Field	Description
P	Child page table present in physical memory (1) or not (0).
R/W	Read-only or read-write access permission for all reachable pages.
U/S	User or supervisor (kernel) mode access permission for all reachable pages.
WT	Write-through or write-back cache policy for the child page table.
CD	Caching disabled or enabled for the child page table.
A	Reference bit (set by MMU on reads and writes, cleared by software).
PS	Page size either 4 KB or 4 MB (defined for level 1 PTEs only).
Base addr	40 most significant bits of physical base address of child page table.
XD	Disable or enable instruction fetches from all pages reachable from this PTE.

Figure 9.23 Format of level 1, level 2, and level 3 page table entries. Ea tr r r a 4 KB t a ta l .

Figure 9.23 shows the format of an entry in a level 1, level 2, or level 3 page table. When P = 1 (which is always the case with Linux), the address field contains a 40-bit physical page number (PPN) that points to the beginning of the appropriate page table. Notice that this imposes a 4 KB alignment requirement on page tables.

Figure 9.24 shows the format of an entry in a level 4 page table. When P = 1, the address field contains a 40-bit PPN that points to the base of some page in physical memory. Again, this imposes a 4 KB alignment requirement on physical pages.

The PTE has three permission bits that control access to the page. The R/W bit determines whether the contents of a page are read/write or read-only. The U/S bit, which determines whether the page can be accessed in user mode, protects code and data in the operating system kernel from user programs. The XD (execute disable) bit, which was introduced in 64-bit systems, can be used to disable instruction fetches from individual memory pages. This is an important new feature that allows the operating system kernel to reduce the risk of buffer overflow attacks by restricting execution to the read-only code segment.

As the MMU translates each virtual address, it also updates two other bits that can be used by the kernel's page fault handler. The MMU sets the A bit, which is known as a reference bit, each time a page is accessed. The kernel can use the reference bit to implement its page replacement algorithm. The MMU sets the D bit, or dirty bit, each time the page is written to. A page that has been modified is sometimes called a dirty page. The dirty bit tells the kernel whether or not it must

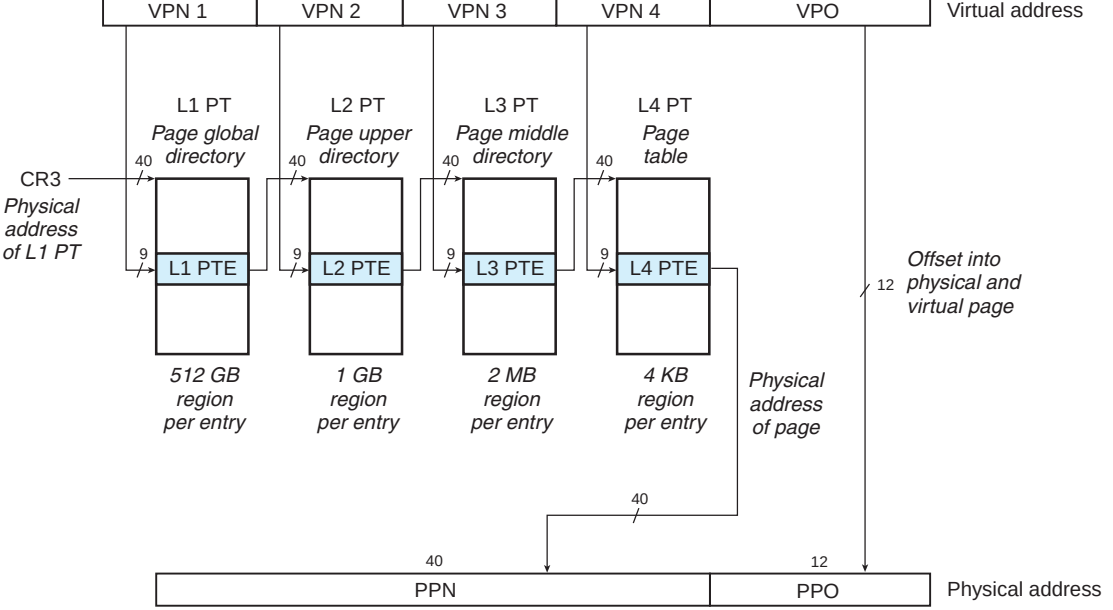
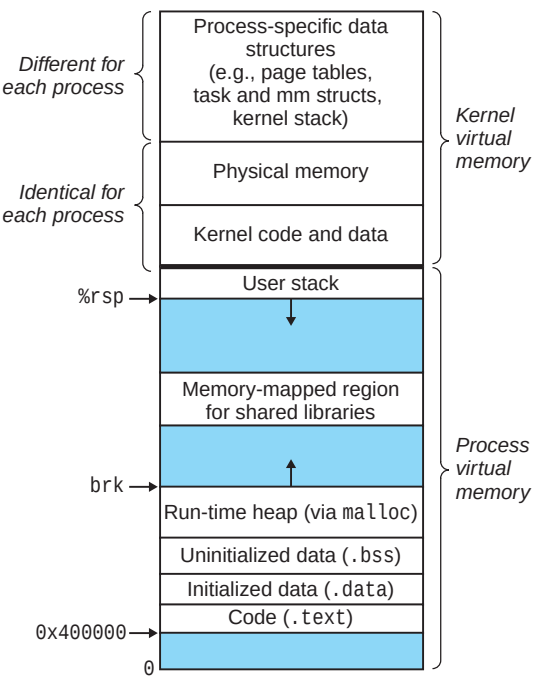


Figure 9.25 Core i7 page table translation. PT: a ta l ; PTE: a ta l tr ; VPN: irt al a r; VPO: irt al a o t; PPN: i al a r; PPO: i al a o t. T Li a o r t o r l l o a ta l ar al o o .

Figure 9.26
The virtual memory of a Linux process.



Aside Optimizing address translation

In our discussion of address translation, we have described a sequential two-step process where the MMU (1) translates the virtual address to a physical address and then (2) passes the physical address to the L1 cache. However, real hardware implementations use a neat trick that allows these steps to be partially overlapped, thus speeding up accesses to the L1 cache. For example, a virtual address on a Core i7 with 4 KB pages has 12 bits of VPO, and these bits are identical to the 12 bits of PPO in the corresponding physical address. Since the 8-way set associative physically addressed L1 caches have 64 sets and 64-byte cache blocks, each physical address has 6 ($\log_2 64$) cache offset bits and 6 ($\log_2 64$) index bits. These 12 bits fit exactly in the 12-bit VPO of a virtual address, which is no accident! When the CPU needs a virtual address translated, it sends the VPN to the MMU and the VPO to the L1 cache. While the MMU is requesting a page table entry from the TLB, the L1 cache is busy using the VPO bits to find the appropriate set and read out the eight tags and corresponding data words in that set. When the MMU gets the PPN back from the TLB, the cache is ready to try to match the PPN to one of these eight tags.

are shared by all processes. For example, each process shares the kernel's code and global data structures. Interestingly, Linux also maps a set of contiguous virtual pages (equal in size to the total amount of DRAM in the system) to the corresponding set of contiguous physical pages. This provides the kernel with a convenient way to access any specific location in physical memory—for example, when it needs to access page tables or to perform memory-mapped I/O operations on devices that are mapped to particular physical memory locations.

Other regions of kernel virtual memory contain data that differ for each process. Examples include page tables, the stack that the kernel uses when it is executing code in the context of the process, and various data structures that keep track of the current organization of the virtual address space.

Linux Virtual Memory Areas

Linux organizes the virtual memory as a collection of **areas** (also called **segments**). An area is a contiguous chunk of existing (allocated) virtual memory whose pages are related in some way. For example, the code segment, data segment, heap, shared library segment, and user stack are all distinct areas. Each existing virtual page is contained in some area, and any virtual page that is not part of some area does not exist and cannot be referenced by the process. The notion of an area is important because it allows the virtual address space to have gaps. The kernel does not keep track of virtual pages that do not exist, and such pages do not consume any additional resources in memory, on disk, or in the kernel itself.

Figure 9.27 highlights the kernel data structures that keep track of the virtual memory areas in a process. The kernel maintains a distinct task structure (`task_struct` in the source code) for each process in the system. The elements of the task structure either contain or point to all of the information that the kernel needs to

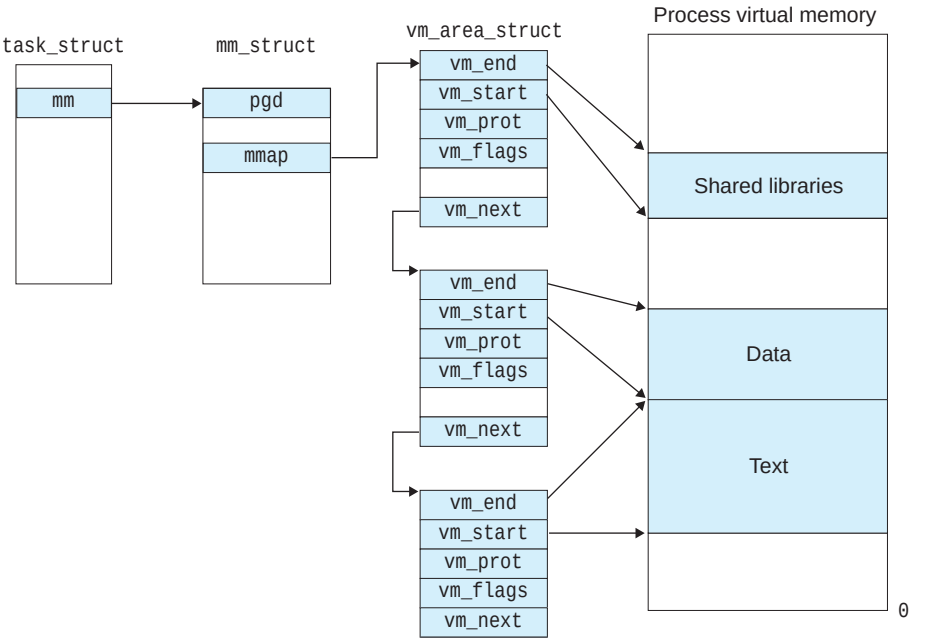


Figure 9.27 How Linux organizes virtual memory.

run the process (e.g., the PID, pointer to the user stack, name of the executable object file, and program counter).

One of the entries in the task structure points to an `mm_struct` that characterizes the current state of the virtual memory. The two fields of interest to us are `pgd`, which points to the base of the level 1 table (the page global directory), and `mmap`, which points to a list of `vm_area_structs` (area structs), each of which characterizes an area of the current virtual address space. When the kernel runs this process, it stores `pgd` in the CR3 control register.

For our purposes, the area struct for a particular area contains the following fields:

`fvm_start`. Points to the beginning of the area.

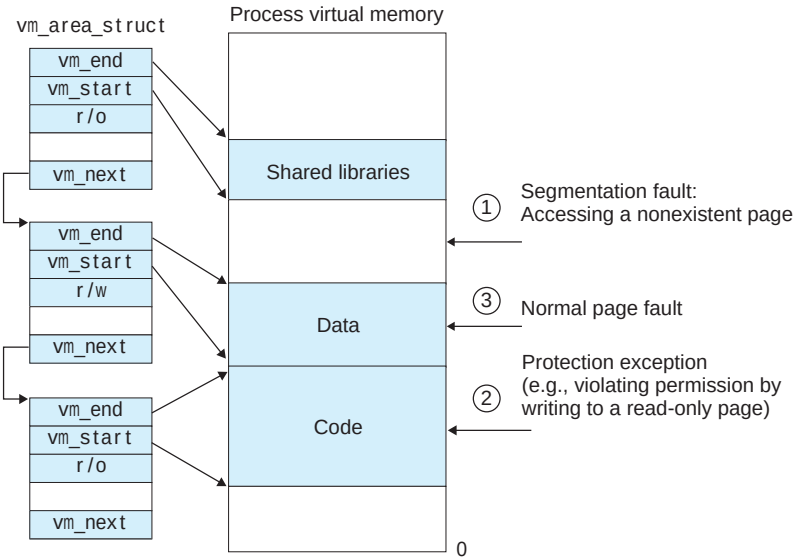
`vm_end`. Points to the end of the area.

`vm_prot`. Describes the read/write permissions for all of the pages contained in the area.

`vm_flags`. Describes (among other things) whether the pages in the area are shared with other processes or private to this process.

`vm_next`. Points to the next area struct in the list.

Figure 9.28
Linux page fault handling.



Linux Page Fault Handling

Suppose the MMU triggers a page fault while trying to translate some virtual address A. The exception results in a transfer of control to the kernel's page fault handler, which then performs the following steps:

1. Is virtual address A legal? In other words, does A lie within an area defined by some area struct? To answer this question, the fault handler searches the list of area structs, comparing A with the `vm_start` and `vm_end` in each area struct. If the instruction is not legal, then the fault handler triggers a segmentation fault, which terminates the process. This situation is labeled "1" in Figure 9.28.

   Because a process can create an arbitrary number of new virtual memory areas (using the `mmap` function described in the next section), a sequential search of the list of area structs might be very costly. So in practice, Linux superimposes a tree on the list, using some fields that we have not shown, and performs the search on this tree.

2. Is the attempted memory access legal? In other words, does the process have permission to read, write, or execute the pages in this area? For example, was the page fault the result of a store instruction trying to write to a read-only page in the code segment? Is the page fault the result of a process running in user mode that is attempting to read a word from kernel virtual memory? If the attempted access is not legal, then the fault handler triggers a protection exception, which terminates the process. This situation is labeled "2" in Figure 9.28.

3. At this point, the kernel knows that the page fault resulted from a legal operation on a legal virtual address. It handles the fault by selecting a victim page, swapping out the victim page if it is dirty, swapping in the new page,

and updating the page table. When the page fault handler returns, the CPU restarts the faulting instruction, which sends A to the MMU again. This time, the MMU translates A normally, without generating a page fault.

9.8 Memory Mapping

Linux initializes the contents of a virtual memory area by associating it with an *object* on disk, a process known as *memory mapping*. Areas can be mapped to one of two types of objects:

1. *Regular file in the Linux file system*: An area can be mapped to a contiguous section of a regular disk file, such as an executable object file. The file section is divided into page-size pieces, with each piece containing the initial contents of a virtual page. Because of demand paging, none of these virtual pages is actually swapped into physical memory until the CPU first *touches* the page (i.e., issues a virtual address that falls within that page's region of the address space). If the area is larger than the file section, then the area is padded with zeros.
2. *Anonymous file*: An area can also be mapped to an anonymous file, created by the kernel, that contains all binary zeros. The first time the CPU touches a virtual page in such an area, the kernel finds an appropriate victim page in physical memory, swaps out the victim page if it is dirty, overwrites the victim page with binary zeros, and updates the page table to mark the page as resident. Notice that no data are actually transferred between disk and memory. For this reason, pages in areas that are mapped to anonymous files are sometimes called *demand-zero pages*.

In either case, once a virtual page is initialized, it is swapped back and forth between a special *swap file* maintained by the kernel. The swap file is also known as the *swap space* or the *swap area*. An important point to realize is that at any point in time, the swap space bounds the total amount of virtual pages that can be allocated by the currently running processes.

9.8.1 Shared Objects Revisited

The idea of memory mapping resulted from a clever insight that if the virtual memory system could be integrated into the conventional file system, then it could provide a simple and efficient way to load programs and data into memory.

As we have seen, the process abstraction promises to provide each process with its own private virtual address space that is protected from errant writes or reads by other processes. However, many processes have identical read-only code areas. For example, each process that runs the Linux shell program `bash` has the same code area. Further, many programs need to access identical copies of read-only run-time library code. For example, every C program requires functions from the standard C library such as `printf`. It would be extremely wasteful for each process to keep duplicate copies of these commonly used codes in physical

memory. Fortunately, memory mapping provides us with a clean mechanism for controlling how objects are shared by multiple processes.

An object can be mapped into an area of virtual memory as either a shared object or a private object. If a process maps a shared object into an area of its virtual address space, then any writes that the process makes to that area are visible to any other processes that have also mapped the shared object into their virtual memory. Further, the changes are also reflected in the original object on disk.

Changes made to an area mapped to a private object, on the other hand, are not visible to other processes, and any writes that the process makes to the area are not reflected back to the object on disk. A virtual memory area into which a shared object is mapped is often called a shared area. Similarly for a private area.

Suppose that process 1 maps a shared object into an area of its virtual memory, as shown in Figure 9.29(a). Now suppose that process 2 maps the same shared ob-

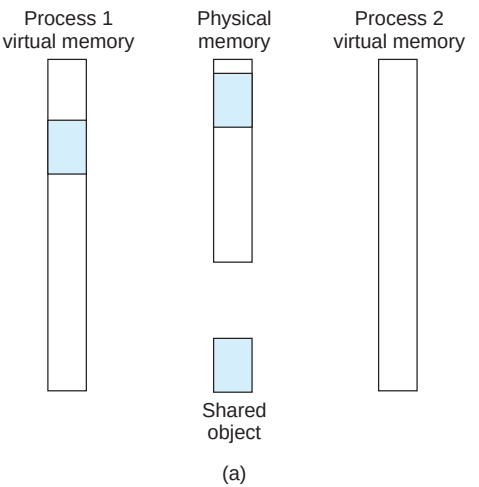
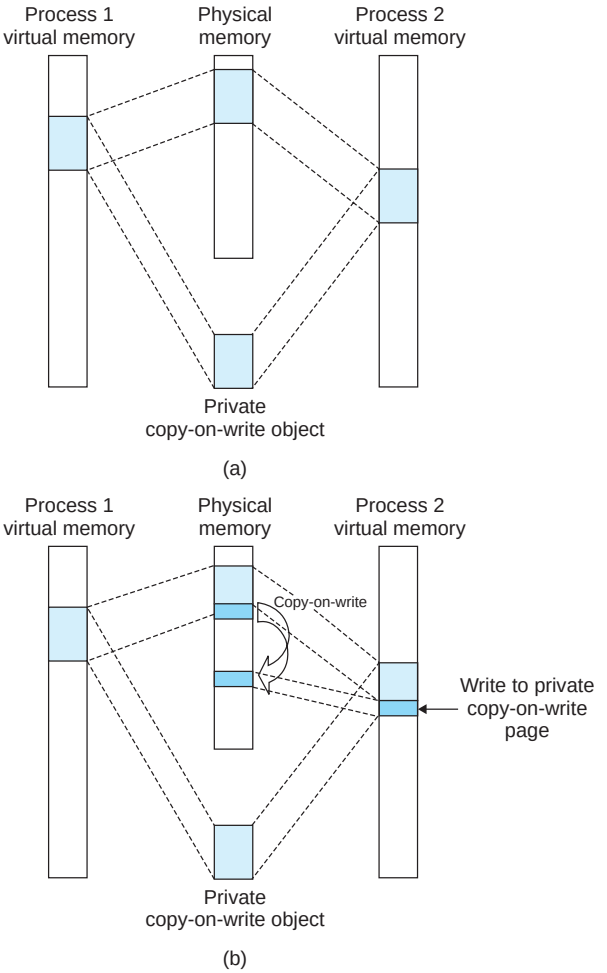


Figure 9.29
A shared object. (a) A t r r 1 a t ar t. ( ) A t r r 2 a t a ar t. (N t t at t i al a ar t ari ti .)

Figure 9.30
A private copy-on-write object. (a) After process 1 maps the private object. (b) After process 2 maps the same private object.



ject into its address space (not necessarily at the same virtual address as process 1), as shown in Figure 9.29(b).

Since each object has a unique filename, the kernel can quickly determine that process 1 has already mapped this object and can point the page table entries in process 2 to the appropriate physical pages. The key point is that only a single copy of the shared object needs to be stored in physical memory, even though the object is mapped into multiple shared areas. For convenience, we have shown the physical pages as being contiguous, but of course this is not true in general.

Private objects are mapped into virtual memory using a clever technique known as copy-on-write. A private object begins life in exactly the same way as a shared object, with only one copy of the private object stored in physical memory. For example, Figure 9.30(a) shows a case where two processes have mapped a private object into different areas of their virtual memories but share the same

physical copy of the object. For each process that maps the private object, the page table entries for the corresponding private area are flagged as read-only, and the area struct is flagged as private copy-on-write. So long as neither process attempts to write to its respective private area, they continue to share a single copy of the object in physical memory. However, as soon as a process attempts to write to some page in the private area, the write triggers a protection fault.

When the fault handler notices that the protection exception was caused by the process trying to write to a page in a private copy-on-write area, it creates a new copy of the page in physical memory, updates the page table entry to point to the new copy, and then restores write permissions to the page, as shown in Figure 9.30(b). When the fault handler returns, the CPU re-executes the write, which now proceeds normally on the newly created page.

By deferring the copying of the pages in private objects until the last possible moment, copy-on-write makes the most efficient use of scarce physical memory.

9.8.2 The `fork` Function Revisited

Now that we understand virtual memory and memory mapping, we can get a clear idea of how the `fork` function creates a new process with its own independent virtual address space.

When the `fork` function is called by the current process, the kernel creates various data structures for the new process and assigns it a unique PID. To create the virtual memory for the new process, it creates exact copies of the current process's `mm_struct`, area structs, and page tables. It flags each page in both processes as read-only, and flags each area struct in both processes as private copy-on-write.

When the `fork` returns in the new process, the new process now has an exact copy of the virtual memory as it existed when the fork was called. When either of the processes performs any subsequent writes, the copy-on-write mechanism creates new pages, thus preserving the abstraction of a private address space for each process.

9.8.3 The `execve` Function Revisited

Virtual memory and memory mapping also play key roles in the process of loading programs into memory. Now that we understand these concepts, we can understand how the `execve` function really loads and executes programs. Suppose that the program running in the current process makes the following call:

```
execve("a.out", NULL, NULL);
```

As you learned in Chapter 8, the `execve` function loads and runs the program contained in the executable object file `a.out` within the current process, effectively replacing the current program with the `a.out` program. Loading and running `a.out` requires the following steps:

Figure 9.31
How the loader maps the areas of the user address space.

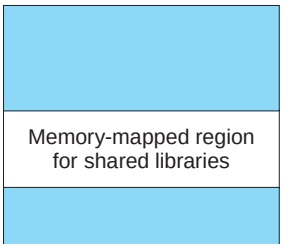
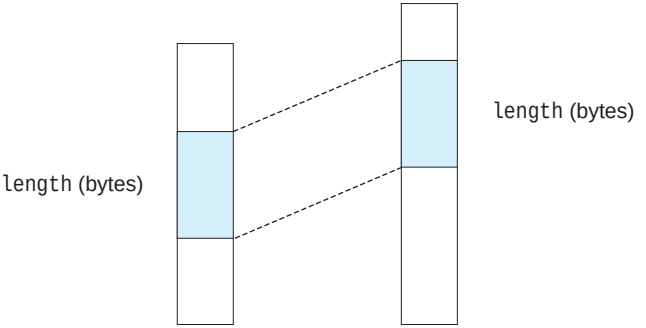


Figure 9.32
Visual interpretation of `mmap` arguments.



asks the kernel to create a new read-only, private, demand-zero area of virtual memory containing `size` bytes. If the call is successful, then `bufp` contains the address of the new area.

The `munmap` function deletes regions of virtual memory:

```
#include <unistd.h>
#include <sys/mman.h>

int munmap(void *start, size_t length);
```

Returns: 0 if OK, −1 on error

The `munmap` function deletes the area starting at virtual address `start` and consisting of the next `length` bytes. Subsequent references to the deleted region result in segmentation faults.

Practice Problem 9.5 (solution page 918)

Write a C program `mmapcopy.c` that uses `mmap` to copy an arbitrary-size disk file to `stdout`. The name of the input file should be passed as a command-line argument.

9.9 Dynamic Memory Allocation

While it is certainly possible to use the low-level `mmap` and `munmap` functions to create and delete areas of virtual memory, C programmers typically find it more convenient and more portable to use a dynamic memory allocator when they need to acquire additional virtual memory at run time.

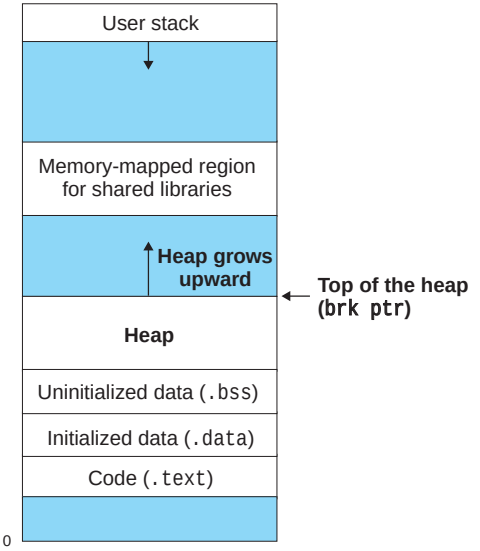
A dynamic memory allocator maintains an area of a process's virtual memory known as the heap (Figure 9.33). Details vary from system to system, but without loss of generality, we will assume that the heap is an area of demand-zero memory that begins immediately after the uninitialized data area and grows upward (toward higher addresses). For each process, the kernel maintains a variable `brk` (pronounced "break") that points to the top of the heap.

An allocator maintains the heap as a collection of various-size blocks. Each block is a contiguous chunk of virtual memory that is either allocated or free. An allocated block has been explicitly reserved for use by the application. A free block is available to be allocated. A free block remains free until it is explicitly allocated by the application. An allocated block remains allocated until it is freed, either explicitly by the application or implicitly by the memory allocator itself.

Allocators come in two basic styles. Both styles require the application to explicitly allocate blocks. They differ about which entity is responsible for freeing allocated blocks.

- Explicit allocators require the application to explicitly free any allocated blocks. For example, the C standard library provides an explicit allocator called the `malloc` package. C programs allocate a block by calling the `malloc`

Figure 9.33
The heap.



function, and free a block by calling the `free` function. The `new` and `delete` calls in C++ are comparable.

Implicit allocators, on the other hand, require the allocator to detect when an allocated block is no longer being used by the program and then free the block. Implicit allocators are also known as garbage collectors, and the process of automatically freeing unused allocated blocks is known as garbage collection. For example, higher-level languages such as Lisp, ML, and Java rely on garbage collection to free allocated blocks.

The remainder of this section discusses the design and implementation of explicit allocators. We will discuss implicit allocators in Section 9.10. For concreteness, our discussion focuses on allocators that manage heap memory. However, you should be aware that memory allocation is a general idea that arises in a variety of contexts. For example, applications that do intensive manipulation of graphs will often use the standard allocator to acquire a large block of virtual memory and then use an application-specific allocator to manage the memory within that block as the nodes of the graph are created and destroyed.

9.9.1 The `malloc` and `free` Functions

The C standard library provides an explicit allocator known as the `malloc` package. Programs allocate blocks from the heap by calling the `malloc` function.

```
#include <stdlib.h>

void *malloc(size_t size);
```

Returns: pointer to allocated block if OK, NULL on error

Aside H i i a ?

Recall from our discussion of machine code in Chapter 3 that Intel refers to 4-byte objects as double words. However, throughout this section, we will assume that words are 4-byte objects and that double words are 8-byte objects, which is consistent with conventional terminology.

The `malloc` function returns a pointer to a block of memory of at least `size` bytes that is suitably aligned for any kind of data object that might be contained in the block. In practice, the alignment depends on whether the code is compiled to run in 32-bit mode (`gcc -m32`) or 64-bit mode (the default). In 32-bit mode, `malloc` returns a block whose address is always a multiple of 8. In 64-bit mode, the address is always a multiple of 16.

If `malloc` encounters a problem (e.g., the program requests a block of memory that is larger than the available virtual memory), then it returns NULL and sets `errno`. `Malloc` does not initialize the memory it returns. Applications that want initialized dynamic memory can use `calloc`, a thin wrapper around the `malloc` function that initializes the allocated memory to zero. Applications that want to change the size of a previously allocated block can use the `realloc` function.

Dynamic memory allocators such as `malloc` can allocate or deallocate heap memory explicitly by using the `mmap` and `munmap` functions, or they can use the `sbrk` function:

```
#include <unistd.h>

void *sbrk(intptr_t incr);
```

Returns: old `brk` pointer on success, −1 on error

The `sbrk` function grows or shrinks the heap by adding `incr` to the kernel's `brk` pointer. If successful, it returns the old value of `brk`, otherwise it returns −1 and sets `errno` to ENOMEM. If `incr` is zero, then `sbrk` returns the current value of `brk`. Calling `sbrk` with a negative `incr` is legal but tricky because the return value (the old value of `brk`) points to `abs(incr)` bytes past the new top of the heap.

Programs free allocated heap blocks by calling the `free` function.

```
#include <stdlib.h>

void free(void *ptr);
```

Returns: nothing

The `ptr` argument must point to the beginning of an allocated block that was obtained from `malloc`, `calloc`, or `realloc`. If not, then the behavior of `free` is undefined. Even worse, since it returns nothing, `free` gives no indication to the application that something is wrong. As we shall see in Section 9.11, this can produce some baffling run-time errors.

Figure 9.34
Allocating and freeing blocks with malloc and free. Ea ar rr o t a o . Ea a r ta l rr t a l o . All at l ar a . Pa r i all at l ar a t a ar r l . Fr l ar a . H a r i r a r l t t ri t.

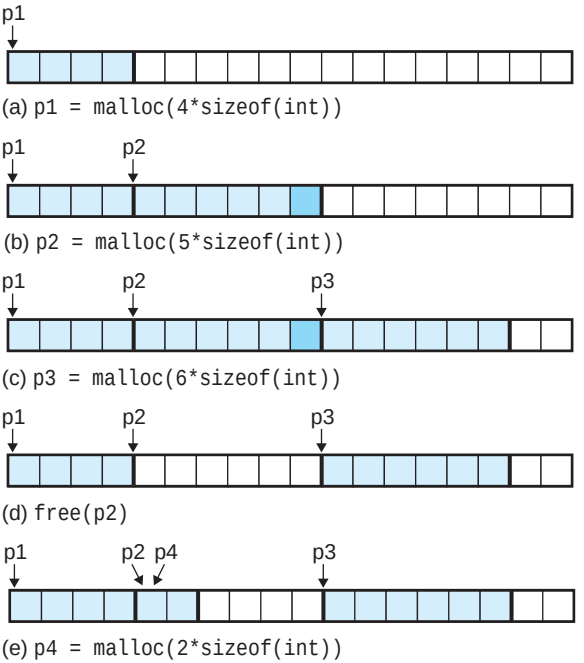


Figure 9.34 shows how an implementation of `malloc` and `free` might manage a (very) small heap of 16 words for a C program. Each box represents a 4-byte word. The heavy-lined rectangles correspond to allocated blocks (shaded) and free blocks (unshaded). Initially, the heap consists of a single 16-word double-word-aligned free block.[1]

Figure 9.34(a). The program asks for a four-word block. `Malloc` responds by carving out a four-word block from the front of the free block and returning a pointer to the first word of the block.

Figure 9.34(b). The program requests a five-word block. `Malloc` responds by allocating a six-word block from the front of the free block. In this example, `malloc` pads the block with an extra word in order to keep the free block aligned on a double-word boundary.

Figure 9.34(c). The program requests a six-word block and `malloc` responds by carving out a six-word block from the free block.

Figure 9.34(d). The program frees the six-word block that was allocated in Figure 9.34(b). Notice that after the call to `free` returns, the pointer `p2`

1. Throughout this section, we will assume that the allocator returns blocks aligned to 8-byte double-word boundaries.

still points to the freed block. It is the responsibility of the application not to use p2 again until it is reinitialized by a new call to malloc.

Figure 9.34(e). The program requests a two-word block. In this case, malloc allocates a portion of the block that was freed in the previous step and returns a pointer to this new block.

9.9.2 Why Dynamic Memory Allocation?

The most important reason that programs use dynamic memory allocation is that often they do not know the sizes of certain data structures until the program actually runs. For example, suppose we are asked to write a C program that reads a list of n ASCII integers, one integer per line, from stdin into a C array. The input consists of the integer n, followed by the n integers to be read and stored into the array. The simplest approach is to define the array statically with some hard-coded maximum array size:

```
1   #include "csapp.h"
2   #define MAXN 15213
3
4   int array[MAXN];
5
6   int main()
7   {
8       int i, n;
9
10      scanf("%d", &n);
11      if (n > MAXN)
12          app_error("Input file too big");
13      for (i = 0; i < n; i++)
14          scanf("%d", &array[i]);
15      exit(0);
16  }
```

Allocating arrays with hard-coded sizes like this is often a bad idea. The value of MAXN is arbitrary and has no relation to the actual amount of available virtual memory on the machine. Further, if the user of this program wanted to read a file that was larger than MAXN, the only recourse would be to recompile the program with a larger value of MAXN. While not a problem for this simple example, the presence of hard-coded array bounds can become a maintenance nightmare for large software products with millions of lines of code and numerous users.

A better approach is to allocate the array dynamically, at run time, after the value of n becomes known. With this approach, the maximum size of the array is limited only by the amount of available virtual memory.

```
#include "csapp.h"

int main()
{
    int *array, i, n;

    scanf("%d", &n);
    array = (int *)Malloc(n * sizeof(int));
    for (i = 0; i < n; i++)
        scanf("%d", &array[i]);
    free(array);
    exit(0);
}
```

Dynamic memory allocation is a useful and important programming technique. However, in order to use allocators correctly and efficiently, programmers need to have an understanding of how they work. We will discuss some of the gruesome errors that can result from the improper use of allocators in Section 9.11.

9.9.3 Allocator Requirements and Goals

Explicit allocators must operate within some rather stringent constraints:

Handling arbitrary request sequences. An application can make an arbitrary sequence of allocate and free requests, subject to the constraint that each free request must correspond to a currently allocated block obtained from a previous allocate request. Thus, the allocator cannot make any assumptions about the ordering of allocate and free requests. For example, the allocator cannot assume that all allocate requests are accompanied by a matching free request, or that matching allocate and free requests are nested.

Making immediate responses to requests. The allocator must respond immediately to allocate requests. Thus, the allocator is not allowed to reorder or buffer requests in order to improve performance.

Using only the heap. In order for the allocator to be scalable, any nonscalar data structures used by the allocator must be stored in the heap itself.

Aligning blocks (alignment requirement). The allocator must align blocks in such a way that they can hold any type of data object.

Not modifying allocated blocks. Allocators can only manipulate or change free blocks. In particular, they are not allowed to modify or move blocks once they are allocated. Thus, techniques such as compaction of allocated blocks are not permitted.

Working within these constraints, the author of an allocator attempts to meet the often conflicting performance goals of maximizing throughput and memory utilization.

Goal 1: Maximizing throughput. Given some sequence of n allocate and free requests

$$R_0, R_1, \ldots, R_k, \ldots, R_{n-1}$$

we would like to maximize an allocator's throughput, which is defined as the number of requests that it completes per unit time. For example, if an allocator completes 500 allocate requests and 500 free requests in 1 second, then its throughput is 1,000 operations per second. In general, we can maximize throughput by minimizing the average time to satisfy allocate and free requests. As we'll see, it is not too difficult to develop allocators with reasonably good performance where the worst-case running time of an allocate request is linear in the number of free blocks and the running time of a free request is constant.

Goal 2: Maximizing memory utilization. Naive programmers often incorrectly assume that virtual memory is an unlimited resource. In fact, the total amount of virtual memory allocated by all of the processes in a system is limited by the amount of swap space on disk. Good programmers know that virtual memory is a finite resource that must be used efficiently. This is especially true for a dynamic memory allocator that might be asked to allocate and free large blocks of memory.

There are a number of ways to characterize how efficiently an allocator uses the heap. In our experience, the most useful metric is peak utilization. As before, we are given some sequence of n allocate and free requests

$$R_0, R_1, \ldots, R_k, \ldots, R_{n-1}$$

If an application requests a block of p bytes, then the resulting allocated block has a payload of p bytes. After request R_k has completed, let the aggregate payload, denoted P_k, be the sum of the payloads of the currently allocated blocks, and let H_k denote the current (monotonically nondecreasing) size of the heap.

Then the peak utilization over the first $k + 1$ requests, denoted by U_k, is given by

$$U_k = \frac{\max_{i \le k} P_i}{H_k}$$

The objective of the allocator, then, is to maximize the peak utilization U_{n-1} over the entire sequence. As we will see, there is a tension between maximizing throughput and utilization. In particular, it is easy to write an allocator that maximizes throughput at the expense of heap utilization. One of the interesting challenges in any allocator design is finding an appropriate balance between the two goals.

Aside Relaxing the monotonically nondecreasing assumption

We could relax the monotonically nondecreasing assumption in our definition of U_k and allow the heap to grow up and down by letting H_k be the high-water mark over the first $k + 1$ requests.

9.9.4 Fragmentation

The primary cause of poor heap utilization is a phenomenon known as fragmentation, which occurs when otherwise unused memory is not available to satisfy allocate requests. There are two forms of fragmentation: internal fragmentation and external fragmentation.

Internal fragmentation occurs when an allocated block is larger than the payload. This might happen for a number of reasons. For example, the implementation of an allocator might impose a minimum size on allocated blocks that is greater than some requested payload. Or, as we saw in Figure 9.34(b), the allocator might increase the block size in order to satisfy alignment constraints.

Internal fragmentation is straightforward to quantify. It is simply the sum of the differences between the sizes of the allocated blocks and their payloads. Thus, at any point in time, the amount of internal fragmentation depends only on the pattern of previous requests and the allocator implementation.

External fragmentation occurs when there is enough aggregate free memory to satisfy an allocate request, but no single free block is large enough to handle the request. For example, if the request in Figure 9.34(e) were for eight words rather than two words, then the request could not be satisfied without requesting additional virtual memory from the kernel, even though there are eight free words remaining in the heap. The problem arises because these eight words are spread over two free blocks.

External fragmentation is much more difficult to quantify than internal fragmentation because it depends not only on the pattern of previous requests and the allocator implementation but also on the pattern of future requests. For example, suppose that after k requests all of the free blocks are exactly four words in size. Does this heap suffer from external fragmentation? The answer depends on the pattern of future requests. If all of the future allocate requests are for blocks that are smaller than or equal to four words, then there is no external fragmentation. On the other hand, if one or more requests ask for blocks larger than four words, then the heap does suffer from external fragmentation.

Since external fragmentation is difficult to quantify and impossible to predict, allocators typically employ heuristics that attempt to maintain small numbers of larger free blocks rather than large numbers of smaller free blocks.

9.9.5 Implementation Issues

The simplest imaginable allocator would organize the heap as a large array of bytes and a pointer p that initially points to the first byte of the array. To allocate

size bytes, malloc would save the current value of p on the stack, increment p by size, and return the old value of p to the caller. Free would simply return to the caller without doing anything.

This naive allocator is an extreme point in the design space. Since each malloc and free execute only a handful of instructions, throughput would be extremely good. However, since the allocator never reuses any blocks, memory utilization would be extremely bad. A practical allocator that strikes a better balance between throughput and utilization must consider the following issues:

Free block organization. How do we keep track of free blocks?

Placement. How do we choose an appropriate free block in which to place a newly allocated block?

Splitting. After we place a newly allocated block in some free block, what do we do with the remainder of the free block?

Coalescing. What do we do with a block that has just been freed?

The rest of this section looks at these issues in more detail. Since the basic techniques of placement, splitting, and coalescing cut across many different free block organizations, we will introduce them in the context of a simple free block organization known as an implicit free list.

9.9.6 Implicit Free Lists

Any practical allocator needs some data structure that allows it to distinguish block boundaries and to distinguish between allocated and free blocks. Most allocators embed this information in the blocks themselves. One simple approach is shown in Figure 9.35.

In this case, a block consists of a one-word **header**, the payload, and possibly some additional **padding**. The header encodes the block size (including the header and any padding) as well as whether the block is allocated or free. If we impose a double-word alignment constraint, then the block size is always a multiple of 8 and the 3 low-order bits of the block size are always zero. Thus, we need to store only the 29 high-order bits of the block size, freeing the remaining 3 bits to encode other information. In this case, we are using the least significant of these bits

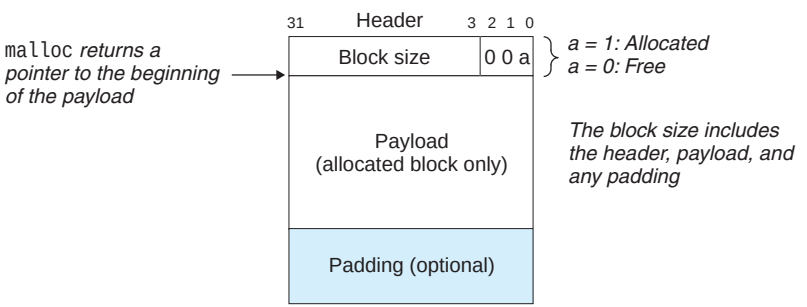


Figure 9.35
Format of a simple heap block.

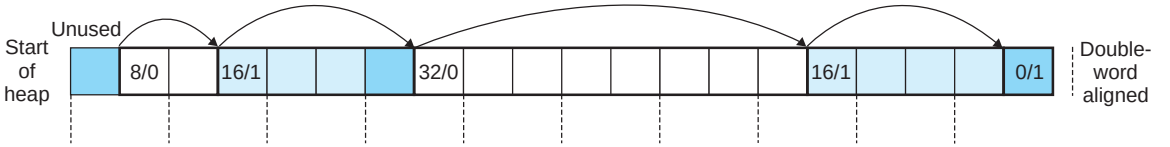


Figure 9.36 Organizing the heap with an implicit free list. All blocks are aligned. Free blocks are shaded. Header labels (size/allocated bit) are shown for each block.

(the allocated bit) to indicate whether the block is allocated or free. For example, suppose we have an allocated block with a block size of 24 (0x18) bytes. Then its header would be

```
0x00000018 | 0x1 = 0x00000019
```

Similarly, a free block with a block size of 40 (0x28) bytes would have a header of

```
0x00000028 | 0x0 = 0x00000028
```

The header is followed by the payload that the application requested when it called `malloc`. The payload is followed by a chunk of unused padding that can be any size. There are a number of reasons for the padding. For example, the padding might be part of an allocator's strategy for combating external fragmentation. Or it might be needed to satisfy the alignment requirement.

Given the block format in Figure 9.35, we can organize the heap as a sequence of contiguous allocated and free blocks, as shown in Figure 9.36.

We call this organization an implicit free list because the free blocks are linked implicitly by the size fields in the headers. The allocator can indirectly traverse the entire set of free blocks by traversing all of the blocks in the heap. Notice that we need some kind of specially marked end block—in this example, a terminating header with the allocated bit set and a size of zero. (As we will see in Section 9.9.12, setting the allocated bit simplifies the coalescing of free blocks.)

The advantage of an implicit free list is simplicity. A significant disadvantage is that the cost of any operation that requires a search of the free list, such as placing allocated blocks, will be linear in the total number of allocated and free blocks in the heap.

It is important to realize that the system's alignment requirement and the allocator's choice of block format impose a minimum block size on the allocator. No allocated or free block may be smaller than this minimum. For example, if we assume a double-word alignment requirement, then the size of each block must be a multiple of two words (8 bytes). Thus, the block format in Figure 9.35 induces a minimum block size of two words: one word for the header and another to maintain the alignment requirement. Even if the application were to request a single byte, the allocator would still create a two-word block.

Practice Problem 9.6 (solution page 919)

Determine the block sizes and header values that would result from the following sequence of `malloc` requests. Assumptions: (1) The allocator maintains double-word alignment and uses an implicit free list with the block format from Figure 9.35. (2) Block sizes are rounded up to the nearest multiple of 8 bytes.

Request	Block size (decimal bytes)	Block header (hex)
`malloc(2)`	_____	_____
`malloc(9)`	_____	_____
`malloc(15)`	_____	_____
`malloc(20)`	_____	_____

9.9.7 Placing Allocated Blocks

When an application requests a block of `k` bytes, the allocator searches the free list for a free block that is large enough to hold the requested block. The manner in which the allocator performs this search is determined by the placement policy. Some common policies are first fit, next fit, and best fit.

First fit searches the free list from the beginning and chooses the first free block that fits. Next fit is similar to first fit, but instead of starting each search at the beginning of the list, it starts each search where the previous search left off. Best fit examines every free block and chooses the free block with the smallest size that fits.

An advantage of first fit is that it tends to retain large free blocks at the end of the list. A disadvantage is that it tends to leave "splinters" of small free blocks toward the beginning of the list, which will increase the search time for larger blocks. Next fit was first proposed by Donald Knuth as an alternative to first fit, motivated by the idea that if we found a fit in some free block the last time, there is a good chance that we will find a fit the next time in the remainder of the block. Next fit can run significantly faster than first fit, especially if the front of the list becomes littered with many small splinters. However, some studies suggest that next fit suffers from worse memory utilization than first fit. Studies have found that best fit generally enjoys better memory utilization than either first fit or next fit. However, the disadvantage of using best fit with simple free list organizations such as the implicit free list is that it requires an exhaustive search of the heap. Later, we will look at more sophisticated segregated free list organizations that approximate a best-fit policy without an exhaustive search of the heap.

9.9.8 Splitting Free Blocks

Once the allocator has located a free block that fits, it must make another policy decision about how much of the free block to allocate. One option is to use the entire free block. Although simple and fast, the main disadvantage is that it

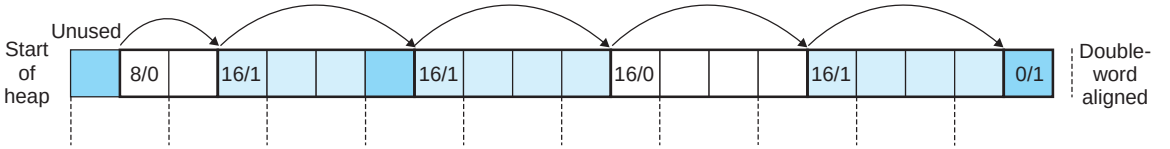


Figure 9.37 Splitting a free block to satisfy a three-word allocation request. All at l ar a . Fr l ar a . H a r ar la l it (i (t)/all at it).

introduces internal fragmentation. If the placement policy tends to produce good fits, then some additional internal fragmentation might be acceptable.

However, if the fit is not good, then the allocator will usually opt to split the free block into two parts. The first part becomes the allocated block, and the remainder becomes a new free block. Figure 9.37 shows how the allocator might split the eight-word free block in Figure 9.36 to satisfy an application's request for three words of heap memory.

9.9.9 Getting Additional Heap Memory

What happens if the allocator is unable to find a fit for the requested block? One option is to try to create some larger free blocks by merging (coalescing) free blocks that are physically adjacent in memory (next section). However, if this does not yield a sufficiently large block, or if the free blocks are already maximally coalesced, then the allocator asks the kernel for additional heap memory by calling the `sbrk` function. The allocator transforms the additional memory into one large free block, inserts the block into the free list, and then places the requested block in this new free block.

9.9.10 Coalescing Free Blocks

When the allocator frees an allocated block, there might be other free blocks that are adjacent to the newly freed block. Such adjacent free blocks can cause a phenomenon known as false fragmentation, where there is a lot of available free memory chopped up into small, unusable free blocks. For example, Figure 9.38 shows the result of freeing the block that was allocated in Figure 9.37. The result is two adjacent free blocks with payloads of three words each. As a result, a subsequent request for a payload of four words would fail, even though the aggregate size of the two free blocks is large enough to satisfy the request.

To combat false fragmentation, any practical allocator must merge adjacent free blocks in a process known as coalescing. This raises an important policy decision about when to perform coalescing. The allocator can opt for immediate coalescing by merging any adjacent blocks each time a block is freed. Or it can opt for deferred coalescing by waiting to coalesce free blocks at some later time. For example, the allocator might defer coalescing until some allocation request fails, and then scan the entire heap, coalescing all free blocks.

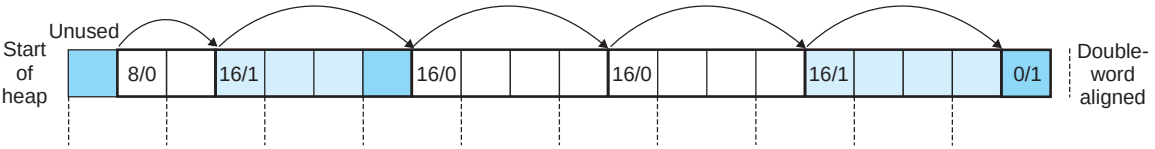


Figure 9.38 An example of false fragmentation. All$_0$ atγ l$_0$ ar a$\gamma\gamma$. Fr l$_0$ ar a$\gamma\gamma$. H aγ r ar la lγ it (i (t)/all$_0$ atγ it).

Immediate coalescing is straightforward and can be performed in constant time, but with some request patterns it can introduce a form of thrashing where a block is repeatedly coalesced and then split soon thereafter. For example, in Figure 9.38, a repeated pattern of allocating and freeing a three-word block would introduce a lot of unnecessary splitting and coalescing. In our discussion of allocators, we will assume immediate coalescing, but you should be aware that fast allocators often opt for some form of deferred coalescing.

9.9.11 Coalescing with Boundary Tags

How does an allocator implement coalescing? Let us refer to the block we want to free as the current block. Then coalescing the next free block (in memory) is straightforward and efficient. The header of the current block points to the header of the next block, which can be checked to determine if the next block is free. If so, its size is simply added to the size of the current header and the blocks are coalesced in constant time.

But how would we coalesce the previous block? Given an implicit free list of blocks with headers, the only option would be to search the entire list, remembering the location of the previous block, until we reached the current block. With an implicit free list, this means that each call to `free` would require time linear in the size of the heap. Even with more sophisticated free list organizations, the search time would not be constant.

Knuth developed a clever and general technique, known as boundary tags, that allows for constant-time coalescing of the previous block. The idea, which is shown in Figure 9.39, is to add a footer (the boundary tag) at the end of each block, where the footer is a replica of the header. If each block includes such a footer, then the allocator can determine the starting location and status of the previous block by inspecting its footer, which is always one word away from the start of the current block.

Consider all the cases that can exist when the allocator frees the current block:

1. The previous and next blocks are both allocated.
2. The previous block is allocated and the next block is free.
3. The previous block is free and the next block is allocated.
4. The previous and next blocks are both free.

Figure 9.39
Format of heap block that uses a boundary tag.

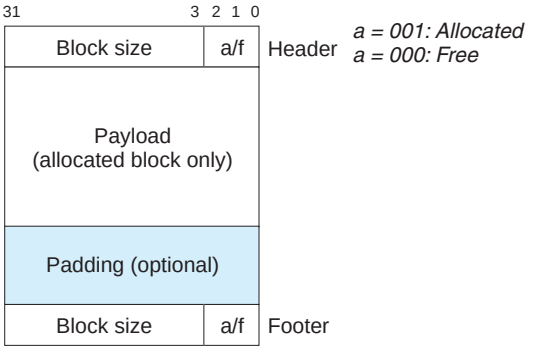


Figure 9.40 shows how we would coalesce each of the four cases.

In case 1, both adjacent blocks are allocated and thus no coalescing is possible. So the status of the current block is simply changed from allocated to free. In case 2, the current block is merged with the next block. The header of the current block and the footer of the next block are updated with the combined sizes of the current and next blocks. In case 3, the previous block is merged with the current block. The header of the previous block and the footer of the current block are updated with the combined sizes of the two blocks. In case 4, all three blocks are merged to form a single free block, with the header of the previous block and the footer of the next block updated with the combined sizes of the three blocks. In each case, the coalescing is performed in constant time.

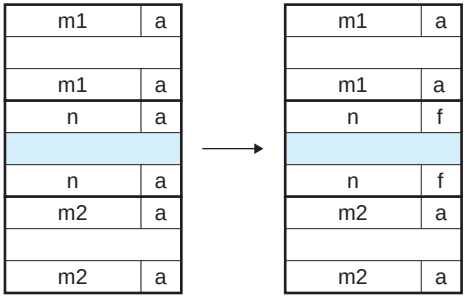
The idea of boundary tags is a simple and elegant one that generalizes to many different types of allocators and free list organizations. However, there is a potential disadvantage. Requiring each block to contain both a header and a footer can introduce significant memory overhead if an application manipulates many small blocks. For example, if a graph application dynamically creates and destroys graph nodes by making repeated calls to `malloc` and `free`, and each graph node requires only a couple of words of memory, then the header and the footer will consume half of each allocated block.

Fortunately, there is a clever optimization of boundary tags that eliminates the need for a footer in allocated blocks. Recall that when we attempt to coalesce the current block with the previous and next blocks in memory, the size field in the footer of the previous block is only needed if the previous block is `free`. If we were to store the allocated/free bit of the previous block in one of the excess low-order bits of the current block, then allocated blocks would not need footers, and we could use that extra space for payload. Note, however, that free blocks would still need footers.

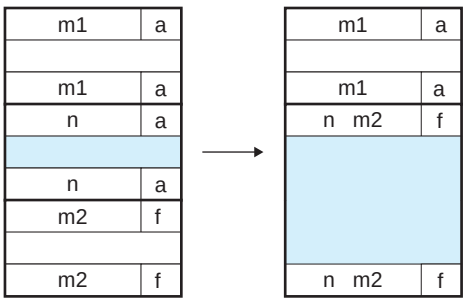
Practice Problem 9.7 (solution page 919)

Determine the minimum block size for each of the following combinations of alignment requirements and block formats. Assumptions: Implicit free list, zero-size payloads are not allowed, and headers and footers are stored in 4-byte words.

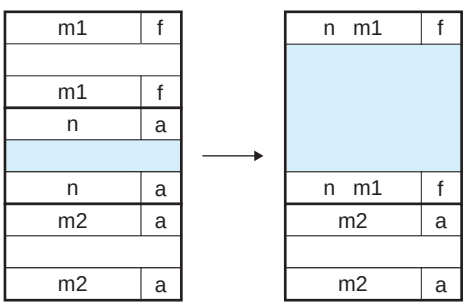
Figure 9.40
Coalescing with boundary tags. Ca 1: r a t all o at . Ca 2: r all o at , t r . Ca 3: r r , t all o at . Ca 4: t a r r .



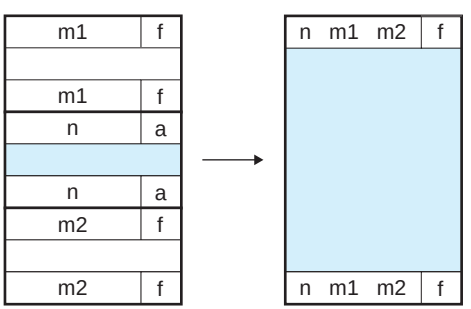
Case 1



Case 2



Case 3



Case 4

Alignment	Allocated block	Free block	Minimum block size (bytes)
Single word	Header and footer	Header and footer	_____
Single word	Header, but no footer	Header and footer	_____
Double word	Header and footer	Header and footer	_____
Double word	Header, but no footer	Header and footer	_____

9.9.12 Putting It Together: Implementing a Simple Allocator

Building an allocator is a challenging task. The design space is large, with numerous alternatives for block format and free list format, as well as placement, splitting, and coalescing policies. Another challenge is that you are often forced to program outside the safe, familiar confines of the type system, relying on the error-prone pointer casting and pointer arithmetic that is typical of low-level systems programming.

While allocators do not require enormous amounts of code, they are subtle and unforgiving. Students familiar with higher-level languages such as C++ or Java often hit a conceptual wall when they first encounter this style of programming. To help you clear this hurdle, we will work through the implementation of a simple allocator based on an implicit free list with immediate boundary-tag coalescing. The maximum block size is 2^{32} = 4 GB. The code is 64-bit clean, running without modification in 32-bit (`gcc -m32`) or 64-bit (`gcc -m64`) processes.

G ral All$_0$ at$_0$r D i

Our allocator uses a model of the memory system provided by the `memlib.c` package shown in Figure 9.41. The purpose of the model is to allow us to run our allocator without interfering with the existing system-level `malloc` package.

The `mem_init` function models the virtual memory available to the heap as a large double-word aligned array of bytes. The bytes between `mem_heap` and `mem_brk` represent allocated virtual memory. The bytes following `mem_brk` represent unallocated virtual memory. The allocator requests additional heap memory by calling the `mem_sbrk` function, which has the same interface as the system's `sbrk` function, as well as the same semantics, except that it rejects requests to shrink the heap.

The allocator itself is contained in a source file (`mm.c`) that users can compile and link into their applications. The allocator exports three functions to application programs:

```
1  extern int mm_init(void);
2  extern void *mm_malloc (size_t size);
3  extern void mm_free (void *ptr);
```

The `mm_init` function initializes the allocator, returning 0 if successful and −1 otherwise. The `mm_malloc` and `mm_free` functions have the same interfaces and semantics as their system counterparts. The allocator uses the block format

code/vm/malloc/memlib.c

```
1  /* Private global variables */
2  static char *mem_heap;     /* Points to first byte of heap */
3  static char *mem_brk;      /* Points to last byte of heap plus 1 */
4  static char *mem_max_addr; /* Max legal heap addr plus 1*/
5
6  /*
7   * mem_init - Initialize the memory system model
8   */
9  void mem_init(void)
10 {
11     mem_heap = (char *)Malloc(MAX_HEAP);
12     mem_brk = (char *)mem_heap;
13     mem_max_addr = (char *)(mem_heap + MAX_HEAP);
14 }
15
16 /*
17  * mem_sbrk - Simple model of the sbrk function. Extends the heap
18  *    by incr bytes and returns the start address of the new area. In
19  *    this model, the heap cannot be shrunk.
20  */
21 void *mem_sbrk(int incr)
22 {
23     char *old_brk = mem_brk;
24
25     if ( (incr < 0) || ((mem_brk + incr) > mem_max_addr)) {
26         errno = ENOMEM;
27         fprintf(stderr, "ERROR: mem_sbrk failed. Ran out of memory...\n");
28         return (void *)-1;
29     }
30     mem_brk += incr;
31     return (void *)old_brk;
32 }
```

code/vm/malloc/memlib.c

Figure 9.41 `memlib.c`: Memory system model.

shown in Figure 9.39. The minimum block size is 16 bytes. The free list is organized as an implicit free list, with the invariant form shown in Figure 9.42.

The first word is an unused padding word aligned to a double-word boundary. The padding is followed by a special prologue block, which is an 8-byte allocated block consisting of only a header and a footer. The prologue block is created during initialization and is never freed. Following the prologue block are zero or more regular blocks that are created by calls to `malloc` or `free`. The heap always ends with a special epilogue block, which is a zero-size allocated block

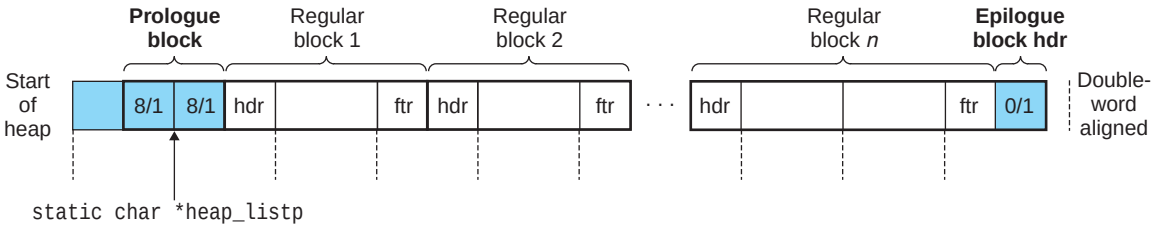


Figure 9.42 Invariant form of the implicit free list.

that consists of only a header. The prologue and epilogue blocks are tricks that eliminate the edge conditions during coalescing. The allocator uses a single private (`static`) global variable (`heap_listp`) that always points to the prologue block. (As a minor optimization, we could make it point to the next block instead of the prologue block.)

Ba i C ta t a Ma r r Ma i lati t Fr Lit

Figure 9.43 shows some basic constants and macros that we will use throughout the allocator code. Lines 2–4 define some basic size constants: the sizes of words (WSIZE) and double words (DSIZE), and the size of the initial free block and the default size for expanding the heap (CHUNKSIZE).

Manipulating the headers and footers in the free list can be troublesome because it demands extensive use of casting and pointer arithmetic. Thus, we find it helpful to define a small set of macros for accessing and traversing the free list (lines 9–25). The PACK macro (line 9) combines a size and an allocate bit and returns a value that can be stored in a header or footer.

The GET macro (line 12) reads and returns the word referenced by argument `p`. The casting here is crucial. The argument `p` is typically a (`void *`) pointer, which cannot be dereferenced directly. Similarly, the PUT macro (line 13) stores `val` in the word pointed at by argument `p`.

The GET_SIZE and GET_ALLOC macros (lines 16–17) return the size and allocated bit, respectively, from a header or footer at address `p`. The remaining macros operate on block pointers (denoted `bp`) that point to the first payload byte. Given a block pointer `bp`, the HDRP and FTRP macros (lines 20–21) return pointers to the block header and footer, respectively. The NEXT_BLKP and PREV_BLKP macros (lines 24–25) return the block pointers of the next and previous blocks, respectively.

The macros can be composed in various ways to manipulate the free list. For example, given a pointer `bp` to the current block, we could use the following line of code to determine the size of the next block in memory:

```
size_t size = GET_SIZE(HDRP(NEXT_BLKP(bp)));
```

code/vm/malloc/mm.c

```
int mm_init(void)
{
    /* Create the initial empty heap */
    if ((heap_listp = mem_sbrk(4*WSIZE)) == (void *)-1)
        return -1;
    PUT(heap_listp, 0);                          /* Alignment padding */
    PUT(heap_listp + (1*WSIZE), PACK(DSIZE, 1)); /* Prologue header */
    PUT(heap_listp + (2*WSIZE), PACK(DSIZE, 1)); /* Prologue footer */
    PUT(heap_listp + (3*WSIZE), PACK(0, 1));     /* Epilogue header */
```

multiple of 2 words (8 bytes) and then requests the additional heap space from the memory system (lines 7–9).

The remainder of the `extend_heap` function (lines 12–17) is somewhat subtle. The heap begins on a double-word aligned boundary, and every call to `extend_heap` returns a block whose size is an integral number of double words. Thus, every call to `mem_sbrk` returns a double-word aligned chunk of memory immediately following the header of the epilogue block. This header becomes the header of the new free block (line 12), and the last word of the chunk becomes the new epilogue block header (line 14). Finally, in the likely case that the previous heap was terminated by a free block, we call the `coalesce` function to merge the two free blocks and return the block pointer of the merged blocks (line 17).

Freeing and Coalescing Blocks

An application frees a previously allocated block by calling the `mm_free` function (Figure 9.46), which frees the requested block (`bp`) and then merges adjacent free blocks using the boundary-tags coalescing technique described in Section 9.9.11.

The code in the `coalesce` helper function is a straightforward implementation of the four cases outlined in Figure 9.40. There is one somewhat subtle aspect. The free list format we have chosen—with its prologue and epilogue blocks that are always marked as allocated—allows us to ignore the potentially troublesome edge conditions where the requested block `bp` is at the beginning or end of the heap. Without these special blocks, the code would be messier, more error prone, and slower because we would have to check for these rare edge conditions on each and every free request.

Allocating Blocks

An application requests a block of `size` bytes of memory by calling the `mm_malloc` function (Figure 9.47). After checking for spurious requests, the allocator must adjust the requested block size to allow room for the header and the footer, and to satisfy the double-word alignment requirement. Lines 12–13 enforce the minimum block size of 16 bytes: 8 bytes to satisfy the alignment requirement and 8 more bytes for the overhead of the header and footer. For requests over 8 bytes (line 15), the general rule is to add in the overhead bytes and then round up to the nearest multiple of 8.

Once the allocator has adjusted the requested size, it searches the free list for a suitable free block (line 18). If there is a fit, then the allocator places the requested block and optionally splits the excess (line 19) and then returns the address of the newly allocated block.

If the allocator cannot find a fit, it extends the heap with a new free block (lines 24–26), places the requested block in the new free block, optionally splitting the block (line 27), and then returns a pointer to the newly allocated block.

code/vm/malloc/mm.c

```
1  void mm_free(void *bp)
2  {
3      size_t size = GET_SIZE(HDRP(bp));
4
5      PUT(HDRP(bp), PACK(size, 0));
6      PUT(FTRP(bp), PACK(size, 0));
7      coalesce(bp);
8  }
9
10 static void *coalesce(void *bp)
11 {
12     size_t prev_alloc = GET_ALLOC(FTRP(PREV_BLKP(bp)));
13     size_t next_alloc = GET_ALLOC(HDRP(NEXT_BLKP(bp)));
14     size_t size = GET_SIZE(HDRP(bp));
15
16
```

code/vm/malloc/mm.c

```
1  void *mm_malloc(size_t size)
2  {
3      size_t asize;      /* Adjusted block size */
4      size_t extendsize; /* Amount to extend heap if no fit */
5      char *bp;
6
7      /* Ignore spurious requests */
8      if (size == 0)
9          return NULL;
10
11     /* Adjust block size to include overhead and alignment reqs. */
12     if (size <= DSIZE)
13         asize = 2*DSIZE;
14     else
15         asize = DSIZE * ((size + (DSIZE) + (DSIZE-1)) / DSIZE);
16
17     /* Search the free list for a fit */
18     if ((bp = find_fit(asize)) != NULL) {
19         place(bp, asize);
20         return bp;
21     }
22
23     /* No fit found. Get more memory and place the block */
24     extendsize = MAX(asize,CHUNKSIZE);
25     if ((bp = extend_heap(extendsize/WSIZE)) == NULL)
26         return NULL;
27     place(bp, asize);
28     return bp;
29 }
```

code/vm/malloc/mm.c

Figure 9.47 `mm_malloc` allocates a block from the free list.

Practice Problem 9.8 (solution page 920)

Implement a `find_fit` function for the simple allocator described in Section 9.9.12.

```
static void *find_fit(size_t asize)
```

Your solution should perform a first-fit search of the implicit free list.

Practice Problem 9.9 (solution page 920)

Implement a `place` function for the example allocator.

```
static void place(void *bp, size_t asize)
```

Your solution should place the requested block at the beginning of the free block, splitting only if the size of the remainder would equal or exceed the minimum block size.

9.9.13 Explicit Free Lists

The implicit free list provides us with a simple way to introduce some basic allocator concepts. However, because block allocation time is linear in the total number of heap blocks, the implicit free list is not appropriate for a general-purpose allocator (although it might be fine for a special-purpose allocator where the number of heap blocks is known beforehand to be small).

A better approach is to organize the free blocks into some form of explicit data structure. Since by definition the body of a free block is not needed by the program, the pointers that implement the data structure can be stored within the bodies of the free blocks. For example, the heap can be organized as a doubly linked free list by including a `pred` (predecessor) and `succ` (successor) pointer in each free block, as shown in Figure 9.48.

Using a doubly linked list instead of an implicit free list reduces the first-fit allocation time from linear in the total number of blocks to linear in the number of free blocks. However, the time to free a block can be either linear or constant, depending on the policy we choose for ordering the blocks in the free list.

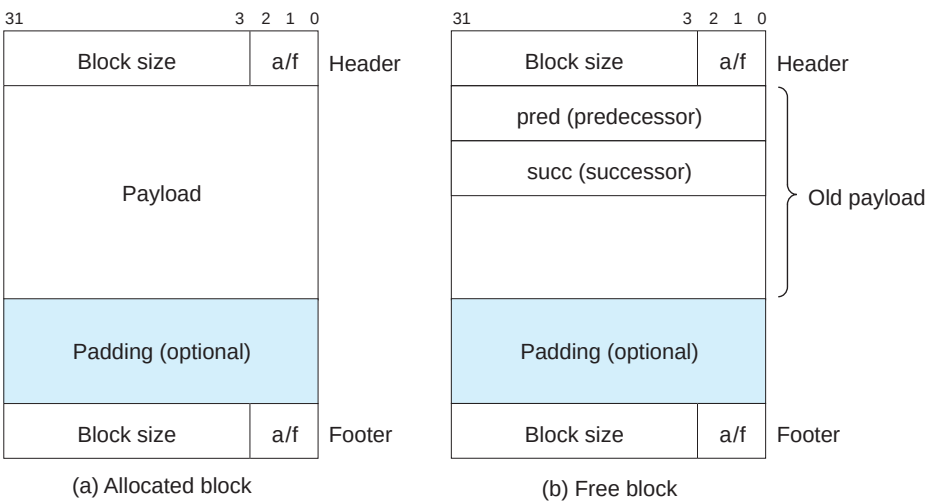


Figure 9.48 Format of heap blocks that use doubly linked free lists.

One approach is to maintain the list in last-in first-out (LIFO) order by inserting newly freed blocks at the beginning of the list. With a LIFO ordering and a first-fit placement policy, the allocator inspects the most recently used blocks first. In this case, freeing a block can be performed in constant time. If boundary tags are used, then coalescing can also be performed in constant time.

Another approach is to maintain the list in address order, where the address of each block in the list is less than the address of its successor. In this case, freeing a block requires a linear-time search to locate the appropriate predecessor. The trade-off is that address-ordered first fit enjoys better memory utilization than LIFO-ordered first fit, approaching the utilization of best fit.

A disadvantage of explicit lists in general is that free blocks must be large enough to contain all of the necessary pointers, as well as the header and possibly a footer. This results in a larger minimum block size and increases the potential for internal fragmentation.

9.9.14 Segregated Free Lists

As we have seen, an allocator that uses a single linked list of free blocks requires time linear in the number of free blocks to allocate a block. A popular approach for reducing the allocation time, known generally as segregated storage, is to maintain multiple free lists, where each list holds blocks that are roughly the same size. The general idea is to partition the set of all possible block sizes into equivalence classes called size classes. There are many ways to define the size classes. For example, we might partition the block sizes by powers of 2:

$$\{1\}, \{2\}, \{3, 4\}, \{5–8\}, \cdots, \{1{,}025–2{,}048\}, \{2{,}049–4{,}096\}, \{4{,}097–\infty\}$$

Or we might assign small blocks to their own size classes and partition large blocks by powers of 2:

$$\{1\}, \{2\}, \{3\}, \cdots, \{1{,}023\}, \{1{,}024\}, \{1{,}025–2{,}048\}, \{2{,}049–4{,}096\}, \{4{,}097–\infty\}$$

The allocator maintains an array of free lists, with one free list per size class, ordered by increasing size. When the allocator needs a block of size n, it searches the appropriate free list. If it cannot find a block that fits, it searches the next list, and so on.

The dynamic storage allocation literature describes dozens of variants of segregated storage that differ in how they define size classes, when they perform coalescing, when they request additional heap memory from the operating system, whether they allow splitting, and so forth. To give you a sense of what is possible, we will describe two of the basic approaches: simple segregated storage and segregated fits.

Simple Segregated Storage

With simple segregated storage, the free list for each size class contains same-size blocks, each the size of the largest element of the size class. For example, if some size class is defined as {17–32}, then the free list for that class consists entirely of blocks of size 32.

To allocate a block of some given size, we check the appropriate free list. If the list is not empty, we simply allocate the first block in its entirety. Free blocks are never split to satisfy allocation requests. If the list is empty, the allocator requests a fixed-size chunk of additional memory from the operating system (typically a multiple of the page size), divides the chunk into equal-size blocks, and links the blocks together to form the new free list. To free a block, the allocator simply inserts the block at the front of the appropriate free list.

There are a number of advantages to this simple scheme. Allocating and freeing blocks are both fast constant-time operations. Further, the combination of the same-size blocks in each chunk, no splitting, and no coalescing means that there is very little per-block memory overhead. Since each chunk has only same-size blocks, the size of an allocated block can be inferred from its address. Since there is no coalescing, allocated blocks do not need an allocated/free flag in the header. Thus, allocated blocks require no headers, and since there is no coalescing, they do not require any footers either. Since allocate and free operations insert and delete blocks at the beginning of the free list, the list need only be singly linked instead of doubly linked. The bottom line is that the only required field in any block is a one-word `succ` pointer in each free block, and thus the minimum block size is only one word.

A significant disadvantage is that simple segregated storage is susceptible to internal and external fragmentation. Internal fragmentation is possible because free blocks are never split. Worse, certain reference patterns can cause extreme external fragmentation because free blocks are never coalesced (Practice Problem 9.10).

Practice Problem 9.10 (solution page 921)

Describe a reference pattern that results in severe external fragmentation in an allocator based on simple segregated storage.

Segregated Fits

With this approach, the allocator maintains an array of free lists. Each free list is associated with a size class and is organized as some kind of explicit or implicit list. Each list contains potentially different-size blocks whose sizes are members of the size class. There are many variants of segregated fits allocators. Here we describe a simple version.

To allocate a block, we determine the size class of the request and do a first-fit search of the appropriate free list for a block that fits. If we find one, then we (optionally) split it and insert the fragment in the appropriate free list. If we cannot find a block that fits, then we search the free list for the next larger size class. We

repeat until we find a block that fits. If none of the free lists yields a block that fits, then we request additional heap memory from the operating system, allocate the block out of this new heap memory, and place the remainder in the appropriate size class. To free a block, we coalesce and place the result on the appropriate free list.

The segregated fits approach is a popular choice with production-quality allocators such as the GNU `malloc` package provided in the C standard library because it is both fast and memory efficient. Search times are reduced because searches are limited to particular parts of the heap instead of the entire heap. Memory utilization can improve because of the interesting fact that a simple first-fit search of a segregated free list approximates a best-fit search of the entire heap.

Buddy Systems

A buddy system is a special case of segregated fits where each size class is a power of 2. The basic idea is that, given a heap of 2^m words, we maintain a separate free list for each block size 2^k, where $0 \leq k \leq m$. Requested block sizes are rounded up to the nearest power of 2. Originally, there is one free block of size 2^m words.

To allocate a block of size 2^k, we find the first available block of size 2^j, such that $k \leq j \leq m$. If $j = k$, then we are done. Otherwise, we recursively split the block in half until $j = k$. As we perform this splitting, each remaining half (known as a buddy) is placed on the appropriate free list. To free a block of size 2^k, we continue coalescing with the free buddies. When we encounter an allocated buddy, we stop the coalescing.

A key fact about buddy systems is that, given the address and size of a block, it is easy to compute the address of its buddy. For example, a block of size 32 bytes with address

$$\text{xxx} \ldots \text{x00000}$$

has its buddy at address

$$\text{xxx} \ldots \text{x10000}$$

In other words, the addresses of a block and its buddy differ in exactly one bit position.

The major advantage of a buddy system allocator is its fast searching and coalescing. The major disadvantage is that the power-of-2 requirement on the block size can cause significant internal fragmentation. For this reason, buddy system allocators are not appropriate for general-purpose workloads. However, for certain application-specific workloads, where the block sizes are known in advance to be powers of 2, buddy system allocators have a certain appeal.

9.10 Garbage Collection

With an explicit allocator such as the C `malloc` package, an application allocates and frees heap blocks by making calls to `malloc` and `free`. It is the application's responsibility to free any allocated blocks that it no longer needs.

Failing to free allocated blocks is a common programming error. For example, consider the following C function that allocates a block of temporary storage as part of its processing:

```
1  void garbage()
2  {
3      int *p = (int *)Malloc(15213);
4
5      return; /* Array p is garbage at this point */
6  }
```

Since p is no longer needed by the program, it should have been freed before garbage returned. Unfortunately, the programmer has forgotten to free the block. It remains allocated for the lifetime of the program, needlessly occupying heap space that could be used to satisfy subsequent allocation requests.

A garbage collector is a dynamic storage allocator that automatically frees allocated blocks that are no longer needed by the program. Such blocks are known as garbage (hence the term "garbage collector"). The process of automatically reclaiming heap storage is known as garbage collection. In a system that supports garbage collection, applications explicitly allocate heap blocks but never explicitly free them. In the context of a C program, the application calls malloc but never calls free. Instead, the garbage collector periodically identifies the garbage blocks and makes the appropriate calls to free to place those blocks back on the free list.

Garbage collection dates back to Lisp systems developed by John McCarthy at MIT in the early 1960s. It is an important part of modern language systems such as Java, ML, Perl, and Mathematica, and it remains an active and important area of research. The literature describes an amazing number of approaches for garbage collection. We will limit our discussion to McCarthy's original Mark&Sweep algorithm, which is interesting because it can be built on top of an existing malloc package to provide garbage collection for C and C++ programs.

9.10.1 Garbage Collector Basics

A garbage collector views memory as a directed reachability graph of the form shown in Figure 9.49. The nodes of the graph are partitioned into a set of root nodes and a set of heap nodes. Each heap node corresponds to an allocated block in the heap. A directed edge p → q means that some location in block p points to some location in block q. Root nodes correspond to locations not in the heap that contain pointers into the heap. These locations can be registers, variables on the stack, or global variables in the read/write data area of virtual memory.

We say that a node p is reachable if there exists a directed path from any root node to p. At any point in time, the unreachable nodes correspond to garbage that can never be used again by the application. The role of a garbage collector is to maintain some representation of the reachability graph and periodically reclaim the unreachable nodes by freeing them and returning them to the free list.

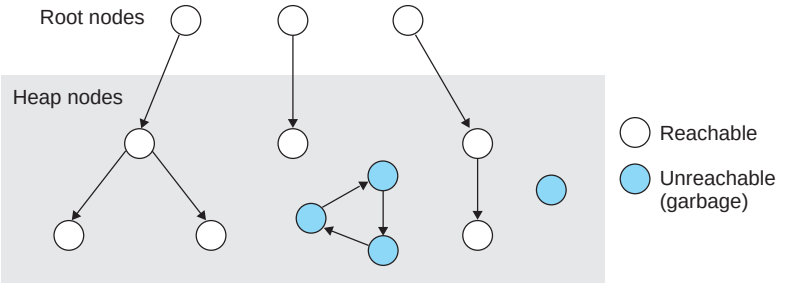


Figure 9.49
A garbage collector's view of memory as a directed graph.

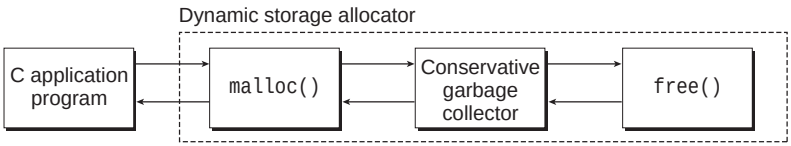


Figure 9.50
Integrating a conservative garbage collector and a C `malloc` package.

Garbage collectors for languages like ML and Java, which exert tight control over how applications create and use pointers, can maintain an exact representation of the reachability graph and thus can reclaim all garbage. However, collectors for languages like C and C++ cannot in general maintain exact representations of the reachability graph. Such collectors are known as conservative garbage collectors. They are conservative in the sense that each reachable block is correctly identified as reachable, while some unreachable nodes might be incorrectly identified as reachable.

Collectors can provide their service on demand, or they can run as separate threads in parallel with the application, continuously updating the reachability graph and reclaiming garbage. For example, consider how we might incorporate a conservative collector for C programs into an existing `malloc` package, as shown in Figure 9.50.

The application calls `malloc` in the usual manner whenever it needs heap space. If `malloc` is unable to find a free block that fits, then it calls the garbage collector in hopes of reclaiming some garbage to the free list. The collector identifies the garbage blocks and returns them to the heap by calling the `free` function. The key idea is that the collector calls `free` instead of the application. When the call to the collector returns, `malloc` tries again to find a free block that fits. If that fails, then it can ask the operating system for additional memory. Eventually, `malloc` returns a pointer to the requested block (if successful) or the NULL pointer (if unsuccessful).

9.10.2 Mark&Sweep Garbage Collectors

A Mark&Sweep garbage collector consists of a mark phase, which marks all reachable and allocated descendants of the root nodes, followed by a sweep phase, which frees each unmarked allocated block. Typically, one of the spare low-order bits in the block header is used to indicate whether a block is marked or not.

The balanced tree approach is correct in the sense that it is guaranteed to mark all of the nodes that are reachable from the roots. This is a necessary guarantee, as application users would certainly not appreciate having their allocated blocks prematurely returned to the free list. However, it is conservative in the sense that it may incorrectly mark blocks that are actually unreachable, and thus it may fail to free some garbage. While this does not affect the correctness of application programs, it can result in unnecessary external fragmentation.

The fundamental reason that Mark&Sweep collectors for C programs must be conservative is that the C language does not tag memory locations with type information. Thus, scalars like `int`s or `float`s can masquerade as pointers. For example, suppose that some reachable allocated block contains an `int` in its payload whose value happens to correspond to an address in the payload of some other allocated block `b`. There is no way for the collector to infer that the data is really an `int` and not a pointer. Therefore, the allocator must conservatively mark block `b` as reachable, when in fact it might not be.

9.11 Common Memory-Related Bugs in C Programs

Managing and using virtual memory can be a difficult and error-prone task for C programmers. Memory-related bugs are among the most frightening because they often manifest themselves at a distance, in both time and space, from the source of the bug. Write the wrong data to the wrong location, and your program can run for hours before it finally fails in some distant part of the program. We conclude our discussion of virtual memory with a look at of some of the common memory-related bugs.

9.11.1 Dereferencing Bad Pointers

As we learned in Section 9.7.2, there are large holes in the virtual address space of a process that are not mapped to any meaningful data. If we attempt to dereference a pointer into one of these holes, the operating system will terminate our program with a segmentation exception. Also, some areas of virtual memory are read-only. Attempting to write to one of these areas terminates the program with a protection exception.

A common example of dereferencing a bad pointer is the classic `scanf` bug. Suppose we want to use `scanf` to read an integer from `stdin` into a variable. The correct way to do this is to pass `scanf` a format string and the address of the variable:

```
scanf("%d", &val)
```

However, it is easy for new C programmers (and experienced ones too!) to pass the contents of `val` instead of its address:

```
scanf("%d", val)
```

In this case, `scanf` will interpret the contents of `val` as an address and attempt to write a word to that location. In the best case, the program terminates immediately with an exception. In the worst case, the contents of `val` correspond to some valid read/write area of virtual memory, and we overwrite memory, usually with disastrous and baffling consequences much later.

9.11.2 Reading Uninitialized Memory

While bss memory locations (such as uninitialized global C variables) are always initialized to zeros by the loader, this is not true for heap memory. A common error is to assume that heap memory is initialized to zero:

```
1   /* Return y = Ax */
2   int *matvec(int **A, int *x, int n)
3   {
4       int i, j;
5
6       int *y = (int *)Malloc(n * sizeof(int));
7
8       for (i = 0; i < n; i++)
9           for (j = 0; j < n; j++)
10              y[i] += A[i][j] * x[j];
11      return y;
12  }
```

In this example, the programmer has incorrectly assumed that vector `y` has been initialized to zero. A correct implementation would explicitly zero `y[i]` or use `calloc`.

9.11.3 Allowing Stack Buffer Overflows

As we saw in Section 3.10.3, a program has a buffer overflow bug if it writes to a target buffer on the stack without examining the size of the input string. For example, the following function has a buffer overflow bug because the `gets` function copies an arbitrary-length string to the buffer. To fix this, we would need to use the `fgets` function, which limits the size of the input string.

```
1   void bufoverflow()
2   {
3       char buf[64];
4
5       gets(buf); /* Here is the stack buffer overflow bug */
6       return;
7   }
```

9.11.4 Assuming That Pointers and the Objects They Point to Are the Same Size

One common mistake is to assume that pointers to objects are the same size as the objects they point to:

```
1   /* Create an nxm array */
2   int **makeArray1(int n, int m)
3   {
4       int i;
5       int **A = (int **)Malloc(n * sizeof(int));
6
7       for (i = 0; i < n; i++)
8           A[i] = (int *)Malloc(m * sizeof(int));
9       return A;
10  }
```

The intent here is to create an array of `n` pointers, each of which points to an array of `m` `int`s. However, because the programmer has written `sizeof(int)` instead of `sizeof(int *)` in line 5, the code actually creates an array of `int`s.

This code will run fine on machines where `int`s and pointers to `int`s are the same size. But if we run this code on a machine like the Core i7, where a pointer is larger than an `int`, then the loop in lines 7–8 will write past the end of the `A` array. Since one of these words will likely be the boundary-tag footer of the allocated block, we may not discover the error until we free the block much later in the program, at which point the coalescing code in the allocator will fail dramatically and for no apparent reason. This is an insidious example of the kind of "action at a distance" that is so typical of memory-related programming bugs.

9.11.5 Making Off-by-One Errors

Off-by-one errors are another common source of overwriting bugs:

```
1   /* Create an nxm array */
2   int **makeArray2(int n, int m)
3   {
4       int i;
5       int **A = (int **)Malloc(n * sizeof(int *));
6
7       for (i = 0; i <= n; i++)
8           A[i] = (int *)Malloc(m * sizeof(int));
9       return A;
10  }
```

This is another version of the program in the previous section. Here we have created an `n`-element array of pointers in line 5 but then tried to initialize `n + 1` of its elements in lines 7 and 8, in the process overwriting some memory that follows the `A` array.

9.11.6 Referencing a Pointer Instead of the Object It Points To

If we are not careful about the precedence and associativity of C operators, then we incorrectly manipulate a pointer instead of the object it points to. For example, consider the following function, whose purpose is to remove the first item in a binary heap of `*size` items and then reheapify the remaining `*size - 1` items:

```
1   int *binheapDelete(int **binheap, int *size)
2   {
3       int *packet = binheap[0];
4
5       binheap[0] = binheap[*size - 1];
6       *size--; /* This should be (*size)-- */
7       heapify(binheap, *size, 0);
8       return(packet);
9   }
```

In line 6, the intent is to decrement the integer value pointed to by the `size` pointer. However, because the unary `--` and `*` operators have the same precedence and associate from right to left, the code in line 6 actually decrements the pointer itself instead of the integer value that it points to. If we are lucky, the program will crash immediately. But more likely we will be left scratching our heads when the program produces an incorrect answer much later in its execution. The moral here is to use parentheses whenever in doubt about precedence and associativity. For example, in line 6, we should have clearly stated our intent by using the expression `(*size)--`.

9.11.7 Misunderstanding Pointer Arithmetic

Another common mistake is to forget that arithmetic operations on pointers are performed in units that are the size of the objects they point to, which are not necessarily bytes. For example, the intent of the following function is to scan an array of `int`s and return a pointer to the first occurrence of `val`:

```
1   int *search(int *p, int val)
2   {
3       while (*p && *p != val)
4           p += sizeof(int); /* Should be p++ */
5       return p;
6   }
```

However, because line 4 increments the pointer by 4 (the number of bytes in an integer) each time through the loop, the function incorrectly scans every fourth integer in the array.

9.11.8 Referencing Nonexistent Variables

Naive C programmers who do not understand the stack discipline will sometimes reference local variables that are no longer valid, as in the following example:

```
1  int *stackref ()
2  {
3      int val;
4
5      return &val;
6  }
```

This function returns a pointer (say, `p`) to a local variable on the stack and then pops its stack frame. Although `p` still points to a valid memory address, it no longer points to a valid variable. When other functions are called later in the program, the memory will be reused for their stack frames. Later, if the program assigns some value to `*p`, then it might actually be modifying an entry in another function's stack frame, with potentially disastrous and baffling consequences.

9.11.9 Referencing Data in Free Heap Blocks

A similar error is to reference data in heap blocks that have already been freed. Consider the following example, which allocates an integer array `x` in line 6, prematurely frees block `x` in line 10, and then later references it in line 14:

```
1  int *heapref(int n, int m)
2  {
3      int i;
4      int *x, *y;
5
6      x = (int *)Malloc(n * sizeof(int));
7
8      . // Other calls to malloc and free go here
       .
9
10     free(x);
11
12     y = (int *)Malloc(m * sizeof(int));
13     for (i = 0; i < m; i++)
14         y[i] = x[i]++; /* Oops! x[i] is a word in a free block */
15
16     return y;
17 }
```

Depending on the pattern of `malloc` and `free` calls that occur between lines 6 and 10, when the program references `x[i]` in line 14, the array `x` might be part of some other allocated heap block and may have been overwritten. As with many

memory-related bugs, the error will only become evident later in the program when we notice that the values in y are corrupted.

9.11.10 Introducing Memory Leaks

Memory leaks are slow, silent killers that occur when programmers inadvertently create garbage in the heap by forgetting to free allocated blocks. For example, the following function allocates a heap block x and then returns without freeing it:

```
1  void leak(int n)
2  {
3      int *x = (int *)Malloc(n * sizeof(int));
4
5      return;  /* x is garbage at this point */
6  }
```

If leak is called frequently, then the heap will gradually fill up with garbage, in the worst case consuming the entire virtual address space. Memory leaks are particularly serious for programs such as daemons and servers, which by definition never terminate.

9.12 Summary

Virtual memory is an abstraction of main memory. Processors that support virtual memory reference main memory using a form of indirection known as virtual addressing. The processor generates a virtual address, which is translated into a physical address before being sent to the main memory. The translation of addresses from a virtual address space to a physical address space requires close cooperation between hardware and software. Dedicated hardware translates virtual addresses using page tables whose contents are supplied by the operating system.

Virtual memory provides three important capabilities. First, it automatically caches recently used contents of the virtual address space stored on disk in main memory. The block in a virtual memory cache is known as a page. A reference to a page on disk triggers a page fault that transfers control to a fault handler in the operating system. The fault handler copies the page from disk to the main memory cache, writing back the evicted page if necessary. Second, virtual memory simplifies memory management, which in turn simplifies linking, sharing data between processes, the allocation of memory for processes, and program loading. Finally, virtual memory simplifies memory protection by incorporating protection bits into every page table entry.

The process of address translation must be integrated with the operation of any hardware caches in the system. Most page table entries are located in the L1 cache, but the cost of accessing page table entries from L1 is usually eliminated by an on-chip cache of page table entries called a TLB.

Modern systems initialize chunks of virtual memory by associating them with chunks of files on disk, a process known as memory mapping. Memory mapping provides an efficient mechanism for sharing data, creating new processes, and loading programs. Applications can manually create and delete areas of the virtual address space using the `mmap` function. However, most programs rely on a dynamic memory allocator such as `malloc`, which manages memory in an area of the virtual address space called the heap. Dynamic memory allocators are application-level programs with a system-level feel, directly manipulating memory without much help from the type system. Allocators come in two flavors. Explicit allocators require applications to explicitly free their memory blocks. Implicit allocators (garbage collectors) free any unused and unreachable blocks automatically.

Managing and using memory is a difficult and error-prone task for C programmers. Examples of common errors include dereferencing bad pointers, reading uninitialized memory, allowing stack buffer overflows, assuming that pointers and the objects they point to are the same size, referencing a pointer instead of the object it points to, misunderstanding pointer arithmetic, referencing nonexistent variables, and introducing memory leaks.

Bibliographic Notes

Kilburn and his colleagues published the first description of virtual memory [63]. Architecture texts contain additional details about the hardware's role in virtual memory [46]. Operating systems texts contain additional information about the operating system's role [102, 106, 113]. Bovet and Cesati [11] give a detailed description of the Linux virtual memory system. Intel Corporation provides detailed documentation on 32-bit and 64-bit address translation on IA processors [52].

Knuth wrote the classic work on storage allocation in 1968 [64]. Since that time, there has been a tremendous amount of work in the area. Wilson, Johnstone, Neely, and Boles have written a beautiful survey and performance evaluation of explicit allocators [118]. The general comments in this book about the throughput and utilization of different allocator strategies are paraphrased from their survey. Jones and Lins provide a comprehensive survey of garbage collection [56]. Kernighan and Ritchie [61] show the complete code for a simple allocator based on an explicit free list with a block size and successor pointer in each free block. The code is interesting in that it uses unions to eliminate a lot of the complicated pointer arithmetic, but at the expense of a linear-time (rather than constant-time) free operation. Doug Lea developed a widely used open-source malloc package called `dlmalloc` [67].

Homework Problems

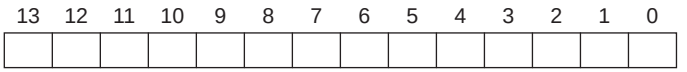
9.11

In the following series of problems, you are to show how the example memory system in Section 9.6.4 translates a virtual address into a physical address and accesses the cache. For the given virtual address, indicate the TLB entry accessed,

the physical address, and the cache byte value returned. Indicate whether the TLB misses, whether a page fault occurs, and whether a cache miss occurs. If there is a cache miss, enter "—" for "Cache byte returned." If there is a page fault, enter "—" for "PPN" and leave parts C and D blank.

Virtual address: `0x027c`

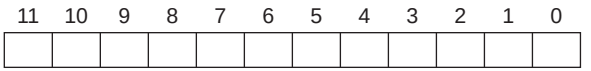
A. Virtual address format



B. Address translation

Parameter	Value
VPN	_____
TLB index	_____
TLB tag	_____
TLB hit? (Y/N)	_____
Page fault? (Y/N)	_____
PPN	_____

C. Physical address format



D. Physical memory reference

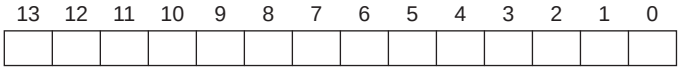
Parameter	Value
Byte offset	_____
Cache index	_____
Cache tag	_____
Cache hit? (Y/N)	_____
Cache byte returned	_____

9.12

Repeat Problem 9.11 for the following address.

Virtual address: `0x03a9`

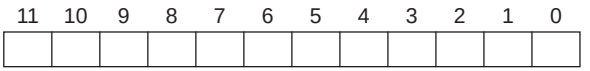
A. Virtual address format



B. Address translation

Parameter	Value
VPN	
TLB index	
TLB tag	
TLB hit? (Y/N)	
Page fault? (Y/N)	
PPN	

C. Physical address format



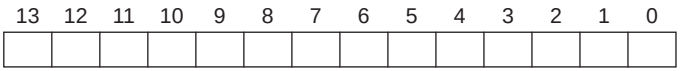
D. Physical memory reference

Parameter	Value
Byte offset	
Cache index	
Cache tag	
Cache hit? (Y/N)	
Cache byte returned	

9.13

Repeat Problem 9.11 for the following address.

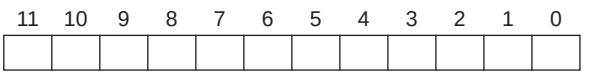
Virtual address: `0x0040`



A. Address translation

Parameter	Value
VPN	
TLB index	
TLB tag	
TLB hit? (Y/N)	
Page fault? (Y/N)	
PPN	

B. Physical address format



C. Physical memory reference

Parameter	Value
Byte offset	_____
Cache index	_____
Cache tag	_____
Cache hit? (Y/N)	_____
Cache byte returned	_____

9.14

Given an input file `hello.txt` that consists of the string `Hello, world!\n`, write a C program that uses `mmap` to change the contents of `hello.txt` to `Jello, world!\n`.

9.15

Determine the block sizes and header values that would result from the following sequence of `malloc` requests. Assumptions: (1) The allocator maintains double-word alignment and uses an implicit free list with the block format from Figure 9.35. (2) Block sizes are rounded up to the nearest multiple of 8 bytes.

Request	Block size (decimal bytes)	Block header (hex)
`malloc(4)`	_____	_____
`malloc(7)`	_____	_____
`malloc(19)`	_____	_____
`malloc(22)`	_____	_____

9.16

Determine the minimum block size for each of the following combinations of alignment requirements and block formats. Assumptions: Explicit free list, 4-byte `pred` and `succ` pointers in each free block, zero-size payloads are not allowed, and headers and footers are stored in 4-byte words.

Alignment	Allocated block	Free block	Minimum block size (bytes)
Single word	Header and footer	Header and footer	_____
Single word	Header, but no footer	Header and footer	_____
Double word	Header and footer	Header and footer	_____
Double word	Header, but no footer	Header and footer	_____

9.17

Develop a version of the allocator in Section 9.9.12 that performs a next-fit search instead of a first-fit search.

9.18

The allocator in Section 9.9.12 requires both a header and a footer for each block in order to perform constant-time coalescing. Modify the allocator so that free blocks require a header and a footer, but allocated blocks require only a header.

9.19

You are given three groups of statements relating to memory management and garbage collection below. In each group, only one statement is true. Your task is to indicate which statement is true.

1. (a) In a buddy system, up to 50% of the space can be wasted due to internal fragmentation.
   (b) The first-fit memory allocation algorithm is slower than the best-fit algorithm (on average).
   (c) Deallocation using boundary tags is fast only when the list of free blocks is ordered according to increasing memory addresses.
   (d) The buddy system suffers from internal fragmentation, but not from external fragmentation.
2. (a) Using the first-fit algorithm on a free list that is ordered according to decreasing block sizes results in low performance for allocations, but avoids external fragmentation.
   (b) For the best-fit method, the list of free blocks should be ordered according to increasing memory addresses.
   (c) The best-fit method chooses the largest free block into which the requested segment fits.
   (d) Using the first-fit algorithm on a free list that is ordered according to increasing block sizes is equivalent to using the best-fit algorithm.
3. Mark&Sweep garbage collectors are called conservative if
   (a) They coalesce freed memory only when a memory request cannot be satisfied.
   (b) They treat everything that looks like a pointer as a pointer.
   (c) They perform garbage collection only when they run out of memory.
   (d) They do not free memory blocks forming a cyclic list.

9.20

Write your own version of `malloc` and `free`, and compare its running time and space utilization to the version of `malloc` provided in the standard C library.

Solutions to Practice Problems

Solution to Problem 9.1 (page 841)

This problem gives you some appreciation for the sizes of different address spaces. At one point in time, a 32-bit address space seemed impossibly large. But now there are database and scientific applications that need more, and you can expect this trend to continue. At some point in your lifetime, expect to find yourself complaining about the cramped 64-bit address space on your personal computer!

Number of address bits (n)	Number of virtual addresses (N)	Largest possible virtual address
4	$2^4 = 16$	$2^4 - 1 = 15$
14	$2^{14} = 16\,K$	$2^{14} - 1 = 16\,K - 1$
24	$2^{24} = 16\,M$	$2^{24} - 1 = 16\,M - 1$
46	$2^{46} = 64\,T$	$2^{46} - 1 = 64\,T - 1$
54	$2^{54} = 16\,P$	$2^{54} - 1 = 16\,P - 1$

Solution to Problem 9.2 (page 843)

Since each virtual page is $P = 2^p$ bytes, there are a total of $2^n/2^p = 2^{n-p}$ possible pages in the system, each of which needs a page table entry (PTE).

n	$P = 2^p$	Number of PTEs
12	1 K	4
16	16 K	4
24	2 M	8
36	1 G	64

Solution to Problem 9.3 (page 852)

You need to understand this kind of problem well in order to fully grasp address translation. Here is how to solve the first subproblem: We are given $n = 64$ virtual address bits and $m = 32$ physical address bits. A page size of $P = 1$ KB means we need $\log_2(1\,K) = 10$ bits for both the VPO and PPO. (Recall that the VPO and PPO are identical.) The remaining address bits are the VPN and PPN, respectively.

	Number of			
P	VPN bits	VPO bits	PPN bits	PPO bits
1 KB	54	10	22	10
2 KB	53	11	21	11
4 KB	52	12	20	12
16 KB	50	14	18	14

Solution to Problem 9.4 (page 860)

Doing a few of these manual simulations is a great way to firm up your understanding of address translation. You might find it helpful to write out all the bits in the addresses and then draw boxes around the different bit fields, such as VPN, TLBI, and so on. In this particular problem, there are no misses of any kind: the TLB has a copy of the PTE and the cache has a copy of the requested data words. See Problems 9.11, 9.12, and 9.13 for some different combinations of hits and misses.

A. `00 0011 1101 0111`

B.

Parameter	Value
VPN	`0xf`
TLB index	`0x3`
TLB tag	`0x3`
TLB hit? (Y/N)	Y
Page fault? (Y/N)	N
PPN	`0xd`

C. `0011 0101 0111`

D.

Parameter	Value
Byte offset	`0x3`
Cache index	`0x5`
Cache tag	`0xd`
Cache hit? (Y/N)	Y
Cache byte returned	`0x1d`

Solution to Problem 9.5 (page 875)

Solving this problem will give you a good feel for the idea of memory mapping. Try it yourself. We haven't discussed the `open`, `fstat`, or `write` functions, so you'll need to read their `man` pages to see how they work.

code/vm/mmapcopy.c

```
1   #include "csapp.h"
2
3   /*
4    * mmapcopy - uses mmap to copy file fd to stdout
5    */
6   void mmapcopy(int fd, int size)
7   {
8       char *bufp; /* ptr to memory-mapped VM area */
9
10      bufp = Mmap(NULL, size, PROT_READ, MAP_PRIVATE, fd, 0);
11      Write(1, bufp, size);
12      return;
13  }
14
15  /* mmapcopy driver */
16  int main(int argc, char **argv)
17  {
18      struct stat stat;
19      int fd;
20
```

```
21      /* Check for required command-line argument */
22      if (argc != 2) {
23          printf("usage: %s <filename>\n", argv[0]);
24          exit(0);
25      }
26
27      /* Copy the input argument to stdout */
28      fd = Open(argv[1], O_RDONLY, 0);
29      fstat(fd, &stat);
30      mmapcopy(fd, stat.st_size);
31      exit(0);
32  }
```

code/vm/mmapcopy.c

Solution to Problem 9.6 (page 885)

This problem touches on some core ideas such as alignment requirements, minimum block sizes, and header encodings. The general approach for determining the block size is to round the sum of the requested payload and the header size to the nearest multiple of the alignment requirement (in this case, 8 bytes). For example, the block size for the `malloc(2)` request is 4 + 2 = 6 rounded up to 8. The block size for the `malloc(20)` request is 20 + 4 = 24 rounded up to 24. No need to round up since it is already aligned.

Request	Block size (decimal bytes)	Block header (hex)
`malloc(2)`	8	`0x9`
`malloc(9)`	16	`0x11`
`malloc(15)`	24	`0x19`
`malloc(20)`	24	`0x19`

Solution to Problem 9.7 (page 888)

The minimum block size can have a significant effect on internal fragmentation. Thus, it is good to understand the minimum block sizes associated with different allocator designs and alignment requirements. The tricky part is to realize that the same block can be allocated or free at different points in time. Thus, the minimum block size is the maximum of the minimum allocated block size and the minimum free block size. For example, in the last subproblem, the minimum allocated block size is a 4-byte header and a 1-byte payload rounded up to 8 bytes. The minimum free block size is a 4-byte header and 4-byte footer, which is already a multiple of 8 and doesn't need to be rounded. So the minimum block size for this allocator is 8 bytes.

Alignment	Allocated block	Free block	Minimum block size (bytes)
Single word	Header and footer	Header and footer	12
Single word	Header, but no footer	Header and footer	8
Double word	Header and footer	Header and footer	16
Double word	Header, but no footer	Header and footer	8

Solution to Problem 9.8 (page 897)

There is nothing very tricky here. But the solution requires you to understand how the rest of our simple implicit-list allocator works and how to manipulate and traverse blocks.

code/vm/malloc/mm.c

```
1  static void *find_fit(size_t asize)
2  {
3      /* First-fit search */
4      void *bp;
5
6      for (bp = heap_listp; GET_SIZE(HDRP(bp)) > 0; bp = NEXT_BLKP(bp)) {
7          if (!GET_ALLOC(HDRP(bp)) && (asize <= GET_SIZE(HDRP(bp)))) {
8              return bp;
9          }
10     }
11     return NULL; /* No fit */
12 #endif
13 }
```

code/vm/malloc/mm.c

Solution to Problem 9.9 (page 897)

This is another warm-up exercise to help you become familiar with allocators. Notice that for this allocator the minimum block size is 16 bytes. If the remainder of the block after splitting would be greater than or equal to the minimum block size, then we go ahead and split the block (lines 6–10). The only tricky part here is to realize that you need to place the new allocated block (lines 6 and 7) before moving to the next block (line 8).

code/vm/malloc/mm.c

```
1   static void place(void *bp, size_t asize)
2   {
3       size_t csize = GET_SIZE(HDRP(bp));
4
5       if ((csize - asize) >= (2*DSIZE)) {
6           PUT(HDRP(bp), PACK(asize, 1));
7           PUT(FTRP(bp), PACK(asize, 1));
8           bp = NEXT_BLKP(bp);
9           PUT(HDRP(bp), PACK(csize-asize, 0));
10          PUT(FTRP(bp), PACK(csize-asize, 0));
11      }
12      else {
13          PUT(HDRP(bp), PACK(csize, 1));
14          PUT(FTRP(bp), PACK(csize, 1));
15      }
16  }
```

code/vm/malloc/mm.c

Solution to Problem 9.10 (page 900)

Here is one pattern that will cause external fragmentation: The application makes numerous allocation and free requests to the first size class, followed by numerous allocation and free requests to the second size class, followed by numerous allocation and free requests to the third size class, and so on. For each size class, the allocator creates a lot of memory that is never reclaimed because the allocator doesn't coalesce, and because the application never requests blocks from that size class again.

Part III

Interaction and Communication between Programs

To this point in our study of computer systems, we have assumed that programs run in isolation, with minimal input and output. However, in the real world, application programs use services provided by the operating system to communicate with I/O devices and with other programs.

This part of the book will give you an understanding of the basic I/O services provided by Unix operating systems and how to use these services to build applications such as Web clients and servers that communicate with each other over the Internet. You will learn techniques for writing concurrent programs, such as Web servers that can service multiple clients at the same time. Writing concurrent application programs can also allow them to execute faster on modern multi-core processors. When you finish this part, you will be well on your way to becoming a power programmer with a mature understanding of computer systems and their impact on your programs.

10

System-Level I/O

10.1 U i I/O 926

10.2 Fi 927

10.3 O i a Cl$_o$ i Fi 929

10.4 R ai a Wri ti Fi 931

10.5 R$_o$ t R ai a Wri ti it t Rio Pa a 933

10.6 R ai Fi M ta ata 939

10.7 R ai Di r t$_o$r C$_o$ t t 941

10.8 S ari Fi 942

10.9 I/O R ir ti$_o$ 945

10.10 Sta an I/O 947

10.11 P tti It T$_o$ t r: W i I/O F ti$_o$ S $_o$ I U ? 947

10.12 S ar 949

Bi li$_o$ ra i N$_o$t 950

H$_o$ $_o$r Pr$_o$ l 950

S$_o$l ti$_o$ t$_o$ Pra ti Pr$_o$ l 951

Input/output (I/O) is the process of copying data between main memory and external devices such as disk drives, terminals, and networks. An input operation copies data from an I/O device to main memory, and an output operation copies data from memory to a device.

All language run-time systems provide higher-level facilities for performing I/O. For example, ANSI C provides the standard I/O library, with functions such as `printf` and `scanf` that perform buffered I/O. The C++ language provides similar functionality with its overloaded << ("put to") and >> ("get from") operators. On Linux systems, these higher-level I/O functions are implemented using system-level Unix I/O functions provided by the kernel. Most of the time, the higher-level I/O functions work quite well and there is no need to use Unix I/O directly. So why bother learning about Unix I/O?

- Understanding Unix I/O will help you understand other systems concepts. I/O is integral to the operation of a system, and because of this, we often encounter circular dependencies between I/O and other systems ideas. For example, I/O plays a key role in process creation and execution. Conversely, process creation plays a key role in how files are shared by different processes. Thus, to really understand I/O, you need to understand processes, and vice versa. We have already touched on aspects of I/O in our discussions of the memory hierarchy, linking and loading, processes, and virtual memory. Now that you have a better understanding of these ideas, we can close the circle and delve into I/O in more detail.
- Sometimes you have no choice but to use Unix I/O. There are some important cases where using higher-level I/O functions is either impossible or inappropriate. For example, the standard I/O library provides no way to access file metadata such as file size or file creation time. Further, there are problems with the standard I/O library that make it risky to use for network programming.

This chapter introduces you to the general concepts of Unix I/O and standard I/O and shows you how to use them reliably from your C programs. Besides serving as a general introduction, this chapter lays a firm foundation for our subsequent study of network programming and concurrency.

10.1 Unix I/O

A Linux file is a sequence of m bytes:

$$B_0, B_1, \ldots, B_k, \ldots, B_{m-1}$$

All I/O devices, such as networks, disks, and terminals, are modeled as files, and all input and output is performed by reading and writing the appropriate files. This elegant mapping of devices to files allows the Linux kernel to export a simple, low-level application interface, known as Unix I/O, that enables all input and output to be performed in a uniform and consistent way:

Opening files. An application announces its intention to access an I/O device by asking the kernel to open the corresponding file. The kernel returns a small nonnegative integer, called a descriptor, that identifies the file in all subsequent operations on the file. The kernel keeps track of all information about the open file. The application only keeps track of the descriptor.

Each process created by a Linux shell begins life with three open files: standard input (descriptor 0), standard output (descriptor 1), and standard error (descriptor 2). The header file <unistd.h> defines constants STDIN_FILENO, STDOUT_FILENO, and STDERR_FILENO, which can be used instead of the explicit descriptor values.

Changing the current file position. The kernel maintains a file position k, initially 0, for each open file. The file position is a byte offset from the beginning of a file. An application can set the current file position k explicitly by performing a seek operation.

Reading and writing files. A read operation copies n > 0 bytes 0(position)]TJ tii1n9a920/T1ly

Aside: End of line (EOL) indicator

One of the clumsy aspects of working with text files is that different systems use different characters to mark the end of a line. Linux and Mac OS X use '\n' (0xa), which is the ASCII line feed (LF) character. However, MS Windows and Internet protocols such as HTTP use the sequence '\r\n' (0xd 0xa), which is the ASCII carriage return (CR) character followed by a line feed (LF). If you create a file `foo.txt` in Windows and then view it in a Linux text editor, you'll see an annoying `^M` at the end of each line, which is how Linux tools display the CR character. You can remove these unwanted CR characters from `foo.txt` in place by running the following command:

```
linux> perl -pi -e "s/\r\n/\n/g" foo.txt
```

least two entries: `.` (dot) is a link to the directory itself, and `..` (dot-dot) is a link to the parent directory in the directory hierarchy (see below). You can create a directory with the `mkdir` command, view its contents with `ls`, and delete it with `rmdir`.

- A socket is a file that is used to communicate with another process across a network (Section 11.4).

Other file types include named pipes, symbolic links, and character and block devices, which are beyond our scope.

The Linux kernel organizes all files in a single directory hierarchy anchored by the root directory named / (slash). Each file in the system is a direct or indirect descendant of the root directory. Figure 10.1 shows a portion of the directory hierarchy on our Linux system.

As part of its context, each process has a current working directory that identifies its current location in the directory hierarchy. You can change the shell's current working directory with the `cd` command.

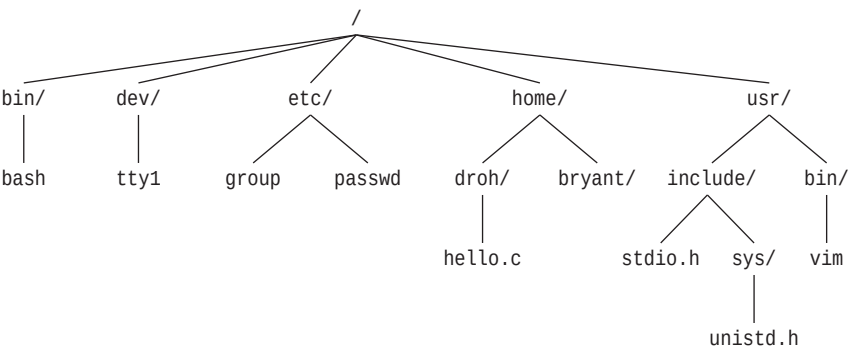


Figure 10.1 Portion of the Linux directory hierarchy. A trailing slash denotes a directory.

Locations in the directory hierarchy are specified by pathnames. A pathname is a string consisting of an optional slash followed by a sequence of filenames separated by slashes. Pathnames have two forms:

- An absolute pathname starts with a slash and denotes a path from the root node. For example, in Figure 10.1, the absolute pathname for `/home/droh/3eoll.cs`.

filA by the

Mask	Description
S_IRUSR	User (owner) can read this file
S_IWUSR	User (owner) can write this file
S_IXUSR	User (owner) can execute this file
S_IRGRP	Members of the owner's group can read this file
S_IWGRP	Members of the owner's group can write this file
S_IXGRP	Members of the owner's group can execute this file
S_IROTH	Others (anyone) can read this file
S_IWOTH	Others (anyone) can write this file
S_IXOTH	Others (anyone) can execute this file

Figure 10.2 Access permission bits. D fi i sys/stat.h.

For example, here is how you might open an existing file with the intent of appending 264v96data:3(of)]Tg

Practice Problem 10.1 (solution page 951)

What is the output of the following program?

```
1   #include "csapp.h"
2
3   int main()
4   {
5       int fd1, fd2;
6
7       fd1 = Open("foo.txt", O_RDONLY, 0);
8       Close(fd1);
9       fd2 = Open("baz.txt", O_RDONLY, 0);
10      printf("fd2 = %d\n", fd2);
11      exit(0);
12  }
```

10.4 Reading and Writing Files

Applications perform input and output by calling the `read` and `write` functions, respectively.

```
#include <unistd.h>

ssize_t read(int fd, void *buf, size_t n);
                        Returns: number of bytes read if OK, 0 on EOF, −1 on error

ssize_t write(int fd, const void *buf, size_t n);
                                Returns: number of bytes written if OK, −1 on error
```

The `read` function copies at most `n` bytes from the current file position of descriptor `fd` to memory location `buf`. A return value of −1 indicates an error, and a return value of 0 indicates EOF. Otherwise, the return value indicates the number of bytes that were actually transferred.

The `write` function copies at most `n` bytes from memory location `buf` to the current file position of descriptor `fd`. Figure 10.3 shows a program that uses `read` and `write` calls to copy the standard input to the standard output, 1 byte at a time.

Applications can explicitly modify the current file position by calling the `lseek` function, which is beyond our scope.

In some situations, `read` and `write` transfer fewer bytes than the application requests. Such *short counts* do *not* indicate an error. They occur for a number of reasons:

Aside What's the difference between ssize_t and size_t?

You might have noticed that the `read` function has a `size_t` input argument and an `ssize_t` return value. So what's the difference between these two types? On x86-64 systems, a `size_t` is defined as an `unsigned long`, and an `ssize_t` (signed size) is defined as a `long`. The `read` function returns a signed size rather than an unsigned size because it must return a -1 on error. Interestingly, the possibility of returning a single -1 reduces the maximum size of a `read` by a factor of 2.

code/io/cpstdin.c

```
1  #include "csapp.h"
2
3  int main(void)
4  {
5      char c;
6
7      while(Read(STDIN_FILENO, &c, 1) != 0)
8          Write(STDOUT_FILENO, &c, 1);
9      exit(0);
10 }
```

code/io/cpstdin.c

Figure 10.3 Using read and write to copy standard input to standard output 1 byte at a time.

Encountering EOF on reads. Suppose that we are ready to read from a file that contains only 20 more bytes from the current file position and that we are reading the file in 50-byte chunks. Then the next `read` will return a short count of 20, and the `read` after that will signal EOF by returning a short count of 0.

Reading text lines from a terminal. If the open file is associated with a terminal (i.e., a keyboard and display), then each `read` function will transfer one text line at a time, returning a short count equal to the size of the text line.

Reading and writing network sockets. If the open file corresponds to a network socket (Section 11.4), then internal buffering constraints and long network delays can cause `read` and `write` to return short counts. Short counts can also occur when you call `read` and `write` on a Linux pipe, an interprocess communication mechanism that is beyond our scope.

In practice, you will never encounter short counts when you read from disk files except on EOF, and you will never encounter short counts when you write to disk files. However, if you want to build robust (reliable) network applications

such as Web servers, then you must deal with short counts by repeatedly calling `read` and `write` until all requested bytes have been transferred.

10.5 Robust Reading and Writing with the Rio Package

In this section, we will develop an I/O package, called the Rio (Robust I/O) package, that handles these short counts for you automatically. The Rio package provides convenient, robust, and efficient I/O in applications such as network programs that are subject to short counts. Rio provides two different kinds of functions:

Unbuffered input and output functions. These functions transfer data directly between memory and a file, with no application-level buffering. They are especially useful for reading and writing binary data to and from networks.

Buffered input functions. These functions allow you to efficiently read text lines and binary data from a file whose contents are cached in an application-level buffer, similar to the one provided for standard I/O functions such as `printf`. Unlike the buffered I/O routines presented in [110], the buffered Rio input functions are thread-safe (Section 12.7.1) and can be interleaved arbitrarily on the same descriptor. For example, you can read some text lines from a descriptor, then some binary data, and then some more text lines.

We are presenting the Rio routines for two reasons. First, we will be using them in the network applications we develop in the next two chapters. Second, by studying the code for these routines, you will gain a deeper understanding of Unix I/O in general.

10.5.1 Rio Unbuffered Input and Output Functions

Applications can transfer data directly between memory and a file by calling the `rio_readn` and `rio_writen` functions.

```
#include "csapp.h"

ssize_t rio_readn(int fd, void *usrbuf, size_t n);
ssize_t rio_writen(int fd, void *usrbuf, size_t n);
```

Returns: number of bytes transferred if OK, 0 on EOF (`rio_readn` only), −1 on error

The `rio_readn` function transfers up to `n` bytes from the current file position of descriptor `fd` to memory location `usrbuf`. Similarly, the `rio_writen` function transfers `n` bytes from location `usrbuf` to descriptor `fd`. The `rio_readn` function can only return a short count if it encounters EOF. The `rio_writen` function never returns a short count. Calls to `rio_readn` and `rio_writen` can be interleaved arbitrarily on the same descriptor.

Figure 10.4 shows the code for `rio_readn` and `rio_writen`. Notice that each function manually restarts the `read` or `write` function if it is interrupted by the return from an application signal handler. To be as portable as possible, we allow for interrupted system calls and restart them when necessary.

10.5.2 Rio Buffered Input Functions

Suppose we wanted to write a program that counts the number of lines in a text file. How might we do this? One approach is to use the `read` function to transfer 1 byte at a time from the file to the user's memory, checking each byte for the newline character. The disadvantage of this approach is that it is inefficient, requiring a trap to the kernel to read each byte in the file.

A better approach is to call a wrapper function (`rio_readlineb`) that copies the text line from an internal read buffer, automatically making a `read` call to refill the buffer whenever it becomes empty. For files that contain both text lines and binary data (such as the HTTP responses described in Section 11.5.3), we also provide a buffered version of `rio_readn`, called `rio_readnb`, that transfers raw bytes from the same read buffer as `rio_readlineb`.

```
#include "csapp.h"

void rio_readinitb(rio_t *rp, int fd);
                                                    Returns: nothing

ssize_t rio_readlineb(rio_t *rp, void *usrbuf, size_t maxlen);
ssize_t rio_readnb(rio_t *rp, void *usrbuf, size_t n);
                    Returns: number of bytes read if OK, 0 on EOF, −1 on error
```

The `rio_readinitb` function is called once per open descriptor. It associates the descriptor `fd` with a read buffer of type `rio_t` at address `rp`.

The `rio_readlineb` function reads the next text line from file `rp` (including the terminating newline character), copies it to memory location `usrbuf`, and terminates the text line with the NULL (zero) character. The `rio_readlineb` function reads at most `maxlen-1` bytes, leaving room for the terminating NULL character. Text lines that exceed `maxlen-1` bytes are truncated and terminated with a NULL character.

The `rio_readnb` function reads up to `n` bytes from file `rp` to memory location `usrbuf`. Calls to `rio_readlineb` and `rio_readnb` can be interleaved arbitrarily on the same descriptor. However, calls to these buffered functions should not be interleaved with calls to the unbuffered `rio_readn` function.

You will encounter numerous examples of the Rio functions in the remainder of this text. Figure 10.5 shows how to use the Rio functions to copy a text file from standard input to standard output, one line at a time.

Figure 10.6 shows the format of a read buffer, along with the code for the `rio_readinitb` function that initializes it. The `rio_readinitb` function sets up an empty read buffer and associates an open file descriptor with that buffer.

code/src/csapp.c

```
ssize_t rio_readn(int fd, void *usrbuf, size_t n)
{
    size_t nleft = n;
    ssize_t nread;
    char *bufp = usrbuf;

    while (nleft > 0) {
        if ((nread = read(fd, bufp, nleft)) < 0) {
            if (errno == EINTR) /* Interrupted by sig handler return */
                nread = 0;      /* and call read() again */
            else
                return -1;      /* errno set by read() */
        }
        else if (nread == 0)
            break;              /* EOF */
        nleft -= nread;
        bufp += nread;
    }
    return (n - nleft);         /* Return >= 0 */
}
```

code/src/csapp.c

code/src/csapp.c

```
ssize_t rio_writen(int fd, void *usrbuf, size_t n)
{
    size_t nleft = n;
    ssize_t nwritten;
    char *bufp = usrbuf;

    while (nleft > 0) {
        if ((nwritten = write(fd, bufp, nleft)) <= 0) {
            if (errno == EINTR)  /* Interrupted by sig handler return */
                nwritten = 0;    /* and call write() again */
            else
                return -1;       /* errno set by write() */
        }
        nleft -= nwritten;
        bufp += nwritten;
    }
    return n;
}
```

code/src/csapp.c

Figure 10.4 The `rio_readn` and `rio_writen` functions.

code/io/cpfile.c

```
#include "csapp.h"

int main(int argc, char **argv)
{
    int n;
    rio_t rio;
    char buf[MAXLINE];

    Rio_readinitb(&rio, STDIN_FILENO);
    while((n = Rio_readlineb(&rio, buf, MAXLINE)) != 0)
        Rio_writen(STDOUT_FILENO, buf, n);
}
```

code/io/cpfile.c

Figure 10.5 Copying a text file from standard input to standard output.

code/include/csapp.h

```
#define RIO_BUFSIZE 8192
typedef struct {
    int rio_fd;                /* Descriptor for this internal buf */
    int rio_cnt;               /* Unread bytes in internal buf */
    char *rio_bufptr;          /* Next unread byte in internal buf */
    char rio_buf[RIO_BUFSIZE]; /* Internal buffer */
} rio_t;
```

code/include/csapp.h

code/src/csapp.c

```
void rio_readinitb(rio_t *rp, int fd)
{
    rp->rio_fd = fd;
    rp->rio_cnt = 0;
    rp->rio_bufptr = rp->rio_buf;
}
```

code/src/csapp.c

Figure 10.6 A read buffer of type `rio_t` and the `rio_readinitb` function that initializes it.

The heart of the Rio read routines is the `rio_read` function shown in Figure 10.7. The `rio_read` function is a buffered version of the Linux `read` function. When `rio_read` is called with a request to read `n` bytes, there are `rp->rio_cnt` unread bytes in the read buffer. If the buffer is empty, then it is replenished with a call to `read`. Receiving a short count from this invocation of `read` is not an error; it simply has the effect of partially filling the read buffer. Once the buffer is

code/src/csapp.c

```
1  static ssize_t rio_read(rio_t *rp, char *usrbuf, size_t n)
2  {
3      int cnt;
4
5      while (rp->rio_cnt <= 0) {  /* Refill if buf is empty */
6          rp->rio_cnt = read(rp->rio_fd, rp->rio_buf,
7                             sizeof(rp->rio_buf));
8          if (rp->rio_cnt < 0) {
9              if (errno != EINTR) /* Interrupted by sig handler return */
10                 return -1;
11         }
12         else if (rp->rio_cnt == 0)  /* EOF */
13             return 0;
14         else
15             rp->rio_bufptr = rp->rio_buf; /* Reset buffer ptr */
16     }
17
18     /* Copy min(n, rp->rio_cnt) bytes from internal buf to user buf */
19     cnt = n;
20     if (rp->rio_cnt < n)
21         cnt = rp->rio_cnt;
22     memcpy(usrbuf, rp->rio_bufptr, cnt);
23     rp->rio_bufptr += cnt;
24     rp->rio_cnt -= cnt;
25     return cnt;
26 }
```

code/src/csapp.c

Figure 10.7 The internal `rio_read` function.

nonempty, `rio_read` copies the minimum of `n` and `rp->rio_cnt` bytes from the read buffer to the user buffer and returns the number of bytes copied.

To an application program, the `rio_read` function has the same semantics as the Linux `read` function. On error, it returns -1 and sets `errno` appropriately. On EOF, it returns 0. It returns a short count if the number of requested bytes exceeds the number of unread bytes in the read buffer. The similarity of the two functions makes it easy to build different kinds of buffered read functions by substituting `rio_read` for `read`. For example, the `rio_readnb` function in Figure 10.8 has the same structure as `rio_readn`, with `rio_read` substituted for `read`. Similarly, the `rio_readlineb` routine in Figure 10.8 calls `rio_read` at most `maxlen-1` times. Each call returns 1 byte from the read buffer, which is then checked for being the terminating newline.

code/src/csapp.c

```
ssize_t rio_readlineb(rio_t *rp, void *usrbuf, size_t maxlen)
{
    int n, rc;
    char c, *bufp = usrbuf;

    for (n = 1; n < maxlen; n++) {
        if ((rc = rio_read(rp, &c, 1)) == 1) {
            *bufp++ = c;
            if (c == '\n') {
                n++;
                break;
            }
        } else if (rc == 0) {
            if (n == 1)
                return 0; /* EOF, no data read */
            else
                break;    /* EOF, some data was read */
        } else
            return -1;    /* Error */
    }
    *bufp = 0;
    return n-1;
}
```

code/src/csapp.c

code/src/csapp.c

```
ssize_t rio_readnb(rio_t *rp, void *usrbuf, size_t n)
{
    size_t nleft = n;
    ssize_t nread;
    char *bufp = usrbuf;

    while (nleft > 0) {
        if ((nread = rio_read(rp, bufp, nleft)) < 0)
            return -1;          /* errno set by read() */
        else if (nread == 0)
            break;              /* EOF */
        nleft -= nread;
        bufp += nread;
    }
    return (n - nleft);         /* Return >= 0 */
}
```

code/src/csapp.c

Figure 10.8 The `rio_readlineb` and `rio_readnb` functions.

Aside Origins of Rio and Robust I/O

The Rio functions are inspired by the `readline`, `readn`, and `writen` functions described by W. Richard Stevens in his classic network programming text [110]. The `rio_readn` and `rio_writen` functions are identical to the Stevens `readn` and `writen` functions. However, the Stevens `readline` function has some limitations that are corrected in Rio. First, because `readline` is buffered and `readn` is not, these two functions cannot be used together on the same descriptor. Second, because it uses a `static` buffer, the Stevens `readline` function is not thread-safe, which required Stevens to introduce a different thread-safe version called `readline_r`. We have corrected both of these flaws with the `rio_readlineb` and `rio_readnb` functions, which are mutually compatible and thread-safe.

10.6 Reading File Metadata

An application can retrieve information about a file (sometimes called the file's metadata) by calling the `stat` and `fstat` functions.

```
#include <unistd.h>
#include <sys/stat.h>

int stat(const char *filename, struct stat *buf);
int fstat(int fd, struct stat *buf);
```

Returns: 0 if OK, −1 on error

The `stat` function takes as input a filename and fills in the members of a `stat` structure shown in Figure 10.9. The `fstat` function is similar, but it takes a file descriptor instead of a filename. We will need the `st_mode` and `st_size` members of the `stat` structure when we discuss Web servers in Section 11.5. The other members are beyond our scope.

The `st_size` member contains the file size in bytes. The `st_mode` member encodes both the file permission bits (Figure 10.2) and the file type (Section 10.2). Linux defines macro predicates in `sys/stat.h` for determining the file type from the `st_mode` member:

S_ISREG(m). Is this a regular file?

S_ISDIR(m). Is this a directory file?

S_ISSOCK(m). Is this a network socket?

Figure 10.10 shows how we might use these macros and the `stat` function to read and interpret a file's `st_mode` bits.

statbuf.h (included by sys/stat.h)

```
/* Metadata returned by the stat and fstat functions */
struct stat {
    dev_t         st_dev;      /* Device */
    ino_t         st_ino;      /* inode */
    mode_t        st_mode;     /* Protection and file type */
    nlink_t       st_nlink;    /* Number of hard links */
    uid_t         st_uid;      /* User ID of owner */
    gid_t         st_gid;      /* Group ID of owner */
    dev_t         st_rdev;     /* Device type (if inode device) */
    off_t         st_size;     /* Total size, in bytes */
    unsigned long st_blksize;  /* Block size for filesystem I/O */
    unsigned long st_blocks;   /* Number of blocks allocated */
    time_t        st_atime;    /* Time of last access */
    time_t        st_mtime;    /* Time of last modification */
    time_t        st_ctime;    /* Time of last change */
};
```

statbuf.h (included by sys/stat.h)

Figure 10.9 The `stat` structure.

code/io/statcheck.c

```
1   #include "csapp.h"
2
3   int main (int argc, char **argv)
4   {
5       struct stat stat;
6       char *type, *readok;
7
8       Stat(argv[1], &stat);
9       if (S_ISREG(stat.st_mode))     /* Determine file type */
10          type = "regular";
11      else if (S_ISDIR(stat.st_mode))
12          type = "directory";
13      else
14          type = "other";
15      if ((stat.st_mode & S_IRUSR)) /* Check read access */
16          readok = "yes";
17      else
18          readok = "no";
19
20      printf("type: %s, read: %s\n", type, readok);
21      exit(0);
22  }
```

code/io/statcheck.c

Figure 10.10 Querying and manipulating a file's `st_mode` bits.

10.7 Reading Directory Contents

Applications can read the contents of a directory with the `readdir` family of functions.

```
#include <sys/types.h>
#include <dirent.h>

DIR *opendir(const char *name);
                                    Returns: pointer to handle if OK, NULL on error
```

The `opendir` function takes a pathname and returns a pointer to a directory stream. A stream is an abstraction for an ordered list of items, in this case a list of directory entries.

```
#include <dirent.h>

struct dirent *readdir(DIR *dirp);
        Returns: pointer to next directory entry if OK, NULL if no more entries or error
```

Each call to `readdir` returns a pointer to the next directory entry in the stream `dirp`, or NULL if there are no more entries. Each directory entry is a structure of the form

```
struct dirent {
    ino_t d_ino;       /* inode number */
    char  d_name[256]; /* Filename */
};
```

Although some versions of Linux include other structure members, these are the only two that are standard across all systems. The `d_name` member is the filename, and `d_ino` is the file location.

On error, `readdir` returns NULL and sets `errno`. Unfortunately, the only way to distinguish an error from the end-of-stream condition is to check if `errno` has been modified since the call to `readdir`.

```
#include <dirent.h>

int closedir(DIR *dirp);
                                    Returns: 0 on success, −1 on error
```

The `closedir` function closes the stream and frees up any of its resources. Figure 10.11 shows how we might use `readdir` to read the contents of a directory.

code/io/readdir.c

```
1  #include "csapp.h"
2
3  int main(int argc, char **argv)
4  {
5      DIR *streamp;
6      struct dirent *dep;
7
8      streamp = Opendir(argv[1]);
9
10     errno = 0;
11     while ((dep = readdir(streamp)) != NULL) {
12         printf("Found file: %s\n", dep->d_name);
13     }
14     if (errno != 0)
15         unix_error("readdir error");
16
17     Closedir(streamp);
18     exit(0);
19 }
```

code/io/readdir.c

Figure 10.11 Reading the contents of a directory.

10.8 Sharing Files

Linux files can be shared in a number of different ways. Unless you have a clear picture of how the kernel represents open files, the idea of file sharing can be quite confusing. The kernel represents open files using three related data structures:

Descriptor table. Each process has its own separate descriptor table whose entries are indexed by the process's open file descriptors. Each open descriptor entry points to an entry in the file table.

File table. The set of open files is represented by a file table that is shared by all processes. Each file table entry consists of (for our purposes) the current file position, a reference count of the number of descriptor entries that currently point to it, and a pointer to an entry in the v-node table. Closing a descriptor decrements the reference count in the associated file table entry. The kernel will not delete the file table entry until its reference count is zero.

v-node table. Like the file table, the v-node table is shared by all processes. Each entry contains most of the information in the `stat` structure, including the `st_mode` and `st_size` members.

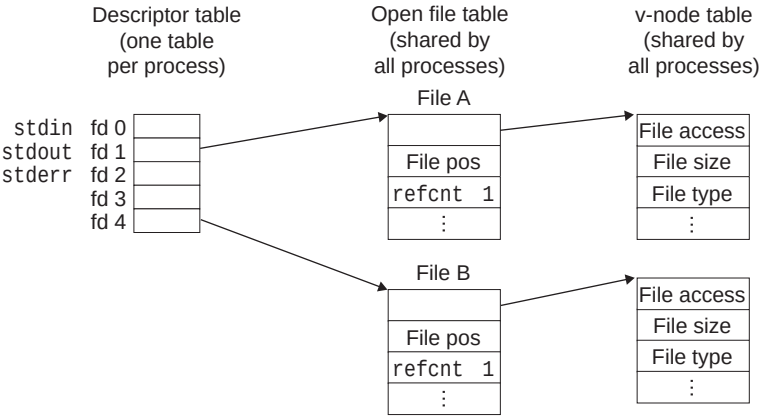


Figure 10.12
Typical kernel data structures for open files. In this example, two descriptors reference two distinct open files. This is the typical situation.

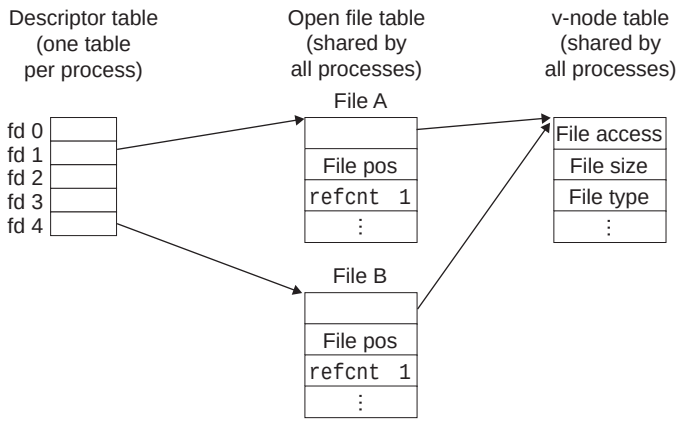


Figure 10.13
File sharing. This configuration can occur when the same file is opened by two separate calls to the `open` function.

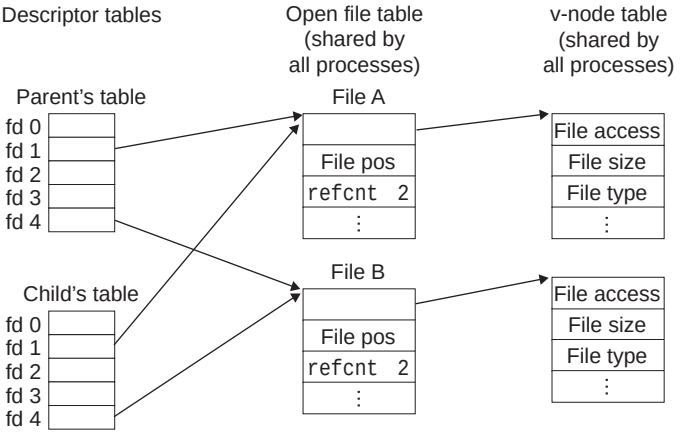
Figure 10.12 shows an example where descriptors 1 and 4 reference two different files through distinct open file table entries. This is the typical situation, where files are not shared and where each descriptor corresponds to a distinct file.

Multiple descriptors can also reference the same file through different file table entries, as shown in Figure 10.13. This might happen, for example, if you were to call the `open` function twice with the same filename. The key idea is that each descriptor has its own distinct file position, so different reads on different descriptors can fetch data from different locations in the file.

We can also understand how parent and child processes share files. Suppose that before a call to `fork`, the parent process has the open files shown in Figure 10.12. Then Figure 10.14 shows the situation after the call to `fork`.

The child gets its own duplicate copy of the parent's descriptor table. Parent and child share the same set of open file tables and thus share the same file position. An important consequence is that the parent and child must both close their descriptors before the kernel will delete the corresponding file table entry.

Figure 10.14
How a child process inherits the parent's open files. T i itial it ati i i Fi r 10.12.



Practice Problem 10.2 (solution page 951)

Suppose the disk file `foobar.txt` consists of the six ASCII characters `foobar`. Then what is the output of the following program?

```
1   #include "csapp.h"
2
3   int main()
4   {
5       int fd1, fd2;
6       char c;
7
8       fd1 = Open("foobar.txt", O_RDONLY, 0);
9       fd2 = Open("foobar.txt", O_RDONLY, 0);
10      Read(fd1, &c, 1);
11      Read(fd2, &c, 1);
12      printf("c = %c\n", c);
13      exit(0);
14  }
```

Practice Problem 10.3 (solution page 951)

As before, suppose the disk file `foobar.txt` consists of the six ASCII characters `foobar`. Then what is the output of the following program?

```
1   #include "csapp.h"
2
3   int main()
4   {
5       int fd;
6       char c;
```

```
7
8       fd = Open("foobar.txt", O_RDONLY, 0);
9       if (Fork() == 0) {
10          Read(fd, &c, 1);
11          exit(0);
12      }
13      Wait(NULL);
14      Read(fd, &c, 1);
15      printf("c = %c\n", c);
16      exit(0);
17  }
```

10.9 I/O Redirection

Linux shells provide I/O redirection operators that allow users to associate standard input and output with disk files. For example, typing

```
linux> ls > foo.txt
```

causes the shell to load and execute the `ls` program, with standard output redirected to disk file `foo.txt`. As we will see in Section 11.5, a Web server performs a similar kind of redirection when it runs a CGI program on behalf of the client. So how does I/O redirection work? One way is to use the `dup2` function.

```
#include <unistd.h>

int dup2(int oldfd, int newfd);
```

Returns: nonnegative descriptor if OK, −1 on error

The `dup2` function copies descriptor table entry `oldfd` to descriptor table entry `newfd`, overwriting the previous contents of descriptor table entry `newfd`. If `newfd` was already open, then `dup2` closes `newfd` before it copies `oldfd`.

Suppose that before calling `dup2(4,1)`, we have the situation in Figure 10.12, where descriptor 1 (standard output) corresponds to file A (say, a terminal) and descriptor 4 corresponds to file B (say, a disk file). The reference counts for A and B are both equal to 1. Figure 10.15 shows the situation after calling `dup2(4,1)`. Both descriptors now point to file B; file A has been closed and its file table and v-node table entries deleted; and the reference count for file B has been incremented. From this point on, any data written to standard output are redirected to file B.

Practice Problem 10.4 (solution page 951)

How would you use `dup2` to redirect standard input to descriptor 5?

Aside Right hoinky

To avoid confusion with other bracket-type operators such as ']' and '[', we have always referred to the shell's '>' operator as a "right hoinky" and the '<' operator as a "left hoinky."

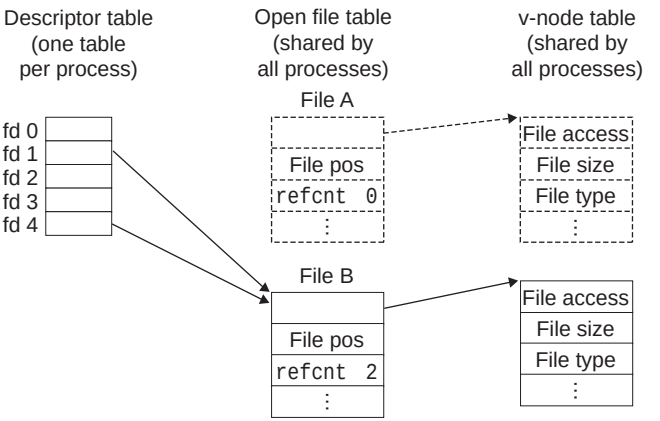


Figure 10.15 Kernel data structures after redirecting standard output by calling dup2(4,1). Initial state is shown in Figure 10.12.

Practice Problem 10.5 (solution page 952)

Assuming that the disk file `foobar.txt` consists of the six ASCII characters `foobar`, what is the output of the following program?

```
1   #include "csapp.h"
2
3   int main()
4   {
5       int fd1, fd2;
6       char c;
7
8       fd1 = Open("foobar.txt", O_RDONLY, 0);
9       fd2 = Open("foobar.txt", O_RDONLY, 0);
10      Read(fd2, &c, 1);
11      Dup2(fd2, fd1);
12      Read(fd1, &c, 1);
13      printf("c = %c\n", c);
14      exit(0);
15  }
```

10.10 Standard I/O

The C language defines a set of higher-level input and output functions, called the *standard I/O library*, that provides programmers with a higher-level alternative to Unix I/O. The library (`libc`) provides functions for opening and closing files (`fopen` and `fclose`), reading and writing bytes (`fread` and `fwrite`), reading and writing strings (`fgets` and `fputs`), and sophisticated formatted I/O (`scanf` and `printf`).

The standard I/O library models an open file as a *stream*. To the programmer, a stream is a pointer to a structure of type `FILE`. Every ANSI C program begins with three open streams, `stdin`, `stdout`, and `stderr`, which correspond to standard input, standard output, and standard error, respectively:

```
#include <stdio.h>
extern FILE *stdin;    /* Standard input (descriptor 0) */
extern FILE *stdout;   /* Standard output (descriptor 1) */
extern FILE *stderr;   /* Standard error (descriptor 2) */
```

A stream of type FILE is an abstraction for a file descriptor and a *stream buffer*. The purpose of the stream buffer is the same as the Rio read buffer: to minimize the number of expensive Linux I/O system calls. For example, suppose we have a program that makes repeated calls to the standard I/O `getc` function, where each invocation returns the next character from a file. When `getc` is called the first time, the library fills the stream buffer with a single call to the `read` function and then returns the first byte in the buffer to the application. As long as there are unread bytes in the buffer, subsequent calls to `getc` can be served directly from the stream buffer.

10.11 Putting It Together: Which I/O Functions Should I Use?

Figure 10.16 summarizes the various I/O packages that we have discussed in this chapter.

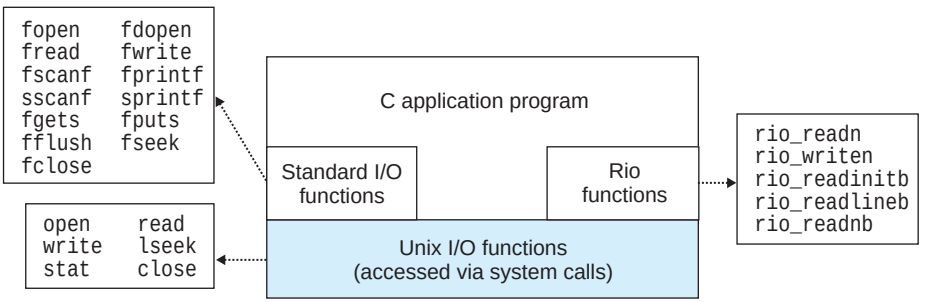


Figure 10.16 Relationship between Unix I/O, standard I/O, and Rio.

The Unix I/O model is implemented in the operating system kernel. It is available to applications through functions such as `open`, `close`, `lseek`, `read`, `write`, and `stat`. The higher-level Rio and standard I/O functions are implemented "on top of" (using) the Unix I/O functions. The Rio functions are robust wrappers for `read` and `write` that were developed specifically for this textbook. They automatically deal with short counts and provide an efficient buffered approach for reading text lines. The standard I/O functions provide a more complete buffered alternative to the Unix I/O functions, including formatted I/O routines such as `printf` and `scanf`.

So which of these functions should you use in your programs? Here are some basic guidelines:

G1: Use the standard I/O functions whenever possible. The standard I/O functions are the method of choice for I/O on disk and terminal devices. Most C programmers use standard I/O exclusively throughout their careers, never bothering with the lower-level Unix I/O functions (except possibly `stat`, which has no counterpart in the standard I/O library). Whenever possible, we recommend that you do likewise.

G2: Don't use `scanf` or `rio_readlineb` to read binary files. Functions like `scanf` and `rio_readlineb` are designed specifically for reading text files. A common error that students make is to use these functions to read binary data, causing their programs to fail in strange and unpredictable ways. For example, binary files might be littered with many `0xa` bytes that have nothing to do with terminating text lines.

G3: Use the Rio functions for I/O on network sockets. Unfortunately, standard I/O poses some nasty problems when we attempt to use it for input and output on networks. As we will see in Section 11.4, the Linux abstraction for a network is a type of file called a socket. Like any Linux file, sockets are referenced by file descriptors, known in this case as socket descriptors. Application processes communicate with processes running on other computers by reading and writing socket descriptors.

Standard I/O streams are full duplex in the sense that programs can perform input and output on the same stream. However, there are poorly documented restrictions on streams that interact badly with restrictions on sockets:

Restriction 1: Input functions following output functions. An input function cannot follow an output function without an intervening call to `fflush`, `fseek`, `fsetpos`, or `rewind`. The `fflush` function empties the buffer associated with a stream. The latter three functions use the Unix I/O `lseek` function to reset the current file position.

Restriction 2: Output functions following input functions. An output function cannot follow an input function without an intervening call to `fseek`, `fsetpos`, or `rewind`, unless the input function encounters an end-of-file.

These restrictions pose a problem for network applications because it is illegal to use the `lseek` function on a socket. The first restriction on stream I/O can be worked around by adopting a discipline of flushing the buffer before every input operation. However, the only way to work around the second restriction is to open two streams on the same open socket descriptor, one for reading and one for writing:

```
FILE *fpin, *fpout;

fpin = fdopen(sockfd, "r");
fpout = fdopen(sockfd, "w");
```

But this approach has problems as well, because it requires the application to call `fclose` on both streams in order to free the memory resources associated with each stream and avoid a memory leak:

```
fclose(fpin);
fclose(fpout);
```

Each of these operations attempts to close the same underlying socket descriptor, so the second `close` operation will fail. This is not a problem for sequential programs, but closing an already closed descriptor in a threaded program is a recipe for disaster (see Section 12.7.4).

Thus, we recommend that you not use the standard I/O functions for input and output on network sockets. Use the robust Rio functions instead. If you need formatted output, use the `sprintf` function to format a string in memory, and then send it to the socket using `rio_writen`. If you need formatted input, use `rio_readlineb` to read an entire text line, and then use `sscanf` to extract different fields from the text line.

10.12 Summary

Linux provides a small number of system-level functions, based on the Unix I/O model, that allow applications to open, close, read, and write files, to fetch file metadata, and to perform I/O redirection. Linux read and write operations are subject to short counts that applications must anticipate and handle correctly. Instead of calling the Unix I/O functions directly, applications should use the Rio package, which deals with short counts automatically by repeatedly performing read and write operations until all of the requested data have been transferred.

The Linux kernel uses three related data structures to represent open files. Entries in a descriptor table point to entries in the open file table, which point to entries in the v-node table. Each process has its own distinct descriptor table, while all processes share the same open file and v-node tables. Understanding the general organization of these structures clarifies our understanding of both file sharing and I/O redirection.

The standard I/O library is implemented on top of Unix I/O and provides a powerful set of higher-level I/O routines. For most applications, standard I/O is the

simpler, preferred alternative to Unix I/O. However, because of some mutually incompatible restrictions on standard I/O and network files, Unix I/O, rather than standard I/O, should be used for network applications.

Bibliographic Notes

Kerrisk gives a comprehensive treatment of Unix I/O and the Linux file system [62]. Stevens wrote the original standard reference text for Unix I/O [111]. Kernighan and Ritchie give a clear and complete discussion of the standard I/O functions [61].

Homework Problems

10.6
What is the output of the following program?

```
1   #include "csapp.h"
2
3   int main()
4   {
5       int fd1, fd2;
6
7       fd1 = Open("foo.txt", O_RDONLY, 0);
8       fd2 = Open("bar.txt", O_RDONLY, 0);
9       Close(fd2);
10      fd2 = Open("baz.txt", O_RDONLY, 0);
11      printf("fd2 = %d\n", fd2);
12      exit(0);
13  }
```

10.7
Modify the `cpfile` program in Figure 10.5 so that it uses the Rio functions to copy standard input to standard output, MAXBUF bytes at a time.

10.8
Write a version of the `statcheck` program in Figure 10.10, called `fstatcheck`, that takes a descriptor number on the command line rather than a filename.

10.9
Consider the following invocation of the `fstatcheck` program from Problem 10.8:

```
linux> fstatcheck 3 < foo.txt
```

You might expect that this invocation of `fstatcheck` would fetch and display metadata for file `foo.txt`. However, when we run it on our system, it fails with a "bad file descriptor." Given this behavior, fill in the pseudocode that the shell must be executing between the `fork` and `execve` calls:

```
if (Fork() == 0) { /* child */
    /* What code is the shell executing right here? */
    Execve("fstatcheck", argv, envp);
}
```

10.10

Modify the `cpfile` program in Figure 10.5 so that it takes an optional command-line argument `infile`. If `infile` is given, then copy `infile` to standard output; otherwise, copy standard input to standard output as before. The twist is that your solution must use the original copy loop (lines 9–11) for both cases. You are only allowed to insert code, and you are not allowed to change any of the existing code.

Solutions to Practice Problems

Solution to Problem 10.1 (page 931)

Unix processes begin life with open descriptors assigned to `stdin` (descriptor 0), `stdout` (descriptor 1), and `stderr` (descriptor 2). The `open` function always returns the lowest unopened descriptor, so the first call to `open` returns descriptor 3. The call to the `close` function frees up descriptor 3. The final call to `open` returns descriptor 3, and thus the output of the program is `fd2 = 3`.

Solution to Problem 10.2 (page 944)

The descriptors `fd1` and `fd2` each have their own open file table entry, so each descriptor has its own file position for `foobar.txt`. Thus, the read from `fd2` reads the first byte of `foobar.txt`, and the output is

```
c = f
```

and not

```
c = o
```

as you might have thought initially.

Solution to Problem 10.3 (page 944)

Recall that the child inherits the parent's descriptor table and that all processes shared the same open file table. Thus, the descriptor `fd` in both the parent and child points to the same open file table entry. When the child reads the first byte of the file, the file position increases by 1. Thus, the parent reads the second byte, and the output is

```
c = o
```

Solution to Problem 10.4 (page 945)

To redirect standard input (descriptor 0) to descriptor 5, we would call `dup2(5,0)`, or equivalently, `dup2(5,STDIN_FILENO)`.

Solution to Problem 10.5 (page 946)

At first glance, you might think the output would be

```
c = f
```

but because we are redirecting `fd1` to `fd2`, the output is really

```
c = o
```

11

Network Programming

11.1 The Client-Server Programming Model 954

11.2 Networks 955

11.3 The Global IP Internet 960

11.4 The Sockets Interface 968

11.5 Web Servers 984

11.6 Putting It Together: The Tiny Web Server 992

11.7 Summary 1000

Bibliographic Notes 1001

Homework Problems 1001

Solutions to Practice Problems 1002

Network applications are everywhere. Any time you browse the Web, send an email message, or play an online game, you are using a network application. Interestingly, all network applications are based on the same basic programming model, have similar overall logical structures, and rely on the same programming interface.

Network applications rely on many of the concepts that you have already learned in our study of systems. For example, processes, signals, byte ordering, memory mapping, and dynamic storage allocation all play important roles. There are new concepts to master as well. You will need to understand the basic client-server programming model and how to write client-server programs that use the services provided by the Internet. At the end, we will tie all of these ideas together by developing a tiny but functional Web server that can serve both static and dynamic content with text and graphics to real Web browsers.

11.1 The Client-Server Programming Model

Every network application is based on the client-server model. With this model, an application consists of a server process and one or more client processes. A server manages some resource, and it provides some service for its clients by manipulating that resource. For example, a Web server manages a set of disk files that it retrieves and executes on behalf of clients. An FTP server manages a set of disk files that it stores and retrieves for clients. Similarly, an email server manages a spool file that it reads and updates for clients.

The fundamental operation in the client-server model is the transaction (Figure 11.1). A client-server transaction consists of four steps:

1. When a client needs service, it initiates a transaction by sending a request to the server. For example, when a Web browser needs a file, it sends a request to a Web server.
2. The server receives the request, interprets it, and manipulates its resources in the appropriate way. For example, when a Web server receives a request from a browser, it reads a disk file.
3. The server sends a response to the client and then waits for the next request. For example, a Web server sends the file back to a client.

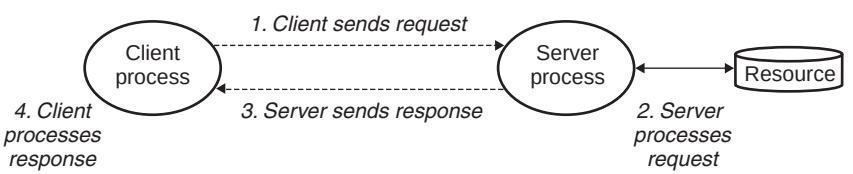


Figure 11.1 A client-server transaction.

Aside Client-server transactions vs. database transactions

Client-server transactions are not database transactions and do not share any of their properties, such as atomicity. In our context, a transaction is simply a sequence of steps carried out by a client and a server.

4. The client receives the response and manipulates it. For example, after a Web browser receives a page from the server, it displays it on the screen.

It is important to realize that clients and servers are processes and not machines, or hosts as they are often called in this context. A single host can run many different clients and servers concurrently, and a client and server transaction can be on the same or different hosts. The client-server model is the same, regardless of the mapping of clients and servers to hosts.

11.2 Networks

Clients and servers often run on separate hosts and communicate using the hardware and software resources of a computer network. Networks are sophisticated systems, and we can only hope to scratch the surface here. Our aim is to give you a workable mental model from a programmer's perspective.

To a host, a network is just another I/O device that serves as a source and sink for data, as shown in Figure 11.2.

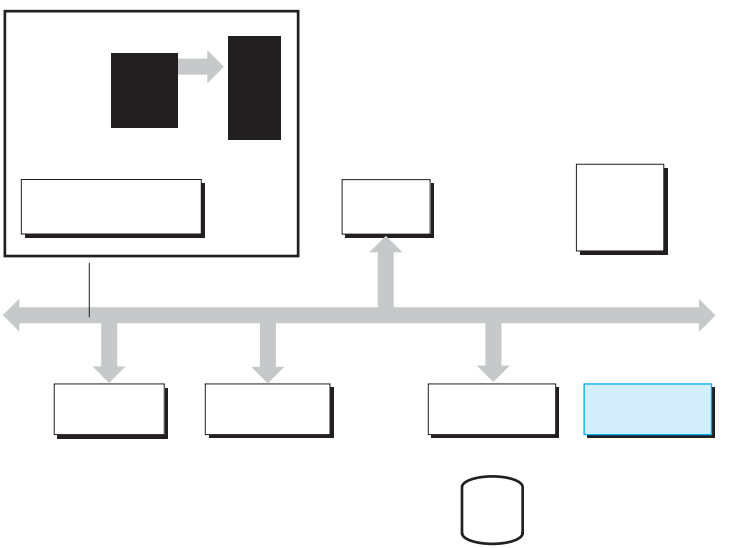
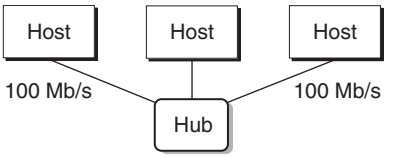


Figure 11.2
Hardware organization of a network host.

Figure 11.3
Ethernet segment.



An adapter plugged into an expansion slot on the I/O bus provides the physical interface to the network. Data received from the network are copied from the adapter across the I/O and memory buses into memory, typically by a DMA transfer. Similarly, data can also be copied from memory to the network.

Physically, a network is a hierarchical system that is organized by geographical proximity. At the lowest level is a LAN (local area network) that spans a building or a campus. The most popular LAN technology by far is Ethernet, which was developed in the mid-1970s at Xerox PARC. Ethernet has proven to be remarkably resilient, evolving from 3 Mb/s to 10 Gb/s.

An Ethernet segment consists of some wires (usually twisted pairs of wires) and a small box called a hub, as shown in Figure 11.3. Ethernet segments typically span small areas, such as a room or a floor in a building. Each wire has the same maximum bit bandwidth, typically 100 Mb/s or 1 Gb/s. One end is attached to an adapter on a host, and the other end is attached to a port on the hub. A hub slavishly copies every bit that it receives on each port to every other port. Thus, every host sees every bit.

Each Ethernet adapter has a globally unique 48-bit address that is stored in a nonvolatile memory on the adapter. A host can send a chunk of bits called a frame to any other host on the segment. Each frame includes some fixed number of header bits that identify the source and destination of the frame and the frame length, followed by a payload of data bits. Every host adapter sees the frame, but only the destination host actually reads it.

Multiple Ethernet segments can be connected into larger LANs, called bridged Ethernets, using a set of wires and small boxes called bridges, as shown in Figure 11.4. Bridged Ethernets can span entire buildings or campuses. In a bridged Ethernet, some wires connect bridges to bridges, and others connect bridges to hubs. The bandwidths of the wires can be different. In our example, the bridge–bridge wire has a 1 Gb/s bandwidth, while the four hub–bridge wires have bandwidths of 100 Mb/s.

Bridges make better use of the available wire bandwidth than hubs. Using a clever distributed algorithm, they automatically learn over time which hosts are reachable from which ports and then selectively copy frames from one port to another only when it is necessary. For example, if host A sends a frame to host B, which is on the segment, then bridge X will throw away the frame when it arrives at its input port, thus saving bandwidth on the other segments. However, if host A sends a frame to host C on a different segment, then bridge X will copy the frame only to the port connected to bridge Y, which will copy the frame only to the port connected to host C's segment.

Aside Internet vs. internet

We will always use lowercase internet to denote the general concept, and uppercase Internet to denote a specific implementation—namely, the global IP Internet.

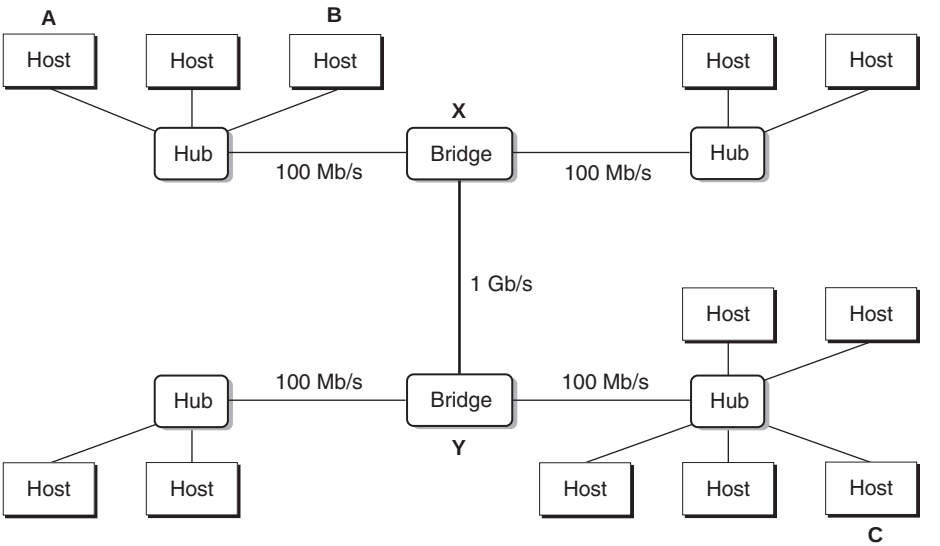
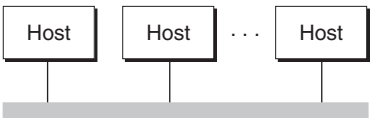


Figure 11.4 Bridged Ethernet segments.

Figure 11.5
Conceptual view of a LAN.



To simplify our pictures of LANs, we will draw the hubs and bridges and the wires that connect them as a single horizontal line, as shown in Figure 11.5.

At a higher level in the hierarchy, multiple incompatible LANs can be connected by specialized computers called routers to form an internet (interconnected network). Each router has an adapter (port) for each network that it is connected to. Routers can also connect high-speed point-to-point phone connections, which are examples of networks known as WANs (wide area networks), so called because they span larger geographical areas than LANs. In general, routers can be used to build internets from arbitrary collections of LANs and WANs. For example, Figure 11.6 shows an example internet with a pair of LANs and WANs connected by three routers.

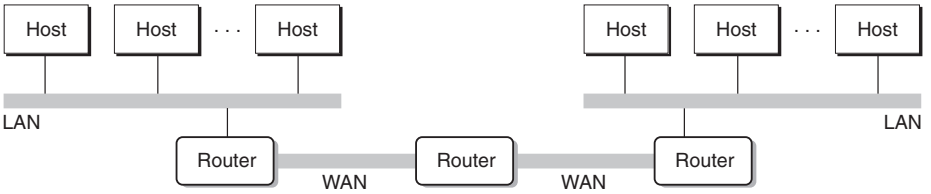


Figure 11.6 A small internet. T o LAN a t o WAN ar o t r r o t r.

The crucial property of an internet is that it can consist of different LANs and WANs with radically different and incompatible technologies. Each host is physically connected to every other host, but how is it possible for some source host to send data bits to another destination host across all of these incompatible networks?

The solution is a layer of protocol software running on each host and router that smoothes out the differences between the different networks. This software implements a protocol that governs how hosts and routers cooperate in order to transfer data. The protocol must provide two basic capabilities:

Naming scheme. Different LAN technologies have different and incompatible ways of assigning addresses to hosts. The internet protocol smoothes these differences by defining a uniform format for host addresses. Each host is then assigned at least one of these internet addresses that uniquely identifies it.

Delivery mechanism. Different networking technologies have different and incompatible ways of encoding bits on wires and of packaging these bits into frames. The internet protocol smoothes these differences by defining a uniform way to bundle up data bits into discrete chunks called packets. A packet consists of a header, which contains the packet size and addresses of the source and destination hosts, and a payload, which contains data bits sent from the source host.

Figure 11.7 shows an example of how hosts and routers use the internet protocol to transfer data across incompatible LANs. The example internet consists of two LANs connected by a router. A client running on host A, which is attached to LAN1, sends a sequence of data bytes to a server running on host B, which is attached to LAN2. There are eight basic steps:

1. The client on host A invokes a system call that copies the data from the client's virtual address space into a kernel buffer.
2. The protocol software on host A creates a LAN1 frame by appending an internet header and a LAN1 frame header to the data. The internet header is addressed to internet host B. The LAN1 frame header is addressed to the router. It then passes the frame to the adapter. Notice that the payload of the LAN1 frame is an internet packet, whose payload is the actual user data. This kind of encapsulation is one of the fundamental insights of internetworking.

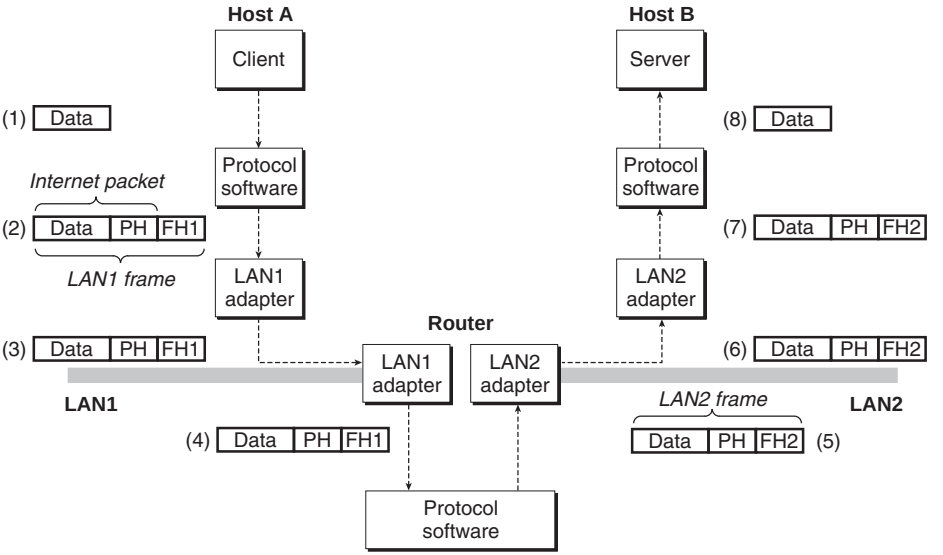
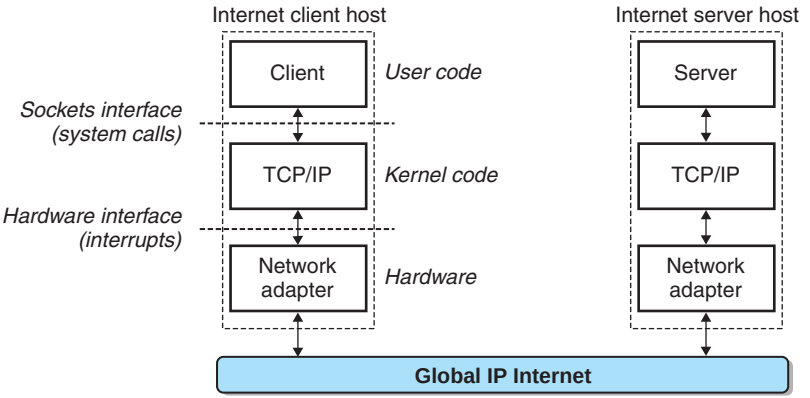

Host A
Client
Server
Host B
(1) Data
(8) Data
Protocol software
Protocol software
Internet packet
(2) Data PH FH1
(7) Data PH FH2
LAN1 frame
LAN1 adapter
LAN2 adapter
Router
(3) Data PH FH1
LAN1 adapter
LAN2 adapter
(6) Data PH FH2
LAN1
LAN2 frame
LAN2
(4) Data PH FH1
Data PH FH2 (5)
Protocol software

Figure 11.8
Hardware and software organization of an Internet application.



11.3 The Global IP Internet

The global IP Internet is the most famous and successful implementation of an internet. It has existed in one form or another since 1969. While the internal architecture of the Internet is complex and constantly changing, the organization of client-server applications has remained remarkably stable since the early 1980s. Figure 11.8 shows the basic hardware and software organization of an Internet client-server application.

Each Internet host runs software that implements the TCP/IP protocol (Transmission Control Protocol/Internet Protocol), which is supported by almost every modern computer system. Internet clients and servers communicate using a mix of sockets interface functions and Unix I/O functions. (We will describe the sockets interface in Section 11.4.) The sockets functions are typically implemented as system calls that trap into the kernel and call various kernel-mode functions in TCP/IP.

TCP/IP is actually a family of protocols, each of which contributes different capabilities. For example, IP provides the basic naming scheme and a delivery mechanism that can send packets, known as datagrams, from one Internet host to any other host. The IP mechanism is unreliable in the sense that it makes no effort to recover if datagrams are lost or duplicated in the network. UDP (Unreliable Datagram Protocol) extends IP slightly, so that datagrams can be transferred from process to process, rather than host to host. TCP is a complex protocol that builds on IP to provide reliable full duplex (bidirectional) connections between processes. To simplify our discussion, we will treat TCP/IP as a single monolithic protocol. We will not discuss its inner workings, and we will only discuss some of the basic capabilities that TCP and IP provide to application programs. We will not discuss UDP.

From a programmer's perspective, we can think of the Internet as a worldwide collection of hosts with the following properties:

- The set of hosts is mapped to a set of 32-bit IP addresses.

Aside IP 4 a IP 6

The original Internet protocol, with its 32-bit addresses, is known as Internet Protocol Version 4 (IPv4). In 1996, the Internet Engineering Task Force (IETF) proposed a new version of IP, called Internet Protocol Version 6 (IPv6), that uses 128-bit addresses and that was intended as the successor to IPv4. However, as of 2015, almost 20 years later, the vast majority of Internet traffic is still carried by IPv4 networks. For example, only 4 percent of users access Google services using IPv6 [42].

Because of its low adoption rate, we will not discuss IPv6 in any detail in this book and will focus exclusively on the concepts behind IPv4. When we talk about the Internet, what we mean is the Internet based on IPv4. Nonetheless, the techniques for writing clients and servers that we will teach you later in this chapter are based on modern interfaces that are independent of any particular protocol.

- The set of IP addresses is mapped to a set of identifiers called Internet domain names.
- A process on one Internet host can communicate with a process on any other Internet host over a connection.

The following sections discuss these fundamental Internet ideas in more detail.

11.3.1 IP Addresses

An IP address is an unsigned 32-bit integer. Network programs store IP addresses in the IP address structure shown in Figure 11.9.

Storing a scalar address in a structure is an unfortunate artifact from the early implementations of the sockets interface. It would make more sense to define a scalar type for IP addresses, but it is too late to change now because of the enormous installed base of applications.

Because Internet hosts can have different host byte orders, TCP/IP defines a uniform network byte order (big-endian byte order) for any integer data item, such as an IP address, that is carried across the network in a packet header. Addresses in IP address structures are always stored in (big-endian) network byte order, even if the host byte order is little-endian. Unix provides the following functions for converting between network and host byte order.

code/netp/netpfragments.c

```
/* IP address structure */
struct in_addr {
    uint32_t  s_addr; /* Address in network byte order (big-endian) */
};
```

code/netp/netpfragments.c

Figure 11.9 IP address structure.

```
#include <arpa/inet.h>

uint32_t htonl(uint32_t hostlong);
uint16_t htons(uint16_t hostshort);
                                        Returns: value in network byte order

uint32_t ntohl(uint32_t netlong);
uint16_t ntohs(unit16_t netshort);
                                        Returns: value in host byte order
```

The `htonl` function converts an unsigned 32-bit integer from host byte order to network byte order. The `ntohl` function converts an unsigned 32-bit integer from network byte order to host byte order. The `htons` and `ntohs` functions perform corresponding conversions for unsigned 16-bit integers. Note that there are no equivalent functions for manipulating 64-bit values.

IP addresses are typically presented to humans in a form known as dotted-decimal notation, where each byte is represented by its decimal value and separated from the other bytes by a period. For example, `128.2.194.242` is the dotted-decimal representation of the address `0x8002c2f2`. On Linux systems, you can use the HOSTNAME command to determine the dotted-decimal address of your own host:

```
linux> hostname -i
128.2.210.175
```

Application programs can convert back and forth between IP addresses and dotted-decimal strings using the functions `inet_pton` and `inet_ntop`.

```
#include <arpa/inet.h>

int inet_pton(AF_INET, const char *src, void *dst);
                    Returns: 1 if OK, 0 if src is invalid dotted decimal, −1 on error

const char *inet_ntop(AF_INET, const void *src, char *dst,
                      socklen_t size);
                    Returns: pointer to a dotted-decimal string if OK, NULL on error
```

In these function names, the "n" stands for network and the "p" stands for presentation. They can manipulate either 32-bit IPv4 addresses (`AF_INET`), as shown here, or 128-bit IPv6 addresses (`AF_INET6`), which we do not cover.

The `inet_pton` function converts a dotted-decimal string (`src`) to a binary IP address in network byte order (`dst`). If `src` does not point to a valid dotted-decimal string, then it returns 0. Any other error returns −1 and sets `errno`. Similarly, the `inet_ntop` function converts a binary IP address in network byte order (`src`) to the corresponding dotted-decimal representation and copies at most `size` bytes of the resulting null-terminated string to `dst`.

Practice Problem 11.1 (solution page 1002)

Complete the following table:

Dotted-decimal address	Hex address
107.212.122.205	_____
64.12.149.13	_____
107.212.96.29	_____
_____	0x00000080
_____	0xFFFFFF00
_____	0x0A010140

Practice Problem 11.2 (solution page 1003)

Write a program `hex2dd.c` that converts its 16-bit hex argument to a 16-bit network byte order and prints the result. For example

```
linux> ./hex2dd 0x400
1024
```

Practice Problem 11.3 (solution page 1003)

Write a program `dd2hex.c` that converts its 16-bit network byte order to a 16-bit hex number and prints the result. For example,

```
linux> ./dd2hex 1024
0x400
```

11.3.2 Internet Domain Names

Internet clients and servers use IP addresses when they communicate with each other. However, large integers are difficult for people to remember, so the Internet also defines a separate set of more human-friendly domain names, as well as a mechanism that maps the set of domain names to the set of IP addresses. A domain name is a sequence of words (letters, numbers, and dashes) separated by periods, such as `whaleshark.ics.cs.cmu.edu`.

The set of domain names forms a hierarchy, and each domain name encodes its position in the hierarchy. An example is the easiest way to understand this. Figure 11.10 shows a portion of the domain name hierarchy.

The hierarchy is represented as a tree. The nodes of the tree represent domain names that are formed by the path back to the root. Subtrees are referred to as subdomains. The first level in the hierarchy is an unnamed root node. The next level is a collection of first-level domain names that are defined by a nonprofit organization called ICANN (Internet Corporation for Assigned Names and Numbers). Common first-level domains include `com`, `edu`, `gov`, `org`, and `net`.

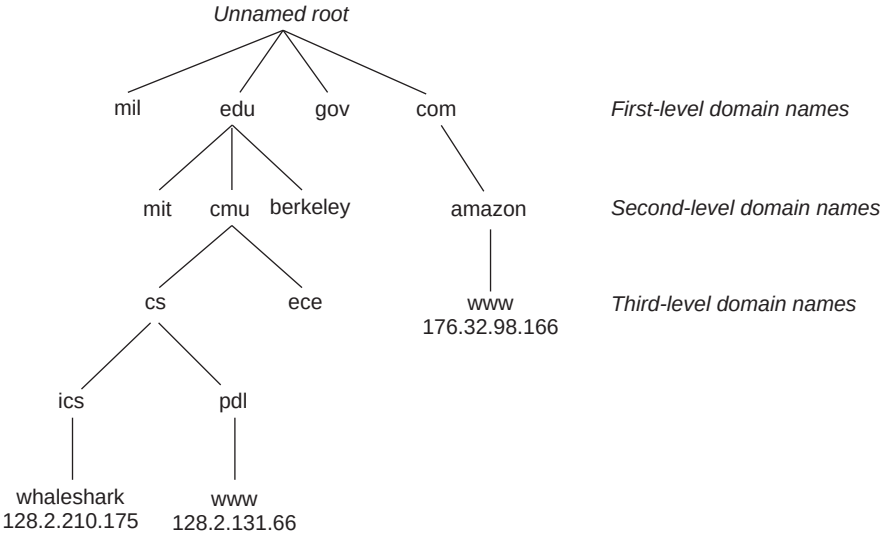


Figure 11.10 Subset of the Internet domain name hierarchy.

At the next level are second-level domain names such as `cmu.edu`, which are assigned on a first-come first-serve basis by various authorized agents of ICANN. Once an organization has received a second-level domain name, then it is free to create any other new domain name within its subdomain, such as `cs.cmu.edu`.

The Internet defines a mapping between the set of domain names and the set of IP addresses. Until 1988, this mapping was maintained manually in a single text file called `HOSTS.TXT`. Since then, the mapping has been maintained in a distributed worldwide database known as DNS (Domain Name System). Conceptually, the DNS database consists of millions of host entries, each of which defines the mapping between a set of domain names and a set of IP addresses. In a mathematical sense, think of each host entry as an equivalence class of domain names and IP addresses. We can explore some of the properties of the DNS mappings with the Linux NSLOOKUP program, which displays the IP addresses associated with a domain name.[1]

Each Internet host has the locally defined domain name `localhost`, which always maps to the loopback address `127.0.0.1`:

```
linux> nslookup localhost
Address: 127.0.0.1
```

The `localhost` name provides a convenient and portable way to reference clients and servers that are running on the same machine, which can be especially useful

1. We've reformatted the output of NSLOOKUP to improve readability.

for debugging. We can use HOSTNAME to determine the real domain name of our local host:

```
linux> hostname
whaleshark.ics.cs.cmu.edu
```

In the simplest case, there is a one-to-one mapping between a domain name and an IP address:

```
linux> nslookup whaleshark.ics.cs.cmu.edu
Address: 128.2.210.175
```

However, in some cases, multiple domain names are mapped to the same IP address:

```
linux> nslookup cs.mit.edu
Address: 18.62.1.6

linux> nslookup eecs.mit.edu
Address: 18.62.1.6
```

In the most general case, multiple domain names are mapped to the same set of multiple IP addresses:

```
linux> nslookup www.twitter.com
Address: 199.16.156.6
Address: 199.16.156.70
Address: 199.16.156.102
Address: 199.16.156.230

linux> nslookup twitter.com
Address: 199.16.156.102
Address: 199.16.156.230
Address: 199.16.156.6
Address: 199.16.156.70
```

Finally, we notice that some valid domain names are not mapped to any IP address:

```
linux> nslookup edu
*** Can't find edu: No answer
linux> nslookup ics.cs.cmu.edu
*** Can't find ics.cs.cmu.edu: No answer
```

11.3.3 Internet Connections

Internet clients and servers communicate by sending and receiving streams of bytes over connections. A connection is point-to-point in the sense that it connects a pair of processes. It is full duplex in the sense that data can flow in both directions

Aside How did it start?

Twice a year since 1987, the Internet Systems Consortium conducts the Internet Domain Survey. The survey, which estimates the number of Internet hosts by counting the number of IP addresses that have been assigned a domain name, reveals an amazing trend. Since 1987, when there were about 20,000 Internet hosts, the number of hosts has been increasing exponentially. By 2015, there were over 1,000,000,000 Internet hosts!

at the same time. And it is reliable in the sense that—barring some catastrophic failure such as a cable cut by the proverbial careless backhoe operator—the stream of bytes sent by the source process is eventually received by the destination process in the same order it was sent.

A socket is an end point of a connection. Each socket has a corresponding socket address that consists of an Internet address and a 16-bit integer port[2] and is denoted by the notation `address:port`.

The port in the client's socket address is assigned automatically by the kernel when the client makes a connection request and is known as an ephemeral port. However, the port in the server's socket address is typically some well-known port that is permanently associated with the service. For example, Web servers typically use port 80, and email servers use port 25. Associated with each service with a well-known port is a corresponding well-known service name. For example, the well-known name for the Web service is `http`, and the well-known name for email is `smtp`. The mapping between well-known names and well-known ports is contained in a file called `/etc/services`.

A connection is uniquely identified by the socket addresses of its two end points. This pair of socket addresses is known as a socket pair and is denoted by the tuple

```
(cliaddr:cliport, servaddr:servport)
```

where cliaddr is the client's IP address, cliport is the client's port, servaddr is the server's IP address, and servport is the server's port. For example, Figure 11.11 shows a connection between a Web client and a Web server.

In this example, the Web client's socket address is

```
128.2.194.242:51213
```

where port `51213` is an ephemeral port assigned by the kernel. The Web server's socket address is

```
208.216.181.15:80
```

2. These software ports have no relation to the hardware ports in network switches and routers.

Aside Ori i o t l t r t

The Internet is one of the most successful examples of government, university, and industry partnership. Many factors contributed to its success, but we think two are particularly important: a sustained 30-year investment by the United States government and a commitment by passionate researchers to what Dave Clarke at MIT has dubbed "rough consensus and working code."

The seeds of the Internet were sown in 1957, when, at the height of the Cold War, the Soviet Union shocked the world by launching Sputnik, the first artificial earth satellite. In response, the United States government created the Advanced Research Projects Agency (ARPA), whose charter was to reestablish the US lead in science and technology. In 1967, Lawrence Roberts at ARPA published plans for a new network called the ARPANET. The first ARPANET nodes were up and running by 1969. By 1971, there were 13 ARPANET nodes, and email had emerged as the first important network application.

In 1972, Robert Kahn outlined the general principles of internetworking: a collection of interconnected networks, with communication between the networks handled independently on a "best-effort basis" by black boxes called "routers." In 1974, Kahn and Vinton Cerf published the first details of TCP/IP, which by 1982 had become the standard internetworking protocol for ARPANET. On January 1, 1983, every node on the ARPANET switched to TCP/IP, marking the birth of the global IP Internet.

In 1985, Paul Mockapetris invented DNS, and there were over 1,000 Internet hosts. The next year, the National Science Foundation (NSF) built the NSFNET backbone connecting 13 sites with 56 Kb/s phone lines. It was upgraded to 1.5 Mb/s T1 links in 1988 and 45 Mb/s T3 links in 1991. By 1988, there were more than 50,000 hosts. In 1989, the original ARPANET was officially retired. In 1995, when there were almost 10,000,000 Internet hosts, NSF retired NSFNET and replaced it with the modern Internet architecture based on private commercial backbones connected by public network access points.

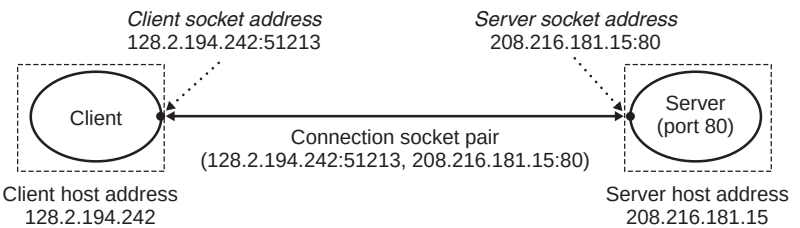


Figure 11.11
Anatomy of an Internet connection.

where port 80 is the well-known port associated with Web services. Given these client and server socket addresses, the connection between the client and server is uniquely identified by the socket pair

```
(128.2.194.242:51213, 208.216.181.15:80)
```

Aside Origins of the sockets interface

The original sockets interface was developed by researchers at University of California, Berkeley, in the early 1980s. For this reason, it is often referred to as Berkeley sockets. The Berkeley researchers developed the sockets interface to work with any underlying protocol. The first implementation was for TCP/IP, which they included in the Unix 4.2BSD kernel and distributed to numerous universities and labs. This was an important event in Internet history. Almost overnight, thousands of people had access to TCP/IP and its source codes. It generated tremendous excitement and sparked a flurry of new research in networking and internetworking.

11.4 The Sockets Interface

The sockets interface is a set of functions that are used in conjunction with the Unix I/O functions to build network applications. It has been implemented on most modern systems, including all Unix variants as well as Windows and Macintosh systems. Figure 11.12 gives an overview of the sockets interface in the context of a typical client-server transaction. You should use this picture as a road map when we discuss the individual functions.

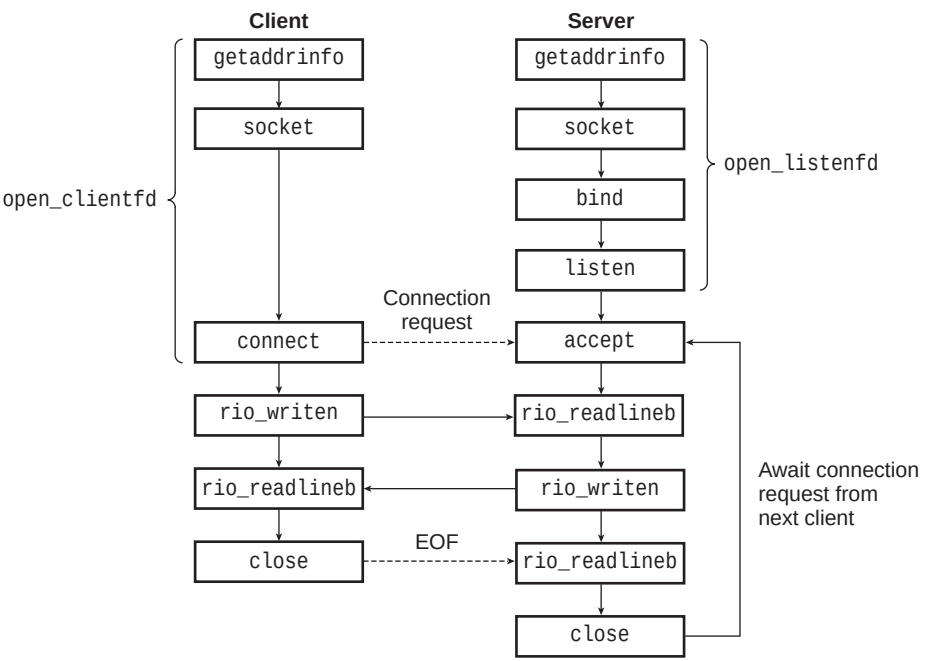


Figure 11.12 Overview of network applications based on the sockets interface.

Aside What's in _in_?

The _in suffix is short for internet, not input.

code/netp/netpfragments.c

```
/* IP socket address structure */
struct sockaddr_in  {
    uint16_t        sin_family;  /* Protocol family (always AF_INET) */
    uint16_t        sin_port;    /* Port number in network byte order */
    struct in_addr  sin_addr;    /* IP address in network byte order */
    unsigned char   sin_zero[8]; /* Pad to sizeof(struct sockaddr) */
};

/* Generic socket address structure (for connect, bind, and accept) */
struct sockaddr {
    uint16_t  sa_family;    /* Protocol family */
    char      sa_data[14];  /* Address data  */
};
```

code/netp/netpfragments.c

Figure 11.13 Socket address structures.

11.4.1 Socket Address Structures

From the perspective of the Linux kernel, a socket is an end point for communication. From the perspective of a Linux program, a socket is an open file with a corresponding descriptor.

Internet socket addresses are stored in 16-byte structures having the type `sockaddr_in`, shown in Figure 11.13. For Internet applications, the `sin_family` field is AF_INET, the `sin_port` field is a 16-bit port number, and the `sin_addr` field contains a 32-bit IP address. The IP address and port number are always stored in network (big-endian) byte order.

The `connect`, `bind`, and `accept` functions require a pointer to a protocol-specific socket address structure. The problem faced by the designers of the sockets interface was how to define these functions to accept any kind of socket address structure. Today, we would use the generic `void *` pointer, which did not exist in C at that time. Their solution was to define sockets functions to expect a pointer to a generic `sockaddr` structure (Figure 11.13) and then require applications to cast any pointers to protocol-specific structures to this generic structure. To simplify our code examples, we follow Stevens's lead and define the following type:

```
typedef struct sockaddr SA;
```

We then use this type whenever we need to cast a `sockaddr_in` structure to a generic `sockaddr` structure.

11.4.2 The `socket` Function

Clients and servers use the `socket` function to create a socket descriptor.

```
#include <sys/types.h>
#include <sys/socket.h>

int socket(int domain, int type, int protocol);
                                    Returns: nonnegative descriptor if OK, −1 on error
```

If we wanted the socket to be the end point for a connection, then we could call `socket` with the following hardcoded arguments:

```
clientfd = Socket(AF_INET, SOCK_STREAM, 0);
```

where AF_INET indicates that we are using 32-bit IP addresses and SOCK_STREAM indicates that the socket will be an end point for a connection. However, the best practice is to use the `getaddrinfo` function (Section 11.4.7) to generate these parameters automatically, so that the code is protocol-independent. We will show you how to use `getaddrinfo` with the `socket` function in Section 11.4.8.

The `clientfd` descriptor returned by `socket` is only partially opened and cannot yet be used for reading and writing. How we finish opening the socket depends on whether we are a client or a server. The next section describes how we finish opening the socket if we are a client.

11.4.3 The `connect` Function

A client establishes a connection with a server by calling the `connect` function.

```
#include <sys/socket.h>

int connect(int clientfd, const struct sockaddr *addr,
            socklen_t addrlen);
                                                  Returns: 0 if OK, −1 on error
```

The `connect` function attempts to establish an Internet connection with the server at socket address `addr`, where `addrlen` is `sizeof(sockaddr_in)`. The `connect` function blocks until either the connection is successfully established or an error occurs. If successful, the `clientfd` descriptor is now ready for reading and writing, and the resulting connection is characterized by the socket pair

```
(x:y, addr.sin_addr:addr.sin_port)
```

where x is the client's IP address and y is the ephemeral port that uniquely identifies the client process on the client host. As with socket, the best practice is to use getaddrinfo to supply the arguments to connect (see Section 11.4.8).

11.4.4 The bind Function

The remaining sockets functions—bind, listen, and accept—are used by servers

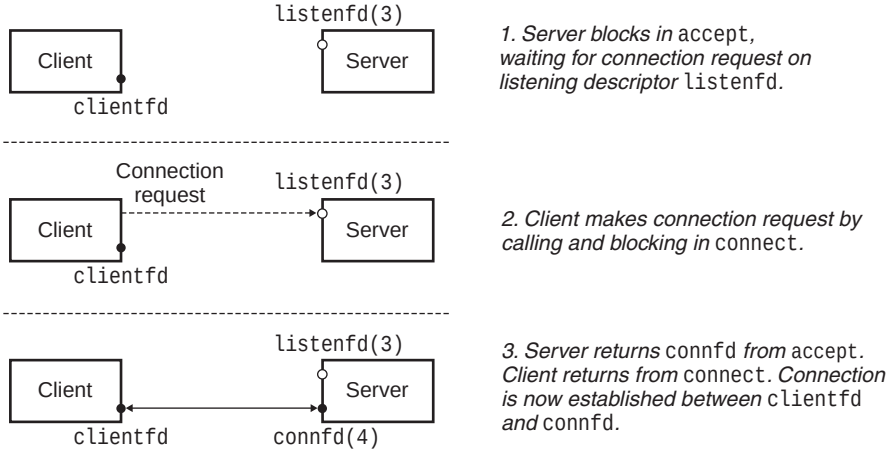


Figure 11.14 The roles of the listening and connected descriptors.

11.4.6 The accept Function

Servers wait for connection requests from clients by calling the `accept` function.

```
#include <sys/socket.h>

int accept(int listenfd, struct sockaddr *addr, int *addrlen);
                                    Returns: nonnegative connected descriptor if OK, −1 on error
```

The `accept` function waits for a connection request from a client to arrive on the listening descriptor `listenfd`, then fills in the client's socket address in `addr`, and returns a *connected descriptor* that can be used to communicate with the client using Unix I/O functions.

The distinction between a listening descriptor and a connected descriptor confuses many students. The listening descriptor serves as an end point for client connection requests. It is typically created once and exists for the lifetime of the server. The connected descriptor is the end point of the connection that is established between the client and the server. It is created each time the server accepts a connection request and exists only as long as it takes the server to service a client.

Figure 11.14 outlines the roles of the listening and connected descriptors. In step 1, the server calls `accept`, which waits for a connection request to arrive on the listening descriptor, which for concreteness we will assume is descriptor 3. Recall that descriptors 0–2 are reserved for the standard files.

In step 2, the client calls the `connect` function, which sends a connection request to `listenfd`. In step 3, the `accept` function opens a new connected descriptor `connfd` (which we will assume is descriptor 4), establishes the connection between `clientfd` and `connfd`, and then returns `connfd` to the application. The

Aside: What's with listening and connected descriptors?

You might wonder why the sockets interface makes a distinction between listening and connected descriptors. At first glance, it appears to be an unnecessary complication. However, distinguishing between the two turns out to be quite useful, because it allows us to build concurrent servers that can process many client connections simultaneously. For example, each time a connection request arrives on the listening descriptor, we might fork a new process that communicates with the client over its connected descriptor. You'll learn more about concurrent servers in Chapter 12.

client also returns from the `connect`, and from this point, the client and server can pass data back and forth by reading and writing `clientfd` and `connfd`, respectively.

11.4.7 Host and Service Conversion

Linux provides some powerful functions, called `getaddrinfo` and `getnameinfo`, for converting back and forth between binary socket address structures and the string representations of hostnames, host addresses, service names, and port numbers. When used in conjunction with the sockets interface, they allow us to write network programs that are independent of any particular version of the IP protocol.

The getaddrinfo Function

The `getaddrinfo` function converts string representations of hostnames, host addresses, service names, and port numbers into socket address structures. It is the modern replacement for the obsolete `gethostbyname` and `getservbyname` functions. Unlike these functions, it is reentrant (see Section 12.7.2) and works with any protocol.

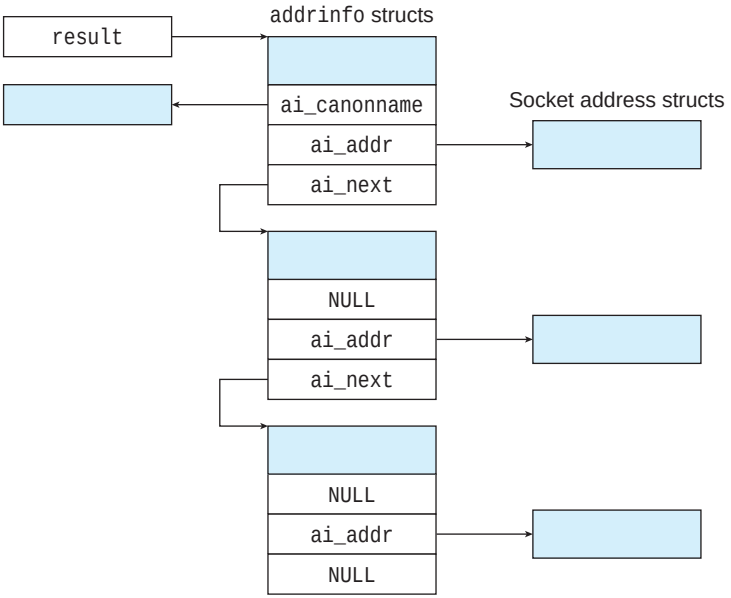
```
#include <sys/types.h>
#include <sys/socket.h>
#include <netdb.h>

int getaddrinfo(const char *host, const char *service,
                const struct addrinfo *hints,
                struct addrinfo **result);
                                    Returns: 0 if OK, nonzero error code on error

void freeaddrinfo(struct addrinfo *result);
                                                          Returns: nothing

const char *gai_strerror(int errcode);
                                                  Returns: error message
```

Figure 11.15
Data structure returned by `getaddrinfo`.



Given `host` and `service` (the two components of a socket address), `getaddrinfo` returns a `result` that points to a linked list of `addrinfo` structures, each of which points to a socket address structure that corresponds to `host` and `service` (Figure 11.15).

After a client calls `getaddrinfo`, it walks this list, trying each socket address in turn until the calls to `socket` and `connect` succeed and the connection is established. Similarly, a server tries each socket address on the list until the calls to `socket` and `bind` succeed and the descriptor is bound to a valid socket address. To avoid memory leaks, the application must eventually free the list by calling `freeaddrinfo`. If `getaddrinfo` returns a nonzero error code, the application can call `gai_strerror` to convert the code to a message string.

The `host` argument to `getaddrinfo` can be either a domain name or a numeric address (e.g., a dotted-decimal IP address). The `service` argument can be either a service name (e.g., `http`) or a decimal port number. If we are not interested in converting the hostname to an address, we can set `host` to NULL. The same holds for `service`. However, at least one of them must be specified.

The optional `hints` argument is an `addrinfo` structure (Figure 11.16) that provides finer control over the list of socket addresses that `getaddrinfo` returns. When passed as a `hints` argument, only the `ai_family`, `ai_socktype`, `ai_protocol`, and `ai_flags` fields can be set. The other fields must be set to zero (or NULL). In practice, we use `memset` to zero the entire structure and then set a few selected fields:

- By default, `getaddrinfo` can return both IPv4 and IPv6 socket addresses. Setting `ai_family` to AF_INET restricts the list to IPv4 addresses. Setting it to AF_INET6 restricts the list to IPv6 addresses.

code/netp/netpfragments.c

```
struct addrinfo {
    int              ai_flags;     /* Hints argument flags */
    int              ai_family;    /* First arg to socket function */
    int              ai_socktype;  /* Second arg to socket function */
    int              ai_protocol;  /* Third arg to socket function  */
    char            *ai_canonname; /* Canonical hostname */
    size_t           ai_addrlen;   /* Size of ai_addr struct */
    struct sockaddr *ai_addr;      /* Ptr to socket address structure */
    struct addrinfo *ai_next;      /* Ptr to next item in linked list */
};
```

code/netp/netpfragments.c

Figure 11.16 The addrinfo structure used by getaddrinfo.

- By default, for each unique address associated with host, the getaddrinfo function can return up to three addrinfo structures, each with a different ai_socktype field: one for connections, one for datagrams (not covered), and one for raw sockets (not covered). Setting ai_socktype to SOCK_STREAM restricts the list to at most one addrinfo structure for each unique address, one whose socket address can be used as the end point of a connection. This is the desired behavior for all of our example programs.
- The ai_flags field is a bit mask that further modifies the default behavior. You create it by oring combinations of various values. Here are some that we find useful:
  - AI_ADDRCONFIG. This flag is recommended if you are using connections [34]. It asks getaddrinfo to return IPv4 addresses only if the local host is configured for IPv4. Similarly for IPv6.
  - AI_CANONNAME. By default, the ai_canonname field is NULL. If this flag is set, it instructs getaddrinfo to point the ai_canonname field in the first addrinfo structure in the list to the canonical (official) name of host (see Figure 11.15).
  - AI_NUMERICSERV. By default, the service argument can be a service name or a port number. This flag forces the service argument to be a port number.
  - AI_PASSIVE. By default, getaddrinfo returns socket addresses that can be used by clients as active sockets in calls to connect. This flag instructs it to return socket addresses that can be used by servers as listening sockets. In this case, the host argument should be NULL. The address field in the resulting socket address structure(s) will be the wildcard address, which tells the kernel that this server will accept requests to any of the IP addresses for this host. This is the desired behavior for all of our example servers.

When `getaddrinfo` creates an `addrinfo` structure in the output list, it fills in each field except for `ai_flags`. The `ai_addr` field points to a socket address structure, the `ai_addrlen` field gives the size of this socket address structure, and the `ai_next` field points to the next `addrinfo` structure in the list. The other fields describe various attributes of the socket address.

One of the elegant aspects of `getaddrinfo` is that the fields in an `addrinfo` structure are opaque, in the sense that they can be passed directly to the functions in the sockets interface without any further manipulation by the application code. For example, `ai_family`, `ai_socktype`, and `ai_protocol` can be passed directly to `socket`. Similarly, `ai_addr` and `ai_addrlen` can be passed directly to `connect` and `bind`. This powerful property allows us to write clients and servers that are independent of any particular version of the IP protocol.

T getnameinfo F ti

The `getnameinfo` function is the inverse of `getaddrinfo`. It converts a socket address structure to the corresponding host and service name strings. It is the modern replacement for the obsolete `gethostbyaddr` and `getservbyport` functions, and unlike those functions, it is reentrant and protocol-independent.

```
#include <sys/socket.h>
#include <netdb.h>

int getnameinfo(const struct sockaddr *sa, socklen_t salen,
                char *host, size_t hostlen,
                char *service, size_t servlen, int flags);
```

Returns: 0 if OK, nonzero error code on error

The `sa` argument points to a socket address structure of size `salen` bytes, `host` to a buffer of size `hostlen` bytes, and `service` to a buffer of size `servlen` bytes. The `getnameinfo` function converts the socket address structure `sa` to the corresponding host and service name strings and copies them to the `host` and `service` buffers. If `getnameinfo` returns a nonzero error code, the application can convert it to a string by calling `gai_strerror`.

If we don't want the hostname, we can set `host` to NULL and `hostlen` to zero. The same holds for the service fields. However, one or the other must be set.

The `flags` argument is a bit mask that modifies the default behavior. You create it by oring combinations of various values. Here are a couple of useful ones:

NI_NUMERICHOST. By default, `getnameinfo` tries to return a domain name in `host`. Setting this flag will cause it to return a numeric address string instead.

NI_NUMERICSERV. By default, `getnameinfo` will look in `/etc/services` and if possible, return a service name instead of a port number. Setting this flag forces it to skip the lookup and simply return the port number.

code/netp/hostinfo.c

```
1  #include "csapp.h"
2
3  int main(int argc, char **argv)
4  {
5      struct addrinfo *p, *listp, hints;
6      char buf[MAXLINE];
7      int rc, flags;
8
9      if (argc != 2) {
10         fprintf(stderr, "usage: %s <domain name>\n", argv[0]);
11         exit(0);
12     }
13
14     /* Get a list of addrinfo records */
15     memset(&hints, 0, sizeof(struct addrinfo));
16     hints.ai_family = AF_INET;       /* IPv4 only */
17     hints.ai_socktype = SOCK_STREAM; /* Connections only */
18     if ((rc = getaddrinfo(argv[1], NULL, &hints, &listp)) != 0) {
19         fprintf(stderr, "getaddrinfo error: %s\n", gai_strerror(rc));
20         exit(1);
21     }
22
23     /* Walk the list and display each IP address */
24     flags = NI_NUMERICHOST; /* Display address string instead of domain name */
25     for (p = listp; p; p = p->ai_next) {
26         Getnameinfo(p->ai_addr, p->ai_addrlen, buf, MAXLINE, NULL, 0, flags);
27         printf("%s\n", buf);
28     }
29
30     /* Clean up */
31     Freeaddrinfo(listp);
32
33     exit(0);
34 }
```

code/netp/hostinfo.c

Figure 11.17 Hostinfo displays the mapping of a domain name to its associated IP addresses.

Figure 11.17 shows a simple program, called HOSTINFO, that uses `getaddrinfo` and `getnameinfo` to display the mapping of a domain name to its associated IP addresses. It is similar to the NSLOOKUP program from Section 11.3.2.

First, we initialize the `hints` structure so that `getaddrinfo` returns the addresses we want. In this case, we are looking for 32-bit IP addresses (line 16)

that can be used as end points of connections (line 17). Since we are only asking `getaddrinfo` to convert domain names, we call it with a NULL `service` argument.

After the call to `getaddrinfo`, we walk the list of `addrinfo` structures, using `getnameinfo` to convert each socket address to a dotted-decimal address string. After walking the list, we are careful to free it by calling `freeaddrinfo` (although for this simple program it is not strictly necessary).

When we run HOSTINFO, we see that `twitter.com` maps to four IP addresses, which is what we saw using NSLOOKUP in Section 11.3.2.

```
linux> ./hostinfo twitter.com
199.16.156.102
199.16.156.230
199.16.156.6
199.16.156.70
```

Practice Problem 11.4 (solution page 1004)

The `getaddrinfo` and `getnameinfo` functions subsume the functionality of `inet_pton` and `inet_ntop`, respectively, and they provide a higher-level of abstraction that is independent of any particular address format. To convince yourself how handy this is, write a version of HOSTINFO (Figure 11.17) that uses `inet_ntop` instead of `getnameinfo` to convert each socket address to a dotted-decimal address string.

11.4.8 Helper Functions for the Sockets Interface

The `getaddrinfo` function and the sockets interface can seem somewhat daunting when you first learn about them. We find it convenient to wrap them with higher-level helper functions, called `open_clientfd` and `open_listenfd`, that clients and servers can use when they want to communicate with each other.

T open_clientfd F ti

A client establishes a connection with a server by calling `open_clientfd`.

```
#include "csapp.h"

int open_clientfd(char *hostname, char *port);
```

Returns: descriptor if OK, −1 on error

The `open_clientfd` function establishes a connection with a server running on host `hostname` and listening for connection requests on port number `port`. It returns an open socket descriptor that is ready for input and output using the Unix I/O functions. Figure 11.18 shows the code for `open_clientfd`.

We call `getaddrinfo`, which returns a list of `addrinfo` structures, each of which points to a socket address structure that is suitable for establishing a con-

T open_listenfd F ti

A server creates a listening descriptor that is ready to receive connection requests by calling the `open_listenfd` function.

```
#include "csapp.h"

int open_listenfd(char *port);
```

Returns: descriptor if OK, −1 on error

The `open_listenfd` function returns a listening descriptor that is ready to receive connection requests on port `port`. Figure 11.19 shows the code for `open_listenfd`.

The style is similar to `open_clientfd`. We call `getaddrinfo` and then walk the resulting list until the calls to `socket` and `bind` succeed. Note that in line 20 we use the `setsockopt` function (not described here) to configure the server so that it can be terminated, be restarted, and begin accepting connection requests immediately. By default, a restarted server will deny connection requests from clients for approximately 30 seconds, which seriously hinders debugging.

Since we have called `getaddrinfo` with the AI_PASSIVE flag and a NULL `host` argument, the address field in each socket address structure is set to the wildcard address, which tells the kernel that this server will accept requests to any of the IP addresses for this host.

Finally, we call the `listen` function to convert `listenfd` to a listening descriptor and return it to the caller. If the `listen` fails, we are careful to avoid a memory leak by closing the descriptor before returning.

11.4.9 Example Echo Client and Server

The best way to learn the sockets interface is to study example code. Figure 11.20 shows the code for an echo client. After establishing a connection with the server, the client enters a loop that repeatedly reads a text line from standard input, sends the text line to the server, reads the echo line from the server, and prints the result to standard output. The loop terminates when `fgets` encounters EOF on standard input, either because the user typed Ctrl+D at the keyboard or because it has exhausted the text lines in a redirected input file.

After the loop terminates, the client closes the descriptor. This results in an EOF notification being sent to the server, which it detects when it receives a return code of zero from its `rio_readlineb` function. After closing its descriptor, the client terminates. Since the client's kernel automatically closes all open descriptors when a process terminates, the `close` in line 24 is not necessary. However, it is good programming practice to explicitly close any descriptors that you have opened.

Figure 11.21 shows the main routine for the echo server. After opening the listening descriptor, it enters an infinite loop. Each iteration waits for a connection request from a client, prints the domain name and port of the connected client, and then calls the `echo` function that services the client. After the echo routine returns,

code/src/csapp.c

```
int open_listenfd(char *port)
{
    struct addrinfo hints, *listp, *p;
    int listenfd, optval=1;

    /* Get a list of potential server addresses */
    memset(&hints, 0, sizeof(struct addrinfo));
    hints.ai_socktype = SOCK_STREAM;             /* Accept connections */
    hints.ai_flags = AI_PASSIVE | AI_ADDRCONFIG; /* ... on any IP address */
    hints.ai_flags |= AI_NUMERICSERV;            /* ... using port number */
    Getaddrinfo(NULL, port, &hints, &listp);

    /* Walk the list for one that we can bind to */
    for (p = listp; p; p = p->ai_next) {
        /* Create a socket descriptor */
        if ((listenfd = socket(p->ai_family, p->ai_socktype, p->ai_protocol)) < 0)
            continue;  /* Socket failed, try the next */

        /* Eliminates "Address already in use" error from bind */
        Setsockopt(listenfd, SOL_SOCKET, SO_REUSEADDR,
                   (const void *)&optval , sizeof(int));

        /* Bind the descriptor to the address */
        if (bind(listenfd, p->ai_addr, p->ai_addrlen) == 0)
            break; /* Success */
        Close(listenfd); /* Bind failed, try the next */
    }

    /* Clean up */
    Freeaddrinfo(listp);
    if (!p) /* No address worked */
        return -1;

    /* Make it a listening socket ready to accept connection requests */
    if (listen(listenfd, LISTENQ) < 0) {
        Close(listenfd);
        return -1;
    }
    return listenfd;
}
```

code/src/csapp.c

Figure 11.19 open_listenfd: Helper function that opens and returns a listening descriptor. It i r tra t a r t l-i t.

code/netp/echoclient.c

```
1   #include "csapp.h"
2
3   int main(int argc, char **argv)
4   {
5       int clientfd;
6       char *host, *port, buf[MAXLINE];
7       rio_t rio;
8
9       if (argc != 3) {
10          fprintf(stderr, "usage: %s <host> <port>\n", argv[0]);
11          exit(0);
12      }
13      host = argv[1];
14      port = argv[2];
15
16      clientfd = Open_clientfd(host, port);
17      Rio_readinitb(&rio, clientfd);
18
19      while (Fgets(buf, MAXLINE, stdin) != NULL) {
20          Rio_writen(clientfd, buf, strlen(buf));
21          Rio_readlineb(&rio, buf, MAXLINE);
22          Fputs(buf, stdout);
23      }
24      Close(clientfd);
25      exit(0);
26  }
```

code/netp/echoclient.c

Figure 11.20 Echo client main routine.

the main routine closes the connected descriptor. Once the client and server have closed their respective descriptors, the connection is terminated.

The `clientaddr` variable in line 9 is a socket address structure that is passed to `accept`. Before `accept` returns, it fills in `clientaddr` with the socket address of the client on the other end of the connection. Notice how we declare `clientaddr` as type `struct sockaddr_storage` rather than `struct sockaddr_in`. By definition, the `sockaddr_storage` structure is large enough to hold any type of socket address, which keeps the code protocol-independent.

Notice that our simple echo server can only handle one client at a time. A server of this type that iterates through clients, one at a time, is called an iterative server. In Chapter 12, we will learn how to build more sophisticated concurrent servers that can handle multiple clients simultaneously.

Finally, Figure 11.22 shows the code for the `echo` routine, which repeatedly reads and writes lines of text until the `rio_readlineb` function encounters EOF in line 10.

code/netp/echoserveri.c

```
#include "csapp.h"

void echo(int connfd);

int main(int argc, char **argv)
{
    int listenfd, connfd;
    socklen_t clientlen;
    struct sockaddr_storage clientaddr;  /* Enough space for any address */
    char client_hostname[MAXLINE], client_port[MAXLINE];

    if (argc != 2) {
        fprintf(stderr, "usage: %s <port>\n", argv[0]);
        exit(0);
    }

    listenfd = Open_listenfd(argv[1]);
    while (1) {
        clientlen = sizeof(struct sockaddr_storage);
        connfd = Accept(listenfd, (SA *)&clientaddr, &clientlen);
        Getnameinfo((SA *) &clientaddr, clientlen, client_hostname, MAXLINE,
                    client_port, MAXLINE, 0);
        printf("Connected to (%s, %s)\n", client_hostname, client_port);
        echo(connfd);
        Close(connfd);
    }
    exit(0);
}
```

code/netp/echoserveri.c

Figure 11.21 Iterative echo server main routine.

code/netp/echo.c

```
#include "csapp.h"

void echo(int connfd)
{
    size_t n;
    char buf[MAXLINE];
    rio_t rio;

    Rio_readinitb(&rio, connfd);
    while((n = Rio_readlineb(&rio, buf, MAXLINE)) != 0) {
        printf("server received %d bytes\n", (int)n);
```

Aside What's the deal with EOF?

The idea of EOF is often confusing to students, especially in the context of Internet connections. First, we need to understand that there is no such thing as an EOF character. Rather, EOF is a condition that is detected by the kernel. An application finds out about the EOF condition when it receives a zero return code from the `read` function. For disk files, EOF occurs when the current file position exceeds the file length. For Internet connections, EOF occurs when a process closes its end of the connection. The process at the other end of the connection detects the EOF when it attempts to read past the last byte in the stream.

11.5 Web Servers

So far we have discussed network programming in the context of a simple echo server. In this section, we will show you how to use the basic ideas of network programming to build your own small, but quite functional, Web server.

11.5.1 Web Basics

Web clients and servers interact using a text-based application-level protocol known as HTTP (hypertext transfer protocol). HTTP is a simple protocol. A Web client (known as a browser) opens an Internet connection to a server and requests some content. The server responds with the requested content and then closes the connection. The browser reads the content and displays it on the screen.

What distinguishes Web services from conventional file retrieval services such as FTP? The main difference is that Web content can be written in a language known as HTML (hypertext markup language). An HTML program (page) contains instructions (tags) that tell the browser how to display various text and graphical objects in the page. For example, the code

```
<b> Make me bold! </b>
```

tells the browser to print the text between the `<b>` and `</b>` tags in boldface type. However, the real power of HTML is that a page can contain pointers (hyperlinks) to content stored on any Internet host. For example, an HTML line of the form

```
<a href="http://www.cmu.edu/index.html">Carnegie Mellon</a>
```

tells the browser to highlight the text object `Carnegie Mellon` and to create a hyperlink to an HTML file called `index.html` that is stored on the CMU Web server. If the user clicks on the highlighted text object, the browser requests the corresponding HTML file from the CMU server and displays it.

Aside Origins of the World Wide Web

The World Wide Web was invented by Tim Berners-Lee, a software engineer working at CERN, a Swiss physics lab. In 1989, Berners-Lee wrote an internal memo proposing a distributed hypertext system

identifies an HTML file called `/index.html` on Google's server, which is managed by `www.google.com`

```
http://bluefish.ics.cs.cmu.edu:8000/cgi-bin/adder?15000&213
```

identifies an executable called `/cgi-bin/adder` that will be passed `15000` and `213`. Clients and servers use diffe

```
http://www.google.com:80
```

to determine what kind of server to contact, where the server is, and what port it

```
/index.html
```

to find the file on its filesystem and to determine whether the request is for static

- There are no standard rules for determining whether a URL refers to static `cgi-bi`
- The initial '`/`' in the suffix does not `/usr/httpd/html/usr/httpd/cgi-bin`.
- The minimal URL suffix is the `/` character, which usually yields and `/index.html`. This explains why it is possible to '/' to the

```
1   linux> telnet www.aol.com 80        Client: open connection to server
2   Trying 205.188.146.23...            Telnet prints 3 lines to the terminal
3   Connected to aol.com.
4   Escape character is '^]'.
5   GET / HTTP/1.1                      Client: request line
6   Host: www.aol.com                   Client: required HTTP/1.1 header
7                                       Client: empty line terminates headers
8   HTTP/1.0 200 OK                     Server: response line
9   MIME-Version: 1.0                   Server: followed by five response headers
10  Date: Mon, 8 Jan 2010 4:59:42 GMT
11  Server:  Apache-Coyote/1.1
12  Content-Type: text/html             Server: expect HTML in the response body
13  Content-Length: 42092               Server: expect 42,092 bytes in the response body
14                                      Server: empty line terminates response headers
15  <html>                              Server: first HTML line in response body
16  ...                                 Server: 766 lines of HTML not shown
17  </html>                             Server: last HTML line in response body
18  Connection closed by foreign host.  Server: closes connection
19  linux>                              Client: closes connection and terminates
```

Figure 11.24 Example of an HTTP transaction that serves static content.

In line 1, we run TELNET from a Linux shell and ask it to open a connection to the AOL Web server. TELNET prints three lines of output to the terminal, opens the connection, and then waits for us to enter text (line 5). Each time we enter a text line and hit the `enter` key, TELNET reads the line, appends carriage return and line feed characters ('\r\n' in C notation), and sends the line to the server. This is consistent with the HTTP standard, which requires every text line to be terminated by a carriage return and line feed pair. To initiate the transaction, we enter an HTTP request (lines 5–7). The server replies with an HTTP response (lines 8–17) and then closes the connection (line 18).

HTTP R t

An HTTP request consists of a request line (line 5), followed by zero or more request headers (line 6), followed by an empty text line that terminates the list of headers (line 7). A request line has the form

method URI version

HTTP supports a number of different methods, including GET, POST, OPTIONS, HEAD, PUT, DELETE, and TRACE. We will only discuss the workhorse GET method, which accounts for a majority of HTTP requests. The GET method instructs the server to generate and return the content identified by the URI

(uniform resource identifier). The URI is the suffix of the corresponding URL that includes the filename and optional arguments.[3]

The version field in the request line indicates the HTTP version to which the request conforms. The most recent HTTP version is HTTP/1.1 [37]. HTTP/1.0 is an earlier, much simpler version from 1996 [6]. HTTP/1.1 defines additional headers that provide support for advanced features such as caching and security, as well as a mechanism that allows a client and server to perform multiple transactions over the same persistent connection. In practice, the two versions are compatible because HTTP/1.0 clients and servers simply ignore unknown HTTP/1.1 headers.

To summarize, the request line in line 5 asks the server to fetch and return the HTML file `/index.html`. It also informs the server that the remainder of the request will be in HTTP/1.1 format.

Request headers provide additional information to the server, such as the brand name of the browser or the MIME types that the browser understands. Request headers have the form

```
header-name: header-data
```

For our purposes, the only header to be concerned with is the `Host` header (line 6), which is required in HTTP/1.1 requests, but not in HTTP/1.0 requests. The `Host` header is used by proxy caches, which sometimes serve as intermediaries between a browser and the origin server that manages the requested file. Multiple proxies can exist between a client and an origin server in a so-called proxy chain. The data in the `Host` header, which identifies the domain name of the origin server, allow a proxy in the middle of a proxy chain to determine if it might have a locally cached copy of the requested content.

Continuing with our example in Figure 11.24, the empty text line in line 7 (generated by hitting `enter` on our keyboard) terminates the headers and instructs the server to send the requested HTML file.

HTTP R o

HTTP responses are similar to HTTP requests. An HTTP response consists of a response line (line 8), followed by zero or more response headers (lines 9–13), followed by an empty line that terminates the headers (line 14), followed by the response body (lines 15–17). A response line has the form

```
version status-code status-message
```

The version field describes the HTTP version that the response conforms to. The status-code is a three-digit positive integer that indicates the disposition of the request. The status-message gives the English equivalent of the error code. Figure 11.25 lists some common status codes and their corresponding messages.

3. Actually, this is only true when a browser requests content. If a proxy server requests content, then the URI must be the complete URL.

Aside Passing arguments in HTTP POST requests

Arguments for HTTP POST requests are passed in the request body rather than in the URI.

Status code	Status message	Description
200	OK	Request was handled without error.
301	Moved permanently	Content has moved to the hostname in the Location header.
400	Bad request	Request could not be understood by the server.
403	Forbidden	Server lacks permission to access the requested file.
404	Not found	Server could not find the requested file.
501	Not implemented	Server does not support the request method.
505	HTTP version not supported	Server does not support version in request.

Figure 11.25 Some HTTP status codes.

The response headers in lines 9–13 provide additional information about the response. For our purposes, the two most important headers are `Content-Type` (line 12), which tells the client the MIME type of the content in the response body, and `Content-Length` (line 13), which indicates its size in bytes.

The empty text line in line 14 that terminates the response headers is followed by the response body, which contains the requested content.

11.5.4 Serving Dynamic Content

If we stop to think for a moment how a server might provide dynamic content to a client, certain questions arise. For example, how does the client pass any program arguments to the server? How does the server pass these arguments to the child process that it creates? How does the server pass other information to the child that it might need to generate the content? Where does the child send its output? These questions are addressed by a de facto standard called CGI (common gateway interface).

How Does the Client Pass Arguments to the Server?

Arguments for GET requests are passed in the URI. As we have seen, a '?' character separates the filename from the arguments, and each argument is separated by an '&' character. Spaces are not allowed in arguments and must be represented with the `%20` string. Similar encodings exist for other special characters.

How Does the Server Pass Arguments to the Child?

After a server receives a request such as

```
GET /cgi-bin/adder?15000&213 HTTP/1.1
```

Environment variable	Description
QUERY_STRING	Program arguments
SERVER_PORT	Port that the parent is listening on
REQUEST_METHOD	GET or POST
REMOTE_HOST	Domain name of client
REMOTE_ADDR	Dotted-decimal IP address of client
CONTENT_TYPE	POST only: MIME type of the request body
CONTENT_LENGTH	POST only: Size in bytes of the request body

Figure 11.26 Examples of CGI environment variables.

it calls `fork` to create a child process and calls `execve` to run the `/cgi-bin/adder` program in the context of the child. Programs like the `adder` program are often referred to as CGI programs because they obey the rules of the CGI standard. Before the call to `execve`, the child process sets the CGI environment variable QUERY_STRING to `15000&213`, which the `adder` program can reference at run time using the Linux `getenv` function.

How Do Servers Pass Other Information to Children?

CGI defines a number of other environment variables that a CGI program can expect to be set when it runs. Figure 11.26 shows a subset.

Where Does Child Output Go?

A CGI program sends its dynamic content to the standard output. Before the child process loads and runs the CGI program, it uses the Linux `dup2` function to redirect standard output to the connected descriptor that is associated with the client. Thus, anything that the CGI program writes to standard output goes directly to the client.

Notice that since the parent does not know the type or size of the content that the child generates, the child is responsible for generating the `Content-type` and `Content-length` response headers, as well as the empty line that terminates the headers.

Figure 11.27 shows a simple CGI program that sums its two arguments and returns an HTML file with the result to the client. Figure 11.28 shows an HTTP transaction that serves dynamic content from the `adder` program.

Practice Problem 11.5 (solution page 1005)

Assume that a CGI program needs to send dynamic content to the client. This is typically done by making the CGI program send its content to the standard output. Explain how this content is sent to the client.

Aside Pa i ar t i HTTP POST r t t_0 CGI r_0 ra

For POST requests, the child would also need to redirect standard input to the connected descriptor. The CGI program would then read the arguments in the request body from standard input.

code/netp/tiny/cgi-bin/adder.c

```
1  #include "csapp.h"
2
3
```

```
linux> telnet kittyhawk.cmcl.cs.cmu.edu 8000  Client: open connection
Trying 128.2.194.242...
Connected to kittyhawk.cmcl.cs.cmu.edu.
Escape character is '^]'.
GET /cgi-bin/adder?15000&213 HTTP/1.0  Client: request line
                                       Client: empty line terminates headers
HTTP/1.0 200 OK                        Server: response line
Server: Tiny Web Server                Server: identify server
Content-length: 115                    Adder: expect 115 bytes in response body
Content-type: text/html                Adder: expect HTML in response body
                                       Adder: empty line terminates headers
Welcome to add.com: THE Internet addition portal. Adder: first HTML line
<p>The answer is: 15000 + 213 = 15213  Adder: second HTML line in response body
<p>Thanks for visiting!                Adder: third HTML line in response body
Connection closed by foreign host.     Server: closes connection
linux>                                 Client: closes connection and terminates
```

Figure 11.28 An HTTP transaction that serves dynamic HTML content.

11.6 Putting It Together: The Tiny Web Server

We conclude our discussion of network programming by developing a small but functioning Web server called Tiny. Tiny is an interesting program. It combines many of the ideas that we have learned about, such as process control, Unix I/O, the sockets interface, and HTTP, in only 250 lines of code. While it lacks the functionality, robustness, and security of a real server, it is powerful enough to serve both static and dynamic content to real Web browsers. We encourage you to study it and implement it yourself. It is quite exciting (even for the authors!) to point a real browser at your own server and watch it display a complicated Web page with text and graphics.

T Tiny main R ti

Figure 11.29 shows Tiny's main routine. Tiny is an iterative server that listens for connection requests on the port that is passed in the command line. After opening a listening socket by calling the `open_listenfd` function, Tiny executes the typical infinite server loop, repeatedly accepting a connection request (line 32), performing a transaction (line 36), and closing its end of the connection (line 37).

T doit F ti

The `doit` function in Figure 11.30 handles one HTTP transaction. First, we read and parse the request line (lines 11–14). Notice that we are using the `rio_readlineb` function from Figure 10.8 to read the request line.

Tiny supports only the GET method. If the client requests another method (such as POST), we send it an error message and return to the main routine

code/netp/tiny/tiny.c

```
/*
 * tiny.c - A simple, iterative HTTP/1.0 Web server that uses the
 *     GET method to serve static and dynamic content
 */
#include "csapp.h"

void doit(int fd);
void read_requesthdrs(rio_t *rp);
int parse_uri(char *uri, char *filename, char *cgiargs);
void serve_static(int fd, char *filename, int filesize);
void get_filetype(char *filename, char *filetype);
void serve_dynamic(int fd, char *filename, char *cgiargs);
void clienterror(int fd, char *cause, char *errnum,
                 char *shortmsg, char *longmsg);

int main(int argc, char **argv)
{
    int listenfd, connfd;
    char hostname[MAXLINE], port[MAXLINE];
    socklen_t clientlen;
    struct sockaddr_storage clientaddr;

    /* Check command-line args */
    if (argc != 2) {
        fprintf(stderr, "usage: %s <port>\n", argv[0]);
        exit(1);
    }

    listenfd = Open_listenfd(argv[1]);
    while (1) {
        clientlen = sizeof(clientaddr);
        connfd = Accept(listenfd, (SA *)&clientaddr, &clientlen);
        Getnameinfo((SA *) &clientaddr, clientlen, hostname, MAXLINE,
                    port, MAXLINE, 0);
        printf("Accepted connection from (%s, %s)\n", hostname, port);
        doit(connfd);
        Close(connfd);
    }
}
```

code/netp/tiny/tiny.c

Figure 11.29 The Tiny Web server.

code/netp/tiny/tiny.c

```
1  void doit(int fd)
2  {
3      int is_static;
4      struct stat sbuf;
5      char buf[MAXLINE], method[MAXLINE], uri[MAXLINE], version[MAXLINE];
6      char filename[MAXLINE], cgiargs[MAXLINE];
7      rio_t rio;
8
9      /* Read request line and headers */
10     Rio_readinitb(&rio, fd);
11     Rio_readlineb(&rio, buf, MAXLINE);
12     printf("Request headers:\n");
13     printf("%s", buf);
14     sscanf(buf, "%s %s %s", method, uri, version);
15     if (strcasecmp(method, "GET")) {
16        clienterror(fd, method, "501", "Not implemented",
17                 "Tiny does not implement this method");
18         return;
19     }
20     read_requesthdrs(&rio);
21
22     /* Parse URI from GET request */
23     is_static = parse_uri(uri, filename, cgiargs);
24     if (stat(filename, &sbuf) < 0) {
25         clienterror(fd, filename, "404", "Not found",
26                     "Tiny couldn't find this file");
27         return;
28     }
29
30     if (is_static) { /* Serve static content */
31         if (!(S_ISREG(sbuf.st_mode)) || !(S_IRUSR & sbuf.st_mode)) {
32             clienterror(fd, filename, "403", "Forbidden",
33                         "Tiny couldn't read the file");
34             return;
35         }
36         serve_static(fd, filename, sbuf.st_size);
37     }
38     else { /* Serve dynamic content */
39         if (!(S_ISREG(sbuf.st_mode)) || !(S_IXUSR & sbuf.st_mode)) {
40             clienterror(fd, filename, "403", "Forbidden",
41                         "Tiny couldn't run the CGI program");
42             return;
43         }
44         serve_dynamic(fd, filename, cgiargs);
45     }
46 }
```

code/netp/tiny/tiny.c

Figure 11.30 Tiny doit handles one HTTP transaction.

(lines 15–19), which then closes the connection and awaits the next connection request. Otherwise, we read and (as we shall see) ignore any request headers (line 20).

Next, we parse the URI into a filename and a possibly empty CGI argument string, and we set a flag that indicates whether the request is for static or dynamic content (line 23). If the file does not exist on disk, we immediately send an error message to the client and return.

Finally, if the request is for static content, we verify that the file is a regular file and that we have read permission (line 31). If so, we serve the static content (line 36) to the client. Similarly, if the request is for dynamic content, we verify that the file is executable (line 39), and, if so, we go ahead and serve the dynamic content (line 44).

T clienterror F ti

Tiny lacks many of the error-handling features of a real server. However, it does check for some obvious errors and reports them to the client. The `clienterror` function in Figure 11.31 sends an HTTP response to the client with the appropriate

code/netp/tiny/tiny.c

```
1  void clienterror(int fd, char *cause, char *errnum,
2                   char *shortmsg, char *longmsg)
3  {
4      char buf[MAXLINE], body[MAXBUF];
5
6      /* Build the HTTP response body */
7      sprintf(body, "<html><title>Tiny Error</title>");
8      sprintf(body, "%s<body bgcolor=""ffffff"">\r\n", body);
9      sprintf(body, "%s%s: %s\r\n", body, errnum, shortmsg);
10     sprintf(body, "%s<p>%s: %s\r\n", body, longmsg, cause);
11     sprintf(body, "%s<hr><em>The Tiny Web server</em>\r\n", body);
12
13     /* Print the HTTP response */
14     sprintf(buf, "HTTP/1.0 %s %s\r\n", errnum, shortmsg);
15     Rio_writen(fd, buf, strlen(buf));
16     sprintf(buf, "Content-type: text/html\r\n");
17     Rio_writen(fd, buf, strlen(buf));
18     sprintf(buf, "Content-length: %d\r\n\r\n", (int)strlen(body));
19     Rio_writen(fd, buf, strlen(buf));
20     Rio_writen(fd, body, strlen(body));
21 }
```

code/netp/tiny/tiny.c

Figure 11.31 Tiny `clienterror` sends an error message to the client.

code/netp/tiny/tiny.c

```
1  void read_requesthdrs(rio_t *rp)
2  {
3      char buf[MAXLINE];
4
5      Rio_readlineb(rp, buf, MAXLINE);
6      while(strcmp(buf, "\r\n")) {
7          Rio_readlineb(rp, buf, MAXLINE);
8          printf("%s", buf);
9      }
10     return;
11 }
```

code/netp/tiny/tiny.c

Figure 11.32 Tiny `read_requesthdrs` reads and ignores request headers.

status code and status message in the response line, along with an HTML file in the response body that explains the error to the browser's user.

Recall that an HTML response should indicate the size and type of the content in the body. Thus, we have opted to build the HTML content as a single string so that we can easily determine its size. Also, notice that we are using the robust `rio_writen` function from Figure 10.4 for all output.

T read_requesthdrs F ti0

Tiny does not use any of the information in the request headers. It simply reads and ignores them by calling the `read_requesthdrs` function in Figure 11.32. Notice that the empty text line that terminates the request headers consists of a carriage return and line feed pair, which we check for in line 6.

T parse_uri F ti0

Tiny assumes that the home directory for static content is its current directory and that the home directory for executables is `./cgi-bin`. Any URI that contains the string `cgi-bin` is assumed to denote a request for dynamic content. The default filename is `./home.html`.

The `parse_uri` function in Figure 11.33 implements these policies. It parses the URI into a filename and an optional CGI argument string. If the request is for static content (line 5), we clear the CGI argument string (line 6) and then convert the URI into a relative Linux pathname such as `./index.html` (lines 7–8). If the URI ends with a '`/`' character (line 9), then we append the default filename (line 10). On the other hand, if the request is for dynamic content (line 13), we extract any CGI arguments (lines 14–20) and convert the remaining portion of the URI to a relative Linux filename (lines 21–22).

code/netp/tiny/tiny.c

```
1   int parse_uri(char *uri, char *filename, char *cgiargs)
2   {
3       char *ptr;
4
5       if (!strstr(uri, "cgi-bin")) {  /* Static content */
6           strcpy(cgiargs, "");
7           strcpy(filename, ".");
8           strcat(filename, uri);
9           if (uri[strlen(uri)-1] == '/')
10              strcat(filename, "home.html");
11          return 1;
12      }
13      else {  /* Dynamic content */
14          ptr = index(uri, '?');
15          if (ptr) {
16              strcpy(cgiargs, ptr+1);
17              *ptr = '\0';
18          }
19          else
20              strcpy(cgiargs, "");
21          strcpy(filename, ".");
22          strcat(filename, uri);
23          return 0;
24      }
25  }
```

code/netp/tiny/tiny.c

Figure 11.33 Tiny `parse_uri` parses an HTTP URI.

T serve_static F ti

Tiny serves five common types of static content: HTML files, unformatted text files, and images encoded in GIF, PNG, and JPEG formats.

The `serve_static` function in Figure 11.34 sends an HTTP response whose body contains the contents of a local file. First, we determine the file type by inspecting the suffix in the filename (line 7) and then send the response line and response headers to the client (lines 8–13). Notice that a blank line terminates the headers.

Next, we send the response body by copying the contents of the requested file to the connected descriptor `fd`. The code here is somewhat subtle and needs to be studied carefully. Line 18 opens `filename` for reading and gets its descriptor. In line 19, the Linux `mmap` function maps the requested file to a virtual memory area. Recall from our discussion of `mmap` in Section 9.8 that the call to `mmap` maps the

```
void serve_static(int fd, char *filename, int filesize)
{
    int srcfd;
    char *srcp, filetype[MAXLINE], buf[MAXBUF];

    /* Send response headers to client */
    get_filetype(filename, filetype);
    sprintf(buf, "HTTP/1.0 200 OK\r\n");
    sprintf(buf, "%sServer: Tiny Web Server\r\n", buf);
    sprintf(buf, "%sConnection: close\r\n", buf);
    sprintf(buf, "%sContent-length: %d\r\n", buf, filesize);
    sprintf(buf, "%sContent-type: %s\r\n\r\n", buf, filetype);
    Rio_writen(fd, buf, strlen(buf));
    printf("Response headers:\n");
    printf("%s", buf);

    /* Send response body to client */
    srcfd = Open(filename, O_RDONLY, 0);
    srcp = Mmap(0, filesize, PROT_READ, MAP_PRIVATE, srcfd, 0);
    Close(srcfd);
    Rio_writen(fd, srcp, filesize);
    Munmap(srcp, filesize);
}

/*
 * get_filetype - Derive file type from filename
 */
void get_filetype(char *filename, char *filetype)
{
    if (strstr(filename, ".html"))
        strcpy(filetype, "text/html");
    else if (strstr(filename, ".gif"))
        strcpy(filetype, "image/gif");
    else if (strstr(filename, ".png"))
        strcpy(filetype, "image/png");
    else if (strstr(filename, ".jpg"))
        strcpy(filetype, "image/jpeg");

```

first `filesize` bytes of file `srcfd` to a private read-only area of virtual memory that starts at address `srcp`.

Once we have mapped the file to memory, we no longer need its descriptor, so we close the file (line 20). Failing to do this would introduce a potentially fatal memory leak. Line 21 performs the actual transfer of the file to the client. The `rio_writen` function copies the `filesize` bytes starting at location `srcp` (which of course is mapped to the requested file) to the client's connected descriptor. Finally, line 22 frees the mapped virtual memory area. This is important to avoid a potentially fatal memory leak.

T serve_dynamic F ti

Tiny serves any type of dynamic content by forking a child process and then running a CGI program in the context of the child.

The `serve_dynamic` function in Figure 11.35 begins by sending a response line indicating success to the client, along with an informational `Server` header. The CGI program is responsible for sending the rest of the response. Notice that this is not as robust as we might wish, since it doesn't allow for the possibility that the CGI program might encounter some error.

After sending the first part of the response, we fork a new child process (line 11). The child initializes the QUERY_STRING environment variable with the CGI arguments from the request URI (line 13). Notice that a real server would

code/netp/tiny/tiny.c

```
1  void serve_dynamic(int fd, char *filename, char *cgiargs)
2  {
3      char buf[MAXLINE], *emptylist[] = { NULL };
4
5      /* Return first part of HTTP response */
6      sprintf(buf, "HTTP/1.0 200 OK\r\n");
7      Rio_writen(fd, buf, strlen(buf));
8      sprintf(buf, "Server: Tiny Web Server\r\n");
9      Rio_writen(fd, buf, strlen(buf));
10
11     if (Fork() == 0) { /* Child */
12         /* Real server would set all CGI vars here */
13         setenv("QUERY_STRING", cgiargs, 1);
14         Dup2(fd, STDOUT_FILENO);         /* Redirect stdout to client */
15         Execve(filename, emptylist, environ); /* Run CGI program */
16     }
17     Wait(NULL); /* Parent waits for and reaps child */
18 }
```

code/netp/tiny/tiny.c

Figure 11.35 Tiny `serve_dynamic` serves dynamic content to a client.

Aside Dali t r at r l l o ' o ti o

Although the basic functions of a Web server are quite simple, we don't want to give you the false impression that writing a real Web server is easy. Building a robust Web server that runs for extended periods without crashing is a difficult task that requires a deeper understanding of Linux systems programming than we've learned here. For example, if a server writes to a connection that has already been closed by the client (say, because you clicked the "Stop" button on your browser), then the first such write returns normally, but the second write causes the delivery of a SIGPIPE signal whose default behavior is to terminate the process. If the SIGPIPE signal is caught or ignored, then the second write operation returns −1 with `errno` set to EPIPE. The `strerr` and `perror` functions report the EPIPE error as a "Broken pipe," a nonintuitive message that has confused generations of students. The bottom line is that a robust server must catch these SIGPIPE signals and check `write` function calls for EPIPE errors.

set the other CGI environment variables here as well. For brevity, we have omitted this step.

Next, the child redirects the child's standard output to the connected file descriptor (line 14) and then loads and runs the CGI program (line 15). Since the CGI program runs in the context of the child, it has access to the same open files and environment variables that existed before the call to the `execve` function. Thus, everything that the CGI program writes to standard output goes directly to the client process, without any intervention from the parent process. Meanwhile, the parent blocks in a call to `wait`, waiting to reap the child when it terminates (line 17).

11.7 Summary

Every network application is based on the client-server model. With this model, an application consists of a server and one or more clients. The server manages resources, providing a service for its clients by manipulating the resources in some way. The basic operation in the client-server model is a client-server transaction, which consists of a request from a client, followed by a response from the server.

Clients and servers communicate over a global network known as the Internet. From a programmer's point of view, we can think of the Internet as a worldwide collection of hosts with the following properties: (1) Each Internet host has a unique 32-bit name called its IP address. (2) The set of IP addresses is mapped to a set of Internet domain names. (3) Processes on different Internet hosts can communicate with each other over connections.

Clients and servers establish connections by using the sockets interface. A socket is an end point of a connection that is presented to applications in the form of a file descriptor. The sockets interface provides functions for opening and closing socket descriptors. Clients and servers communicate with each other by reading and writing these descriptors.

Web servers and their clients (such as browsers) communicate with each other using the HTTP protocol. A browser requests either static or dynamic content from the server. A request for static content is served by fetching a file from the server's disk and returning it to the client. A request for dynamic content is served by running a program in the context of a child process on the server and returning its output to the client. The CGI standard provides a set of rules that govern how the client passes program arguments to the server, how the server passes these arguments and other information to the child process, and how the child sends its output back to the client. A simple but functioning Web server that serves both static and dynamic content can be implemented in a few hundred lines of C code.

Bibliographic Notes

The official source of information for the Internet is contained in a set of freely available numbered documents known as RFCs (requests for comments). A searchable index of RFCs is available on the Web at

http://rfc-editor.org

RFCs are typically written for developers of Internet infrastructure, and thus they are usually too detailed for the casual reader. However, for authoritative information, there is no better source. The HTTP/1.1 protocol is documented in RFC 2616. The authoritative list of MIME types is maintained at

http://www.iana.org/assignments/media-types

Kerrisk is the bible for all aspects of Linux programming and provides a detailed discussion of modern network programming [62]. There are a number of good general texts on computer networking [65, 84, 114]. The great technical writer W. Richard Stevens developed a series of classic texts on such topics as advanced Unix programming [111], the Internet protocols [109, 120, 107], and Unix network programming [108, 110]. Serious students of Unix systems programming will want to study all of them. Tragically, Stevens died on September 1, 1999. His contributions are greatly missed.

Homework Problems

11.6

A. Modify TINY so that it echoes every request line and request header.

B. Use your favorite browser to make a request to TINY for static content. Capture the output from TINY in a file.

C. Inspect the output from TINY to determine the version of HTTP your browser uses.

D. Consult the HTTP/1.1 standard in RFC 2616 to determine the meaning of each header in the HTTP request from your browser. You can obtain RFC 2616 from www.rfc-editor.org/rfc.html.

11.7
Extend TINY so that it serves MPG video files. Check your work using a real browser.

11.8
Modify TINY so that it reaps CGI children inside a SIGCHLD handler instead of explicitly waiting for them to terminate.

11.9
Modify TINY so that when it serves static content, it copies the requested file to the connected descriptor using `malloc`, `rio_readn`, and `rio_writen`, instead of `mmap` and `rio_writen`.

11.10
A. Write an HTML form for the CGI `adder` function in Figure 11.27. Your form should include two text boxes that users fill in with the two numbers to be added together. Your form should request content using the GET method.

B. Check your work by using a real browser to request the form from TINY, submit the filled-in form to TINY, and then display the dynamic content generated by `adder`.

11.11
Extend TINY to support the HTTP HEAD method. Check your work using TELNET as a Web client.

11.12
Extend TINY so that it serves dynamic content requested by the HTTP POST method. Check your work using your favorite Web browser.

11.13
Modify TINY so that it deals cleanly (without terminating) with the SIGPIPE signals and EPIPE errors that occur when the `write` function attempts to write to a prematurely closed connection.

Solutions to Practice Problems

Solution to Problem 11.1 (page 963)

Dotted-decimal address	Hex address
`107.212.122.205`	`0x6BD47ACD`
`64.12.149.13`	`0x400C950D`
`107.212.96.29`	`0x6BD4601D`
`[0.0].[0.128]`	`0x00000080`

Dotted-decimal address	Hex address
[255.255].[255.0]	0xFFFFFF00
[10.1].[1.64]	0x0A010140

Solution to Problem 11.2 (page 963)

code/netp/global-hex2dd.c

```
1  #include "csapp.h"
2
3  int main(int argc, char **argv)
4  {
5      struct in_addr inaddr;  /* Address in network byte order */
6      uint16_t addr;          /* Address in host byte order */
7      char buf[MAXBUF];       /* Buffer for dotted-decimal string */
8
9      if (argc != 2) {
10         fprintf(stderr, "usage: %s <hex number>\n", argv[0]);
11         exit(0);
12     }
13     sscanf(argv[1], "%x", &addr);
14     inaddr.s_addr = htons(addr);
15
16     if (!inet_ntop(AF_INET, &inaddr, buf, MAXBUF))
17         unix_error("inet_ntop");
18     printf("%s\n", buf);
19
20     exit(0);
21 }
```

code/net/global-hex2dd.c

Solution to Problem 11.3 (page 963)

code/netp/global-dd2hex.c

```
1  #include "csapp.h"
2
3  int main(int argc, char **argv)
4  {
5      struct in_addr inaddr;  /* Address in network byte order */
6      int rc;
7
8      if (argc != 2) {
9          fprintf(stderr, "usage: %s <network byte order>\n", argv[0]);
10         exit(0);
11     }
12
13     rc = inet_pton(AF_INET, argv[1], &inaddr);
14     if (rc == 0)
15         app_error("inet_pton error: invalid network byte order");
```

```
16      else if (rc < 0)
17          unix_error("inet_pton error");
18
19      printf("0x%x\n", ntohs(inaddr.s_addr));
20      exit(0);
21  }
```

code/netp/global-dd2hex.c

Solution to Problem 11.4 (page 978)

Here's a solution. Notice how much more difficult it is to use `inet_ntop`, which requires messy casting and deep structure references. The `getnameinfo` function is much simpler because it does all of that work for us.

code/netp/hostinfo-ntop.c

```
1   #include "csapp.h"
2
3   int main(int argc, char **argv)
4   {
5       struct addrinfo *p, *listp, hints;
6       struct sockaddr_in *sockp;
7       char buf[MAXLINE];
8       int rc;
9
10      if (argc != 2) {
11          fprintf(stderr, "usage: %s <domain name>\n", argv[0]);
12          exit(0);
13      }
14
15      /* Get a list of addrinfo records */
16      memset(&hints, 0, sizeof(struct addrinfo));
17      hints.ai_family = AF_INET;       /* IPv4 only */
18      hints.ai_socktype = SOCK_STREAM; /* Connections only */
19      if ((rc = getaddrinfo(argv[1], NULL, &hints, &listp)) != 0) {
20          fprintf(stderr, "getaddrinfo error: %s\n", gai_strerror(rc));
21          exit(1);
22      }
23
24      /* Walk the list and display each associated IP address */
25      for (p = listp; p; p = p->ai_next) {
26          sockp = (struct sockaddr_in *)p->ai_addr;
27          Inet_ntop(AF_INET, &(sockp->sin_addr), buf, MAXLINE);
28          printf("%s\n", buf);
29      }
30
```

```
31      /* Clean up */
32      Freeaddrinfo(listp);
33
34      exit(0);
35  }
```

code/netp/hostinfo-ntop.c

Solution to Problem 11.5 (page 990)

Before the process that runs the CGI program is loaded, a Linux `dup2` function is used to redirect standard output to the connected descriptor that is associated with the client. Thus, anything that the CGI program writes to standard output goes directly to the client.

12

Concurrent Programming

12.1 C_o rr t Pr_o ra i it Pr_o 1009

12.2 C_o rr t Pr_o ra i it I/O M lti l i 1013

12.3 C_o rr t Pr_o ra i it T r á 1021

12.4 S ar Varia l i T r á Pr_o ra 1028

12.5 S r_o i i T r á it S a _o r 1031

12.6 U i T r á _o r Parall li 1049

12.7 Ot r C_o rr l 1056

12.8 S ar 1066

Bi li_o ra i N_o t 1066

H_o _o r Pr_o l 1067

S_o l ti_o t_o Pra ti Pr_o l 1072

As we learned in Chapter 8, logical control flows are concurrent if they overlap in time. This general phenomenon, known as concurrency, shows up at many different levels of a computer system. Hardware exception handlers, processes, and Linux signal handlers are all familiar examples.

Thus far, we have treated concurrency mainly as a mechanism that the operating system kernel uses to run multiple application programs. But concurrency is not just limited to the kernel. It can play an important role in application programs as well. For example, we have seen how Linux signal handlers allow applications to respond to asynchronous events such as the user typing Ctrl+C or the program accessing an undefined area of virtual memory. Application-level concurrency is useful in other ways as well:

- Accessing slow I/O devices. When an application is waiting for data to arrive from a slow I/O device such as a disk, the kernel keeps the CPU busy by running other processes. Individual applications can exploit concurrency in a similar way by overlapping useful work with I/O requests.
- Interacting with humans. People who interact with computers demand the ability to perform multiple tasks at the same time. For example, they might want to resize a window while they are printing a document. Modern windowing systems use concurrency to provide this capability. Each time the user requests some action (say, by clicking the mouse), a separate concurrent logical flow is created to perform the action.
- Reducing latency by deferring work. Sometimes, applications can use concurrency to reduce the latency of certain operations by deferring other operations and performing them concurrently. For example, a dynamic storage allocator might reduce the latency of individual `free` operations by deferring coalescing to a concurrent "coalescing" flow that runs at a lower priority, soaking up spare CPU cycles as they become available.
- Servicing multiple network clients. The iterative network servers that we studied in Chapter 11 are unrealistic because they can only service one client at a time. Thus, a single slow client can deny service to every other client. For a real server that might be expected to service hundreds or thousands of clients per second, it is not acceptable to allow one slow client to deny service to the others. A better approach is to build a concurrent server that creates a separate logical flow for each client. This allows the server to service multiple clients concurrently and precludes slow clients from monopolizing the server.
- Computing in parallel on multi-core machines. Many modern systems are equipped with multi-core processors that contain multiple CPUs. Applications that are partitioned into concurrent flows often run faster on multi-core machines than on uniprocessor machines because the flows execute in parallel rather than being interleaved.

Applications that use application-level concurrency are known as concurrent programs. Modern operating systems provide three basic approaches for building concurrent programs:

- Processes. With this approach, each logical control flow is a process that is scheduled and maintained by the kernel. Since processes have separate virtual address spaces, flows that want to communicate with each other must use some kind of explicit interprocess communication (IPC) mechanism.
- I/O multiplexing. This is a form of concurrent programming where applications explicitly schedule their own logical flows in the context of a single process. Logical flows are modeled as state machines that the main program explicitly transitions from state to state as a result of data arriving on file descriptors. Since the program is a single process, all flows share the same address space.
- Threads. Threads are logical flows that run in the context of a single process and are scheduled by the kernel. You can think of threads as a hybrid of the other two approaches, scheduled by the kernel like process flows and sharing the same virtual address space like I/O multiplexing flows.

This chapter investigates these three different concurrent programming techniques. To keep our discussion concrete, we will work with the same motivating application throughout—a concurrent version of the iterative echo server from Section 11.4.9.

12.1 Concurrent Programming with Processes

The simplest way to build a concurrent program is with processes, using familiar functions such as `fork`, `exec`, and `waitpid`. For example, a natural approach for building a concurrent server is to accept client connection requests in the parent and then create a new child process to service each new client.

To see how this might work, suppose we have two clients and a server that is listening for connection requests on a listening descriptor (say, 3). Now suppose that the server accepts a connection request from client 1 and returns a connected descriptor (say, 4), as shown in Figure 12.1. After accepting the connection request, the server forks a child, which gets a complete copy of the server's descriptor table. The child closes its copy of listening descriptor 3, and the parent closes its copy of connected descriptor 4, since they are no longer needed. This gives us the situation shown in Figure 12.2, where the child process is busy servicing the client.

Since the connected descriptors in the parent and child each point to the same file table entry, it is crucial for the parent to close its copy of the connected

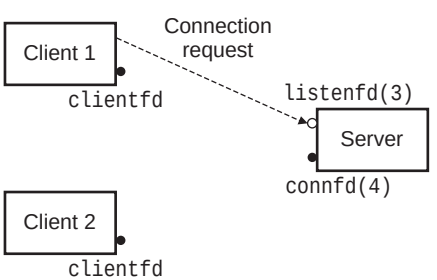


Figure 12.1
Step 1: Server accepts connection request from client.

Figure 12.2
Step 2: Server forks a child process to service the client.

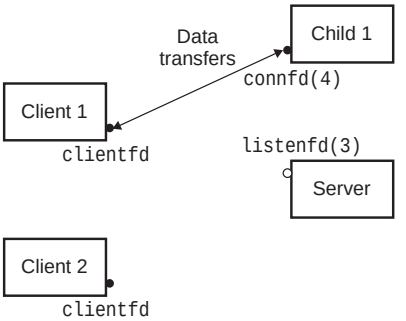
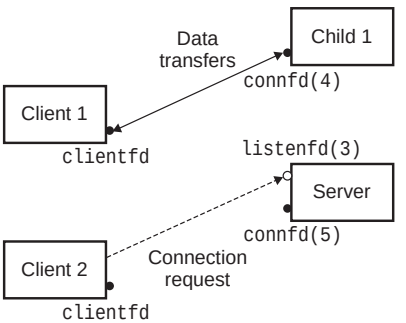


Figure 12.3
Step 3: Server accepts another connection request.



descriptor. Otherwise, the file table entry for connected descriptor 4 will never be released, and the resulting memory leak will eventually consume the available memory and crash the system.

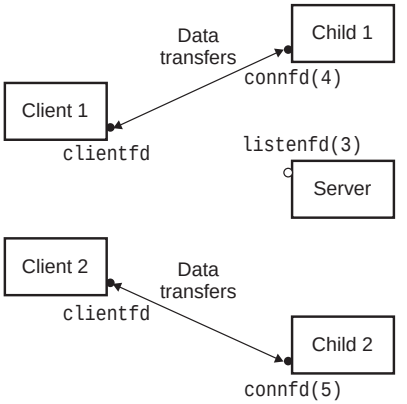
Now suppose that after the parent creates the child for client 1, it accepts a new connection request from client 2 and returns a new connected descriptor (say, 5), as shown in Figure 12.3. The parent then forks another child, which begins servicing its client using connected descriptor 5, as shown in Figure 12.4. At this point, the parent is waiting for the next connection request and the two children are servicing their respective clients concurrently.

12.1.1 A Concurrent Server Based on Processes

Figure 12.5 shows the code for a concurrent echo server based on processes. The `echo` function called in line 29 comes from Figure 11.22. There are several important points to make about this server:

- First, servers typically run for long periods of time, so we must include a SIGCHLD handler that reaps zombie children (lines 4–9). Since SIGCHLD signals are blocked while the SIGCHLD handler is executing, and since Linux signals are not queued, the SIGCHLD handler must be prepared to reap multiple zombie children.
- Second, the parent and the child must close their respective copies of `connfd` (lines 33 and 30, respectively). As we have mentioned, this is especially im-

Figure 12.4
Step 4: Server forks another child to service the new client.



portant for the parent, which must close its copy of the connected descriptor to avoid a memory leak.

- Finally, because of the reference count in the socket's file table entry, the connection to the client will not be terminated until both the parent's and child's copies of `connfd` are closed.

12.1.2 Pros and Cons of Processes

Processes have a clean model for sharing state information between parents and children: file tables are shared and user address spaces are not. Having separate address spaces for processes is both an advantage and a disadvantage. It is impossible for one process to accidentally overwrite the virtual memory of another process, which eliminates a lot of confusing failures—an obvious advantage.

On the other hand, separate address spaces make it more difficult for processes to share state information. To share information, they must use explicit IPC (interprocess communications) mechanisms. (See the Aside on page 1013.) Another disadvantage of process-based designs is that they tend to be slower because the overhead for process control and IPC is high.

Practice Problem 12.1 (solution page 1072)

Figure 12.5 demonstrates a concurrent server in which the parent process creates a child process to handle each new connection request. Trace the value of the reference counter for the associated file table for Figure 12.5.

Practice Problem 12.2 (solution page 1072)

If we were to delete line 33 of Figure12.5, which closes the connected descriptor, the code would still be correct, in the sense that there would be no memory leak. Why?

code/conc/echoserverp.c

```
1  #include "csapp.h"
2  void echo(int connfd);
3
4  void sigchld_handler(int sig)
5  {
6      while (waitpid(-1, 0, WNOHANG) > 0)
7          ;
8      return;
9  }
10
11 int main(int argc, char **argv)
12 {
13     int listenfd, connfd;
14     socklen_t clientlen;
15     struct sockaddr_storage clientaddr;
16
17     if (argc != 2) {
18         fprintf(stderr, "usage: %s <port>\n", argv[0]);
19         exit(0);
20     }
21
22     Signal(SIGCHLD, sigchld_handler);
23     listenfd = Open_listenfd(argv[1]);
24     while (1) {
25         clientlen = sizeof(struct sockaddr_storage);
26         connfd = Accept(listenfd, (SA *) &clientaddr, &clientlen);
27         if (Fork() == 0) {
28             Close(listenfd); /* Child closes its listening socket */
29             echo(connfd);    /* Child services client */
30             Close(connfd);   /* Child closes connection with client */
31             exit(0);         /* Child exits */
32         }
33         Close(connfd); /* Parent closes connected socket (important!) */
34     }
35 }
```

code/conc/echoserverp.c

Figure 12.5 Concurrent echo server based on processes. T ar t r a t a l a ti r t.

Aside Unix IPC

You have already encountered several examples of IPC in this text. The `waitpid` function and signals from Chapter 8 are primitive IPC mechanisms that allow processes to send tiny messages to processes running on the same host. The sockets interface from Chapter 11 is an important form of IPC that allows processes on different hosts to exchange arbitrary byte streams. However, the term Unix IPC is typically reserved for a hodgepodge of techniques that allow processes to communicate with other processes that are running on the same host. Examples include pipes, FIFOs, System V shared memory, and System V semaphores. These mechanisms are beyond our scope. The book by Kerrisk [62] is an excellent reference.

12.2 Concurrent Programming with I/O Multiplexing

Suppose you are asked to write an echo server that can also respond to interactive commands that the user types to standard input. In this case, the server must respond to two independent I/O events: (1) a network client making a connection request, and (2) a user typing a command line at the keyboard. Which event do we wait for first? Neither option is ideal. If we are waiting for a connection request in `accept`, then we cannot respond to input commands. Similarly, if we are waiting for an input command in `read`, then we cannot respond to any connection requests.

One solution to this dilemma is a technique called I/O multiplexing. The basic idea is to use the `select` function to ask the kernel to suspend the process, returning control to the application only after one or more I/O events have occurred, as in the following examples:

- Return when any descriptor in the set {0, 4} is ready for reading.
- Return when any descriptor in the set {1, 2, 7} is ready for writing.
- Time out if 152.13 seconds have elapsed waiting for an I/O event to occur.

`Select` is a complicated function with many different usage scenarios. We will only discuss the first scenario: waiting for a set of descriptors to be ready for reading. See [62, 110] for a complete discussion.

```
#include <sys/select.h>

int select(int n, fd_set *fdset, NULL, NULL, NULL);
                          Returns: nonzero count of ready descriptors, −1 on error

FD_ZERO(fd_set *fdset);           /* Clear all bits in fdset */
FD_CLR(int fd, fd_set *fdset);    /* Clear bit fd in fdset */
FD_SET(int fd, fd_set *fdset);    /* Turn on bit fd in fdset */
FD_ISSET(int fd, fd_set *fdset);  /* Is bit fd in fdset on? */
                                  Macros for manipulating descriptor sets
```

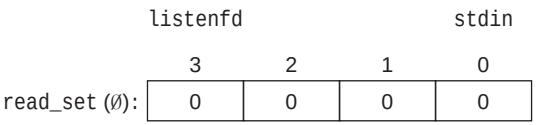
The `select` function manipulates sets of type `fd_set`, which are known as *descriptor sets*. Logically, we think of a descriptor set as a bit vector (introduced in Section 2.1) of size `n`:

$$b_{n-1}, \ldots, b_1, b_0$$

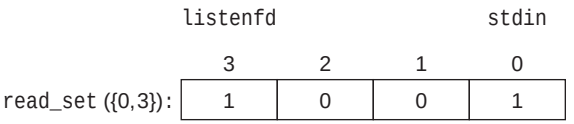
Each bit b_k corresponds to descriptor `k`. Descriptor `k` is a member of the descriptor set if and only if $b_k = 1$. You are only allowed to do three things with descriptor sets: (1) allocate them, (2) assign one variable of this type to another, and (3) modify and inspect them using the FD_ZERO, FD_SET, FD_CLR, and FD_ISSET macros.

For our purposes, the `select` function takes two inputs: a descriptor set (`fdset`) called the *read set*, and the cardinality (`n`) of the read set (actually the maximum cardinality of any descriptor set). The `select` function blocks until at least one descriptor in the read set is ready for reading. A descriptor `k` is *ready for reading* if and only if a request to read 1 byte from that descriptor would not block. As a side effect, `select` modifies the `fd_set` pointed to by argument `fdset` to indicate a subset of the read set called the *ready set*, consisting of the descriptors in the read set that are ready for reading. The value returned by the function indicates the cardinality of the ready set. Note that because of the side effect, we must update the read set every time `select` is called.

The best way to understand `select` is to study a concrete example. Figure 12.6 shows how we might use `select` to implement an iterative echo server that also accepts user commands on the standard input. We begin by using the `open_listenfd` function from Figure 11.19 to open a listening descriptor (line 16), and then using FD_ZERO to create an empty read set (line 18):



Next, in lines 19 and 20, we define the read set to consist of descriptor 0 (standard input) and descriptor 3 (the listening descriptor), respectively:



At this point, we begin the typical server loop. But instead of waiting for a connection request by calling the `accept` function, we call the `select` function, which blocks until either the listening descriptor or standard input is ready for reading (line 24). For example, here is the value of `ready_set` that `select` would return if the user hit the enter key, thus causing the standard input descriptor to

code/conc/select.c

```
1   #include "csapp.h"
2   void echo(int connfd);
3   void command(void);
4
5   int main(int argc, char **argv)
6   {
7       int listenfd, connfd;
8       socklen_t clientlen;
9       struct sockaddr_storage clientaddr;
10      fd_set read_set, ready_set;
11
12      if (argc != 2) {
13          fprintf(stderr, "usage: %s <port>\n", argv[0]);
14          exit(0);
15      }
16      listenfd = Open_listenfd(argv[1]);
17
18      FD_ZERO(&read_set);              /* Clear read set */
19      FD_SET(STDIN_FILENO, &read_set); /* Add stdin to read set */
20      FD_SET(listenfd, &read_set);     /* Add listenfd to read set */
21
22      while (1) {
23          ready_set = read_set;
24          Select(listenfd+1, &ready_set, NULL, NULL, NULL);
25          if (FD_ISSET(STDIN_FILENO, &ready_set))
26              command(); /* Read command line from stdin */
27          if (FD_ISSET(listenfd, &ready_set)) {
28              clientlen = sizeof(struct sockaddr_storage);
29              connfd = Accept(listenfd, (SA *)&clientaddr, &clientlen);
30              echo(connfd); /* Echo client input until EOF */
31              Close(connfd);
32          }
33      }
34  }
35
36  void command(void) {
37      char buf[MAXLINE];
38      if (!Fgets(buf, MAXLINE, stdin))
39          exit(0); /* EOF */
40      printf("%s", buf); /* Process the input command */
41  }
```

code/conc/select.c

Figure 12.6 An iterative echo server that uses I/O multiplexing. T r r select t ai t o r ti r t a lit i ri t ra o a o ta ar i t.

become ready for reading:

	listenfd			stdin
	3	2	1	0
ready_set ({0}):	0	0	0	1

Once `select` returns, we use the FD_ISSET macro to determine which descriptors are ready for reading. If standard input is ready (line 25), we call the `command` function, which reads, parses, and responds to the command before returning to the main routine. If the listening descriptor is ready (line 27), we call `accept` to get a connected descriptor and then call the `echo` function from Figure 11.22, which echoes each line from the client until the client closes its end of the connection.

While this program is a good example of using `select`, it still leaves something to be desired. The problem is that once it connects to a client, it continues echoing input lines until the client closes its end of the connection. Thus, if you type a command to standard input, you will not get a response until the server is finished with the client. A better approach would be to multiplex at a finer granularity, echoing (at most) one text line each time through the server loop.

Practice Problem 12.3 (solution page 1072)

In Linux systems, typing Ctrl+D indicates EOF on standard input. What happens if you type Ctrl+D to the program in Figure 12.6 while it is echoing each line of the client?

12.2.1 A Concurrent Event-Driven Server Based on I/O Multiplexing

I/O multiplexing can be used as the basis for concurrent *event-driven* programs, where flows make progress as a result of certain events. The general idea is to model logical flows as state machines. Informally, a *state machine* is a collection of *states*, *input events*, and *transitions* that map states and input events to states. Each transition maps an (input state, input event) pair to an output state. A *self-loop* is a transition between the same input and output state. State machines are typically drawn as directed graphs, where nodes represent states, directed arcs represent transitions, and arc labels represent input events. A state machine begins execution in some initial state. Each input event triggers a transition from the current state to the next state.

For each new client k, a concurrent server based on I/O multiplexing creates a new state machine s_k and associates it with connected descriptor d_k. As shown in Figure 12.7, each state machine s_k has one state ("waiting for descriptor d_k to be ready for reading"), one input event ("descriptor d_k is ready for reading"), and one transition ("read a text line from descriptor d_k").

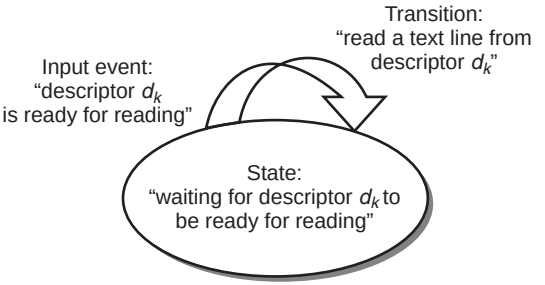


Figure 12.7
State machine for a logical flow in a concurrent event-driven echo server.

The server uses the I/O multiplexing, courtesy of the `select` function, to detect the occurrence of input events. As each connected descriptor becomes ready for reading, the server executes the transition for the corresponding state machine—in this case, reading and echoing a text line from the descriptor.

Figure 12.8 shows the complete example code for a concurrent event-driven server based on I/O multiplexing. The set of active clients is maintained in a `pool` structure (lines 3–11). After initializing the pool by calling `init_pool` (line 27), the server enters an infinite loop. During each iteration of this loop, the server calls the `select` function to detect two different kinds of input events: (1) a connection request arriving from a new client, and (2) a connected descriptor for an existing client being ready for reading. When a connection request arrives (line 35), the server opens the connection (line 37) and calls the `add_client` function to add the client to the pool (line 38). Finally, the server calls the `check_clients` function to echo a single text line from each ready connected descriptor (line 42).

The `init_pool` function (Figure 12.9) initializes the client pool. The `clientfd` array represents a set of connected descriptors, with the integer `-1` denoting an available slot. Initially, the set of connected descriptors is empty (lines 5–7), and the listening descriptor is the only descriptor in the `select` read set (lines 10–12).

The `add_client` function (Figure 12.10) adds a new client to the pool of active clients. After finding an empty slot in the `clientfd` array, the server adds the connected descriptor to the array and initializes a corresponding RIO read buffer so that we can call `rio_readlineb` on the descriptor (lines 8–9). We then add the connected descriptor to the `select` read set (line 12), and we update some global properties of the pool. The `maxfd` variable (lines 15–16) keeps track of the largest file descriptor for `select`. The `maxi` variable (lines 17–18) keeps track of the largest index into the `clientfd` array so that the `check_clients` function does not have to search the entire array.

The `check_clients` function in Figure 12.11 echoes a text line from each ready connected descriptor. If we are successful in reading a text line from the descriptor, then we echo that line back to the client (lines 15–18). Notice that in line 15, we are maintaining a cumulative count of total bytes received from all clients. If we detect EOF because the client has closed its end of the connection, then we close our end of the connection (line 23) and remove the descriptor from the pool (lines 24–25).

code/conc/echoservers.c

```
1   #include "csapp.h"
2
3   typedef struct { /* Represents a pool of connected descriptors */
4       int maxfd;        /* Largest descriptor in read_set */
5       fd_set read_set;  /* Set of all active descriptors */
6       fd_set ready_set; /* Subset of descriptors ready for reading  */
7       int nready;       /* Number of ready descriptors from select */
8       int maxi;         /* High water index into client array */
9       int clientfd[FD_SETSIZE];    /* Set of active descriptors */
10      rio_t clientrio[FD_SETSIZE]; /* Set of active read buffers */
11  } pool;
12
13  int byte_cnt = 0; /* Counts total bytes received by server */
14
15  int main(int argc, char **argv)
16  {
17      int listenfd, connfd;
18      socklen_t clientlen;
19      struct sockaddr_storage clientaddr;
20      static pool pool;
21
22      if (argc != 2) {
23          fprintf(stderr, "usage: %s <port>\n", argv[0]);
24          exit(0);
25      }
26      listenfd = Open_listenfd(argv[1]);
27      init_pool(listenfd, &pool);
28
29      while (1) {
30          /* Wait for listening/connected descriptor(s) to become ready */
31          pool.ready_set = pool.read_set;
32          pool.nready = Select(pool.maxfd+1, &pool.ready_set, NULL, NULL, NULL);
33
34          /* If listening descriptor ready, add new client to pool */
35          if (FD_ISSET(listenfd, &pool.ready_set)) {
36              clientlen = sizeof(struct sockaddr_storage);
37              connfd = Accept(listenfd, (SA *)&clientaddr, &clientlen);
38              add_client(connfd, &pool);
39          }
40
41          /* Echo a text line from each ready connected descriptor */
42          check_clients(&pool);
43      }
44  }
```

code/conc/echoservers.c

Figure 12.8 Concurrent echo server based on I/O multiplexing. Ea r r it rati o o a t t li r o a r a v v ri t o r.

code/conc/echoservers.c

```
1  void init_pool(int listenfd, pool *p)
2  {
3      /* Initially, there are no connected descriptors */
4      int i;
5      p->maxi = -1;
6      for (i=0; i< FD_SETSIZE; i++)
7          p->clientfd[i] = -1;
8
9      /* Initially, listenfd is only member of select read set */
10     p->maxfd = listenfd;
11     FD_ZERO(&p->read_set);
12     FD_SET(listenfd, &p->read_set);
13 }
```

code/conc/echoservers.c

Figure 12.9 `init_pool` initializes the pool of active clients.

code/conc/echoservers.c

```
1  void add_client(int connfd, pool *p)
2  {
3      int i;
4      p->nready--;
5      for (i = 0; i < FD_SETSIZE; i++) /* Find an available slot */
6          if (p->clientfd[i] < 0) {
7              /* Add connected descriptor to the pool */
8              p->clientfd[i] = connfd;
9              Rio_readinitb(&p->clientrio[i], connfd);
10
11             /* Add the descriptor to descriptor set */
12             FD_SET(connfd, &p->read_set);
13
14             /* Update max descriptor and pool high water mark */
15             if (connfd > p->maxfd)
16                 p->maxfd = connfd;
17             if (i > p->maxi)
18                 p->maxi = i;
19             break;
20         }
21     if (i == FD_SETSIZE) /* Couldn't find an empty slot */
22         app_error("add_client error: Too many clients");
23 }
```

code/conc/echoservers.c

Figure 12.10 `add_client` adds a new client connection to the pool.

code/conc/echoservers.c

```
1  void check_clients(pool *p)
2  {
3      int i, connfd, n;
4      char buf[MAXLINE];
5      rio_t rio;
6
7      for (i = 0; (i <= p->maxi) && (p->nready > 0); i++) {
8          connfd = p->clientfd[i];
9          rio = p->clientrio[i];
10
11         /* If the descriptor is ready, echo a text line from it */
12         if ((connfd > 0) && (FD_ISSET(connfd, &p->ready_set))) {
13             p->nready--;
14             if ((n = Rio_readlineb(&rio, buf, MAXLINE)) != 0) {
15                 byte_cnt += n;
16                 printf("Server received %d (%d total) bytes on fd %d\n",
17                        n, byte_cnt, connfd);
18                 Rio_writen(connfd, buf, n);
19             }
20
21             /* EOF detected, remove descriptor from pool */
22             else {
23                 Close(connfd);
24                 FD_CLR(connfd, &p->read_set);
25                 p->clientfd[i] = -1;
26             }
27         }
28     }
29 }
```

code/conc/echoservers.c

Figure 12.11 `check_clients` services ready client connections.

In terms of the finite state model in Figure 12.7, the `select` function detects input events, and the `add_client` function creates a new logical flow (state machine). The `check_clients` function performs state transitions by echoing input lines, and it also deletes the state machine when the client has finished sending text lines.

Practice Problem 12.4 (solution page 1072)

In the server in Figure 12.8, `pool.nready` is reinitialized with the value obtained from the call to `select`. Why?

Aside **E** **t** **ri** **W** **r** **r**

Despite the disadvantages outlined in Section 12.2.2, modern high-performance servers such as Node.js, nginx, and Tornado use event-driven programming based on I/O multiplexing, mainly because of the significant performance advantage compared to processes and threads.

12.2.2 Pros and Cons of I/O Multiplexing

The server in Figure 12.8 provides a nice example of the advantages and disadvantages of event-driven programming based on I/O multiplexing. One advantage is that event-driven designs give programmers more control over the behavior of their programs than process-based designs. For example, we can imagine writing an event-driven concurrent server that gives preferred service to some clients, which would be difficult for a concurrent server based on processes.

Another advantage is that an event-driven server based on I/O multiplexing runs in the context of a single process, and thus every logical flow has access to the entire address space of the process. This makes it easy to share data between flows. A related advantage of running as a single process is that you can debug your concurrent server as you would any sequential program, using a familiar debugging tool such as GDB. Finally, event-driven designs are often significantly more efficient than process-based designs because they do not require a process context switch to schedule a new flow.

A significant disadvantage of event-driven designs is coding complexity. Our event-driven concurrent echo server requires three times more code than the process-based server. Unfortunately, the complexity increases as the granularity of the concurrency decreases. By granularity, we mean the number of instructions that each logical flow executes per time slice. For instance, in our example concurrent server, the granularity of concurrency is the number of instructions required to read an entire text line. As long as some logical flow is busy reading a text line, no other logical flow can make progress. This is fine for our example, but it makes our event-driven server vulnerable to a malicious client that sends only a partial text line and then halts. Modifying an event-driven server to handle partial text lines is a nontrivial task, but it is handled cleanly and automatically by a process-based design. Another significant disadvantage of event-based designs is that they cannot fully utilize multi-core processors.

12.3 Concurrent Programming with Threads

To this point, we have looked at two approaches for creating concurrent logical flows. With the first approach, we use a separate process for each flow. The kernel schedules each process automatically, and each process has its own private address space, which makes it difficult for flows to share data. With the second approach, we create our own logical flows and use I/O multiplexing to explicitly schedule the flows. Because there is only one process, flows share the entire address space.

with a process form a pool of peers, independent of which threads were created by which other threads. The main thread is distinguished from other threads only in the sense that it is always the first thread to run in the process. The main impact of this notion of a pool of peers is that a thread can kill any of its peers or wait for any of its peers to terminate. Further, each peer can read and write the same shared data.

12.3.2 Posix Threads

Posix threads (Pthreads) is a standard interface for manipulating threads from C programs. It was adopted in 1995 and is available on all Linux systems. Pthreads defines about 60 functions that allow programs to create, kill, and reap threads, to share data safely with peer threads, and to notify peers about changes in the system state.

Figure 12.13 shows a simple Pthreads program. The main thread creates a peer thread and then waits for it to terminate. The peer thread prints `Hello, world!\n` and terminates. When the main thread detects that the peer thread has terminated, it terminates the process by calling `exit`. This is the first threaded program we have seen, so let us dissect it carefully. The code and local data for a thread are encapsulated in a thread routine. As shown by the prototype in line 2, each thread routine takes as input a single generic pointer and returns a generic pointer. If you want to pass multiple arguments to a thread routine, then you should put the arguments into a structure and pass a pointer to the structure. Similarly, if you

code/conc/hello.c

```
1  #include "csapp.h"
2  void *thread(void *vargp);
3
4  int main()
5  {
6      pthread_t tid;
7      Pthread_create(&tid, NULL, thread, NULL);
8      Pthread_join(tid, NULL);
9      exit(0);
10 }10
```

want the thread routine to return multiple arguments, you can return a pointer to a structure.

Line 4 marks the beginning of the code for the main thread. The main thread declares a single local variable `tid`, which will be used to store the thread ID of the peer thread (line 6). The main thread creates a new peer thread by calling the `pthread_create` function (line 7). When the call to `pthread_create` returns, the main thread and the newly created peer thread are running concurrently, and `tid` contains the ID of the new thread. The main thread waits for the peer thread to terminate with the call to `pthread_join` in line 8. Finally, the main thread calls `exit` (line 9), which terminates all threads (in this case, just the main thread) currently running in the process.

Lines 12–16 define the thread routine for the peer thread. It simply prints a string and then terminates the peer thread by executing the `return` statement in line 15.

12.3.3 Creating Threads

Threads create other threads by calling the `pthread_create` function.

```
#include <pthread.h>
typedef void *(func)(void *);

int pthread_create(pthread_t *tid, pthread_attr_t *attr,
                   func *f, void *arg);
                                        Returns: 0 if OK, nonzero on error
```

The `pthread_create` function creates a new thread and runs the thread routine `f` in the context of the new thread and with an input argument of `arg`. The `attr` argument can be used to change the default attributes of the newly created thread. Changing these attributes is beyond our scope, and in our examples, we will always call `pthread_create` with a NULL `attr` argument.

When `pthread_create` returns, argument `tid` contains the ID of the newly created thread. The new thread can determine its own thread ID by calling the `pthread_self` function.

```
#include <pthread.h>

pthread_t pthread_self(void);
                                        Returns: thread ID of caller
```

12.3.4 Terminating Threads

A thread terminates in one of the following ways:

- The thread terminates implicitly when its top-level thread routine returns.

- The thread terminates explicitly by calling the `pthread_exit` function. If the main thread calls `pthread_exit`, it waits for all other peer threads to terminate and then terminates the main thread and the entire process with a return value of `thread_return`.

```
#include <pthread.h>

void pthread_exit(void *thread_return);
                                                    Never returns
```

- Some peer thread calls the Linux `exit` function, which terminates the process and all threads associated with the process.
- Another peer thread terminates the current thread by calling the `pthread_cancel` function with the ID of the current thread.

```
#include <pthread.h>

int pthread_cancel(pthread_t tid);
                                        Returns: 0 if OK, nonzero on error
```

12.3.5 Reaping Terminated Threads

Threads wait for other threads to terminate by calling the `pthread_join` function.

```
#include <pthread.h>

int pthread_join(pthread_t tid, void **thread_return);
                                        Returns: 0 if OK, nonzero on error
```

The `pthread_join` function blocks until thread `tid` terminates, assigns the generic `(void *)` pointer returned by the thread routine to the location pointed to by `thread_return`, and then reaps any memory resources held by the terminated thread.

Notice that, unlike the Linux `wait`

be reaped or killed by other threads. Its memory resources are freed automatically by the system when it terminates.

By default, threads are created joinable. In order to avoid memory leaks, each joinable thread should be either explicitly reaped by another thread or detached by a call to the `pthread_detach` function.

```
#include <pthread.h>

int pthread_detach(pthread_t tid);
```

Returns: 0 if OK, nonzero on error

The `pthread_detach` function detaches the joinable thread `tid`. Threads can detach themselves by calling `pthread_detach` with an argument of `pthread_self()`.

Although some of our examples will use joinable threads, there are good reasons to use detached threads in real programs. For example, a high-performance Web server might create a new peer thread each time it receives a connection request from a Web browser. Since each connection is handled independently by a separate thread, it is unnecessary—and indeed undesirable—for the server to explicitly wait for each peer thread to terminate. In this case, each peer thread should detach itself before it begins processing the request so that its memory resources can be reclaimed after it terminates.

12.3.7 Initializing Threads

The `pthread_once` function allows you to initialize the state associated with a thread routine.

```
#include <pthread.h>

pthread_once_t once_control = PTHREAD_ONCE_INIT;

int pthread_once(pthread_once_t *once_control,
                 void (*init_routine)(void));
```

Always returns 0

The `once_control` variable is a global or static variable that is always initialized to PTHREAD_ONCE_INIT. The first time you call `pthread_once` with an argument of `once_control`, it invokes `init_routine`, which is a function with no input arguments that returns nothing. Subsequent calls to `pthread_once` with the same `once_control` variable do nothing. The `pthread_once` function is useful whenever you need to dynamically initialize global variables that are shared by multiple threads. We will look at an example in Section 12.5.5.

12.3.8 A Concurrent Server Based on Threads

Figure 12.14 shows the code for a concurrent echo server based on threads. The overall structure is similar to the process-based design. The main thread repeatedly waits for a connection request and then creates a peer thread to handle the request. While the code looks simple, there are a couple of general and somewhat subtle issues we need to look at more closely. The first issue is how to pass

code/conc/echoservert.c

```
1   #include "csapp.h"
2
3   void echo(int connfd);
4   void *thread(void *vargp);
5
6   int main(int argc, char **argv)
7   {
8       int listenfd, *connfdp;
9       socklen_t clientlen;
10      struct sockaddr_storage clientaddr;
11      pthread_t tid;
12
13      if (argc != 2) {
14          fprintf(stderr, "usage: %s <port>\n", argv[0]);
15          exit(0);
16      }
17      listenfd = Open_listenfd(argv[1]);
18
19      while (1) {
20          clientlen=sizeof(struct sockaddr_storage);
21          connfdp = Malloc(sizeof(int));
22          *connfdp = Accept(listenfd, (SA *) &clientaddr, &clientlen);
23          Pthread_create(&tid, NULL, thread, connfdp);
24      }
25  }
26
27  /* Thread routine */
28  void *thread(void *vargp)
29  {
30      int connfd = *((int *)vargp);
31      Pthread_detach(pthread_self());
32      Free(vargp);
33      echo(connfd);
34      Close(connfd);
35      return NULL;
36  }
```

code/conc/echoservert.c

Figure 12.14 Concurrent echo server based on threads.

the connected descriptor to the peer thread when we call pthread_create. The obvious approach is to pass a pointer to the descriptor, as in the following:

```
connfd = Accept(listenfd, (SA *) &clientaddr, &clientlen);
Pthread_create(&tid, NULL, thread, &connfd);
```

Then we have the peer thread dereference the pointer and assign it to a local variable, as follows:

```
void *thread(void *vargp) {
    int connfd = *((int *)vargp);
    .
    .
    .
}
```

This would be wrong, however, because it introduces a race between the assignment statement in the peer thread and the accept statement in the main thread. If the assignment statement completes before the next accept, then the local connfd variable in the peer thread gets the correct descriptor value. However, if the assignment completes after the accept, then the local connfd variable in the peer thread gets the descriptor number of the next connection. The unhappy result is that two threads are now performing input and output on the same descriptor. In order to avoid the potentially deadly race, we must assign each connected descriptor returned by accept to its own dynamically allocated memory block, as shown in lines 21–22. We will return to the issue of races in Section 12.7.4.

Another issue is avoiding memory leaks in the thread routine. Since we are not explicitly reaping threads, we must detach each thread so that its memory resources will be reclaimed when it terminates (line 31). Further, we must be careful to free the memory block that was allocated by the main thread (line 32).

Practice Problem 12.5 (solution page 1072)

In the process-based server in Figure 12.5, we observed that there is no memory leak and the code remains correct even when line 33 is deleted. In the threads-based server in Figure 12.14, are there any chances of memory leak if lines 31 or 32 are deleted. Why?

12.4 Shared Variables in Threaded Programs

From a programmer's perspective, one of the attractive aspects of threads is the ease with which multiple threads can share the same program variables. However, this sharing can be tricky. In order to write correctly threaded programs, we must have a clear understanding of what we mean by sharing and how it works.

There are some basic questions to work through in order to understand whether a variable in a C program is shared or not: (1) What is the underlying memory model for threads? (2) Given this model, how are instances of the variable mapped to memory? (3) Finally, how many threads reference each of these

code/conc/sharing.c

```
1   #include "csapp.h"
2   #define N 2
3   void *thread(void *vargp);
4
5   char **ptr;  /* Global variable */
6
7   int main()
8   {
9       int i;
10      pthread_t tid;
11      char *msgs[N] = {
12          "Hello from foo",
13          "Hello from bar"
14      };
15
16      ptr = msgs;
17      for (i = 0; i < N; i++)
18          Pthread_create(&tid, NULL, thread, (void *)i);
19      Pthread_exit(NULL);
20  }
21
22  void *thread(void *vargp)
23  {
24      int myid = (int)vargp;
25      static int cnt = 0;
26      printf("[%d]: %s (cnt=%d)\n", myid, ptr[myid], ++cnt);
27      return NULL;
28  }
```

code/conc/sharing.c

Figure 12.15 Example program that illustrates different aspects of sharing.

instances? The variable is shared

program counter, condition codes, and general-purpose register values. Each thread shares the rest of the process context with the other threads. This includes the entire user virtual address space, which consists of read-only text (code), read/write data, the heap, and any shared library code and data areas. The threads also share the same set of open files.

In an operational sense, it is impossible for one thread to read or write the register values of another thread. On the other hand, any thread can access any location in the shared virtual memory. If some thread modifies a memory location, then every other thread will eventually see the change if it reads that location. Thus, registers are never shared, whereas virtual memory is always shared.

The memory model for the separate thread stacks is not as clean. These stacks are contained in the stack area of the virtual address space and are usually accessed independently by their respective threads. We say usually rather than always, because different thread stacks are not protected from other threads. So if a thread somehow manages to acquire a pointer to another thread's stack, then it can read and write any part of that stack. Our example program shows this in line 26, where the peer threads reference the contents of the main thread's stack indirectly through the global `ptr` variable.

12.4.2 Mapping Variables to Memory

Variables in threaded C programs are mapped to virtual memory according to their storage classes:

Global variables. A global variable is any variable declared outside of a function. At run time, the read/write area of virtual memory contains exactly one instance of each global variable that can be referenced by any thread. For example, the global `ptr` variable declared in line 5 has one run-time instance in the read/write area of virtual memory. When there is only one instance of a variable, we will denote the instance by simply using the variable name—in this case, `ptr`.

Local automatic variables. A local automatic variable is one that is declared inside a function without the `static` attribute. At run time, each thread's stack contains its own instances of any local automatic variables. This is true even if multiple threads execute the same thread routine. For example, there is one instance of the local variable `tid`, and it resides on the stack of the main thread. We will denote this instance as `tid.m`. As another example, there are two instances of the local variable `myid`, one instance on the stack of peer thread 0 and the other on the stack of peer thread 1. We will denote these instances as `myid.p0` and `myid.p1`, respectively.

Local static variables. A local static variable is one that is declared inside a function with the `static` attribute. As with global variables, the read/write area of virtual memory contains exactly one instance of each local static

variable declared in a program. For example, even though each peer thread in our example program declares `cnt` in line 25, at run time there is only one instance of `cnt` residing in the read/write area of virtual memory. Each peer thread reads and writes this instance.

12.4.3 Shared Variables

We say that a variable `v` is shared if and only if one of its instances is referenced by more than one thread. For example, variable `cnt` in our example program is shared because it has only one run-time instance and this instance is referenced by both peer threads. On the other hand, `myid` is not shared, because each of its two instances is referenced by exactly one thread. However, it is important to realize that local automatic variables such as `msgs` can also be shared.

Practice Problem 12.6 (solution page 1072)

A. Using the analysis from Section 12.4, fill each entry in the following table with "Yes" or "No" for the example program in Figure 12.15. In the first column, the notation `v.t` denotes an instance of variable `v` residing on the local stack for thread `t`, where `t` is either `m` (main thread), `p0` (peer thread 0), or `p1` (peer thread 1).

Variable instance	Referenced by		
	main thread?	peer thread 0?	peer thread 1?
`ptr`			
`cnt`			
`i.m`			
`msgs.m`			
`myid.p0`			
`myid.p1`			

B. Given the analysis in part A, which of the variables `ptr`, `cnt`, `i`, `msgs`, and `myid` are shared?

12.5 Synchronizing Threads with Semaphores

Shared variables can be convenient, but they introduce the possibility of nasty synchronization errors. Consider the `badcnt.c` program in Figure 12.16, which creates two threads, each of which increments a global shared counter variable called `cnt`.

Since each thread increments the counter `niters` times, we expect its final value to be 2 × `niters`. This seems quite simple and straightforward. However, when we run `badcnt.c` on our Linux system, we not only get wrong answers, we get different answers each time!

code/conc/badcnt.c

```
1   /* WARNING: This code is buggy! */
2   #include "csapp.h"
3
4   void *thread(void *vargp);  /* Thread routine prototype */
5
6   /* Global shared variable */
7   volatile long cnt = 0; /* Counter */
8
9   int main(int argc, char **argv)
10  {
11      long niters;
12      pthread_t tid1, tid2;
13
14      /* Check input argument */
15      if (argc != 2) {
16          printf("usage: %s <niters>\n", argv[0]);
17          exit(0);
18      }
19      niters = atoi(argv[1]);
20
21      /* Create threads and wait for them to finish */
22      Pthread_create(&tid1, NULL, thread, &niters);
23      Pthread_create(&tid2, NULL, thread, &niters);
24      Pthread_join(tid1, NULL);
25      Pthread_join(tid2, NULL);
26
27      /* Check result */
28      if (cnt != (2 * niters))
29          printf("BOOM! cnt=%ld\n", cnt);
30      else
31          printf("OK cnt=%ld\n", cnt);
32      exit(0);
33  }
34
35  /* Thread routine */
36  void *thread(void *vargp)
37  {
38      long i, niters = *((long *)vargp);
39
40      for (i = 0; i < niters; i++)
41          cnt++;
42
43      return NULL;
44  }
```

code/conc/badcnt.c

Figure 12.16 badcnt.c: An improperly synchronized counter program.

```
linux>  ./badcnt 1000000
BOOM! cnt=1445085

linux>  ./badcnt 1000000
BOOM! cnt=1915220

linux>  ./badcnt 1000000
BOOM! cnt=1404746
```

So what went wrong? To understand the problem clearly, we need to study the assembly code for the counter loop (lines 40–41), as shown in Figure 12.17. We will find it helpful to partition the loop code for thread i into five parts:

H_i: The block of instructions at the head of the loop

L_i: The instruction that loads the shared variable `cnt` into the accumulator register $\%rdx_i$, where $\%rdx_i$ denotes the value of register `%rdx` in thread i

U_i: The instruction that updates (increments) $\%rdx_i$

S_i: The instruction that stores the updated value of $\%rdx_i$ back to the shared variable `cnt`

T_i: The block of instructions at the tail of the loop

Notice that the head and tail manipulate only local stack variables, while L_i, U_i, and S_i manipulate the contents of the shared counter variable.

When the two peer threads in `badcnt.c` run concurrently on a uniprocessor, the machine instructions are completed one after the other in some order. Thus, each concurrent execution defines some total ordering (or interleaving) of the instructions in the two threads. Unfortunately, some of these orderings will produce correct results, but others will not.

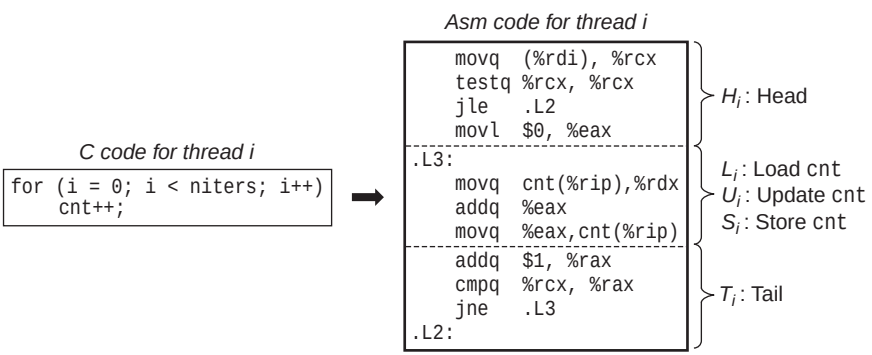


Figure 12.17 Assembly code for the counter loop (lines 40–41) in `badcnt.c`.

(a) Correct ordering

Step	Thread	Instr.	%rdx$_1$	%rdx$_2$	cnt
1	1	H_1	—	—	0
2	1	L_1	0	—	0
3	1	U_1	1	—	0
4	1	S_1	1	—	1
5	2	H_2	—	—	1
6	2	L_2	—	1	1
7	2	U_2	—	2	1
8	2	S_2	—	2	2
9	2	T_2	—	2	2
10	1	T_1	1	—	2

(b) Incorrect ordering

Step	Thread	Instr.	%rdx$_1$	%rdx$_2$	cnt
1	1	H_1	—	—	0
2	1	L_1	0	—	0
3	1	U_1	1	—	0
4	2	H_2	—	—	0
5	2	L_2	—	0	0
6	1	S_1	1	—	1
7	1	T_1	1	—	1
8	2	U_2	—	1	1
9	2	S_2	—	1	1
10	2	T_2	—	1	1

Figure 12.18 Instruction orderings for the first loop iteration in `badcnt.c`.

Here is the crucial point: In general, there is no way for you to predict whether the operating system will choose a correct ordering for your threads. For example, Figure 12.18(a) shows the step-by-step operation of a correct instruction ordering. After each thread has updated the shared variable `cnt`, its value in memory is 2, which is the expected result.

On the other hand, the ordering in Figure 12.18(b) produces an incorrect value for `cnt`. The problem occurs because thread 2 loads `cnt` in step 5, after thread 1 loads `cnt` in step 2 but before thread 1 stores its updated value in step 6. Thus, each thread ends up storing an updated counter value of 1. We can clarify these notions of correct and incorrect instruction orderings with the help of a device known as a progress graph, which we introduce in the next section.

Practice Problem 12.7 (solution page 1073)

Complete the table for the following instruction ordering of `badcnt.c`:

Step	Thread	Instr.	%rdx$_1$	%rdx$_2$	cnt
1	1	H_1	—	—	0
2	1	L_1			
3	2	H_2			
4	2	L_2			
5	2	U_2			
6	2	S_2			
7	1	U_1			
Step	Thread	Instr.	%rdx$_1$	%rdx$_2$	cnt
8	1	S_1			
9	1	T_1			

10 2 T_2 _____ _____ _____

Does this ordering result in a correct value for `cnt`?

12.5.1 Progress Graphs

A *progress graph* models the execution of n concurrent threads as a trajectory through an n-dimensional Cartesian space. Each axis k corresponds to the progress of thread k. Each point $(I_1, I_2, \ldots, I_n)$ represents the state where thread k ($k = 1, \ldots, n$) has completed instruction I_k. The origin of the graph corresponds to the *initial state* where none of the threads has yet completed an instruction.

Figure 12.19 shows the two-dimensional progress graph for the first loop iteration of the `badcnt.c` program. The horizontal axis corresponds to thread 1, the vertical axis to thread 2. Point (L_1, S_2) corresponds to the state where thread 1 has completed L_1 and thread 2 has completed S_2.

A progress graph models instruction execution as a *transition* from one state to another. A transition is represented as a directed edge from one point to an adjacent point. Legal transitions move to the right (an instruction in thread 1 completes) or up (an instruction in thread 2 completes). Two instructions cannot complete at the same time—diagonal transitions are not allowed. Programs never run backward so transitions that move down or to the left are not legal either.

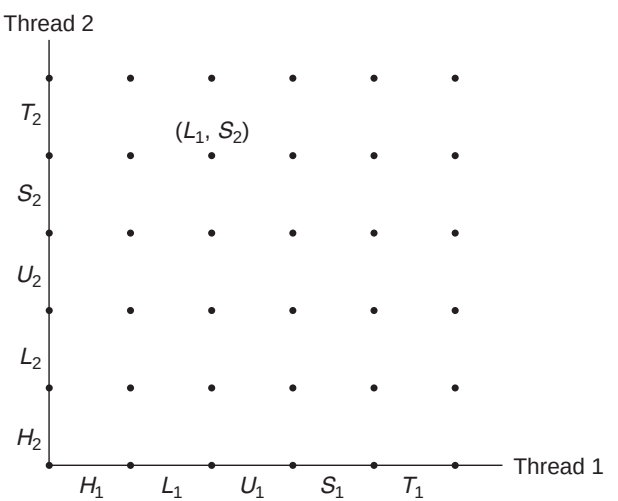
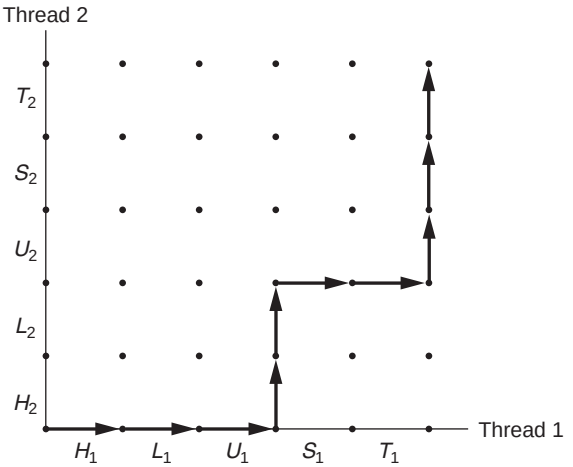


Figure 12.19
Progress graph for the first loop iteration of `badcnt.c`.

Figure 12.20
An example trajectory.



The execution history of a program is modeled as a trajectory through the state space. Figure 12.20 shows the trajectory that corresponds to the following instruction ordering:

$$H_1, L_1, U_1, H_2, L_2, S_1, T_1, U_2, S_2, T_2$$

For thread i, the instructions (L_i, U_i, S_i) that manipulate the contents of the shared variable `cnt` constitute a critical section (with respect to shared variable `cnt`) that should not be interleaved with the critical section of the other thread. In other words, we want to ensure that each thread has mutually exclusive access to the shared variable while it is executing the instructions in its critical section. The phenomenon in general is known as mutual exclusion.

On the progress graph, the intersection of the two critical sections defines a region of the state space known as an unsafe region. Figure 12.21 shows the unsafe region for the variable `cnt`. Notice that the unsafe region abuts, but does not include, the states along its perimeter. For example, states (H_1, H_2) and (S_1, U_2) abut the unsafe region, but they are not part of it. A trajectory that skirts the unsafe region is known as a safe trajectory. Conversely, a trajectory that touches any part of the unsafe region is an unsafe trajectory. Figure 12.21 shows examples of safe and unsafe trajectories through the state space of our example `badcnt.c` program. The upper trajectory skirts the unsafe region along its left and top sides, and thus is safe. The lower trajectory crosses the unsafe region, and thus is unsafe.

Any safe trajectory will correctly update the shared counter. In order to guarantee correct execution of our example threaded program—and indeed any concurrent program that shares global data structures—we must somehow synchronize the threads so that they always have a safe trajectory. A classic approach is based on the idea of a semaphore, which we introduce next.

Aside Origin of P and V

Edsger Dijkstra (1930–2002) was originally from the Netherlands. The names P and V come from the Dutch words proberen (to test) and verhogen (to increment).

The test and decrement operations in P occur indivisibly, in the sense that once the semaphore s becomes nonzero, the decrement of s occurs without interruption. The increment operation in V also occurs indivisibly, in that it loads, increments, and stores the semaphore without interruption. Notice that the definition of V does not define the order in which waiting threads are restarted. The only requirement is that the V must restart exactly one waiting thread. Thus, when several threads are waiting at a semaphore, you cannot predict which one will be restarted as a result of the V.

The definitions of P and V ensure that a running program can never enter a state where a properly initialized semaphore has a negative value. This property, known as the semaphore invariant, provides a powerful tool for controlling the trajectories of concurrent programs, as we shall see in the next section.

The Posix standard defines a variety of functions for manipulating semaphores.

```
#include <semaphore.h>

int sem_init(sem_t *sem, 0, unsigned int value);
int sem_wait(sem_t *s);   /* P(s) */
int sem_post(sem_t *s);   /* V(s) */
```

Returns: 0 if OK, −1 on error

The `sem_init` function initializes semaphore `sem` to `value`. Each semaphore must be initialized before it can be used. For our purposes, the middle argument is always 0. Programs perform P and V operations by calling the `sem_wait` and `sem_post` functions, respectively. For conciseness, we prefer to use the following equivalent P and V wrapper functions instead:

```
#include "csapp.h"

void P(sem_t *s);   /* Wrapper function for sem_wait */
void V(sem_t *s);   /* Wrapper function for sem_post */
```

Returns: nothing

12.5.3 Using Semaphores for Mutual Exclusion

Semaphores provide a convenient way to ensure mutually exclusive access to shared variables. The basic idea is to associate a semaphore s, initially 1, with

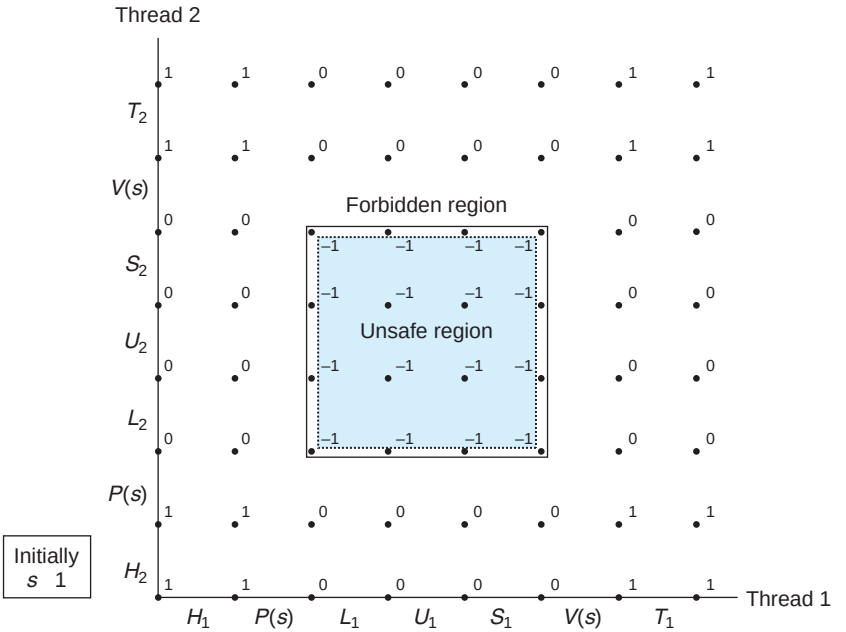


Figure 12.22 Using semaphores for mutual exclusion. T i a i l tat r s < 0 fi a forbidden region t at rr t a r i a r t a a i l tra t r r t i t a r i .

each shared variable (or related set of shared variables) and then surround the corresponding critical section with P (s) and V (s) operations.

A semaphore that is used in this way to protect shared variables is called a binary semaphore because its value is always 0 or 1. Binary semaphores whose purpose is to provide mutual exclusion are often called mutexes. Performing a P operation on a mutex is called locking the mutex. Similarly, performing the V operation is called unlocking the mutex. A thread that has locked but not yet unlocked a mutex is said to be holding the mutex. A semaphore that is used as a counter for a set of available resources is called a counting semaphore.

The progress graph in Figure 12.22 shows how we would use binary semaphores to properly synchronize our example counter program.

Each state is labeled with the value of semaphore s in that state. The crucial idea is that this combination of P and V operations creates a collection of states, called a forbidden region, where s < 0. Because of the semaphore invariant, no feasible trajectory can include one of the states in the forbidden region. And since the forbidden region completely encloses the unsafe region, no feasible trajectory can touch any part of the unsafe region. Thus, every feasible trajectory is safe, and regardless of the ordering of the instructions at run time, the program correctly increments the counter.

Aside Limitations of progress graphs

Progress graphs give us a nice way to visualize concurrent program execution on uniprocessors and to

/*Semaphore that protects counter */ sem_t mutex; Sem_init(&mutex, 0, 1);

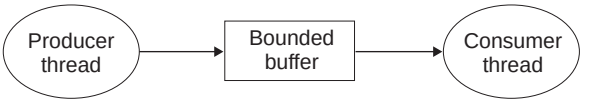


Figure 12.23 Producer-consumer problem. T r rat t a i rt t i t a o r. T o r r o it r t r a t o t .

operation to notify another thread that some condition in the program state has become true. Two classical and useful examples are the producer-consumer and readers-writers problems.

Pr r-C r Pr l

The producer-consumer problem is shown in Figure 12.23. A producer and consumer thread share a bounded buffer with n slots. The producer thread repeatedly produces new items and inserts them in the buffer. The consumer thread repeatedly removes items from the buffer and then consumes (uses) them. Variants with multiple producers and consumers are also possible.

Since inserting and removing items involves updating shared variables, we must guarantee mutually exclusive access to the buffer. But guaranteeing mutual exclusion is not sufficient. We also need to schedule accesses to the buffer. If the buffer is full (there are no empty slots), then the producer must wait until a slot becomes available. Similarly, if the buffer is empty (there are no available items), then the consumer must wait until an item becomes available.

Producer-consumer interactions occur frequently in real systems. For example, in a multimedia system, the producer might encode video frames while the consumer decodes and renders them on the screen. The purpose of the buffer is to reduce jitter in the video stream caused by data-dependent differences in the encoding and decoding times for individual frames. The buffer provides a reservoir of slots to the producer and a reservoir of encoded frames to the consumer. Another common example is the design of graphical user interfaces. The producer detects mouse and keyboard events and inserts them in the buffer. The consumer removes the events from the buffer in some priority-based manner and paints the screen.

In this section, we will develop a simple package, called SBUF, for building producer-consumer programs. In the next section, we look at how to use it to build an interesting concurrent server based on prethreading. SBUF manipulates bounded buffers of type `sbuf_t` (Figure 12.24). Items are stored in a dynamically allocated integer array (`buf`) with `n` items. The `front` and `rear` indices keep track of the first and last items in the array. Three semaphores synchronize access to the buffer. The `mutex` semaphore provides mutually exclusive buffer access. Semaphores `slots` and `items` are counting semaphores that count the number of empty slots and available items, respectively.

code/conc/sbuf.h

```
1  typedef struct {
2      int *buf;          /* Buffer array */
3      int n;             /* Maximum number of slots */
4      int front;         /* buf[(front+1)%n] is first item */
5      int rear;          /* buf[rear%n] is last item */
6      sem_t mutex;       /* Protects accesses to buf */
7      sem_t slots;       /* Counts available slots */
8      sem_t items;       /* Counts available items */
9  } sbuf_t;
```

code/conc/sbuf.h

Figure 12.24 `sbuf_t`: Bounded buffer used by the Sbuf package.

Figure 12.25 shows the implementation of the SBUF package. The `sbuf_init` function allocates heap memory for the buffer, sets `front` and `rear` to indicate an empty buffer, and assigns initial values to the three semaphores. This function is called once, before calls to any of the other three functions. The `sbuf_deinit` function frees the buffer storage when the application is through using it. The `sbuf_insert` function waits for an available slot, locks the mutex, adds the item, unlocks the mutex, and then announces the availability of a new item. The `sbuf_remove` function is symmetric. After waiting for an available buffer item, it locks the mutex, removes the item from the front of the buffer, unlocks the mutex, and then signals the availability of a new slot.

Practice Problem 12.9 (solution page 1074)

Let `p` denote the number of producers, `c` the number of consumers, and `n` the buffer size in units of items. For each of the following scenarios, indicate whether the mutex semaphore in `sbuf_insert` and `sbuf_remove` is necessary or not.

A. $p = 1, c = 1, n > 1$

B. $p = 1, c = 1, n = 1$

C. $p > 1, c > 1, n = 1$

Readers-Writers Problem

The readers-writers problem is a generalization of the mutual exclusion problem. A collection of concurrent threads is accessing a shared object such as a data structure in main memory or a database on disk. Some threads only read the object, while others modify it. Threads that modify the object are called writers. Threads that only read it are called readers. Writers must have exclusive access to the object, but readers may share the object with an unlimited number of other readers. In general, there are an unbounded number of concurrent readers and writers.

code/conc/sbuf.c

```
#include "csapp.h"
#include "sbuf.h"

/* Create an empty, bounded, shared FIFO buffer with n slots */
void sbuf_init(sbuf_t *sp, int n)
{
    sp->buf = Calloc(n, sizeof(int));
    sp->n = n;                       /* Buffer holds max of n items */
    sp->front = sp->rear = 0;        /* Empty buffer iff front == rear */
    Sem_init(&sp->mutex, 0, 1);      /* Binary semaphore for locking */
    Sem_init(&sp->slots, 0, n);      /* Initially, buf has n empty slots */
    Sem_init(&sp->items, 0, 0);      /* Initially, buf has zero data items */
}

/* Clean up buffer sp */
void sbuf_deinit(sbuf_t *sp)
{
    Free(sp->buf);
}

/* Insert item onto the rear of shared buffer sp */
void sbuf_insert(sbuf_t *sp, int item)
{
    P(&sp->slots);                          /* Wait for available slot */
    P(&sp->mutex);                          /* Lock the buffer */
    sp->buf[(++sp->rear)%(sp->n)] = item;   /* Insert the item */
    V(&sp->mutex);                          /* Unlock the buffer */
    V(&sp->items);                          /* Announce available item */
}

/* Remove and return the first item from buffer sp */
int sbuf_remove(sbuf_t *sp)
{
    int item;
    P(&sp->items);                          /* Wait for available item */
    P(&sp->mutex);                          /* Lock the buffer */
    item = sp->buf[(++sp->front)%(sp->n)];  /* Remove the item */
    V(&sp->mutex);                          /* Unlock the buffer */
    V(&sp->slots);                          /* Announce available slot */
    return item;
}
```

code/conc/sbuf.c

Figure 12.25 Sbuf: A package for synchronizing concurrent access to bounded buffers.

Readers-writers interactions occur frequently in real systems. For example, in an online airline reservation system, an unlimited number of customers are allowed to concurrently inspect the seat assignments, but a customer who is booking a seat must have exclusive access to the database. As another example, in a multithreaded caching Web proxy, an unlimited number of threads can fetch existing pages from the shared page cache, but any thread that writes a new page to the cache must have exclusive access.

The readers-writers problem has several variations, each based on the priorities of readers and writers. The first readers-writers problem, which favors readers, requires that no reader be kept waiting unless a writer has already been granted permission to use the object. In other words, no reader should wait simply because a writer is waiting. The second readers-writers problem, which favors writers, requires that once a writer is ready to write, it performs its write as soon as possible. Unlike the first problem, a reader that arrives after a writer must wait, even if the writer is also waiting.

Figure 12.26 shows a solution to the first readers-writers problem. Like the solutions to many synchronization problems, it is subtle and deceptively simple. The `w` semaphore controls access to the critical sections that access the shared object. The `mutex` semaphore protects access to the shared `readcnt` variable, which counts the number of readers currently in the critical section. A writer locks the `w` mutex each time it enters the critical section and unlocks it each time it leaves. This guarantees that there is at most one writer in the critical section at any point in time. On the other hand, only the first reader to enter the critical section locks `w`, and only the last reader to leave the critical section unlocks it. The `w` mutex is ignored by readers who enter and leave while other readers are present. This means that as long as a single reader holds the `w` mutex, an unbounded number of readers can enter the critical section unimpeded.

A correct solution to either of the readers-writers problems can result in starvation, where a thread blocks indefinitely and fails to make progress. For example, in the solution in Figure 12.26, a writer could wait indefinitely while a stream of readers arrived.

Practice Problem 12.10 (solution page 1074)

The solution to the first readers-writers problem in Figure 12.26 gives priority to readers, but this priority is weak in the sense that a writer leaving its critical section might restart a waiting writer instead of a waiting reader. Describe a scenario where this weak priority would allow a collection of writers to starve a reader.

12.5.5 Putting It Together: A Concurrent Server Based on Prethreading

We have seen how semaphores can be used to access shared variables and to schedule accesses to shared resources. To help you understand these ideas more clearly, let us apply them to a concurrent server based on a technique called prethreading.

```
/* Global variables */
int readcnt;    /* Initially = 0 */
sem_t mutex, w; /* Both initially = 1 */

void reader(void)
{
    while (1) {
        P(&mutex);
        readcnt++;
        if (readcnt == 1) /* First in */
            P(&w);
        V(&mutex);

        /* Critical section */
        /* Reading happens  */

        P(&mutex);
        readcnt--;
        if (readcnt == 0) /* Last out */
            V(&w);
        V(&mutex);
    }
}

void writer(void)
{
    while (1) {
        P(&w);

        /* Critical section */
        /* Writing happens  */

        V(&w);
    }
}
```

Figure 12.26 Solution to the first readers-writers problem. Fa r r a r r it r .

In the concurrent server in Figure 12.14, we created a new thread for each new client. A disadvantage of this approach is that we incur the nontrivial cost of creating a new thread for each new client. A server based on prethreading tries to reduce this overhead by using the producer-consumer model shown in Figure 12.27. The server consists of a main thread and a set of worker threads. The main thread repeatedly accepts connection requests from clients and places

Aside Other synchronization techniques

We have shown you how to synchronize threads using semaphores, mainly because they are simple, classical, and have a clean semantic model. But you should know that other synchronization techniques exist as well. For example, Java threads are synchronized with a mechanism called a Java monitor [48], which provides a higher-level abstraction of the mutual exclusion and scheduling capabilities of semaphores; in fact, monitors can be implemented with semaphores. As another example, the Pthreads interface defines a set of synchronization operations on mutex and condition variables. Pthreads mutexes are used for mutual exclusion. Condition variables are used for scheduling accesses to shared resources, such as the bounded buffer in a producer-consumer program.

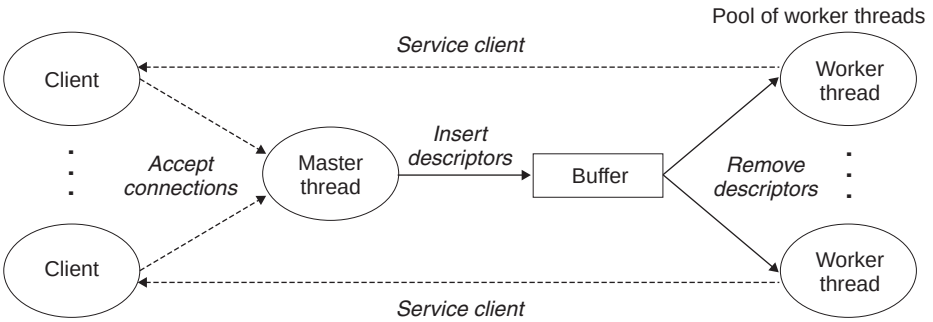


Figure 12.27 Organization of a prethreaded concurrent server. A [illegible]

the resulting connected descriptors in a bounded buffer. Each worker thread repeatedly removes a descriptor from the buffer, services the client, and then waits for the next descriptor.

Figure 12.28 shows how we would use the SBUF package to implement a prethreaded concurrent echo server. After initializing buffer `sbuf` (line 24), the main thread creates the set of worker threads (lines 25–26). Then it enters the infinite server loop, accepting connection requests and inserting the resulting connected descriptors in `sbuf`. Each worker thread has a very simple behavior. It waits until it is able to remove a connected descriptor from the buffer (line 39) and then calls the `echo_cnt` function to echo client input.

The `echo_cnt` function in Figure 12.29 is a version of the `echo` function from Figure 11.22 that records the cumulative number of bytes received from all clients in a global variable called `byte_cnt`. This is interesting code to study because it shows you a general technique for initializing packages that are called from thread routines. In our case, we need to initialize the `byte_cnt` counter and the `mutex` semaphore. One approach, which we used for the SBUF and RIO packages, is to require the main thread to explicitly call an initialization function. Another approach, shown here, uses the `pthread_once` function (line 19) to call

code/conc/echoservert-pre.c

```
1   #include "csapp.h"
2   #include "sbuf.h"
3   #define NTHREADS  4
4   #define SBUFSIZE  16
5
6   void echo_cnt(int connfd);
7   void *thread(void *vargp);
8
9   sbuf_t sbuf; /* Shared buffer of connected descriptors */
10
11  int main(int argc, char **argv)
12  {
13      int i, listenfd, connfd;
14      socklen_t clientlen;
15      struct sockaddr_storage clientaddr;
16      pthread_t tid;
17
18      if (argc != 2) {
19          fprintf(stderr, "usage: %s <port>\n", argv[0]);
20          exit(0);
21      }
22      listenfd = Open_listenfd(argv[1]);
23
24      sbuf_init(&sbuf, SBUFSIZE);
25      for (i = 0; i < NTHREADS; i++)  /* Create worker threads */
26          Pthread_create(&tid, NULL, thread, NULL);
27
28      while (1) {
29          clientlen = sizeof(struct sockaddr_storage);
30          connfd = Accept(listenfd, (SA *) &clientaddr, &clientlen);
31          sbuf_insert(&sbuf, connfd); /* Insert connfd in buffer */
32      }
33  }
34
35  void *thread(void *vargp)
36  {
37      Pthread_detach(pthread_self());
38      while (1) {
39          int connfd = sbuf_remove(&sbuf); /* Remove connfd from buffer */
40          echo_cnt(connfd);                /* Service client */
41          Close(connfd);
42      }
43  }
```

code/conc/echoservert-pre.c

Figure 12.28 A prethreaded concurrent echo server. T r r a r o r- o r o l i t o r o r a l ti l o r .

code/conc/echo-cnt.c

```
1   #include "csapp.h"
2
3   static int byte_cnt;  /* Byte counter */
4   static sem_t mutex;   /* and the mutex that protects it */
5
6   static void init_echo_cnt(void)
7   {
8       Sem_init(&mutex, 0, 1);
9       byte_cnt = 0;
10  }
11
12  void echo_cnt(int connfd)
13  {
14      int n;
15      char buf[MAXLINE];
16      rio_t rio;
17      static pthread_once_t once = PTHREAD_ONCE_INIT;
18
19      Pthread_once(&once, init_echo_cnt);
20      Rio_readinitb(&rio, connfd);
21      while((n = Rio_readlineb(&rio, buf, MAXLINE)) != 0) {
22          P(&mutex);
23          byte_cnt += n;
24          printf("server received %d (%d total) bytes on fd %d\n",
25                 n, byte_cnt, connfd);
26          V(&mutex);
27          Rio_writen(connfd, buf, n);
28      }
29  }
```

code/conc/echo-cnt.c

Figure 12.29 `echo_cnt`: A version of `echo` that counts all bytes received from clients.

the initialization function the first time some thread calls the `echo_cnt` function. The advantage of this approach is that it makes the package easier to use. The disadvantage is that every call to `echo_cnt` makes a call to `pthread_once`, which most times does nothing useful.

Once the package is initialized, the `echo_cnt` function initializes the Rio buffered I/O package (line 20) and then echoes each text line that is received from the client. Notice that the accesses to the shared `byte_cnt` variable in lines 23–25 are protected by P and V operations.

Aside Event-driven programming alternatives

I/O multiplexing is not the only way to write an event-driven program. For example, you might have noticed that the concurrent prethreaded server that we just developed is really an event-driven server with simple state machines for the main and worker threads. The main thread has two states ("waiting for connection request" and "waiting for available buffer slot"), two I/O events ("connection request arrives" and "buffer slot becomes available"), and two transitions ("accept connection request" and "insert buffer item"). Similarly, each worker thread has one state ("waiting for available buffer item"), one I/O event ("buffer item becomes available"), and one transition ("remove buffer item").

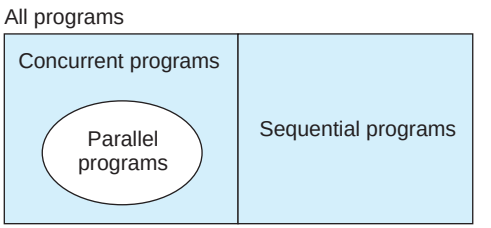


Figure 12.30 Relationships between the sets of sequential, concurrent, and parallel programs.

12.6 Using Threads for Parallelism

Thus far in our study of concurrency, we have assumed concurrent threads executing on uniprocessor systems. However, most modern machines have multi-core processors. Concurrent programs often run faster on such machines because the operating system kernel schedules the concurrent threads in parallel on multiple cores, rather than sequentially on a single core. Exploiting such parallelism is critically important in applications such as busy Web servers, database servers, and large scientific codes, and it is becoming increasingly useful in mainstream applications such as Web browsers, spreadsheets, and document processors.

Figure 12.30 shows the set relationships between sequential, concurrent, and parallel programs. The set of all programs can be partitioned into the disjoint sets of sequential and concurrent programs. A sequential program is written as a single logical flow. A concurrent program is written as multiple concurrent flows. A parallel program is a concurrent program running on multiple processors. Thus, the set of parallel programs is a proper subset of the set of concurrent programs.

A detailed treatment of parallel programs is beyond our scope, but studying a few simple example programs will help you understand some important aspects of parallel programming. For example, consider how we might sum the sequence of integers $0, \ldots, n-1$ in parallel. Of course, there is a closed-form solution for this particular problem, but nonetheless it is a concise and easy-to-understand exemplar that will allow us to make some interesting points about parallel programs.

The most straightforward approach for assigning work to different threads is to partition the sequence into t disjoint regions and then assign each of t different

threads to work on its own region. For simplicity, assume that n is a multiple of t, such that each region has n/t elements. Let's look at some of the different ways that multiple threads might work on their assigned regions in parallel.

The simplest and most straightforward option is to have the threads sum into a shared global variable that is protected by a mutex. Figure 12.31 shows how we might implement this. In lines 28–33, the main thread creates the peer threads and then waits for them to terminate. Notice that the main thread passes a small integer to each peer thread that serves as a unique thread ID. Each peer thread will use its thread ID to determine which portion of the sequence it should work on. This idea of passing a small unique thread ID to the peer threads is a general technique that is used in many parallel applications. After the peer threads have terminated, the global variable gsum contains the final sum. The main thread then uses the closed-form solution to verify the result (lines 36–37).

Figure 12.32 shows the function that each peer thread executes. In line 4, the thread extracts the thread ID from the thread argument and then uses this ID to determine the region of the sequence it should work on (lines 5–6). In lines 9–13, the thread iterates over its portion of the sequence, updating the shared global variable gsum on each iteration. Notice that we are careful to protect each update with P and V mutex operations.

When we run psum-mutex on a system with four cores on a sequence of size $n = 2^{31}$ and measure its running time (in seconds) as a function of the number of threads, we get a nasty surprise:

	Number of threads				
Version	1	2	4	8	16
psum-mutex	68	432	719	552	599

Not only is the program extremely slow when it runs sequentially as a single thread, it is nearly an order of magnitude slower when it runs in parallel as multiple threads. And the performance gets worse as we add more cores. The reason for this poor performance is that the synchronization operations (P and V) are very expensive relative to the cost of a single memory update. This highlights an important lesson about parallel programming: Synchronization overhead is expensive and should be avoided if possible. If it cannot be avoided, the overhead should be amortized by as much useful computation as possible.

One way to avoid synchronization in our example program is to have each peer thread compute its partial sum in a private variable that is not shared with any other thread, as shown in Figure 12.33. The main thread (not shown) defines a global array called psum, and each peer thread i accumulates its partial sum in psum[i]. Since we are careful to give each peer thread a unique memory location to update, it is not necessary to protect these updates with mutexes. The only necessary synchronization is that the main thread must wait for all of the children to finish. After the peer threads have terminated, the main thread sums up the elements of the psum vector to arrive at the final result.

code/conc/psum-mutex.c

```
1   #include "csapp.h"
2   #define MAXTHREADS 32
3
4   void *sum_mutex(void *vargp); /* Thread routine */
5
6   /* Global shared variables */
7   long gsum = 0;           /* Global sum */
8   long nelems_per_thread;  /* Number of elements to sum */
9   sem_t mutex;             /* Mutex to protect global sum */
10
11  int main(int argc, char **argv)
12  {
13      long i, nelems, log_nelems, nthreads, myid[MAXTHREADS];
14      pthread_t tid[MAXTHREADS];
15
16      /* Get input arguments */
17      if (argc != 3) {
18          printf("Usage: %s <nthreads> <log_nelems>\n", argv[0]);
19          exit(0);
20      }
21      nthreads = atoi(argv[1]);
22      log_nelems = atoi(argv[2]);
23      nelems = (1L << log_nelems);
24      nelems_per_thread = nelems / nthreads;
25      sem_init(&mutex, 0, 1);
26
27      /* Create peer threads and wait for them to finish */
28      for (i = 0; i < nthreads; i++) {
29          myid[i] = i;
30          Pthread_create(&tid[i], NULL, sum_mutex, &myid[i]);
31      }
32      for (i = 0; i < nthreads; i++)
33          Pthread_join(tid[i], NULL);
34
35      /* Check final answer */
36      if (gsum != (nelems * (nelems-1))/2)
37          printf("Error: result=%ld\n", gsum);
38
39      exit(0);
40  }
```

code/conc/psum-mutex.c

Figure 12.31 Main routine for psum-mutex. U ltil t r a t l t o a i t a ar l o al aria l r o t t a t .

code/conc/psum-mutex.c

```
1  /* Thread routine for psum-mutex.c */
2  void *sum_mutex(void *vargp)
3  {
4      long myid = *((long *)vargp);          /* Extract the thread ID */
5      long start = myid * nelems_per_thread; /* Start element index */
6      long end = start + nelems_per_thread;  /* End element index */
7      long i;
8
9      for (i = start; i < end; i++) {
10         P(&mutex);
11         gsum += i;
12         V(&mutex);
13     }
14     return NULL;
15 }
```

code/conc/psum-mutex.c

Figure 12.32 Thread routine for psum-mutex. Ea rt r â¶ i t o a ar\ l o al aria l r o t t\ a t .

code/conc/psum-array.c

```
1  /* Thread routine for psum-array.c */
2  void *sum_array(void *vargp)
3  {
4      long myid = *((long *)vargp);          /* Extract the thread ID */
5      long start = myid * nelems_per_thread; /* Start element index */
6      long end = start + nelems_per_thread;  /* End element index */
7      long i;
8
9      for (i = start; i < end; i++) {
10         psum[myid] += i;
11     }
12     return NULL;
13 }
```

code/conc/psum-array.c

Figure 12.33 Thread routine for psum-array. Ea rt r â¶ a lat it artial i a ri at arra l tt at i o t ar\ it a o t r rt r â¶.

When we run `psum-array` on our four-core system, we see that it runs orders of magnitude faster than `psum-mutex`:

	Number of threads				
Version	1	2	4	8	16
`psum-mutex`	68.00	432.00	719.00	552.00	599.00
`psum-array`	7.26	3.64	1.91	1.85	1.84

In Chapter 5, we learned how to use local variables to eliminate unnecessary memory references. Figure 12.34 shows how we can apply this principle by having each peer thread accumulate its partial sum into a local variable rather than a global variable. When we run `psum-local` on our four-core machine, we get another order-of-magnitude decrease in running time:

	Number of threads				
Version	1	2	4	8	16
`psum-mutex`	68.00	432.00	719.00	552.00	599.00
`psum-array`	7.26	3.64	1.91	1.85	1.84
`psum-local`	1.06	0.54	0.28	0.29	0.30

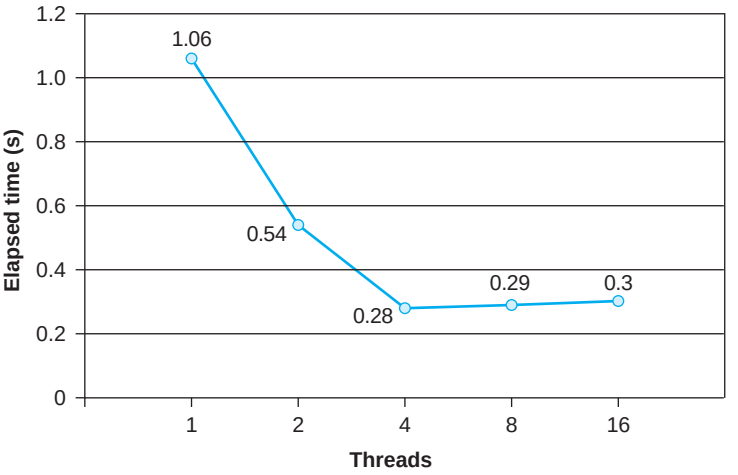
code/conc/psum-local.c

```
1   /* Thread routine for psum-local.c */
2   void *sum_local(void *vargp)
3   {
4       long myid = *((long *)vargp);          /* Extract the thread ID */
5       long start = myid * nelems_per_thread; /* Start element index */
6       long end = start + nelems_per_thread;  /* End element index */
7       long i, sum = 0;
8
9       for (i = start; i < end; i++) {
10          sum += i;
11      }
12      psum[myid] = sum;
13      return NULL;
14  }
```

code/conc/psum-local.c

Figure 12.34 Thread routine for `psum-local`. Each thread accumulates its partial sum in a local variable.

Figure 12.35
Performance of psum-local (Figure 12.34). S i a o 2^{31} l t i o r r o o r o r .



An important lesson to take away from this exercise is that writing parallel programs is tricky. Seemingly small changes to the code have a significant impact on performance.

C ara t ri i t P r o r a o Parall l Pr o ra

Figure 12.35 plots the total elapsed running time of the psum-local program in Figure 12.34 as a function of the number of threads. In each case, the program runs on a system with four processor cores and sums a sequence of $n = 2^{31}$ elements. We see that running time decreases as we increase the number of threads, up to four threads, at which point it levels off and even starts to increase a little.

In the ideal case, we would expect the running time to decrease linearly with the number of cores. That is, we would expect running time to drop by half each time we double the number of threads. This is indeed the case until we reach the point ($t > 4$) where each of the four cores is busy running at least one thread. Running time actually increases a bit as we increase the number of threads because of the overhead of context switching multiple threads on the same core. For this reason, parallel programs are often written so that each core runs exactly one thread.

Although absolute running time is the ultimate measure of any program's performance, there are some useful relative measures that can provide insight into how well a parallel program is exploiting potential parallelism. The speedup of a parallel program is typically defined as

$$S_p = \frac{T_1}{T_p}$$

where p is the number of processor cores and T_k is the running time on k cores. This formulation is sometimes referred to as strong scaling. When T_1 is the execution

Threads (t)	1	2	4	8	16
Cores (p)	1	2	4	4	4
Running time (T_p)	1.06	0.54	0.28	0.29	0.30
Speedup (S_p)	1	1.9	3.8	3.7	3.5
Efficiency (E_p)	100%	98%	95%	91%	88%

Figure 12.36 Speedup and parallel efficiency for the execution times in Figure 12.35.

time of a sequential version of the program, then S_p is called the absolute speedup. When T_1 is the execution time of the parallel version of the program running on one core, then S_p is called the relative speedup. Absolute speedup is a truer measure of the benefits of parallelism than relative speedup. Parallel programs often suffer from synchronization overheads, even when they run on one processor, and these overheads can artificially inflate the relative speedup numbers because they increase the size of the numerator. On the other hand, absolute speedup is more difficult to measure than relative speedup because measuring absolute speedup requires two different versions of the program. For complex parallel codes, creating a separate sequential version might not be feasible, either because the code is too complex or because the source code is not available.

A related measure, known as efficiency, is defined as

$$E_p = \frac{S_p}{p} = \frac{T_1}{pT_p}$$

and is typically reported as a percentage in the range (0, 100]. Efficiency is a measure of the overhead due to parallelization. Programs with high efficiency are spending more time doing useful work and less time synchronizing and communicating than programs with low efficiency.

Figure 12.36 shows the different speedup and efficiency measures for our example parallel sum program. Efficiencies over 90 percent such as these are very good, but do not be fooled. We were able to achieve high efficiency because our problem was trivially easy to parallelize. In practice, this is not usually the case. Parallel programming has been an active area of research for decades. With the advent of commodity multi-core machines whose core count is doubling every few years, parallel programming continues to be a deep, difficult, and active area of research.

There is another view of speedup, known as weak scaling, which increases the problem size along with the number of processors, such that the amount of work performed on each processor is held constant as the number of processors

Weak scaling is often a truer measure than strong scaling because it more accurately reflects our desire to use bigger machines to do more work. This is particularly true for scientific codes, where the problem size can be easily increased and where bigger problem sizes translate directly to better predictions of nature. However, there exist applications whose sizes are not so easily increased, and for these applications strong scaling is more appropriate. For example, the amount of work performed by real-time signal-processing applications is often determined by the properties of the physical sensors that are generating the signals. Changing the total amount of work requires using different physical sensors, which might not be feasible or necessary. For these applications, we typically want to use parallelism to accomplish a fixed amount of work as quickly as possible.

Practice Problem 12.11 (solution page 1074)

Fill in the blanks for the parallel program in the following table. Assume strong scaling.

Threads (t)	1	4	8
Cores (p)	1	4	8
Running time (T_p)	16	8	4
Speedup (S_p)	1	_____	_____
Efficiency (E_p)	100%	_____	_____

12.7 Other Concurrency Issues

You probably noticed that life got much more complicated once we were asked to synchronize accesses to shared data. So far, we have looked at techniques for mutual exclusion and producer-consumer synchronization, but this is only the tip of the iceberg. Synchronization is a fundamentally difficult problem that raises issues that simply do not arise in ordinary sequential programs. This section is a survey (by no means complete) of some of the issues you need to be aware of when you write concurrent programs. To keep things concrete, we will couch our discussion in terms of threads. Keep in mind, however, that these are typical of the issues that arise when concurrent flows of any kind manipulate shared resources.

12.7.1 Thread Safety

When we program with threads, we must be careful to write functions that have a property called thread safety. A function is said to be thread-safe if and only if it will always produce correct results when called repeatedly from multiple concurrent threads. If a function is not thread-safe, then we say it is thread-unsafe.

We can identify four (nondisjoint) classes of thread-unsafe functions:

Class 1: Functions that do not protect shared variables. We have already encountered this problem with the `thread` function in Figure 12.16, which

code/conc/rand.c

```
1  unsigned next_seed = 1;
2
3  /* rand - return pseudorandom integer in the range 0..32767 */
4  unsigned rand(void)
5  {
6      next_seed = next_seed*1103515245 + 12543;
7      return (unsigned)(next_seed>>16) % 32768;
8  }
9
10 /* srand - set the initial seed for rand() */
11 void srand(unsigned new_seed)
12 {
13     next_seed = new_seed;
14 }
```

code/conc/rand.c

Figure 12.37 A thread-unsafe pseudorandom number generator. (Ba [61])

increments an unprotected global counter variable. This class of thread-unsafe functions is relatively easy to make thread-safe: protect the shared variables with synchronization operations such as P and V. An advantage is that it does not require any changes in the calling program. A disadvantage is that the synchronization operations slow down the function.

Class 2: Functions that keep state across multiple invocations. A pseudorandom number generator is a simple example of this class of thread-unsafe functions. Consider the pseudorandom number generator package in Figure 12.37.

The `rand` function is thread-unsafe because the result of the current invocation depends on an intermediate result from the previous iteration. When we call `rand` repeatedly from a single thread after seeding it with a call to `srand`, we can expect a repeatable sequence of numbers. However, this assumption no longer holds if multiple threads are calling `rand`.

The only way to make a function such as `rand` thread-safe is to rewrite it so that it does not use any `static` data, relying instead on the caller to pass the state information in arguments. The disadvantage is that the programmer is now forced to change the code in the calling routine as well. In a large program where there are potentially hundreds of different call sites, making such modifications could be nontrivial and prone to error.

Class 3: Functions that return a pointer to a static variable. Some functions, such as `ctime` and `gethostbyname`, compute a result in a `static` variable and then return a pointer to that variable. If we call such functions from

code/conc/ctime-ts.c

```
1  char *ctime_ts(const time_t *timep, char *privatep)
2  {
3      char *sharedp;
4
5      P(&mutex);
6      sharedp = ctime(timep);
7      strcpy(privatep, sharedp); /* Copy string from shared to private */
8      V(&mutex);
9      return privatep;
10 }
```

code/conc/ctime-ts.c

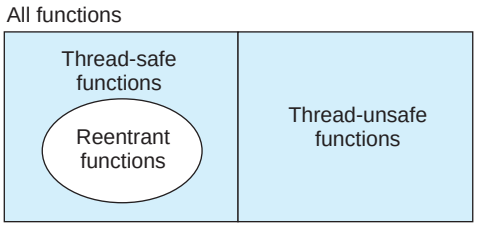
Figure 12.38 Thread-safe wrapper function for the C standard library `ctime` function. T i a l t l$_0$ -a\'$_0$- $_0$ t i t$_0$ all a la 3 t r a'$_0$- a ti$_0$.

concurrent threads, then disaster is likely, as results being used by one thread are silently overwritten by another thread.

There are two ways to deal with this class of thread-unsafe functions. One option is to rewrite the function so that the caller passes the address of the variable in which to store the results. This eliminates all shared data, but it requires the programmer to have access to the function source code.

If the thread-unsafe function is difficult or impossible to modify (e.g., the code is very complex or there is no source code available), then another option is to use the lock-and-copy technique. The basic idea is to associate a mutex with the thread-unsafe function. At each call site, lock the mutex, call the thread-unsafe function, copy the result returned by the function to a private memory location, and then unlock the mutex. To minimize changes to the caller, you should define a thread-safe wrapper function that performs the lock-and-copy and then replrnedcall casby eof thread-unsafe function with casbs to eof wrappegor exampley

Figure 12.39
Relationships between the sets of reentrant, thread-safe, and thread-unsafe functions.



code/conc/rand-r.c

```
1  /* rand_r - return a pseudorandom integer on 0..32767 */
2  int rand_r(unsigned int *nextp)
3  {
4      *nextp = *nextp * 1103515245 + 12345;
5      return (unsigned int)(*nextp / 65536) % 32768;
6  }
```

code/conc/rand-r.c

Figure 12.40 `rand_r`: A reentrant version of the `rand` function from Figure 12.37.

12.7.2 Reentrancy

There is an important class of thread-safe functions, known as reentrant functions, that are characterized by the property that they do not reference any shared data when they are called by multiple threads. Although the terms thread-safe and reentrant are sometimes used (incorrectly) as synonyms, there is a clear technical distinction that is worth preserving. Figure 12.39 shows the set relationships between reentrant, thread-safe, and thread-unsafe functions. The set of all functions is partitioned into the disjoint sets of thread-safe and thread-unsafe functions. The set of reentrant functions is a proper subset of the thread-safe functions.

Reentrant functions are typically more efficient than non-reentrant thread-safe functions because they require no synchronization operations. Furthermore, the only way to convert a class 2 thread-unsafe function into a thread-safe one is to rewrite it so that it is reentrant. For example, Figure 12.40 shows a reentrant version of the `rand` function from Figure 12.37. The key idea is that we have replaced the static `next` variable with a pointer that is passed in by the caller.

Is it possible to inspect the code of some function and declare a priori that it is reentrant? Unfortunately, it depends. If all function arguments are passed by value (i.e., no pointers) and all data references are to local automatic stack variables (i.e., no references to static or global variables), then the function is explicitly reentrant, in the sense that we can assert its reentrancy regardless of how it is called.

However, if we loosen our assumptions a bit and allow some parameters in our otherwise explicitly reentrant function to be passed by reference (i.e., we allow them to pass pointers), then we have an implicitly reentrant function, in the sense that it is only reentrant if the calling threads are careful to pass pointers

to nonshared data. For example, the `rand_r` function in Figure 12.40 is implicitly reentrant.

We always use the term reentrant to include both explicit and implicit reentrant functions. However, it is important to realize that reentrancy is sometimes a property of both the caller and the callee, and not just the callee alone.

Practice Problem 12.12 (solution page 1074)

The `rand_r` function in Figure 12.40 is implicitly reentrant. Explain.

12.7.3 Using Existing Library Functions in Threaded Programs

Most Linux functions, including the functions defined in the standard C library (such as `malloc`, `free`, `realloc`, `printf`, and `scanf`), are thread-safe, with only a few exceptions. Figure 12.41 lists some common exceptions. (See [110] for a complete list.) The `strtok` function is a deprecated function (one whose use is discouraged) for parsing strings. The `asctime`, `ctime`, and `localtime` functions are popular functions for converting back and forth between different time and date formats. The `gethostbyaddr`, `gethostbyname`, and `inet_ntoa` functions are obsolete network programming functions that have been replaced by the reentrant `getaddrinfo`, `getnameinfo`, and `inet_ntop` functions, respectively (see Chapter 11). With the exceptions of `rand` and `strtok`, they are of the class 3 variety that return a pointer to a static variable. If we need to call one of these functions in a threaded program, the least disruptive approach to the caller is to lock and copy. However, the lock-and-copy approach has a number of disadvantages. First, the additional synchronization slows down the program. Second, functions that return pointers to complex structures of structures require a deep copy of the structures in order to copy the entire structure hierarchy. Third, the lock-and-copy approach will not work for a class 2 thread-unsafe function such as `rand` that relies on static state across calls.

Thread-unsafe function	Thread-unsafe class	Linux thread-safe version
`rand`	2	`rand_r`
`strtok`	2	`strtok_r`
`asctime`	3	`asctime_r`
`ctime`	3	`ctime_r`
`gethostbyaddr`	3	`gethostbyaddr_r`
`gethostbyname`	3	`gethostbyname_r`
`inet_ntoa`	3	(none)
`localtime`	3	`localtime_r`

Figure 12.41 Common thread-unsafe library functions.

Therefore, Linux systems provide reentrant versions of most thread-unsafe functions. The names of the reentrant versions always end with the _r suffix. For example, the reentrant version of asctime is called asctime_r. We recommend using these functions whenever possible.

12.7.4 Races

A race occurs when the correctness of a program depends on one thread reaching point x in its control flow before another thread reaches point y. Races usually occur because programmers assume that threads will take some particular trajectory through the execution state space, forgetting the golden rule that threaded programs must work correctly for any feasible trajectory.

An example is the easiest way to understand the nature of races. Consider the simple program in Figure 12.42. The main thread creates four peer threads and passes a pointer to a unique integer ID to each one. Each peer thread copies the

code/conc/race.c

```
1   /* WARNING: This code is buggy! */
2   #include "csapp.h"
3   #define N 4
4
5   void *thread(void *vargp);
6
7   int main()
8   {
9       pthread_t tid[N];
10      int i;
11
12      for (i = 0; i < N; i++)
13          Pthread_create(&tid[i], NULL, thread, &i);
14      for (i = 0; i < N; i++)
15          Pthread_join(tid[i], NULL);
16      exit(0);
17  }
18
19  /* Thread routine */
20  void *thread(void *vargp)
21  {
22      int myid = *((int *)vargp);
23      printf("Hello from thread %d\n", myid);
24      return NULL;
25  }
```

code/conc/race.c

Figure 12.42 A program with a race.

ID passed in its argument to a local variable (line 22) and then prints a message containing the ID. It looks simple enough, but when we run this program on our system, we get the following incorrect result:

```
linux> ./race
Hello from thread 1
Hello from thread 3
Hello from thread 2
Hello from thread 3
```

The problem is caused by a race between each peer thread and the main thread. Can you spot the race? Here is what happens. When the main thread creates a peer thread in line 13, it passes a pointer to the local stack variable `i`. At this point, the race is on between the next increment of `i` in line 12 and the dereferencing and assignment of the argument in line 22. If the peer thread executes line 22 before the main thread increments `i` in line 12, then the `myid` variable gets the correct ID. Otherwise, it will contain the ID of some other thread. The scary thing is that whether we get the correct answer depends on how the kernel schedules the execution of the threads. On our system it fails, but on other systems it might work correctly, leaving the programmer blissfully unaware of a serious bug.

To eliminate the race, we can dynamically allocate a separate block for each integer ID and pass the thread routine a pointer to this block, as shown in Figure 12.43 (lines 12–14). Notice that the thread routine must free the block in order to avoid a memory leak.

When we run this program on our system, we now get the correct result:

```
linux> ./norace
Hello from thread 0
Hello from thread 1
Hello from thread 2
Hello from thread 3
```

Practice Problem 12.13 (solution page 1075)

In Figure 12.43, we might be tempted to free the allocated memory block immediately after line 14 in the main thread, instead of freeing it in the peer thread. But this would be a bad idea. Why?

Practice Problem 12.14 (solution page 1075)

A. In Figure 12.43, we eliminated the race by allocating a separate block for each integer ID. Outline a different approach that does not call the `malloc` or `free` functions.

B. What are the advantages and disadvantages of this approach?

code/conc/norace.c

```
#include "csapp.h"
#define N 4

void *thread(void *vargp);

int main()
{
    pthread_t tid[N];
    int i, *ptr;

    for (i = 0; i < N; i++) {
        ptr = Malloc(sizeof(int));
        *ptr = i;
        Pthread_create(&tid[i], NULL, thread, ptr);
    }
    for (i = 0; i < N; i++)
        Pthread_join(tid[i], NULL);
    exit(0);
}

/* Thread routine */
void *thread(void *vargp)
{
    int myid = *((int *)vargp);
    Free(vargp);
    printf("Hello from thread %d\n", myid);
    return NULL;
}
```

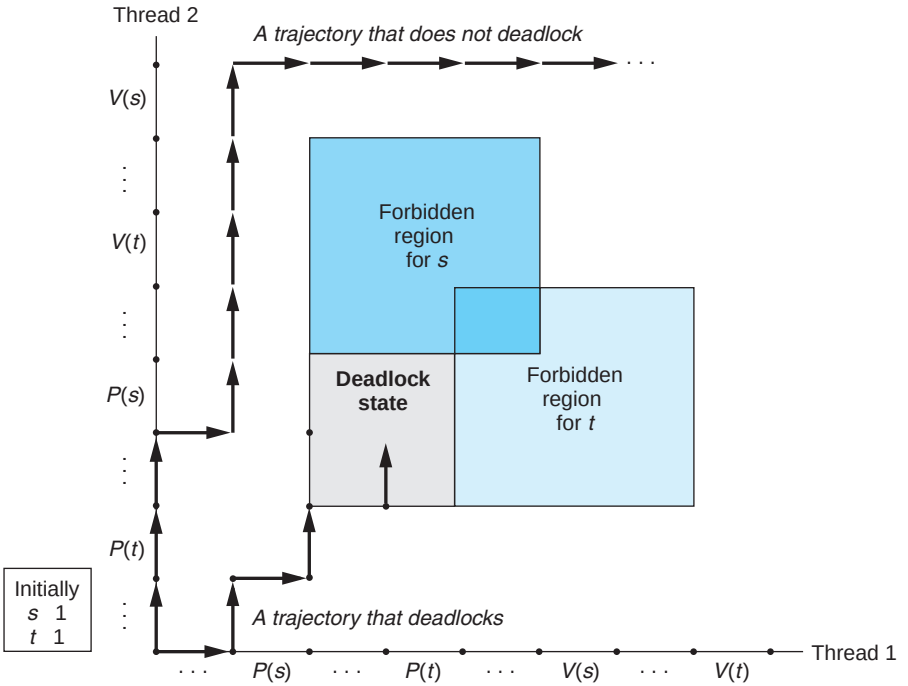
code/conc/norace.c

Figure 12.43 A correct version of the program in Figure 12.42 without a race.

12.7.5 Deadlocks

Semaphores introduce the potential for a nasty kind of run-time error, called deadlock, where a collection of threads is blocked, waiting for a condition that will never be true. The progress graph is an invaluable tool for understanding deadlock. For example, Figure 12.44 shows the progress graph for a pair of threads that use two semaphores for mutual exclusion. From this graph, we can glean some important insights about deadlock:

- The programmer has incorrectly ordered the P and V operations such that the forbidden regions for the two semaphores overlap. If some execution trajectory happens to reach the deadlock state d, then no further progress is


Thread 2
A trajectory that does not deadlock
V(s)
V(t)
P(s)
P(t)
Forbidden region for s
Deadlock state
Forbidden region for t
Initially
s 1
t 1
A trajectory that deadlocks
Thread 1
P(s)
P(t)
V(s)
V(t)

C. If so, what simple change to the initial semaphore values will eliminate the potential for deadlock?

D. Draw the progress graph for the resulting deadlock-free program.

12.8 Summary

A concurrent program consists of a collection of logical flows that overlap in time. In this chapter, we have studied three different mechanisms for building concurrent programs: processes, I/O multiplexing, and threads. We used a concurrent network server as the motivating application throughout.

Processes are scheduled automatically by the kernel, and because of their separate virtual address spaces, they require explicit IPC mechanisms in order to share data. Event-driven programs create their own concurrent logical flows, which are modeled as state machines, and use I/O multiplexing to explicitly schedule the flows. Because the program runs in a single process, sharing data between flows is fast and easy. Threads are a hybrid of these approaches. Like flows based on processes, threads are scheduled automatically by the kernel. Like flows based on I/O multiplexing, threads run in the context of a single process, and thus can share data quickly and easily.

Regardless of the concurrency mechanism, synchronizing concurrent accesses to shared data is a difficult problem. The P and V operations on semaphores have been developed to help deal with this problem. Semaphore operations can be used to provide mutually exclusive access to shared data, as well as to schedule access to resources such as the bounded buffers in producer-consumer systems and shared objects in readers-writers systems. A concurrent prethreaded echo server provides a compelling example of these usage scenarios for semaphores.

Concurrency introduces other difficult issues as well. Functions that are called by threads must have a property known as thread safety. We have identified four classes of thread-unsafe functions, along with suggestions for making them thread-safe. Reentrant functions are the proper subset of thread-safe functions that do not access any shared data. Reentrant functions are often more efficient than non-reentrant functions because they do not require any synchronization primitives. Some other difficult issues that arise in concurrent programs are races and deadlocks. Races occur when programmers make incorrect assumptions about how logical flows are scheduled. Deadlocks occur when a flow is waiting for an event that will never happen.

Bibliographic Notes

Semaphore operations were introduced by Dijkstra [31]. The progress graph concept was introduced by Coffman [23] and later formalized by Carson and Reynolds [16]. The readers-writers problem was introduced by Courtois et al [25]. Operating systems texts describe classical synchronization problems such as the dining philosophers, sleeping barber, and cigarette smokers problems in more de-

tail [102, 106, 113]. The book by Butenhof [15] is a comprehensive description of the Posix threads interface. The paper by Birrell [7] is an excellent introduction to threads programming and its pitfalls. The book by Reinders [90] describes a C/C++ library that simplifies the design and implementation of threaded programs. Several texts cover the fundamentals of parallel programming on multi-core systems [47, 71]. Pugh identifies weaknesses with the way that Java threads interact through memory and proposes replacement memory models [88]. Gustafson proposed the weak-scaling speedup model [43] as an alternative to strong scaling.

Homework Problems

12.16

Write a version of `hello.c` (Figure 12.13) that creates and reaps `n` joinable peer threads, where `n` is a command-line argument.

12.17

A. The program in Figure 12.46 has a bug. The thread is supposed to sleep for 1 second and then print a string. However, when we run it on our system, nothing prints. Why?

B. You can fix this bug by replacing the `exit` function in line 10 with one of two different Pthreads function calls. Which ones?

code/conc/hellobug.c

```
1   /* WARNING: This code is buggy! */
2   #include "csapp.h"
3   void *thread(void *vargp);
4
5   int main()
6   {
7       pthread_t tid;
8
9       Pthread_create(&tid, NULL, thread, NULL);
10      exit(0);
11  }
12
13  /* Thread routine */
14  void *thread(void *vargp)
15  {
16      Sleep(1);
17      printf("Hello, world!\n");
18      return NULL;
19  }
```

code/conc/hellobug.c

Figure 12.46 Buggy program for Problem 12.17.

12.18

Using the progress graph in Figure 12.21, classify the following trajectories as either safe or unsafe.

A. $H_2, L_2, U_2, H_1, L_1, S_2, U_1, S_1, T_1, T_2$

B. $H_2, H_1, L_1, U_1, S_1, L_2, T_1, U_2, S_2, T_2$

C. $H_1, L_1, H_2, L_2, U_2, S_2, U_1, S_1, T_1, T_2$

12.19

The solution to the first readers-writers problem in Figure 12.26 gives a somewhat weak priority to readers because a writer leaving its critical section might restart a waiting writer instead of a waiting reader. Derive a solution that gives stronger priority to readers, where a writer leaving its critical section will always restart a waiting reader if one exists.

12.20

Consider a simpler variant of the readers-writers problem where there are at most N readers. Derive a solution that gives equal priority to readers and writers, in the sense that pending readers and writers have an equal chance of being granted access to the resource. Hint: You can solve this problem using a single counting semaphore and a single mutex.

12.21

Derive a solution to the second readers-writers problem, which favors writers instead of readers.

12.22

Test your understanding of the `select` function by modifying the server in Figure 12.6 so that it echoes at most one text line per iteration of the main server loop.

12.23

The event-driven concurrent echo server in Figure 12.8 is flawed because a malicious client can deny service to other clients by sending a partial text line. Write an improved version of the server that can handle these partial text lines without blocking.

12.24

The functions in the Rio I/O package (Section 10.5) are thread-safe. Are they reentrant as well?

12.25

In the prethreaded concurrent echo server in Figure 12.28, each thread calls the `echo_cnt` function (Figure 12.29). Is `echo_cnt` thread-safe? Is it reentrant? Why or why not?

12.26

Use the lock-and-copy technique to implement a thread-safe non-reentrant version of `gethostbyname` called `gethostbyname_ts`. A correct solution will use a deep copy of the `hostent` structure protected by a mutex.

12.27

Some network programming texts suggest the following approach for reading and writing sockets: Before interacting with the client, open two standard I/O streams on the same open connected socket descriptor, one for reading and one for writing:

```
FILE *fpin, *fpout;

fpin = fdopen(sockfd, "r");
fpout = fdopen(sockfd, "w");
```

When the server finishes interacting with the client, close both streams as follows:

```
fclose(fpin);
fclose(fpout);
```

However, if you try this approach in a concurrent server based on threads, you will create a deadly race condition. Explain.

12.28

In Figure 12.45, does swapping the order of the two V operations have any effect on whether or not the program deadlocks? Justify your answer by drawing the progress graphs for the four possible cases:

Case 1		Case 2		Case 3		Case 4	
Thread 1	Thread 2	Thread 1	Thread 2	Thread 1	Thread 2	Thread 1	Thread 2
P(s)	P(s)	P(s)	P(s)	P(s)	P(s)	P(s)	P(s)
P(t)	P(t)	P(t)	P(t)	P(t)	P(t)	P(t)	P(t)
V(s)	V(s)	V(s)	V(t)	V(t)	V(s)	V(t)	V(t)
V(t)	V(t)	V(t)	V(s)	V(s)	V(t)	V(s)	V(s)

12.29

Can the following program deadlock? Why or why not?

```
Initially: a = 1, b = 1, c = 1.

Thread 1:       Thread 2:
   P(a);           P(c);
   P(b);           P(b);
   V(b);           V(b);
   P(c);           V(c);
   V(c);
   V(a);
```

12.30

Consider the following program that deadlocks.

```
Initially: a = 1, b = 1, c = 1.

Thread 1:       Thread 2:       Thread 3:
   P(a);           P(c);           P(c);
   P(b);           P(b);           V(c);
   V(b);           V(b);           P(b);
   P(c);           V(c);           P(a);
   V(c);           P(a);           V(a);
   V(a);           V(a);           V(b);
```

A. For each thread, list the pairs of mutexes that it holds simultaneously.

B. If `a < b < c`, which threads violate the mutex lock ordering rule?

C. For these threads, show a new lock ordering that guarantees freedom from deadlock.

12.31

Implement a version of the standard I/O `fgets` function, called `tfgets`, that times out and returns `NULL` if it does not receive an input line on standard input within 5 seconds. Your function should be implemented in a package called `tfgets-proc.c` using processes, signals, and nonlocal jumps. It should not use the Linux `alarm` function. Test your solution using the driver program in Figure 12.47.

code/conc/tfgets-main.c

```
1   #include "csapp.h"
2
3   char *tfgets(char *s, int size, FILE *stream);
4
5   int main()
6   {
7       char buf[MAXLINE];
8
9       if (tfgets(buf, MAXLINE, stdin) == NULL)
10          printf("BOOM!\n");
11      else
12          printf("%s", buf);
13
14      exit(0);
15  }
```

code/conc/tfgets-main.c

Figure 12.47 Driver program for Problems 12.31–12.33.

12.32
Implement a version of the `tfgets` function from Problem 12.31 that uses the `select` function. Your function should be implemented in a package called `tfgets-select.c`. Test your solution using the driver program from Problem 12.31. You may assume that standard input is assigned to descriptor 0.

12.33
Implement a threaded version of the `tfgets` function from Problem 12.31. Your function should be implemented in a package called `tfgets-thread.c`. Test your solution using the driver program from Problem 12.31.

12.34
Write a parallel threaded version of an $N \times M$ matrix multiplication kernel. Compare the performance to the sequential case.

12.35
Implement a concurrent version of the TINY Web server based on processes. Your solution should create a new child process for each new connection request. Test your solution using a real Web browser.

12.36
Implement a concurrent version of the TINY Web server based on I/O multiplexing. Test your solution using a real Web browser.

12.37
Implement a concurrent version of the TINY Web server based on threads. Your solution should create a new thread for each new connection request. Test your solution using a real Web browser.

12.38
Implement a concurrent prethreaded version of the TINY Web server. Your solution should dynamically increase or decrease the number of threads in response to the current load. One strategy is to double the number of threads when the buffer becomes full, and halve the number of threads when the buffer becomes empty. Test your solution using a real Web browser.

12.39
A Web proxy is a program that acts as a middleman between a Web server and browser. Instead of contacting the server directly to get a Web page, the browser contacts the proxy, which forwards the request to the server. When the server replies to the proxy, the proxy sends the reply to the browser. For this lab, you will write a simple Web proxy that filters and logs requests:

A. In the first part of the lab, you will set up the proxy to accept requests, parse the HTTP, forward the requests to the server, and return the results to the browser. Your proxy should log the URLs of all requests in a log file on disk, and it should also block requests to any URL contained in a filter file on disk.

B. In the second part of the lab, you will upgrade your proxy to deal with multiple open connections at once by spawning a separate thread to handle each request. While your proxy is waiting for a remote server to respond to a request so that it can serve one browser, it should be working on a pending request from another browser.

Check your proxy solution using a real Web browser.

Solutions to Practice Problems

Solution to Problem 12.1 (page 1011)

When the parent process on the concurrent server starts executing, the reference counter increments from 0 to 1 for the associated file table. When this parent process forks the child process, the reference counter is incremented from 1 to 2. When the parent closes its copy of the descriptor, the reference count is decremented from 2 to 1. Similarly, when the child's end of connection closes, the reference counter is decremented from 1 to 0.

Solution to Problem 12.2 (page 1011)

When a process terminates for any reason, the kernel closes all open descriptors. Thus, the parent's copy of the connected file descriptor will be closed automatically when the parent exits.

Solution to Problem 12.3 (page 1016)

Recall that the `echo` function from Figure 11.22 echoes each line from the client until the client loses its end of the connection. If Ctrl+D is typed when the `echo` function is under execution, the server would consider it to be the EOF and may assume that the client has closed its end of connection and hence, may stop echoing back to the client.

Solution to Problem 12.4 (page 1020)

`pool.nready` is an integer variable. We reinitialize the `pool.nready` variable with the value obtained from the call to `select` so as to store the total number of ready descriptors returned by `select`.

Solution to Problem 12.5 (page 1028)

Yes, there are chances of memory leak if lines 31 or 32 are deleted from Figure 12.14. Since the threads are not explicitly reaped, each thread must be detached so that its memory resource will be reclaimed when it terminates. Similarly, it is important to free the memory block that was allocated by the main thread.

Solution to Problem 12.6 (page 1031)

The main idea here is that stack variables are private, whereas global and static variables are shared. Static variables such as `cnt` are a little tricky because the sharing is limited to the functions within their scope—in this case, the thread routine.

A. Here is the table:

Variable instance	Referenced by		
	main thread?	peer thread 0?	peer thread 1?
`ptr`	yes	yes	yes
`cnt`	no	yes	yes
`i.m`	yes	no	no
`msgs.m`	yes	yes	yes
`myid.p0`	no	yes	no
`myid.p1`	no	no	yes

Notes:

`ptr` A global variable that is written by the main thread and read by the peer threads.

`cnt` A static variable with only one instance in memory that is read and written by the two peer threads.

`i.m` A local automatic variable stored on the stack of the main thread. Even though its value is passed to the peer threads, the peer threads never reference it on the stack, and thus it is not shared.

`msgs.m` A local automatic variable stored on the main thread's stack and referenced indirectly through `ptr` by both peer threads.

`myid.p0` and `myid.p1` Instances of a local automatic variable residing on the stacks of peer threads 0 and 1, respectively.

B. Variables `ptr`, `cnt`, and `msgs` are referenced by more than one thread and thus are shared.

Solution to Problem 12.7 (page 1034)

The important idea here is that you cannot make any assumptions about the ordering that the kernel chooses when it schedules your threads.

Step	Thread	Instr.	$\%rdx_1$	$\%rdx_2$	cnt
1	1	H_1	—	—	0
2	1	L_1	0	—	0
3	2	H_2	—	—	0
4	2	L_2	—	0	0
5	2	U_2	—	1	0
6	2	S_2	—	1	1
7	1	U_1	1	—	1
8	1	S_1	1	—	1
9	1	T_1	1	—	1
10	2	T_2	—	1	1

Variable `cnt` has a final incorrect value of 1.

functions pass arguments only by value and all data references are to local automatic stack variables.

Solution to Problem 12.13 (page 1062)

If we free the block immediately after the call to `pthread_create` in line 14, then we will introduce a new race, this time between the call to `free` in the main thread and the assignment statement in line 24 of the thread routine.

Solution to Problem 12.14 (page 1062)

A. Another approach is to pass the integer `i` directly, rather than passing a pointer to `i`:

```
for (i = 0; i < N; i++)
    Pthread_create(&tid[i], NULL, thread, (void *)i);
```

In the thread routine, we cast the argument back to an `int` and assign it to `myid`:

```
int myid = (int) vargp;
```

B. The advantage is that it reduces overhead by eliminating the calls to `malloc` and `free`. A significant disadvantage is that it assumes that pointers are at least as large as `int`s. While this assumption is true for all modern systems, it might not be true for legacy or future systems.

Solution to Problem 12.15 (page 1065)

A. The progress graph for the original program is shown in Figure 12.48 on the next page.

B. The program always deadlocks, since any feasible trajectory is eventually trapped in a deadlock state.

C. To eliminate the deadlock potential, initialize the binary semaphore `t` to 1 instead of 0.

D. The progress graph for the corrected program is shown in Figure 12.49.

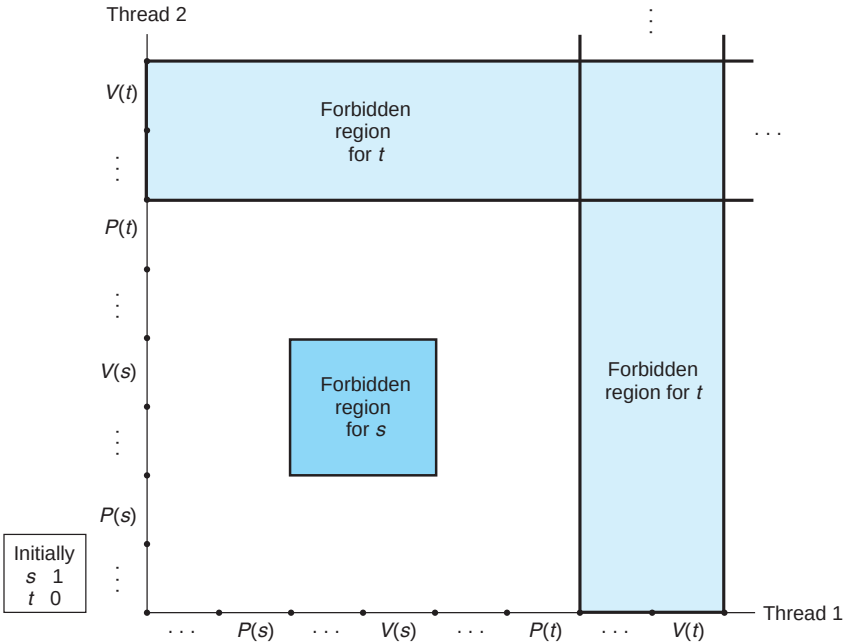


Figure 12.48 Progress graph for a program that deadlocks.

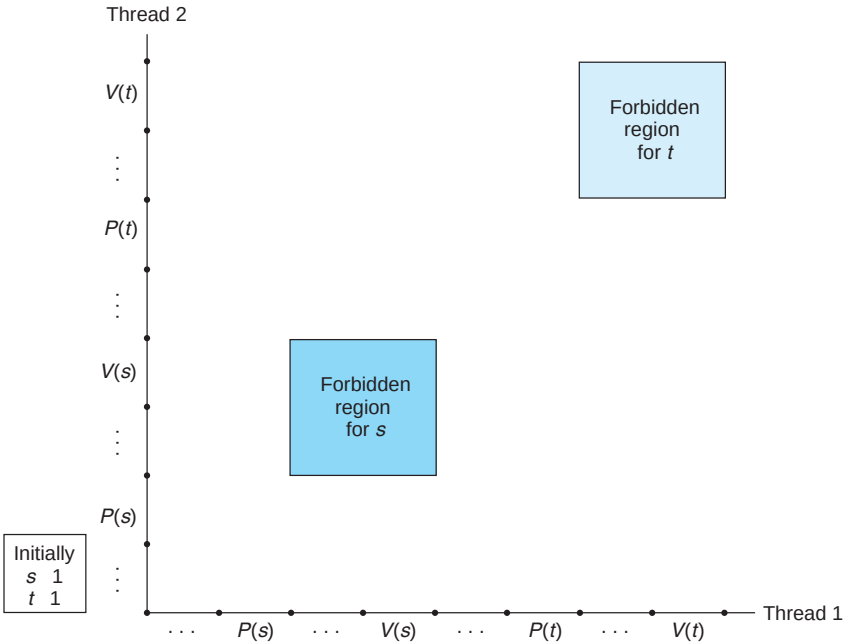


Figure 12.49 Progress graph for the corrected deadlock-free program.

Error Handling

Programmers should always check the error codes returned by system-level functions. There are many subtle ways that things can go wrong, and it only makes sense to use the status information that the kernel is able to provide us. Unfortunately, programmers are often reluctant to do error checking because it clutters their code, turning a single line of code into a multi-line conditional statement. Error checking is also confusing because different functions indicate errors in different ways.

We were faced with a similar problem when writing this text. On the one hand, we would like our code examples to be concise and simple to read. On the other hand, we do not want to give students the wrong impression that it is OK to skip error checking. To resolve these issues, we have adopted an approach based on error-handling wrappers that was pioneered by W. Richard Stevens in his network programming text [110].

The idea is that given some base system-level function `foo`, we define a wrapper function `Foo` with identical arguments, but with the first letter capitalized. The wrapper calls the base function and checks for errors. If it detects an error, the wrapper prints an informative message and terminates the process. Otherwise, it returns to the caller. Notice that if there are no errors, the wrapper behaves exactly like the base function. Put another way, if a program runs correctly with wrappers, it will run correctly if we render the first letter of each wrapper in lowercase and recompile.

The wrappers are packaged in a single source file (`csapp.c`) that is compiled and linked into each program. A separate header file (`csapp.h`) contains the function prototypes for the wrappers.

This appendix gives a tutorial on the different kinds of error handling in Unix systems and gives examples of the different styles of error-handling wrappers. Copies of the `csapp.h` and `csapp.c` files are available at the CS:APP Web site.

A.1 Error Handling in Unix Systems

The systems-level function calls that we will encounter in this book use three different styles for returning errors: Unix-style, Posix-style, and GAI-style.

Unix-Style Error Handling

Functions such as `fork` and `wait` that were developed in the early days of Unix (as well as some older Posix functions) overload the function return value with both error codes and useful results. For example, when the Unix-style `wait` function encounters an error (e.g., there is no child process to reap), it returns -1 and sets the global variable `errno` to an error code that indicates the cause of the error. If `wait` completes successfully, then it returns the useful result, which is the PID of the reaped child. Unix-style error-handling code is typically of the following form:

```
1       if ((pid = wait(NULL)) < 0) {
2           fprintf(stderr, "wait error: %s\n", strerror(errno));
3           exit(0);
4       }
```

The `strerror` function returns a text description for a particular value of `errno`.

Posix-Style Error Handling

Many of the newer Posix functions such as Pthreads use the return value only to indicate success (zero) or failure (nonzero). Any useful results are returned in function arguments that are passed by reference. We refer to this approach as Posix-style error handling. For example, the Posix-style `pthread_create` function indicates success or failure with its return value and returns the ID of the newly created thread (the useful result) by reference in its first argument. Posix-style error-handling code is typically of the following form:

```
1   if ((retcode = pthread_create(&tid, NULL, thread, NULL)) != 0) {
2       fprintf(stderr, "pthread_create error: %s\n", strerror(retcode));
3       exit(0);
4   }
```

The `strerror` function returns a text description for a particular value of `retcode`.

GAI-Style Error Handling

The `getaddrinfo` (GAI) and `getnameinfo` functions return zero on success and a nonzero value on failure. GAI error-handling code is typically of the following form:

```
1   if ((retcode = getaddrinfo(host, service, &hints, &result)) != 0) {
2       fprintf(stderr, "getaddrinfo error: %s\n", gai_strerror(retcode));
3       exit(0);
4   }
```

The `gai_strerror` function returns a text description for a particular value of `retcode`.

S ar o Err or-R ort i F ti o

Thoughout this book, we use the following error-reporting functions to accommodate different error-handling styles.

```
#include "csapp.h"

void unix_error(char *msg);
void posix_error(int code, char *msg);
void gai_error(int code, char *msg);
void app_error(char *msg);
```

Returns: nothing

As their names suggest, the `unix_error`, `posix_error`, and `gai_error` functions report Unix-style, Posix-style, and GAI-style errors and then terminate. The `app_error` function is included as a convenience for application errors. It simply prints its input and then terminates. Figure A.1 shows the code for the error-reporting functions.

A.2 Error-Handling Wrappers

Here are some examples of the different error-handling wrappers.

Unix-style error-handling wrappers. Figure A.2 shows the wrapper for the Unix-style `wait` function. If the `wait` returns with an error, the wrapper prints an informative message and then exits. Otherwise, it returns a PID to the caller. Figure A.3 shows the wrapper for the Unix-style `kill` function. Notice that this function, unlike `wait`, returns `void` on success.

Posix-style error-handling wrappers. Figure A.4 shows the wrapper for the Posix-style `pthread_detach` function. Like most Posix-style functions, it does not overload useful results with error-return codes, so the wrapper returns `void` on success.

GAI-style error-handling wrappers. Figure A.5 shows the error-handling wrapper for the GAI-style `getaddrinfo` function.

code/src/csapp.c

```
1  void unix_error(char *msg) /* Unix-style error */
2  {
3      fprintf(stderr, "%s: %s\n", msg, strerror(errno));
4      exit(0);
5  }
6
7  void posix_error(int code, char *msg) /* Posix-style error */
8  {
9      fprintf(stderr, "%s: %s\n", msg, strerror(code));
10     exit(0);
11 }
12
13 void gai_error(int code, char *msg) /* Getaddrinfo-style error */
14 {
15     fprintf(stderr, "%s: %s\n", msg, gai_strerror(code));
16     exit(0);
17 }
18
19 void app_error(char *msg) /* Application error */
20 {
21     fprintf(stderr, "%s\n", msg);
22     exit(0);
23 }
```

code/src/csapp.c

Figure A.1 Error-reporting functions.

code/src/csapp.c

```
1  pid_t Wait(int *status)
2  {
3      pid_t pid;
4
5      if ((pid  = wait(status)) < 0)
6          unix_error("Wait error");
7      return pid;
8  }
```

code/src/csapp.c

Figure A.2 Wrapper for Unix-style `wait` function.

code/src/csapp.c

```
1  void Kill(pid_t pid, int signum)
2  {
3      int rc;
4
5      if ((rc = kill(pid, signum)) < 0)
6          unix_error("Kill error");
7  }
```

code/src/csapp.c

Figure A.3 Wrapper for Unix-style `kill` function.

code/src/csapp.c

```
1  void Pthread_detach(pthread_t tid) {
2      int rc;
3
4      if ((rc = pthread_detach(tid)) != 0)
5          posix_error(rc, "Pthread_detach error");
6  }
```

code/src/csapp.c

Figure A.4 Wrapper for Posix-style `pthread_detach` function.

code/src/csapp.c

```
1  void Getaddrinfo(const char *node, const char *service,
2                   const struct addrinfo *hints, struct addrinfo **res)
3  {
4      int rc;
5
6      if ((rc = getaddrinfo(node, service, hints, res)) != 0)
7          gai_error(rc, "Getaddrinfo error");
8  }
```

code/src/csapp.c

Figure A.5 Wrapper for GAI-style `getaddrinfo` function.

[19] S. Chellappa, F. Franchetti, and M. Püschel. How to write fast numerical code: A small introduction. In Generative and Transformational Techniques in Software Engineering II, volume 5235 of Lecture Notes in Computer Science, pages 196–259. Springer-Verlag, 2008.

[20] P. Chen, E. Lee, G. Gibson, R. Katz, and D. Patterson. RAID: High-performance, reliable secondary storage. ACM Computing Surveys 26(2):145–185, June 1994.

[21] S. Chen, P. Gibbons, and T. Mowry. Improving index performance through prefetching. In Proceedings of the 2001 ACM SIGMOD International Conference on Management of Data, pages 235–246. ACM, May 2001.

[22] T. Chilimbi, M. Hill, and J. Larus. Cache-conscious structure layout. In Proceedings of the 1999 ACM Conference on Programming Language Design and Implementation (PLDI), pages 1–12. ACM, May 1999.

[23] E. Coffman, M. Elphick, and A. Shoshani. System deadlocks. ACM Computing Surveys 3(2):67–78, June 1971.

[24] D. Cohen. On holy wars and a plea for peace. IEEE Computer 14(10):48–54, October 1981.

[25] P. J. Courtois, F. Heymans, and D. L. Parnas. Concurrent control with "readers" and "writers." Communications of the ACM 14(10):667–668, 1971.

[26] C. Cowan, P. Wagle, C. Pu, S. Beattie, and J. Walpole. Buffer overflows: Attacks and defenses for the vulnerability of the decade. In DARPA Information Survivability Conference and Expo (DISCEX), volume 2, pages 119–129, March 2000.

[27] J. H. Crawford. The i486 CPU: Executing instructions in one clock cycle. IEEE Micro 10(1):27–36, February 1990.

[28] V. Cuppu, B. Jacob, B. Davis, and T. Mudge. A performance comparison of contemporary DRAM architectures. In Proceedings of the 26th International Symposium on Computer Architecture (ISCA), pages 222–233, ACM, 1999.

[29] B. Davis, B. Jacob, and T. Mudge. The new DRAM interfaces: SDRAM, RDRAM, and variants. In Proceedings of the 3rd International Symposium on High Performance Computing (ISHPC), volume 1940 of Lecture Notes in Computer Science, pages 26–31. Springer-Verlag, October 2000.

[30] E. Demaine. Cache-oblivious algorithms and data structures. In Lecture Notes from the EEF Summer School on Massive Data Sets. BRICS, University of Aarhus, Denmark, 2002.

[31] E. W. Dijkstra. Cooperating sequential processes. Technical Report EWD-123, Technological University, Eindhoven, the Netherlands, 1965.

[32] C. Ding and K. Kennedy. Improving cache performance of dynamic applications through data and computation reorganizations at run time. In Proceedings of the 1999 ACM Conference on Programming Language Design and Implementation (PLDI), pages 229–241. ACM, May 1999.

[33] M. Dowson. The Ariane 5 software failure. SIGSOFT Software Engineering Notes 22(2):84, 1997.

[34] U. Drepper. User-level IPv6 programming introduction. Available at http://www.akkadia.org/drepper/userapi-ipv6.html, 2008.

[35] M. W. Eichen and J. A. Rochlis. With microscope and tweezers: An analysis of the Internet virus of November, 1988. In Proceedings of the IEEE Symposium on Research in Security and Privacy, pages 326–343. IEEE, 1989.

[36] ELF-64 Object File Format, Version 1.5 Draft 2, 1998. Available at http://www.uclibc.org/docs/elf-64-gen.pdf.

[37] R. Fielding, J. Gettys, J. Mogul, H. Frystyk, L. Masinter, P. Leach, and T. Berners-Lee. Hypertext transfer protocol - HTTP/1.1. RFC 2616, 1999.

[38] M. Frigo, C. E. Leiserson, H. Prokop, and S. Ramachandran. Cache-oblivious algorithms. In Proceedings of the 40th IEEE Symposium on Foundations of Computer Science (FOCS), pages 285–297. IEEE, August 1999.

[39] M. Frigo and V. Strumpen. The cache complexity of multithreaded cache oblivious algorithms. In Proceedings of the 18th Symposium on Paral-

lelism in Algorithms and Architectures (SPAA), pages 271–280. ACM, 2006.

[40] G. Gibson, D. Nagle, K. Amiri, J. Butler, F. Chang, H. Gobioff, C. Hardin, E. Riedel, D. Rochberg, and J. Zelenka. A cost-effective, high-bandwidth storage architecture. In Proceedings of the 8th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), pages 92–103. ACM, October 1998.

[41] G. Gibson and R. Van Meter. Network attached storage architecture. Communications of the ACM 43(11):37–45, November 2000.

[42] Google. IPv6 Adoption. Available at http://www.google.com/intl/en/ipv6/statistics.html.

[43] J. Gustafson. Reevaluating Amdahl's law. Communications of the ACM 31(5):532–533, August 1988.

[44] L. Gwennap. New algorithm improves branch prediction. Microprocessor Report 9(4), March 1995.

[45] S. P. Harbison and G. L. Steele, Jr. C, A Reference Manual, Fifth Edition. Prentice Hall, 2002.

[46] J. L. Hennessy and D. A. Patterson. Computer Architecture: A Quantitative Approach, Fifth Edition. Morgan Kaufmann, 2011.

[47] M. Herlihy and N. Shavit. The Art of Multiprocessor Programming. Morgan Kaufmann, 2008.

[48] C. A. R. Hoare. Monitors: An operating system structuring concept. Communications of the ACM 17(10):549–557, October 1974.

[49] Intel Corporation. Intel 64 and IA-32 Architectures Optimization Reference Manual. Available at http://www.intel.com/content/www/us/en/processors/architectures-software-developer-manuals.html.

[50] Intel Corporation. Intel 64 and IA-32 Architectures Software Developer's Manual, Volume 1: Basic Architecture. Available at http://www.intel.com/content/www/us/en/processors/architectures-software-developer-manuals.html.

[51] Intel Corporation. Intel 64 and IA-32 Architectures Software Developer's Manual, Volume 2: Instruction Set Reference. Available at http://www.intel.com/content/www/us/en/processors/architectures-software-developer-manuals.html.

[52] Intel Corporation. Intel 64 and IA-32 Architectures Software Developer's Manual, Volume 3a: System Programming Guide, Part 1. Available at http://www.intel.com/content/www/us/en/processors/architectures-software-developer-manuals.html.

[53] Intel Corporation. Intel Solid-State Drive 730 Series: Product Specification. Available at http://www.intel.com/content/www/us/en/solid-state-drives/ssd-730-series-spec.html.

[54] Intel Corporation. Tool Interface Standards Portable Formats Specification, Version 1.1, 1993. Order number 241597.

[55] F. Jones, B. Prince, R. Norwood, J. Hartigan, W. Vogley, C. Hart, and D. Bondurant. Memory—a new era of fast dynamic RAMs (for video applications). IEEE Spectrum, pages 43–45, October 1992.

[56] R. Jones and R. Lins. Garbage Collection: Algorithms for Automatic Dynamic Memory Management. Wiley, 1996.

[57] M. Kaashoek, D. Engler, G. Ganger, H. Briceo, R. Hunt, D. Maziers, T. Pinckney, R. Grimm, J. Jannotti, and K. MacKenzie. Application performance and flexibility on Exokernel systems. In Proceedings of the 16th ACM Symposium on Operating System Principles (SOSP), pages 52–65. ACM, October 1997.

[58] R. Katz and G. Borriello. Contemporary Logic Design, Second Edition. Prentice Hall, 2005.

[59] B. W. Kernighan and R. Pike. The Practice of Programming. Addison-Wesley, 1999.

[60] B. Kernighan and D. Ritchie. The C Programming Language, First Edition. Prentice Hall, 1978.

[61] B. Kernighan and D. Ritchie. The C Programming Language, Second Edition. Prentice Hall, 1988.

[62] Michael Kerrisk. The Linux Programming Interface. No Starch Press, 2010.

[63] T. Kilburn, B. Edwards, M. Lanigan, and F. Sumner. One-level storage system. IRE

Transactions on Electronic Computers EC-11:223–235, April 1962.

[64] D. Knuth. The Art of Computer Programming, Volume 1: Fundamental Algorithms, Third Edition. Addison-Wesley, 1997.

[65] J. Kurose and K. Ross. Computer Networking: A Top-Down Approach, Sixth Edition. Addison-Wesley, 2012.

[66] M. Lam, E. Rothberg, and M. Wolf. The cache performance and optimizations of blocked algorithms. In Proceedings of the 4th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), pages 63–74. ACM, April 1991.

[67] D. Lea. A memory allocator. Available at http://gee.cs.oswego.edu/dl/html/malloc.html, 1996.

[68] C. E. Leiserson and J. B. Saxe. Retiming synchronous circuitry. Algorithmica 6(1–6), June 1991.

[69] J. R. Levine. Linkers and Loaders. Morgan Kaufmann, 1999.

[70] David Levinthal. Performance Analysis Guide for Intel Core i7 Processor and Intel Xeon 5500 Processors. Available at https://software.intel.com/sites/products/collateral/hpc/vtune/performance_analysis_guide.pdf.

[71] C. Lin and L. Snyder. Principles of Parallel Programming. Addison Wesley, 2008.

[72] Y. Lin and D. Padua. Compiler analysis of irregular memory accesses. In Proceedings of the 2000 ACM Conference on Programming Language Design and Implementation (PLDI), pages 157–168. ACM, June 2000.

[73] J. L. Lions. Ariane 5 Flight 501 failure. Technical Report, European Space Agency, July 1996.

[74] S. Macguire. Writing Solid Code. Microsoft Press, 1993.

[75] S. A. Mahlke, W. Y. Chen, J. C. Gyllenhal, and W. W. Hwu. Compiler code transformations for superscalar-based high-performance systems. In Proceedings of the 1992 ACM/IEEE Conference on Supercomputing, pages 808–817. ACM, 1992.

[76] E. Marshall. Fatal error: How Patriot overlooked a Scud. Science, page 1347, March 13, 1992.

[77] M. Matz, J. Hubička, A. Jaeger, and M. Mitchell. System V application binary interface AMD64 architecture processor supplement. Technical Report, x86-64.org, 2013. Available at http://www.x86-64.org/documentation_folder/abi-0.99.pdf.

[78] J. Morris, M. Satyanarayanan, M. Conner, J. Howard, D. Rosenthal, and F. Smith. Andrew: A distributed personal computing environment. Communications of the ACM, pages 184–201, March 1986.

[79] T. Mowry, M. Lam, and A. Gupta. Design and evaluation of a compiler algorithm for prefetching. In Proceedings of the 5th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), pages 62–73. ACM, October 1992.

[80] S. S. Muchnick. Advanced Compiler Design and Implementation. Morgan Kaufmann, 1997.

[81] S. Nath and P. Gibbons. Online maintenance of very large random samples on flash storage. In Proceedings of VLDB, pages 970–983. VLDB Endowment, August 2008.

[82] M. Overton. Numerical Computing with IEEE Floating Point Arithmetic. SIAM, 2001.

[83] D. Patterson, G. Gibson, and R. Katz. A case for redundant arrays of inexpensive disks (RAID). In Proceedings of the 1998 ACM SIGMOD International Conference on Management of Data, pages 109–116. ACM, June 1988.

[84] L. Peterson and B. Davie. Computer Networks: A Systems Approach, Fifth Edition. Morgan Kaufmann, 2011.

[85] J. Pincus and B. Baker. Beyond stack smashing: Recent advances in exploiting buffer overruns. IEEE Security and Privacy 2(4):20–27, 2004.

[86] S. Przybylski. Cache and Memory Hierarchy Design: A Performance-Directed Approach. Morgan Kaufmann, 1990.

[87] W. Pugh. The Omega test: A fast and practical integer programming algorithm for depen-

dence analysis. Communications of the ACM 35(8):102–114, August 1992.

[88] W. Pugh. Fixing the Java memory model. In Proceedings of the ACM Conference on Java Grande, pages 89–98. ACM, June 1999.

[89] J. Rabaey, A. Chandrakasan, and B. Nikolic. Digital Integrated Circuits: A Design Perspective, Second Edition. Prentice Hall, 2003.

[90] J. Reinders. Intel Threading Building Blocks. O'Reilly, 2007.

[91] D. Ritchie. The evolution of the Unix time-sharing system. AT&T Bell Laboratories Technical Journal 63(6 Part 2):1577–1593, October 1984.

[92] D. Ritchie. The development of the C language. In Proceedings of the 2nd ACM SIGPLAN Conference on History of Programming Languages, pages 201–208. ACM, April 1993.

[93] D. Ritchie and K. Thompson. The Unix time-sharing system. Communications of the ACM 17(7):365–367, July 1974.

[94] M. Satyanarayanan, J. Kistler, P. Kumar, M. Okasaki, E. Siegel, and D. Steere. Coda: A highly available file system for a distributed workstation environment. IEEE Transactions on Computers 39(4):447–459, April 1990.

[95] J. Schindler and G. Ganger. Automated disk drive characterization. Technical Report CMU-CS-99-176, School of Computer Science, Carnegie Mellon University, 1999.

[96] F. B. Schneider and K. P. Birman. The monoculture risk put into context. IEEE Security and Privacy 7(1):14–17, January 2009.

[97] R. C. Seacord. Secure Coding in C and C++, Second Edition. Addison-Wesley, 2013.

[98] R. Sedgewick and K. Wayne. Algorithms, Fourth Edition. Addison-Wesley, 2011.

[99] H. Shacham, M. Page, B. Pfaff, E.-J. Goh, N. Modadugu, and D. Boneh. On the effectiveness of address-space randomization. In Proceedings of the 11th ACM Conference on Computer and Communications Security (CCS), pages 298–307. ACM, 2004.

[100] J. P. Shen and M. Lipasti. Modern Processor Design: Fundamentals of Superscalar Processors. McGraw Hill, 2005.

[101] B. Shriver and B. Smith. The Anatomy of a High-Performance Microprocessor: A Systems Perspective. IEEE Computer Society, 1998.

[102] A. Silberschatz, P. Galvin, and G. Gagne. Operating Systems Concepts, Ninth Edition. Wiley, 2014.

[103] R. Skeel. Roundoff error and the Patriot missile. SIAM News 25(4):11, July 1992.

[104] A. Smith. Cache memories. ACM Computing Surveys 14(3), September 1982.

[105] E. H. Spafford. The Internet worm program: An analysis. Technical Report CSD-TR-823, Department of Computer Science, Purdue University, 1988.

[106] W. Stallings. Operating Systems: Internals and Design Principles, Eighth Edition. Prentice Hall, 2014.

[107] W. R. Stevens. TCP/IP Illustrated, Volume 3: TCP for Transactions, HTTP, NNTP and the Unix Domain Protocols. Addison-Wesley, 1996.

[108] W. R. Stevens. Unix Network Programming: Interprocess Communications, Second Edition, volume 2. Prentice Hall, 1998.

[109] W. R. Stevens and K. R. Fall. TCP/IP Illustrated, Volume 1: The Protocols, Second Edition. Addison-Wesley, 2011.

[110] W. R. Stevens, B. Fenner, and A. M. Rudoff. Unix Network Programming: The Sockets Networking API, Third Edition, volume 1. Prentice Hall, 2003.

[111] W. R. Stevens and S. A. Rago. Advanced Programming in the Unix Environment, Third Edition. Addison-Wesley, 2013.

[112] T. Stricker and T. Gross. Global address space, non-uniform bandwidth: A memory system performance characterization of parallel systems. In Proceedings of the 3rd International Symposium on High Performance Computer Architecture (HPCA), pages 168–179. IEEE, February 1997.

[113] A. S. Tanenbaum and H. Bos. Modern Operating Systems, Fourth Edition. Prentice Hall, 2015.

[114] A. S. Tanenbaum and D. Wetherall. Computer Networks, Fifth Edition. Prentice Hall, 2010.

[115] K. P. Wadleigh and I. L. Crawford. Software Optimization for High-Performance Computing: Creating Faster Applications. Prentice Hall, 2000.

[116] J. F. Wakerly. Digital Design Principles and Practices, Fourth Edition. Prentice Hall, 2005.

[117] M. V. Wilkes. Slave memories and dynamic storage allocation. IEEE Transactions on Electronic Computers, EC-14(2), April 1965.

[118] P. Wilson, M. Johnstone, M. Neely, and D. Boles. Dynamic storage allocation: A survey and critical review. In International Workshop on Memory Management, volume 986 of Lecture Notes in Computer Science, pages 1–116. Springer-Verlag, 1995.

[119] M. Wolf and M. Lam. A data locality algorithm. In Proceedings of the 1991 ACM Conference on Programming Language Design and Implementation (PLDI), pages 30–44, June 1991.

[120] G. R. Wright and W. R. Stevens. TCP/IP Illustrated, Volume 2: The Implementation. Addison-Wesley, 1995.

[121] J. Wylie, M. Bigrigg, J. Strunk, G. Ganger, H. Kiliccote, and P. Khosla. Survivable information storage systems. IEEE Computer 33:61–68, August 2000.

[122] T.-Y. Yeh and Y. N. Patt. Alternative implementation of two-level adaptive branch prediction. In Proceedings of the 19th Annual International Symposium on Computer Architecture (ISCA), pages 451–461. ACM, 1998.

Index

Page numbers of de ning references areitalicized. Entries that belong to a hardware or software system are followed by a tag in brackets that identi es the system, along with a brief description to jog your memory. Here is the list of tags and their meanings.

[C] C language construct
[C Stdlib] C standard library function
[CS:APP] Program or function developed in this text
[HCL] HCL language construct
[Unix] Unix program, function, variable, or constant
[x86-64] x86-64 machine-language instruction
[Y86-64] Y86-64 machine-language instruction

! [HCL] not operation, 409
$ for immediate operands, 217
& [C] address of operation
  local variables, 284
  logic gates, 409
  pointers, 84, 224, 293, 313
* [C] dereference pointer operation, 224
-> [C] dereference and select eld operation, 302
. (periods) in dotted-decimal notation, 962
|| [HCL] or operation, 409
< operator for left hoinkies, 945
<< "put to" operator (C++), 926
> operator for right hoinkies, 945
>> "get from" operator (C++), 926
$+^t_w$ (two's-complement addition), 96, 126
$*^t_w$ (two's-complement multiplication), 96, 133
$-^t_w$ (two's-complement negation), 96, 131
$+^u_w$ (unsigned addition), 96, 121, 125
$*^u_w$ (unsigned multiplication), 96, 132
$-^u_w$ (unsigned negation), 96, 125

8086 microprocessor, 203
8087 oating-point coprocessor, 145, 173, 203
80286 microprocessor, 203

.a archive les, 722
a.out object le, 709
Abel, Niels Henrik, 125
abelian group, 125
ABI (application binary interface), 346
abort exception class, 762
aborts, 764
absolute addressing relocation type, 727, 729–730
absolute pathnames, 929
absolute speedup of parallel programs, 1055
abstract operation model for Core i7, 561–567
abstractions, 63
accept [Unix] wait for client connection request, 969, 972, 972–973
access
  disks, 633–636
  IA32 registers, 215–216
  main memory, 623–625
  x86-64 registers
    data movement, 218–225
    operand speci ers, 216–218
access permission bits, 930
access time for disks, 629, 629–631
accumulator variable expansion, 606
accumulators, multiple, 572–577
Acorn RISC machine (ARM)
  ISAs, 388
  processor architecture, 399
actions, signal, 798
active sockets, 971
actuator arms, 628
acyclic networks, 410
adapters, 45, 633
add [instruction class] add, 228
add_client function, 1017, 1019
add every signal to signal set instruction, 801
add instruction, 228
add operation in execute stage, 444
add signal to signal set instruction, 801
adder [CS:APP] CGI adder, 991
addition
  oating point, 158–160, 338
  two's complement, 126, 126–131
  unsigned, 120–126, 121
  Y86-64, 392
additive inverse, 88

addq [Y86-64] add, 392, 438
address exceptions, status code for, 440
address of operator (&) [C]
  local variables, 284
  logic gates, 409
  pointers, 84, 224, 293, 313
address order of free lists, 899
address partitioning in caches, 651, 651–652
address-space layout randomization (ASLR), 321, 321–322
address spaces, 840
  child processes, 777
  linear, 840
  private, 770
  virtual, 840–841
address translation, 840
  caches and VM integration, 853
  Corress if30T* -1.1829d-to-29dntegratio7-304(i130T* -1.18multi-leve Td [(lpagJ T* [(t)-250(variab855-304(5730T* -1.18o)-2-28(

arithmetic, 69, 227
discussion, 232–233
floating-point code, 338–340
integer. See integer arithmetic
latency and issue time, 559
load effective address, 227–229
pointers, 293–294, 909
saturating, 170
shift operations, 94, 140–142, 228, 230–232
special, 233–236
unary and binary, 230–232
arithmetic/logic units (ALUs), 46
combinational circuits, 416
in execute stage, 421
sequential Y86-64 implementation, 444–445
ARM (Acorn RISC machine), 79
ISAs, 388
processor architecture, 399
ARM A7 microprocessor, 389
arms, actuator, 628
ARPA (Advanced Research Projects Administration), 967
ARPANET, 967
arrays, 291
basic principles, 291–293
declarations, 291–292, 299
DRAM, 618
fixed-size, 296–298
machine-code representation, 207
nested, 294–296
pointer arithmetic, 293–294
pointer relationships, 84, 313
stride, 642
variable-size, 298–301
ASCII standard, 39
character codes, 85
limitations, 86
asctime function, 1060
ASLR (address-space layout randomization), 321, 321–322
asm directive, 214
assembler directives, 402
assemblers, 41, 41, 200, 206
assembly code, 41, 200
with C programs, 325–326
formatting, 211–213
Y86-64, 395
assembly phase, 41
associate socket address with descriptor function, 971, 971
associative caches, 660–662
associative memory, 661
associativity
caches, 669
floating-point addition, 159–160
asterisks (*) dereference pointer operation, 224, 293, 313
asymmetric ranges in two's-complement representation, 102, 113
async-signal-safe function, 802
async-signal safety, 802
asynchronous interrupts, 762
atomic reads and writes, 806
ATT assembly code format, 213, 330, 347
argument listing, 342
condition codes, 237–238
cqo instruction, 235
vs. Intel, 213
operands, 217, 228
Y86-64, 392
automatic variables, 1030
AVX (advanced vector extensions) instructions, 312, 330, 582–583
%ax [x86-64] low order 16 bits of register %rax, 216

B2T (binary to two's-complement conversion), 96, 100, 108, 133
B2U (binary to unsigned conversion), 96, 98, 108, 118, 133
background processes, 789, 789–792
backlogs for listening sockets, 971
backups for disks, 647
backward compatibility, 71
backward taken, forward not taken (BTFNT) branch prediction strategy, 464
bad pointers and virtual memory, 906–907
badcnt.c [CS:APP] improperly synchronized program, 1031–1035, 1032
bandwidth, read, 675
Barracuda 7400 drives, 636
base pointers, 326
base registers, 217
bash [Unix] Unix shell program, 789
basic blocks, 605
Bell Laboratories, 71
Berkeley sockets, 968
Berners-Lee, Tim, 985
best-fit block placement policy, 885, 885
bi-endian ordering convention, 79
biased number encoding, 149, 149–153
biasing in division, 142
big-endian ordering convention, 78, 78–80
bigrams statistics, 601
bijections, 100, 100
/bin/kill program, 796
binary files, 39, 927
binary notation, 68
binary points, 146, 146–147
binary representations
conversions
with hexadecimal, 72–73
signed and unsigned, 106–112
to two's complement, 100, 108–109, 133
to unsigned, 98–99
fractional, 145–148
machine language, 230
binary semaphores, 1039
binary tree structure, 306–307
bind [Unix] associate socket address with descriptor, 969, 971, 971
binding, lazy, 742
binutils package, 749
bistable memory cells, 617
bit-level operations, 90–92
bit representation expansion, 112–116
bit vectors, 87, 87–88
bits, 39
overview, 68
union access to, 307–308
bitwise operations, 341–342
%bl [x86-64] low order 8 of register %rbx, 216
block and unblock signals instruction, 801
block devices, 928
block offset bits, 652
block pointers, 892
block size
caches, 669
minimum, 884
blocked bit vectors, 795
blocked signals, 794, 795, 800–801
blocking
signals, 800–801
for temporal locality, 683
blocks
aligning, 880
allocated, 875, 885
vs. cache lines, 670
caches, 647, 647–648, 651, 669
coalescing, 886–887, 896
epilogue, 891
free lists, 883–885
freeing, 896
heap, 875
logical disk, 631, 631–632, 637
prologue, 891

blocks (continued)
referencing data in, 910–911
splitting, 885–886
bodies, response, 988
bool [HCL] bit-level signal, 410
Boole, George, 86
Boolean algebra and functions, 86
HCL, 410–411
logic gates, 409
properties, 88
working with, 86–89
Boolean rings, 88
bottlenecks, 598
profilers, 601–604
program profiling, 598–600
bottom of stack, 226
boundary tags, 887, 887–890, 895
bounded buffers, 1040, 1041–1042
bounds
latency, 554, 560
throughput, 554, 560
%bp [x86-64] low order 16 bits of register %rbp, 216
%bpl [x86-64] low order 8 of register %rbp, 216
branch prediction, 555, 555
misprediction handling, 479–480
performance, 585–589
Y86-64 pipelining, 464
branch prediction logic, 251
branches, conditional, 208, 245
assembly form, 247
condition codes, 237–238
condition control, 245–249
moves, 250–256, 586–589
switch, 268–274
break command
in GDB, 316
with switch, 269
break multstore command in GDB, 316
breakpoints, 315–316
bridged Ethernet, 956, 957
bridges
Ethernet, 956
I/O, 623
browsers, 984, 985
.bss section, 710
BTFNT (backward taken, forward not taken) branch prediction strategy, 464
bubbles, pipeline, 470, 470–471, 495–496
buddies, 901
buddy systems, 901, 901
buffer overflow, 315
execution code regions limits for, 325–326
memory-related bugs, 907
overview, 315–320
stack corruption detection for, 322–325
stack randomization for, 320–322
vulnerabilities, 43
buffered I/O functions, 934–938
buffers
bounded, 1040, 1041–1042
read, 934, 936–937
store, 593–594
streams, 947
bus transactions, 623
buses, 44, 623
designs, 624, 634
I/O, 632
memory, 623
bypassing for data hazards, 472–475
byte data connections in hardware diagrams, 434
byte order, 78–85
disassembled code, 245
network, 961
unions, 308
bytes, 39, 70
copying, 169
range, 72
register operations, 217
Y86 encoding, 395–396
%bx [x86-64] low order 16 bits of register %rbx, 216

C language
bit-level operations, 90–92
floating-point representation, 160–162
history, 71
logical operations, 92–93
origins, 40
shift operations, 93–95
static libraries, 720–724
C++ language, 713
linker symbols, 716
objects, 302–303
software exceptions, 759–760, 822
.c source files, 707
C standard library, 40–41, 42
C11 standard, 71
C90 standard, 71
C99 standard, 71
fixed data sizes, 77
integral data types, 103
cache block offset (CO), 859
cache blocks, 651
cache-friendly code, 669–675, 670
cache lines
cache sets, 651
vs. sets and blocks, 670
cache-oblivious algorithms, 685
cache set index (CI), 859
cache tags (CT), 859
cached pages, 842
caches and cache memory, 646, 651
address translation, 859
anatomy, 667
associativity, 669
cache-friendly code, 669–675, 670
data, 556, 667, 667
direct-mapped. See direct-mapped caches
DRAM, 842

security vulnerability, 136–137
callq [x86-64] procedure call, 277
calls, 53, 763–764
error handling, 773–774
Linux/x86-64 systems, 766–767
in performance, 548–549
canary values, 322–323
canceling mispredicted branch handling, 480
capacity
caches, 651
disks, 627, 627–628
functional units, 559
capacity misses, 649
cards, graphics, 633
carriage return (CR) characters, 928
carry flag condition code, 237, 342
CAS (column access strobe) requests, 619
case expressions in HCL, 414, 414
casting, 80
explicit, 111
floating-point values, 161
pointers, 314, 890
signed values, 106–107
catching signals, 794, 797, 799
cells
DRAM, 618, 619
SRAM, 617
central processing units (CPUs), 45, 45–46
Core i7. See Core i7 microprocessors
early instruction sets, 397
effective cycle time, 638
embedded, 399
Intel. See Intel microprocessors
logic design. See logic design
many-core, 507
multi-core, 52, 60–61, 204, 641, 1008
overview, 388–390
pipelining. See pipelining
RAM, 420
sequential Y86 implementation. See sequential Y86-64 implementation
superscalar, 62, 507, 554
trends, 638–639
Y86. See Y86-64 instruction set architecture
Cerf, Vinton, 967
CERT (Computer Emergency Response Team), 136
CF [x86-64] carry flag condition code, 237, 342
CGI (common gateway interface) program, 989, 989–991
CGI adder function, 991
chains, proxy, 988
char [C] data types, 76, 97
character codes, 85
character devices, 928
check_clients function, 1017, 1020
child processes, 776
creating, 777–779
default behavior, 780
error conditions, 781–782
exit status, 781
reaping, 779, 779–785
waitpid function, 782–785
CI (cache set index), 859
circuits
combinational, 410, 410–416
retiming, 457
sequential, 417
CISC (complex instruction set computers), 397, 397–399
%cl [x86-64] low order 8 of register %rcx, 216
Clarke, Dave, 967
classes
data hazards, 471
exceptions, 762–764
instructions, 218
size, 899
storage, 1030–1031
clear bit in descriptor set macro, 1014
clear descriptor set macro, 1014
clear signal set instruction, 801
client-server model, 954, 954–955
clienterror [CS:APP] Tiny helper function, 995–996
clients
client-server model, 954
telnet, 57
clock signals, 417
clocked registers, 437–438
clocking in logic design, 417–420
close [Unix] close file, 930, 930–931
close operations for files, 927, 930–931
close shared library function, 738
closedir functions, 941
cltq [x86-64] Sign extend %eax to %rax, 221
cmova [x86-64] move if unsigned greater, 253
cmovae [x86-64] move if unsigned greater or equal, 253
cmovb [x86-64] move if unsigned less, 253
cmovbe [x86-64] move if unsigned less or equal, 253
cmove [Y86-64] move when equal, 393
cmovg [x86-64] move if greater, 253, 393
cmovge [x86-64] move if greater or equal, 253, 393
cmovl [x86-64] move if less, 253, 393
cmovle [x86-64] move if less or equal, 253, 393
cmovna [x86-64] move if not unsigned greater, 253
cmovnae [x86-64] move if unsigned greater or equal, 253
cmovnb [x86-64] move if not unsigned less, 253
cmovnbe [x86-64] move if not unsigned less or equal, 253
cmovne [x86-64] move if not equal, 253, 393
cmovng [x86-64] move if not greater, 253
cmovnge [x86-64] move if not greater or equal, 253
cmovnl [x86-64] move if not less, 253
cmovnle [x86-64] move if not less or equal, 253
cmovns [x86-64] move if nonnegative, 253
cmovnz [x86-64] move if not zero, 253
cmovp [x86-64] move if even parity, 360
cmovs [x86-64] move if negative, 253
cmovz [x86-64] move if zero, 253
CMP [instruction class] Compare, 238
cmpb [x86-64] compare byte, 238
cmpl [x86-64] compare double word, 238
cmpq [x86-64] compare double word, 238
cmpw [x86-64] compare word, 238
cmtest script, 501
CO (cache block offset), 859
coalescing blocks, 896
with boundary tags, 887–890
free, 886
memory, 883
Cocke, John, 397
code
performance strategies, 597–598
profilers, 598–600
representing, 85–86
self-modifying, 471
Y86 instructions, 394, 395–396
code motion, 544

code segments, 732, 733–734
Cohen, Danny, 79
cold caches, 648
cold misses, 648
Cold War, 967
collectors, garbage, 875, 902
  basics, 902–903
  conservative, 903, 905–906
  Mark&Sweep, 903–906
column access strobe (CAS) requests, 619
column-major sum function, 672
combinational circuits, 410, 410–416
combinational pipelines, 448–450, 496–498
common gateway interface (CGI) program, 989, 989–991
Compaq Computer Corp. RISC processors, 399
compare byte instruction, 238
compare double precision, 342
compare double word instruction, 238
compare instructions, 238
compare single precision, 342
compare word instruction, 238
comparison operations for floating-point code, 342–345
compilation phase, 41
compilation systems, 42, 42–43
compile time, 706
compile-time interpositioning, 744–745
compiler drivers, 40, 707–708
compilers, 42, 200
  optimizing capabilities and limitations, 534–538
  process, 205–206
  purpose, 207
complement instruction, 228
complex instruction set computers (CISC), 397, 397–399
compulsory misses, 648
computation stages in pipelining, 457–458
computed `goto`, 269
Computer Emergency Response Team (CERT), 136
computer systems, 38
concurrency, 1008
  ECF for, 759
  flow synchronizing, 812–814
  and parallelism, 60
  run, 769
  thread-level, 60–62
concurrent execution, 769
concurrent flow, 769, 769–770
concurrent processes, 51, 52
concurrent programming, 1008–1009
  deadlocks, 1063–1066
  with I/O multiplexing, 1014–1021
  library functions in, 1060–1061
  with processes, 1009–1013
  races, 1061–1063
  reentrancy issues, 1059–1060
  shared variables, 1028–1031
  summary, 1066
  threads, 1021–1028
    for parallelism, 1049–1054
    safety issues, 1056–1058
concurrent programs, 1008
concurrent servers, 1008
  based on prethreading, 1041–1049
  based on processes, 1010–1011
  based on threads, 1027–1028
condition code registers, 207
  hazards, 471
  SEQ timing, 437–438
condition codes, 237, 237–238
  accessing, 238–241
  x86-64, 237
  Y86-64, 391–393
condition variables, 1046
conditional branches, 208, 245
  assembly form, 247
  condition codes, 237–238
  condition control, 245–249
  moves, 250–256, 586–589
  `switch`, 268–274
conflict misses, 649, 658–660
`connect` [Unix] establish connection with server, 970, 970–971
connected descriptors, 972, 972–973
connections
  EOF on, 984
  Internet, 961, 965–967
  I/O devices, 632–633
  persistent, 988
conservative garbage collectors, 903, 905–906
constant words in Y86-64, 395
constants
  floating-point code, 340–341
  free lists, 892–893
  maximum and minimum values, 104
  multiplication, 137–139
  for ranges, 103–104
  Unix, 782
content
  dynamic, 989–990
  serving, 985
  Web, 984, 985–986
context switches, 52, 772–773
contexts, 772
  processes, 52, 768
  thread, 1022, 1029
`continue` command, 316
Control Data Corporation 6600 processor, 558
control dependencies in pipelining, 455, 465
control flow, 758
  exceptional. See exceptional control flow (ECF)
  logical, 768, 768–769
  machine-language procedures, 275
control hazards, 465
control logic blocks, 434, 434, 441, 462
control logic in pipelining, 491
  control mechanism combinations, 496–498
  control mechanisms, 495–496
  design testing and verifying, 501
  implementation, 498–500
  special cases, 491–493
  special conditions, 493–495
control structures, 236–237
  condition codes, 236–241
  conditional branches, 245–249
  conditional move instructions, 250–256
  jumps, 241–245
  loops. See loops
  `switch` statements, 268–274
control transfer, 277–281, 758
controllers
  disk, 631, 631–632
  I/O devices, 45
  memory, 619, 620
conventional DRAMs, 618–620
conversions
  binary
    with hexadecimal, 72–73
    signed and unsigned, 106–112
    to two's complement, 100, 108–109, 133
    to unsigned, 98–99
  floating point, 161, 332–337
  lowercase, 545–547
  number systems, 72–75
convert active socket to listening socket function, 971
convert application-to-network function, 962
convert double precision to integer instruction, 333

convert double precision to quad-word integer instruction, 333
convert double to single precision instruction, 335
convert host and service names function, 973, 973–976
convert host-to-network long function, 961
convert host-to-network short function, 961
convert integer to double precision instruction, 333
convert integer to single precision instruction, 333
convert network-to-application function, 962
convert network-to-host long function, 961
convert network-to-host short function, 961
convert packed single to packed double precision instruction, 334
convert quad-word integer to double precision instruction, 333
convert quad-word integer to single precision instruction, 333
convert quad word to oct word instruction, 234
convert single precision to integer instruction, 333
convert single precision to quad-word integer instruction, 333
convert single to double precision instruction, 334
convert socket address to host and service names function, 976, 976–978
copy_elements function, 136
copy file descriptor function, 945
copy_from_kernel function, 122–123
copy-on-write technique, 871, 871–872
copying
  bytes in memory, 169
  descriptor tables, 945
  text files, 936
Core 2 microprocessors, 204, 624
Core i7 microprocessors, 61
  abstract operation model, 561–567
  address translation, 862–864
  caches, 667
  Haswell, 543
  memory mountain, 677
  Nehalem, 204
  page table entries, 862–864
  QuickPath interconnect, 624
  virtual memory, 861–864
core memory, 793
cores in multi-core processors, 204, 641, 1008
correct signal handling, 806–810
counting semaphores, 1039
CPE (cycles per element) metric, 538, 540, 543–544
cpfile [CS:APP] text file copy, 936
CPI (cycles per instruction)
  five-stage pipelines, 507
  in performance analysis, 500–504
CPUs. See central processing units (CPUs)
cqto [x86-64] convert quad word to oct word, 234, 235
CR (carriage return) characters, 928
CR3 register, 862
Cray 1 supercomputer, 389
create/change environment variable function, 788
create child process function, 776, 777–779
create thread function, 1024
critical path analysis, 534
critical paths, 561, 565
critical sections in progress graphs, 1036
CS:APP
  header files, 782
  wrapper functions, 774, 1077
csapp.c [CS:APP] CS:APP wrapper functions, 774, 1077
csapp.h [CS:APP] CS:APP header file, 774, 782, 1077
csh [Unix] Unix shell program, 789
CT (cache tags), 859
ctest script, 501
ctime function, 1060
ctime_ts [CS:APP] thread-safe non-reentrant wrapper for ctime, 1058
Ctrl+C key
  nonlocal jumps, 821
  signals, 794, 797, 831
Ctrl+Z key, 797, 831
current working directory, 928
cvtsd2ss [x86-64] convert double to single precision, 335
cvtss2sd [x86-64] convert single to double precision, 334
cycles per element (CPE) metric, 538, 540, 543–544
cycles per instruction (CPI)
  five-stage pipelines, 507
  in performance analysis, 500–504
cylinders
  disk, 627
  spare, 632
%cx [x86-64] low order 16 bits of register %rcx, 216

d-caches (data caches), 556, 667
data
  conditional transfers, 250–256
  forwarding, 472–475, 473
  sizes, 75–78
data alignment, 309, 309–312
data caches (d-caches), 556, 667
data dependencies in pipelining, 455, 465–467
data-flow graphs, 561–566
data formats in machine-level programming, 213–215
data hazards, 465
  avoiding, 477–480
  classes, 471
  forwarding for, 472–475
  load/use, 475–477
  stalling, 469–472
  Y86-64 pipelining, 465–469
data memory in SEQ timing, 437
data movement instructions, 218–225
data references
  locality, 642–643
  PIC, 740–741
.data section, 710
data segments, 732
data structures, 301
  data alignment, 309–312
  structures, 301–305
  unions, 305–309
data transfer, procedures, 281–284
data types. See types
database transactions, 955
datagrams, 960
DDD debugger with graphical user interface, 315
DDR SDRAM (double data-rate synchronous DRAM), 622
deadlocks, 1063, 1063–1066
deallocate heap storage function, 877
.debug section, 711
debugging, 315–316
DEC [instruction class] decrement, 228
decimal notation, 68
decimal system conversions, 73–75
declarations
  arrays, 291–292, 299
  pointers, 77

declarations (continued)
public and private, 713
structures, 301–305
p786rs, 351–395

dynamic linkers, 735
dynamic linking, 735, 735–737
dynamic memory allocation
  allocated block placement, 885
  allocator design, 890–892
  allocator requirements and goals, 880–881
  coalescing free blocks, 886–887
  coalescing with boundary tags, 887–890
  explicit free lists, 898–899
  fragmentation, 882
  heap memory requests, 886
  implementation issues, 882–883
  implicit free lists, 883–885
  `malloc` and `free` functions, 876–879
  overview, 875–876
  purpose, 879–880
  segregated free lists, 899–901
  splitting free blocks, 885–886
dynamic memory allocators, 875–876
dynamic RAM (DRAM), 45, 618
  caches, 842, 844, 844–845
  conventional, 618–620
  enhanced, 621–622
  historical popularity, 622
  modules, 620, 621
  vs. SRAM, 618
  trends, 638–639
dynamic Web content, 985
`%dx` [x86-64] low order 16 bits of register `%rdx`, 216

E-way set associative caches, 660–661
`%eax` [x86-64] low order 32 bits of register `%rax`, 216
`%ebp` [x86-64] low order 32 bits of register `%rbp`, 216
`%ebx` [x86-64] low order 32 bits of register `%rbx`, 216
ECF. See exceptional control flow (ECF)
ECHILD return code, 782–783
`echo` [CS:APP] read and echo input lines, 983
`echo` function, 317–318, 323
`echo_cnt` [CS:APP] counting version of `echo`, 1048
`echoclient.c` [CS:APP] echo client, 980–981
`echoserveri.c` [CS:APP] iterative echo server, 972–973, 983
`echoservert.c` [CS:APP] concurrent echo server based on threads, 1027
`echoservert_pre.c` [CS:APP] prethreaded concurrent echo server, 1047
`%ecx` [x86-64] low order 32 bits of register `%rcx`, 216
`%edi` [x86-64] low order 32 bits of register `%rdi`, 216
EDO DRAM (extended data out DRAM), 622
`%edx` [x86-64] low order 32 bits of register `%rdx`, 216
EEPROMs (electrically erasable programmable ROMs), 623
effective addresses, 217, 726
effective cycle time, 638
efficiency of parallel programs, 1055, 1055
EINTR return code, 782
electrically erasable programmable ROMs (EEPROMs), 623
ELF. See executable and linkable format (ELF)
EM64T processors, 204
embedded processors, 399
encapsulation, 958
encodings in machine-level programming, 205–206
  code examples, 208–211
  code overview, 206–207
  formatting, 211–213
  Y86-64 instructions, 394–396
end-of-file (EOF) condition, 927, 984
end of line (EOL) indicators, 928
entry points, 732, 733–734
environment variables lists, 787–788
EOF (end-of-file) condition, 927, 984
EOL (end of line) indicators, 928
ephemeral ports, 966
epilogue blocks, 891
EPIPE error return code, 1000
erasable programmable ROMs (EPROMs), 623
`errno` [Unix] Unix error variable, 1078
error-correcting codes for memory, 618
error handling
  system calls, 773–774
  Unix systems, 1078–1079
  wrappers, 774, 1077, 1079–1081
error-reporting functions, 773
errors
  child processes, 781–782
  link-time, 43
  off-by-one, 908
  race, 812, 812–814
  reporting, 1079
  synchronization, 1031
`%esi` [x86-64] low order 32 bits of register `%rsi`, 216
`%esp` [x86-64] low order 32 bits of stack pointer register `%rsp`, 216
establish connection with server functions, 970, 970–971, 978–980
establish listening socket function, 980, 980
etest script, 501
Ethernet segments, 956, 956
Ethernet technology, 956
EUs (execution units), 554, 556
`eval` [CS:APP] shell helper routine, 790, 791
event-driven programs, 1016
  based on I/O multiplexing, 1016–1021
  based on threads, 1049
events, 759
  scheduling, 799
  state machines, 1016
evicting blocks, 648
exabytes, 75
excepting instructions, 481
exception handlers, 760, 760
exception handling
  in instruction processing, 421
  Y86-64, 399–400, 480–483
exception numbers, 761
exception table base registers, 761
exception tables, 761, 761
exceptional control flow (ECF), 758
  exceptions, 759–767
  importance, 758–759
  nonlocal jumps, 817–822
  process control. See processes
  signals. See signals
  summary, 823
  system call error handling, 773–774
exceptions, 759
  anatomy, 759–760
  asynchronous, 762
  classes, 762–764
  data alignment, 312
  handling, 760–762
  Linux/x86-64 systems, 765–767
  status code for, 440
  synchronous, 763
  Y86, 392
exclamation points ! for NOT operation, 409
EXCLUSIVE-OR Boolean operation, 87
exclusive-or instruction
  x86-64, 228
  Y86-64, 392

EXCLUSIVE-OR operation in execute stage, 444
exclusive-or packed double precision instruction, 341
exclusive-or packed single precision instruction, 341
executable and linkable format (ELF), 709
  executable object files, 731–732
  header tables, 710, 732
  headers, 710–711
  relocation, 726
  symbol tables, 711–715
executable code, 206
executable object files, 40
  creating, 708
  description, 708
  fully linked, 732
  loading, 733–734
  running, 43–44
executable object programs, 40
execute access, 325
execute disable bit, 863
execute stage
  instruction processing, 421, 423–433
  PIPE processor, 489–490
  sequential processing, 436
  sequential Y86-64 implementation, 444–445
  Y86-64 pipelining, 459
execution
  concurrent, 769
  parallel, 770
  speculative, 555, 555, 585–586
  tracing, 423, 430–431, 439
execution code regions, 325–326
execution units (EUs), 554, 556
execve [Unix] load program, 786
  arguments and environment variables, 786–788
  child processes, 735, 737
  loading programs, 733
  running programs, 789–792
  virtual memory, 872–873
exit [C Stdlib] terminate process, 775
exit status, 775, 781
expanding bit representation, 112–116
expansion slots, 633
explicit allocator requirements and goals, 880–881
explicit dynamic memory allocators, 875–876
explicit free lists, 898–899
explicit thread termination, 1024
explicit waiting for, signals, 814–817
explicitly reentrant functions, 1059
exploit code, 320
exponents in floating-point representation, 148
extend_heap [CS:APP] allocator: extend heap, 894
extended data out DRAM (EDO DRAM), 622
extended precision floating-point representation, 173, 173
external exceptions in pipelining, 480
external fragmentation, 882, 882

fall through in switch statements, 269
false fragmentation, 886
fast page mode DRAM (FPM DRAM), 621
fault exception class, 762
faulting instructions, 763
faults, 764
  Linux/x86-64 systems, 765, 868–869
  Y86-64 pipelining caches, 506
FD_CLR [Unix] clear bit in descriptor set, 1013, 1014
FD_ISSET [Unix] bit turned on in descriptor set, 1013, 1014, 1016
FD_SET [Unix] set bit in descriptor set, 1013, 1014
FD_ZERO [Unix] clear descriptor set, 1013, 1014
feedback in pipelining, 455–457, 461
feedback paths, 432, 455
fetch file metadata function, 939
fetch stage
  instruction processing, 420, 423–433
  PIPE processor, 483–485
  SEQ, 440–442
  sequential processing, 436
  Y86-64 pipelining, 459
fetches, locality, 643–644
fgets function, 318
Fibonacci (Pisano), 68
field-programmable gate arrays (FPGAs), 503
FIFOs, 1013
file descriptors, 927
file position, 927
file tables, 772, 942
FILE type, 947
filenames, 927
files, 55
  as abstraction, 63
  anonymous, 869
  binary, 39
  metadata, 939–940
  object. See object files
  register, 46, 207, 394–395, 418–419, 437, 557
  regular, 869
  sharing, 942–944
  system-level I/O. See system-level I/O
  types, 927–929
  Unix, 926, 926–927
FINGER command, 320
fingerd daemon, 320
finish command, 316
firmware, 623
first-fit block placement policy, 885, 885
first-level domain names, 963
first readers-writers problem, 1044
fits, segregated, 899, 900–901
five-stage pipelines, 507
fixed-size arithmetic, 121
fixed-size arrays, 296–298
fixed-size integer types, 77, 103
flash memory, 623
flash translation layers, 636–637
flat addressing, 203
float [C] single-precision floating point, 160
float floating-point declaration, 214
floating-point code
  architecture, 329, 329–332
  arithmetic operations, 338–340
  bitwise operations, 341–342
  comparison operations, 342–345
  constants, 340–341
  movement and conversion operations, 332–337
  observations, 345
  in procedures, 337–338
floating-point representation and programs, 144–145
  arithmetic, 69
  C, 160–162
  denormalized values, 150, 150–152
  encodings, 68
  extended precision, 173, 173
  fractional binary numbers, 145–148
  IEEE, 148–150
  normalized value, 149–150
  operations, 158–160
  overflow, 163
  pi, 176
  rounding, 156, 156–158
  special values, 151
  support, 76
  x87 processors, 203
flows
  concurrent, 769, 769–770
  control, 758
  logical, 768, 768–769
  parallel, 770
  synchronizing, 812–814

flushed instructions, 558
FNONE [Y86-64] default function code, 440
footers of blocks, 887
for [C] general loop statement, 264–268
  guarded-do translation, 261
  jump-to-middle translation, 259
forbidden regions, 1039
foreground processes, 789
fork [Unix] create child process, 776
  child processes, 737
  example, 777–779
  running programs, 789–792
  virtual memory, 872
fork.c [CS:APP] fork example, 777
formal verification in pipelining, 502
format strings, 83
formats for machine-level data, 213–215
formatted disk capacity, 632
formatted printing, 83
formatting
  disks, 632
  machine-level code, 211–213
forwarding
  for data hazards, 472–475
  load, 513
forwarding priority, 487–488
FPGAs (field-programmable gate arrays), 503
FPM DRAM (fast page mode DRAM), 621
fprintf [C Stdlib] function, 83
fractional binary numbers, 145–148
fractional floating-point representation, 148–156, 173
fragmentation, 882
  dynamic memory allocation, 882
  false, 886
frame pointers, 326
frames
  Ethernet, 956
  stack, 276, 276–277, 312, 326–329
free blocks, 875
  coalescing, 886–887
  splitting, 885–886
free bounded buffer function, 1043
free [C Stdlib] deallocate heap storage, 877, 877–879
  interpositioning libraries, 744
  wrappers for, 747
free heap block function, 896
free heap blocks, referencing data in, 910–911
free lists
  creating, 893–895
  dynamic memory allocation, 883–885
  explicit, 898–899
  implicit, 884
  manipulating, 892–893
  segregated, 899–901
free software, 42
free up getaddrinfo resources function, 973
freeaddrinfo [Unix] free up getaddrinfo resources, 973, 974
FreeBSD open-source operating system, 122–123
freeing blocks, 896
Freescale
  processor family, 388
  RISC design, 397
front side bus (FSB), 624
fstat [Unix] fetch file metadata, 939
full duplex connections, 965
full duplex streams, 948
fully associative caches, 662
  line matching and word selection, 663–664
  set selection, 663
fully linked executable object files, 732
fully pipelined functional units, 559
function calls
  performance strategies, 597
  PIC, 741–743
function part in Y86-64 instruction specifier, 394
functional units, 556–557, 559–560
functions
  pointers to, 314
  reentrant, 802, 1059
  static libraries, 720–724
  system-level, 766
  thread-safe and thread-unsafe, 1056, 1056–1058
  wrapper, 747
  in Y86 instructions, 395

gai_error [CS:APP] reports GAI-style errors, 1079
gai_strerror [Unix] print getaddrinfo error message, 974
GAI-style error handling, 1078, 1078–1079
gaps between disk sectors, 626, 632
garbage, 902
garbage collection, 876, 902
  garbage collectors, 876, 902
    basics, 902–903
    conservative, 903, 905–906
    Mark&Sweep, 903–906
  overview, 901–902
gates, logic, 409
GCC (GNU compiler collection) compiler
  code formatting, 211–212
  inline assembly, 214
  options, 71
  working with, 204–205
GDB GNU debugger, 209, 315, 315–316
general protection faults, 765
general-purpose registers, 215, 215–216
geometry of disks, 626–627
get address of shared library symbol function, 738
"get from" operator (C++), 926
GET method in HTTP, 987
get parent process ID function, 775
get process group ID function, 795
get process ID function, 775
get thread ID function, 1024
getaddrinfo [Unix] convert host and service names, 973, 973–976
getenv [C Stdlib] read environment variable, 787
gethostbyaddr [Unix] get DNS host entry, 1060
gethostbyname [Unix] get DNS host entry, 1060
getnameinfo [Unix] convert socket address to host and service names, 976, 976–978
getpeername function [C Stdlib] security vulnerability, 122–123
getpgrp [Unix] get process group ID, 795
getpid [Unix] get process ID, 775
getppid [Unix] get parent process ID, 775
getrusage [Unix] function, 847
gets function, 315, 317–318
GHz (gigahertz), 538
giga-instructions per second (GIPS), 449
gigabytes, 628
gigahertz (GHz), 538
GIPS (giga-instructions per second), 449
global IP Internet. See Internet
Global Offset Table (GOT), 741, 741–743
global symbols, 711
global variable mapping, 1030–1031

GNU compiler collection. See GCC (GNU compiler collection) compiler
GNU project, 42
GOT (global offset table), 741, 741–743
goto [C] control transfer statement, 246, 269
goto code, 246
GPROF Unix profiler, 598, 598–599
gradual underflow, 151
granularity of concurrency, 1021
graphic user interfaces for debuggers, 315
graphics adapters, 632
graphs
  data-flow, 561–566
  process, 777, 778
  progress. See progress graphs
  reachability, 902
greater than signs >
  deferencing operation, 302
  “get from” operator, 926
  right hoinkies, 945
groups
  abelian, 125
  process, 795
guard values, 322
guarded-do translation, 261

.h header files, 722
half-precision floating-point representation, 173, 173
halt [Y86-64] halt instruction execution, 393
  code for, 440–441
  exceptions, 400, 480–483
  in pipelining, 498
handlers
  exception, 760, 760
  interrupt, 762
  signal, 794, 799
handling signals
  blocking and unblocking, 800–801
  portable, 810–811
hardware caches. See caches and cache memory
hardware control language (HCL), 408
  Boolean expressions, 410–411
  integer expressions, 412–416
  logic gates, 409
hardware descri7 1 z6t(sating-point)]TJ 2 9 2 4 8 0 /T1_1 7 s

IA32 (Intel Architecture 32-bit) microprocessors, 81, 204
machine language, 201–202
registers, 215–216
iaddq [Y86-64] immediate add, 405
IBM
Freescale microprocessors, 388, 397
out-of-order processing, 558
RISC design, 397–399
ICALL [Y86-64] instruction code for call instruction, 440
ICANN (Internet Corporation for Assigned Names and Numbers), 963
icode (instruction code), 420, 441
ICUs (instruction control units), 554
identifiers, register, 394
idivl [x86-64] signed divide, 235
idivq [x86-64] signed divide, 234
IDs (identifiers)
processes, 775–776
register, 394–395
IEEE. See Institute for Electrical and Electronics Engineers (IEEE)
if [C] conditional statement, 247–249
ifun (instruction function), 420, 441
IHALT [Y86-64] instruction code for halt instruction, 440
IIRMOVQ [Y86-64] instruction code for irmovq instruction, 440
ijk matrix multiplication, 680–682, 681
IJXX [Y86-64] instruction code for jump instructions, 440
ikj matrix multiplication, 680–682, 681
illegal instruction exceptions, 440
imem_error signal, 441
immediate add instruction, 405
immediate coalescing, 886
immediate offset, 217
immediate operands, 217
immediate to register move instruction, 392
implicit dynamic memory allocators, 876
implicit free lists, 883–885, 884
implicit thread termination, 1024
implicitly reentrant functions, 1059
implied leading 1 representation, 150
IMRMOVQ [Y86-64] instruction code for mrmovq instruction, 440
IMUL [instruction class] multiply, 228
imulq [x86-64] signed multiply, 234, 234
in [HCL] set membership test, 417
in_addr [Unix] IP address structure, 961
INC [instruction class] increment, 228
include files, 722
#include [C] preprocessor directive, 206
incq instruction, 230
increment instruction, 228, 230
indefinite integer values, 161
index.html file, 986
index registers, 217
indexes for direct-mapped caches, 658–660
indirect jumps, 242, 270
inefficiencies in loops, 544–548
inet_ntoa [Unix] convert network-to-application, 1060
inet_ntop [Unix] convert network-to-application, 962
inet_pton [Unix] convert application-to-network, 962
infinity
constants, 160
representation, 150–151
info frame command, 316
info registers command, 316
information, 38–40
information access with x86-64 registers, 215–216
data movement, 218–225
operand specifiers, 216–218
information storage, 70
addressing and byte ordering, 78–85
bit-level operations, 90–92
Boolean algebra, 86–89
code, 85–86
data sizes, 75–78
disks. See disks
floating point. See floating-point representation and programs
hexadecimal, 72–75
integers. See integers
locality. See locality
logical operations, 92–93
memory. See memory
segregated, 899
shift operations, 93–95
strings, 85
summary, 684
init function, 779
init_pool function, 1017, 1019
initial state in progress graphs, 1035
initialize nonlocal handler jump function, 819
initialize nonlocal jump functions, 819
initialize read buffer function, 934, 936
initialize semaphore function, 1038
initialize thread function, 1026
initializing threads, 1026
inline assembly, 214
inline substitution, 537
inlining, 537
INOP [Y86-64] instruction code for nop instruction, 440
input events, 1016
input/output. See I/O (input/output)
insert item in bounded buffer function, 1043
install portable handler function, 811
installing signal handlers, 799
Institute for Electrical and Electronics Engineers (IEEE)
description, 145
floating-point representation and programs, 148–150
denormalized, 150
normalized, 149–150
special values, 151
Standard 754, 145
standards, 145
Posix standards, 52
instr_valid signal, 441–442
instruction caches (i-caches), 554, 667
instruction code (icode), 420, 441
instruction control units (ICUs), 554
instruction function (ifun), 420, 441
instruction-level parallelism, 62, 533, 554, 598
instruction memory in SEQ timing, 437
instruction set architectures (ISAs), 46, 63, 206, 388
instruction set simulators, 402
instructions
classes, 218
decoding, 554
excepting, 481
fetch locality, 643–644
issuing, 463–464
jump, 46, 241–245
load, 46
low-level. See machine-level programming
move, 250–256, 586–589
operate, 46
pipelining, 504–505, 585
privileged, 771
store, 46
update, 45–46
Y86-64. See Y86-64 instruction set architecture
instructions per cycle (IPC), 507
int [C] integer data type, 76

int [HCL] integer signal, 412
int data types, integral, 97
INT_MAX constant, maximum signed integer, 104
INT_MIN constant, minimum signed integer, 104
int32_t [Unix] fixed-size, 77
integer arithmetic, 120, 228
  division by powers of 2, 139–143
  multiplication by constants, 137–139
  overview, 143–144
  two's complement addition, 126–131
  two's complement multiplication, 133–137
  two's complement negation, 131
  unsigned addition, 120–126
integer bits in floating-point representation, 173
integer expressions in HCL, 412–416
integer indefinite values, 161
integer operation instruction, 440
integer registers in x86-64, 215–216
integers, 68, 95–96
  arithmetic operations. See integer arithmetic
  bit-level operations, 90–92
  bit representation expansion, 112–116
  byte order, 79–80
  data types, 96–98
  shift operations, 93–95
  signed and unsigned conversions, 106–112
  signed vs. unsigned guidelines, 119–120
  truncating, 117–118
  two's complement representation, 100–106
  unsigned encoding, 98–100
integral data types, 96, 96–98
integration of caches and VM, 853
Intel assembly-code format, 213, 330, 347
Intel Corporation, 201
Intel microprocessors
  8086, 62, 203
  80286, 203
  Core 2, 204, 624
  Core i7. See Core i7 microprocessors
  data alignment, 312
  evolution, 203–204
  floating-point representation, 173
  Haswell, 204, 251, 330, 559
  i386, 203
  i486, 203
  northbridge and southbridge chipsets, 624
  out-of-order processing, 558
  Pentium, 203
  Pentium II, 203
  Pentium III, 203–204
  Pentium 4, 204
  Pentium 4E, 204
  PentiumPro, 203, 558
  Sandy Bridge, 204
  x86-64. See x86-64 microprocessors
  Y86-64. See Y86-64 instruction set architecture
interconnected networks (internets), 957, 957–958
interfaces
  bus, 624
  host bus, 633
interlocks, load, 477
internal exceptions in pipelining, 480
internal fragmentation, 882
internal read function, 937
International Standards Organization (ISO), 40, 71
Internet, 957
  connections, 965–967
  domain names, 963–965
  IP addresses, 961–963
  organization, 960–961
  origins, 967
internet addresses, 958
Internet Corporation for Assigned Names and Numbers (ICANN), 963
Internet domain names, 961
Internet Domain Survey, 966
Internet hosts, number of, 966
Internet Protocol (IP), 960
Internet Software Consortium, 966
Internet worms, 320
internets (interconnected networks), 957, 957–958
interpositioning libraries, 743, 743–744
  compile-time, 744–745
  link-time, 744, 746
  run-time, 746–748
interpretation of bit patterns, 68
interprocess communication (IPC), 1013
interrupt handlers, 762
interruptions, 800
interrupts, 762, 762–763
interval counting schemes, 600
INTN_MAX [C] maximum value of N-bit signed data type, 103
INTN_MIN [C] minimum value of N-bit signed data type, 103
intN_t [C] N-bit signed integer data type, 103
<inttypes.h> fixed-size integer types, 234
invalid address status code, 400
invariants, semaphore, 1038
I/O (input/output), 45, 926
  memory-mapped, 634
  ports, 634
  redirection, 945, 945–946
  system-level. See system-level I/O
  Unix, 55, 926, 926–927
I/O bridges, 623
I/O buses, 624, 632, 634
I/O devices, 45
  addressing, 634
  connecting, 632–633
I/O multiplexing, 1009
  concurrent programming with, 1014–1021
  event-driven servers based on, 1016–1021
  pros and cons, 1021
IOPL [Y86-64] instruction code for integer operation instruction, 440
IP (Internet Protocol), 960
IP address structure, 961, 962
IP addresses, 960, 961–963
IPC (instructions per cycle), 507
IPC (interprocess communication), 1013
iPhone 5S, 389
IPOPQ [Y86-64] instruction code for popq instruction, 440
IPUSHQ [Y86-64] instruction code for pushq instruction, 440
IPv6, 961
IRET [Y86-64] instruction code for ret instruction, 440
IRMMOVQ [Y86-64] instruction code for rmmovq instruction, 440
irmovq [Y86-64] immediate to register move, 392, 440
IRRMOVQ [Y86-64] instruction code for rrmovq instruction, 440
ISAs (instruction set architectures), 46, 63, 206, 388
ISO (International Standards Organization), 40, 71
ISO C11 C standard, 71
ISO C90 C standard, 71
ISO C99 C standard, 71, 77, 360
  integral data types, 103

less than signs <
  left hoinkies, 945
  "put to" operator, 926
levels
  optimization, 534
  storage, 645–646
LF (line feed) characters, 928
LFU (least-frequently-used) replacement policies, 662
libc library, 947
__libc_start_main, 734
libraries
  in concurrent programming, 1060–1061
  header files, 119
  interpositioning, 743, 743–748
  shared, 55, 735, 735–737
  standard I/O, 947
  static, 720, 720–724
LIFO (last-in first-out) free list order, 899
<limits.h> file for numeric limit declarations, 103–104, 113
line feed (LF) characters, 928
line matching
  direct-mapped caches, 654
  fully associative caches, 662
  set associative caches, 661–662
line replacement
  direct-mapped caches, 655
  set associative caches, 662
.line section, 711
linear address spaces, 840
link-time errors, 43
link-time interpositioning, 744, 746
linkers and linking, 41, 200, 206
  compiler drivers, 707–708
  dynamic, 735, 735–737
  library interpositioning, 743, 743–748
  object files, 709, 709–710
    executable, 731–734
    loading, 733–734
    relocatable, 710–711
    tools for, 749
  overview, 706–707
  position-independent code, 740–743
  relocation, 725–731
  shared libraries from applications, 737–739
  static, 708
  summary, 749–750
  symbol resolution, 715–725
  symbol tables, 711–715
  virtual memory for, 847–848
linking phase, 42
links in directories, 927
Linux operating system, 56, 81
  code segments, 733–734
  dynamic linker interfaces, 738
  and ELF, 709
  exceptions, 765–767
  files, 927–929
  signals, 792
  static libraries, 721–722
  virtual memory, 866–869
Lisp language, 121
listen [Unix] convert active socket to listening socket, 971
listening descriptors, 972–973
listening sockets, 971
little-endian ordering convention, 78, 78–80
load effective address instruction, 227–229, 313
load forwarding in PIPE, 513
load instructions, 46
load interlocks, 477
load operations
  example, 624
  process, 555–556
load penalty in CPI, 503
load performance of memory, 590–591
load program function, 786
load-store architecture in CISC vs. RISC, 398
load time for code, 706
load/use data hazards, 475, 475–477
loaders, 708, 733
loading
  concepts, 735
  executable object files, 733–734
  process, 733
  programs, 786–788
  shared libraries from applications, 737–739
  virtual memory for, 848
local area networks (LANs), 956, 956–958
local automatic variables, 1030
local registers, 563
local static variables, 1030, 1030–1031
local storage
  registers, 287–289
  stack, 284–287
local symbols, 712
locality, 49, 616, 640–641
  blocking for, 683
  caches, 679–683, 846
  exploiting, 683
  forms, 640, 650
  instruction fetches, 643–644
  program data references, 642–643
  summary, 644–645
localtime function, 1060
lock-and-copy technique, 1058, 1058
locking mutexes
  lock ordering rule, 1065
  for semaphores, 1039
logic design, 408
  combinational circuits, 410–416, 449
  logic gates, 409, 409
  memory and clocking, 417–420
  set membership, 416–417
logic gates, 409
logic synthesis, 391, 409, 503
logical blocks
  disks, 631, 631–632
  SSDs, 637
logical control flow, 768, 768–769
logical operations, 92–93, 227
  discussion, 232–233
  load effective address, 227–229
  shift, 94, 140, 228, 230–232
  special, 233–236
  unary and binary, 230
long [C] integer data type, 76–77, 97–98
long double [C] extended-precision floating point, 161, 173
long double floating-point declaration, 214
long words in machine-level data, 215
longjmp [C Stdlib] nonlocal jump, 759, 819, 819
loop registers, 563
loop unrolling, 538, 540, 567
  Core i7, 608
  k × 1, 567
  k × 1a, 580
  k × k, 575–576
  overview, 567–571
  with reassociation transformations, 577–579
loopback addresses, 964
loops, 256
  do-while, 256–259
  for, 264–268
  inefficiencies, 544–548
  reverse engineering, 258
  segments, 562–563
  for spatial locality, 679–683
  while, 259–264
low-level instructions. See machine-level programming
low-level optimizations, 598

lowercase conversions, 545–547
LRU (least-recently-used) replacement policies, 648, 662
ls command, 928
lseek [Unix] function, 932–933
lvalue (C) assignable value for pointers, 313

Mac OS X (Apple Macintosh) operating system, 63
machine checks, 765
machine code, 200
machine-level programming
  arithmetic. See arithmetic
  arrays. See arrays
  buffer overflow. See buffer overflow
  control. See control structures
  data-flow graphs from, 561–565
  data formats, 213–215
  data movement instructions, 218–225
  encodings, 205–213
  floating point. See floating-point code
  GDB debugger, 315–316
  heterogeneous data structures. See heterogeneous data structures
  historical perspective, 202–205
  information access, 215–216
  instructions, 40
  operand specifiers, 216–218
  overview, 200–202
  pointer principles, 314
  procedures. See procedures
  x86-64. See x86-64 microprocessors
macros for storage allocators, 892–893
main memory, 45
  accessing, 623–625
  memory modules, 620
main threads, 1022
malloc [C Stdlib] allocate heap storage, 71, 360, 733, 875–876, 876
  alignment with, 312
  declaration, 170–171
  dynamic memory allocation, 876–879
  interpositioning libraries, 744
  wrappers for, 747
man ascii command, 84
mandatory alignment, 312
mangling process (C++ and Java), 716
many-core processors, 507
map disk object into memory function, 873
mapping
  memory. See memory mapping
  variables, 1030–1031
mark phase in Mark&Sweep, 903
Mark&Sweep algorithm, 902
Mark&Sweep garbage collectors, 903, 903–906
masking operations, 91
matrices
  adjacency, 696
  multiplying, 679–683
maximum floating-point instructions, 338
maximum two's complement number, 102
maximum unsigned number function, 99
maximum values, constants for, 104
McCarthy, John, 902
McIlroy, Doug, 52
media instructions, 330
mem_init [CS:APP] heap model, 891
mem_sbrk [CS:APP] sbrk emulator, 891
membership, set, 416–417
memcpy [Unix] copy bytes from one region of memory to another, 169
memory, 616
  accessing, 623–625
  aliasing, 535, 536
  associative, 661
  caches. See caches and cache memory
  copying bytes in, 169
  data alignment in, 309–312
  data hazards, 471
  design, 420
  dynamic. See dynamic memory allocation
  hazards, 471
  hierarchy, 50, 50, 645–650
  leaks, 911, 1028
  load performance, 590–591
  in logic design, 397–400
  machine-language procedures, 275
  machine-level programming, 206
  main, 45, 620, 623–625
  mapping. See memory mapping
  nonvolatile, 623
  performance, 589–597
  pipelining, 505–506
  protecting, 325, 848–849
  RAM. See random access memory (RAM)
  ROM, 623
  threads, 1029–1030
  trends, 638–640
  virtual. See virtual memory (VM)
  Y86, 392
memory buses, 623
memory controllers, 619, 620
memory management units (MMUs), 840, 843
memory-mapped I/O, 634
memory mapping, 848
  areas, 869, 869
  execve function, 872–873
  fork function, 872
  in loading, 735
  objects, 869–872
  user-level, 873–875
memory mountains, 675
  Core i7 microprocessors, 677
  overview, 675–679
memory references
  operands, 217
  out of bounds. See buffer overflow
  in performance, 550–553
memory stage
  instruction processing, 421, 423–433
  PIPE processor, 490–491
  sequential processing, 436
  sequential Y86-64 implementation, 445–447
  Y86-64 pipelining, 459
memory system, 616
memory utilization, 881, 881
memset function, declaration, 170–171
metadata, 939, 939–940
metastable states, 617
methods
  hypertext transfer protocol, 987–988
  objects, 303
micro-operations, 555
microarchitecture, 46, 553
microprocessors. See central processing units (CPUs)
Microsoft Windows operating system, 81
MIME (multipurpose internet mail extensions) types, 985
minimum block size, 884
minimum floating-point instructions, 338
minimum two's complement number, 102
minimum values
  constants, 104
  two's complement representation, 102

mispredicted branches
handling, 479–480
performance penalties, 503, 556, 585–589
miss rates, 667
misses, caches, 506, 648
kinds, 648–649
penalties, 668, 842
rates, 667
mkdir command, 928
mm_coalesce [CS:APP] allocator: boundary tag coalescing, 896
mm_free [CS:APP] allocator: free heap block, 896
mm-ijk [CS:APP] matrix multiply ijk, 681
mm-ikj [CS:APP] matrix multiply ikj, 681
mm_init [CS:APP] allocator: initialize heap, 894
mm-jik [CS:APP] matrix multiply jik, 681
mm-jki [CS:APP] matrix multiply jki, 681
mm-kij [CS:APP] matrix multiply kij, 681
mm-kji [CS:APP] matrix multiply kji, 681
mm_malloc [CS:APP] allocator: allocate heap block, 896, 897
mmap [Unix] map disk object into memory, 873, 873–875
MMUs (memory management units), 840, 843
MMX media instructions, 203, 330
Mockapetris, Paul, 967
mode bits, 771
modern processor performance, 554–567
modes
kernel, 762, 764
processes, 770–772, 771
user, 762, 764
modified sequential processor implementation, 457–458
modular arithmetic, 121–122, 125
modules
DRAM, 620, 621
object, 709
monitors, Java, 1046
monotonicity assumption, 882
monotonicity property, 160
Moore, Gordon, 205
Moore's Law, 205, 205
MOSAIC browser, 985
motherboards, 45
Motorola RISC processors, 399
MOV [instruction class] move data, 218, 218–219
movabsq [x86-64] move absolute quad word, 219, 219
movb [x86-64] move byte, 219
move absolute quad word instruction, 219, 219
move aligned, packed double precision instruction, 332
move aligned, packed single precision instruction, 332
move and sign-extend instruction, 220, 221
move byte instruction, 219
move data instructions, 218–225

mrmovq instruction, 440
mulq [x86-64] unsigned multiply, 234, 234
multi-core processors, 52, 60–61, 204, 641, 1008
multi-level page tables, 855–857
multi-threading, 53–54, 61
Multics, 52
multicycle instructions, 504–505
multidimensional arrays, 294–296
multiple accumulators in parallelism, 572–577
multiple zone recording, 628
multiplexing, I/O, 1009
  concurrent programming with, 1014–1021
  event-driven servers based on, 1016–1021
  pros and cons, 1021
multiplexors, 410, 410–411
  HCL with case expression, 414
  word-level, 414–416
multiplication
  constants, 137–139
  floating point, 160, 338
  instructions, 234
  matrices, 679–683
  two's complement, 133–137
  unsigned, 132–133, 234, 234
multiply instruction, 228
multiported random access memory, 418
multiprocessor systems, 60
multipurpose internet mail extensions (MIME) types, 985
multitasking, 769
multiway branch statements, 268–274
munmap [Unix] unmap disk object, 875
mutexes
  lock ordering rule, 1065
  Pthreads, 1046
  for semaphores, 1039
mutual exclusion
  progress graphs, 1036
  semaphores for, 1038–1040
mutually exclusive access, 1036

\n (newline character), 39, 927
n-gram statistics, 601
named pipes, 928
names
  domain, 961, 963–965
  mangling and demangling processes (C++ and Java), 716, 716
  protocols, 958
  types, 83
  Y86-64 pipelines, 463
NaN (not a number)
  constants, 160
  floating point, 342
  representation, 150, 151
nanoseconds (ns), 538
National Science Foundation (NSF), 967
need_regids signal, 441
need_valC signal, 441
NEG [instruction class] negate, 228
negate instruction, 228
negation, two's complement, 131
negative overflow, 126, 126–127
nested arrays, 294–296
nested structures, 304
network adapters, 633
network byte order, 961
network clients, 57, 954
Network File System (NFS), 646
network programming, 954
  client-server model, 954–955
  Internet. See Internet
  networks, 955–959
  sockets interface. See sockets interface
  summary, 1000–1001
  TINY Web server, 992–1000
  Web servers, 984–992
network servers, 57, 954
networks, 56–57
  acyclic, 410
  LANs, 956, 956–958
  WANs, 957, 957–958
never taken (NT) branch prediction strategy, 464
newline character (\n), 39, 927
next-fit block placement policy, 885, 885
nexti command, 316
NFS (Network File System), 646
NM tool, 749
no-execute (NX) memory protection, 325
no operation nop instruction, 322, 440
  instruction code for, 441
  pipelining, 466–467
  in stack randomization, 322
no-write-allocate approach, 666
nodes, root, 902
nondeterminism, 784
nondeterministic behavior, 784
nonexistent variables, referencing, 910
nonlocal jumps, 759, 817, 817–822
nonuniform partitioning, 452–454
nonvolatile memory, 622
nop [x86-64] no operation instruction, 322, 440
  instruction code for, 441
  pipelining, 466–467
  in stack randomization, 322
nop sleds, 322
norace.c [CS:APP] Pthreads program without a race, 1063
normal operation status code, 400, 440
normalized values, floating-point, 149, 149–150
northbridge chipsets, 624
not a number (NaN)
  constants, 160
  floating point, 342
  representation, 150, 151
NOT [instruction class] complement, 228
NOT operation
  Boolean, 87–88
  C operators, 92–93
  logic gates, 409
ns (nanoseconds), 538
NSF (National Science Foundation), 967
NSFNET, 967
NSLOOKUP program, 964
ntohl [Unix] convert network-to-host long, 961
ntohs [Unix] convert network-to-host short, 961
number systems conversions. See conversions
numeric limit declarations, 113
numeric ranges
  C standards, 97
  integral types, 96–98
  Java standard, 104
NX (no-execute) memory protection, 325

.o files, 209, 708
-O1 optimization flag, 206
-O2 optimization flag, 206
OBJDUMP GNU machine-code file reader, 209, 315, 728, 749
object code, 206, 209
object files, 209
  executable. See executable object files
  formats, 709
  forms, 709
  relocatable, 41, 708, 709–711
  shared, 709
  tools, 749
object modules, 709

objects
C++ and Java, 302
memory-mapped, 869–872
private, 870, 870
program, 70
shared, 735, 869–872
as struct, 302–303
oct word, 233, 233–234
OF [x86-64] overflow flag condition code, 237, 391
off-by-one errors, 908
offsets
GOTs, 741, 741–743
memory references, 217
PPOs, 850
unions, 306
VPOs, 850
-Og optimization flag, 206, 599
one-operand multiply instructions, 234
ones'-complement representation, 104
open [Unix] open file, 927, 929–931
open_clientfd [CS:APP] establish connection with server, 978, 978–980
open_listenfd [CS:APP] establish a listening socket, 980, 980
open operations for files, 927, 929–931
open shared library function, 737
open-source operating systems, 122–123
opendir functions, 941
operand specifiers, 216–218
operate instruction, 46
operating systems (OS), 51
files, 55
hardware management, 50–51
kernels, 55
Linux, 56, 81
processes, 51–53
threads, 53–54
Unix, 71
virtual memory, 54–55
Windows, 81
operations
bit-level, 90–92
logical, 92–93
shift, 93–95
optest script, 501
optimization
address translation, 866
compiler, 206
levels, 534
program performance. See performance
optimization blockers, 532–533, 536
OPTIONS method, 987
OR [instruction class] or, 228
OR operation
Boolean, 87–88
C operators, 92–93
HCL expressions, 410–411
logic gates, 409
order, bytes, 78–85
disassembled code, 246
network, 961
unions, 308
origin servers, 988
OS. See operating systems (OS)
Ossanna, Joe, 52
out-of-bounds memory references. See buffer overflow
out-of-order execution, 554
five-stage pipelines, 507
history, 558
overflow
arithmetic, 123, 123–125, 170
buffer. See buffer overflow
floating-point values, 163
identifying, 128–129
infinity representation, 151
multiplication, 138
negative, 126, 126–127
operations, 68
positive, 126, 126–127
overflow flag condition code, 237, 391
overloaded functions (C++ and Java), 716

P semaphore operation, 1037, 1037–1038
P [CS:APP] wrapper function for Posix sem_wait, 1038
P6 microarchitecture, 203
PA (physical addresses), 839
vs. virtual, 839–840
Y86-64, 392
packages, processor, 861
packet headers, 958
packets, 958
padding
alignment, 310–311
blocks, 883
page faults
DRAM caches, 844, 844–845
Linux/x86-64 systems, 765, 869–869
memory caches, 506
pipelining caches, 844
page frames, 841
page hits in caches, 844
page table base registers (PTBRs), 850
page table entries (PTEs), 843, 843–844
Core i7, 862–864
TLBs for, 853–857, 859
page table entry addresses (PTEAs), 853
page tables, 772, 859
caches, 842–844, 843
multi-level, 855–857
paged-in pages, 845
paged-out pages, 845
pages
allocation, 846
demand zero, 869
dirty, 863
physical, 841, 841–842
SSDs, 637
virtual, 325, 841, 841–842
paging
demand, 846
description, 845
parallel execution, 770
parallel flows, 770, 770
parallel programs, 1049
parallelism, 60, 572
instruction-level, 62, 533, 554, 598
multiple accumulators, 572–577
reassociation transformations, 577–582
SIMD, 62, 582–583
thread-level, 62
threads for, 1049–1054
parent directories, 928
parent processes, 775, 775–776
parity flag condition code, 214, 342
parse_uri [CS:APP] TINY helper function, 996
parseline [CS:APP] shell helper routine, 792
partitioning
addresses, 651–652
nonuniform in pipelining, 452–454
passing data
machine-language procedures, 275
pointers to structures, 302
pathnames, 929
Patterson, David, 397, 507
pause [Unix] suspend until signal arrives, 786
payloads
aggregate, 881
Ethernet, 956
protocol, 958
PC. See program counters (PCs)
PC-relative addressing
jumps, 243, 243–245

symbol references, 726, 728–729
Y86-64, 395
PC selection stage in PIPE processor, 483–485
PC update stage
instruction processing, 421, 423–431
sequential processing, 436
sequential Y86-64 implementation, 447
PCI (peripheral component interconnect), 634
PCIe (PCI express), 634
PE (Portable Executable) format, 709
peak utilization metric, 880–881, 881
peer threads, 1022
pending bit vectors, 795
pending signals, 794
Pentium II microprocessor, 203
Pentium III microprocessor, 203–204
Pentium 4 microprocessor, 204
Pentium 4E microprocessor, 204
Pentium microprocessor, 203
PentiumPro microprocessor, 203, 558
performance, 42
Amdahl's law, 58–60
basic strategies, 597–598
bottlenecks, 598–604
branch prediction and misprediction penalties, 585–589
caches, 589, 667–669, 675–683
compiler capabilities and limitations, 534–538
expressing, 538–540
limiting factors, 584–589
loop inefficiencies, 544–548
loop unrolling, 567, 567–571
memory, 589–597
memory references, 550–553
modern processors, 554–567
overview, 532–534
parallelism. See parallelism
procedure calls, 548–549
program example, 540–544
program profiling, 598–600
register spilling, 584–585
results summary, 583–584
sequential Y86-64 implementation, 448
summary, 604–605
Y86-64 pipelining, 500–504
periods (.) in dotted-decimal notation, 962
persistent connections in HTTP, 988
PF [x86-64] parity flag condition code, 213–214, 342
physical address spaces, 840
physical addresses (PA), 839
vs. virtual, 839–840
Y86-64, 392
physical page numbers (PPNs), 850
physical page offset (PPO), 850
physical pages (PPs), 841, 841–842
pi in floating-point representation, 176
PIC (position-independent code), 740
data references, 740–741
function calls, 741–743
picoseconds (ps), 449, 538
PIDs (process IDs), 775
pins, DRAM, 618–619
PIPE− processor, 457, 458, 462–466
PIPE processor stages, 475–476, 483
decode and write-back, 485–489
execute, 489–490
memory, 490–491
PC selection and fetch, 483–485
pipelining, 62, 251, 448
bubble, 470
combinational, 448–450
deep, 454–455
diagram, 449
five-stage, 507
functional units, 559–560
instruction, 585
limitations, 452–454
nonuniform partitioning, 452–454
operation, 450–452
registers, 449, 463
store operation, 591–592
systems with feedback, 455–457
Y86-64. See Y86-64 pipelined implementations
pipes, 1013
Pisano, Leonardo (Fibonacci), 68
placement
memory blocks, 883, 885
policies, 648, 885
platters, disk, 626, 627
PLT (procedure linkage table), 742, 742–743
PMAP tool, 822
point-to-point connections, 965
pointers, 70
arithmetic, 293–2950(numbecm4/T1_2 1 Tf 1.52h50(partitioning,)-250(452–4418
phy0-250(feedbaT* [(Y8d [(ph* [(PC)-250(s906.52h909965)Tj /T42 1 Tf -46824
Pisa82 644182 of226454965 885
Pisa83iti46 [(limitatiositi46 [(iti46 x79 -1.1i46 m)-21 -1.147(408250(feedbaa83iti)]TJ T* [(sy64418
m0 le, PEl)50(485f(4mat [arithm9250(feedbaa8r 0 le,)-250sig60)]TJ Th85)l,)--1.182 T1-250(81-250(a
Pisr]TJ T* ,f TsTd [(ph418(T)-2j /T42 1 Tf -46824 00f -46824398.3941 013.6301 Tm (.pos Td [(poi
five-stag 1.751_6 1 Tf -2.J PPO,physic0742five-stag 2,
965PisPPs,physic0742

principle of locality, 640, 640
print command, 316
print getaddrinfo error message function, 974
printf [C Stdlib] formatted printing function
  formatted printing, 83
  numeric values with, 111
printing, formatted, 83
priorities
  PIPE processor forwarding sources, 487–488
  write ports, 444
private address space, 770
private areas, 870
private copy-on-write structures, 872
private declarations (C++ and Java), 713
private objects, 870, 870
privileged instructions, 771
/proc filesystem, 771, 771–772, 822
procedure linkage table (PLT), 742, 742–743
procedure return instruction, 393
procedures, 274–275
  call performance, 548–549
  control transfer, 277–281
  data transfer, 281–284
  floating-point code in, 337–338
  recursive, 289–291
  register usage conventions, 287–289
  run-time stack, 275–277
process contexts, 52, 772
process graphs, 777, 778
process groups, 795
process IDs, 775
process tables, 772
processes, 51, 768, 774
  background, 789
  child, 776
  concurrent flow, 768–770, 769
  concurrent programming with, 1009–1013
  concurrent servers based on, 1010–1011
  context switches, 772–773
  creating and terminating, 775–779
  default behavior, 780
  error conditions, 781–782
  exit status, 781
  foreground, 789
  group, 795
  IDs, 775–776
  loading programs, 735, 786–788
  overview, 51–53
  parent, 775, 776
  preempted, 769
  private address space, 770
  vs. programs, 789
  pros and cons, 1011
  reaping, 779, 779–785
  running programs, 786–792
  sleeping, 785–786
  tools, 822–823
  user and kernel modes, 770–771
  waitpid function, 782–785
  zombie, 779
processor-memory gap, 49, 640
processor packages, 861
processor states, 759
processors. See central processing units (CPUs)
producer-consumer problem, 1040, 1041–1042
profilers code, 533
profiling, program, 598–600
program counters (PCs), 45, 80
  in fetch stage, 420
  hazards, 471
  machine-language procedures, 275
  %rip, 207
  SEQ timing, 437
  Y86-64 instruction set architecture, 392
  Y86-64 pipelining, 459, 463–465
program data references locality, 642–643
program header tables, 732, 732
program registers
  clocked, 417–420
  data hazards, 471
  Y86-64, 391–392
programmable ROMs (PROMs), 623
programmer-visible state, 391, 391–392
programs
  code and data, 54
  concurrent. See concurrent programming
  forms, 40–41
  loading and running, 786–788
  machine-level. See machine-level programming
  objects, 70
  vs. processes, 789
  profiling, 598–600
  running, 46–48, 789–792
  Y86-64, 400–406
progress graphs, 1035, 1035–1037
  deadlock regions, 1063–1064, 1064
  forbidden regions, 1039
  limitations, 1040
prologue blocks, 891
PROMs (programmable ROMs), 623
protection, memory, 848–849
protocol software, 958
protocols, 958
proxy caches, 988
proxy chains, 988
ps (picoseconds), 449, 538
ps tool, 822
pseudorandom number generator functions, 1057
psum-array.c [CS:APP] parallel sum program using array, 1052
psum-local.c [CS:APP] parallel sum program using local variables, 1053
psum-mutex.c [CS:APP] parallel sum program using mutex, 1051
PTBRs (page table base registers), 850
PTEAs (page table entry addresses), 853
PTEs (page table entries), 843, 843–844
  Core i7, 862–864
  TLBs for, 853–857, 859
pthread_cancel [Unix] terminate another thread, 1025
pthread_create [Unix] create a thread, 1024
pthread_detach [Unix] detach thread, 1026, 1026
pthread_exit [Unix] terminate current thread, 1025
pthread_join [Unix] reap a thread, 1025
pthread_once [Unix] initialize a thread, 1026, 1048
pthread_self [Unix] get thread ID, 1024
Pthreads, 1023, 1023–1024, 1046
public declarations (C++ and Java), 713
push instructions in x86-64 models, 408
push operations on stack, 225, 225–227
pushq [x86-64] push quad word, 209, 226, 226, 393
  code for, 440
  processing steps, 406–407, 428
  run-time stack, 275
PUT method in HTTP, 987
"put to" operator (C++), 926

qsort function, 602
quad words, 213
QuickPath interconnect, 624, 862
quit command, 316

R_X86_64_32 (absolute addressing), 727
R_X86_64_PC32 (PC-relative addressing), 726
  symbol table entry, 713
  and Unix, 709
%r8 [Y86-64] program register, 216, 391
%r8d [x86-64] low order 32 bits of register %r8, 216
%r8w [x86-64] low order 16 bits of register %r8, 216
%r9 [Y86-64] program register, 216, 391
%r9d [x86-64] low order 32 bits of register %r9, 216
%r9w [x86-64] low order 16 bits of register %r9, 216
%r10 [Y86-64] program register, 216, 391
%r10d [x86-64] low order 32 bits of register %r10, 216
%r10w [x86-64] low order 16 bits of register %r10, 216
%r11 [Y86-64] program register, 216, 391
%r11d [x86-64] low order 32 bits of register %r11, 216
%r11w [x86-64] low order 16 bits of register %r11, 216
%r12 [Y86-64] program register, 216, 391
%r12d [x86-64] low order 32 bits of register %r12, 216
%r12w [x86-64] low order 16 bits of register %r12, 216
%r13 [Y86-64] program register, 216, 391
%r13d [x86-64] low order 32 bits of register %r13, 216
%r13w [x86-64] low order 16 bits of register %r13, 216
%r14 [Y86-64] program register, 216, 391
%r14d [x86-64] low order 32 bits of register %r14, 216
%r14w [x86-64] low order 16 bits of register %r14, 216
%r15 [x86-64] program register, 216, 391
%r15d [x86-64] low order 32 bits of register %r15, 216
%r15w [x86-64] low order 16 bits of register %r15, 216
race.c [CS:APP] program with a race, 1061
race conditions, 812, 1028
  concurrent programming, 1061, 1061–1063
  signals, 812–814
RAM. See random access memory (RAM)
rand [CS:APP] pseudorandom number generator, 1057, 1060
rand_r function, 1060
random access memory (RAM), 417, 617
  dynamic. See dynamic RAM (DRAM)
  multiported, 418
  processors, 420
  SEQ timing, 437
  static. See static RAM (SRAM)
random operations in SSDs, 636
random replacement policies, 648
ranges
  asymmetric, 102, 113
  bytes, 72
  constants for, 103–104
  data types, 76
  integral types, 96–98
  Java standard, 104
RAS (row access strobe) requests, 619
%rax [Y86-64] program register, 216, 391
%rbp [Y86-64] program register, 216, 391
%rbx [Y86-64] program register, 216, 391
%rcx [Y86-64] program register, 216, 391
%rdi [Y86-64] program register, 216, 391
%rdx [Y86-64] program register, 216, 391
reachability graphs, 902
reachable nodes, 902
read access, 325
read and echo input lines function, 983
read bandwidth, 675
read environment variable function, 787
read/evaluate steps, 789
read [Unix] read file, 931, 931–933
read-only memory (ROM), 622
read-only register, 563
read operations
  buffered, 934, 936–937
  disk sectors, 633–635
  file metadata, 939–940
  files, 927, 931–933
  SSDs, 637
  unbuffered, 933–934
  uninitialized memory, 907
read ports, 418
read_requesthdrs [CS:APP] Tiny helper function, 996
read sets, 1014
read throughput, 675
read transactions
  descriptions, 623
  example of, 624–625
read/write heads, 628
readdir functions, 941
READELF GNU object file reader, 714, 749
readers-writers problem, 1042, 1044
reading
  directory contents, 941–942
  disk sectors, 633
readline function, 939
readn function, 939
ready read descriptors, 1014
ready sets, 1014
realloc function, 877
reap thread function, 1025
reaping
  child processes, 779, 779–785
  threads, 1025
rearranging signals in pipelining, 462–463
reassociation transformations, 577, 577–582, 606
receiving signals, 794, 798–800
recording density, 627
recording zones, 628
recursive procedures, 289–291
redirection of I/O, 945, 945–946
reduced instruction set computers (RISC), 397
  vs. CISC, 397–399
  SPARC processors, 507
reentrancy issues, 1059–1060
reentrant functions, 802, 1059
reference bits, 863
reference counts, 942
reference machines, 543
referencing
  data in free heap blocks, 910–911

referencing (continued)
nonexistent variables, 910
refresh, DRAM, 618
regions, deadlock, 1063–1064, 1064
register files, 46, 394
contents, 418–419, 557
purpose, 394–395
SEQ timing, 437
register identifier (ID), 394–395
register operands, 217
register specifier bytes in Y86-64 instruction, 394
register to memory move instruction, 392
register to register move instruction, 392
registers, 45
clocked, 417
data hazards, 471
data transfer, 281–284
hardware, 417–420
local, 563
local storage, 287–289
loop, 563
pipeline, 449, 463
program, 391–392, 417–420, 471
read-only, 563
register files, 207
renaming, 558
spilling, 584–585
updating conventions, 215
write-only, 563
x86-64 integer, 215, 215–216
Y86-64, 395, 458–462
regular files, 869, 927
.rel.data section, 711
.rel.text section, 711
relabeling signals, 462–463
relative pathnames, 929
relative speedup in parallel programs, 1055
reliable connections, 966
relocatable object files, 41, 708, 709–711
relocation, 709, 725–726
algorithm, 727
entries, 726, 726–727
PC-relative references, 728–729
practice problems, 730–731
remove item from bounded buffer function, 1043
renaming registers, 558
rep [x86-64] string repeat instruction used as no-op, 244
replacement policies, 649
replacing blocks, 648
report shared library error function, 738
reporting errors, 1079
request headers in HTTP, 987
request lines in HTTP, 987
requests
client-server model, 954
HTTP, 987, 987–988
requests for comments (RFCs), 1001
reset configuration in pipelining, 496
resident sets, 846
resources
client-server model, 954
shared, 1040–1044
RESP [Y86-64] register ID for %rsp, 440
response bodies in HTTP, 988
response headers in HTTP, 988
response lines in HTTP, 988
responses
client-server model, 954
HTTP, 988, 988–989
restart.c [CS:APP] nonlocal jump example, 821
restrictions, alignment, 309–312
ret [Y86-64] procedure return, 393
ret [x86-64] return from procedure call, 244, 277–278
ret instruction, 440
processing steps, 431
Y86-64 pipelining, 464–465, 491–493, 497–498
retiming circuits, 457
retirement units, 557
retq [x86-64] return from procedure, 277
return addresses, 277
predicting, 465
procedures, 276
return penalty in CPI, 503
reverse engineering
loops, 258
machine code, 201
revolutions per minute (RPM), 626
RFCs (requests for comments), 1001
ridges in memory mountains, 677
right hoinkies (>), 946
right shift operations, 93–94, 228
rings, Boolean, 88
RIO [CS:APP] Robust I/O package, 933
buffered functions, 934–938
origins, 939
unbuffered functions, 933–934
rio_read [CS:APP] internal read function, 937
rio_readinitb [CS:APP] init read buffer, 934, 936
rio_readlineb [CS:APP] robust buffered read, 934, 938
rio_readn [CS:APP] robust unbuffered read, 933, 933–935, 937, 939
rio_readnb [CS:APP] robust buffered read, 934, 938
rio_t [CS:APP] read buffer, 936
rio_writen [CS:APP] robust unbuffered write, 933, 933–935, 939
rip [x86-64] program counter, 207
%rip program counter, 207
RISC (reduced instruction set computers), 397
vs. CISC, 397–399
SPARC processors, 507
Ritchie, Dennis, 38, 40, 52, 71, 950
rmdir command, 928
rmmovq [Y86-64] register to memory move, 392, 426, 440
RNONE [Y86-64] ID for indicating no register, 440
Roberts, Lawrence, 967
robust buffered read functions, 934, 938
Robust I/O (RIO) package, 933
buffered functions, 934–938
origins, 939
unbuffered functions, 933–934
robust unbuffered read function, 933, 933–935
robust unbuffered write function, 933, 933–935
.rodata section, 710
ROM (read-only memory), 622
root directory, 928
root nodes, 902
rotating disks term, 627
rotational latency of disks, 630
rotational rate of disks, 626
round-down mode, 157, 157
round-to-even mode, 156, 156–157, 160
round-to-nearest mode, 156, 156
round-toward-zero mode, 156, 156–157
round-up mode, 157, 157
rounding
in division, 141–142
floating-point representation, 156–158
rounding modes, 156, 156–158
routers, Ethernet, 957

routines, thread, 1023
row access strobe (RAS) requests, 619
row-major array order, 294, 642
row-major sum function, 671, 671
RPM (revolutions per minute), 626
rrmovq [Y86-64] register to register move, 392, 440
%rsi [x86-64] program register, 216
%rsp [Y86-64] stack pointer program register215–216, 391
run command, 316
run concurrency, 769
run time
  interpositioning, 746–748
  linking, 706
  shared libraries, 735
  stacks, 207, 275–277
running
  in parallel, 770
  processes, 775
  programs, 46–48, 786–792

.s assembly language files, 708
SA [CS:APP] shorthand for struct sockaddr, 969
SADR [Y86-64] status code for address exception, 440
safe optimization, 534, 534–535
safe signal handling, 802–806
safe trajectories in progress graphs, 1036
safely emit error message and terminate instruction, 802, 804
safely emit long int instruction, 802, 804
safely emit string instruction, 802, 804
SAL [instruction class] shift left, 228
salb [x86-64] shift left, 231
salq [x86-64] shift left, 231
salw [x86-64] shift left, 231
Sandy Bridge microprocessor, 204
SAOK [Y86-64] status code for normal operation, 440
SAR [instruction class] shift arithmetic right, 228, 231
SATA interfaces, 633
saturating arithmetic, 170
sbrk [C Stdlib] extend the heap, 877, 877
  emulator, 891
  heap memory, 886
Sbuf [CS:APP] shared bounded buffer package, 1041, 1042
sbuf_deinit [CS:APP] free bounded buffer, 1043
sbuf_init [CS:APP] allocate and init bounded buffer, 1043
sbuf_insert [CS:APP] insert item in a bounded buffer, 1043
sbuf_remove [CS:APP] remove item from bounded buffer, 1043
sbuf_t [CS:APP] bounded buffer used by SBUF package, 1042
scalar code performance summary, 583–584
scalar format data, 330
scalar instructions, 332
scale factor in memory references, 217
scaling parallel programs, 1055, 1055–1056
scanf function, 906–907
schedule alarm to self function, 798
schedulers, 772
scheduling, 772
  events, 799
  shared resources, 1040–1044
SCSI interfaces, 633
SDRAM (synchronous DRAM), 622
second-level domain names, 964
second readers-writers problem, 1044
sectors, disk, 626, 626–628
  access time, 629–631
  gaps, 632
  reading, 633–635
security monoculture, 321
security vulnerabilities, 43
  getpeername function, 122–123
  XDR library, 136
seeds for pseudorandom number generators, 1057
seek operations, 629, 927
seek time for disks, 629, 629
segmentation faults, 765
segmented addressing, 323–324
segments
  code, 732, 733–734
  data, 732
  Ethernet, 956, 956
  loops, 562–563
  virtual memory, 866
segregated fits, 899, 900–901
segregated free lists, 899–901
segregated storage, 899
select [Unix] wait for I/O events, 1013
self-loops, 1016
self-modifying code, 471
sem_init [Unix] initialize semaphore, 1038
sem_post [Unix] V operation, 1038
sem_wait [Unix] P operation, 1038
semaphores, 1037, 1037–1038
  concurrent server example, 1041–1049
  for mutual exclusion, 1038–1040
  for scheduling shared resources, 1040–1044
sending signals, 771, 795–798
separate compilation, 706
SEQ+ pipelined implementations, 457, 457–458
SEQ Y86-64 processor design. See sequential Y86-64 implementation
sequential circuits, 417
sequential execution, 236–237
sequential operations in SSDs, 636
sequential reference patterns, 642
sequential Y86-64 implementation, 420, 457
  decode and write-back stage, 442–444
  execute stage, 444–445
  fetch stage, 440–442
  hardware structure, 432–436
  instruction processing stages, 420–431
  memory stage, 445–447
  PC update stage, 447
  performance, 448
  SEQ+ implementations, 457, 457–458
  timing, 436–439
serve_dynamic [CS:APP] TINY helper function, 999–1000
serve_static [CS:APP] TINY helper function, 997–999
servers, 57
  client-server model, 954
  concurrent. See concurrent servers
  network, 57
  Web. See Web servers
service conversions in sockets interface, 973–978
services in client-server model, 954
serving
  dynamic content, 989–990
  Web content, 985
set associative caches, 660
  line matching and word selection, 661–662
  line replacement, 661
  set selection, 661, 661
set bit in descriptor set macro, 1014
set index bits, 651, 651–652
set on equal instruction, 239
set on greater instruction, 239

set on greater or equal instruction, 239
set on less instruction, 239
set on less or equal instruction, 239
set on negative instruction, 239
set on nonnegative instruction, 239
set on not equal instruction, 239
set on not greater instruction, 239
set on not greater or equal instruction, 239
set on not less instruction, 239
set on not less or equal instruction, 239
set on not zero instruction, 239
set on unsigned greater instruction, 239
set on unsigned greater or equal instruction, 239
set on unsigned less instruction, 239
set on unsigned less or equal instruction, 239
set on unsigned not greater instruction, 239
set on unsigned not less instruction, 239
set on unsigned not less or equal instruction, 239
set on zero instruction, 239
set process group ID function, 795
set selection
  direct-mapped caches, 654
  fully associative caches, 661
  set associative caches, 661
seta [x86-64] set on unsigned greater, 239
setae [x86-64] set on unsigned greater or equal, 239
setb [x86-64] set on unsigned less, 239
setbe [x86-64] set on unsigned less or equal, 239
sete [x86-64] set on equal, 239
setenv [Unix] create/change environment variable, 788
setg [x86-64] set on greater, 239
setge [x86-64] set on greater or equal, 239
setjmp [C Stdlib] init nonlocal jump, 759, 817, 819
setjmp.c [CS:APP] nonlocal jump example, 820
setl [x86-64] set on less, 239
setle [x86-64] set on less or equal, 239
setna [x86-64] set on unsigned not greater, 239
setnae [x86-64] set on unsigned not less or equal, 239
setnb [x86-64] set on unsigned not less, 239
setnbe [x86-64] set on unsigned not less or equal, 239
setne [x86-64] set on not equal, 239
setng [x86-64] set on not greater, 239
setnge [x86-64] set on not greater or equal, 239
setnl [x86-64] set on not less, 239
setnle [x86-64] set on not less or equal, 239
setns [x86-64] set on nonnegative, 239
setnz [x86-64] set on not zero, 239
setpgid [Unix] set process group ID, 795
sets
  vs. cache lines, 670
  membership, 416–417
sets [x86-64] set on negative, 239
setz [x86-64] set on zero, 239
SF [x86-64] sign flag condition code, 237, 391
sh [Unix] Unix shell program, 789
Shannon, Claude, 87
shared areas, 870
shared libraries, 55, 735
  dynamic linking with, 735–737
  loading and linking from applications, 737–739
shared object files, 709
shared objects, 735, 869–872, 870
shared resources, scheduling, 1040–1044
shared variables, 1028–1031, 1029
sharing
  files, 942–944
  virtual memory for, 848
sharing.c [CS:APP] sharing in Pthreads programs, 1029
shellex.c [CS:APP] shell main routine, 790
shells, 43, 789
shift arithmetic right instruction, 228
shift left instruction, 228
shift logical right instruction, 228
shift operations, 93, 93–95
  for division, 139–143
  machine language, 230–232
  for multiplication, 137–139
  shift arithmetic right instruction, 228
  shift left instruction, 228
  shift logical right instruction, 228
SHL [instruction class] shift left, 228, 231
SHLT [Y86-64] status code for halt, 440
short counts, 931
short [C] integer data type, 76, 97
SHR [instruction class] shift logical right, 228, 231
%si [x86-64] low order 16 bits of register %rsi, 216
side effects, 536
sig_atomic_t type, 806
sigaction [Unix] install portable handler, 811
sigaddset [Unix] add signal to signal set, 801
sigdelset [Unix] delete signal from signal set, 801
sigemptyset [Unix] clear a signal set, 801
sigfillset [Unix] add every signal to signal set, 801
sigint.c [CS:APP] catches SIGINT signal, 799
sigismember [Unix] test signal set membership, 801
siglongjmp [Unix] init nonlocal jump, 819, 821
sign bits
  floating-point representation, 173
  two's complement representation, 100
sign extension, 113, 113, 219–220
sign flag condition code, 237, 391
sign-magnitude representation, 104
Signal [CS:APP] portable version of signal, 811
signal handlers, 794
  installing, 799
  writing, 802–811
  Y86-64, 400
signal1.c [CS:APP] flawed signal handler, 807
signal2.c [CS:APP] flawed signal handler, 808
signals, 758, 792–794
  blocking and unblocking, 800–801
  correct handling, 806–810
  enabling and disabling, 88
  flow synchronizing, 812–814
  portable handling, 810–811
  processes, 775
  receiving, 798, 798–800
  safe handling, 802–806
  sending, 794, 795–798
  terminology, 794–795
  waiting for, 814–817

Y86-64 pipelined implementations, 462–463
signed [C] integer data type, 77
signed divide instruction, 234, 235
signed integers, 68, 76, 97–98, 103
alternate representations, 104
shift operations, 94
two's complement encoding, 100–106
unsigned conversions, 106–112
signed multiply instruction, 234, 234
signed number representation
guidelines, 119–120
ones' complement, 104
sign magnitude, 104
signed size type, 932
significands in floating-point representation, 148
signs for floating-point representation, 148, 148–149
SIGPIPE signal, 1000
sigprocmask [Unix] block and unblock signals, 801, 817
sigsetjmp [Unix] init nonlocal handler jump, 817, 821
sigsuspend [Unix] wait for a signal, 817
%sil [x86-64] low order 8 of register %rsi, 216
SimAquarium game, 673–674
SIMD (single-instruction, multiple-data) parallelism, 62, 330, 582, 583
SIMD streaming extensions (SSE) instructions, 312
simple segregated storage, 899, 899–900
simplicity in instruction processing, 421
simulated concurrency, 60
simultaneous multi-threading, 61
single-bit data connections, 434
single-instruction, multiple-data (SIMD) parallelism, 62, 330, 582–583
single-precision floating-point representation
IEEE, 149, 149
machine-level data, 214
support for, 77
SINS [Y86-64] status code for illegal instruction exception, 440
sio_error [CS:APP] safely emit error message and terminate, 802, 804
sio_ltoa [CS:APP] safely emit string, 804
sio_putl [CS:APP] safely emit long int, 802, 804
sio_puts [CS:APP] safely emit string, 802, 804
sio_strlen [CS:APP] safely emit string, 804
size
blocks, 884
caches, 668–669
data, 75–78
word, 44, 75
size classes, 899
size_t [Unix] unsigned size type for designating sizes, 80, 119–120, 122, 135, 932
SIZE tool, 749
sizeof [C] compute size of object, 81, 165–167, 169
slashes (/) for root directory, 928
sleep [Unix] suspend process, 785
slow system calls, 810
.so shared object file, 735
sockaddr [Unix] generic socket address structure, 969
sockaddr_in [Unix] Internet-style socket address structure, 969
socket addresses, 966
socket descriptors, 948, 970
socket function, 970
socket pairs, 966
sockets, 928, 966
sockets interface, 968, 968–969
accept function, 972–973
address structures, 969–970
bind function, 971
connect function, 970–971
example, 980–983
helper functions, 978–980
host and service conversions, 973–978
listen function, 971
open_clientfd function, 970–971
socket function, 970
Software Engineering Institute, 136
software exceptions
C++ and Java, 822
ECF for, 759–760
vs. hardware, 760
Solaris Sun Microsystems operating system, 52, 81
solid state disks (SSDs), 627, 636
benefits, 623
operation, 636–638
sorting performance, 602–603
source files, 39
source hosts, 958
source programs, 39
southbridge chipsets, 624
Soviet Union, 967
%sp [x86-64] low order 16 bits of stack pointer register %rsp, 216
SPARC
five-stage pipelines, 507
RISC processors, 399
Sun Microsystems processor, 81
spare cylinders, 632
spatial locality, 640
caches, 679–683
exploiting, 650
special arithmetic operations, 233–236
special control conditions in Y86-64 pipelining
detecting, 493–495
handling, 491–493
specifiers, operand, 216–218
speculative execution, 555, 555, 585–586
speedup of parallel programs, 1054, 1054–1055
spilling, register, 584–585
spin loops, 814
spindles, disks, 626
%spl [x86-64] low order 8 of stack pointer register %rsp, 216
splitting
free blocks, 885–886
memory blocks, 883
sprintf [C Stdlib] function, 83, 318
Sputnik, 967
sqrtsd [x86-64] double-precision square root, 338
sqrtss [x86-64] single-precision square root, 338
square root floating-point instructions, 338
squashing mispredicted branch handling, 480
SRAM (static RAM), 49, 617, 617–618
cache. See caches and cache memory
vs. DRAM, 618
trends, 638–639
SRAM cells, 617
srand [CS:APP] pseudorandom number generator seed, 1057
SSDs (solid state disks), 627, 636
benefits, 623
operation, 636–638
SSE (streaming SIMD extensions) instructions, 203–204, 330
alignment exceptions, 312
parallelism, 582–583
ssize_t [Unix] signed size type, 932

stack corruption detection, 322–325
stack frames, 276, 276–277
  alignment on, 312
  variable-size, 326–329
stack pointers, 275
stack protectors, 322–323
stack randomization, 320–322
stack storage allocation function, 326, 360
stacks, 55, 225, 225–227
  bottom, 226
  buffer overflow, 907
  with execve function, 787–788
  local storage, 284–287
  machine-level programming, 207
  overflow. See buffer overflow
  recursive procedures, 289–291
  run time, 275–277
  top, 226
  Y86-64 pipelining, 465
stages, SEQ, 420–431
  decode and write-back, 442–444
  execute, 444–445
  fetch, 440–442
  memory stage, 445–447
  PC update, 447
stalling
  for data hazards, 478
  pipeline, 469–472, 495–496
Stallman, Richard, 42, 52
standard C library, 40, 40–41
standard error files, 927
standard I/O library, 947, 947
standard input files, 927
standard output files, 927
Standard Unix Specification, 52
_start, 734
starvation in readers-writers problem, 1044
stat [Unix] fetch file metadata, 939–940
state machines, 1016
states
  bistable memory, 617
  deadlock, 1063
  processor, 759
  programmer-visible, 391, 391–392
  progress graphs, 1035
  state machines, 1016
static libraries, 720, 720–724
static linkers, 708
static linking, 708
static RAM (SRAM), 49, 617–618
  cache. See caches and cache memory
  vs. DRAM, 618
  trends, 638–639
static [C] variable and function attribute, 712, 713, 1030
static variables, 1030, 1030–1031
static Web content, 985
status code registers, 471
status codes
  HTTP, 989
  Y86-64, 399–400, 400
status messages in HTTP, 989
status register hazards, 471
STDERR_FILENO [Unix] constant for standard error descriptor, 927
stderr stream, 947
STDIN_FILENO [Unix] constant for standard input descriptor, 927
stdin stream, 947
stdint.h file, 103
<stdio.h> [Unix] standard I/O library header file, 120, 122
stdlib, 40, 40–41
STDOUT_FILENO [Unix] constant for standard output descriptor, 927
stdout stream, 947
stepi command, 316
stepi4 command, 316
Stevens, W. Richard, 939, 950, 1001, 1077
stopped processes, 775
storage. See also information storage
  device hierarchy, 50
  registers, 287–289
  stack, 284–287
storage classes for variables, 1030–1031
store buffers, 593–594
store instructions, 46
store operations
  example, 624
  processors, 557
store performance of memory, 591–597
STRACE tool, 822
straight-line code, 236–237
strcat [C Stdlib] string concatenation function, 318
strcpy [C Stdlib] string copy function, 318
streaming SIMD extensions (SSE) instructions, 203–204, 330
  alignment exceptions, 312
  parallelism, 582–583
streams, 947
  buffers, 947
  directory, 941
  full duplex, 948
strerror function, 774
stride-1 reference patterns, 642
stride-k reference patterns, 642
string concatenation function, 318
string copy function, 318
string generation function, 318
strings
  in buffer overflow, 315, 317
  length, 119
  lowercase conversions, 545–547
  representing, 85
STRINGS tool, 749
STRIP tool, 749
strlen [C Stdlib] string length function, 119, 545–547
strong scaling, 1055
strong symbols, 716
.strtab section, 711
strtok [C Stdlib] string function, 1060
struct [C] structure data type, 301
structures
  address, 969–970
  heterogeneous. See heterogeneous data structures
  machine-level programming, 207
SUB [instruction class] subtract, 228
subdomains, 963
subq [Y86-64] subtract, 392, 424
substitution, inline, 537
subtract instruction, 228
subtract operation in execute stage, 444
subtraction, floating-point, 338
sumarraycols [CS:APP] column-major sum, 672
sumarrayrows [CS:APP] row-major sum,

swapped-out pages, 845
swapping pages, 845
sweep phase in Mark&Sweep garbage collectors, 903
Swift, Jonathan, 79
switch [C] multiway branch statement, 268–274
switches, context, 772–773
symbol resolution, 709, 715
  duplicate symbol names, 716–720
  static libraries, 720–724
symbol tables, 711, 711–715
symbolic links, 928
symbolic methods, 502
symbols
  address translation, 850
  caches, 653
  global, 711
  local, 712
  relocation, 725–731
  strong and weak, 716
.symtab section, 711
synchronization
  flow, 812–814
  Java threads, 1046
  progress graphs, 1036
  threads, 1031–1035
    progress graphs, 1035–1037
    with semaphores. See semaphores
synchronization errors, 1031
synchronous DRAM (SDRAM), 622
synchronous exceptions, 763
/sys filesystem, 772
syscall function, 766
system bus, 623
system calls, 53, 763, 763–764
  error handling, 773–774
  Linux/x86-64 systems, 766–767
  slow, 810
system-level functions, 766
system-level I/O
  closing files, 930–931
  file metadata, 939–940
  I/O redirection, 945–946
  opening files, 929–931
  packages summary, 947–949
  reading files, 931–933
  RIO package, 933–939
  sharing files, 942–944
  standard, 947
  summary, 949–950
  Unix I/O, 926–927
  writing files, 932–933
system startup function, 734
System V Unix, 52
  semaphores, 1013
  shared memory, 1013

T2B (two's complement to binary conversion), 96, 101, 107
T2U (two's complement to unsigned conversion), 96, 107, 107–109
tables
  descriptor, 943, 945
  exception, 761, 761
  GOTs, 741, 741–743
  hash, 603–604
  header, 710, 732
  jump, 269, 270–271, 761
  page, 772, 842–844, 843, 855–857, 859
  program header, 732, 732
  symbol, 711, 711–715
tag bits, 651, 652
tags, boundary, 887, 887–890, 895
Tanenbaum, Andrew S., 56
target functions in interpositioning libraries, 744
targets, jump, 242, 242–245
TCP (Transmission Control Protocol), 960
TCP/IP (Transmission Control Protocol/Internet Protocol), 960
tcsh [Unix] Unix shell program, 789
TELNET remote login program, 986, 986–987
temporal locality, 640
  blocking for, 683
  exploiting, 650
terminate another thread function, 1025
terminate current thread function, 1025
terminate process function, 775
terminated processes, 775
terminating
  processes, 775–779
  threads, 1024–1025
TEST [instruction class] Test, 238
test byte instruction, 238
test double word instruction, 238
test instructions, 238
test quad word instruction, 238
test signal set membership instruction, 801
test word instruction, 238
testb [x86-64] test byte, 238
testing Y86-64 pipeline design, 501
testl [x86-64] test double word, 238
testq [x86-64] test quad word, 238
testw [x86-64] test word, 238
text files, 39, 927, 928, 936
text lines, 927, 934
text representation
  ASCII, 85
  Unicode, 86
.text section, 710
Thompson, Ken, 52
thrashing
  direct-mapped caches, 658, 658–659
  pages, 846
thread contexts, 1022, 1029
thread IDs (TIDs), 1022
thread-level concurrency, 60–62
thread-level parallelism, 62
thread routines, 1023, 1024
thread-safe functions, 1056, 1056–1058
thread-unsafe functions, 1056, 1056–1058
threads, 53, 54, 1009, 1021–1022
  concurrent server based on, 1027–1028
  creating, 1024
  detaching, 1025–1026
  execution model, 1022–1023
  initializing, 1026
  library functions for, 1060–1061
  mapping variables in, 1030–1031
  memory models, 1029–1030
  for parallelism, 1049–1054
  Posix, 1023–1024
  races, 1061–1063
  reaping, 1025
  safety issues, 1056–1058
  shared variables with, 1028–1031, 1029
  synchronizing, 1031–1035
    progress graphs, 1035–1037
    with semaphores. See semaphores
  terminating, 1024–1025
three-stage pipelines, 450–452
throughput, 560
  dynamic memory allocators, 881
  pipelining for. See pipelining
  read, 675
throughput bounds, 554, 560
TIDs (thread IDs), 1022
time slicing, 769
timing, SEQ, 436–439
TINY [CS:APP] Web server, 992, 992–1000
TLB index (TLBI), 853
TLB tags (TLBT), 853, 859
TLBI (TLB index), 853